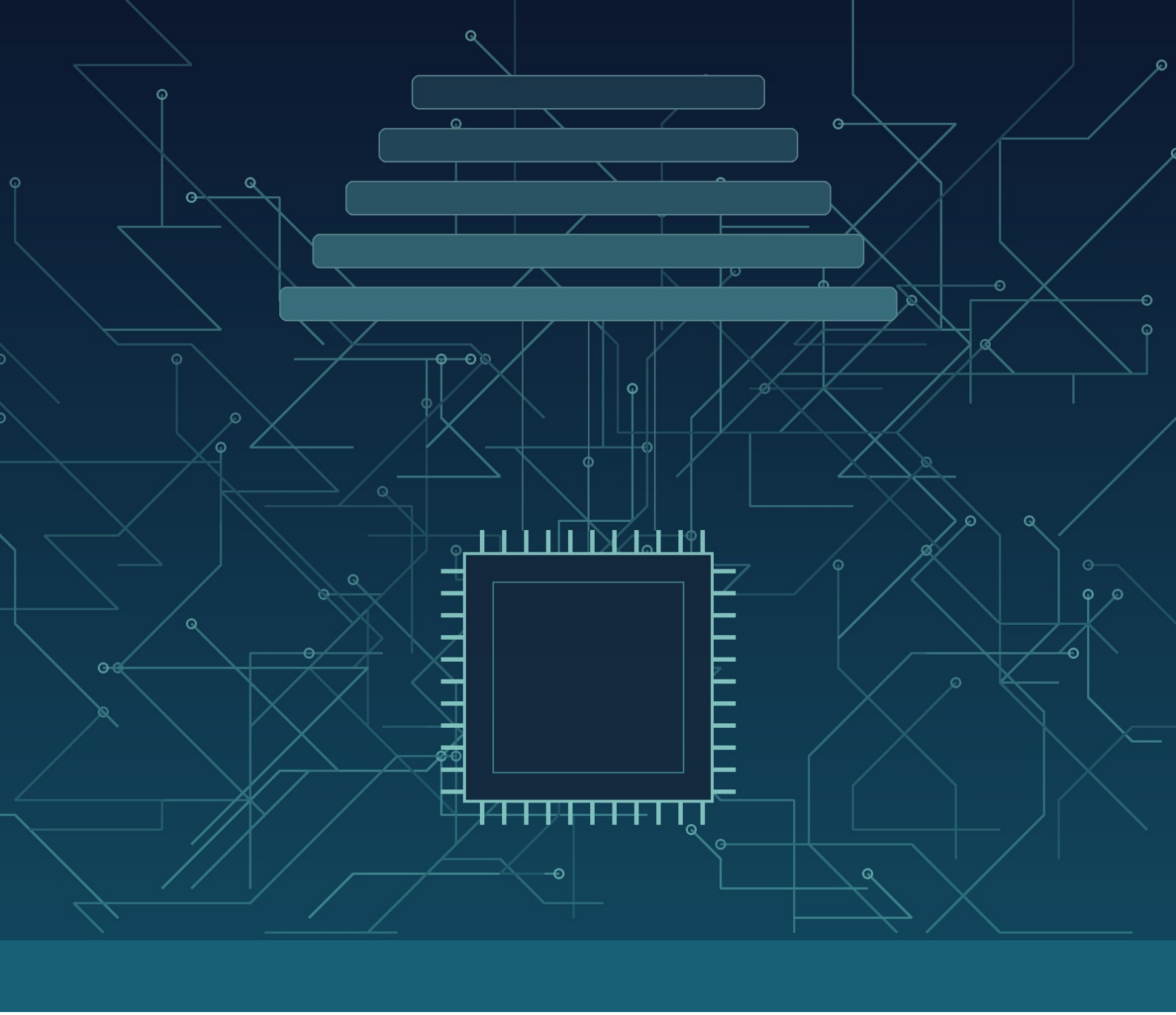
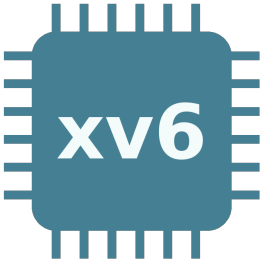


从裸机到操作系统

基于 xv6 的简单实现 —— Raspberry Pi 3 / AArch64

作者：Jiming Kang
组织：xv6-aarch64 项目
时间：2026 年 10 月
版本：0.1
源码：xv6-aarch64（分支 rpi3-vfs）



从上电后的第一条指令，到能远程 SSH 登录、连 Wi-Fi、拍照的多核内核。

目录

第一章 序言 **1**

1.1 从一条指令开始 . . . 1
1.2 为什么是 xv6、AArch64 和树莓派 . . . 1
1.3 这个项目走过的路 . . . 1
1.4 本书的结构 . . . 2
1.5 阅读方法与约定 . . . 3
1.6 致谢 . . . 3

第二章 项目组织 **4**

2.1 目录结构 . . . 4
2.2 内核子系统 . . . 4
2.3 构建流程 . . . 4
2.3.1 编译选项 . . . 4
2.3.2 从目标文件到 `kernel8.img` . . . 5
2.3.3 用户程序与文件系统镜像 . . . 6
2.4 安装、运行与调试 . . . 6
2.5 内核初始化顺序 . . . 7

第三章 基于汇编的裸机代码 **9**

3.1 xv6-aarch64 的启动代码 . . . 9
3.1.1 上电之后发生了什么 . . . 9
3.1.2 链接地址与装载地址 . . . 10
3.1.3 统一异常级别 . . . 10
3.1.4 不依赖任何环境的早期串口 . . . 13
3.2 异常与中断 . . . 15
3.2.1 异常与中断区别 . . . 15
3.2.2 异常向量 . . . 15
3.2.3 异常级别与栈指针 . . . 17
3.2.4 各种事件落到哪一个入口 . . . 17
3.2.5 保存寄存器状态 . . . 18
3.2.6 设置向量表 . . . 20
3.2.7 屏蔽与解除屏蔽中断 . . . 20
3.2.8 配置中断控制器 . . . 21
3.2.9 通用 IRQ 处理程序 . . . 23
3.2.10 初始化定时器 . . . 25
3.2.11 处理定时器中断 . . . 26
3.2.12 小结 . . . 27

第四章 MMU **28**

4.1 AArch64 的地址翻译 . . . 28
4.1.1 地址空间划分 . . . 28
4.1.2 三级页表 . . . 28

4.1.3 硬件如何遍历页表 . . . 29
4.1.4 描述符的属性位 . . . 29
4.1.5 内存属性间接表 . . . 29
4.2 启动页表 . . . 31
4.2.1 清 BSS 与早期栈 . . . 31
4.2.2 两棵临时页表 . . . 31
4.2.3 打开 MMU：TCR、MAIR 与 SCTLR . . . 33
4.2.4 跳到高地址 . . . 34
4.3 多核启动 . . . 35
4.4 从设备树到 memblock . . . 36
4.4.1 设备树从哪里来 . . . 36
4.4.2 解析 FDT . . . 36
4.4.3 memblock：两张有序区间表 . . . 37
4.5 物理内存初始化与伙伴系统 . . . 37
4.5.1 从空闲链表到伙伴系统 . . . 37
4.5.2 `struct page` 与 `mem_map` . . . 38
4.5.3 伙伴分配与合并 . . . 39
4.5.4 把内存交给伙伴系统 . . . 40
4.5.5 接口与防错 . . . 41
4.5.6 每 CPU 页缓存 . . . 41
4.5.7 观察：启动自检与 `/proc/buddyinfo` . . . 42
4.6 slab 与 `kmalloc()` . . . 43
4.6.1 元数据放进 `struct page` . . . 43
4.6.2 尺寸类与 `kfree()` . . . 44
4.6.3 观察：`/proc/slabinfo` . . . 44
4.7 调试视图：`/proc/meminfo` 与 `/proc/iomem` . . . 45
4.7.1 逐条跟踪：`MM_TRACE` . . . 46
4.8 最终内核页表 . . . 47
4.8.1 `kvmmake()` . . . 47
4.8.2 `walk()` 与 `mappages()` . . . 47
4.8.3 页表的规模 . . . 48
4.9 用户地址空间 . . . 49
4.9.1 内核空间与用户空间 . . . 49
4.9.2 `exec()` 之后的布局 . . . 49
4.9.3 第一个用户进程的页表 . . . 49
4.9.4 `fork`：写时复制 . . . 50
4.9.5 TLB 与地址空间切换 . . . 52
4.9.6 缺页与按需分配 . . . 52
4.10 `mmap()`：VMA、按需调页与设备映射 . . . 53
4.10.1 `vm_area_struct` 与地址布局 . . . 53
4.10.2 按需调页：`handle_mm_fault()` . . . 54
4.10.3 共享页：`pcache_page` 与 `shm_object` . . . 55
4.10.4 设备 mmap：`f_op->mmap` 与 `remap_pfn_range()` . . . 55
4.10.5 内核替用户访问 mmap 区 . . . 56
4.10.6 fork、vfork、exit 与 `exec` . . . 56

4.10.7 测试与限制 . . . 56
4.11 MMIO、总线地址与 DMA . . . 57
4.11.1 MMIO . . . 57
4.11.2 三种地址视图 . . . 57
4.11.3 PCIe：BAR、DMA 与 IOMMU . . . 58
4.12 缓存与 TLB：三个真机故障 . . . 59
4.12.1 故障一：停在 BPH0abde . . . 59
4.12.2 故障二：ls 跳到了不属于它的代码 . . . 59
4.12.3 故障三：sbrk 之后的旧 TLB . . . 60
4.13 实验分支：引用计数、写时复制与页缓存 . . . 60
4.13.1 物理页引用计数 . . . 60
4.13.2 写时复制 . . . 61
4.13.3 页缓存 . . . 62
4.13.4 实验分支里的 mmap() . . . 62

第五章 进程与调度 64
5.1 进程由什么组成 . . . 64
5.1.1 struct proc . . . 64
5.1.2 两种现场 . . . 65
5.2 动态进程表与内核栈 . . . 65
5.3 第一个用户进程 . . . 66
5.3.1 initcode 做了什么 . . . 66
5.3.2 为什么需要 initcode . . . 67
5.3.3 不用 initcode 的写法 . . . 67
5.4 fork() 逐行解析 . . . 68
5.4.1 新进程的第一次运行 . . . 70
5.4.2 回到用户态：trapret 中的中断竞争 . . . 70
5.4.3 实例：fork 快照中的虚拟地址与物理页 . . . 71
5.5 clone 标志与 vfork . . . 75
5.6 上下文切换与调度器 . . . 76
5.6.1 swtch() . . . 76
5.6.2 context 中保存的值 . . . 77
5.6.3 调度循环 . . . 78
5.6.4 把一次调度切换打印出来 . . . 78
5.6.5 调度对进程是透明的 . . . 79
5.6.6 谁触发调度 . . . 79
5.6.7 关中断与禁止抢占 . . . 80
5.6.8 如何选择下一个进程 . . . 80
5.6.9 跟踪一次时间片轮转 . . . 81
5.6.10 实例：log8 中的调度、栈与 context . . . 81
5.7 exec、exit 与 wait . . . 84
5.7.1 exec . . . 84
5.7.2 exit 与 wait . . . 85
5.7.3 在系统调用里睡眠不是异常 . . . 86
5.8 睡眠、唤醒与同步原语 . . . 86

5.8.1 sleep/wakeup . . . 86
5.8.2 内核 mutex 与 rw semaphore . . . 87
5.8.3 用户态同步原语 . . . 87
5.9 多核软实时调度 . . . 87

第六章 文件系统 **89**
6.1 分层结构 . . . 89
6.2 块设备：从 ramdisk 到 SD 卡 . . . 90
6.2.1 三个阶段 . . . 90
6.2.2 SD 卡初始化 . . . 90
6.2.3 命令超时与恢复 . . . 90
6.3 xv6 文件系统格式：superblock、inode 与目录 . . . 91
6.3.1 superblock 与磁盘布局 . . . 91
6.3.2 inode：文件的元数据与块映射 . . . 92
6.3.3 目录与路径查找 . . . 93
6.3.4 从路径到 SD 扇区 . . . 94
6.3.5 mkfs 怎样生成 fs.img . . . 94
6.3.6 更新 p2 上的镜像 . . . 94
6.4 块缓存、日志与 inode . . . 95
6.5 VFS 中间层 . . . 95
6.5.1 从路径前缀表到统一 VFS . . . 95
6.5.2 super_block、inode_ops 与 file_operations . . . 98
6.5.3 一次跨文件系统的路径解析 . . . 100
6.5.4 文件描述符与系统调用 . . . 100
6.5.5 当前目录与 exec . . . 101
6.5.6 三类实现 . . . 102
6.5.7 由用户态决定挂载 . . . 103
6.6 FAT32 与启动分区 . . . 103
6.6.1 bootfs 里有什么，谁来读 . . . 103
6.6.2 磁盘上的 FAT32 . . . 104
6.6.3 FAT 表与簇链 . . . 105
6.6.4 目录项与长文件名 . . . 106
6.6.5 函数一览与调用关系 . . . 106
6.6.6 写入：分配、追加与截断 . . . 107
6.7 根文件系统放在哪里 . . . 108
6.8 Linux 式命名空间引导 . . . 109
6.8.1 三个“根”：rootfs、根目录 / 与根文件系统 . . . 109
6.8.2 PC 上的 Linux：initramfs 与用户态找根 . . . 110
6.8.3 嵌入式 Linux：内核按 root= 直接挂载 . . . 110
6.8.4 xv6 的选择：模仿嵌入式 Linux . . . 111
6.8.5 启动调用链总览 . . . 112
6.8.6 阶段一：登记块设备、选出 ROOT_DEV . . . 115
6.8.7 阶段二：forkret() 中的 prepare_namespace() . . . 116
6.8.8 支撑这四步的机制：超级块、日志、挂载表与 namex() . . . 117
6.8.9 ext2 根的挂载细节 . . . 118

6.8.10 阶段三：/init 与用户态看到的结果 . . . 121
6.8.11 用 fs-trace 看 rootfs 与 devtmpfs . . . 122
6.9 烧录 SD 卡 . . . 126
6.9.1 分区布局 . . . 126
6.9.2 步骤 . . . 126
6.9.3 日常更新 . . . 126

第七章 驱动架构 **127**
7.1 设备模型 . . . 127
7.1.1 为什么不再只用 devsw[] . . . 127
7.1.2 kobject：一切皆对象 . . . 128
7.1.3 sysfs：把对象树渲染成文件系统 . . . 129
7.1.4 属性文件与 DEVICE_ATTR . . . 129
7.1.5 uevent 与 drivers_probe：手工触发热插拔 . . . 129
7.1.6 /sys/class 与符号链接 . . . 130
7.2 上半部与下半部 . . . 130
7.3 控制台、UART 与 TTY . . . 130
7.3.1 Mini UART . . . 131
7.3.2 控制台与行规程 . . . 131
7.3.3 TTY 层 . . . 131
7.3.4 绑定：设备、驱动、设备文件与会话 . . . 131
7.3.5 数据通路与调用链 . . . 132
7.3.6 锁 . . . 135
7.3.7 已知问题 . . . 136
7.3.8 与 Linux tty 的对照 . . . 137
7.4 USB 与 HID 键盘 . . . 137
7.4.1 三层软件与硬件路径 . . . 137
7.4.2 内核软件框架：五层栈与三条抽象边界 . . . 140
7.4.3 事务格式：Token、Data、Handshake 三相 . . . 142
7.4.4 描述符：设备、配置、接口与端点 . . . 144
7.4.5 枚举：control 传输与请求序列 . . . 145
7.4.6 Split transaction：高速 Hub 后面的低速设备 . . . 150
7.4.7 URB 与中断驱动的接收 . . . 152
7.4.8 HID 报告与输入子系统 . . . 155
7.4.9 中断传输：小、周期、低延迟 . . . 157
7.4.10 Bulk 传输：量大、可等待、不许出错 . . . 158
7.4.11 真机排错：从 STALL 到第一份报告 . . . 158
7.5 I2C . . . 161
7.5.1 开漏电气规则与总线时序 . . . 161
7.5.2 OV5647 寄存器收发序列 . . . 162
7.6 网络协议栈 . . . 164
7.7 SDIO 无线网卡：BCM43430/43455 . . . 164
7.7.1 硬件路径 . . . 164
7.7.2 硬件结构：Arasan、SDIO 总线与 FullMAC 芯片 . . . 165
7.7.3 SDIO 枚举 . . . 166

7.7.4 固件装载 . . . 166
7.7.5 SDPCM 与 BCDC . . . 166
7.7.6 WPA2 四次握手 . . . 167
7.7.7 数据流与并发 . . . 167
7.8 USB 无线网卡：MT7601U . . . 168
7.8.1 USB 拓扑与枚举 . . . 168
7.8.2 设备、配置、接口与端点描述符 . . . 169
7.8.3 探测时序：从枚举到 `wlan1` . . . 170
7.8.4 主机与芯片之间的四种报文 . . . 172
7.8.5 启动日志分析：USB 总线上发生了什么 . . . 173
7.8.6 空口：扫描、认证、关联、WPA2 四次握手 . . . 176
7.8.7 调试实录 . . . 178
7.8.8 现状与待改进 . . . 179
7.9 CSI-2 摄像头：OV5647 . . . 179
7.9.1 路线选择 . . . 179
7.9.2 硬件结构：I2C、CSI-2、Unicam 与 DMA . . . 179
7.9.3 `sensor_ops`：仿照 V4L2 子设备 . . . 181
7.9.4 板级资源 . . . 181
7.9.5 硬件连接与工作流程 . . . 182
7.9.6 上电与探测 . . . 184
7.9.7 拍一帧 . . . 184
7.9.8 从 D-PHY 到内存：FIFO、双缓冲与帧通知 . . . 186
7.9.9 用户程序 `camshot` . . . 189
7.9.10 一次成功拍照的日志解读 . . . 189
7.9.11 真机调试记录 . . . 191
7.9.12 连续取流与 HTTP 推流 . . . 196
7.9.13 提高推流帧率 . . . 198
7.9.14 当前状态 . . . 200
7.10 Windows 风格窗口系统：win32k、消息与 GDI . . . 201
7.10.1 边界：哪些对象在用户态，哪些必须进内核 . . . 201
7.10.2 创建窗口与系统调用服务表 . . . 202
7.10.3 消息驱动：等待，而不是轮询 . . . 202
7.10.4 输入如何找到 WNDPROC . . . 203
7.10.5 无效区域与 `WM_PAINT` . . . 203
7.10.6 GDI：HDC、批处理、光栅化与合成 . . . 204
7.10.7 并发：锁的划分与前后缓冲 . . . 204
7.10.8 实例与局限 . . . 205

第八章 用户进程、线程与 libc **206**
8.1 用户程序的形态 . . . 206
8.1.1 为什么需要系统调用 . . . 206
8.1.2 链接与装载 . . . 206
8.1.3 系统调用网关 . . . 206
8.1.4 内核如何分发系统调用 . . . 207
8.1.5 两个栈指针与返回级别 . . . 208

8.2 从 `init` 到 shell . . . 208
8.3 用户程序一览 . . . 209
8.4 用户线程 . . . 209
8.4.1 设计 . . . 209
8.4.2 系统调用 . . . 210
8.4.3 `clone` 为什么“返回两次” . . . 210
8.4.4 多核下的细节 . . . 211
8.4.5 实例：threadtest 的运行过程 . . . 212
8.5 libc 与 POSIX 源码兼容层 . . . 212
8.5.1 `libc.a` . . . 212
8.5.2 动态链接 `libc.so` . . . 212
8.5.3 一次库函数调用：PLT 与 GOT . . . 215
8.5.4 共享 libc 代码页 . . . 216
8.5.5 用户态解释器 `/lib/ld-xv6.so` . . . 216
8.5.6 图解：`/bin/echo` 的文件布局与运行时绑定 . . . 217
8.5.7 进一步：用 `mmap` 装载并共享（实验分支） . . . 221
8.5.8 调试动态链接 . . . 222
8.5.9 POSIX 层 . . . 222
8.6 在 xv6 里编译 C 程序 . . . 223
8.7 一个大型用户程序：`sshd` . . . 223

第九章 热插拔与驱动自动装载 **225**
9.1 Hub 中断驱动的 USB 热插拔 . . . 225
9.2 复合 HID 接口与鼠标驱动自动装载（真机日志 046d:c534） . . . 228

第十章 可选工作 **231**
10.1 ext2 移植 . . . 231
10.1.1 动机与分区布局 . . . 231
10.1.2 ext2 的磁盘结构 . . . 232
10.1.3 从文件偏移到块号 . . . 233
10.1.4 分配与释放 . . . 234
10.1.5 接入 VFS：按 inode 工作 . . . 234
10.1.6 持久化：`sdflush()` 写屏障 . . . 235
10.1.7 外置事务日志 xjournal . . . 236
10.1.8 源码组织与编译 . . . 238
10.1.9 测试：WAL 程序与随机断电 . . . 239
10.1.10 给 SD 卡分出 ext2 分区 . . . 239
10.1.11 格式化 ext2 . . . 241
10.1.12 安装与真机调试 . . . 242
10.1.13 当前边界 . . . 243

参考文献 **244**

第一章 序言

本章内容

- 为什么要从裸机写起
- 为什么选 xv6、AArch64 和 Raspberry Pi 3
- 这个项目走过的路
- 本书的结构与阅读方法
- 开发环境与约定

1.1 从一条指令开始

一台树莓派插上电源后，GPU 固件把一段二进制文件拷到物理地址 `0x80000`，然后让 CPU0 从那里开始取指。此时没有栈，没有堆，没有 `printf`，没有页表，甚至不知道自己处于哪个异常级别。本书要回答的问题是：从这条指令出发，要依次搭起哪些东西，才能得到一个可以多核调度、读写 SD 卡、通过 Wi-Fi 被 SSH 远程登录、还能驱动摄像头的操作系统？

教科书通常按“进程、内存、文件、设备”分章讲操作系统原理，每一章都假设别的部分已经存在。真正动手时顺序恰好相反：要先让串口吐出一个字符，才能知道代码跑到了哪里；要先建好页表，内存分配器才有地方放空闲链表；要先有块设备驱动，文件系统才能读到 superblock。本书沿着这条“依赖链”组织内容，每一章都只依赖前面已经建好的东西。

1.2 为什么是 xv6、AArch64 和树莓派

xv6. xv6 是 MIT 为操作系统课程 6.S081/6.1810 编写的教学内核 [6]，用 C 重写了第六版 Unix 的核心思想。它只有一万行左右，却完整包含了进程、虚拟内存、系统调用、日志文件系统、管道和 shell。代码短，意味着每一个机制都能读到底；结构清楚，意味着加新功能时知道该放在哪一层。

AArch64. 原版 xv6 运行在 RISC-V 上。本项目的起点是一个面向 QEMU `virt` 机器的 AArch64 移植。ARMv8-A 的异常级别（EL0 ~ EL3）、两套页表基址寄存器（`TTBR0_EL1`/`TTBR1_EL1`）、内存属性间接表（`MAIR_EL1`）以及显式的缓存维护指令，都比 RISC-V 更接近手机和服务器上真实运行的系统，也更容易踩到“QEMU 正常、真机失败”的坑——而这些坑恰恰最有教学价值。

Raspberry Pi 3. 树莓派 3B/3B+ 使用 BCM2837 SoC（四核 Cortex-A53），价格低、资料多，Linux 驱动全部开源，可以对照阅读。它的外设足够丰富：Mini UART、SD 卡控制器、DWC2 USB 主机、板载 SDIO Wi-Fi、I2C、CSI-2 摄像头接口……几乎每一类典型驱动都能在这块板子上找到一个真实的例子。同时 QEMU 提供了 `raspi3b` 机器模型，可以先在模拟器里调通流程，再上真机验证。

1.3 这个项目走过的路

项目的提交记录大致反映了“从裸机到操作系统”的顺序（表 1.1）。起点是 QEMU `virt` 上的 xv6；转到树莓派以后的两周里，几乎每天都在真机上解决一个新问题，先让串口、SD 卡、网络和 Wi-Fi 跑起来；此后的三周转向“像 Linux 一样组织”：initcall、内核模块与热插拔、ext2 根文件系统和统一的 VFS、可睡眠锁、mmap 与 clone。

这些功能都附有一篇专题文档（`doc/readme/` 目录），记录了实现思路和调试过程。本书就是把这些文档按依赖顺序重新组织、补全前因后果之后得到的。

表 1.1: 主要里程碑（取自 rpi3-lock 分支的 git 提交记录，2026 年）

日期	内容
05-27	初始提交：QEMU `virt` 上的 xv6-aarch64
09-23	从 `virt` 转到 Raspberry Pi 3；SD 卡与 FAT32 分区
09-24	真机串口与内存属性问题；D/I-cache 一致性；SD 命令出错重试
09-27	进程间同步原语与生产者/消费者；网络协议栈与 `ping`
09-28 ~ 09-29	TTY 中间层；Linux 风格的总线/设备/驱动层；SDIO 板载 Wi-Fi
09-30	`sshd` 远程登录；SDIO 中断；workqueue 下半部
10-01	每设备私有 work、delayed work、NAPI；FAT32 启动分区挂载到 `/boot`；非阻塞 epoll；TFTP 客户端；信号；USB 键盘（轮询到中断）
10-02	USB URB 接口；从键盘驱动分离出 input 子系统
10-03	软实时调度；类 pthread 的用户线程
10-04	用户 API 改为 POSIX 风格
10-05	CSI-2 摄像头（OV5647 + Unicam）；TFTP 服务器；本书初稿
10-06	可读写的 ext2（p3 分区）；netfs；`exec` 走 VFS，`cwd` 改为路径
10-07	摄像头中断与 DMA 双缓冲、MJPEG 推流三级流水线；仿嵌入式 Linux 的根文件系统加载（rootfs、`root=`）；inode 层改为 VFS 兼容
10-09 ~ 10-11	USB 键盘热插拔；`module_init` 与 initcall；可装载的 `.ko` 模块与 `request_module`
10-12	摄像头上电问题；WPA2 在线重新握手后未装新密钥的修复
10-13 ~ 10-14	合入 kobject/sysfs；`libc.so` 动态加载与 `ld.so`
10-21 ~ 10-23	目录重组；VFS 分层向 Linux 靠近；qemu-gdb 调试；FIFO mutex、rw_semaphore 与 inode 读写锁
10-25	`register_filesystem`；Linux 式 clone 标志（`files_struct`、`fs_struct`、`CLONE_VFORK`）
10-26	普通文件与设备文件的 `mmap`；合入 clone/vfork 分支

1.4 本书的结构

- **第 2 章项目组织**：目录结构、构建流程、镜像如何生成，以及内核初始化顺序。这一章是全书的地图。
- **第 3 章基于汇编的裸机代码**：固件把控制权交给内核之后，`entry.S` 如何统一异常级别、点亮早期串口、建立临时页表、打开 MMU，并唤醒其余三个核；异常向量表和中断控制器也在这一章。
- **第 4 章 MMU**：39 位虚拟地址、三级页表、内存属性；启动页表、多核启动与最终内核页表；用户地址空间与 `mmap()`；MMIO 与 DMA 地址；以及三个与缓存和 TLB 有关的真机故障。
- **第 5 章进程与调度**：`struct proc`、第一个用户进程、`fork()` 的每一步、clone 标志与 `vfork`、上下文切换和调度器，以及在此基础上加入的同步原语和软实时调度。
- **第 6 章文件系统**：xv6 日志文件系统的分层，块设备与 SD 卡，仿嵌入式 Linux 的根文件系统加载（rootfs、devtmpfs、ext2 根），VFS 中间层（procfs、ext2、FAT32），以及如何把系统烧录到 SD 卡。
- **第 7 章驱动架构**：设备模型与上下半部；控制台、UART 与 TTY；USB 与输入子系统；I2C；网络协议栈；SDIO 与 USB 两种无线网卡；CSI-2 摄像头。
- **第 8 章用户进程、线程与 libc**：系统调用桩、`init` 与 shell、用户线程、libc 与 POSIX 兼容层、一个能在 xv6 里运行的小型 C 编译器，以及最大的用户程序 `sshd`。
- **第 9 章热插拔与驱动自动装载**：Hub 中断驱动的 USB 热插拔，拔出时拆除整棵设备子树；匹配不到驱动时由 MODALIAS 经 `request_module()` 从 `/lib/modules` 装载 `.ko`，并用一份复合 HID 接收器的真机日志串起全过程。
- **第 10 章可选工作**：ext2 驱动本身的实现细节：可读写的 ext2、`sdflush()` 写屏障和外置事务日志 xjournal，以及用 WAL 程序做的断电测试。第 6 章讲 ext2 怎样成为根文件系统，这一章讲它怎样读写磁盘。

第 2 章之后的各章大体按启动顺序排列：第 3、4 章是 `main()` 之前和刚进入 `main()` 时的事，第 5 章造出第一个进程，第 6 章在第一个进程里挂上根文件系统，第 7 章是挂在各条总线上的设备，第 8 章回到用户态。第 9、10 章是在前面基础上的专题，可以按需选读。

1.5 阅读方法与约定

本书以源码为准。正文中的文件路径都相对于仓库根目录，例如 `kernel/arch/arm64/entry.S`。代码片段尽量保持原样，只删去与讨论无关的行。描述某个函数时，一般按“它解决什么问题 → 调用路径 → 关键代码 → 容易出错的地方”的顺序展开。



笔记 QEMU 的 `raspi3b` 模型与真机有不少差别：QEMU 对内存属性和缓存一致性非常宽容，不模拟 SDIO Wi-Fi、摄像头和板载 USB 网卡，固件 mailbox 也只实现了一部分。本书凡是只在 QEMU 上验证过的结论都会注明。一条经验贯穿全书：**QEMU 能跑通不代表真机正确，原子操作、缓存维护和设备时序必须在 Cortex-A53 上验证。**

开发环境如下：

- 交叉工具链：`aarch64-linux-gnu-`、`aarch64-elf-` 或 `aarch64-unknown-elf-` 均可，Makefile 会自动探测；
- 模拟器：`qemu-system-aarch64 -machine raspi3b`；
- 真机：Raspberry Pi 3B 或 3B+，一张 microSD 卡，一根 3.3 V 的 USB-TTL 串口线（GPIO14/15，115200 8N1）；
- 宿主机：macOS（文中的 `diskutil`、`/dev/rdiskN` 等命令以 macOS 为例，Linux 下换成 `fdisk`/`dd` 的等价写法即可）。

1.6 致谢

本项目从以下工作中获益良多：MIT 的 xv6[6]；Sergey Matyukevich 的 *raspberry-pi-os*[7]，它对树莓派裸机异常与中断的讲解非常清楚；rockytriton 的 LLD 裸机教程 [12]，其中的 EMMC 驱动是本项目 SD 驱动的参考；Circle 裸机框架 [15]；以及 Raspberry Pi Linux 内核中的 `brcmfmac`、`mt7601u`、`bcm2835-unicam`、`ov5647` 等驱动 [11]。

第二章 项目组织

本章内容

- ❑ 仓库目录结构
- ❑ 内核子系统与源文件
- ❑ 从源码到镜像的构建流程
- ❑ 安装、运行与调试目标
- ❑ 内核初始化顺序：全书的地图

2.1 目录结构

仓库的顶层目录如表 2.1 所示。它保留了 xv6 的基本布局：`kernel/` 是内核，`user/` 是用户程序和用户库，`mkfs/` 是在宿主机上运行的文件系统镜像生成工具。在此之上，为了支撑 POSIX 源码兼容、真机部署和调试，又增加了 `include/`、`firmware/`、`tools/`、`doc/` 等目录。

表 2.1: 仓库顶层目录

路径	内容
`kernel/`	内核源码：启动汇编、异常、内存、进程、文件系统、VFS、网络栈、全部驱动
`user/`	用户程序（`sh`、`ls`、`sshd`、`wifi`、`camshot` 等）和用户库（`ulib`、`printf`、`umalloc`、线程、POSIX 包装）
`include/`	头文件：`include/kernel/` 是内核内部接口，`include/uapi/` 是内核与用户态共用的定义（系统调用号、`KERNBASE` 等），其余是面向 POSIX 源码的标准头文件：`stdio.h`、`unistd.h`、`pthread.h`、`sys/socket.h` 等
`mkfs/`	宿主机工具 `mkfs`，把用户程序打包成 xv6 原生文件系统镜像 `fs.img`
`doc/`	`doc/readme/` 下 40 余篇专题文档，记录每个功能的设计与调试过程；`doc/svg/` 是本书部分插图的源文件
`firmware/`	无线网卡固件：BCM43430/43455 的 `.bin/.txt/.clm_blob`，MT7601U 的 `mt7601u.bin`
`tools/`	调试脚本：串口跟踪的采集、回放和网页导出（`trace_xv6.py` 等）
`ports/`	向其他开发板移植的实验（如全志 T113）
`config.txt`	树莓派固件配置文件，随内核一起拷到启动分区
`cmdline.txt`、`cmdline_ext2.txt`	内核命令行（`root=`、`rootfstype=`），安装时拷到启动分区的 `/cmdline.txt`（6.7 节）
`Makefile`	构建、打包、安装、QEMU 运行的全部规则

2.2 内核子系统

内核约 4.2 万行（含头文件），用户空间约 1.7 万行。`kernel/` 下的子目录仿照 Linux 的源码树划分（表 2.2），路径本身就说明了一个文件属于哪个子系统。阅读时建议先看 `arch/arm64`、`mm` 和 `core`，它们是其余一切的基础。

2.3 构建流程

2.3.1 编译选项

内核是一个独立运行的程序：没有 C 运行库，没有动态链接器，也不能假设 CPU 已经打开浮点单元。关键的编译选项如下：

```
CFLAGS = -Wall -Werror -Os -g -fno-omit-frame-pointer -mcpu=cortex-a53+nofp
CFLAGS += -ffreestanding -fno-common -nostdlib
```

表 2.2: kernel/ 下的目录与主要文件

目录	主要文件
arch/arm64	启动与异常：entry.S、start.c、kernelvec.S、uservec.S、trapasm.S、swtch.S、timer.c、gicv3.c、bcm2837.c
init	main.c（初始化顺序）、cmdline.c（内核命令行）、do_mounts.c（挂根文件系统）
mm	kalloc.c（物理页分配）、vm.c（页表）、mmap.c（VMA 与缺页）
core	进程与同步：proc.c、trap.c、syscall.c、sysproc.c、spinlock.c、mutex.c、rwsem.c、sleeplock.c、sync.c、workqueue.c
fs	bio.c、log.c、fs.c、vfs.c、file.c、fdtable.c、sysfile.c、exec.c、pipe.c、epoll.c；子目录 ext2/（ext2.c、xjournal.c）、fat/、netfs/
net	协议栈 net.c 与套接字系统调用 sysnet.c
drivers/base	设备模型：device.c、kobject.c、module.c（.ko 装载）、export.c
drivers/block	blkdev.c（块设备表与分区）、ramdisk.c、rootdev.c（选出 ROOT_DEV）
drivers/ tty、char、input	uart.c、console.c、tty.c、pty.c；mem.c（/dev/mem、/dev/zero）；input.c、usbkbd.c、usbmouse.c
drivers/mmc	host/sdhost.c（外置 SD 卡）、host/sd.c（早期 Arasan SD 驱动）、host/arasan_sdio.c、core/sdio.c
drivers/usb、net	host/dwc2.c（主机控制器）、core/usb.c（USB core 与 URB）；net/usb/usbnet.c、net/wireless/（brcmfmac.c、wpa_crypto.c、mt7601u.c）
drivers/ media、video、firmware	i2c/i2c.c、i2c/ov5647.c、platform/unicam.c；fbdev/bcm2708_fb.c（/dev/fb0）；mbox.c（固件 mailbox）
lib	printf.c、string.c、rng.c（硬件随机数）

```
CFLAGS += -I. -Iinclude -fno-stack-protector -fno-pie -no-pie
LDFLAGS = -z max-page-size=4096
```

- `-mcpu=cortex-a53+nofp`：生成 A53 指令，但禁止编译器使用浮点/SIMD 寄存器。内核上下文切换只保存通用寄存器，若编译器偷偷用 q0 做 memcpy，用户态的浮点状态就会被破坏。
- `-ffreestanding -nostdlib`：不假设标准库存在，memset、memmove 等由 kernel/lib/string.c 自己实现。
- `-fno-pie`：内核链接在固定的高虚拟地址，不需要位置无关代码。
- `-Werror`：所有警告视为错误，真机调试的成本很高，宁可在编译期多拦住一些问题。

2.3.2 从目标文件到 kernel8.img

图 2.1 是完整的构建流水线。

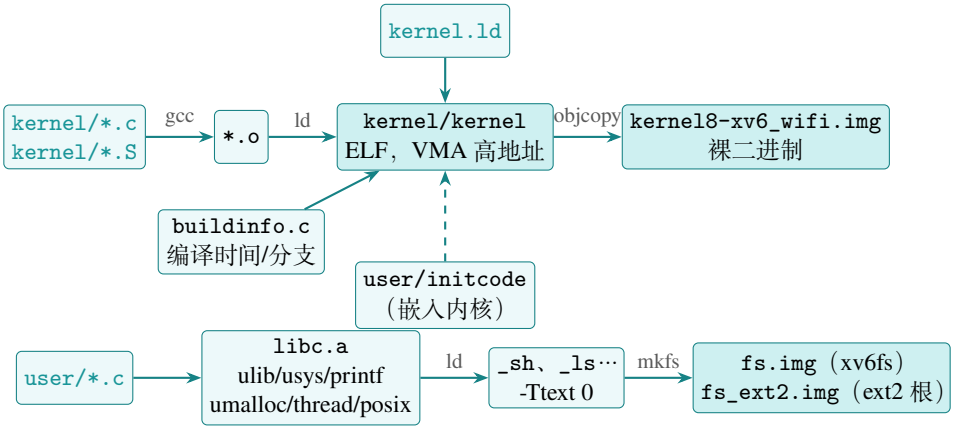


图 2.1: 构建流水线：上方生成内核镜像，下方生成文件系统镜像

链接脚本 kernel/kernel.ld 同时给出两个地址：

```
. = 0xffffff8000080000;          /* VMA：运行时的虚拟地址 */
```

```
.text : AT(0x80000) { ... }      /* LMA: 镜像装入的物理地址 */
```

ELF 中的所有符号都按高虚拟地址解析，但固件并不解析 ELF，它只是把文件内容原样拷到物理地址 0x80000。因此需要 objcopy -O binary 去掉 ELF 头，得到“从第一个字节开始就是第一条指令”的裸镜像。在 MMU 打开之前，代码实际运行在物理地址上，这段代码必须只使用 PC 相对寻址（adr/adrp），这是第 3 章要处理的问题。

每次链接都会重新生成 kernel/buildinfo.c，写入当时的日期和 git 分支、提交号，启动时打印出来：

```
kernel image built: 2026-10-05 14:02:11 CDT (rpi3-vfs ebefc6c+dirty)
```

真机调试时经常要确认“SD 卡上的内核是不是刚编译的那个”，这一行能省去很多困惑。

2.3.3 用户程序与文件系统镜像

用户程序先编译成 .o，再与 libc.a 静态链接。每个程序都链接在虚拟地址 0、入口为 main：

```
LIBC_OBJS = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o $U/thread.o \
            $U/posix.o $U/tweetnacl.o $U/ssh_crypto.o
_%: %.o $(ULIB)
    $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
```

usys.S 由 Perl 脚本 user/usys.pl 生成，每个系统调用对应三条指令（第 8 章）。程序最后被装进两种镜像。一种是 mkfs 生成的 32 MiB xv6 原生文件系统镜像 fs.img，烧到 SD 卡的 p2：

```
fs.img: mkfs/mkfs $(UPROGS)
    mkfs/mkfs fs.img $(UPROGS)
    truncate -s 32M fs.img
```

另一种是根文件系统用的 ext2 镜像 fs_ext2.img：make ext2root-image 先用 e2fsprogs 的 mke2fs 格式化一个空镜像，再用 debugfs 建目录、写入 /init、/bin/*、/lib/libc.so、/lib/ld-xv6.so 和 /lib/modules 下的 .ko，烧到 SD 卡的 p3（6.8.9 节）。

笔记 mkfs 是宿主机程序。如果仓库在 macOS 上编译过，mkfs/mkfs 就是一个 Mach-O 可执行文件，换到 Linux 构建时必须先删掉它重新编译，否则 make fs.img 会报“Syntax error”。

2.4 安装、运行与调试

Makefile 提供了几组目标，覆盖从模拟器到真机的完整工作流（表 2.3）。

表 2.3: 常用 make 目标

目标	作用
make	构建内核与 fs.img，并把内核同步到本机 TFTP 目录
make qemu	用 tools/ mkqemu_sd.py 拼出一张带 MBR 的虚拟 SD 卡（启动分区、fs.img、fs_ext2.img），再用 raspi3b 机器启动
make qemu-net	额外接入 QEMU 的 USB 网卡（CDC-ECM）和用户态 NAT
make qemu-gdb	启动后暂停，等待 GDB 连接
make install-rpi3	把内核、config.txt、Wi-Fi 固件拷到已挂载的启动分区；若给出 RPI3_XV6_DEV 则同时烧写根文件系统
make install-rpi3-rawfs	只把 fs.img 写入 SD 卡上的 xv6 原始分区
make install-rpi3-ext2root	重建 fs_ext2.img 并写入 RPI3_EXT2_DEV 指定的分区，同时把内核、config.txt 和 cmdline_ext2.txt（作为 cmdline.txt）装到启动分区
make install-tftp	把内核拷到宿主机 TFTP 目录，真机上用 tftp 命令下载到 /boot

QEMU 的 raspi3b 有两个串口：第一个是 PL011，第二个是 AUX Mini UART。本内核使用 Mini UART，因此 QEMU 参数要把第一个串口丢弃：

```
qemu-system-aarch64 -machine raspi3b -kernel kernel/kernel8-xv6_wifi.img \
    -display none -serial null -serial mon:stdio \
    -drive file=build/qemu-rpi3-sd.img,if=sd,format=raw
```

真机通过启动分区里的 config.txt 指定内核文件名和几个关键参数：

```
arm_64bit=1                     # 以 AArch64 状态启动 CPU
enable_uart=1                   # 打开 GPIO14/15 上的串口
uart_2ndstage=1                 # 固件自身的启动日志也输出到串口
disable_commandline_tags=1      # cmdline.txt 留给 xv6 自己读
core_freq=250                   # 固定 VPU 时钟：Mini UART 波特率依赖它
core_freq_min=250
kernel=kernel8-xv6_wifi.img
camera_auto_detect=0            # 不让固件接管摄像头
```

其中 core_freq=250 很重要：Mini UART 的波特率由 VPU 核心时钟分频得到，如果固件动态调频，串口就会乱码（第 3.1.4 节）。disable_commandline_tags=1 让固件不去处理 cmdline.txt，内核挂上启动分区后自己读它，从中取出 root= 等参数（6.7 节）。

2.5 内核初始化顺序

前面几节讲的是“文件放在哪里、怎样编出来”，这一节讲“启动后按什么顺序运行”。kernel/init/main.c 是整个内核的 C 入口，它的初始化顺序就是全书的目录。下面的代码去掉了注释中的细节，保留了调用顺序：

```
void main() {
  if(cpuid() == 0){
    kinit1(end, P2V(EARLYTOP));  // 只用启动页表覆盖的前 4 MiB     -> 第 4 章
    kvminit();  kvminithart();   // 建立并切换到最终内核页表       -> 第 4 章
    kinit2(P2V(EARLYTOP), P2V(PHYSTOP));
    kobject_subsys_init(); device_init();                        // -> 第 7 章
    sdio_bus_init(); usb_bus_init();
    consoleinit(); ttyinit(); printfinit();                      // 从这里起 printf 可用
    procinit(); mmap_init(); workqueue_init();                   // -> 第 5、4 章
    rnginit(); syncinit(); netinit();
    trapinit(); trapinithart(); gicv3init(); gicv3inithart();    // -> 第 3 章
    timerinit();
    binit(); iinit(); fileinit();                                // -> 第 6 章
    do_initcalls();                                              // 各驱动与文件系统，见下
    userinit();                                                  // 第一个用户进程
    start_secondary_cpus();                                      // 唤醒 CPU1~3
    started = 1;
  } else {
    while(started == 0) ;
    kvminithart(); trapinithart(); gicv3inithart(); timerinit();
  }
  scheduler();
}
```

驱动和文件系统不再手写在 main() 里。每个模块用 subsys_initcall()、fs_initcall()、module_init()、late_initcall() 这类宏把自己放进链接器生成的 .initcall.list 表，do_initcalls() 按级别从低到高、同级按序号依次调用（序号相同的按链接顺序）：

```
subsys   input_init()                输入子系统核心
         i2c_core_init()             I2C 核心
         blkdev_init()               块设备表，主设备号 BLOCKDEV
fs       fat32init()                 FAT32 驱动的全局状态
device   sd_driver_init()            SD 卡：add_disk(mmcblk0)，块层扫描 MBR，
                                     登记 mmcblk0p1 ~ p4
         arasan_sdio_driver_init()   1 SDIO 控制器（Wi-Fi 芯片挂在上面）
         usbkbd_driver_init()        2 USB 键盘
         dwc2_driver_init()          4 USB 主控制器
         ov5647_init()               5 摄像头传感器
late     rootdev_init()              挂起 FAT32 启动分区、读 cmdline.txt，
                                     选出 ROOT_DEV              -> 第 6 章
         brcmfmac_driver_init()      2 Wi-Fi，固件在启动分区里
         mt7601u_driver_init()       3 USB Wi-Fi，固件在启动分区里
         camera_init()               4 unicam 摄像头接口
         ext2init()                  5 只登记 ext2 文件系统类型
         vfsinit()                   6 登记 procfs、sysfs、fat32、netfs
```

几条顺序约束值得先记住，后面各章会解释原因：

- 页表必须在内存分配器之前就绪，但建页表本身又要分配页——所以分配器分两次初始化（kinit1/kinit2）。
- 设备核心（device_init）要在任何驱动注册之前初始化；控制台要尽早，因为之后的错误都要靠 printf 报告。
- 异常向量、中断控制器、定时器依次初始化，但真正打开 CPU 中断要等到 scheduler()。
- Wi-Fi 驱动需要从 FAT32 启动分区读取固件，所以放在 late 级、排在 rootdev_init() 之后——由它挂起 FAT32 启动分区；分区表则早在 device 级 SD 驱动 add_disk() 时由块层扫描好。
- 其余三个 CPU 最后才被唤醒，它们只做每核相关的初始化，然后直接进入调度器。
- 根文件系统不在 main() 里挂载：读超级块要用会睡眠的块缓存，必须等第一个进程在 forkret() 里调用 prepare_namespace()（6.8.5 节）。

下一章从 main() 之前开始：固件把 CPU 交给内核的第一条指令时，以上这些都还不存在。

第三章 基于汇编的裸机代码

本章内容

- ❑ 上电、链接地址与装载地址
- ❑ 异常级别：从 ARM9 的七种模式到 AArch64 的 EL0 ~ EL3
- ❑ 不依赖任何环境的早期串口
- ❑ 异常和中断：向量表、现场保存、中断控制器与定时器

3.1 节讲本项目的启动代码：从固件交出控制权、统一异常级别，到点亮早期串口。3.2 节讲异常和中断：向量表、现场保存、两级中断控制器和通用定时器。清 BSS、建启动页表、打开 MMU 和多核启动与页表关系密切，放在第 4 章的 4.2、4.3 节。这两节的代码全部来自 xv6-aarch64；它要兼容真机固件、QEMU 和多核，比教学内核复杂，所以每一步都先讲清楚背后的原理，这些原理主要参考了 Sergey Matyukevich 的 *raspberry-pi-os*[7]（下文简称 RPi OS）第 1 ~ 3 课。RPi OS 后三课的原理也按同样的方式融入了后面各章：第 4 课（进程调度器）在第 5 章，第 5 课（用户进程与系统调用）在第 8 章，第 6 课（虚拟内存管理）在第 4 章，那些地方同样只用本项目的代码来讲。

3.1 xv6-aarch64 的启动代码

3.1.1 上电之后发生了什么

树莓派的启动与 PC 不同：最先运行的不是 ARM 核，而是 VideoCore GPU。

1. GPU 内部 ROM 从 SD 卡的 FAT32 分区读取 `bootcode.bin`，再加载 `start.elf`（GPU 固件）；
2. `start.elf` 读取 `config.txt`，初始化 SDRAM 和时钟，把内核镜像 `kernel8-xv6_wifi.img` 拷到物理地址 `0x80000`；
3. 固件还在物理地址 0 处放一段很小的 *armstub*。它让四个 Cortex-A53 核一起开始执行：CPU0 跳到 `0x80000`，CPU1 ~ 3 在 *spin-table* 上循环等待——各自读取 `0xe0`、`0xe8`、`0xf0` 处的 64 位地址，非零就跳过去；
4. 固件自带的 64 位 armstub 会先把核从 EL3 降到 EL2，因此内核通常在 EL2 拿到控制权；QEMU 也是如此。

项目早期曾使用自制的极简 armstub（`kernel/arch/arm64/armstub.S`），它只做一次跳转、保留在 EL3。后来发现这样做会让固件的 property mailbox 在真机上完全没有响应——而 SD 时钟查询、USB 上电、摄像头供电都依赖 mailbox。所以现在直接使用固件自带的 armstub，内核启动代码则要能同时处理 EL3、EL2 两种入口。

固件、`config.txt` 与镜像文件名. 树莓派的固件是闭源的，ARM 一侧能控制它的只有启动分区里的 `config.txt`。从一张空白 SD 卡启动一个裸机内核，至少需要四个文件：GPU 引导程序 `bootcode.bin`、GPU 固件 `start.elf`、`config.txt` 和内核镜像。固件根据镜像文件名决定 CPU 的执行状态：以 `8` 结尾的 `kernel8.img` 表示 ARMv8，固件会让四个核以 AArch64 状态启动；本项目用 `kernel=` 指定了自己的文件名，同时写了 `arm_64bit=1`。

另一种启动方式：`kernel_old`. 最简单的裸机教程（如 raspberry-pi-os[7]）在 `config.txt` 中写 `kernel_old=1`。这时固件把镜像装在物理地址 0，并且不安装 armstub：四个核都从地址 0 开始执行同一段内核代码，而且停留在 EL3。内核必须自己读 `MPIDR_EL1` 的低 8 位区分核号，把非 0 号核挂进死循环，再自己从 EL3 降到 EL1。这种方式的好处是“什么都由自己掌控”，代价是失去了固件 armstub 提供的 spin-table 多核唤醒协议，也失去了 armstub 对 EL3 安全状态的初始化。本项目选择相反的路线：让固件做它擅长的事，内核只在 `_entry` 中兼容两种入口级别。

3.1.2 链接地址与装载地址

内核在高地址 0xffffff8000080000 链接，却被装到物理地址 0x80000。kernel/memlayout.h 用两个宏描述这种对应关系：

```
#define KERNBASE  0xffffff8000000000L     // 内核虚拟地址空间起点
#define KERNLINK  (KERNBASE + KERNPA)     // 内核链接地址
#define V2P(a) (((uint64)(a)) - KERNBASE)
#define P2V(a) ((void *)(((char *)(a)) + KERNBASE))
```

KERNBASE 选在 39 位地址空间的高半部分起点：当 TCR_EL1.T1SZ= 25 时，TTBR1_EL1 负责的范围恰好是 0xffffff8000000000 ~ 0xffffffffffffffff。内核把全部物理内存线性映射到 KERNBASE + PA，所以物理地址与内核虚拟地址之间只差一个常数。

MMU 打开之前，CPU 用物理地址取指。此时如果执行 ldr x0, =symbol 得到的是高虚拟地址，访问它会立即出错；只有 PC 相对寻址的 adr/adrp 能得到正确的物理地址。这一限制决定了 entry.S 前半部分的写法。

独立环境下的编译. 内核不能依赖任何运行时环境，编译时要明确告诉编译器这一点：不链接 C 标准库（库中的大多数函数最终都要调用操作系统，而这里下面已经没有操作系统了）；不使用标准启动文件（设置栈、初始化静态数据、调用 main 这些事都由启动汇编自己完成）；声明为独立（freestanding）环境，使编译器不对 memcpy 之类的函数做任何假设；只使用通用寄存器，不让编译器生成 NEON/浮点指令，否则上下文切换就必须额外保存这些寄存器。本项目对应的选项是 -ffreestanding -nostdlib -fno-common 和 -mcpu=cortex-a53+nofp（第 2 章）。

为什么要把 ELF 变成裸镜像. 链接器输出的是 ELF 文件，它需要一个“装载器”去解析段表、把各段放到指定地址。固件不做这件事，它只是把文件按字节原样拷到内存，然后从第一个字节开始执行。所以要用 objcopy -O binary 抽出所有代码段和数据段，拼成一个裸镜像，并且保证启动代码位于镜像的最开头：常见的做法是把启动代码放进单独的 .text.boot 段并在链接脚本中排在第一位；本项目则让 entry.o 成为 OBJS 的第一个目标文件，_entry 自然落在 .text 的起点。

3.1.3 统一异常级别

AArch64 有四个异常级别：EL0 运行用户程序，EL1 运行操作系统内核，EL2 运行虚拟机监控器，EL3 运行安全监控固件。xv6 只需要 EL0 和 EL1。

为什么需要异常级别. 异常级别可以看成处理器的几种执行模式，每种模式只能使用一部分指令和寄存器。操作系统要实现进程隔离，就必须让用户程序运行在特权最低的 EL0：在这一级只能使用通用寄存器、栈指针和普通的访存指令，不能修改页表基址等系统寄存器，因此一个进程既看不到也改不了别的进程的内存。操作系统本身运行在 EL1，可以配置虚拟内存和大多数系统寄存器。EL2 留给虚拟机监控器：宿主系统在 EL2 隔离各个运行在 EL1 的客户系统，就像操作系统隔离用户进程一样。EL3 负责在“安全世界”和“非安全世界”之间切换，两边的隔离由硬件保证，非安全世界的软件无论如何都访问不到安全世界的指令和数据。

ARM9 时代：七种处理器模式. 四个异常级别是 ARMv8 才有的说法。在 ARM9（ARM920T、ARM926EJ-S 等 ARMv4T/v5 内核，S3C2440、AT91 这类板子上最常见）时代，处理器用的是处理器模式（processor mode），当前模式记在 CPSR 的低 5 位 M[4:0] 里，一共七种（表 3.1）：用户模式（User）运行应用程序；系统模式（System）是“有特权的用户模式”；其余五种各对应一类异常——复位和 swi 指令进入管理模式（Supervisor，SVC），未定义指令进入未定义模式（Undefined），取指或数据访问出错进入中止模式（Abort），普通中断和快速中断分别进入 IRQ 模式和 FIQ 模式。

模式不等于特权级。 七种模式里只有 User 是非特权的，其余六种的特权完全相同，都能改 CPSR、访问 CP15 协处理器（MMU、缓存的控制寄存器都在这里）。所以 ARM9 实际上只有两个特权级，模式之间的区别不在于“能做什么”，而在于用哪一组寄存器：每个异常模式有自己私有（banked）的 r13（SP）、r14（LR）和 SPSR，FIQ 还额外私有 r8 ~ r12，这样快速中断处理程序不用保存任何通用寄存器就能开工。异常的类型直接决定进入哪个模式；向量表固定在地址 0x00000000（或 CP15 选择的高端 0xffff0000），每类异常占一个 4 字节的槽，只放得下一条跳转指令。

这种设计给操作系统带来不少麻烦。内核必须在启动时依次切进 IRQ、Abort、Undefined 各模式，为每个模式单独设置栈指针；而内核真正想在一个统一的上下文里处理所有陷入，所以 Linux 的 ARM 移植在每个异常入口只往该模式的小栈里存三个字（r0、lr、spsr），随即切换到 SVC 模式，在进程的内核栈上继续处理——七种模式在软件里又被折回成“用户态”和“内核态”两种。另外，特权代码可以直接用 msr cpsr_c, #0xd3 改写 CPSR 进入任意模式，模式切换并不一定经过异常。

TrustZone 与虚拟化：再加两种模式。 ARMv6K 和 ARMv7 引入安全扩展（TrustZone），处理器多了一个“安全/非安全”状态位，以及负责在两个世界之间切换的监控模式（Monitor），由 smc 指令进入。ARMv7 的虚拟化扩展（Cortex-A15 起）又加了一个 Hyp 模式，供虚拟机监控器使用，由 hvc 进入。为了说清楚这些模式之间的高低，ARMv7 手册开始使用特权级（PL0：User；PL1：六种原有特权模式加 Monitor；PL2：Hyp）。到这时已经有九种模式，而“特权高低”和“用哪组寄存器”这两件事仍然搅在一起。

AArch64：把模式折叠成异常级别。 ARMv8 的 AArch64 状态把这两件事拆开了。异常级别只表示特权的高低：EL0 对应原来的 User，EL1 一次吸收了 SVC、IRQ、FIQ、Abort、Undefined、System 六种模式，EL2 对应 Hyp，EL3 对应 Monitor。异常的类型不再决定模式，而由两样东西表达：

- 异常发生时跳到目标级别向量表里的哪一项。VBAR_ELx 指向的表有 16 项，按“来自哪一级、用哪个栈 × 同步/IRQ/FIQ/SError”排列，每项 128 字节，能直接放下一段保存现场的代码（3.2 节）；
- 同步异常的具体原因写进 ESR_ELx 的 EC 字段：svc 是 0x15，来自 EL0 的数据中止是 0x24，未定义指令是 0x00……原来要靠“当前处于 Abort 模式还是 Undefined 模式”来区分的事情，现在读一个寄存器就知道。

私有寄存器也大大精简：31 个通用寄存器不再分组，每一级只保留自己的栈指针 SP_ELx、返回地址 ELR_ELx 和保存状态 SPSR_ELx，FIQ 的快速寄存器组被取消。原来的 System 模式（特权级、但用 User 的寄存器）也有对应物：EL1t 在 EL1 下借用 SP_EL0，EL1h 使用自己的 SP_EL1（见下文 SPSR 的模式字段）。最重要的一点是：AArch64 里没有任何指令能直接改写当前级别，升级只能靠异常，降级只能靠 eret。

表 3.1: ARM9/ARMv7 处理器模式与 AArch64 异常级别

模式	CPSR.M	出现于	AArch64	何时进入（旧架构）
User (usr)	10000	ARMv4	EL0	应用程序运行
FIQ (fiq)	10001	ARMv4	EL1	快速中断；私有 r8 ~ r14
IRQ (irq)	10010	ARMv4	EL1	普通中断
Supervisor (svc)	10011	ARMv4	EL1	复位、swi/svc 系统调用
Abort (abt)	10111	ARMv4	EL1	预取中止、数据中止
Undefined (und)	11011	ARMv4	EL1	未定义指令
System (sys)	11111	ARMv4	EL1	无对应异常，由特权代码直接切入
Monitor (mon)	10110	ARMv6K/v7	EL3	smc，安全/非安全世界切换
Hyp (hyp)	11010	ARMv7VE	EL2	hvc、虚拟化陷入

这张对应关系并不只是类比。ARMv8 处理器仍能在 EL0、EL1 上运行 32 位代码（AArch32 状态），这时旧的模式依旧存在，而且严格按表 3.1 落在对应的异常级别上；那些私有寄存器也被映射到 64 位通用寄存器上（例如 IRQ 模式的 r13/r14 就是 X17/X16），所以 64 位内核切换到 32 位进程时不需要另外保存它们。对比早期面向树莓派 1（ARMv6）的 xv6 移植：它要在启动时为 IRQ、Abort、Undefined 各设一个栈，并在每个入口切回 SVC 模式；本项目在 AArch64 上只需要每个核一个 SP_EL1，所有异常都在同一个 EL1 里处理。

异常级别只能靠异常来改变. 没有更高特权软件的参与，程序无法提升自己的级别——否则隔离就形同虚设。级别只在异常发生和返回时改变。发生异常（非法访存、svc 指令、硬件中断等）并由 EL*n* 处理时，硬件依次做三件事：把返回地址存入 ELR_ELn，把当时的处理器状态存入 SPSR_ELn，跳到异常向量；处理程序结束时执行 eret，硬件再从这两个寄存器恢复状态和地址。关键在于这两个寄存器都是可写的，处理程序不一定要“回到原处”。启动时降级正是利用这一点。

当前级别可以从 CurrentEL 读出：它保存在第 [3:2] 位，右移 2 位即得 0 ~ 3。用不同的固件或 config.txt 启动时，读到的值可能是 3 或 2，所以 kernel/arch/arm64/entry.S 的第一件事就是读 CurrentEL，不论从 EL3 还是 EL2 进入，都降到 EL1：

```
_entry:
        mrs x15, mpidr_el1
        and x15, x15, #0xff          // 记下核号，后面区分主核与副核
        mrs x0, CurrentEL
        cmp x0, #0xc                 // CurrentEL[3:2] == 3 ?
        b.ne 2f
        mov x0, #(1 << 31)           // HCR_EL2.RW: EL1 使用 AArch64
        msr hcr_el2, x0
        msr cntvoff_el2, xzr         // 虚拟计数器偏移清零
        mov x0, #3
        msr cnthctl_el2, x0          // 允许 EL1 访问物理计数器/定时器
        mov x0, #0x431               // SCR_EL3: RES1 | RW | NS
        msr scr_el3, x0
        mov x0, #0x3c5               // 目标 EL1h，屏蔽 D/A/I/F
        msr spsr_el3, x0
        adr x0, 3f
        msr elr_el3, x0
        eret
2:      cmp x0, #0x8                 // EL2 路径与上面相同，只是写 *_el2
        ...
        eret
3:      ldr x0, =((3 << 28) | (3 << 22) | (1 << 20) | (1 << 11))
        msr sctlr_el1, x0            // 只保留 RES1 位：MMU、缓存先全部关闭
        isb
        cbnz x15, secondary_wait     // 副核去等待
        bl early_mini_uart           // 主核点亮串口
        b entry
```

“降级”的方式很有意思：处理器没有一条“切换到 EL1”的指令，只能伪造一次异常返回。把目标状态写进 SPSR_ELx、把返回地址写进 ELR_ELx，然后执行 eret，CPU 就会“返回”到 EL1 的标号 3 处。表 3.2 列出了涉及的寄存器。

表 3.2: 降级到 EL1 时设置的系统寄存器

寄存器	值	含义
HCR_EL2	bit 31	RW=1：下一级 EL1 以 AArch64 运行，否则会掉进 AArch32
CNTVOFF_EL2	0	虚拟计数器 = 物理计数器，内核用的是虚拟定时器
CNTHCTL_EL2	3	EL1/EL0 可以访问物理计数器和物理定时器
SCR_EL3	0x431	非安全状态，下一级为 AArch64
SPSR_ELx	0x3c5	M[3:0]=0101 即 EL1h（使用 SP_EL1）；DAIF 四位全部置 1
SCTLR_EL1	RES1 位	不继承固件留下的 MMU/缓存状态

SPSR 中的 D、A、I、F 四位分别屏蔽调试异常、SError、IRQ 和 FIQ。启动期间必须全部屏蔽：此时既没有异常向量表，也没有栈，任何一个中断都会让 CPU 跳到不可预知的地方。

SPSR 里保存的处理器状态. 写入 SPSR_ELx 的值就是 eret 之后的 PSTATE，它由三部分组成：条件标志 N、Z、C、V（上一次运算结果是否为负、为零、无符号溢出、有符号溢出，条件分支指令据此跳转）；中断屏蔽位 D、A、I、F；以及模式字段 M[3:0]，它决定返回到哪一级、使用哪个栈指针。EL1 有两种栈模式：EL1h 使用 EL1 自己的 SP_EL1，EL1t 借用 EL0 的 SP_EL0。内核总是使用 EL1h，这样陷入内核时自动换到内核栈，用户栈指针原样留在 SP_EL0 中。有的教学内核只屏蔽 A、I、F 三位（值 7<<6），本项目连同调试异常 D 一起屏蔽（0xf<<6）。

SCTLR_EL1 的初值. SCTLR_EL1 控制 EL1 下的 MMU、数据缓存、指令缓存和字节序。手册把其中若干位标为 RES1（保留，必须写 1），即上面那个 (3<<28)|(3<<22)|(1<<20)|(1<<11)；其余位全写 0，含义是：EL1 和 EL0 的数据访问都是小端，指令缓存、数据缓存和 MMU 全部关闭。不继承固件留下的任何状态，后面的代码才有确定的起点。

HCR_EL2 与 SCR_EL3. 即使不实现虚拟机监控器，也必须设置 HCR_EL2 的 RW 位，因为它决定 EL1 运行在 AArch64 还是 AArch32；同理，SCR_EL3 的 RW 位决定 EL2 的执行状态，NS 位让所有更低的级别处于非安全状态。漏掉 RW 位，eret 之后处理器会以 32 位状态执行 64 位指令，表现为“毫无输出地死掉”。

3.1.4 不依赖任何环境的早期串口

普通的 printf 依赖一长串设施：有效的栈、清零的 BSS、页表、内存分配器、自旋锁和控制台驱动。如果内核在这些设施就绪之前挂掉，串口上什么也看不到，连“死在哪一步”都无从判断。项目在真机上调通的第一个关键是：用纯汇编写一个不需要栈、不需要内存、不需要 C 运行环境的串口输出。

内存映射的设备寄存器. BCM2837 上所有外设都通过内存映射寄存器访问：树莓派 3 把 0x3F000000 以上的一段物理地址留给外设，每个寄存器是 32 位，读写某一位就是配置或启动设备的某项功能。Broadcom 手册中的地址以 0x7E 开头，那是 VideoCore 总线上的地址，ARM 一侧对应的物理地址要换成 0x3F 开头（第 4 章）。

树莓派 3 的 GPIO14/15 可以连到 PL011 UART0 或 AUX Mini UART。本内核使用 Mini UART，即 GPIO14/15 的 ALT5 功能。树莓派有两个 UART：PL011 功能完整，但在树莓派 3 上默认被板载蓝牙占用；Mini UART 结构简单，只需几个寄存器就能工作，代价是它的波特率依赖 VPU 核心时钟。初始化分三步：

第一步：选择 GPIO 复用功能. 大多数 GPIO 引脚可以连接不同的外设，每个引脚有 0 ~ 5 共六种“复用功能”，用 3 位编码，十个引脚共用一个 GPFSELn 寄存器。GPIO14、15 由 GPFSEL1 的第 14:12 位和 17:15 位控制；它们的 ALT5 分别是 Mini UART 的 TXD1 和 RXD1，ALT5 的编码是 010。所以要先把这两个 3 位字段清零，再各写入 2。注意 GPIO 编号与排针针脚号不同：GPIO14 是第 8 脚，GPIO15 是第 10 脚，第 6 脚是地。

第二步：关闭内部上下拉. 一个悬空的输入引脚读到的值是随机的；上拉或下拉电阻让它在悬空时稳定地读到 1 或 0。串口两根线始终接着对端，不需要上下拉，而且上下拉状态在重启后仍然保持，所以使用前要显式撤销。BCM2837 改变上下拉状态需要一个带时钟的握手过程：先在 GPPUD 中写入目标状态（0 表示都不要），等待约 150 个周期让控制信号建立；再在 GPPUDCLK0 中把要修改的引脚对应位写 1，把控制信号“打”进这些引脚，再等 150 个周期保持；最后清除 GPPUD 和 GPPUDCLK0。只有收到时钟的引脚会改变，其余引脚保持原状。

第三步：配置 Mini UART. 顺序很重要：先在 AUX 使能寄存器中打开 Mini UART，因为在此之前它的其他寄存器都不可访问；配置期间先关闭收发器，并永久关闭自动流控（它需要额外的 RTS/CTS 引脚，普通 USB-TTL 线不支持）；关闭收发中断（早期阶段没有中断处理）；把数据位设为 8 位——手册写的是第 0 位，实际硬件要第 0、1 两位都置 1，即写 3；RTS 置为常高；写波特率除数；最后打开收发器。

early_mini_uart 依次完成：

```
early_mini_uart:
        mov x13, x30                    // 没有栈，用寄存器保存返回地址
        ldr x9, =0x3f200004             // GPFSEL1: GPIO14/15 = ALT5 (功能码 2)
        ldr w10, [x9]
        ldr w11, =0x0003f000
        bic w10, w10, w11
        ldr w11, =0x00012000
        orr w10, w10, w11
        str w10, [x9]
        ...                             // GPPUD/GPPUDCLK0 时序：关闭内部上下拉
        ldr x9, =0x3f215000             // AUX 基址
        ...                             // 使能 AUX、8N1、清 FIFO
        mov w10, #270
        str w10, [x9, #0x68]            // AUX_MU_BAUD: 115200 @ 250 MHz
        mov w10, #3
        str w10, [x9, #0x60]            // 打开收发
        mov w12, #'E'
        bl early_mini_uart_putc
        ...
```

这里用的都是物理地址，因为 MMU 还没打开。Mini UART 的波特率公式是

$$\text{baud} = \frac{f_{\text{core}}}{8\,(\text{divisor}+1)}, \qquad \text{divisor} = \frac{250\,000\,000}{8 \times 115\,200} - 1 \approx 270.$$

它依赖 VPU 核心频率，所以 config.txt 必须固定 core_freq=250。

收发都用轮询：AUX_MU_LSR 的第 5 位为 1 表示发送器空闲，可以向 AUX_MU_IO 写一个字符；第 0 位为 1 表示收到数据，可以从 AUX_MU_IO 读出。早期汇编版只需要发送，所以 early_mini_uart_putc 只有三条指令：读 LSR、测第 5 位、写 IO。

在没有调试器的裸机上，串口输出几乎是了解程序内部状态的唯一途径。所以一个裸机项目通常要做的第二件事，就是在串口之上移植一个小型 printf——只需要提供一个发送单个字符的函数。本项目把这件事分成两级：汇编阶段只能输出单个标记字符，C 环境就绪后才由控制台驱动提供完整的 printf。

此后，启动代码在每个关键节点输出一个字符，形成一串“启动指纹”（表 3.3）。真机串口上看到的最后一个字符，就指明了内核停在哪一步。

表 3.3: 启动标记字符

标记	含义	标记	含义
E	进入 EL1，物理地址串口可用	1	kinit1() 完成
B	BSS 清零完成	2	最终内核页表建立
P	启动页表与系统寄存器就绪	3	切换到最终页表
H	MMU 已开，跳到高地址并建好栈	4	kinit2() 完成
0	进入 main()	a ~ f	kalloc.c 内部细分

正常启动时串口先打出 E，然后是 BPH0abdefc1234，接着才是 printf 的第一行“xv6 kernel is booting”。第 4 章会讲到一次停在 BPH0abde 的故障：它说明第一次 kfree() 已经完成 memset，却没能拿到 kmem.lock。

MMU 打开之后，物理地址 0x3f215000 不再可用。boot_uart_mark() 改用高地址 KERNBASE + 0x3f215000，所以启动页表里专门为 Mini UART 所在的 2 MiB 块加了一个设备映射。

到这里，主核已经处在 EL1、关着 MMU，并且有了一个能报告进度的串口；副核在 secondary_wait 中等待。接下来的 entry 要清 BSS、建启动页表、打开 MMU，这些步骤与页表的关系更紧密，放到第 4 章讲。下一

节先看另一件同样要在 main() 之前想清楚的事：异常和中断怎样进入内核。

3.2 异常与中断

与设备打交道的大多数场合都是异步的：内核向设备发出一条命令，设备并不立即完成，而是在工作结束后“通知”处理器。这种通知打断了正在执行的指令流，迫使处理器转去执行一段处理程序，所以叫“中断”。对操作系统来说，最重要的中断源是定时器：调度器靠周期性的定时器中断度量每个进程运行了多久，并在时间片用完时把 CPU 交给别人。

本节沿着一次中断从发生到返回的路径，依次讲解 xv6-aarch64 的实现：kernel/arch/arm64/trapasm.S（向量表与现场保存）、kernel/core/trap.c（分发）、kernel/arch/arm64/bcm2837.c（中断控制器）、kernel/arch/arm64/timer.c（定时器）和 kernel/core/spinlock.c（中断屏蔽的嵌套）。

3.2.1 异常与中断区别

在 ARMv8 中，中断只是“异常”的一种。异常分为四类，表 3.4 列出了它们在 xv6-aarch64 中的用途。

表 3.4: 四类异常在 xv6-aarch64 中的用途

类型	性质	在 xv6-aarch64 中
同步异常	由当前指令引起	用户态 svc #0 即系统调用；用户态缺页先交给 handle_mm_fault()（4.10 节），处理不了才打印诊断并杀死进程；内核态同步异常直接 panic
IRQ	外部硬件产生，与当前指令无关	通用定时器、Mini UART、DWC2 USB、Arasan SDIO、Unicam CSI、核间中断（IPI）
FIQ	高优先级的“快速中断”	不使用
SError	外部产生的系统错误	不处理，进入即停机

同步异常的具体原因记录在 ESR_EL1（异常综合征寄存器）中：高 6 位 [31:26] 是异常类别 EC，低 25 位 ISS 是该类别下的细节；出错的指令地址在 ELR_EL1，出错的数据地址在 FAR_EL1。xv6 只识别一种“正常”的同步异常——EC = 0x15（21），即来自 AArch64 EL0 的 svc；其余同步异常都按错误处理。内核态的同步异常不尝试恢复：

```
void kerneltrap()
{
  uint64 esr = r_esr_el1();
  w_esr_el1(0);
  if(intr_get() != 0)
    panic("kerneltrap: interrupts enabled");
  uint64 ec = (esr >> 26) & 0x3f;
  uint64 iss = esr & 0x1ffffff;
  printf("esr %p %p %p\n", esr, ec, iss);
  printf("elr=%p far=%p\n", r_elr_el1(), r_far_el1());
  panic("kerneltrap");
}
```

3.2.2 异常向量

每类异常都要有自己的处理入口；而且同一类异常，从不同的执行状态进入时也要用不同的入口。以运行在 EL1 的内核为视角，进入异常时的执行状态有四种：EL1t（EL1 使用 SP_EL0）、EL1h（EL1 使用自己的 SP_EL1，内核正是这种模式）、来自 64 位 EL0、来自 32 位 EL0。四类异常乘以四种状态，共 16 个入口，它们排成一张异常向量表：表必须 2 KiB 对齐，每个入口占 0x80 字节（表 3.5）。

表 3.5: VBAR_EL1 向量表布局

偏移	来源	同步	IRQ	本内核
0x000	当前 EL，使用 SP_EL0	+0x000	+0x080	未用（b .）
0x200	当前 EL，使用 SP_ELx	el1sync	el1irq	内核态异常/中断
0x400	低 EL，AArch64	el0sync	el0irq	系统调用、用户异常/中断
0x600	低 EL，AArch32	+0x600	+0x680	未用

每组的第三、四个入口是 FIQ 和 SError。kernel/arch/arm64/trapasm.S 用 .balign 保证对齐，每个入口只放一条跳转指令，真正的处理放在表外（0x80 字节只够放 32 条指令）：

```
.global alltraps
.balign 0x800
alltraps:
// Current EL with sp0
  b .
.balign 0x80
  b .
.balign 0x80
  b .
.balign 0x80
  b .

// Current EL with spx
.balign 0x80
  b el1sync
.balign 0x80
  b el1irq
.balign 0x80
  b .
.balign 0x80
  b .

// Lower EL using aarch64
.balign 0x80
  b el0sync
.balign 0x80
  b el0irq
.balign 0x80
  b .
.balign 0x80
  b .

// Lower EL using aarch32
  ...                       // 四个 b .
```

只有四个入口真正被使用。其余 12 个入口是 b .，一旦进入就原地停住。这在调试初期足够了，但也有缺点：FIQ 或 SError 发生时，现象只是“毫无输出地死机”。更好的做法是让这些入口也保存现场，然后打印入口编号、ESR_EL1 和 ELR_EL1 再停机。

3.2.3 异常级别与栈指针

ARMv8 有四个异常级别，本系统实际只常驻其中两个（表 3.6）。

表 3.6: 四个异常级别在 xv6-aarch64 中的用途

级别	一般用途	本移植
EL3	安全监控（Secure Monitor）	只在启动时经过：用自定义 armstub 时 CPU 从 EL3 起步
EL2	虚拟机管理器（Hypervisor）	只在启动时经过：固件的 armstub 和 QEMU 把 CPU 交到 EL2
EL1	操作系统内核	xv6 内核全部运行在这里，始终是 EL1h
EL0	用户程序	initcode、init、sh 等，EL0t、AArch64

3.1.3 节已经看到，`entry.S` 用伪造的异常返回（`SPSR = 0x3c5`，即 EL1h、DAIF 全屏蔽）把 CPU 从 EL3 或 EL2 降到 EL1。此后 EL2、EL3 不再使用，也没有安装 `VBAR_EL2`、`VBAR_EL3`。进入 EL0 用的是同一个技巧：`trapframe->spsr = 0`（`M = 0b0000`，即 EL0t，`DAIF` 全清），然后 `urestorereg; eret`。

每个异常级别都有自己的栈指针寄存器 `SP_EL0` ~ `SP_EL3`，指令里的 `sp` 指哪一个由 `PSTATE.M`（其中的 SPSel 位）决定，见表 3.7。

表 3.7: 执行模式与 `sp` 的对应

`PSTATE.M`	名称	级别	指令里的 `sp`
`0b0000`	EL0t	EL0	`SP_EL0`（EL0 只能用它）
`0b0100`	EL1t	EL1	`SP_EL0`
`0b0101`	EL1h	EL1	`SP_EL1`

后缀 t 指 thread，h 指 handler。向量表第一组“当前 EL，使用 `SP_EL0`”对应的就是异常发生时 CPU 处于 EL1t 的情形。有的内核让每个线程在 EL1t 下以 `SP_EL0` 作为线程栈，把 `SP_EL1` 留作每个 CPU 的专用异常栈（与 Cortex-M 的 PSP/MSP 双栈同一思路），这样线程栈坏了，异常处理仍有一块可靠的栈。xv6 不这样用：内核始终是 EL1h，`SP_EL0` 专门保存用户栈指针。

- **用户态（EL0t）**：`sp` 就是 `SP_EL0`，值是 `exec()` 设置的用户栈虚拟地址，经 `TTBR0_EL1` 翻译；
- **陷入时**：硬件把 `sp` 切到 `SP_EL1`，即当前进程内核栈的顶端；`SP_EL0` 保持用户的值不动，由 `usavereg` 用 `mrs x11, sp_el0` 存进 trapframe；
- **返回时**：`urestorereg` 用 `msr sp_el0, x11` 写回，`eret` 回到 EL0t。

用户无法读写 `SP_EL1`（`msr sp_el1` 是特权指令），所以用户把自己的栈弄坏了，陷入内核后用的仍是可靠的内核栈。这正是硬件为每个 EL 单独准备一个栈指针的意义。

3.2.4 各种事件落到哪一个入口

选择入口的规则只有一条：*看异常发生的那一刻* CPU 在哪一级、用的是哪个栈指针。处理程序本身总是运行在 EL1h、使用 `SP_EL1`。所以 `el0sync` 的意思是“来自 EL0 的同步异常”，它自己的每一条指令都在 EL1 执行；`el1sync` 则表示异常来自 EL1 本身。

“低一级 EL”又按低一级的执行状态分成 AArch64 和 AArch32 两组。32 位用户程序的寄存器视图不同（`r0` ~ `r14` 映射到 `x0` ~ `x14` 的低 32 位），系统调用的 EC 也不同（`0x11` SVC32 对 `0x15` SVC64），硬件直接让它们进不同的入口，内核不必先判断。Linux 打开 `CONFIG_COMPAT` 后就用 `0x600` 组运行 armhf 程序。本系统只装入 64 位 ELF，`spsr` 的 `M[4]` 总是 0，而执行状态只能在内核 `eret` 时切换，用户代码改不了，所以 `0x600` 组不可能被触发。也就是说，*在本系统里用户态的一切异常都落在 0x400 组*。

表 3.8 把常见事件逐一对照。

两点值得单独说明。

表 3.8: 常见事件与向量表入口

事件	入口	结果
用户执行 `svc`	`0x400 el0sync`	EC `0x15`，进入 `syscall()`；`ELR_EL1` 指向 `svc` 的下一条
系统调用里睡眠	不经过向量表	只是 `sleep` → `sched` → `swtch` 的函数调用
用户态时钟中断	`0x480 el0irq`	时间片用完则 `yield`；`ELR_EL1` 指向被打断的那条，返回后重新执行它
系统调用中来了中断	`0x280 el1irq`	在当前内核栈上再压 272 字节内核现场
内核访存出错、坏指令	`0x200 el1sync`	`kerneltrap()` 打印后 `panic`
用户访问 mmap 区里还没装入的页	`0x400`	EC `0x24`，`handle_mm_fault()` 装好 PTE 后返回，重新执行那条指令（4.10 节）
用户空指针、野指针、用户栈溢出	`0x400`	EC `0x24`（Data Abort），`FAR_EL1` 为出错地址；不在任何 VMA 里，进程被杀
用户跳到未映射地址	`0x400`	EC `0x20`（Instruction Abort）；进程被杀
用户执行未定义指令	`0x400`	EC `0x00`；进程被杀
用户执行浮点/SIMD 指令	`0x400`	EC `0x07`（`CPACR_EL1` 未开放 FP）；进程被杀
`sp` 或 PC 未对齐	`0x400`	EC `0x26` / `0x22`；进程被杀
用户整数除以 0	不产生异常	`sdiv/udiv` 结果为 0，程序继续运行
FIQ、SError、EL1t、AArch32	各自入口	`b .`，无声挂死

整数除以 0 不陷入. ARMv8 规定整数除法的除数为 0 时结果为 0，不产生任何异常，这与 x86 的 `#DE` 不同。要像 Linux 那样对除零报 `SIGFPE`，只能由编译器在除法前插入检查（例如 `-fsanitize=integer-divide-by-zero`）。浮点也走不到这一步：Makefile 用 `-mcpu=cortex-a53+nofp`，用户程序不会生成浮点指令；即使打开 FP，1.0/0.0 的结果也是 ∞，Cortex-A53 不支持 `FPCR` 的浮点异常陷入。

无声挂死的几个来源. 板子“突然不动、也没有 panic”时，可以怀疑以下几种情况：访问了不存在的外设地址引起 SError；某段代码误执行了 `msr spsel, #0` 后又发生异常，落进 EL1t 那一组；内核栈溢出。每个内核栈只有一页，`KSTACK()` 在栈与栈之间留了一页不映射的保护页，溢出时会触发 EL1 同步异常；但 `el1sync` 的 `savereg` 仍要往这个坏栈上压寄存器，于是再次出错、反复陷入，结果同样是没有任何输出。

3.2.5 保存寄存器状态

处理完异常回到原来的代码时，所有通用寄存器必须和异常发生前一模一样，否则一个与当前代码毫不相干的中断就会悄悄改变它的行为。进入异常时，硬件只做三件事：把返回地址存入 `ELR_EL1`，把原来的 PSTATE 存入 `SPSR_EL1`，换到 `SP_EL1`。通用寄存器要由软件自己保存。

`trapasm.S` 为两种来源准备了两对宏。来自内核的异常用 `savereg`/`restorereg`，现场压在当前内核栈上：

```
.macro savereg
        sub sp, sp, #272              // 17 组 × 16 字节
        stp x0, x1, [sp, #16 * 0]
        stp x2, x3, [sp, #16 * 1]
        ...
        stp x28, x29, [sp, #16 * 14]
        mrs x9, elr_el1
        mrs x10, spsr_el1
        add x11, sp, #272             // 异常发生前的内核 sp
        stp x30, x9, [sp, #16 * 15]
        stp x10, x11, [sp, #16 * 16]
.endm
```

来自用户态的异常用 usavereg/urestorereg，区别是最后一个字保存的是用户栈指针 SP_EL0，返回时也要写回：

```
.macro usavereg
        sub sp, sp, #272
        stp x0, x1, [sp, #16 * 0]
        ...
        stp x28, x29, [sp, #16 * 14]
        mrs x9, elr_el1
        mrs x10, spsr_el1
        mrs x11, sp_el0                 // 用户栈指针
        stp x30, x9, [sp, #16 * 15]
        stp x10, x11, [sp, #16 * 16]
.endm

.macro urestorereg
        ldp x30, x9, [sp, #16 * 15]
        ldp x10, x11, [sp, #16 * 16]
        msr elr_el1, x9
        msr spsr_el1, x10
        msr sp_el0, x11
        ldp x0, x1, [sp, #16 * 0]
        ...
        ldp x28, x29, [sp, #16 * 14]
        add sp, sp, #272
.endm
```

这 272 字节的布局与 struct trapframe（x0 ~ x30、elr、spsr、sp）完全一致。allocproc() 把每个进程的 trapframe 放在它内核栈的最顶端，而用户态陷入时内核栈恰好是空的，所以 usavereg 压栈的位置正好就是 p->trapframe。trap.c 开头的注释画出了两种现场同时存在时一个内核栈的样子：

```
低地址   p->kstack
           |  普通内核栈空间
           |  +---------------------------+  <- 内核态中断后 sp = S - 272
           |  | el1irq savereg 临时现场   |
           |  | x0..x30, elr, spsr, sp    |
           |  +---------------------------+  <- 中断前 sp = S
           |  函数调用帧、局部变量……
          +------------------------------
           |  struct trapframe            |  <- p->trapframe (usavereg 的位置)
           |  x0..x30, elr, spsr, 用户 sp |
高地址   p->kstack + PGSIZE
```

也就是说：进程在内核里执行系统调用时又被中断，会在同一个内核栈上再压一层 savereg 现场；如果中断发生在调度器中，用的则是这个 CPU 自己的调度器栈（stack0）。

有了调度器之后，“以一个进程的身份进入中断、以另一个进程的身份离开”是完全合法的：eret 依赖的 elr 和 spsr 都保存在当前进程自己的内核栈上，所以 ELR_EL1、SPSR_EL1 必须和通用寄存器一起保存和恢复。新进程第一次回到用户态走的是 usertrapret，它直接把 sp 指向 trapframe，借用 el0irq 的后半段返回：

```
el0irq:
```

```
        usavereg
        mov x0, sp
        bl userirq
trapret:
        urestorereg
        eret

.global usertrapret
usertrapret:                    // void usertrapret(struct trapframe *tf)
        mov sp, x0
        b trapret
```

3.2.6 设置向量表

向量表准备好了，还要告诉处理器它在哪里：把地址写入 VBAR_EL1（向量基址寄存器）。这是每个核各自的系统寄存器，所以每个 CPU 都要执行一次：

```
void trapinithart(void)
{
  w_vbar_el1((uint64)alltraps);
}

static inline void w_vbar_el1(uint64 x)     // kernel/aarch64.h
{
  asm volatile("msr vbar_el1, %0" : : "r" (x) );
}
```

alltraps 是链接时确定的高虚拟地址，所以这一步必须在 MMU 打开、跳到高地址之后进行。主核在 main() 中依次调用 trapinit()（只初始化 tickslock）和 trapinithart()；副核被唤醒后只调用 trapinithart()。

3.2.7 屏蔽与解除屏蔽中断

有些代码绝不能被中断打断。例如 usavereg 执行到一半时若又来一个中断，现场就会被覆盖。所以硬件在进入任何异常时都会自动屏蔽中断；保存好现场之后再解除屏蔽、允许嵌套，也是完全合法的。

PSTATE 中有四个屏蔽位：D（调试异常）、A（SError，也叫异步中止）、I（IRQ）、F（FIQ），用 msr daifset/daifclr, #imm 设置或清除，立即数是一个 4 位掩码，从高到低对应 D、A、I、F。xv6 的封装在 kernel/aarch64.h 中：

```
static inline void intr_on()  { asm volatile("msr daifclr, #0xf" ::: "memory"); }
static inline void intr_off() { asm volatile("msr daifset, #0xf" ::: "memory"); }

static inline int intr_get()          // IRQ 是否打开？
{
  uint64 x = daif();
  return ((x >> 6) & 0x2) == 0;       // DAIF 寄存器中 I 在 bit 7
}
```

注意这里用的是 #0xf，一次改变全部四位，而不只是 I 位（#2）。对 xv6 来说效果相同，因为它只用 IRQ；更精确的写法应只操作 I 位，以免意外打开调试异常或 SError。

3.2.7.1 自旋锁与中断的嵌套

如果一个 CPU 持有某把自旋锁时被中断，而中断处理程序又去获取同一把锁，这个 CPU 就会永远自旋下去。所以 xv6 规定：持有任何自旋锁期间，本 CPU 的中断都是关闭的。`acquire()` 调用 `push_off()`，`release()` 调用 `pop_off()`，二者成对嵌套计数：

```
void push_off(void)
{
  int old = intr_get();
  intr_off();
  if(mycpu()->noff == 0)
    mycpu()->intena = old;          // 只记录最外层进入前的状态
  mycpu()->noff += 1;
}

void pop_off(void)
{
  struct cpu *c = mycpu();
  if(intr_get())
    panic("pop_off - interruptible");
  if(c->noff < 1)
    panic("pop_off");
  c->noff -= 1;
  if(c->noff == 0 && c->intena)     // 最外层释放、且原来是开着的，才重新打开
    intr_on();
}
```

3.2.7.2 中断在哪里被打开

- 启动时，`entry.S` 通过 `SPSR = 0x3c5` 进入 EL1，四位全部屏蔽；
- 每个 CPU 第一次真正接收中断，是在 `scheduler()` 循环中调用 `intr_on()`；
- 用户态陷入后，`usertrap()` 识别出系统调用，在调用 `syscall()` 之前打开中断，让长时间的系统调用也能被设备中断和定时器打断；返回用户态前再 `intr_off()`，因为 `urestorereg` 期间不能被打断；
- `kerneltrap()` 进入时检查 `intr_get()`，若中断是开着的就说明某处逻辑出错，直接 `panic`。

3.2.8 配置中断控制器

设备一般不直接中断 CPU，而是经过中断控制器：控制器负责打开或关闭各个中断源，并告诉 CPU 当前是谁在请求中断。BCM2837 没有 ARM 标准的 GIC，而是两个级联的模块（图 3.1）：

- **legacy 中断控制器**（`0x3f00b000`）管理 SoC 共享外设：Mini UART（IRQ 29）、DWC2 USB（IRQ 9）、Arasan SDIO（IRQ 62）、Unicam CSI1（IRQ 39）等。中断号 0 ~ 31 由 `ENABLE_IRQS_1/IRQ_PENDING_1` 管理，32 ~ 63 由 `ENABLE_IRQS_2/IRQ_PENDING_2` 管理。所有外设中断汇总成一根“GPU IRQ”；
- **ARM-local 中断路由**（`0x40000000`）位于 CPU 簇旁，管理每个核私有的中断源：通用定时器、核间 mailbox、PMU，以及从 legacy 控制器汇总来的 GPU IRQ，最终连到四个核的 IRQ/FIQ 输入。

为了少改 `trap.c`，项目保留了原 GIC 版本的函数名 `gicv3init()`、`gicv3inithart()`、`gic_iar()`、`gic_eoi()`，实现换成了 `kernel/arch/arm64/bcm2837.c`。全局初始化先关掉 legacy 控制器的全部中断，再只打开已经有驱动的几个：

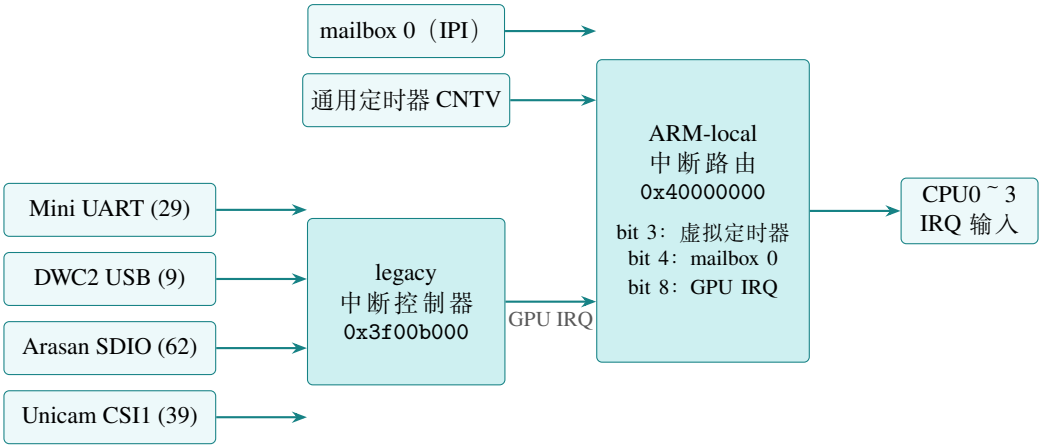


图 3.1: BCM2837 的两级中断结构

```
void gicv3init(void)
{
  irqwrite(DISABLE_IRQS_1, ~0U);
  irqwrite(DISABLE_IRQS_2, ~0U);
  irqwrite(DISABLE_BASIC_IRQS, ~0U);
  irqwrite(ENABLE_IRQS_1, (1U << UART0_IRQ) | (1U << DWC2_IRQ));
  irqwrite(ENABLE_IRQS_2, 1U << (SDIO_IRQ - 32));
}
```

后来加入的驱动（如摄像头）在 probe 成功后调用 `bcm2837_enable_irq(irq)` 打开对应位，按中断号小于还是大于 32 选择 `ENABLE_IRQS_1` 或 `_2`。

每个核还要各自配置 ARM-local 路由：把本核的虚拟定时器接到 IRQ，并打开 mailbox 0 作为“重新调度”核间中断：

```
#define CORE_TIMER_CONTROL(c) (0x40 + 4 * (c))
#define CORE_MBOX_CONTROL(c)  (0x50 + 4 * (c))
#define CORE_IRQ_SOURCE(c)    (0x60 + 4 * (c))
#define CORE_MBOX0_SET(c)     (0x80 + 0x10 * (c))   // 写 1 发出
#define CORE_MBOX0_CLR(c)     (0xc0 + 0x10 * (c))   // 写 1 清除
#define CORE_VIRTUAL_TIMER    (1 << 3)
#define CORE_MBOX0            (1 << 4)

void gicv3inithart(void)
{
  int cpu = cpuid();
  localwrite(CORE_TIMER_CONTROL(cpu), CORE_VIRTUAL_TIMER);
  localwrite(CORE_MBOX0_CLR(cpu), ~0U);     // 先清掉遗留的 mailbox 位
  localwrite(CORE_MBOX_CONTROL(cpu), 1);
}

void send_resched_ipi(int cpu)              // 让 cpu 尽快重新调度
{
  asm volatile("dsb sy" ::: "memory");      // 先让 need_resched 的写入可见
  localwrite(CORE_MBOX0_SET(cpu), 1);
}
```

一个外设中断要真正到达 CPU，三道开关缺一不可：设备自己的中断使能位（例如 Mini UART 的 MU_IER）、legacy 控制器的 ENABLE_IRQS 位，以及 CPU 的 PSTATE.I = 0。

3.2.9 通用 IRQ 处理程序

所有 IRQ 按来源分为两条路径：来自用户态的走前面已经看过的 el0irq（usavereg 后调用 userirq()），来自内核的走 el1irq，它用 savereg/restorereg 把现场压在当前内核栈上：

```
el1irq:                    // 内核执行时来的 IRQ
        savereg
        mov x0, sp
        bl kernelirq
        restorereg
        eret
```

两个 C 处理函数都先调用 devintr() 找出并服务中断源，再决定是否让出 CPU：

```
void userirq(void)
{
  struct proc *p = myproc();
  devintr();
  if(resched_pending())      // 时间片用完，或更高优先级进程就绪（定时器或 IPI）
    yield();
  if(p->killed)
    exit(-1);
  intr_off();
}

void kernelirq()
{
  devintr();
  // 内核态也可以被抢占：只有不持任何自旋锁时中断才是开着的
  if(myproc() != 0 && myproc()->state == RUNNING && resched_pending())
    yield();
}
```

devintr() 先问中断控制器“是谁”，再调用对应设备的处理函数，返回 2 表示定时器、1 表示其他设备、0 表示无法识别：

```
int devintr()
{
  uint32 iar = gic_iar();
  uint32 irq = gic_iar_irq(iar);
  int dev = 0;

  if(irq == UART0_IRQ){         uartintr();         dev = 1; }
  else if(irq == DWC2_IRQ){     dwc2_irq();         dev = 1; }
  else if(irq == SDIO_IRQ){     arasan_sdio_irq(); dev = 1; }
  else if(irq == CSI1_IRQ){     unicam_irq();       dev = 1; }
  else if(irq == TIMER0_IRQ){
    int logical_tick = timerintr();
```

```
    if(cpuid() == 0 && logical_tick)
      clockintr();
    sched_tick();
    dev = 2;
  } else if(irq == IPI_RESCHED_IRQ){
    resched_ipi_ack();          // need_resched 已由发送方设置
    dev = 1;
  } else if(irq == 1023){
    // 没有可处理的中断
  } else if(irq){
    printf("unexpected interrupt irq=%d\n", irq);
  }
  if(dev)
    gic_eoi(iar);
  return dev;
}
```

“是谁”的判断在 gic_iar() 中。它先查本核的 ARM-local 源寄存器（定时器和 IPI 是每核私有的），再查 legacy 控制器的挂起寄存器，把两套硬件的状态翻译成一个类似 GIC 的中断号：

```
uint32 gic_iar(void)
{
  int cpu = cpuid();
  uint32 local = localread(CORE_IRQ_SOURCE(cpu));
  if(local & CORE_VIRTUAL_TIMER)  return TIMER0_IRQ;        // 27
  if(local & CORE_MBOX0)          return IPI_RESCHED_IRQ;   // 伪中断号 1000
  if(irqread(IRQ_PENDING_1) & (1U << UART0_IRQ))       return UART0_IRQ;
  if(irqread(IRQ_PENDING_1) & (1U << DWC2_IRQ))        return DWC2_IRQ;
  if(irqread(IRQ_PENDING_2) & (1U << (SDIO_IRQ - 32))) return SDIO_IRQ;
  if(irqread(IRQ_PENDING_2) & (1U << (CSI1_IRQ - 32))) return CSI1_IRQ;
  return 1023;
}
```

BCM2837 没有 GIC 那样的 EOI 握手，gic_eoi() 是空函数：中断要在源设备处清除——定时器靠重新装载，UART 靠读空接收 FIFO，USB 靠写 HCINT，IPI 靠写 mailbox 清除寄存器。设备的挂起位清除之后，legacy 控制器中对应的位才会消失。多个中断可能同时挂起，devintr() 每次只处理优先级最高的一个；其余的会在 eret 之后立即再次触发中断，所以不会丢失。

3.2.9.1 同步异常：系统调用路径

同步异常走 el0sync/el1sync 入口，保存现场的方式与 IRQ 相同，只是调用的 C 函数不同。用户程序执行 svc #0 时，系统调用号放在 x7，参数在 x0 ~ x5：

```
user:  mov x7, #SYS_write ; svc #0
  -> VBAR_EL1 + 0x400 -> el0sync -> usavereg (现场 = trapframe)
  -> usertrap(): ESR_EL1.EC == 0x15
       -> intr_on(); syscall(): syscalls[x7](), 返回值写回 trapframe->x0
       -> 若唤醒了更高优先级进程: yield()
  -> intr_off(); urestorereg -> eret -> 回到 svc 的下一条指令
```

```
void usertrap(void)
{
  struct proc *p = myproc();
  uint64 esr = r_esr_el1();
  uint64 ec = (esr >> 26) & 0x3f;
  w_esr_el1(0);
  if(ec == 21){                          // SVC from AArch64
    if(p->killed) exit(-1);
    intr_on();
    syscall();
    if(resched_pending()) yield();
  } else if((ec == 0x24 || ec == 0x20) && user_page_fault(p, esr)){
    // 缺页落在某个 mmap() VMA 里，已经装好 PTE（4.7 节）
  } else {
    printf("usertrap(): unexpected esr=%p ec=%p pid=%d\n", esr, ec, p->pid);
    printf("            elr=%p far=%p\n", r_elr_el1(), r_far_el1());
    ...
    uvmdump(p->vm->pagetable, p->pid, p->name, "usertrap");
    p->killed = 1;
  }
  if(p->killed) exit(-1);
  intr_off();
}
```

硬件保存在 ELR_EL1 中的已经是 svc 的下一条指令地址，所以不需要像 RISC-V 那样手工给 PC 加 4。缺页则相反：ELR_EL1 指向出错的那条指令本身，user_page_fault() 装好页表项后直接返回，eret 会重新执行它。

笔记 早期版本的 usertrap() 先读出 EC 再清零 ESR_EL1，打印错误信息时又重新读一次寄存器，结果永远显示 ec=0。现在的代码先把完整的 ESR 存进局部变量 esr。调试信息本身的正确性，同样需要检查。

3.2.10 初始化定时器

xv6-aarch64 没有使用 BCM 自己的 System Timer，而是使用 ARM 架构定义的通用定时器（Generic Timer）。它的好处是：每个核都有自己的比较器，中断经 ARM-local 的 bit 3 直接送到本核，不经过 legacy 控制器；寄存器是系统寄存器，用 mrs/msr 访问，不需要 MMIO 映射。内核使用虚拟定时器那一组：

- CNTFRQ_EL0：计数器频率（由固件设置）；
- CNTVCT_EL0：当前虚拟计数值，所有核共享，可用来测量时间（clock_us()、delay() 都基于它）；
- CNTV_TVAL_EL0：距离触发还剩多少个计数，写入即开始倒计时；
- CNTV_CTL_EL0：bit 0 ENABLE、bit 1 IMASK、bit 2 ISTATUS。

timerinit() 的顺序是“关闭—装载—打开”，避免改写比较值的过程中产生一个中间中断：

```
#define CNTV_CTL_ENABLE   (1<<0)
#define CNTV_CTL_IMASK    (1<<1)

void timerinit()
{
  disable_timer();
  reload_timer();
  enable_timer();
}
```

```
static void disable_timer()
{
  uint64 c = r_cntv_ctl_el0();
  c &= ~CNTV_CTL_ENABLE;
  c |= CNTV_CTL_IMASK;
  w_cntv_ctl_el0(c);
}

static void reload_timer()
{
  uint64 interval = 10000;                                    // 微秒，即 10 ms
  uint64 interval_clk = interval * (r_cntfrq_el0() / 1000000);
  w_cntv_tval_el0(interval_clk);
}

static void enable_timer()
{
  uint64 c = r_cntv_ctl_el0();
  c |= CNTV_CTL_ENABLE;
  c &= ~CNTV_CTL_IMASK;
  w_cntv_ctl_el0(c);
}
```

timerinit() 由每个核各自调用。执行完之后定时器已经开始倒计时，但 PSTATE.I 仍然屏蔽着 IRQ，要等进入 scheduler() 调用 intr_on() 时 CPU 才会真正接收它。

3.2.11 处理定时器中断

定时器到期后，中断经 ARM-local 路由到本核，gic_iar() 返回 TIMER0_IRQ，devintr() 调用 timerintr()：

```
int timerintr()
{
  int cpu = cpuid();
  int logical_tick;

  disable_timer();
  reload_timer();                          // 重新装载即清除了中断条件
  timer_divider[cpu]++;
  logical_tick = timer_divider[cpu] == 10;
  if(logical_tick)
    timer_divider[cpu] = 0;
  if(cpu == 0)
    workqueue_timer_tick();                // 推进 delayed-work 时钟
  enable_timer();
  return logical_tick;                     // 每 10 次 (100 ms) 返回 1
}
```

一次 10 ms 的硬件中断在 devintr() 中引出三件事：

1. **系统时间**：每 10 次中断构成一个 xv6 逻辑 tick。只有 CPU0 调用 clockintr() 把全局 ticks 加一并

`wakeup(&ticks)`，避免四个核把时间推进四倍。`sleep()`、`uptime()` 因此保持 xv6 原来 100 ms 一个 tick 的语义：

```
void clockintr()
{
  acquire(&tickslock);
  ticks++;
  wakeup(&ticks);
  release(&tickslock);
  net_udp_timeout_tick();       // 检查带超时的 UDP 接收是否到期
}
```

2. **下半部时钟**：CPU0 推进 workqueue 的 delayed-work 时钟，TCP 重传、Wi-Fi 连接超时等到期的工作被放进各自的工作队列，由内核线程执行，而不是在中断里直接运行；
3. **调度节拍**：每次中断都调用 `sched_tick()`，扣减当前进程的时间片、累计实时任务的运行时间，必要时设置 `need_resched`（第 5 章）。随后 `userirq()`/`kernelirq()` 检查 `resched_pending()`，决定是否 `yield()`。

`reload_timer()` 写的是相对倒计时，所以下一次中断发生在“本次处理程序重新装载的时刻 + 10 ms”，处理程序的延迟会累积成时钟漂移。若要更稳定的时基，可以改写绝对比较值 `CNTV_CVAL_EL0`，每次在上一个截止时间上加固定间隔。

3.2.12 小结

把本节的代码按 `main()` 中的调用顺序串起来：

```
trapinit();         // 初始化 tickslock
trapinithart();     // VBAR_EL1 = alltraps                    (每个核)
gicv3init();        // legacy 控制器：全关，再打开 UART/USB/SDIO
gicv3inithart();    // ARM-local：本核虚拟定时器 + mailbox 0   (每个核)
timerinit();        // 10 ms 虚拟定时器开始倒计时              (每个核)
...
scheduler();        // intr_on()：第一次真正接收中断
```

顺序不能随意交换：先安装向量表，保证中断来了有入口；再配置控制器和路由；再启动定时器；最后才解除 PSTATE 的屏蔽。要收到第一个定时器中断，下面这些条件必须同时满足：

- 处于 EL1h，且 EL2 已允许 EL1 访问计数器和定时器（`CNTHCTL_EL2`）；
- `VBAR_EL1` 指向 2 KiB 对齐的 `alltraps`；
- `CORE_TIMER_CONTROL(cpu)` 打开了虚拟定时器路由；
- `CNTV_TVAL_EL0` 装入了非零值，`CNTV_CTL_EL0` 的 ENABLE = 1、IMASK = 0；
- PSTATE.I = 0，且当前内核栈有效，有空间保存 272 字节的现场。

调试中断问题时，可以在启动阶段打印 `CurrentEL`、`VBAR_EL1`、`DAIF`、`CNTFRQ_EL0` 和 `CNTV_CTL_EL0`；对未知中断打印 `IRQ_PENDING_1/2` 和 `CORE_IRQ_SOURCE`。但不要在定时器中断里持续 `printf`：串口远比中断慢，会改变调度时序，甚至带来新的锁问题。

到这里，异常和中断的通路已经齐备，但 `main()` 能走到 `trapinithart()`，前提是 MMU 已经打开、内核已经跳到高地址运行。下一章回到 `entry.S`，看启动页表和多核启动。

第四章 MMU

本章内容

- 39 位虚拟地址与三级页表
- 硬件如何遍历页表、块映射
- 描述符格式与内存属性（MAIR）
- 清 BSS、启动页表（恒等映射加高地址映射）、TCR/MAIR/SCTLR、跳到高地址
- 多核启动：两道闸门
- 两阶段的物理内存初始化：struct page、mem_map 与伙伴系统
- 最终内核页表与用户页表
- 第一个进程、fork 与地址空间切换；TLB；缺页与按需分配
- mmap()：VMA、按需调页、共享页与设备映射
- MMIO、总线地址与 DMA
- 三个缓存与 TLB 故障
- 实验分支：引用计数与写时复制

第 3 章结束时，CPU 已经处在 EL1，但 MMU 还关着。本章先讲 AArch64 怎样翻译地址，再沿着启动顺序看页表怎样一步步建起来：`entry.S` 的启动页表、副核唤醒、物理页分配器、`main()` 里的最终内核页表，然后是每个进程的用户页表和 `mmap()`。最后几节转向页表的另一面——它不只决定地址，还决定 CPU 怎样访问一块内存：设备寄存器、DMA 缓冲区，以及三个只在真机上出现的缓存与 TLB 故障。

4.1 AArch64 的地址翻译

4.1.1 地址空间划分

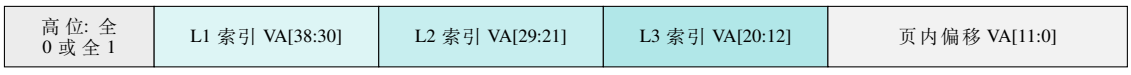
本内核使用 4 KiB 页、39 位虚拟地址。`TCR_EL1` 中 `T0SZ=T1SZ= 25`，即每个地址空间有 $2^{64-25} = 2^{39}$ 字节（512 GiB）。AArch64 用虚拟地址的高位选择页表基址寄存器：

- 高位全 0：`0x0000000000000000` ~ `0x0000007fffffffff`，由 `TTBR0_EL1` 翻译，用于**用户空间**，每个进程一张；
- 高位全 1：`0xffffff8000000000` ~ `0xffffffffffffffff`，由 `TTBR1_EL1` 翻译，用于**内核空间**，所有进程共享一张；
- 中间的地址不合法，访问即产生翻译错误。

两个基址寄存器分开带来一个很大的好处：进程切换时只需改写 `TTBR0_EL1`，内核映射保持不变，也不用像 xv6-riscv 那样在每个用户页表里复制一份内核映射和 trampoline 页。

4.1.2 三级页表

39 位地址用 4 KiB 粒度时，从第 1 级开始遍历，共三级（没有第 0 级）。每级页表占一页，含 512 个 8 字节表项，用虚拟地址中的 9 位索引（图 4.1）：



每级 9 位 = 512 项；L1 一项管 1 GiB，L2 一项管 2 MiB，L3 一项管 4 KiB

图 4.1: 39 位虚拟地址的划分

`kernel/aarch64.h` 用两个宏取出各级索引：

```
#define PXMASK          0x1FF                   // 9 位
#define PXSHIFT(level)  (39-(level)*9)          // L1: 30, L2: 21, L3: 12
#define PX(level, va)   ((((uint64)(va)) >> PXSHIFT(level)) & PXMASK)
```

第 1、2 级表项可以是*表描述符*（指向下一级页表）或*块描述符*（直接映射 1 GiB/2 MiB 的连续物理内存）；第 3 级表项是*页描述符*。启动页表用 2 MiB 块，最终页表全部用 4 KiB 页。

4.1.3 硬件如何遍历页表

虚拟内存给每个进程一个错觉：整个地址空间都归它一个人用。进程发出的每个地址都是虚拟地址，由 MMU（内存管理单元）在访存的路上翻译成物理地址，进程本身完全察觉不到。这样做同时解决了两个问题：一是*隔离*，进程只能访问自己页表里映射了的内存，碰不到别人的数据，也碰不到内核；二是*分配简单*，每个进程的程序都可以从地址 0 开始链接，不必关心物理内存里哪些地方已经被别人占了。

MMU 翻译一个地址的过程叫*页表遍历*（table walk），完全由硬件完成，软件只负责事先把页表在内存里摆好、再把根页表的物理地址写进 `TTBR0_EL1` 或 `TTBR1_EL1`。以用户地址为例：

1. 从 `TTBR0_EL1` 取出 L1 页表的**物理**地址；
2. 用 VA[38:30] 作下标读出 L1 表项。若它是表描述符，其中 [47:12] 位就是 L2 页表的物理地址；
3. 用 VA[29:21] 在 L2 页表中读出表项，同理得到 L3 页表的物理地址；
4. 用 VA[20:12] 在 L3 页表中读出页描述符，得到物理页框号；
5. 物理页框号拼上 VA[11:0] 的页内偏移，就是最终的物理地址。

每一步读到的地址都是物理地址——MMU 遍历页表时不可能再去翻译页表自己的地址。这就是为什么 `walk()` 往表项里写的是 `V2P(pagetable)`，读出来以后又要 `P2V()` 才能被内核软件访问（4.8.2 节）。途中任何一级遇到无效表项（bit 0 为 0），遍历就停下来，CPU 产生一个*翻译错误*（translation fault），并把出错的级别记在 `ESR_EL1` 里、出错的虚拟地址记在 `FAR_EL1` 里。

块映射. 有时候要映射一大片连续的物理内存，例如启动时的整个内核镜像。这时可以让 L2 表项直接是一个*块描述符*，指向一个 2 MiB 对齐的物理块，遍历在第 3 步就结束：省去一级页表，页内偏移变成 VA[20:0] 共 21 位（2^{21} = 2 MiB）。同理，L1 块描述符映射 1 GiB。描述符最低两位区分这几种情况：在 L1/L2 上 `01` 是块、`11` 是表；在 L3 上只有 `11` 合法，表示一个 4 KiB 页——同样是 `11`，含义随所在的级别而变。xv6-aarch64 只在启动页表里用块映射（4.2 节）。

实例：一次真实的三级翻译. 图 4.2 用一个进程快照里的真实页表项，把上面五步走了一遍：pid 16 的用户栈地址 `0x89d0`。`TCR_EL1.T0SZ = 25`，即 39 位 VA。VA[63:39] 全为 0，选 `TTBR0_EL1 = 0x6eb8000`；L1、L2、L3 的下标分别是 0、0、8，页内偏移是 `0x9d0`。MMU 依次读三个物理地址：

- 读 `0x6eb8000`，得到表描述符 `0x6eb5003`；
- 读 `0x6eb5000`，得到表描述符 `0x6ec0003`；
- 读 `0x6ec0040`，得到页描述符 `0x6ece74b`。

页框 `0x6ece000` 拼上偏移，得到物理地址 `0x6ece9d0`。描述符低 12 位 `0x74b` 的含义见下一小节：AttrIndx=2 表示普通可缓存内存，AP=01 表示 EL0 可读写，SH=11，AF=1，UXN 和 PXN 都是 0。

4.1.4 描述符的属性位

表 4.1 是项目中用到的属性位。

4.1.5 内存属性间接表

页表项里并不直接写“可缓存”“设备内存”之类的属性，而是写一个 3 位的索引，真正的属性放在 `MAIR_EL1` 的 8 个字节里。项目定义了三种类型：

```
#define AI_DEVICE_nGnRnE_IDX  0x0
#define AI_NORMAL_NC_IDX      0x1
```

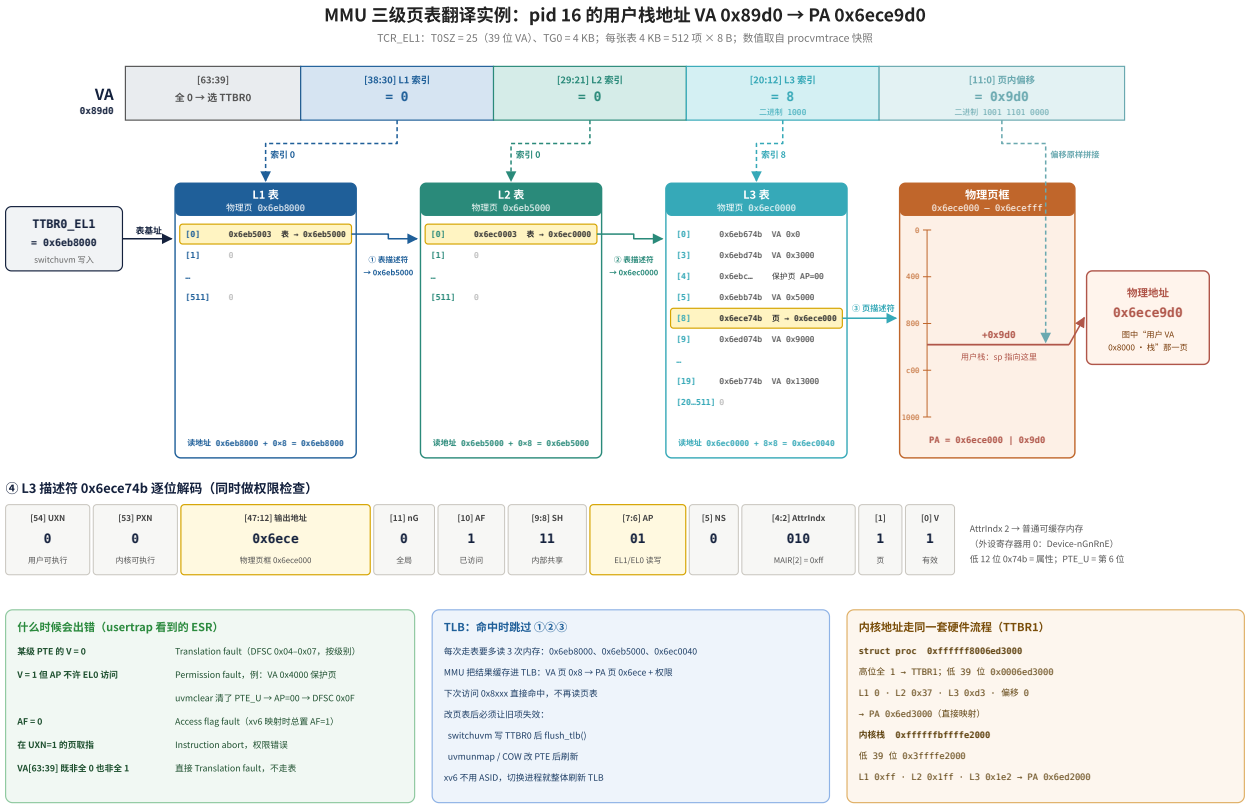


图 4.2: MMU 三级页表翻译实例：VA 0x89d0 → PA 0x6ece9d0（数值取自进程快照）

表 4.1: 页表项属性位（kernel/aarch64.h）

宏	位	含义
PTE_VALID / PTE_TABLE	[1:0]	L1/L2：01 块，11 表；L3：11 页（PTE_V）
PTE_INDX(i)	[4:2]	AttrIndx：选择 MAIR_EL1 中的第 *i* 个内存类型
PTE_U、PTE_RO	[7:6]	AP：EL0 是否可访问、是否只读
PTE_SH_INNER	[9:8]	共享属性：11 为 Inner Shareable
PTE_AF	[10]	Access Flag：必须置 1，否则首次访问触发 Access Fault
PTE_PXN、PTE_UXN	53、54	内核态 / 用户态禁止执行

```
#define AI_NORMAL_IDX          0x2
#define MT_DEVICE_nGnRnE  0x00    // 设备内存：不聚合、不重排、无提前写确认
#define MT_NORMAL_NC      0x44    // 普通内存，不可缓存
#define MT_NORMAL         0xff    // 普通内存，Inner/Outer Write-Back

#define PTE_DEVICE    PTE_INDX(AI_DEVICE_nGnRnE_IDX)
#define PTE_NORMAL_NC PTE_INDX(AI_NORMAL_NC_IDX)
#define PTE_NORMAL    (PTE_INDX(AI_NORMAL_IDX) | PTE_SH_INNER)
```

用途划分很简单：RAM、内核、用户页面和页表本身用 PTE_NORMAL；UART、GPIO、SD 控制器等外设寄存器用 PTE_DEVICE；摄像头 DMA 缓冲区用 PTE_NORMAL_NC，这样就不必在每帧前后做缓存维护（第 7 章）。

4.2 启动页表

第 3 章的启动代码把处理器降到 EL1、点亮串口之后，内核还运行在物理地址上，MMU 和缓存都是关着的。本节接着讲从这里到跳进高地址的三步：清 BSS、建临时页表、打开 MMU。

4.2.1 清 BSS 与早期栈

C 语言假定未初始化的全局变量为 0，这些变量位于 .bss 段。镜像文件里并不包含 BSS 的内容，所以要先清零。链接脚本导出了 bss_start 和以 8 字节为单位的 bss_size：

```
entry:
        adrp x1, bss_start          // PC 相对：MMU 未开时得到物理地址
        ldr w2, =bss_size
1:      cbz w2, 2f
        str xzr, [x1], #8
        sub w2, w2, #1
        b 1b
2:      ...                         // 输出 'B'
```

把需要清零的数据单独放在 .bss 段，是为了节省镜像空间：ELF 只记录这个段的大小，不存它的内容。代价是装载之后必须由启动代码自己清零，所以链接脚本要导出段的起止位置，并让起点按 8 字节对齐。对齐在这里不只是为了效率：MMU 关闭时所有数据访问都按 Device 内存处理，非对齐访问会直接触发对齐异常，因此一次写 8 字节的 str xzr 要求地址必须是 8 的倍数。

栈也是同样的道理：C 代码需要一个可用的栈指针。最简单的做法是把 sp 设到内核镜像上方一个“足够远”的固定地址（例如内核装在 0 时设为 4 MiB），栈向下增长也碰不到内核。本项目要支持四个核同时运行，所以在 BSS 中定义了 stack0[4096 * NCPU]，每个核按核号取其中一页（见 4.2.3 节末尾的 _start）。清完 BSS 以后，主核先临时把 sp 设成 stack0 + 4096（这时还是物理地址），因为下一步建启动页表要调用一个 C 函数。

4.2.2 两棵临时页表

MMU 打开前要先建两棵只有两级的小页表，四张页表页（l1entrypgt、l2entrypgt、l1kpgt、l2kpgt）静态定义在 start.c 中、按页对齐。

如图 4.3，恒等映射保证“打开 MMU 的那条指令之后还能取到下一条指令”，高地址映射保证“跳到链接地址之后一切照常”。两者都用 2 MiB 块描述符，所以每棵树只要一张 L1 和一张 L2。早期版本在 entry.S 里用汇编循环填表；现在 entry.S 只把四张表的物理地址和 EARLYTOP 放进 x0 ~ x4，调用 start.c 里的 C 函数

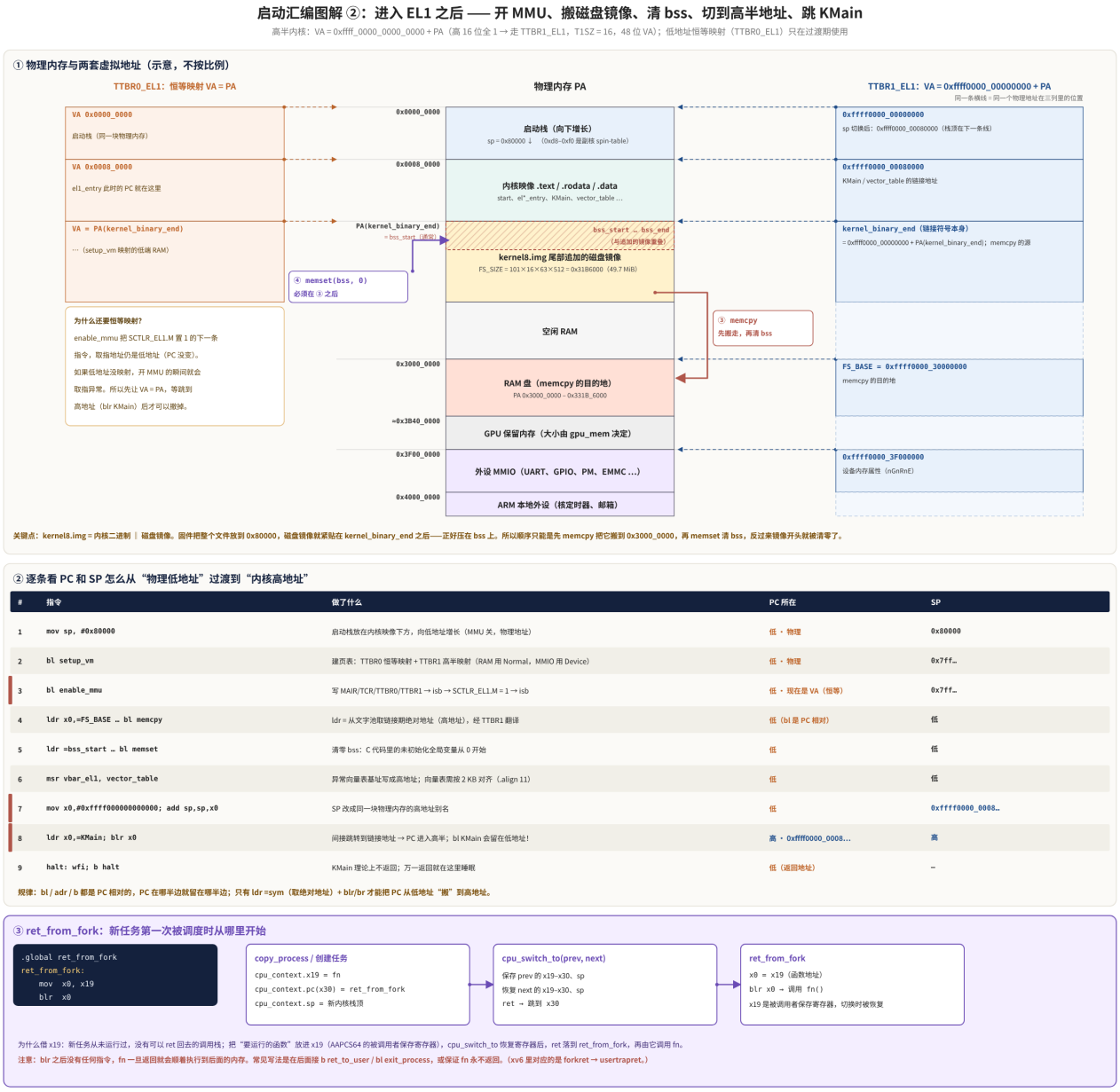


图 4.3: 启动页表：同一段物理内存同时映射到低地址和高地址。TTBR0_EL1 恒等映射 VA = PA，只在开 MMU 的过渡期使用；TTBR1_EL1 映射 VA = KERNBASE + PA。图中按通用写法标的高地址 0xffff0000_…，在 xv6 里一律读作 KERNBASE = 0xffffff80_00000000 加物理地址，例如物理 0x80000 对应 0xffffff80_00080000，外设 0x3f000000 对应 0xffffff80_3f000000

entrypgtinit()。MMU 没开时 C 代码同样运行在物理地址上，只要它不访问全局变量的绝对地址，就和汇编没有区别：

```
void entrypgtinit(pte_t *l1entry, pte_t *l2entry, pte_t *l1kernel, pte_t *l2kernel,
                  uint64 pa_end)                    // pa_end = EARLYTOP
{
  const uint64 l2block = 1UL << PXSHIFT(2);         // 2 MiB
  for(uint64 pa = 0; pa < pa_end; pa += l2block)    // 恒等映射
    l2entry[PX(2, pa)] = PA2PTE(pa) | PTE_AF | PTE_NORMAL | PTE_VALID;
  l1entry[PX(1, 0)] = PA2PTE((uint64)l2entry) | PTE_TABLE | PTE_VALID;

  for(uint64 pa = 0; pa < pa_end; pa += l2block)    // KERNBASE + PA
    l2kernel[PX(2, KERNBASE + pa)] = PA2PTE(pa) | PTE_AF | PTE_NORMAL | PTE_VALID;
  uint64 uart_pa = 0x3f200000UL;                    // Mini UART 所在的 2 MiB
  l2kernel[PX(2, KERNBASE + uart_pa)] = PA2PTE(uart_pa) | PTE_AF | PTE_DEVICE | PTE_VALID;
  l1kernel[PX(1, KERNBASE)] = PA2PTE((uint64)l2kernel) | PTE_TABLE | PTE_VALID;
}
```

启动页表覆盖物理内存的前 4 MiB（EARLYTOP=0x400000，两个 2 MiB 块），外加 Mini UART 所在的一个设备块，供 boot_uart_mark() 在开 MMU 之后继续输出标记。EARLYTOP 原来是 2 MiB，只盖住内核镜像；后来内核的 BSS 变大，kinit1() 在 [end, 2 MiB) 里只剩 65 页，而 kvminit() 逐页映射 112 MiB 内存需要约 70 个页表页，启动时就 panic: kvmmap，所以改成 4 MiB。main() 一开始仍检查内核镜像连同 BSS 没有越过这个边界——否则后面的分配器会把内核自己的内存当成空闲页发出去：

```
if(V2P(end) > EARLYTOP){
  for(char *m = "\nkernel image extends past EARLYTOP\n"; *m; m++)
    boot_uart_mark(*m);
  for(;;) asm volatile("wfe");
}
```

4.2.3 打开 MMU：TCR、MAIR 与 SCTLR

页表摆好以后，还要告诉 MMU 怎样解释它们。主核在 _entry、副核在 entryothers 里执行的是同一段代码：

```
        adrp x0, l1entrypgt
        adrp x1, l1kpgt
        msr  ttbr0_el1, x0        // 低地址：恒等映射
        msr  ttbr1_el1, x1        // 高地址：内核
        ldr  x0, =(TCR_T0SZ(25)|TCR_IRGN0(1)|TCR_ORGN0(1)|TCR_SH0(3)|TCR_TG0(0)| \
                   TCR_T1SZ(25)|TCR_IRGN1(1)|TCR_ORGN1(1)|TCR_SH1(3)|TCR_TG1(2)|TCR_IPS(0))
        msr  tcr_el1, x0
        ldr  x1, =((MT_DEVICE_nGnRnE<<(8*AI_DEVICE_nGnRnE_IDX)) | \
                   (MT_NORMAL_NC<<(8*AI_NORMAL_NC_IDX)) | (MT_NORMAL<<(8*AI_NORMAL_IDX)))
        msr  mair_el1, x1
        isb
        ldr  x1, =_start          // 链接地址（高地址）
        dsb  ish
        tlbi vmalle1              // 丢掉固件可能留下的翻译
        dsb  ish
```

```
        isb
        ic   iallu
        dsb  sy
        isb
        mrs  x0, sctlr_el1
        orr  x0, x0, #1           // M: 打开 MMU
        orr  x0, x0, #(1 << 2)    // C: 数据缓存
        orr  x0, x0, #(1 << 12)   // I: 指令缓存
        msr  sctlr_el1, x0
        isb
        br   x1                   // 跳到高地址
```

TCR_EL1（Translation Control Register）的各个字段决定了前面所有的“规则”：

- T0SZ=T1SZ= 25：两个地址空间都是 39 位，于是遍历从 L1 开始、一共三级；
- TG0= 0、TG1= 2：两边都是 4 KiB 粒度。两个字段的编码不一样（TG0 中 00 是 4 KiB，TG1 中 10 才是 4 KiB），抄错一个数字，高地址的页表就会被按 64 KiB 或 16 KiB 粒度解释；
- IRGN/ORGN= 1、SH= 3：MMU 遍历页表时本身也要读内存，这几位规定这些读取走 Write-Back 缓存、属于 Inner Shareable 域。它们必须和页表页本身的映射属性（PTE_NORMAL）一致，否则 CPU 写进缓存的新表项，MMU 可能看不到（第 4.12.1 节的故障与此同源）；
- IPS= 0：物理地址 32 位（4 GiB），对 BCM2837 足够。

MAIR_EL1 把三种内存类型写进第 0、1、2 字节，页表项的 AttrIndx 就是这里的下标。SCTLR_EL1 是总开关：M 位打开地址翻译，C、I 位打开数据和指令缓存。打开前的 tlbi vmalle1 和 ic iallu 清掉固件阶段可能残留的 TLB 和指令缓存；写 SCTLR_EL1 之后的 isb 保证后续取指都已经按新的翻译进行。此时 PC 仍是低地址，靠 TTBR0 的恒等映射继续执行，直到 br x1 跳进高地址——这就是启动页表要同时建两棵树的原因。

4.2.4 跳到高地址

ldr x1, =_start 从文字池中取出链接时确定的高虚拟地址；这里不能写 b _start：打开 MMU 并不会改变 PC，而 b 和 adr 都是相对当前 PC 计算目标的，会落到“没开 MMU 时 _start 所在的低地址”。必须事先用 ldr 从文字池取出链接器算好的高地址绝对值；这条 ldr 本身也是 PC 相对的，所以要在打开 MMU 之前执行。br x1 之后，CPU 就运行在 TTBR1_EL1 映射的高地址上了。_start 为每个核在 stack0 中划出 4 KiB 栈，然后跳进 C 函数 main()：

```
_start:
        ldr x0, =stack0
        mov x1, #1024*4
        mrs x2, mpidr_el1
        and x2, x2, #0x3
        add x2, x2, #1
        mul x1, x1, x2
        add x0, x0, x1              // sp = stack0 + (cpuid + 1) * 4096
        mov sp, x0
        mov w0, #'H'
        bl boot_uart_mark
        b main
```

4.3 多核启动

四个核在上电后都会执行 `_entry`——CPU0 由 armstub 直接跳入，CPU1 ~ 3 则要等主核在 spin-table 里写入地址。如果像 `kernel_old=1` 那样没有 armstub，四个核会同时从同一个入口开始执行，内核要立刻用 `MPIDR_EL1` 的低 8 位区分核号，把除 0 号核以外的所有核挂进死循环——这是单核教学内核的标准做法，但被挂起的核就再也没有机会醒来了。本项目需要四核同时调度，所以副核不能死循环，而要在一个可以被唤醒的地方等待。整个过程有两道“闸门”：

第一道闸门：启动页表。 副核到达 `_entry` 后，统一好异常级别（3.1.3 节）就进入 `secondary_wait`，用 `ldar`（带 acquire 语义的加载）读取标志 `secondary_release`，为 0 就 `wfe` 休眠。主核建好启动页表后，用 `stlr`（带 release 语义的存储）把标志置 1，再执行 `sev` 唤醒所有在 `wfe` 上休眠的核：

```
        adrp x10, secondary_release           // 主核
        add x10, x10, :lo12:secondary_release
        mov w11, #1
        stlr w11, [x10]
        sev

secondary_wait:                               // 副核
        adrp x10, secondary_release
        add x10, x10, :lo12:secondary_release
4:      ldar w11, [x10]
        cbnz w11, entryothers
        wfe
        b 4b
```

acquire/release 配对保证：副核看到标志为 1 时，主核在此之前写入的页表内容也一定可见。

第二道闸门：内核初始化。 主核在 `main()` 的最后调用 `start_secondary_cpus()`，把 `_entry` 的物理地址写进三个 spin-table 槽，用 `dc cvac` 把这几行缓存写回内存（此时副核的 MMU 和缓存都没打开，只能看到内存里的值），再 `sev`：

```
static void start_secondary_cpus(void) {
  volatile uint64 *spin = (volatile uint64*)P2V(0xe0);
  uint64 entry = V2P((uint64)_entry);
  for(int cpu = 1; cpu < NCPU; cpu++){
    spin[cpu - 1] = entry;
    asm volatile("dc cvac, %0" :: "r"(&spin[cpu - 1]) : "memory");
  }
  asm volatile("dsb sy\n\tsev" ::: "memory");
}
```

副核进入 `main()` 后在 `while(started == 0)` 处等待主核完成全部初始化，然后只做每核相关的部分：切换到最终页表、安装异常向量、打开本核的定时器中断路由，最后进入调度器。真机串口上会看到：

```
PHOhart 3 starting
hart 2 starting
hart 1 starting
```

三个副核几乎同时出发，谁先抢到 `printf` 的锁谁先打印，所以顺序不固定；`PH0` 是不经过锁的早期标记，会与其他输出交错。

4.4 从设备树到 memblock

建立最终页表需要分配页表页，分配页表页又需要物理页分配器，而物理页分配器的 `mem_map` 又要按 RAM 的大小来开——这是一个“先有鸡还是先有蛋”的问题。早先的 xv6 用常量回避它：`PHYSTOP=112 MiB` 是 RAM 的大小，`kinit1()` 先把 `[end, EARLYTOP)` 交给分配器建页表，`kinit2()` 再交出 `[EARLYTOP, PHYSTOP)`。树莓派 3 实际有 1 GiB 内存（扣掉 GPU 部分约 960 MiB），常量让八成内存闲置；换一块板子还得改源码重编。

Linux 的回答分两步：先从固件拿到内存布局（x86 的 e820、UEFI 内存表，ARM 上是设备树），记进 `memblock` 这个只有两张区间表的启动期分配器；等页表和 `struct page` 数组都就绪，`memblock_free_all()` 再把剩下的空闲内存一次性移交给伙伴系统。当前源码树照这个形状实现了一个迷你版本（`kernel/mm/fdt.c`、`kernel/mm/memblock.c`，合计约 500 行），`main()` 的开头变成：

```
memblock_init(boot_dtb_pa);  // 从 DTB 读出 RAM 与保留区
kvminit();                   // 最终内核页表，页表页来自 memblock_alloc()
kvminithart();               // 切换 TTBR1，作废 TLB
page_alloc_init();           // mem_map 也来自 memblock，然后 memblock_free_all()
```

4.4.1 设备树从哪里来

AArch64 的启动约定是：引导程序把扁平设备树（flattened device tree，FDT，也叫 DTB）的物理地址放在 `x0` 里跳进内核。树莓派固件按 `config.txt` 加载 `bcm2710-rpi-3-b.dtb`，填好 `/memory` 节点后传给内核；QEMU 的 `-dtb` 选项做同样的事，并把 `/memory` 改写成模拟的内存大小。`entry.S` 的第一条指令就把 `x0` 存进 `x21`——后面清 BSS、建启动页表都不能碰它——等启动页表建好后再写进全局变量 `boot_dtb_pa`。

此时只有启动页表，它只覆盖物理前 4 MiB，而 QEMU 把 DTB 放在 `0x08000000`，固件则通常放在 RAM 顶部附近。所以 `entrypgtinit()` 多收一个参数：DTB 地址非零、8 字节对齐、低于外设窗口时，把它所在的 2 MiB 块和下一块（DTB 不超过 2 MiB，可能跨块）也映射进高地址的 `l2kernel`，`memblock_init()` 就能经 `P2V()` 直接读它。

4.4.2 解析 FDT

FDT 是一段大端格式的二进制：40 字节的头（魔数 `0xd00dfeed`、总长、结构块和字符串块的偏移、版本），之后是 `/memreserve/` 表（若干 `< 地址，长度 >` 对，以 0 结尾）、结构块和字符串块。结构块是一串 32 位的 token：

```
FDT_BEGIN_NODE 1   // 后跟以 NUL 结尾、4 字节对齐的节点名
FDT_END_NODE   2
FDT_PROP       3   // 后跟 len、nameoff（指向字符串块）和 len 字节的值
FDT_NOP        4
FDT_END        9
```

`fdt_scan()` 不建树，只顺序走一遍 token，用一个深度不超过 8 的小栈记住每层的 `#address-cells/#size-cells`——它们描述的是子节点 `reg` 的格式，所以解析一个节点的 `reg` 要用父节点的值，而且属性可能出现在 `reg` 之后，于是 `reg` 留到 `FDT_END_NODE` 才上报。它关心三类节点：

- 根下名为 `memory` 或 `memory@...`、或 `device_type = "memory"` 的节点：`reg` 里每一对都回调 `memblock_add()`；

- /reserved-memory 的子节点：有 reg 的回调 memblock_reserve()；只有 size 的（如 CMA 池）是“请内核找个地方”，迷你版不支持，跳过；
- 根节点的 model 字符串，仅用于打印。

/memreserve/ 表里的区间同样进 reserved（树莓派的 DTB 用它保护物理第 0 页的 armstub 与 spin-table）。头部校验失败（魔数不对、地址不对齐、长度超过 2 MiB、版本低于 16）或扫描中遇到越界 token，memblock_init() 就丢掉已经加进去的区间，退回内置布局。

4.4.3 memblock：两张有序区间表

```
struct memblock_region { uint64 base, size; };
struct memblock_type   { int cnt; uint64 total; char *name;
                         struct memblock_region regions[32]; };
struct memblock { uint64 current_limit;
                  struct memblock_type memory, reserved; };
```

两张表都按地址排序，memblock_add() 和 memblock_reserve() 插入时与重叠或相邻的区间合并，memblock_remove() 必要时把一个区间劈成两段。表是静态数组：此时还没有任何分配器，Linux 也一样先用静态的 128 项数组。memblock_init() 在 DTB 之外再登记几样东西：

```
if(memblock.memory.cnt == 0){           // 没有 DTB，或 DTB 不可用
  memblock_source = "built-in";
  memblock_add(0, PHYSTOP);             // 退回原来的 112 MiB
}
memblock_remove(PERIPHERAL_BASE_PA, ~0ULL - PERIPHERAL_BASE_PA);
memblock_reserve(0, PGROUNDUP(V2P(end)));                  // 第 0 页与内核镜像
memblock_reserve(CARVEOUT_START, CARVEOUT_END - CARVEOUT_START); // ramdisk/摄像头 DMA/模块区
memblock_set_current_limit(EARLYTOP);   // 现在只有前 4 MiB 有映射
```

PHYSTOP 因此从“RAM 的大小”降格为“没有 DTB 时的内置大小”。0x07000000 起的 6 MiB 窗口原先在 PHYSTOP 之上，分配器天然碰不到；现在 RAM 远大于它，就得显式保留。DTB 本身也被保留，以后还可以再读。

memblock_alloc(size, align) 和 Linux 一样**自顶向下**找：从最高的 memory 区间、current_limit 以下开始，取对齐的候选地址；若与某个 reserved 区间重叠，就退到那个区间的下方再试。分到的区间立即记进 reserved，所以它永远不会再交给伙伴系统。current_limit 是这里的关键：建最终页表时它等于 EARLYTOP，保证页表页落在启动页表已经映射的 4 MiB 里；切换到最终页表后它抬到 RAM 顶部，mem_map 就可以放到任何地方。

4.5 物理内存初始化与伙伴系统

在伙伴系统就绪之前，kalloc() 直接转给 memblock_alloc(PGSIZE, PGSIZE)，kvmmake() 和 walk() 不用知道背后是谁。内核页表永不释放，这些页留在 reserved 里正合适；在 buddy_ready 置位之前调用 kfree() 会直接 panic。

4.5.1 从空闲链表到伙伴系统

最初的分配器就是经典 xv6 的空闲链表：每个空闲页的前 8 字节存放下一页的地址，kalloc() 从表头取，kfree() 往表头放。它简单可靠，但有三个缺陷：

- **只能分配单页**。链表里相邻的两项在物理上毫无关系，拿不到连续的多页。于是需要连续物理内存的地方只能另想办法：virtio 驱动用静态数组 `pages[2*PGSIZE]`，`init_module` 用静态的 `module_image[]`，摄像头 DMA 缓冲和模块区则干脆划在当时的 RAM 上限 `PHYSTOP` 之上，绕开分配器。
- **页没有元数据**。分配器不知道一页是空闲还是在用、被谁用，重复 `kfree()` 同一页不会报错，只会让链表成环，很久以后才以莫名其妙的方式崩溃。也没有地方放引用计数，写时复制无从谈起。
- **统计要遍历**。`/proc/meminfo` 的 MemFree 靠 `kfreepages()` 把整条链表走一遍。

现在的 `kernel/mm/kalloc.c` 换成了 Linux 式的两件套：为每个物理页框准备一个描述符 `struct page`，组成数组 `mem_map`；空闲内存按 2^{order} 页的块组织在 `free_area[order]` 里，也就是伙伴系统。`kalloc()`/`kfree()` 的接口原样保留，变成 order 0 的薄封装，全内核四十多处调用点一行都不用改。

4.5.2 struct page 与 mem_map

`include/kernel/page.h` 定义了描述符：

```
struct list_head {
  struct list_head *next, *prev;
};

struct page {
  struct list_head lru;          // PG_buddy: 挂在 free_area[order] 上
                                 // PG_pcp: 挂在每 CPU 链表上
                                 // PG_slab: 挂在 kmem_cache 的 partial 链表上
  uint8  flags;                  // PG_*
  uint8  order;                  // 空闲块或多页分配的阶
  uint16 inuse;                  // PG_slab: 这个 slab 已分出的对象数
  int    refcount;               // 0 空闲，分配后为 1
  struct kmem_cache *slab_cache;// PG_slab: 所属的 cache
  void  *freelist;               // PG_slab: 这个 slab 的第一个空闲对象
};

#define PG_reserved 0x01         // 从不交给伙伴系统（内核镜像、mem_map 本身）
#define PG_buddy    0x02         // 空闲块的首页，挂在 free_area[] 上
#define PG_head     0x04         // order > 0 分配块的首页
#define PG_slab     0x08         // 属于某个 kmem_cache（见 slab 一节）
#define PG_pcp      0x10         // 空闲，暂存在每 CPU 链表上
```

一个描述符 40 字节（最初只有前 24 字节，第 4.6 节为 slab 加了后两个字段；Linux 是 64 字节，同样按页的用途复用字段），RAM 里每 4 KiB 一个，物理页号 `pfn` 就是数组下标。地址换算因此都是常数时间：

```
struct page *pfn_to_page(uint64 pfn) { return mem_map + pfn; }
uint64 page_to_pfn(struct page *pg)  { return pg - mem_map; }
struct page *virt_to_page(void *kva) { return pfn_to_page(V2P(kva) >> PGSHIFT); }
void *page_address(struct page *pg)  { return P2V(page_to_pfn(pg) << PGSHIFT); }
```

`mem_map` 自己也要占物理内存，而它的长度由 `memblock_end_of_DRAM()` 决定：QEMU 给出 960 MiB 时是 245 760 个描述符、9 600 KiB，没有 DTB 时是 28 672 个、1 120 KiB。9.4 MiB 已经放不进启动页表的 4 MiB，所以 `page_alloc_init()` 放在 `kvminithart()` 之后——此时最终页表已经线性映射了全部 RAM，可以先抬高 `current_limit`，再像 Linux 的 `alloc_node_mem_map()` 那样从 memblock 里要一块：

```
max_pfn = memblock_end_of_DRAM() >> PGSHIFT;
```

```
size = max_pfn * sizeof(struct page);
memblock_set_current_limit(max_pfn << PGSHIFT);  // 最终页表已覆盖全部 RAM
if((pa = memblock_alloc(PGROUNDUP(size), PGSIZE)) == 0)
  boot_halt("\npage_alloc_init: no memory for mem_map\n");
mem_map = (struct page *)P2V(pa);

memset(mem_map, 0, size);
for(uint64 pfn = 0; pfn < max_pfn; pfn++)
  mem_map[pfn].flags = PG_reserved;       // 先全部保留
memblock_free_all(free_range);            // 只有空闲区间才清掉 PG_reserved
buddy_ready = 1;
```

自顶向下的分配把 mem_map 放在 RAM 的最高处（QEMU 下是 0x3b6a0000），低端留给 DMA 等可能在意地址的用途，这也是 Linux 的习惯。

4.5.3 伙伴分配与合并

空闲内存按块组织。free_area[order] 是一条双向链表，挂着所有大小为 2^{order} 页的空闲块，MAX_ORDER=11，块大小从 4 KiB 到 4 MiB。每个块都**自然对齐**：首页的 pfn 是 2^{order} 的倍数。只有块的首页带 PG_buddy 并挂在链表上，块内其余页的描述符不参与。

自然对齐带来了伙伴系统名字的由来：一个 order 阶的块，与它“结伴”的另一半的首页号只差第 order 位，即

$$\text{buddy_pfn} = \text{pfn} \oplus 2^{order}.$$

分配时从请求的阶往上找第一个非空链表，取下一块；如果块比需要的大，就不断对半切开，把上半块挂回低一阶的链表（Linux 里叫 expand()）：

```
for(o = order; o < MAX_ORDER; o++){
  if(list_empty(&zone.free_area[o].free_list))
    continue;
  pg = lru_to_page(zone.free_area[o].free_list.next);
  del_from_free_area(pg, o);
  while(o > order){              // 把用不到的上半块还回低阶链表
    o--;
    add_to_free_area(pg + (1UL << o), o);
  }
  break;
}
```

释放时反过来：只要伙伴也是同阶的完整空闲块，就把它从链表摘下、合成高一阶的块，一直合到伙伴不空闲或到达最高阶为止（__free_one_page()）：

```
while(order < MAX_ORDER - 1){
  uint64 buddy_pfn = pfn ^ (1UL << order);
  struct page *buddy;

  if(buddy_pfn >= max_pfn)
    break;
  buddy = pfn_to_page(buddy_pfn);
```

```
  if((buddy->flags & PG_buddy) == 0 || buddy->order != order)
    break;
  del_from_free_area(buddy, order);
  pfn &= ~(1UL << order);        // 合并后的块从低半块开始
  order++;
}
add_to_free_area(pfn_to_page(pfn), order);
```

PG_reserved 页永远不带 PG_buddy，合并自然在内核镜像和 mem_map 的边界处停下，不需要任何特判。

图 4.4 以从一个空的 order 3 块里分配一页为例：8 页的块被切三次，留下 order 2、1、0 各一块；这一页释放时，三次合并按相反顺序发生，恢复成原来的 order 3 块。

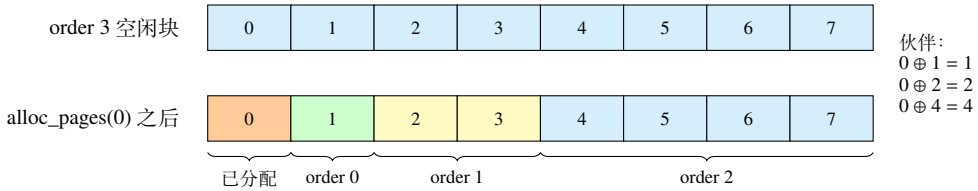


图 4.4: 从 order 3 块分配一页：切三次，上半块依次挂回 order 2、1、0；释放时按 pfn $\oplus 2^{order}$ 找伙伴逐级合并

4.5.4 把内存交给伙伴系统

memblock_free_all() 遍历 memory 减去 reserved 后剩下的每一段 [start, end)，对每段调用 free_range()。最早的实现逐页 kfree()，每页还要先填一遍垃圾；新实现像 Linux 的 __free_pages_memory() 那样，每次取当前位置能放下的**最大对齐块**直接挂上去：

```
while(pfn < end_pfn){
  int order = MAX_ORDER - 1;
  while(order > 0 &&
        ((pfn & ((1UL << order) - 1)) != 0 || pfn + (1UL << order) > end_pfn))
    order--;
  for(uint64 i = 0; i < (1UL << order); i++){
    struct page *pg = pfn_to_page(pfn + i);
    pg->flags &= ~PG_reserved;
    pg->refcount = 0;
  }
  __free_one_page(pfn, order);  // 可能与上一段交出的块合并
  zone.managed_pages += 1UL << order;
  pfn += 1UL << order;
}
```

保留区把空闲内存切成几段，相邻两段交出的块在 __free_one_page() 里照常合并。在 QEMU 上，受管理的页数是 managed_pages= 241 333（约 943 MiB），/proc/meminfo 的 MemTotal 报告的就是这个数，而不是 RAM 的总大小——与 Linux 的 MemTotal 只计 managed pages 一致。启动时 memblock_dump() 打印两张表：

```
memblock: memory map from DTB at pa 0x0000000008000000 (89494 bytes),
          model "Raspberry Pi 3 Model B"
  memory.cnt = 1, total = 983040 KiB
    [0] 0x0000000000000000-0x000000003bffffff 983040 KiB
  reserved.cnt = 5, total = 17707 KiB
```

```
    [0] 0x0000000000000000-0x00000000001c3fff 1808 KiB
    [1] 0x00000000003ef000-0x00000000003fffff 68 KiB
    [2] 0x0000000007000000-0x00000000075fffff 6144 KiB
    [3] 0x0000000008000000-0x0000000008015d95 87 KiB
    [4] 0x000000003b6a0000-0x000000003bffffff 9600 KiB
buddy: 241333 pages managed, 241333 free,
       mem_map 9600 KiB at pa 0x000000003b6a0000
```

五个保留区依次是：第 0 页加内核镜像与 BSS；最终内核页表的 17 个页表页（自顶向下，紧贴 EARLYTOP 之下）；ramdisk、摄像头 DMA 与模块区窗口；DTB 本身；mem_map。不带 DTB 启动（make qemu QEMU_DTB=）时第一行变成 memory map from built-in，RAM 回到 112 MiB，受管理的约 2.8 万页。

4.5.5 接口与防错

对外的接口与 Linux 同名：

```
struct page *alloc_pages(int order);          // 2^order 个物理连续页
void         __free_pages(struct page *pg, int order);
void        *__get_free_pages(int order);     // 返回内核虚拟地址
void         free_pages(void *kva, int order);

void *kalloc(void)   { return __get_free_pages(0); }
void  kfree(void *va);   // kalloc() 的页，或 kmalloc() 的内存
```

kfree() 把页填成 0x01、kalloc() 把页填成 0x05 的做法现在由编译开关 PAGE_POISONING 控制（对应 Linux 的 CONFIG_PAGE_POISONING），slab 对象释放时填 0x01 也归它管。它默认关闭：每分配、释放一页都要把 4 KiB 整个写一遍，fork、缺页、管道这些热路径都为此付出代价，而需要清零的地方（用户页、页表页、kmem_cache_alloc() 返回的对象）本来就各自显式 memset。追查内存错误时用 make PAGE_POISONING=1 重新编译——第 8 章会看到，用户态异常地址 0x0505050505050505 正是一页刚分配、尚未初始化的内存，这类线索只有打开开关才会出现。有了描述符，释放路径还能做空闲链表做不到的检查，出错时直接打印物理地址并 panic：

- 释放 PG_reserved 页（内核镜像、mem_map）；
- 重复释放：页已经带 PG_buddy 或 PG_pcp；
- 把 slab 页（PG_slab）当普通页释放；
- 阶不匹配：用 order 3 分配、却按 order 0 释放，或反之（靠 PG_head 和 order 判断）；
- 释放时 refcount 仍大于 1。

refcount 由 get_page()/put_page()/page_count() 维护：分配时置 1，put_page() 减到 0 时把页还给伙伴系统。这正是写时复制所需的那一份“这一页还有谁在用”，第 4.9.4 节的 COW fork 就建立在它之上。计数的加减用 ldaxr/stlxr 写成的 LL/SC 循环，而不是 GCC 的 __atomic 内建函数：新版 aarch64 工具链默认把它们编译成对 __aarch64_ldadd4_acq_rel 等“外置原子操作”辅助函数的调用，而内核不链接 libgcc，链接时会报未定义符号。

4.5.6 每 CPU 页缓存

四个核共用一把 zone.lock，而 kalloc()/kfree() 是内核里最频繁的调用之一：缺页、写时复制、管道和文件缓冲都要一页一页地拿和还。Linux 的办法是给每个 CPU 一个小的单页缓存（struct per_cpu_pages，简称 pcp）：大多数单页分配只碰本 CPU 的链表，zone.lock 每批才拿一次。xv6 照搬了这个形状：

```
#define PCP_HIGH  64            // 超过这么多页就还一批给伙伴系统
```

```
#define PCP_BATCH 16            // 每次与伙伴系统之间搬运的页数

struct per_cpu_pages {
  struct spinlock lock;
  int count;
  struct list_head list;        // 空闲的 order-0 页，表头是最近释放的（最“热”）
} __attribute__((aligned(64))); // 各占一条 cache line，避免伪共享

static struct per_cpu_pages pcp[NCPU];
```

- **分配**（rmqueue_pcplist()）：本 CPU 的链表为空时，拿一次 zone.lock，用 __rmqueue(0) 搬 PCP_BATCH 页过来；然后从表头取一页。order > 0 的请求照旧直接走伙伴系统。
- **释放**（free_pages_ok() 的 order-0 分支，对应 Linux 的 free_unref_page()）：页挂到本 CPU 表头，标上 PG_pcp；数量到 PCP_HIGH 时，从表尾（最“冷”的一端）还 PCP_BATCH 页给伙伴系统，让它们有机会合并。
- **排空**（drain_all_pages()）：任何分配失败时先把所有 CPU 的链表还给伙伴系统再试一次——别的核链表里的页，以及因此无法合并出的大块，都可能正好满足这次请求。

早期 Linux 靠“只有本 CPU 访问、访问时关中断”保护 pcp，排空别的 CPU 的链表得发 IPI 让它自己来做；5.19 以后每个 pcp 带一把自旋锁，平时只有本 CPU 拿，几乎不会争抢，而排空可以直接跨核进行。xv6 采用后一种：this_pcp_lock() 在关中断的情况下读 cpuid() 并拿本 CPU 的锁，锁的顺序固定为先 pcp、后 zone.lock。

PG_pcp 让 pcp 上的页和伙伴系统里的页区分开：__free_one_page() 只与带 PG_buddy 的伙伴合并，不会把 pcp 链表上的页摘走；对 pcp 上的页再释放一次，同样会被当作重复释放抓住。

空闲页的统计也换成了计数器。原来的 kfreepages() 每次都拿着 zone.lock 把 11 个阶的 nr_free 加一遍；现在 add_to_free_area()/del_from_free_area() 顺手维护 zone.nr_free（对应 Linux 的 NR_FREE_PAGES），kfreepages() 不拿锁，返回它再加上各 CPU 的 count。pcp 上的页也算空闲，所以 /proc/meminfo 的 MemFree 不会因为页暂存在 pcp 上而“丢失”，cowtest 用 MemFree 做的泄漏检查依然精确；另有一行 PerCPUPages 显示 pcp 上的总量（四个核最多 4×64 页，即 1 MiB）。

4.5.7 观察：启动自检与 /proc/buddyinfo

main() 在 printf 可用之后立刻调用 buddy_selftest()：先排空 pcp，记下各阶的空闲块数，分配 order 3、order 0 和 order 10 各一块，检查对齐、计数和引用计数，再全部释放并再次排空 pcp，要求每一阶的空闲块数都恢复原样——这同时验证了切分与合并。随后 slab_selftest() 检查第 4.6 节的 kmalloc。两者通过后各打印一行：

```
buddy: 241333 pages managed, 241333 free, mem_map 9600 KiB at pa 0x000000003b6a0000
slab: 7 kmalloc caches (32..2048 B), self-test ok, 1 page(s) kept by kmalloc-64
```

/proc/buddyinfo 按 Linux 的格式列出 order 0 到 10 各有多少空闲块。下面是 112 MiB 时的一次观察（带 DTB 启动时 order 10 一列是 230 多个）。刚登录时，几乎所有内存都还是 26 个完整的 4 MiB 块；跑完 mmaptest 后，大块被切开，低阶链表上出现了大量碎块，order 10 的块少了一个：

```
$ cat /proc/buddyinfo
Node 0, zone   Normal       1      0      0      0      1      1      1      1      1      1     26
$ mmaptest
...
mmaptest: OK (0 failures)
$ cat /proc/buddyinfo
Node 0, zone   Normal       0     10     18     18     17     17      9      3      0      0     25
```

表 4.2 把这一实现与 Linux 对照。

表 4.2: xv6 与 Linux 的物理页管理

	Linux	xv6
内存探测	e820 / UEFI / DTB	DTB 的 `/memory`；没有时用常量 `PHYSTOP`
早期分配器	memblock（memory / reserved 两张表）	迷你 memblock，各 32 项静态数组
页描述符	`struct page`，约 64 字节	`struct page`，40 字节
`mem_map` 来源	`memblock_alloc()`	`memblock_alloc()`，自顶向下
节点与 zone	多 NUMA 节点，DMA/DMA32/Normal	单节点、单 zone
阶数	`MAX_ORDER`（4 KiB ~ 4 MiB）	同为 11 阶
每 CPU 缓存	`per_cpu_pages`，带锁（5.19 起）	同样形状，high 64 / batch 16
空闲页统计	`NR_FREE_PAGES` 计数器	`zone.nr_free` 加各 pcp 的 `count`
小对象	SLUB，`kmalloc-8` 到 `kmalloc-8k`	简化的 SLUB，`kmalloc-32` 到 `kmalloc-2048`

物理 `0x07000000` 起的 6 MiB 仍保留给 QEMU 时代的 ramdisk、摄像头 DMA 区和模块区，它们登记在 memblock 的保留表里，分配器永远不会把它们发出去。有了 `alloc_pages(order)`，这类“必须物理连续”的缓冲以后可以改从伙伴系统里分配，或者像 Linux 那样写进设备树的保留内存节点，而不必在源码里手工划地盘。

4.6 slab 与 kmalloc()

伙伴系统的最小单位是一页。内核里大量对象只有几十到几百字节：一个 VMA 88 字节，一个 `struct file` 168 字节，伪终端的打开状态只有一个 `int`。为它们各要一页，浪费的是 95% 以上的内存。xv6 原先已经有 `kmem_cache`：每个 cache 从 `kalloc()` 拿页，切成等长的对象，用对象自身的第一个字串成空闲链表。但它有两个缺陷：

- 只有类型专用的 cache，没有按大小分配的 `kmalloc()`。于是 `unicam.c` 为 4 字节的 `struct cam_file`、`pty.c` 为一个 `struct pty` 都直接 `kalloc()` 整页。
- 页一旦进了 cache 就永不归还。所有对象被释放后，空闲链表把不同页里的对象混在一起，cache 既不知道一个对象属于哪一页，也不知道某一页是否已经全空。

4.6.1 元数据放进 struct page

第二个缺陷的根源是空闲链表挂在 cache 上，而不是挂在页上。Linux 的 SLUB 把每个 slab（一页或几页）的元数据放在它的 `struct page` 里：哪个 cache、页内的空闲链表、已分出几个对象。xv6 照做，于是 `struct page` 多了 `slab_cache`、`freelist`、`inuse` 三个字段（第 4.5 节的定义），`flags` 里多了 `PG_slab`，`lru` 在 slab 页上用来挂 cache 的 partial 链表：

```
struct kmem_cache {
  struct spinlock lock;
  const char *name;
  uint object_size, stride, alignment;
  uint objs_per_slab;           // PGSIZE / stride
  struct list_head partial;     // 还有空闲对象的 slab
  uint nr_partial;
  uint pages, total, inuse;     // 统计，供 /proc/slabinfo
  struct kmem_cache *next;      // 所有 cache 串成一条链
};
```

- **分配**：取 partial 链表上第一个 slab，从它的 `freelist` 摘一个对象，`inuse++`；摘空了就把这个 slab 移出 partial（满的 slab 不挂在任何链表上，和 SLUB 一样）。partial 为空时先放开 *cache* 锁再 `alloc_pages(0)`

——页分配器可能排空 pcp，调用者也可能还拿着别的锁——拿到页后在 `struct page` 里建好空闲链表，再挂进 partial。对象依旧在返回前清零。

- **释放**：`virt_to_page(obj)` 找到 slab，核对 `PG_slab`、`slab_cache` 和对象边界，然后把对象推回这个 slab 的 `freelist`。原来是满的就重新挂回 partial。`inuse` 减到 0 且 cache 还有别的 partial slab 时，清掉 `PG_slab`，把页还给伙伴系统；只剩这一个时留着它，避免一对 alloc/free 让同一页在 slab 和伙伴系统之间来回搬（Linux 用 `min_partial` 做同样的事）。

整页都用来放对象，没有页内头部，所以 `kmalloc-2048` 一页正好两个对象。

4.6.2 尺寸类与 kfree()

`kmalloc_init()` 建 7 个通用 cache：`kmalloc-32`、`-64`、……、`-2048`，每个按自身大小对齐。`kmalloc(size)` 向上取到 2 的幂选 cache；超过 2 KiB 的请求像 Linux 的 `kmalloc_large()` 那样直接向伙伴系统要 2^{order} 页。

释放只有一个入口 `kfree()`，它和 Linux 的 `kfree()` 一样靠页描述符判断内存从哪里来：

```
void kfree(void *va) {
  struct page *pg = virt_to_page(va);
  if(pg->flags & PG_slab){            // kmalloc() 的小对象
    kmem_cache_free(pg->slab_cache, va);
    return;
  }
  if(pg->flags & PG_head){            // kmalloc() 的大块
    __free_pages(pg, pg->order);
    return;
  }
  free_pages(va, 0);                  // kalloc() 的一页
}
```

这样原有上百处 `kfree()` 调用一处不用改，`unicam.c`、`pty.c` 把 `kalloc()` 换成 `kmalloc(sizeof(*p))` 即可，释放端照旧 `kfree()`。`kmalloc` 也加进了模块的导出符号表，`.ko` 驱动可以直接用。

4.6.3 观察：/proc/slabinfo

每个 cache 在 `kmem_cache_init()` 时串进一条全局链表，`/proc/slabinfo` 按 Linux slabinfo 2.1 的列序打印它们。登录后不久的样子（节选）：

```
# name            <active_objs> <num_objs> <objsize> <objperslab> <pagesperslab> <num_slabs>
file_cache                2        23        168          23              1           1
vma_cache                 3        42         88          42              1           1
proc_cache               16        20        384          10              1           2
kmalloc-2048              0         0       2048           2              1           0
kmalloc-64                0         0         64          64              1           0
kmalloc-32                0         0         32         128              1           0
```

`num_slabs` 为 0 的行说明 cache 此刻一页都没占：空 slab 已经还给了伙伴系统。启动时的 `slab_selftest()` 分配 300 个 40 字节对象（落在 `kmalloc-64`，占 5 页），检查对齐、清零和页描述符，再全部释放，要求除 cache 自愿保留的一页外，空闲页数精确回到测试前的值。

4.7 调试视图：/proc/meminfo 与 /proc/iomem

物理内存现在由 DTB、memblock、伙伴系统、pcp 和 slab 五层共同管理，出了问题首先要知道“内存都去哪儿了”。两个 procfs 文件按 Linux 的格式回答这个问题。

`/proc/meminfo` 给出各层的数量。MemTotal 是交给伙伴系统的页（managed pages），MemFree 是伙伴系统加上各 CPU pcp 里的空闲页，Slab 是所有 `kmem_cache` 占用的页，Reserved 是 memblock 保留区与 RAM 的交集；DirectMap4k/DirectMap2M 遍历内核页表，统计线性映射里分别有多少字节是 4 KiB 页、多少是 2 MiB 块，可以直接验证第 4.4 节的块映射是否生效：

```
MemTotal:       965332 kB
MemFree:        963916 kB
MemAvailable:   963916 kB
Slab:           40 kB
PerCPUPages:    192 kB
Reserved:       17707 kB
DirectMap4k:    7684 kB
DirectMap2M:    972800 kB
CowShared:      76 pages
...
MemorySource:   DTB
```

`/proc/iomem` 给出物理地址空间的布局：先按 memblock 的 memory 表列出 System RAM，再把 reserved 表里落在其中的区间缩进一层列在下面，并认出各自是什么——第 0 页到内核装载地址之间的 spin-table/armstub、内核代码与数据、启动时分配的内核页表、carve-out 窗口里的三块缓冲、DTB 本身、`mem_map`。RAM 之后到外设窗口之间，带 DTB 时是 VideoCore 的那部分内存（帧缓冲 `0x3c100000` 就在这里），不带 DTB 时则如实标成“未描述”。最后是各外设控制器的 MMIO 子区间：

```
0x0000000000000000-0x000000003bffffff : System RAM
  0x0000000000000000-0x000000000007ffff : Reserved (spin-table, armstub)
  0x0000000000080000-0x00000000000bbfff : Kernel code
  0x00000000000bc000-0x00000000001c316f : Kernel data/bss
  0x00000000003ef000-0x00000000003fffff : Kernel page tables
  0x0000000007000000-0x00000000075fffff : Reserved (carve-out window)
    0x0000000007000000-0x00000000070fffff : QEMU xv6 ramdisk
    0x0000000007200000-0x00000000073fffff : camera DMA buffer (non-cacheable)
    0x0000000007400000-0x00000000074fffff : loadable-module arena (RWX)
  0x0000000008000000-0x0000000008015d95 : Flattened Device Tree
  0x000000003b6a0000-0x000000003bffffff : mem_map (struct page[])
0x000000003c000000-0x000000003effffff : Reserved (VideoCore / GPU memory, ...)
0x000000003f000000-0x000000003fffffff : BCM2837 peripheral MMIO window
  0x000000003f003000-0x000000003f003fff : BCM system timer
  ...
0x0000000040000000-0x0000000040000fff : BCM2836/7 ARM-local interrupt/timer MMIO
```

两者都只读现成的数据结构（memblock 的两张表在启动后保留下来，正像 Linux 保留 `memblock` 供 `/sys/kernel/debug/memblock` 查看），不额外维护任何状态。

4.7.1 逐条跟踪：MM_TRACE

/proc 给出的是快照；要看一次 fork 或一次 sbrk 之后内存到底是怎么一页页分出去的，需要逐事件的跟踪。与 FS_TRACE、VM_TRACE 一样，它是一个编译开关，定义在 include/kernel/mm_trace.h：

```
make MM_TRACE=1 qemu                          # 只跟踪用户进程（默认）
make MM_TRACE=1 MM_TRACE_PID=0 qemu           # 全部，包括内核线程和中断
make MM_TRACE=1 MM_TRACE_PID=7 MM_TRACE_LIMIT=500 qemu
```

关闭时 MMTRACE(...) 展开成 if(0) printf(...)：参数照样做类型检查，代码和格式串则被编译器整个删掉，热路径上没有任何代价。打开后，每一层在事件发生、锁已经放开之后各打印一行，前缀是当前进程的 pid 和名字：

- **page**：alloc_pages/kalloc 的阶、物理地址、来自 pcp 还是伙伴系统；__free_pages/kfree/put_page 的释放；pcp 的批量补充、超过 high 后的回吐和 drain_all_pages。
- **slab**：kmem_cache_alloc/free 的对象地址、所在 slab 及其占用（如 16/25）、新建 slab、空 slab 还给伙伴系统；kmalloc 选中的尺寸类或大块的阶。
- **walk**：walk() 为某个虚拟地址新建的 L2/L3 页表页；每次用户缺页处理完后，该地址经过的 L1/L2/L3 三个描述符和最终物理地址。
- **fault** 与 **cow**：缺页地址、读写、是否来自内核的 copyin/copyout，以及结果——写时复制拷贝或原地复用、brk 堆的零页、VMA 缺页，或 OOM/SIGSEGV/SIGBUS。
- **brk**、**mmap**、**munmap**：系统调用的参数和返回值。

页、slab 和 kmalloc 的行以调用者的返回地址结尾（__builtin_return_address(0)）。内核里没有符号表，主机端的 tools/mm_trace_sym.py 用同一次构建的 kernel/kernel.sym 把它换成“函数名 + 偏移”。下面是 shell 执行 echo hi 时的一段（地址略去高位）：

```
mm[16 sh] slab: proc_cache alloc ...8029780 (slab pa 0x8029000 6/10) <- allocproc+0x40
mm[16 sh] page: alloc order 0 pa 0x3b638000 (pcp) <- alloc_kstack+0x18
mm[16 sh] page: alloc order 0 pa 0x3b636000 (pcp) <- walk+0xd8
mm[16 sh] walk: va 0x0: new L2 table pa 0x3b636000 in L1[0]
mm[16 sh] slab: vma_cache alloc ...80310c0 (slab pa 0x8031000 3/42) <- dup_mmap+0x5c
mm[16 sh] page: alloc order 0 pa 0x3b632000 (pcp) <- cow_break+0x134
mm[16 sh] cow: va 0xa000 copied pa 0x3b641000 -> new pa 0x3b632000
mm[17 sh] page: put_page -> free order 0 pa 0x3b641000 <- cow_break+0x1ac
mm[17 sh] cow: va 0xa000 copied pa 0x3b641000 -> new pa 0x3b61f000
mm[17 sh] brk: sbrk(131072) 0xb000 -> 0x2b000 (pages are faulted in lazily)
mm[17 sh] page: alloc order 0 pa 0x3b61e000 (pcp) <- handle_user_mm_fault+0x180
mm[17 sh] fault: va 0xb008 write -> brk heap, mapped
mm[17 sh] walk: va 0xb008  L1[0]=0x3b636003 L2[0]=0x3b635003 L3[11]=0x6000003b61e74b
                -> pa 0x3b61e008
mm[17 sh] page: alloc order 0 pa 0x3b619000 (pcp) <- e2_file_read_at+0xb0
mm[17 sh] page: free  order 0 pa 0x3b619000 <- e2_file_read_at+0x128
```

一次 fork 的全过程在这里一目了然：allocproc 从 slab 拿 struct proc，从 pcp 拿内核栈，walk 为子进程建页表，dup_mmap 复制 VMA；随后父子进程几乎同时写同一页（0xa000），两边检查时引用计数都还是 2，于是各自复制，后放手的那次 put_page 把旧页的引用降到 0、交还分配器；子进程的 sbrk 只挪动 brk，真正的页在第一次写时才由缺页处理分配。L3 描述符 0x6000003b61e74b 也可以对照第 4.8.2 节逐位读出：UXN/PXN 置位（数据不可执行）、AF、Inner Shareable、EL0 可读写、Normal 内存。

跟踪对时序影响很大——串口每行要花几毫秒——所以它适合在单个命令上打开，用 MM_TRACE_PID 和 MM_TRACE_LIMIT 收窄范围；跑 usertests 这类压力测试时应保持关闭。

4.8 最终内核页表

4.8.1 kvmmake()

```
pagetable_t kvmmake(void) {
  pagetable_t kpgtbl = (pagetable_t) kalloc();
  memset(kpgtbl, 0, PGSIZE);
  // BCM2837 外设窗口 16 MiB，以及 ARM-local 外设一页：设备内存，不可执行
  kvmmap(kpgtbl, PERIPHERAL_BASE, PERIPHERAL_BASE_PA, 0x01000000, PTE_DEVICE | PTE_XN);
  kvmmap(kpgtbl, LOCAL_BASE, LOCAL_BASE_PA, PGSIZE, PTE_DEVICE | PTE_XN);
  kvmmap(kpgtbl, RAMDISK, RAMDISK_PA, RAMDISK_SIZE, PTE_NORMAL | PTE_XN);
  // 摄像头 DMA：不可缓存的普通内存
  kvmmap(kpgtbl, CAMDMA, CAMDMA_PA, CAMDMA_SIZE, PTE_NORMAL_NC | PTE_XN);
  // 可装载模块区：.ko 的代码拷进来后要执行，所以这里不加 PTE_XN
  kvmmap(kpgtbl, MODAREA, MODAREA_PA, MODAREA_SIZE, PTE_NORMAL);
  // 物理第 0 页：spin-table 槽 0xe0..0xf0，唤醒副核时要写
  kvmmap(kpgtbl, KERNBASE, 0, PGSIZE, PTE_NORMAL | PTE_XN);
  // 内核代码：只读、可执行
  kvmmap(kpgtbl, KERNLINK, V2P(KERNLINK), (uint64)etext-KERNLINK, PTE_NORMAL | PTE_RO);
  // 内核数据和启动窗口其余部分：4 KiB 页，可读写、不可执行
  kvmmap(kpgtbl, (uint64)etext, V2P(etext), (uint64)P2V(EARLYTOP)-(uint64)etext,
         PTE_NORMAL | PTE_XN);
  // memblock 知道的其余 RAM：尽量用 2 MiB 块（Linux 的 map_mem()）
  for(int i = 0; i < memblock.memory.cnt; i++){
    uint64 start = max(memblock.memory.regions[i].base, EARLYTOP);
    uint64 stop  = memblock.memory.regions[i].base + memblock.memory.regions[i].size;
    // 跳过 [CARVEOUT_START, CARVEOUT_END)，它已按自己的属性映射
    kvmmap_linear(kpgtbl, start, stop, PTE_NORMAL | PTE_XN);
  }
  return kpgtbl;
}
```

代码段只读可执行，数据段可写不可执行（W^X），这样即使内核出现缓冲区溢出，也很难把数据当指令执行。唯一的例外是模块区 MODAREA（物理 0x07400000 起）：模块装载器把 .ko 的代码拷进来再执行，它只能可写又可执行（第 9 章）。

4.8.2 walk() 与 mappages()

所有映射最终都落到这两个函数上。walk() 从 L1 走到 L3，遇到缺失的中间页表就（按需）分配一页：

```
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) {
  if(va >= MAXVA) panic("walk");
  for(int level = 1; level < 3; level++) {
    pte_t *pte = &pagetable[PX(level, va)];
    if((*pte & PTE_VALID) && (*pte & PTE_TABLE)) {
      pagetable = (pagetable_t)P2V(PTE2PA(*pte));
    } else {
      if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
        return 0;
```

```
      memset(pagetable, 0, PGSIZE);
      *pte = PA2PTE(V2P(pagetable)) | PTE_TABLE | PTE_VALID;
    }
  }
  return &pagetable[PX(3, va)];
}

int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, uint64 perm) {
  uint64 a = PGROUNDDOWN(va), last = PGROUNDDOWN(va + size - 1);
  for(;;){
    pte_t *pte = walk(pagetable, a, 1);
    if(pte == 0) return -1;
    if(*pte & PTE_AF) panic("mappages: remap");
    *pte = PA2PTE(pa) | perm | PTE_AF | PTE_V;
    if(a == last) break;
    a += PGSIZE; pa += PGSIZE;
  }
  return 0;
}
```

注意页表项里存的是**物理地址**（硬件遍历页表时用物理地址），而内核软件读写页表时要先 `P2V()` 转成虚拟地址。页表页本身也位于被直接映射的 RAM 中，所以内核总能通过 `KERNBASE + PA` 访问任何一张页表——这不是递归页表技巧，只是直接映射的自然结果。

4.8.3 页表的规模

用户命令 `vmmap` 会调用 `kvmdump()` 打印内核页表，每个 2 MiB 窗口汇总一行，再逐个打印各进程的用户页表。表 4.3 是一次快照的统计（当时还保留着 ramdisk 映射）。

表 4.3: 最终内核页表的组成（一次运行时快照）

L1 项	虚拟地址	L2 有效项	内容
`L1[0]`	`KERNBASE`+0 ~ 1 GiB	65	内核和 RAM（56 项）、ramdisk（1 项）、BCM2837 MMIO（8 项）
`L1[1]`	`KERNBASE`+1 ~ 2 GiB	1	ARM-local 外设 `0x40000000`
`L1[255]`	接近 `MAXVA`	1	进程内核栈及其间的保护页
合计：L1 一张、L2 三张、L3 67 张，共 71 页 = 284 KiB；有效 L3 叶子 32 961 个			

表中的快照来自全部用 4 KiB 页的旧版本：112 MiB 就要 56 张 L3 表；照此映射 960 MiB 需要约 480 张、近 2 MiB，而且必须在 `EARLYTOP` 以下分配，启动页表根本放不下。所以当前版本把 4 MiB 以上的 RAM 交给 `kvmmap_linear()`：起止地址都 2 MiB 对齐的部分由 `kvmmap_block()` 直接在 L2 写一个块描述符（`PTE_VALID` 置位、`PTE_TABLE` 清零，输出地址的低 21 位为 0），只有首尾不对齐的零头才落到 4 KiB 页。960 MiB 的 RAM 只占 L2 的约 475 项，启动时整张内核页表一共 17 页（见上面 memblock 的 reserved `[1]`）。

细粒度属性仍然保得住：代码段只读、数据段不可执行都发生在前 4 MiB，那里依旧是 4 KiB 页；carve-out 窗口里的 `CAMDMA` 是不可缓存的、`MODAREA` 是可执行的，线性映射就绕开整个窗口。代价是 `walk()` 现在可能在 L2 遇到块描述符：查询（`alloc=` 0）时返回 0，要求分配时则 `panic("walk: block")`——内核不会在线性映射里再去改单页的属性。

4.9 用户地址空间

4.9.1 内核空间与用户空间

每个进程有自己的 L1 页表，进程切换时写入 TTBR0_EL1：

```
void switchuvm(struct proc *p) {
  w_ttbr0_el1(V2P(p->vm->pagetable));
  flush_tlb();
}
```

4.9.2 exec() 之后的布局

exec() 建立的用户地址空间从虚拟地址 0 开始（图 4.5）：先是 ELF 的代码和数据，然后是一页不可访问的保护页和 4 页（16 KiB）用户栈，栈之上是 sbrk() 扩展的堆。用户页面的属性是 PTE_NORMAL | PTE_U，代码页可执行。动态链接的程序在主镜像之后还有一页多的 /lib/ld.so；mmap() 的区域则远在其上：[MMAP_MIN, MMAP_TOP)，从 64 GiB（0x10_0000_0000）到 256 GiB−256 MiB，自顶向下分配，只登记 VMA、按需调页（第 4.10 节）。

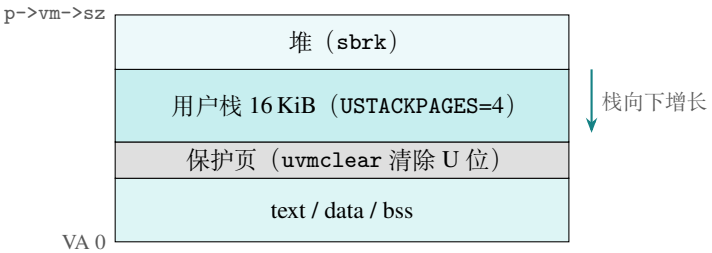


图 4.5: exec() 之后的用户地址空间

进程的内核栈不在用户页表里，而在内核页表高端的 KSTACK(slot) 处，每个栈一页，相邻栈之间隔一页不映射的保护页。内核栈溢出会立刻触发翻译错误，而不是悄悄覆盖相邻进程的栈。第 5 章会介绍内核栈的动态分配。

4.9.3 第一个用户进程的页表

所有用户地址空间都由同一组函数建立。uvmcreate() 分配一页并清零，作为空的 L1 页表——全 0 意味着全部表项无效，这一点很重要：任何从 kalloc() 拿来做页表的页都必须先清零，否则 0x05 填充的垃圾会被 MMU 当成有效描述符。第一个进程没有 ELF 文件可读，内核把一小段编进内核镜像的 initcode 直接拷进一页：

```
void uvminit(pagetable_t pagetable, uchar *src, uint sz) {
  if(sz >= PGSIZE) panic("inituvm: more than a page");
  char *mem = kalloc();
  memset(mem, 0, PGSIZE);
  mappages(pagetable, 0, PGSIZE, V2P(mem), PTE_NORMAL|PTE_U);
  memmove(mem, src, sz);              // 通过内核直接映射写入
  uvmsync_icache(pagetable, sz);      // 写入的是指令，要同步 I-cache
}
```

userinit() 再把 trapframe 的 elr 设为 0、sp 设为 PGSIZE：进程第一次“返回”用户态时，从虚拟地址 0 取第一条指令，栈从这一页的顶部向下长。同一个物理页此时有两个虚拟地址：内核通过 KERNBASE+PA 写它，用户通过 VA 0 执行它。拷贝时用的是前者，所以内核根本不需要先切换到这个进程的页表。

4.9.4 fork：写时复制

fork() 要给子进程一份与父进程内容相同、但互不影响的内存。最直接的做法是对 [0, sz) 的每一页 kalloc() 一页、拷贝、再映射进子进程——早期的 uvmcopy() 就是这样。代价是 fork 一个 8 MiB 的进程就要复制 8 MiB，而 shell 的子进程往往紧接着 exec()，复制的内容立刻被丢弃。vfork 能解决 fork+exec 的场合（第 5.5 节），但它要求子进程在 exec 之前什么都不改。

有了第 4.5 节的 struct page 和它的 refcount，当前源码树改用写时复制（copy-on-write，COW）：fork 时**只共享、不复制**，谁先写谁复制。

fork 时：共享并写保护。 uvmcopy() 把父进程的每个私有页以同一物理地址映射进子进程，并给这一页加一份引用；原来用户可写的页，在父子两边都改成“只读 + PTE_COW”：

```
if(*pte & PTE_SHARED){                    // libc.so 代码：归缓存所有，照旧共享
  mappages(new, i, PGSIZE, pa, flags | PTE_SHARED);
  continue;
}
if((*pte & PTE_U) && (*pte & PTE_RO) == 0)
  *pte |= PTE_RO | PTE_COW;               // 父进程：可写 -> COW
if(*pte & PTE_COW)
  flags |= PTE_RO | PTE_COW;
mappages(new, i, PGSIZE, pa, flags);      // 子进程：同一物理页
get_page(virt_to_page(P2V(pa)));          // refcount++
```

PTE_COW 是描述符里留给软件的第 57 位（MMU 忽略 [58:55]，第 55、56 位已被 PTE_SHARED、PTE_SPECIAL 占用），意思是“只读只是因为共享，写了就复制”。本来就只读的私有页（代码段、只读数据、栈的保护页）不加 PTE_COW，只增加引用、两边共享，写它们仍是致命错误。dup_mmap() 对 MAP_PRIVATE 的 VMA 页做同样的处理，共享映射和设备帧则照旧。复制不再写入新字节，所以原来跟在 uvmcopy() 后面的 uvmsync_icache() 也删掉了。

整个过程在父进程的 vm->lock 下进行；结束后一次 flush_tlb()（TLBI VMALLE1IS）广播作废，父进程其他线程在别的 CPU 上缓存的“可写”翻译随之失效。

写的时候：cow_break()。 写一个 COW 页会产生权限错误（ESR_EL1 的 EC = 0x24、WnR = 1、DFSC = 0b0011xx）。usertrap() 照常把它交给 handle_user_mm_fault()，后者在按地址区分 brk 堆和 VMA 之前，先看这是不是一个 COW 页——代码段、数据段和栈不属于任何 VMA，不先判断就会被当成越界：

```
pa = PTE2PA(*pte);
pg = virt_to_page(P2V(pa));
if(page_count(pg) == 1){                  // 别人都已复制或退出：就地收归己有
  *pte &= ~(PTE_RO | PTE_COW);
  flush_tlb();
  return 1;
}
mem = kalloc();                           // 复制一份
memmove(mem, P2V(pa), PGSIZE);
if((*pte & PTE_UXN) == 0)                 // 可执行页还要同步 I-cache
```

```
  sync_icache(mem);
*pte = PA2PTE(V2P(mem)) | (*pte & ~(PA2PTE(~0ULL) | PTE_RO | PTE_COW));
flush_tlb();                              // 先让所有 CPU 忘掉旧翻译
put_page(pg);                             // 再放弃对共享页的那一份引用
```

顺序很重要：必须先广播作废 TLB 再 put_page()。另一方可能恰好是最后一个使用者，引用一减到 0，伙伴系统就会把这页发给别人；如果此时某个 CPU 上还留着旧翻译，就会写进别人的内存。

返回用户态后，那条 str 重新执行，这次写进自己的页。cow_break() 不会睡眠（kalloc() 只用自旋锁），这一点后面会用到。

释放：kfree 换成 put_page. 有了共享，“释放一页”的含义变成“放弃我这一份”。uvmunmap()（进程退出、exec 换镜像）、growproc() 缩小堆、zap_range() 撤销私有 VMA，这几处原来的 kfree() 都改成 put_page(virt_to_page(...))，最后一个使用者离开时页才回到伙伴系统。第 4.5 节的检查在这里派上了用场：如果哪条路径漏改，仍对共享页调用 kfree()，会立刻因“still has 2 references”而 panic，而不是悄悄地把别的进程还在用的页放回空闲链表。

copyout() 也要遵守. 内核写用户内存走的是自己的直接映射，用户 PTE 的 AP 位对它不起作用。如果不处理，子进程 read(fd, buf, n) 时内核会把数据直接写进仍与父进程共享的物理页，父进程的内存就被改了。copyout() 原来就会对只读页调用 uva2ka_writable() 拒绝写入，现在在拒绝之前先试 cow_break()。而且对当前地址空间，检查、破除 COW 和这一页的 memmove 都在 vm->lock 下完成：fork 也持这把锁，于是另一个线程的 fork() 不可能恰好在检查和写入之间把这一页重新变成共享页。因为 cow_break() 不睡眠，持有自旋锁的调用者（比如在锁内回写退出状态的 wait()）也能正确写入 COW 页；只有需要按需调页的情形才在锁外走 uva2ka_fault()，然后在锁内重试这一页。

统计与测试. /proc/meminfo 多了三行：CowShared（fork 时共享出去的页数）、CowCopied（写时复制的次数）、CowReused（最后一个使用者就地收回的次数）。新的用户程序 cowtest 覆盖五种情形：

- **big**：先 sbrk 占用三分之二的空闲内存，再 fork——不共享的话需要两倍内存，必然失败；
- **isolate**：父子写同一批页，各自只看到自己的修改；
- **three**：两个子进程先后复制并退出，父进程最后写时 refcount 已是 1，走就地收回的路径；
- **copyout**：子进程 read() 一个管道到 COW 页，父进程的页不变；
- **mmap**：MAP_PRIVATE|MAP_ANONYMOUS 页同样写时复制。

全部测试在一个子进程里运行，子进程退出后检查空闲页数回到原值：

```
$ cowtest
PASS big: sbrk two thirds of free memory
PASS big: fork succeeds (would need 2x memory without COW)
PASS big: child reads parent's data
PASS big: pages were shared
...
PASS mmap: parent's MAP_PRIVATE pages unchanged
MemFree before 110524 kB, after 110524 kB
cowtest: OK (0 failures)
```

行为上的变化. COW 把“内存不够”从 fork 时推迟到了写的时候：fork 几乎总能成功，真正复制时如果没有空闲页，cow_break() 返回 OOM，进程被杀（out of memory at va=...）——这与 Linux 的 OOM 行为一致。另一个可见的变化在 usertests 的 forktest：本内核没有固定大小的进程表，原来 fork 不到 1000 次就会因内存耗

尽而失败，测试据此判定“fork 能正确报告失败”。有了 COW，每个子进程只花几页（内核栈、trapframe、页表页），fork 一直能成功到三千多个子进程，测试反而报错；更糟的是，最终耗尽内存的往往是父进程自己写 COW 页时的那次复制，于是父进程被 OOM 杀掉，几千个 `struct proc`、vmspace 所在的 slab 页也被 `kmem_cache` 永久留作缓存（表 4.2 中 slab 一行的“无收缩”）。所以 `param.h` 加了任务数上限 `NPROC_MAX`，取 Linux `threads-max` 的默认算法“内存页数除以 32”，112 MiB 对应 896；`newprocslot()` 在已经建立 896 个进程描述符时直接返回失败，`fork()` 返回 −1，相当于 Linux 的 `EAGAIN`。描述符在进程回收后会被复用，所以这个上限约束的是同时存在的任务数。

4.9.5 TLB 与地址空间切换

如果每次访存都要走三级页表，一次 `ldr` 就会变成四次内存访问。MMU 因此把最近用过的翻译结果缓存在 TLB（Translation Lookaside Buffer）里，命中时不再遍历页表。代价是：一旦软件改了页表，TLB 里可能还留着旧的翻译，必须由软件显式作废。

进程切换正是最大的一次“改页表”——`TTBR0` 换成另一棵树，之前缓存的低地址翻译全都过期了：

```
void switchuvm(struct proc *p) {
  if(p->vm == 0 || p->vm->pagetable == 0) panic("switchuvm: no pagetable");
  w_ttbr0_el1(V2P(p->vm->pagetable));
  flush_tlb();                  // dsb ishst; tlbi vmalle1is; dsb ish; isb
}
```

`flush_tlb()` 作废全部 EL1 翻译，连内核的也一起丢掉了。ARMv8 提供了 ASID（地址空间标识）：把一个 8 或 16 位的进程编号写进 `TTBR0` 的高位，TLB 表项会带上这个标签，切换时就不必全部作废。xv6-aarch64 没有使用 ASID，每次切换都整体作废，换来的是实现简单、不必管理 ASID 的回收；对于这个规模的系统，多出来的 TLB 缺失是可以接受的。注意内核页表（`TTBR1`）在整个运行期间只装一次（`kvminithart()`），这也是把两个地址空间分开的好处。

4.9.6 缺页与按需分配

如果用户进程访问了一个没有映射的地址，页表遍历在某一级遇到无效项，CPU 产生同步异常进入 `usertrap()`。此时 `ESR_EL1` 的 EC 字段是 `0x24`（来自 EL0 的数据中止）或 `0x20`（来自 EL0 的指令中止）；ISS 的低 6 位 DFSC 说明原因，`0b0001xx` 是第 xx 级的翻译错误，`0b0011xx` 是权限错误；bit 6（WnR）表示是写；出错的虚拟地址在 `FAR_EL1` 里。

很多内核利用这个异常实现*按需分配*（demand paging）：进程第一次碰到某页时触发翻译错误，内核检查 `FAR_EL1` 落在合法区间里，就当场分配、映射，然后 `eret` 重新执行那条出错的指令。进程看到的只是一条稍微慢一点的 `ldr`。

xv6-aarch64 对两段地址采用不同的策略：

- `[0, sz)`：`exec()` 和 `growproc()` 调用 `uvmalloc()` 立即分配并清零所有页，合法访问永远不会缺页；
- mmap 区：`mmap()` 只登记 VMA，第一次访问时由 `user_page_fault()` → `handle_mm_fault()` 按 VMA 分配或读入（第 4.10 节）。共享文件页的第一次写也走这里，借权限错误标记脏页。

其余异常——访问 `[0, sz)` 之外又不在任何 VMA 里的地址、写只读页、执行不可执行页——一律打印 `esr`、`elr`、`far`，调用 `uvmdump()` 打印这个进程的整张页表（mmap 区里的错误只打印一行 SIGSEGV/SIGBUS），然后杀掉进程：

```
} else if((ec == 0x24 || ec == 0x20) && user_page_fault(p, esr)){
  // mmap 区的缺页已处理，或已打印 SIGSEGV/SIGBUS 并设 killed
} else {
  printf("usertrap(): unexpected esr=%p ec=%p pid=%d\n", esr, ec, p->pid);
```

```
  printf("            elr=%p far=%p\n", r_elr_el1(), r_far_el1());
  uvmdump(p->vm->pagetable, p->pid, p->name, "usertrap");
  p->killed = 1;
}
```

这份输出正是 4.12 节分析真机故障时用的线索。

4.10 mmap()：VMA、按需调页与设备映射

前面的用户地址空间 [0, sz) 由 exec() 和 sbrk() 一次性建好，页表里有什么，进程就有什么。mmap() 换了一种思路，仿照 Linux：先用一个 vm_area_struct（VMA）描述一段地址应该装着什么——哪个文件的哪一段、是否与别的进程共享、是否是设备的物理帧——页表只记录已经碰过的页。第一次访问某页时 MMU 报缺页，内核按 VMA 的描述把这一页“变出来”，再让那条指令重新执行。图 4.6 是数据结构，图 4.7 是三条主要路径。

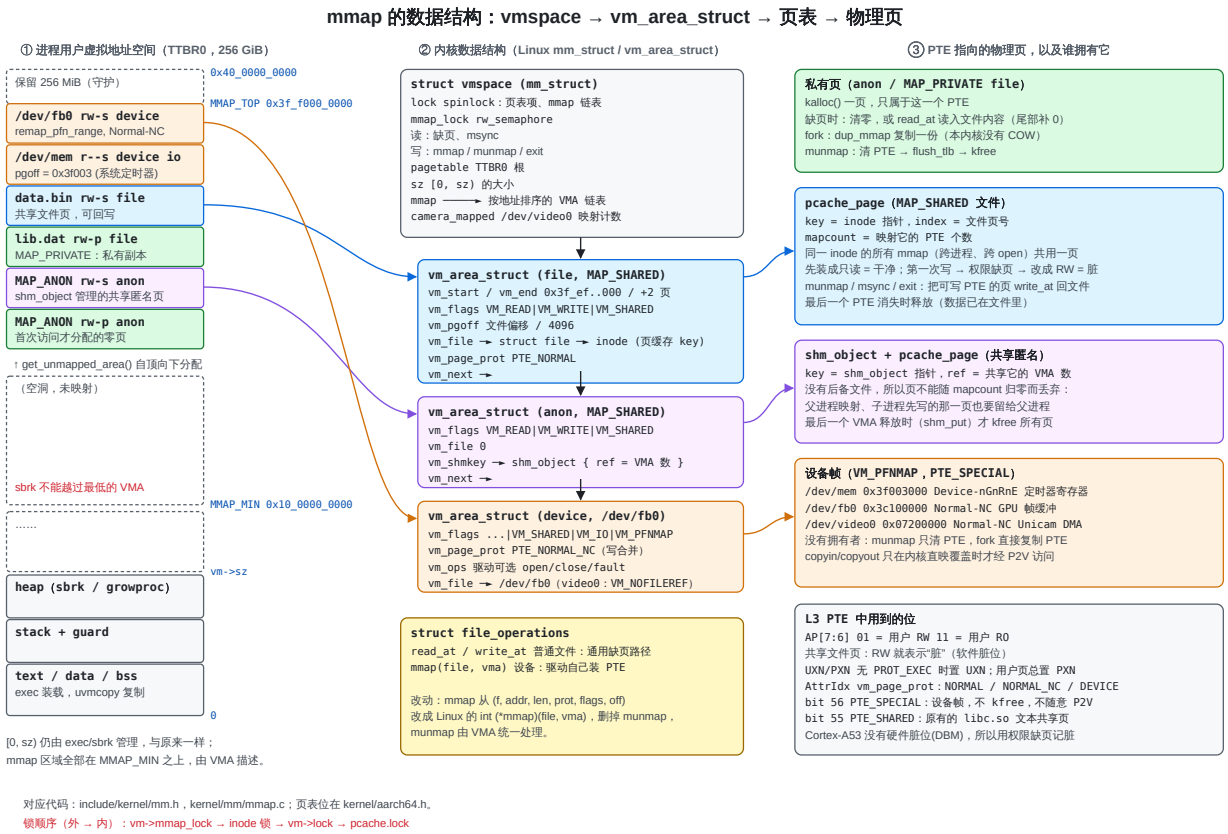


图 4.6: mmap 的数据结构：地址空间布局、vmspace/vm_area_struct、四类物理页的归属与 PTE 位

4.10.1 vm_area_struct 与地址布局

struct vmspace（相当于 Linux 的 mm_struct）多了两个成员：按地址排序的 VMA 链表 mmap，以及一把可睡眠的读写信号量 mmap_lock。缺页处理拿读锁，mmap/munmap 拿写锁；改页表本身仍由自旋锁 vm->lock 保护。VMA 定义在 include/kernel/mm.h，字段是 Linux 的一个子集，vm_flags 的位值也照抄 Linux：

```
struct vm_area_struct {
  uint64 vm_start, vm_end;    // [start, end)，页对齐
```

```
  uint64 vm_flags;              // VM_READ/WRITE/EXEC/SHARED/MAYWRITE/PFNMAP/IO...
  uint64 vm_page_prot;          // 内存类型：PTE_NORMAL / PTE_NORMAL_NC / PTE_DEVICE
  uint64 vm_pgoff;              // 文件偏移（或物理帧号），以页为单位
  struct file *vm_file;         // 后备文件，匿名映射为 0
  const struct vm_operations_struct *vm_ops;   // open/close/fault
  void *vm_private_data;
  uint64 vm_shmkey;             // 共享匿名内存的页缓存键
  struct vmspace *vm_mm;
  struct vm_area_struct *vm_next;
};
```

mmap 区在 [MMAP_MIN，MMAP_TOP)，即 64 GiB 到 256 GiB−256 MiB，不带 MAP_FIXED 时 get_unmapped_area() 自顶向下找空洞；growproc() 不允许堆长进最低的 VMA。do_mmap() 检查参数、分配 VMA，然后分两种情况：普通文件和匿名映射到此为止，一页都不分配；字符设备则调用驱动的 f_op->mmap(f，vma)，签名与 Linux 相同。

4.10.2 按需调页：handle_mm_fault()

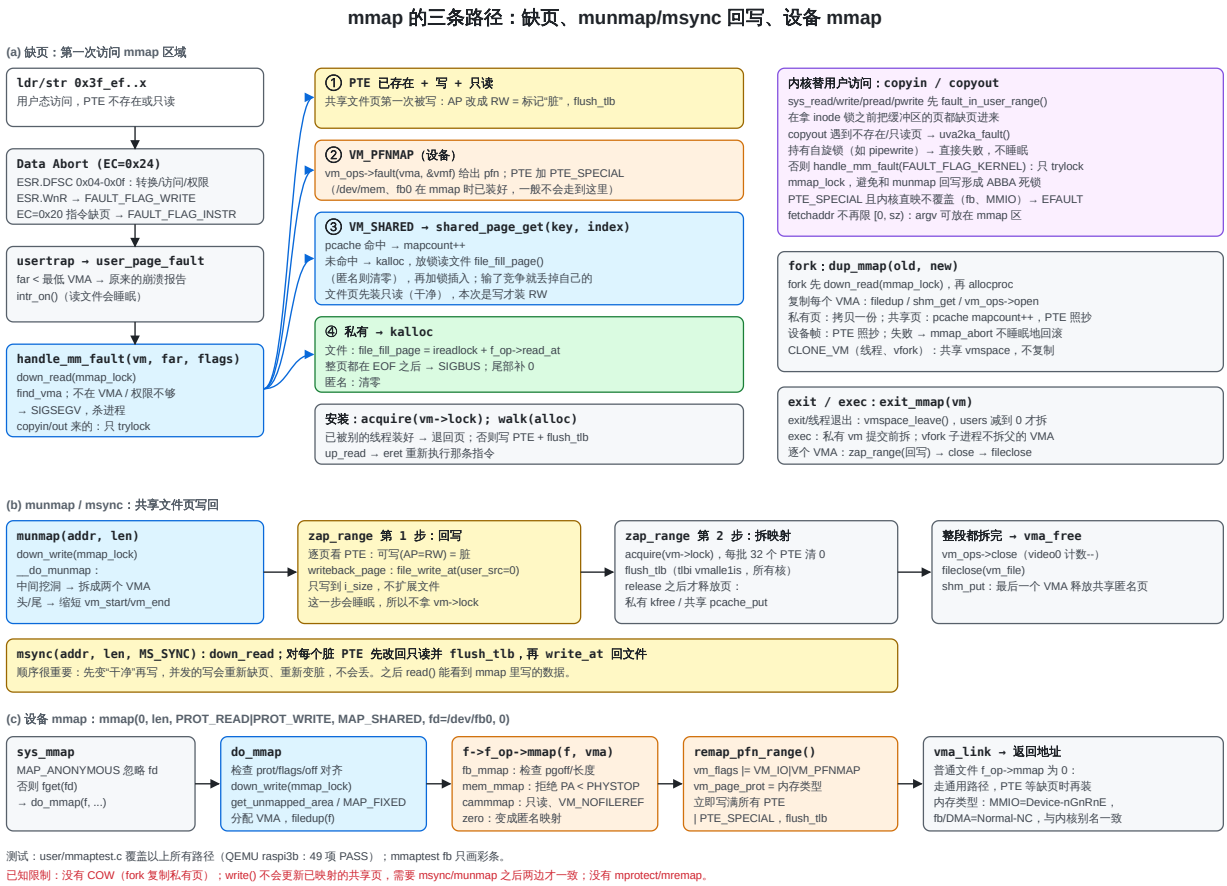


图 4.7: mmap 的三条路径：(a) 缺页，(b) munmap/msync 回写，(c) 设备 mmap

usertrap() 遇到 EC = 0x24（数据中止）或 0x20（指令中止）时先交给 user_page_fault()。地址低于最低 VMA 的缺页仍走原来的详细报告；否则打开中断（读文件可能睡眠）调用 handle_mm_fault()：

```
if(fsc < 0x04 || fsc > 0x0f || far < mmap_lowest(p->vm))
```

```
  return 0;                                    // 不是 mmap 区：保留原来的崩溃报告
if(ec == 0x20)                          flags |= FAULT_FLAG_INSTR;
else if((iss & (1<<6)) && !(iss & (1<<8))) flags |= FAULT_FLAG_WRITE;  // WnR
intr_on();
r = handle_mm_fault(p->vm, far, flags);       // 失败：打印 SIGSEGV/SIGBUS 并杀进程
```

handle_mm_fault() 在 mmap_lock 读锁下找到 VMA，检查权限（不在任何 VMA 里、写只读区、执行不可执行区都是 SIGSEGV），然后按四种情况产生物理页：

1. **PTE 已存在**：只可能是“写一个干净的共享文件页”，把 AP 改成可写即可（见下文的脏页跟踪）；
2. VM_PFNMAP（设备）：由 vm_ops->fault 给出物理帧号；
3. VM_SHARED：到共享页缓存里找或建这一页；
4. **私有**：kalloc() 一页，文件映射用 f_op->read_at 读入（整页都在文件末尾之后则 SIGBUS，尾部补 0），匿名映射清零。

最后在 vm->lock 下 walk() 并写入 PTE。如果另一个线程抢先装好了同一页，就把自己的页退回；然后返回，eret 重新执行那条 ldr/str。

4.10.3 共享页：pcache_page 与 shm_object

MAP_SHARED 要求所有映射同一文件同一页的 PTE 指向同一个物理页，内核因此需要一个以“对象 + 页号”为键的表：

```
struct pcache_page {
  struct pcache_page *next;   // 64 桶哈希链
  uint64 key;                 // inode* 或 shm_object*
  uint64 index;               // 页号
  uint64 pa;
  int mapcount;               // 有多少个 PTE 指向它
};
```

文件页的键是 inode 指针，所以两个进程各自 open() 同一文件再 mmap，也会共享同一页；mapcount 降到 0 时页被释放。共享匿名内存（MAP_SHARED|MAP_ANONYMOUS 和 /dev/zero）没有后备文件，键是一个按 VMA 计数的 shm_object，页要活到最后一个 VMA 消失——否则 fork 出的子进程先写、随后退出，那一页就跟着丢了（mmaptest 抓到过这个错误）。

脏页跟踪. Cortex-A53 是 ARMv8.0，没有硬件脏位（DBM 是 ARMv8.1 的特性）。所以共享文件页第一次总是装成只读；第一次写触发权限错误，handle_mm_fault() 的情况 1 把 AP 改成可写并刷 TLB。于是“共享文件 VMA 里可写的 PTE”就等于“脏页”。

回写. munmap()、进程退出和 exec() 都经过 zap_range()：先对可写 PTE 指向的页调用 file_write_at() 写回文件（只写到 i_size，不扩展文件），这一步会睡眠，所以不持 vm->lock；然后在 vm->lock 下每批 32 个 PTE 清零、flush_tlb()，最后才释放物理页。msync(MS_SYNC) 顺序相反：先把脏 PTE 改回只读并刷 TLB，再写回——写回期间的并发写会重新缺页、重新变脏，不会丢。

4.10.4 设备 mmap：f_op->mmap 与 remap_pfn_range()

设备没有页缓存，驱动知道的是物理地址。驱动的 mmap 回调检查偏移和长度后调用 remap_pfn_range() 立即装好整段 PTE，例如帧缓冲：

```
static int fb_mmap(struct file *f, struct vm_area_struct *vma) {
```

```
  uint64 size = vma->vm_end - vma->vm_start, len = PGROUNDUP(fb.size);
  if(!fb.ready || vma->vm_pgoff > len / PGSIZE ||
     size > len - vma->vm_pgoff * PGSIZE || (vma->vm_flags & VM_EXEC))
    return -1;
  return remap_pfn_range(vma, vma->vm_start,
                         (fb.base >> PGSHIFT) + vma->vm_pgoff, size, PTE_NORMAL_NC);
}
```

remap_pfn_range() 给 VMA 加上 VM_IO|VM_PFNMAP，并在每个 PTE 上打软件位 PTE_SPECIAL（bit 56）：这一页不归 kalloc 管，munmap 只清 PTE、不释放，fork 照抄 PTE。内存类型由驱动决定：MMIO 必须是 Device-nGnRnE，帧缓冲和 DMA 缓冲用 Normal-NC（写合并，并与内核对同一物理页的别名属性一致）。

表 4.4: 支持 mmap 的设备

设备	主/次	行为
/dev/mem	8/1	偏移即物理地址；拒绝落在 memblock 内存里的区间（carve-out 窗口除外，相当于 STRICT_DEVMEM）；MMIO 用 Device-nGnRnE，其余 Normal-NC
/dev/zero	8/5	变成匿名映射；MAP_SHARED 即共享匿名内存
/dev/fb0	9/0	bcm2708_fb.c：首次 open 用一条多 tag 的 mailbox 消息分配 640×480×32 帧缓冲；支持 FBIOGET_VSCREENINFO/FSCREENINFO
/dev/video0	5	Unicam DMA 缓冲；地址由内核选；VM_NOFILEREF 让映射不持有 file，最后一个 fd 关闭即停流；vm_ops 的 open/close 维护 camera_mapped 计数

4.10.5 内核替用户访问 mmap 区

copyin/copyout 现在可能碰到还没缺页进来的地址。规则有三条：

- read/write/pread/pwrite 在拿 inode 锁之前先 fault_in_user_range()，把整块用户缓冲区缺页进来；
- copyout 仍遇到缺页时调用 uva2ka_fault()：若当前持有自旋锁（例如 pipewrite、consoleread 持锁拷贝）直接失败，不睡眠；否则以 FAULT_FLAG_KERNEL 调 handle_mm_fault()，只 down_read_trylock(mmap_lock)，避免与正在回写的 munmap 形成 ABBA 死锁；
- PTE_SPECIAL 且不在内核直接映射范围内的页（帧缓冲、MMIO）一律返回 EFAULT，内核不去碰没有映射的物理地址。

fetchaddr() 也不再要求地址在 [0, sz)，只要在 256 GiB 用户空间内，这样 argv 可以放在 mmap 区里。

4.10.6 fork、vfork、exit 与 exec

fork() 在 allocproc() 之前拿 mmap_lock 读锁，复制完 [0, sz) 后调用 dup_mmap()：每个 VMA 复制一份（filedup、shm_get、vm_ops->open），私有页按写时复制共享（第 4.9.4 节），共享页 mapcount++、PTE 照抄，设备帧照抄 PTE；中途失败用不睡眠的 mmap_abort() 回滚。带 CLONE_VM 的线程和 vfork 子进程直接共享 vmspace，不复制。

共享 vmspace 带来一个问题：vfork 子进程退出时不能把父进程的映射拆掉。所以 vmspace 有两个计数（第 5.5 节）：refcount 记录还有谁持有页表，users 记录还有几个活着的任务在用。exit()、线程退出和 vfork 子进程的 exec() 都调用 vmspace_leave()，只有 users 减到 0 时才 exit_mmap()。exec() 对私有 vmspace 在提交新镜像前 exit_mmap()；vfork 子进程则换到 vmspace_wrap() 新建的私有 vmspace，旧的 VMA 原样留给父进程。

4.10.7 测试与限制

user/mmaptest.c 在 QEMU raspi3b 上 49 项全部通过，连续运行 /proc/meminfo 的 MemFree 不变。它覆盖：匿名零页、munmap 中间一页拆分 VMA、访问空洞或只读页被杀、MAP_FIXED、fork 后的共享与私有语义、子

进程先写的共享页、vfork 子进程写父进程的映射并退出、MAP_PRIVATE 文件（关闭 fd 后仍可用、尾部补零、写不回文件）、MAP_SHARED 文件（msync、munmap 回写、文件大小不变、两个进程各自 open 后共享页）、read() 写进未触碰的映射、超出 EOF 的 SIGBUS、/dev/zero、用 /dev/mem 读系统定时器、/dev/mem 拒绝内核 RAM、/dev/fb0 画彩条。

已知限制：没有写时复制，fork 复制所有已触碰的私有页；pcache_page 只在页被映射期间存在，不是统一的页缓存，write() 不会更新已映射的共享页；没有 mprotect、mremap、madvise 和页回收。

4.11 MMIO、总线地址与 DMA

4.11.1 MMIO

SoC 把外设寄存器和 RAM 放进同一个物理地址空间，CPU 用普通的 ldr/str 访问它们，由片内地址译码器决定一次访问是去 DDR 控制器还是去某个外设（表 4.5）。

表 4.5: Raspberry Pi 3 的物理地址布局（本内核使用的部分）

物理地址	用途
0x00000000 ~ 0x3fffffff	RAM，以 DTB 的 /memory 为准（QEMU 960 MiB；无 DTB 时 112 MiB）
0x00080000	内核镜像装载地址
0x07200000 ~ 0x072fffff	摄像头 DMA 区（CAMDMA）
0x3f003000	BCM System Timer
0x3f00b000	legacy 中断控制器；+0x880 处为 property mailbox
0x3f101000	时钟管理器（摄像头 CM_CAM1）
0x3f200000	GPIO
0x3f202000	SDHOST（外置 SD 卡）
0x3f205000	BSC0（I2C）
0x3f215000	AUX / Mini UART
0x3f300000	Arasan SDHCI（板载 Wi-Fi）
0x3f801000	Unicam CSI1
0x3f980000	DWC2 USB 主机
0x40000000	ARM-local 中断与定时器路由

外设寄存器必须映射为 Device 内存。若映射成可缓存的普通内存，写操作可能停留在缓存里迟迟不到达设备，读操作可能拿到旧值，两次写还可能被合并或重排——而对设备来说，“写 FIFO”这个动作本身就有副作用。运行时可以用 cat /proc/iomem 查看内核登记的全部外设地址区间。

4.11.2 三种地址视图

Broadcom 的文档用 VideoCore 总线地址描述外设，例如 Mini UART 在 0x7e215000。同一组寄存器在不同观察者眼里地址不同：

```
VideoCore 总线地址   0x7e215000      (文档、设备树里的写法)
ARM 物理地址         0x3f215000      (CPU 发出的地址)
内核虚拟地址         0xffffff803f215000  (经 TTBR1 翻译)
```

DMA 方向的地址转换更容易出错。DWC2 USB、Unicam 这些总线主设备自己访问内存时，看到的是 VideoCore 总线地址，必须把 ARM 物理地址加上别名前缀：

```
#define DWC2_DMA_BUS(a) (0xc0000000U | (uint32)V2P(a))
```

0xC0000000 别名表示“绕过 VideoCore 的 L2 缓存直接访问 SDRAM”。这不是新的内存，也不是 CPU 虚拟地址，而是同一片 RAM 在 GPU 总线上的另一种编址。

4.11.3 PCIe：BAR、DMA 与 IOMMU

PCIe 把两条方向相反的访问路径放在同一条互连上，因而最容易把地址概念混在一起。CPU 访问设备时走入站路径：操作系统给设备的 BAR 分配一段 CPU 物理地址，CPU 对这段 MMIO 窗口执行 `ldr/str`，Root Complex 再把访问路由成 PCIe 请求。设备主动访问主机内存时走出站路径：设备自己的 DMA 引擎发出 PCIe Memory Read/Write 请求，目标是驱动交给设备的 DMA 地址。BAR 和 DMA 因而不是同一套地址，也不是互相替代的功能。

一块 PCIe 设备通常同时涉及四种地址：

表 4.6: PCIe 设备涉及的地址空间

地址	含义
设备内部地址	设备片内 SRAM、FIFO 或寄存器在设备内部互连上的地址，通常只由设备逻辑直接使用
BAR/MMIO 地址	CPU 访问设备寄存器或可见设备内存窗口时使用的主机物理地址
DMA/IOVA 地址	设备 DMA 请求中携带的地址；驱动把它写进设备描述符或 DMA 寄存器
主机物理地址	数据在主机 DDR 中实际占用的位置

没有 IOMMU 时，DMA 地址通常就是设备可达的主机物理地址；有 IOMMU 时，设备使用 IOVA，IOMMU 硬件依据内核建立的页表把它翻译成主机物理地址并检查访问权限。IOMMU 不搬运数据，真正发起读写、产生 PCIe TLP 的仍是设备内部 DMA 引擎。它的价值是隔离设备、支持虚拟机、让连续 IOVA 映射到离散物理页，以及适配设备的地址位宽限制。

DMA 的“数据源”由方向和设备功能决定，并不固定为设备 SRAM。设备向主机写数据时，来源可以是设备 SRAM，也可以是接收 FIFO、硬件流水线的实时输出，或者经控制器读出的外部介质数据：网卡从 PHY/MAC 接收包，NVMe 控制器从 NAND 读数据，采集卡从传感器得到数据。它们可能先暂存在 SRAM/FIFO，也可能边产生边由 DMA 写入主机 DDR：

```
设备 SRAM / RX FIFO / 硬件流水线 / 外部输入
        -> 设备 DMA 引擎
        -> PCIe Memory Write TLP
        -> IOMMU（可选：IOVA -> 主机 PA）
        -> 主机 DDR
```

反方向传输时，主机 DDR 才是数据源。以网卡发送为例，驱动在 DDR 中准备数据包与描述符，把 DMA 地址写入网卡；网卡 DMA 引擎发出 Memory Read，从 DDR 取回数据，送进设备 TX FIFO，最后由 MAC/PHY 发出：

```
主机 DDR
        -> IOMMU（可选）
        -> PCIe Memory Read completion
        -> 设备 DMA 引擎
        -> 设备 SRAM / TX FIFO / 处理单元
```

所以“CPU 通过 BAR 读设备 SRAM”和“设备通过 DMA 把数据写进 DDR”是两件不同的事；即便同一块设备 SRAM 同时参与二者，前者经过 MMIO 窗口，后者由设备 DMA 引擎完成。PCIe 设备也不一定必须实现 DMA：低速控制设备可以只暴露 BAR；网卡、NVMe、GPU 等高吞吐设备则必须依靠 DMA，避免 CPU 逐字节搬运。

4.12 缓存与 TLB：三个真机故障

QEMU 不模拟缓存，也几乎不检查内存属性，下面三个问题都只在真机上出现。它们说明：页表不只是“虚拟地址到物理地址”的映射，还决定了 CPU 怎样访问这块内存。

4.12.1 故障一：停在 BPH0abde

真机串口输出 `BPH0abde` 后不再有动静。对照表 3.3，`d` 表示进入了第一次 `kfree()`，`e` 表示 `memset` 已完成，缺少的 `f` 表示 `acquire(&kmem.lock)` 没有返回。

当时只有 CPU0 在运行，不可能有别的核持锁。真正的原因在于 `acquire()` 的实现：

```
retry:  ldaxr w1, [x0]      // 独占加载，建立“独占监视”
        cbnz  w1, retry
        stxr  w2, w3, [x0]  // 独占存储，失败时 w2 != 0
        cbnz  w2, retry
```

`ldaxr` 读到锁为 0，但 `stxr` 一直返回失败——这是活锁，不是死锁。当时普通 RAM 被映射成 *Normal Non-cacheable*，也没有 Inner Shareable 属性，数据缓存也没有打开。在 Cortex-A53 上，独占监视器依赖缓存一致性机制工作，对这种属性的内存，独占存储可能永远不成功。

修复包括四处：增加 `MT_NORMAL`（Write-Back）类型并让 RAM 使用它；页表项加 `PTE_SH_INNER`；`TCR_EL1` 的页表遍历属性设为 Write-Back、Inner Shareable；`SCTLR_EL1` 打开 C 和 I 位。之后串口变成 `BPH0abdefc1234`，系统顺利进入 shell。

4.12.2 故障二：ls 跳到了不属于它的代码

打开缓存之后，在 shell 里执行 `ls` 出现用户异常：

```
usertrap(): unexpected esr=0x92000047 ec=0x24 pid=3
            elr=0x13dc far=0x13f48
```

EC=0x24 是来自 EL0 的数据中止，ISS 表示 L3 翻译错误、写访问。奇怪的是 `ls` 的代码只到 `0xbd7`，PC 却在 `0x13dc`。

原因在于 D-cache 与 I-cache 不一致。`exec()` 通过内核高地址把新程序的指令写进物理页，新指令此时还在 D-cache 里；用户态从低地址取指，走的是 I-cache，里面可能还留着同一虚拟地址上一个程序（`sh`）的指令。AArch64 不保证数据写入对取指自动可见，必须显式维护：

```
void uvmsync_icache(pagetable_t pagetable, uint64 sz) {
  for(uint64 va = 0; va < PGROUNDUP(sz); va += PGSIZE){
    uint64 kva = uva2ka(pagetable, va);
    if(kva == 0) continue;
    for(uint64 p = kva; p < kva + PGSIZE; p += 64)
      asm volatile("dc cvau, %0" :: "r"(p) : "memory");   // 数据清到统一点
  }
  asm volatile("dsb ish" ::: "memory");
  asm volatile("ic ialluis\n\tdsb ish\n\tisb" ::: "memory"); // 广播作废所有核的 I-cache
}
```

第一版用 `ic ivau` 按内核高地址作废 I-cache，仍然失败：同一物理页有内核高地址和用户低地址两个别名，按其中一个别名维护不能保证覆盖另一个。改用 `ic iallu` 作废整个 I-cache 后问题消失。但打开四核以后它又以另一种形式回来了：`ic iallu` 只作废执行它的那个核的 I-cache，`exec()` 在一个核上装好新程序，进程迁移

到另一个核后，那里仍能命中旧程序的指令——表现为 ELR 指向一条根本不访存的指令却报数据中止（第 7 章摄像头一节）。最终版本用广播到 Inner Shareable 域的 `ic ialluis`。代价是每次 `exec()`/`fork()` 清一次 I-cache，对 xv6 的规模完全可以接受。`exec()`、`uvminit()`（第一个进程）、`uvmcopy()`（`fork`）三处都会调用它。

4.12.3 故障三：sbrk 之后的旧 TLB

进一步的日志显示，上面那次异常还有另一层原因：`sh` 通过 `sbrk()` 新增了 `0x13000` 这一页，但 `growproc()` 修改页表后没有作废 TLB，CPU 继续使用之前缓存的“该地址无效”的翻译结果。修复是在页表扩展或收缩后执行广播式 TLB 失效：

```
        dsb  ishst          // 确保页表写入完成
        tlbi vmalle1is      // 作废 Inner Shareable 域内所有核的 EL1 TLB
        dsb  ish
        isb
```

`vmalle1is` 中的 `is` 很关键：多核以后，同一进程的线程可能正在别的核上运行，只作废本核 TLB 是不够的。收缩地址空间时顺序必须是“清页表项 → 广播作废 TLB → 释放物理页”，否则别的核可能通过陈旧的 TLB 访问一页已经被重新分配的内存。

笔记 经验：在 AArch64 上，改了“将要执行的内存”要做 D/I-cache 同步，改了“页表”要做 TLB 失效，两者不能互相替代。QEMU 两件事都不检查，所以必须上真机验证。

4.13 实验分支：引用计数、写时复制与页缓存

前面各节描述的都是当前源码树。本节换一个角度，记录实验分支 `wip-cow-mmap-ldso` 的设计，它没有整体合入当前源码树：其中的写时复制 fork 已经以另一种方式实现在主线里（第 4.9.4 节），区别在于主线用 `struct page` 的 `refcount` 而不是单独的 `pageref[]` 数组，`PTE_COW` 用第 57 位（第 55 位已被 `PTE_SHARED` 占用），用每个地址空间的 `vm->lock` 而不是全局的 `pte_lock`；页缓存仍只在实验分支里，`mmap` 的实现见第 4.10 节。下面的设计是让多个地址空间共用物理页——fork 后的父子进程、所有进程里的 `libc.so` 代码、同一文件的多个 `mmap`——所需的基础，前提是先回答“这一页还有谁在用”。

4.13.1 物理页引用计数

实验分支写于伙伴系统之前，当时 `kalloc.c` 还是空闲链表，没有页描述符，只好为它能分配的每一页另开一个 16 位计数数组，以物理页号为下标，受 `kmem.lock` 保护。当前源码树的 `struct page` 已经带有 `refcount` 和 `get_page()`/`put_page()`（第 4.5 节），把这套设计移植过来时，下面的 `pageref[]` 就直接换成 `mem_map[pfn].refcount`：

```
static uint16 pageref[PHYSTOP / PGSIZE];

void *kalloc(void)      // 取出空闲页时置为 1
void  kref_inc(uint64 pa)   // 再多一个映射（COW、页缓存、共享库）
int   kref_get(uint64 pa)
void  kfree(void *va)   // 减一；减到 0 才真正放回空闲链表
```

于是“释放”的含义变成了“放弃我这一份”：`uvmunmap()`、`growproc()` 缩小内存、进程退出，一律调用 `kfree()`，共享页只会在最后一个使用者离开时回到空闲链表。不归 `kalloc` 管的物理页（相机 DMA 缓冲、模块区）不计数，也不允许传给这些函数。

这一步也修正了第一版共享 libc 时的隐患：那时共享页用软件位 PTE_SHARED 标记、各处“见到就别释放”，结果 sbrk(-sz) 走的是 growproc() 自己的释放循环，绕过了检查，把所有进程共用的 libc 代码页还给了空闲链表。usertests 的 sbrkbugs 正是这样暴露问题的：子进程释放全部内存后，父进程、sh、init 相继在 libc 的同一地址遇到未定义指令（EC = 0），最终 panic: init exiting。有了引用计数，这类特判全部消失。

4.13.2 写时复制

fork 之后父子共享的可写页在两边都是“只读 + PTE_COW”。PTE_COW 是 ARMv8 留给软件的描述符位 55（MMU 忽略 [58:55]），它的意思是“只读只是因为共享，写了就复制”。写这种页会产生权限错误：

```
} else if(ec == 0x24 && ((esr >> 6) & 1) && (esr & 0x3c) == 0x0c &&
          (cow = uvm_cowfault(p->vm->pagetable, r_far_el1())) != -1){
  if(cow == -2){                        // 没有空闲页可复制
    printf("pid %d %s: out of memory for a copy-on-write fault at %p\n", ...);
    p->killed = 1;
  }
  // 否则返回用户态，那条 str 重新执行，这次写进自己的页
}
```

判断条件逐位对应 ESR_EL1：EC 0x24 是来自 EL0 的数据中止，ISS 的 bit 6（WnR）为 1 表示写，DFSC 0b0011xx 是权限错误。uvm_cowfault() 在 pte_lock 下处理这一页：

```
attrs = (*pte & ~(PA2PTE(~0ULL) | PTE_COW | PTE_AP(3))) | PTE_U;  // EL0 可写
if(kref_get(pa) == 1){              // 别人都已复制或退出：直接收归己有
  *pte = PA2PTE(pa) | attrs;
} else {
  mem = kalloc();                   // 复制一份
  memmove(mem, P2V(pa), PGSIZE);
  if((attrs & PTE_UXN) == 0)        // 可执行页还要同步 I-cache
    uvmsync_icache_kva(mem);
  *pte = PA2PTE(V2P(mem)) | attrs;
  kfree(P2V(pa));                   // 放弃对共享页的那一份引用
}
```

改完页表项后 flush_tlb() 广播作废，旧的只读翻译从所有 CPU 上消失。真正“只读”的页（共享库代码、PROT_READ 映射）没有 PTE_COW，写它们仍是致命错误。

copyout() 也要遵守。 内核写用户内存走的是自己的直接映射，用户页表项的 AP 位对它不起作用。如果不加检查，子进程 read(fd, buf, n) 时内核会把数据直接写进仍与父进程共享的物理页，父进程的内存就被改了；同样，read(fd, libc_text, n) 能让一个进程改写所有进程共用的 libc 代码。所以 copyout() 对每一页先在 pte_lock 下检查：COW 页先复制，只读页拒绝（返回 −1），并且持锁完成这一页的拷贝，免得另一个线程的 fork() 恰好在检查和写入之间把它重新变成共享页。

一把锁。 pte_lock 串行化所有“把页变成 COW”和“把 COW 页变回可写”的操作：fork 标记、写保护异常、copyout()、区间撤销映射。没有它，一个线程可能正把 kref_get()==1 的页就地改成可写，而同一地址空间里另一个线程的 fork() 恰好把同一页共享给了子进程。

效果与代价。 新增的 cowtest 实测：8 MiB 的进程 fork 只用 52 KiB；子进程写遍 8 MiB 后才真正复制 8 MiB；连续 fork 50 次空闲内存不变。代价是内存从“fork 时不够就失败”变成“写的时候不够才失败”：usertests 的

countfree() 子进程先用 sbrk 吃光内存，再写一个仍与父进程共享的数据页，于是在写保护异常里因没有空闲页而被杀——这正是 Linux OOM 的行为。另一个副作用是 fork 太便宜，进程数不再被内存限制住；动态增长的进程表和线性扫描的调度器经不起几万个进程，因此 param.h 加了 NPROC_MAX = 1024 的上限，作用相当于 Linux 的 threads-max。

4.13.3 页缓存

要让不同进程共享同一个文件的页（典型是 libc.so 的代码），内核需要一个以“文件 + 页号”为键的缓存。kernel/pcache.c 是一个很小的版本：

```
struct pcfile {
  struct inode *ip;     // 持有 inode 引用，像 Linux 的 address_space
  uint64 *pa;           // 一页数组：文件第 n 页 -> 物理页
  uint64 lastuse;
};
uint64 pcache_getpage(struct inode *ip, uint64 pgidx);  // 返回 pa，并为调用者 kref_inc
void   pcache_invalidate(struct inode *ip);             // 文件变了：丢弃缓存页
```

- 缓存页是普通的 kalloc() 页，缓存自己持有一份引用，每个映射再各持一份；文件末尾之后的字节读作 0。读入后统一做一次 I-cache 同步，因为它可能被映射成可执行的库代码。
- 缓存最多 8 个文件（NINODE 只有 50），每个文件最多 512 页（2 MiB）；满了就按最近使用淘汰。被淘汰文件的 inode 引用不能在持锁时释放，先放进待释放列表，由 pcache_reap() 在锁外、自开日志事务再 iput()。
- 一致性：filewrite、ftruncate 和 open(O_TRUNC) 之后调用 pcache_invalidate()。已有的映射保留它们拿到的页，相当于一份 MAP_PRIVATE 快照；新的 mmap 看到新内容。
- 加锁顺序固定为“先 ilock(ip)，再页缓存的睡眠锁”；pcache_invalidate() 在 inode 解锁之后调用。

4.13.4 实验分支里的 mmap()

用户地址空间在 sz 之上多了一块区域 [MMAP_BASE, MMAP_TOP)，即 64 GiB 到相机窗口下方。每个 struct vmspace 有一张 32 项的 VMA 表记录这里的区间：

```
struct vma { uint64 start, end; int prot, flags; };   // start == end 表示空闲
```

kernel/mmap.c 在调用时就把整段映射好（没有按需调页），所以 VMA 只需记录区间：

表 4.7: mmap() 支持的形式

形式	实现
MAP_ANONYMOUS	新分配的清零页，PROT_WRITE 时可写
文件 + MAP_PRIVATE	页缓存页；只读映射直接共享，可写映射以“只读 + PTE_COW”共享，写时复制，缓存中的原页不变
文件 + MAP_SHARED	只允许只读（没有回写）
MAP_FIXED	先撤销该区间原有的映射，再映射到指定地址
munmap()	可以只撤销区间的一部分，必要时把一个 VMA 切成两个

PROT_EXEC 决定 UXN 位；内核永远不执行用户页（PXN）；PROT_NONE 映射成 AP = 00，EL0 无法访问。fork 时 vma_copy() 对每个区间调用 uvmcopy_range()，因此 mmap 区域同样写时复制；进程退出和 exec() 替换镜像时 vma_unmap_all() 撤销全部区间。sbrk 不允许越过 MMAP_BASE；相机的 /dev/video0 仍由驱动自己提供页，sys_mmap() 遇到字符设备就交给 chrdev_mmap()。

实验分支的 mmaptest 覆盖了这些路径：只读映射的内容与文件一致；私有可写映射被写后，另一个映射和文件本身都不变；写 PROT_READ 映射的子进程被杀；fork 后子进程的写入不影响父进程；匿名页为零；munmap

掉中间一页后访问它会出错；`MAP_FIXED` 覆盖已有映射；文件被改写后新的映射能看到新内容。下一章的用户态动态链接器 `ld.so` 正是用这套接口装载 `libc.so` 的（第 8.5.7 节）。

有了页表和物理页分配器，内核就可以为每个进程准备一个独立的地址空间。下一章看进程本身：`struct proc` 里有什么，第一个进程怎样造出来，`fork()`、`exec()` 和调度器怎样使用本章的这些函数。

第五章 进程与调度

本章内容

- 进程的数据结构
- 动态进程表与按需分配的内核栈
- 第一个用户进程
- fork() 逐行解析
- clone 标志与 vfork
- 上下文切换与调度器
- exec、exit 与 wait
- 睡眠、唤醒与同步原语
- 多核软实时调度

上一章结束时，内核已经有了页表和物理页分配器，但还只有 main() 这一条执行流。本章造出第一个进程，再沿着 fork()、exec()、exit()/wait() 看进程怎样繁衍和消亡；中间穿插调度器怎样在多个进程之间切换，最后是建立在调度器之上的睡眠、同步原语和软实时调度。

5.1 进程由什么组成

在 xv6 里，一个进程就是一个 struct proc，加上它引用的几样东西：一张用户页表、一个内核栈、一个 trapframe、一组打开的文件。图 5.1 画出了它们的关系。

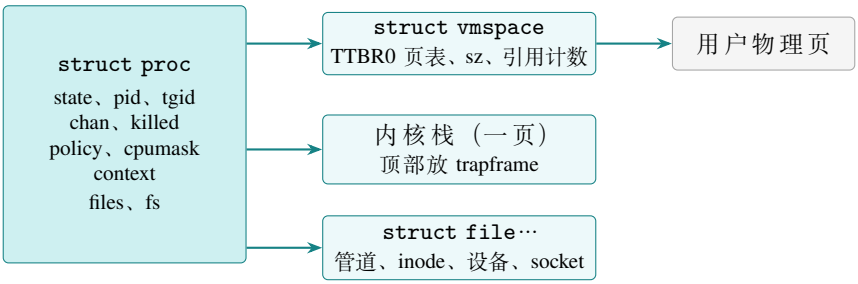


图 5.1: 一个进程的组成

5.1.1 struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

struct proc {
  struct spinlock lock;
  // 以下字段需持有 p->lock
  enum procstate state;
  void *chan;                  // 非 0 表示睡在这个“通道”上
  int killed, xstate;
  int pid, tgid, is_thread;    // tid、线程组 id
  int sid, pgid, ctty;         // 会话、进程组、控制终端
  int policy, rt_priority, nice;
  uint cpumask;                // CPU 亲和性
  int slice; uint64 rq_seq;    // 时间片、入队序号
  // 以下字段需持有 wait_lock
  struct proc *parent;
  int vfork_pending;           // CLONE_VFORK 父进程等子进程 exec/exit
```

```
  // 以下字段只由进程自己访问
  uint64 kstack, kstack_pa, kstack_slot;
  struct proc *next;           // 动态进程表链表
  struct vmspace *vm;          // 用户地址空间，CLONE_VM 共享
  struct trapframe *trapframe; // 用户态现场，位于内核栈顶
  struct context context;      // 内核态现场，swtch() 用
  struct files_struct *files;  // 描述符表，CLONE_FILES 共享
  struct fs_struct *fs;        // cwd 与 cwdpath，CLONE_FS 共享
  struct file *fpin[NFPIN];    // 本次系统调用钉住的 file 引用
  int nfpin;
  char name[16];
};
```

字段按“谁在什么锁下访问”分组，这是 xv6 的一贯风格。与原版相比，这里多了会话和进程组（TTY 作业控制要用）、调度属性（软实时调度）、线程组 id，并且用户地址空间、描述符表和当前目录都被抽成了可共享、带引用计数的对象（`struct vmspace`、`files_struct`、`fs_struct`），由 `clone` 标志决定共享还是复制（第 5.5 节）。

5.1.2 两种现场

进程有两份寄存器现场，初学时很容易混淆：

- **trapframe**：用户态被打断时（系统调用、中断、异常）由 `usavereg` 保存，包含全部 31 个通用寄存器、`elr`、`spsr` 和用户 `sp`。它描述“回到用户态时从哪里继续”。
- **context**：内核态主动让出 CPU 时由 `swtch()` 保存，只包含被调用者保存寄存器 `x18 ~ x30` 和 `sp`。它描述“这个进程的内核线程停在了哪里”。

AArch64 的调用约定规定 `x19 ~ x28`、`x29`（帧指针）和 `sp` 由被调用者保存，`x30` 是返回地址。`swtch()` 是一次普通的函数调用，调用者保存寄存器已经由编译器在调用前存好，所以 context 只需要这 14 个值。

5.2 动态进程表与内核栈

原版 xv6 用静态数组 `proc[NPROC]`，并在启动时为 64 个槽位全部分配内核栈。本项目改成了按需增长的链表：

```
static struct proc* allocproc(void) {
  acquire(&proc_table_lock);
  for(p = proc_head; p; p = p->next) {        // 先找一个 UNUSED 节点复用
    acquire(&p->lock);
    if(p->state == UNUSED) {
      if(p->kstack_pa == 0 && alloc_kstack(p) < 0) { ... return 0; }
      goto found;
    }
    release(&p->lock);
  }
  p = newprocslot();                          // 没有就新建一个节点
  acquire(&p->lock);
found:
  release(&proc_table_lock);
  p->pid = allocpid();  p->tgid = p->pid;
```

```
  sched_defaults(p);
  p->state = USED;
  sp = (char*)p->kstack + PGSIZE;
  sp -= sizeof(*p->trapframe);                  // trapframe 放在内核栈顶
  p->trapframe = (struct trapframe*)sp;
  p->vm = vmspace_alloc();                      // 空的用户地址空间
  memset(&p->context, 0, sizeof(p->context));
  p->context.x30 = (uint64)forkret;             // 第一次被调度时“返回”到 forkret
  p->context.sp  = (uint64)sp;
  return p;                                     // 持有 p->lock 返回
}
```

几个设计要点：

- 每个节点分到一个永不改变的 kstack_slot，内核栈虚拟地址为 KSTACK(slot)，相邻栈之间隔一页保护页；
- 节点空闲时释放内核栈的物理页，但节点本身永远留在链表里。调度器、wait()、wakeup() 都无锁遍历这个链表，保留节点就不会出现悬空的 next 指针，省去了 RCU 一类的复杂机制；
- 释放内核栈时先清页表项，再广播作废 TLB，最后才 kfree 物理页（第 4 章的规则）。

forktest 同时创建并回收 128 个子进程，验证已经越过了旧的 64 个槽位上限。

5.3 第一个用户进程

所有用户进程都是 fork() 出来的，但第一个进程只能由内核手工构造。userinit() 分配一个进程，把一段内置的机器码 initcode 拷到它的第 0 页：

```
void userinit(void) {
  struct proc *p = allocproc();
  initproc = p;
  uvminit(p->vm->pagetable, initcode, sizeof(initcode));  // 拷贝并同步 I-cache
  p->vm->sz = PGSIZE;
  p->trapframe->elr  = 0;          // 用户态 PC
  p->trapframe->spsr = 0;          // M[3:0]=0: EL0t
  p->trapframe->sp   = PGSIZE;     // 用户栈顶
  safestrcpy(p->name, "initcode", sizeof(p->name));
  // p->fs->cwd 由 forkret 中的 prepare_namespace() 设置
  make_runnable(p, 0);
  release(&p->lock);
}
```

spsr = 0 有两层含义：M[3:0] = 0 表示回到 EL0t，DAIF 全部清零表示用户态开着中断。代码和栈共用这一页：代码只占开头 0x48 字节，sp 从 0x1000 往下长，而 initcode 根本不用栈。

5.3.1 initcode 做了什么

initcode 由 user/initcode.S 汇编而来，只做一件事：调用 exec("/init")。kernel/core/proc.c 里的 initcode[] 是它的机器码，按地址展开如下：

```
0x00  ldr x0, =init        // 字面量池 0x38 中是 0x1c，即 "/init"
0x04  ldr x1, =argv        // 字面量池 0x40 中是 0x28，argv = {0x1c, 0}
```

```
0x08  mov x7, #SYS_exec    // 本移植用 x7 传系统调用号 (7)
0x0c  svc #0
0x10  mov x7, #SYS_exit    // 只有 exec 失败才会走到这里
0x14  svc #0
0x18  b   0x10
0x1c  "/init\0"
0x28  argv: .quad 0x1c, 0
```

整个过程是：

1. 第一次被调度时，swtch() 恢复的 x30 是 forkret，于是“返回”到 forkret()。它释放调度器替它拿着的 p->lock；作为系统中第一个进程，它还要调用 prepare_namespace() 挂载根文件系统（读 superblock、恢复日志，期间可能睡眠，所以不能放在 main() 里），然后 usertrapret()，eret 到 EL0 的地址 0；
2. svc 陷入，el0sync 把寄存器压成 trapframe：x0 是 "/init"，x1 是 argv，x7 = 7；
3. syscall() 按 x7 调用 sys_exec()，后者调用 exec("/init", argv)；
4. exec() 改写 trapframe：elr 设为 /init 的入口，sp 设为新用户栈，x1 设为 argv，返回值 argc 写进 x0；
5. eret 进入 /init 的 main(1, {"/init"})。initcode 那一页随旧页表一起释放；进程还是 pid 1，initproc 不变，只是换了镜像。

如果第 4 步失败，例如根文件系统里没有 /init，svc 返回 −1，执行落到 0x10 的 exit 循环。exit() 发现 p == initproc，于是 panic("init exiting")。如果 ext2 根镜像里漏装了 /init（6.8.9 节），看到的就是这个 panic。

5.3.2 为什么需要 initcode

内核刚启动时只有 main()，它运行在 EL1，没有任何进程，因此有三件事做不到：

- **读文件**：/init 在磁盘上，读它要睡眠等待 I/O，而 sleep() 需要 myproc()；根文件系统也要到 forkret() 才挂载；
- **exec**：exec() 要改写 p->trapframe、替换 p->vm->pagetable，前提是已经有一个进程；
- **进入 EL0**：唯一的办法是 eret，而 eret 需要一个已经映射好的用户程序。

initcode 是编译进内核镜像的一小段字节，不需要文件系统。它的作用是让 pid 1 存在，并让 pid 1 在用户态用普通的 svc 发起第一次 exec。这样，第一个程序和以后所有程序走的是同一条路径（svc → el0sync → usertrap → sys_exec → exec），内核里没有“直接加载第一个程序”的特殊代码。顺带，开机时就把 EL0 → EL1 → EL0 的往返、系统调用分派和 trapframe 布局完整验证了一遍。

5.3.3 不用 initcode 的写法

initcode 不是必需的。Linux 的 kernel_init 就是在内核态直接调用 kernel_execve()，再像系统调用返回一样回到用户态。在本移植中，可以在 forkret() 第一次运行时直接调用内核的 exec()：

```
if(first){
  first = 0;
  prepare_namespace();
  blkdev_selftest();
  static char *argv[] = { "/init", 0 };
  int argc = exec("/init", argv);   // 内核态直接装入 /init
  if(argc < 0)
    panic("no /init on root filesystem");
  tf->x0 = argc;                    // 没经过 syscall(), x0 要手动写
}
```

```
usertrapret(tf);
```

同时，userinit() 删掉 uvminit()、sz = PGSIZE 和设置 elr、sp、spsr 的三行，这些都由 exec() 设置。这样做是可行的：forkret() 运行在 pid 1 的进程上下文里，allocproc() 已经建好一张空页表供 exec() 替换；exec() 用 copyout 把参数拷进用户栈，参数来自内核数组也没有问题。两种做法的取舍见表 5.1。

表 5.1: 保留 initcode 与在 forkret 中直接 exec

	保留 initcode	forkret 中直接 exec
特殊路径	无，第一个程序也走 svc	forkret 多一段只运行一次的分支
维护负担	手抄的 initcode[] 须与 initcode.S 同步	无
找不到 /init	报 init exiting，提示较绕	直接报 no /init on root filesystem
启动自检	开机即测一次完整的系统调用往返	要到 /init 运行后才有第一次系统调用

即使保留 initcode，也值得让 Makefile 用 xxd -i 从编译出的 user/initcode 自动生成这个数组，避免改了汇编却忘了改字节。

5.4 fork() 逐行解析

fork() 现在只是一行：return do_clone(SIGCHLD, 0, 0, 0, 0, 0);。do_clone() 仿照 Linux 的 copy_process()，是 fork、线程库的 clone 和 Linux 风格 clone(flags, ...) 的共同实现；flags 决定地址空间、描述符表和当前目录是共享还是复制（第 5.5 节）。不带任何共享标志时，它走的就是 fork 的路径：

```
int do_clone(uint64 flags, uint64 stack, uint64 ptid, uint64 ctid,
             uint64 entry, uint64 arg) {
  ...                                                    // 检查 flags 组合
  files = (flags & CLONE_FILES) ? files_get(p->files) : files_dup(p->files);  // (1)
  fs    = (flags & CLONE_FS)    ? fs_get(p->fs)       : fs_copy(p->fs);
  if(!(flags & CLONE_VM))
    down_read(&vm->mmap_lock);                           // (2)
  if((np = allocproc()) == 0) goto bad;                  // (3)

  if(flags & CLONE_VM){                                  // 线程、vfork：共享 vmspace
    vm->refcount++; vm->users++;
    vmspace_put(np->vm); np->vm = vm;
  } else {                                               // (4) fork：复制
    acquire(&vm->lock);
    uvmcopy(vm->pagetable, np->vm->pagetable, vm->sz);
    np->vm->sz = vm->sz;
    dup_mmap(vm, np->vm);                                // mmap 区按 VMA 复制
    release(&vm->lock);
    up_read(&vm->mmap_lock);
  }
  np->tgid = (flags & CLONE_THREAD) ? p->tgid : np->pid;

  *np->trapframe = *p->trapframe;                        // (5)
  np->trapframe->x0 = 0;                                 // (6)
  if(stack) np->trapframe->sp = stack;

  np->files = files;  np->fs = fs;                       // (7)
```

```
  np->sid = p->sid;  np->pgid = p->pgid;  np->ctty = p->ctty;
  np->policy = p->policy; ... np->slice = timeslice(np);     // (8)

  acquire(&wait_lock);                                       // (9)
  np->parent = p;
  np->vfork_pending = (flags & CLONE_VFORK) != 0;
  release(&wait_lock);
  acquire(&np->lock); make_runnable(np, 0); release(&np->lock);  // (10)
  if(flags & CLONE_VFORK)                                    // (11)
    while(np->vfork_pending && !p->killed)
      sleep(&np->vfork_pending, &wait_lock);
  return np->pid;
}
```

1. fork 复制描述符表（`files_dup()`）和当前目录（`fs_copy()`）。复制的是 `struct file` 的引用，父子共享同一个文件偏移，这正是 sh 实现重定向和管道的基础；
2. `dup_mmap()` 要遍历父进程的 VMA，复制期间不能有 mmap/munmap 插进来。`mmap_lock` 是可睡眠的读写信号量，必须在 `allocproc()` 之前拿——`allocproc()` 返回时已经持有 `np->lock` 这把自旋锁；
3. 分配进程描述符、内核栈和空的地址空间，context 设为“从 `forkret` 开始”；
4. 复制父进程的用户内存：`uvmcopy()` 逐页复制 `[0, sz)`（只读共享库页共享），`dup_mmap()` 复制 mmap 区：私有页拷贝，共享页 `mapcount++`，设备帧照抄 PTE（第 4.10 节）；
5. 复制整个 trapframe——子进程回到用户态时，PC、栈指针、全部寄存器都和父进程在 `svc` 之后一模一样；
6. 唯一的区别：子进程的 `x0`（系统调用返回值）改成 0；
7. 换上第 (1) 步准备好的描述符表和 `fs_struct`，继承会话、进程组和控制终端；
8. 像 Linux 一样，子进程继承调度策略、优先级、nice 和 CPU 亲和性；
9. 设置父进程指针，受 `wait_lock` 保护，与 `wait()`、`exit()` 同步；
10. 把子进程放入运行队列，从这一刻起别的 CPU 就可能开始运行它；
11. 只有 vfork 才在这里睡眠，等子进程 `exec` 或退出；fork 直接返回子进程的 pid。

图 5.2 对比了父子两条返回路径。

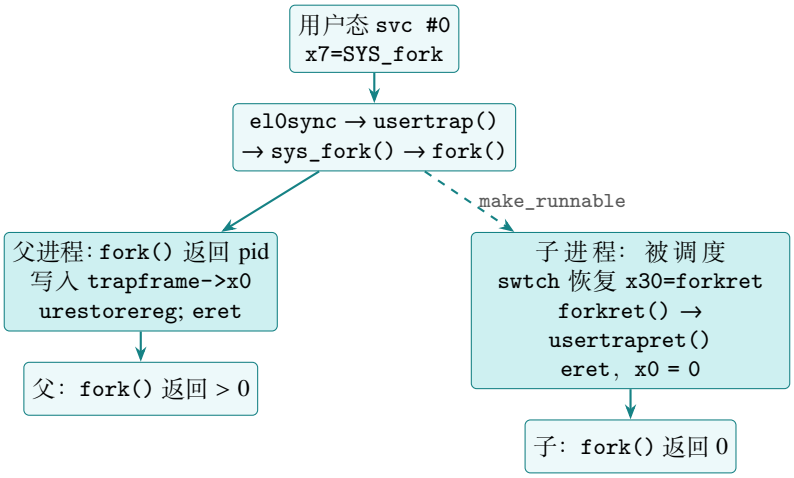


图 5.2: `fork()` 的两条返回路径

子进程从来没有执行过 `fork()` 的前半段，它的内核栈上只有一个 trapframe。调度器第一次运行它时，`swtch()` 恢复的返回地址是 `forkret`，`forkret()` 再调用 `usertrapret()`，从复制来的 trapframe “返回”到用户态——在用户程序看来，就好像子进程也刚刚从 `fork()` 返回，只是返回值为 0。

5.4.1 新进程的第一次运行

allocproc() 为新进程伪造了一个“刚刚调用过 swtch()”的上下文：context.sp 指向内核栈上 trapframe 的下方，context.x30（返回地址）指向 forkret。调度器第一次 swtch() 到它时，ret 指令就跳进了一个从未被调用过的函数。这是所有操作系统创建新任务的共同技巧：教学内核里常见的写法是把返回地址设成一段叫 ret_from_fork 的汇编，Linux 也是如此。

这里有一个细节容易忽略：调度器在 swtch() 之前持有新进程的 p->lock，而持锁期间本 CPU 的中断是关闭的，所以新进程从 forkret 开始执行的那一刻不会被抢占。forkret() 的第一件事就是释放这把锁，相当于“完成交接、重新允许抢占”。若这时被定时器打断，也只是正常的抢占。然后它调用 usertrapret()，把父进程复制来的 trapframe 装回寄存器，eret 到用户态：

```
void forkret(void)
{
  static int first = 1;
  struct proc *p = myproc();
  struct trapframe *tf = p->trapframe;

  release(&p->lock);            // 调度器替我们拿着的锁
  if (first) {                  // 第一个进程：在进程上下文中挂载根文件系统
    first = 0;
    prepare_namespace();        // rootfs、devtmpfs、挂载 root=、MS_MOVE、chroot
    blkdev_selftest();
  }
  usertrapret(tf);              // 从 trapframe 返回用户态
}
```

要分清 forkret 的角色：它只是新进程回到用户态的一块跳板，本身运行在 EL1，用的就是 context.sp 指定的那段栈，即紧挨在 trapframe 下方的内核栈。sp 的初值之所以设在 trapframe 下沿，是为了让 forkret 以及它调用的 prepare_namespace() 从那里往下用栈，不覆盖 trapframe；trapframe 里准备好的用户现场要一直保持到 eret 那一刻。

context 和 trapframe 在这里各管一段：swtch 用 context 换栈、跳到 forkret；usertrapret 用 trapframe 恢复用户寄存器并 eret。context 自己不参与返回用户态，它也不在栈上，而是 struct proc 的一个字段，context.sp 只是一个指向内核栈的数值。

5.4.2 回到用户态：trapret 中的中断竞争

forkret() 的最后一步是调用 usertrapret()，经 trapret 返回用户态。每个新 fork 出的子进程第一次运行都走这条路，而这里藏着一个早就存在的竞争，引入块设备层后时序变化才在 QEMU 上暴露出来。

trapret 先用 urestorereg 装载 ELR_EL1/SPSR_EL1，再 eret。forkret() 开头的 release(&p->lock) 会经 pop_off() 按调度器当时的状态重新开中断，于是 trapret 是在开中断的情况下运行的。IRQ 若恰好落在 msr 与 eret 之间，就会把这两个寄存器改写成它自己的 EL1 返回现场，eret 回到的是 EL1 的 trapret+0x14，此时 sp 已等于内核栈顶，下一条 ldp 读到栈上方未映射的保护页。跑 usertests forkfork/concreate 时看到的正是 elr=trapret+0x14，far 正好是内核栈顶。修复是在 urestorereg 前无条件屏蔽中断：

```
trapret:
    msr daifset, #0xf     // 屏蔽：ELR/SPSR 加载到 eret 之间不能有 EL1 IRQ
    urestorereg
    eret
```

笔记 daifset 只影响 EL1 当前的 PSTATE；eret 会从 SPSR_EL1 恢复 DAIF，回到用户态后的中断状态由 SPSR 决定，不受这条指令影响。

5.4.3 实例：fork 快照中的虚拟地址与物理页

为了看清这些结构在内存里的真实位置，在 fork() 末尾调用 procvmtrace(np, "process-create:fork")，打印子进程的 proc、内核栈、trapframe、context 和用户页表。开机后的两份输出分别属于：

- **pid 16**：init 刚 fork 出、即将 exec /bin/sh 的子进程；
- **pid 17**：在 sh 中输入 ls 后，sh fork 出、即将 exec /bin/ls 的子进程。

两者都是 state=1（USED），已经建好，还没有被调度过。

一个进程：从虚拟地址到物理页. 图 5.3 选的是 pid 17，也就是后面执行 /bin/ls 的那个进程；fork 刚完成时它还是 sh 的拷贝。上半部分是虚拟地址视角，下半部分把物理内存 0x6e9e000–0x6ecffff 按页横向排开。10 条箭头分别从 struct proc、p->context、p->vm、内核栈、trapframe、三级页表、用户 sp 和 elr 连到各自的物理页。内核对象的地址换算见表 5.2。

表 5.2: pid 17 的内核对象：虚拟地址与物理地址

对象	内核虚拟地址	物理地址	换算
struct proc	0xffffff8006eb7000	0x6eb7000	直接映射：PA = VA − KERNBASE
p->context	0xffffff8006eb70b0	0x6eb70b0	嵌在 proc 内，偏移 +0xb0
vmspace	0xffffff8006eb5000	0x6eb5000	直接映射
内核栈（槽 15）	0xffffffbfffe0000	0x6ec0000	KSTACK(slot)，经内核页表映射
trapframe	0xffffffbfffe0ef0	0x6ec0ef0	内核栈顶 − 272
页表根（L1）	0xffffff8006eb8000	0x6eb8000	TTBR0 指向它
L2 页表	0xffffff8006ecf000	0x6ecf000	L1[0] 指向它
L3 页表	0xffffff8006ecc000	0x6ecc000	L2[0] 指向它

从这张图还能读出以下几点：

- 这几页并不相邻：它们大多是 pid 16 exec sh 时刚释放、又被 kalloc 发出来的页，见本节后面的“两个进程：物理页的复用”。
- trapframe 里的值都能对上。elr = 0xe5bc 和 x17 = 0xe59c 是 sh 的 libc 基址 0xa000 加 0x459c（__syscall），elr 指向其中 svc #0 的下一条指令；x16 = 0x143e0 是 libc 的 GOT 起点；x0 = 0 是子进程中 fork() 的返回值。
- 用户地址要查 L3 叶子再加页内偏移：sp 0x9830 经 L3[9] 得到物理地址 0x6ea9830，elr 0xe5bc 经 L3[14] 得到 0x6ea45bc。sh 比 init 大，所以 ld-xv6 装在 0x3000，libc 从 0xa000 开始。
- VA 0x5000 是保护页。exec 为它分配了物理页，但清掉了 PTE_U，而 dump 只打印用户可访问的页；按页号空缺推测是 0x6ec7000。

context.sp 什么时候变. fork 刚完成时 context.sp 等于 trapframe 的地址。之后：

- **每次 svc 或中断**：el0sync 从内核栈顶执行 sub sp, sp, #272，trapframe 总在同一位置被重写，context.sp 不受影响——只有 swtch 会写它。
- **在系统调用里睡眠或被抢占**：sleep/yield → sched → swtch 把当时的内核 sp 存入 context.sp。这个值低于 trapframe，因为 usertrap → syscall → sys_xxx → sleep → sched 的调用帧压在 trapframe 下面。
- **再次被调度**：swtch 恢复这个 sp，从 sched 返回，沿调用链回到 usertrap，最后经 trapframe 回到 EL0。

不睡眠的系统调用（如 getpid）全程不会改变 context.sp。

两个进程：物理页的复用. 两份快照的页表根都是 0x6eb8000，但这不是共享页表。在两次快照之间，pid 16 exec 了 sh，旧的 init 映像被释放；sh 再 fork 出 pid 17 时，kalloc 后进先出，最先发出的正是这些刚释放的页：

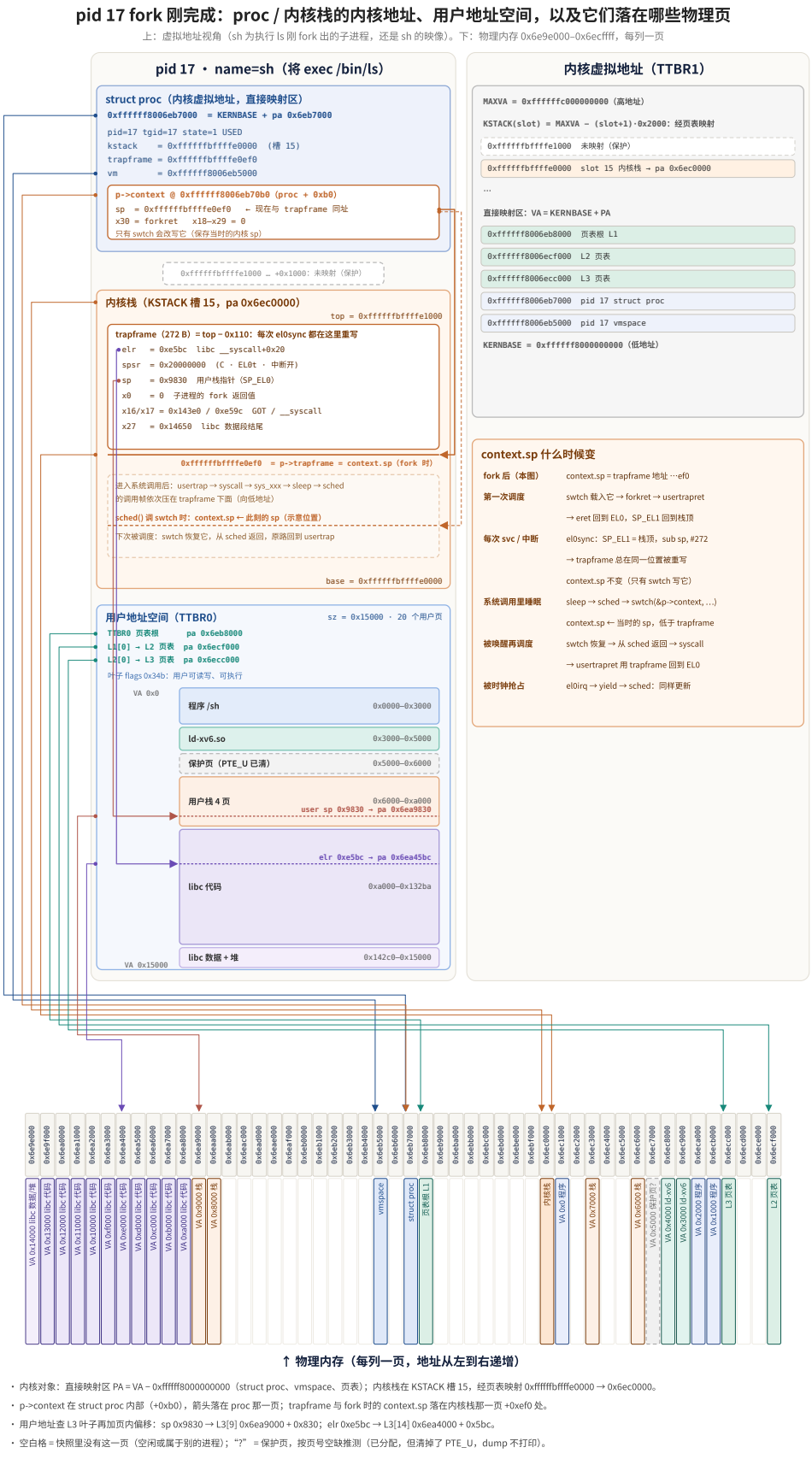


图 5.3: pid 17 fork 刚完成：上半部分是虚拟地址视角，下半部分是物理页，箭头连接各对象所在的物理页

- 0x6eb8000：pid 16 的页表根 → pid 17 的页表根；
- 0x6eb5000：pid 16 的 L2 页表 → pid 17 的 vmspace；
- 0x6ec0000：pid 16 的 L3 页表 → pid 17 的内核栈；
- 0x6eb7000：pid 16 的用户页 VA 0x13000 → pid 17 的 struct proc。

同一个物理页在很短的时间里就换了用途，所以 kfree 之后仍保留旧指针（use-after-free）的错误，在这里会立刻表现为另一个进程的数据被改坏。另外，从 pid 17 的寄存器可以看出 sh 比 init 大：ld-xv6 装在 0x3000，libc 基址是 0xa000，后面的区域整体后移了一页。

同一个进程：exec 之后与第一个系统调用. 接着图 5.3，pid 17 执行 exec("/bin/ls")。打开 VM_TRACE 后在两个时刻打印快照：A 在 exec() 刚 switchuvm 换上新页表、还没 return 时；B 在 exec 之后第一个系统调用即将返回时。这个调用是 ld-xv6 的 open("/lib/libc.so")，返回 fd 3。图 5.4 的左、中两列分别是 A 和 B，内核栈页按比例画；下方是涉及的物理页，实线箭头来自 A，虚线箭头来自 B。

表 5.3: pid 17 内核栈中的三个位置（页内偏移）

	A：exec-image:first	B：first-syscall:return
当前内核 sp	+0x940：还在 sys_exec 的深层调用链里，已用 1728 B	+0xe80：只剩 usertrap、syscall 两帧，272 + 112 B
context.sp	+0x530：exec 读 ELF 时等磁盘睡眠	+0x690：open 在 namei/bread 中睡眠
trapframe	+0xef0	+0xef0：每次 el0sync 都在原地重写

表 5.3 印证了前面的结论：context.sp 只在睡眠时由 swtch 改写，两次都低于 trapframe，深度取决于睡眠时的调用链。图中还能看到：

- **exec 只换用户页表。** proc、vmspace、内核栈都沿用 fork 时的页；页表根从 0x6eb8000 换成 0x6e8c000。新映像只有 8 个用户页：/bin/ls、ld-xv6、保护页和 4 页栈，sz = 0x9000。物理页按分配顺序递减：根 0x6e8c → VA 0 页 0x6e8b → L2 0x6e8a、L3 0x6e89 → 用户页 0x6e88 到 0x6e81。
- **trapframe 已指向新程序。** A 的 elr = 0x2750 是 ld-xv6 的入口，sp = 0x8f60 指向 argv 表；但 x0 = 0x97d0 还是旧值，要等 return argc 后才由 syscall 写入。B 的 elr = 0x2b9c 是 ld-xv6 自带的 __syscall（0xb7c）+ 0x20，x0 = 3 是 open 返回的 fd。
- **exec 不清零通用寄存器。** x16/x17 在 A、B 中都还是 sh 留下的 0x143e0/0xe59c。Linux 的 exec 会清零，这里没有，算是一个小的信息泄露点。
- 这次的构建在 struct proc 里加了追踪字段，p->context 从 proc + 0xb0 移到了 proc + 0xb8。

在 B 这一刻 libc 还没有装入；接下来 ld-xv6 用 pread 和 sbrk 把它装到 0x9000，过程见第 8.5.6 节。

读图：用户地址怎样落到物理页. ls 的 8 个用户页逐页映射：VA 0x0 和 0x1000（程序）分别映射到 0x6e8b000 和 0x6e88000，0x2000、0x3000（ld-xv6）映射到 0x6e87000、0x6e86000，栈 0x5000–0x8000 映射到 0x6e84000–0x6e81000。有三点值得注意：

- 图中的“VA 0x2000”是页的起始地址，不是大小。0x2000 同时也是 ld-xv6 的装载基址：ls 结束于 0x1ac8，向上取页得到 0x2000。
- ls 的两页在物理上不相邻。uvmalloc 每轮先 kalloc 一个数据页，再调 mappages；第一次 walk() 发现缺 L2、L3，当场分配了 0x6e8a 和 0x6e89，于是 VA 0x1000 只能拿到 0x6e88。kalloc 是单页分配器，本来就不保证物理连续。
- 0x6e87750 正是 ls 执行的第一条用户指令：ld-xv6.so 文件偏移 0x750 处的 ldso_start，第一条指令是 sub sp, sp, #0x530。因此 B 时刻的用户 sp 0x8a30 恰好等于 0x8f60 − 0x530。

内核对象在哪个地址空间. 物理条带里，用户页和 struct proc 挨在一起，只说明它们来自同一个页池，并不表示 proc 在用户空间。p = 0xffffff8006eb7000、p->vm = 0xffffff8006eb5000 和页表页都位于直接映射区（VA

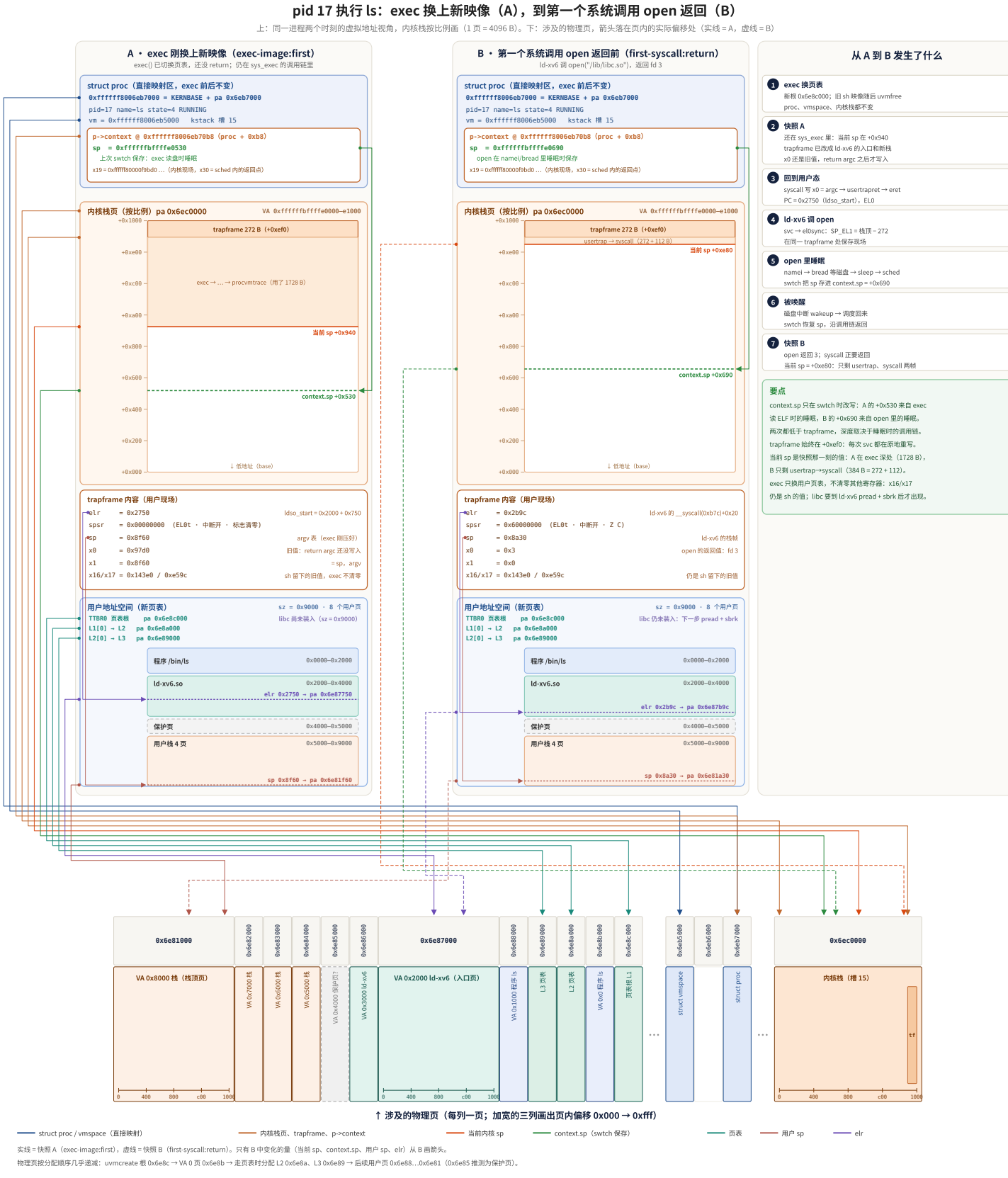


图 5.4: pid 17 执行 ls：exec 换上新映像（A）与第一个系统调用 open 返回（B）；内核栈按比例，下方物理页带页内偏移

= KERNBASE + PA），由 TTBR1_EL1 翻译，PTE 不给 EL0 权限，用户页表中也没有任何一项指向它们。struct vmspace 同样是内核数据结构：它用 pagetable 和 sz 描述一个用户地址空间，从 proc 中拆出来是为了让线程共享（第 8.4.5 节）。内核栈则在 KSTACK 槽位（0xffffffbffffe0000），经内核页表映射到 0x6ec0000；槽中另一页不映射，作为保护页。同一个物理页可以先后有完全不同的身份，例如 0x6eb7000 在 pid 16 时是用户页，到 pid 17 时变成了 struct proc。地址翻译的硬件细节见图 4.2。

5.5 clone 标志与 vfork

Linux 没有“进程”和“线程”两套创建接口，只有一个 clone(flags, ...)：flags 逐项说明子任务与父任务共享什么。xv6-aarch64 照此把 struct proc 里原来内嵌的三样东西拆成三个带引用计数的对象：

表 5.4: struct proc 指向的三个可共享对象

对象	标志	复制函数	内容
struct vmspace	CLONE_VM	uvmcopy + dup_mmap	页表、sz、VMA 链表、mmap_lock；refcount 与 users 两个计数
struct files_struct	CLONE_FILES	files_dup	fd[NOFILE]，原来的 p->ofile[]
struct fs_struct	CLONE_FS	fs_copy	cwd inode 和规范化的 cwdpath

标志的取值与 Linux 相同（include/uapi/sched.h），检查规则也相同：CLONE_THREAD 要求 CLONE_SIGHAND，CLONE_SIGHAND 要求 CLONE_VM；共享地址空间又不是 vfork 时必须给出新栈，因为两个同时运行的任务不能用同一个用户栈。三种常用组合见图 5.5：

- fork()：SIGCHLD，三样都复制；
- thread_create()：CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD，三样都共享。这是一处行为变化：以前线程有各自的 fd 表，现在像 pthread 一样，一个线程 close(fd)，其他线程也看不到这个 fd 了；
- vfork()：CLONE_VM|CLONE_VFORK|SIGCHLD（0x4111），共享地址空间，复制描述符表和当前目录。

用户接口. 原有 ABI 不变：fork() 仍走 SYS_fork，线程库改调 thread_clone()，底下仍是固定参数的 SYS_clone。新增的 SYS_clone_flags（95）按 arm64 Linux 的参数顺序接收 (flags, stack, ptid, tls, ctid)，libc 在其上提供 glibc 形式的 clone(fn, stack, flags, arg, ...) 和 vfork()。CLONE_PARENT_SETTID/CLONE_CHILD_SETTID 在子进程运行之前把 TID 写到 ptid/ctid；TLS 不支持，tls 被忽略。

共享描述符表. files_struct 共享之后，一个任务在系统调用里使用的 struct file 可能被另一个任务 close() 掉。所以 fdget() 在表被共享（count > 1）时给 file 多拿一个引用，钉在 p->fpin[] 里，系统调用返回时由 fdput_all() 放掉；私有表不会被别人改，省掉这一步。open() 等到 struct file 完全建好才 fd_install()，别的线程不会看到半成品。

vfork. vfork 子进程借用父进程的全部内存，包括栈：它在父进程的栈上运行，可以直接写父进程的变量和 mmap 区。为了不让两边同时改同一个栈，父进程在 do_clone() 末尾睡在 &np->vfork_pending 上，直到子进程调用 vfork_done()——在 exec() 成功换上自己的地址空间之后，或者在 exit() 一开始。父进程被 kill 时也会醒来。

两个计数. 共享 vmspace 的任务退出时，最后一个才能拆 mmap 区；但页表要留到僵尸被 wait() 回收。Linux 用 mm_users 和 mm_count 区分这两件事，xv6 对应的是 users 和 refcount：

- refcount：有多少个 struct proc（包括僵尸）持有它。freeproc() 调 vmspace_put()，减到 0 释放页表；

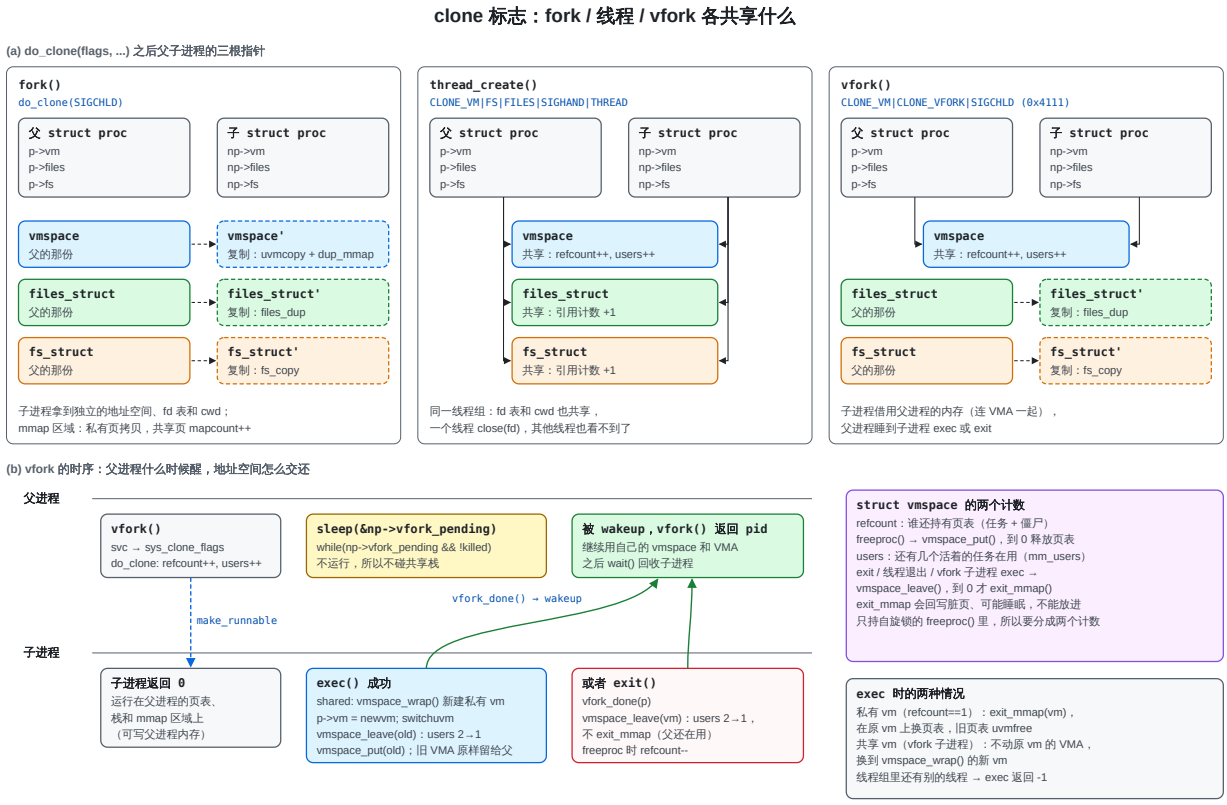


图 5.5: fork、线程和 vfork 各共享什么，以及 vfork 父进程何时醒来

- `users`：有多少个活着的任务在用它。`exit()`、线程退出和 vfork 子进程的 `exec()` 调 `vmspace_leave()`，减到 0 才 `exit_mmap()`。

分成两个是因为 `exit_mmap()` 要回写脏页、可能睡眠，不能放进只持自旋锁的 `freeproc()`；而如果只在 `exit()` 里无条件 `exit_mmap()`，vfork 子进程退出时就会把父进程的映射一起拆掉——`mmaptest` 的 vfork 测试正是检查这一点。

测试。 `user/clonetest.c` 覆盖 24 项：fork 与线程的旧接口、`CLONE_FILES` 下子进程 open/close 对父进程可见、不带它时互不影响、`CLONE_FS` 下 `chdir()` 共享、`CLONE_VM` 的写入共享、`CLONE_VFORK` 父进程等待子进程退出或 exec、TID 回写，以及非法标志组合被拒绝。

5.6 上下文切换与调度器

5.6.1 swtch()

```
swtch:                          // void swtch(struct context *old, struct context *new)
        mov x9, x0
        mov x10, sp
        stp x10, x18, [x9, #16 * 0]   // 保存 sp 和 x18..x30
        stp x19, x20, [x9, #16 * 1]
        ...
        stp x29, x30, [x9, #16 * 6]
        mov x9, x1
        ldp x10, x18, [x9, #16 * 0]   // 恢复新的 sp 和 x18..x30
```

```
    ...
    ldp x29, x30, [x9, #16 * 6]
    mov sp, x10
    ret                          // 跳到新上下文的 x30
```

ret 跳到恢复出来的 x30：对于一个曾经让出过 CPU 的进程，那是它当年调用 swtch() 的下一条指令（在 sched() 里）；对于新进程，就是 forkret。

5.6.2 context 中保存的值

allocproc() 设的 x30 = forkret、sp = trapframe 只是第一次调度时用的初值。进程每次调用 sched() 让出 CPU，swtch 都会把当时真实的 sp 和 x30 写进 context，覆盖掉初值。下面是 sched() 的反汇编（地址取自当前编译出的 kernel.asm，重新编译后会变化）：

```
ffffff800008ed18:  add  x0, x21, #0xb0       // x0 = &p->context
ffffff800008ed20:  add  x1, x1, #0x8         // x1 = &mycpu()->context
ffffff800008ed24:  bl   swtch                // x30 <- 下一条指令的地址
ffffff800008ed28:  mrs  x0, mpidr_el1        // mycpu()->intena = intena;
ffffff800008ed2c:  and  x0, x0, #0xff
ffffff800008ed30:  madd x0, x0, x20, x19
ffffff800008ed34:  str  w22, [x0, #124]
ffffff800008ed38:  ldp  x19, x20, [sp, #16]
ffffff800008ed3c:  ldp  x21, x22, [sp, #32]
ffffff800008ed40:  ldp  x29, x30, [sp], #48  // 弹出 sched 自己的栈帧
ffffff800008ed44:  ret                       // 回到 yield / sleep / exit
```

x30. bl swtch 把下一条指令的地址 0xffffff800008ed28 放进 x30；swtch 是叶子函数，不压栈，直接把它存进 p->context.x30。因为 sched() 是让出 CPU 的唯一入口，所有让出过 CPU 的进程的 context.x30 都是这同一个地址，无论它是被抢占、在睡眠还是正在退出。调度器自己的 c->context.x30 则是 scheduler() 中 bl swtch 的下一条，紧接着执行 switchkvm()。两者构成一对往返。

sp. context.sp 是调用 swtch 那一刻的 sp，也就是 sched() 栈帧的底部。它不再紧挨着 trapframe，深度取决于进程是从哪条路径进入 sched() 的。以用户态被定时器抢占为例，调用链是 el0irq → userirq → yield → sched → swtch，各函数序言的 stp x29, x30, [sp, #-N]! 给出了栈帧大小：

```
p->kstack+4096 +-------------------------+
               | trapframe       272 B   |  <- el0irq 的 usavereg
               +-------------------------+  <- p->trapframe
               | userirq 栈帧     32 B   |
               | yield 栈帧       48 B   |
               | sched 栈帧       48 B   |   (含 x19..x22)
               +-------------------------+  <- context.sp = 栈顶 - 400
               |         (空)            |
p->kstack      +-------------------------+
```

在系统调用里睡眠时（usertrap → syscall → sys_wait → wait → sleep → sched），context.sp 更深；内核态被中断抢占时，栈上还多一份 el1irq 压的 272 字节内核现场。

恢复。 context.x30 只负责把执行流带回 sched() 这一层，再往上靠的是栈上各层函数自己保存的 x29/x30。以上图为例：

1. 调度器 swtch(&c->context, &p->context)：sp 设为“栈顶 – 400”，ret 到 0x…ed28；
2. sched 写回 intena，ldp x29, x30 弹出栈帧，ret 回到 yield；
3. yield 释放 p->lock，返回 userirq；
4. userirq 关中断后返回到 el0irq 中 bl userirq 的下一条，即 trapret；
5. urestorereg 从 trapframe 恢复用户寄存器，sp 回到栈顶，eret。

新进程的栈上没有这些可以逐层返回的调用帧，所以 allocproc() 伪造了一个“刚从 swtch 返回”的 context：用 forkret 代替 0x…ed28，用 trapframe 下沿代替“栈顶 – 400”。forkret 再直接调用 usertrapret(tf)，一步跳到第 5 步。两条路最终都汇合到 urestorereg；eret，用的是同一个 trapframe。

5.6.3 调度循环

每个 CPU 都有一个调度器线程，使用启动时的 stack0 栈：

```
void scheduler(void) {
  struct cpu *c = mycpu();
  int id = cpuid();
  c->proc = 0;  c->online = 1;
  for(;;){
    intr_on();                                  // 让设备有机会中断
    if((p = pick_next(c, id)) == 0) continue;   // 选优先级最高的可运行进程
    acquire(&p->lock);
    if(p->state == RUNNABLE && (p->cpumask & (1U << id))){
      p->state = RUNNING;  p->last_cpu = id;
      c->proc = p;  c->cur_prio = effective_prio(p, c);
      switchuvm(p);                             // TTBR0 = 进程页表
      swtch(&c->context, &p->context);          // 运行进程，直到它 sched()
      switchkvm();                              // TTBR0 = 0
      c->proc = 0;  c->cur_prio = PRIO_IDLE;
    }
    release(&p->lock);
  }
}
```

进程让出 CPU 的唯一入口是 sched()：调用者必须持有且只持有 p->lock，并已把状态改成非 RUNNING。锁在两次 swtch() 之间“传递”——进程持锁切到调度器，调度器在 swtch() 返回后释放；反方向同理，新进程在 forkret() 或 sched() 返回后释放。这保证了“设置状态”和“切换上下文”之间不会有别的 CPU 插进来运行同一个进程。

5.6.4 把一次调度切换打印出来

为了同时观察 proc、context 和 trapframe，构建系统提供了独立于 VM_TRACE 的开关：

```
make PROC_SCHED_TRACE=1 PROC_SCHED_TRACE_LIMIT=256 PROC_SCHED_TRACE_PID=-1
```

PID 为 0 表示所有任务，正数表示精确 PID；−1 表示 initproc 及其经 fork/clone 创建的全部用户后代，并排除 kthread_fn != 0 的 workqueue 和驱动内核线程。本工程在 userinit() 前已创建多个内核线程，所以不能假定 initproc 是 pid 1。事件上限默认为 256，设成 0 会无限打印。该跟踪不遍历页表，因而比 VM_TRACE 适

合观察频繁的上下文切换。日志只在当前 CPU 选中的可见用户任务确实改变时产生：

```
scheduler:first-dispatch 第一个用户任务即将装入 CPU
  ...用户任务 A 换出，context 已由 swtch 保存...
scheduler:switch-out     输出 A 保存后的现场
scheduler:switch-in      输出即将装入 CPU 的用户任务 B
```

如果 A 在 sleep/yield 后仍被选中，便不输出；A -> kworker -> A 也不输出，而 A -> kworker -> B 会折叠成一组 A -> B。userinit:runnable 仍单独记录 initproc 刚构造完的状态（它不一定是 pid 1）。第一次调度时 context.sp 等于 trapframe 下沿、context.x30=forkret；经历一次 sleep/yield 后，二者被实时的 sched 栈指针和 swtch 返回地址覆盖。

输出中的指针必须分层理解：cpu->proc 指向动态分配的 struct proc；p->context 是嵌在 proc 内的内核现场，只有其中的 sp 指向内核栈；p->trapframe 固定指向内核栈顶以下 272 字节；trapframe.sp 却是用户地址空间的 SP_EL0。context 的 x29/x30 是内核帧指针/返回地址，trapframe 的 x29/x30 是用户帧指针/链接寄存器，elr 是用户返回 PC，spsr 决定 eret 后的异常级别和 PSTATE。scheduler:switch-out 是在下一用户任务被选中时延迟打印上一任务已经保存的 context；scheduler:switch-in 是即将装载的新现场。两者都在调度器栈上打印，所以日志中的 current-sp 不属于两个进程的内核栈。

这次跟踪还暴露了一个初始化遗漏：kalloc() 用 0x05 污染空闲页，旧的 allocproc() 在内核栈顶划出 trapframe 后没有清零；userinit() 又只设置 elr/spsr/sp，于是其余用户寄存器第一次进入 EL0 时会是 0x050505…。现在划出 trapframe 后立即清零 272 字节，再由 userinit() 或 do_clone() 写入有效现场，既避免泄露内核栈残留，也让初始现场具有确定值。

5.6.5 调度对进程是透明的

调度的目标是让多个进程分享 CPU 时间，而每个进程都以为自己独占 CPU。能做到这一点，靠的正是上面两份现场：被中断的那一刻，用户寄存器全部存在 trapframe 里；让出 CPU 的那一刻，内核寄存器存在 context 里。从进程自己的角度看，它只是调用了一次 swtch()，过了一段（也许很长的）时间后函数返回了，期间别的进程运行过这件事它察觉不到。也正因为切换只发生在一次函数调用之内，按调用约定可以被调用者破坏的 x0 ~ x17 不需要保存。

5.6.6 谁触发调度

进程让出 CPU 有两种情形：

- **主动让出**：进程在内核中没事可做了，例如 sleep() 等待磁盘、管道或网络，wait() 等待子进程，exit() 结束自己。它们把状态改成 SLEEPING 或 ZOMBIE，然后调用 sched()；
- **被动抢占**：每次 10 ms 定时器中断，sched_tick() 给当前进程的时间片减一，减到 0 就设置本 CPU 的 need_resched；另一个 CPU 唤醒了更高优先级的进程时，会通过核间中断（IPI）设置这个标志。中断返回前，userirq()/kernelirq() 检查 resched_pending()，若为真就调用 yield()，把自己放回运行队列再 sched()。

```
void sched_tick(void)                    // 每个 CPU 每 10 ms 调用一次
{
  struct cpu *c = mycpu();
  struct proc *p = c->proc;
  if(p){
    acquire(&p->lock);
    p->run_ticks++;
    if(p->policy != SCHED_FIFO && --p->slice <= 0)
```

```
      c->need_resched = 1;              // 时间片用完：下次中断返回时让出
    release(&p->lock);
  }
  ...                                   // 实时任务节流计数
}
```

5.6.7 关中断与禁止抢占

教学内核里常见一个计数器 `preempt_count`：它不为 0 时，定时器中断不会触发重新调度。xv6 没有单独的计数器，但效果相同：调度相关的代码总是持有 `p->lock`，而持有任何自旋锁时本 CPU 的中断都是关着的（`push_off`），也就不可能被定时器抢占。`sched()` 开头的检查把这个约定写成了断言：必须持有 `p->lock`、只持这一把锁、状态已不是 RUNNING、中断是关着的。

“关中断”与“禁止抢占”并不总是一回事。调度器循环在每一轮开头都要 `intr_on()`：如果此时没有任何可运行的进程，CPU 就在循环里空转，而让某个进程重新变成 RUNNABLE 的，往往正是一个设备中断（磁盘读完、网络包到达）。如果调度器关着中断空转，就永远等不到这个中断。早期 Linux 的调度函数也有同样的要求：它在运行期间禁止嵌套调度，但必须允许中断发生。

5.6.8 如何选择下一个进程

最早的 Linux（0.01 版）用一个很巧妙的计数器算法：每个任务有优先级 `priority` 和剩余时间 `counter`，每个定时器节拍当前任务的 `counter` 减一；调度时选 `counter` 最大的可运行任务；若所有可运行任务的 `counter` 都为 0，就给所有任务（包括在睡眠的）执行 $\text{counter} \leftarrow \text{counter}/2 + \text{priority}$。睡得越久的任务累积得越多，但不会超过 $2 \times \text{priority}$（这是迭代的不动点），所以交互式任务醒来后能优先得到 CPU。

xv6-aarch64 的 `pick_next()` 把“剩余时间”和“谁先运行”分开：时间片只决定什么时候让出，选择则按有效优先级，同优先级按入队先后（`rq_seq`）：

```
static struct proc* pick_next(struct cpu *c, int id)
{
  struct proc *best = 0;
  int best_prio = 0;
  uint64 best_seq = 0;

  for(struct proc *p = proc_head; p; p = p->next){
    acquire(&p->lock);
    if(p->state == RUNNABLE && (p->cpumask & (1U << id))){
      int prio = effective_prio(p, c);
      if(best == 0 || prio > best_prio ||
         (prio == best_prio && p->rq_seq < best_seq)){
        best = p;
        best_prio = prio;
        best_seq = p->rq_seq;
      }
    }
    release(&p->lock);
  }
  return best;
}
```

时间片用完的进程在 `yield()` 中重新入队、拿到更大的 `rq_seq`，于是排到同优先级的末尾——这就是轮转（round robin）。完整的优先级规则见第 5.9 节。

5.6.9 跟踪一次时间片轮转

调度代码里有大量异步交互，单看函数很难想清楚整个系统的状态如何随时间变化。下面跟踪进程 A 在用户态运行时被定时器抢占、换成进程 B、之后又换回 A 的全过程，注意每一步发生在哪个栈上：

1. A 在 EL0 运行，使用自己的用户栈（`SP_EL0`）；
2. 定时器中断到来。硬件切到 `SP_EL1`，此时它指向 A 内核栈的顶端；`el0irq` 的 `usavereg` 把 A 的全部用户寄存器存进 A 的 trapframe；
3. `userirq()` → `devintr()` → `sched_tick()`：A 的时间片用完，设置 `need_resched`；
4. `userirq()` 发现 `resched_pending()`，调用 `yield()`：拿 `A->lock`，状态改为 RUNNABLE，`sched()` → `swtch(&A->context, &cpu->context)`。A 的内核寄存器存进 `A->context`，`sp` 换成本 CPU 的调度器栈；
5. 调度器从上次的 `swtch()` 之后继续：`switchkvm()`，释放 `A->lock`，下一轮 `pick_next()` 选中 B；
6. 调度器拿 `B->lock`，状态改为 RUNNING，`switchuvm(B)` 把 B 的页表装进 `TTBR0_EL1`，`swtch(&cpu->context, &B->context)`；
7. B 从它自己上次的 `swtch()` 之后继续——如果 B 当初也是被定时器抢占的，那就是它的 `sched()` 返回到 `yield()`，释放 `B->lock`，再一路返回到 `userirq()`、`el0irq`，`urestorereg` 从 B 的 trapframe 恢复寄存器，`eret` 回到 B 的用户态；如果 B 是新进程，则 `ret` 进入 `forkret`；
8. 若干时间片之后，B 以同样的方式让出，调度器再次选中 A，`swtch()` 恢复 `A->context`：A 回到第 4 步的 `sched()` 里面，沿 `yield()` → `userirq()` → `el0irq` 返回，`eret` 之后 A 从被打断的那条指令继续执行。

这一过程中，A 在内核里的调用链 `el0irq` → `userirq` → `yield` → `sched` → `swtch` 一直“冻结”在 A 自己的内核栈上：

```
A 的内核栈（高地址在下）              调度器栈（stack0）
+-----------------------------+    +---------------------------+
| swtch/sched/yield 的栈帧    |    | scheduler() 的栈帧        |
| userirq 的栈帧              |    |  （每个 CPU 一个）        |
+-----------------------------+    +---------------------------+
| trapframe：A 的用户寄存器   |
+-----------------------------+ <- p->kstack + PGSIZE
```

这就是“以 A 的身份进入中断、以 B 的身份离开中断”之所以合法的原因：每个进程的中断现场（trapframe 中的 `elr`、`spsr`）和内核调用链都保存在它自己的内核栈上，`eret` 使用的永远是当前进程自己的那一份。与把任务结构放在栈页底部的教学内核相比，xv6 的每个进程在调度器之外还多了一个独立的调度器上下文，切换总是“进程 → 调度器 → 进程”两步，而不是进程之间直接切换；代价是多一次 `swtch()`，好处是调度器有自己干净的栈，进程退出时也不必在自己的栈上释放自己。

5.6.10 实例：log8 中的调度、栈与 context

上一小节讲的是一次轮转的原理。下面用一次真实运行把它对上号：开机后 init 拉起 wifi，再拉起 login，登录进 sh，然后运行 `forktest`。默认的 `PROC_SCHED_TRACE_PID=-1` 看不到内核线程，要把每一次调度都记下来，需要再加 `PROC_SCHED_TRACE_ALL=1`。在这个模式下，内核线程和用户进程的换入、换出都会打印，事件上限默认为 0，即不设上限。`make_runnable()` 还会多打一行 `sched-wake`，说明是谁、从哪个函数唤醒了睡眠的进程：

```
make PROC_SCHED_TRACE=1 PROC_SCHED_TRACE_ALL=1 qemu 2>&1 | tee log8
python3 tools/sched_bt_sym.py kernel/kernel.sym < log8 > log8.sym
```

```
sched-wake[46] cpu=0 wake pid=16 name=login chan=... <cons+0x98> by pid=-1 name=-
               caller.lr=... <wakeup+0x60> from.lr=... <consoleintr+0xbc>
```

`sched-trace[n]` 和 `sched-wake[n]` 共用一个全局编号，而且都在同一把打印锁之内取号，所以编号顺序就是事件发生的顺序。log8 一共 348 条记录，其中约 320 条属于用户进程。图 5.6 只画了用户进程，内核线程的记录被隐藏，所以图中的编号不连续。

时间线. 整个过程可以分成六段：

1. **init 读盘**，[26]–[35]：init(14) 在 `sdsector` 里忙等 SD 卡。时间片一用完，它就被定时器 IRQ `yield` 出去，在 CPU2、CPU3、CPU1 之间漂移。
2. **fork wifi**，[35]–[39]：[36] 时子进程 15 刚被设成 RUNNABLE，空闲的 CPU0 就把它换入了（首次换入，`x30=forkret`）。这时 init 还在 `do_clone` 里。[37] init 被抢占，[38] 它在 CPU2 上把 fork 做完，[39] 进入 `wait` 睡眠。
3. **wifi 退出**，[40]–[44]：[40] wifi 在 `exit` 里唤醒 init，[41] init 在 CPU1 上换入，[42] wifi 才真正换出。init 回收 wifi 之后 fork 出 16。16 复用了 wifi 刚释放的内核栈槽 `...e2000`，在 [43] 首次换入，随后 exec 成 login。[44] init 再次进入 `wait`。
4. **登录**，[45]–[53]：login/sh(16) 在 `consoleread` 里睡在 `&cons.r` 上。[46]、[49]、[52] 是三次键盘中断，都打在空闲的 CPU0 上，由 `consoleintr` 唤醒它；之后分别由 CPU3、CPU1、CPU0 把它换入。
5. **forktest**，[54]–[341]：sh fork 出 17，17 exec 成 forktest；[55] sh 在 `wait` 里睡下，chan 是 sh 自己的 proc。17 连续 fork 出 128 个子进程（pid 18–145）。每个子进程在某个空闲 CPU 上首次换入后立刻 `exit`，并且各占一个新的内核栈槽（VA 每次减 `0x2000`）。17 自己则多次在 `do_clone` 和 `wait` 里被定时器抢占。
6. **回到 shell**，[342]–[347]：17 退出时唤醒 sh。sh 在 CPU0 上换入后，刚从 `sleep` 出来就被 IRQ 抢占，换到 CPU2 继续执行，最后又在 `consoleread` 里睡下。

fork 与 wait 的时序. `user/init.c` 先 fork 出 wifi，然后 `wait(0)`；wait 返回之后，才进入循环去 fork login。日志里能看到三个细节。

- 子进程可能比父进程的 fork 先跑起来：`sched-q#0 fork 14 -> 15` 这一行打印在 [36] 之后。
- 两个进程真正并行的时间只有 [36]–[39] 这一小段。这段时间里 wifi 在 CPU0 上执行，init 先后在 CPU1、CPU2 上把 fork 收尾；[37] 的回溯是 `yield <- el1irq <- do_clone+0x380 <- sys_fork`。
- wakeup 之后并不能马上回收子进程。[40] 的 `wakeup(p->parent)` 在 `exit()` 里执行，这时 wifi 还没被标成 ZOMBIE。init 在 [41] 换入后，`wait()` 里的 `acquire(&np->lock)` 要一直等到 wifi 在 [42] 换出、调度器释放它的 `p->lock` 之后才能拿到。所以 `fork 14 -> 16` 出现在 [42] 之后。

时间片与空闲 CPU. init 每次被换出时，`run_ticks` 依次是 20、30、40、50。SCHED_OTHER、nice 0 时 `timeslice()` 是 $(20-0)/2 = 10$ 个 tick，正好对上：每次都是时间片用完被换出的。`kernelirq()` 允许抢占内核代码，所以 init 即使还在 `sdsector` 或 `do_clone` 里，也照样被换出。`pick_next()` 不考虑 `last_cpu`。[37] 时 CPU1 把 init 换出，init 要等 CPU1 的调度器释放 `p->lock` 之后才能被别的 CPU 选中，结果同样空闲的 CPU2 先抢到了它（[38]）。CPU1 再扫描时已经没有可运行的进程，就在 `scheduler()` 里空转，一直到 [41]。

中断用的是哪块栈. 这取决于中断到来的那一刻 CPU 上在跑什么，见表 5.5。[40] 不属于其中任何一种：它是 wifi 在 `exit` 系统调用里同步调用 `wakeup`，没有中断帧。

wakeup 与中断. 图中橙色闪电标的是“有人调用了 `wakeup`”，红色循环箭头标的是“被中断抢占”，这两件事不在同一个层次上。`wakeup(chan)` 是一个普通函数：它只把睡在 chan 上的进程改成 RUNNABLE，不做任何切换；被唤醒的进程什么时候运行、在哪个 CPU 上运行，由之后的 `scheduler()` 决定。它的调用者可以是进程上

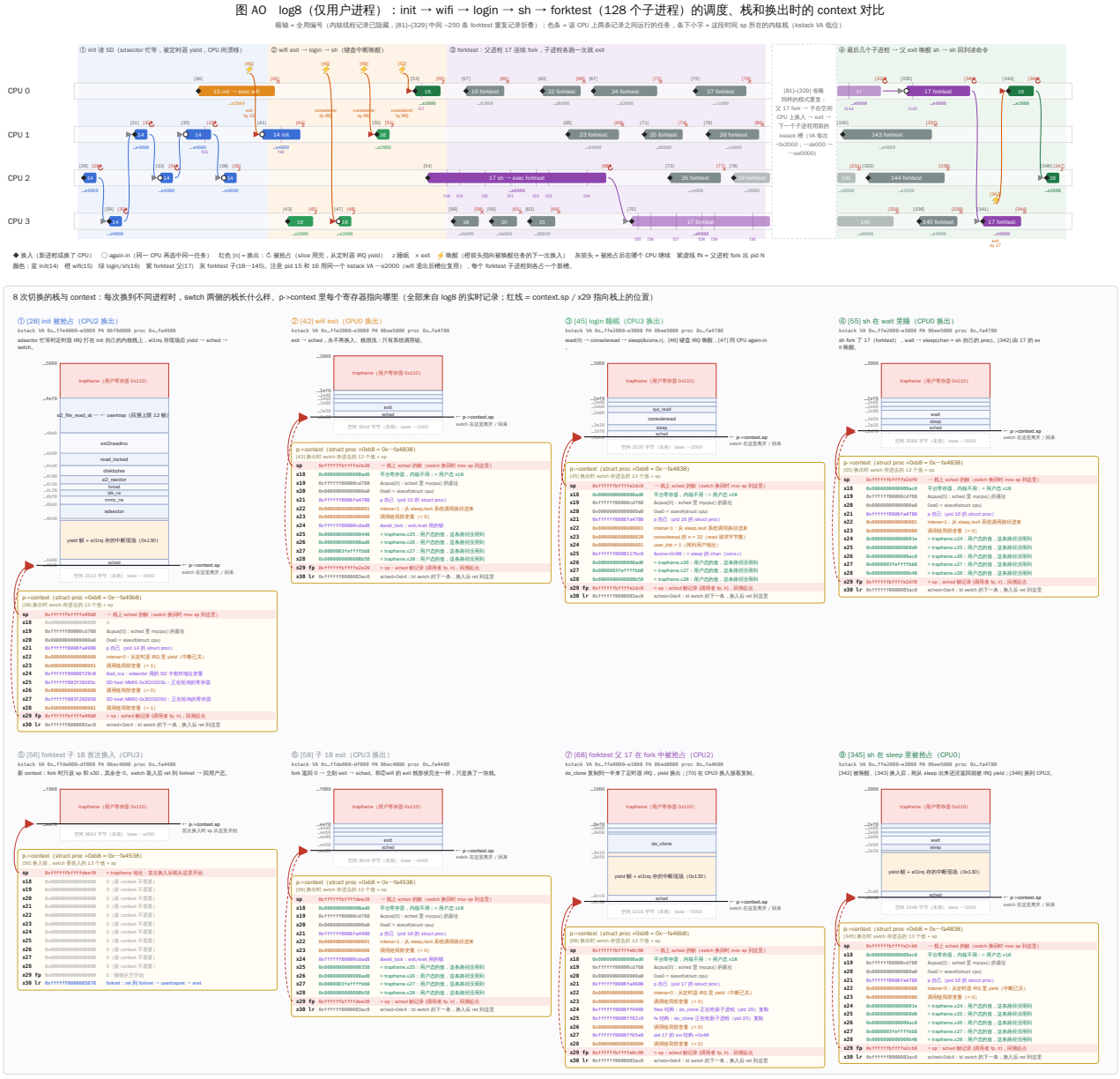


图 5.6: log8：上部是按 CPU 分道的调度时间线（[81]–[329] 的 forktest 重复段已折叠）；中部是 8 次切换的内核栈和 `p->context`，红线表示 `context.sp/x29` 指向栈上的位置；底部是寄存器横向对比

表 5.5: 中断压在哪块栈上

中断时 CPU 在做什么	入口	压在哪块栈上	log8 中的例子
内核态运行某进程	el1irq	该进程内核栈当前 sp 的下方，多出约 0x130 字节的中断现场	[28] [37] [68] [345]
用户态运行某进程	el0irq	切换到 SP_EL1（该进程内核栈顶），先存 trapframe，再经 usertrap 调用 yield	用户态时间片抢占
CPU 空闲	el1irq	该 CPU 的调度器栈 stack0，记录中显示 by pid=-1	[46] [49] [52] 键盘

下文（exit、piperead），也可以是中断上下文（uartintr -> consoleintr）。中断则是硬件事件，会立刻打断本 *CPU* 正在执行的东西；处理函数可能调用 wakeup（键盘中断），也可能根本不调（定时器中断走 sched_tick -> yield）。[46] 把两者串在了一起：键盘中断到来，wakeup 把 login 设成 RUNNABLE，中断返回后 CPU0 继续空转，最后由 CPU3 在 [47] 把 login 换入。

context 的对比。 表 5.6 列出图 5.6 中 8 次切换保存下来的 context。

表 5.6: log8 中 8 次切换的 p->context（... 表示省略了高位）

切换	sp / x29	x21	x22	x23–x28	x30
[28] init 读盘被抢占	...e49d0	proc 14	0	&sd_rca、SD 控制器 MMIO	sched+0xb4
[42] wifi exit	...e2e20	proc 15	1	&wait_lock，其余为用户态值	sched+0xb4
[45] login 睡在 consoleread	...e2dc0	proc 16	1	n=32、user_dst=1、&cons.r	sched+0xb4
[55] sh 在 wait 里睡	...e2df0	proc 16	1	用户态值	sched+0xb4
[56] 子 18 首次换入	...deef0 / 0	0	0	全 0	**forkret**
[58] 子 18 exit	...dee20	proc 18	1	&wait_lock，其余为用户态值	sched+0xb4
[68] 17 在 fork 中被抢占	...e0c90	proc 17	0	新子进程 25 的 files/fs，17 的 vm	sched+0xb4
[345] sh 在 sleep 中被抢占	...e2cb0	proc 16	0	用户态值	sched+0xb4

从这 8 次切换可以看出几点：

1. x19（&cpus[0]）、x20（0xa0）、x30（sched+0xb4）在所有 live context 里都一样。它们是 sched() 自己的寄存器，不属于进程。
2. 能区分进程的是 sp/x29，它们指向各自内核栈上 sched 的帧；还有 x21，它就是进程自己的 struct proc。wifi 和 login/sh 的 sp 都在 ...e2xxx，因为同一个 VA 槽先后给了 15 和 16，对应的物理页并不相同。
3. x22 是 intena，它说明了换出的原因：0 表示从 IRQ 里 yield，1 表示自己走 sleep 或 exit。x23–x28 是换出路径上保存的 callee-saved 寄存器，路径上没用到的就保留用户态的值。
4. 新进程的 context 只设置了 sp=trapframe 和 x30=forkret，其余全为 0。它第一次换入时不经过 sched()，直接 ret 到 forkret，再回到用户态。
5. 被抢占的三列（[28]、[68]、[345]）栈里都有一块 el1irq 中断现场。下次在别的 CPU 上换入时（[29]、[70]、[346]），就从这里 eret 回去，所以换一个 CPU 继续执行并不需要搬动栈。

5.7 exec、exit 与 wait

5.7.1 exec

exec(path, argv) 读取 ELF 文件头，在一张新页表上为每个 PT_LOAD 段分配并装入内存，然后在其上方分配一页保护页和 4 页用户栈，把参数字符串和 argv[] 指针数组压栈：

```
sz_top +----------------+  <- sp 初值
```

```
       | argv 字符串      |  每个都按 16 字节对齐
       | argv[] 指针数组 |  <- x1
       |  ...            |  USTACKPAGES = 4 页
       +-----------------+  stackbase
       | 保护页          |  uvmclear 去掉 PTE_U，溢出即 Data Abort
       +-----------------+
       | text/data/bss   |
0      +-----------------+
```

全部就绪后才替换旧的地址空间——任何一步失败，原进程都不受影响。提交之前，exec() 还要像 Linux 的 unshare_files() 一样给自己一份私有的描述符表，并拆掉旧地址空间的 mmap 区（exit_mmap()）；如果调用者是与父进程共享地址空间的 vfork 子进程，则不动原来的 vmspace，而是把新页表包进 vmspace_wrap() 新建的私有 vmspace，成功后 vfork_done() 唤醒父进程。提交时改写 trapframe：elr 设为 ELF 入口，sp 设为新栈顶，spsr = 0，x1 指向 argv；返回值 argc 由 syscall() 写入 x0。随后 uvmsync_icache()（ARM 的指令缓存与数据缓存不一致，装入代码后必须同步）、switchuvm() 立即换 TTBR0_EL1，最后释放旧页表。eret 之后，用户态从 main(argc, argv) 开始执行。

exec 不建新进程，也不碰 context，只把 trapframe 改写成“一个新程序刚开始”的样子；pid、打开的文件、当前目录都保留，shell 的重定向正是建立在这一点上。

5.7.2 exit 与 wait

exit() 先唤醒可能在等它的 vfork 父进程（vfork_done()），调用 vmspace_leave()——若它是地址空间的最后一个使用者，就拆掉全部 mmap 映射并回写脏页——再放下描述符表和当前目录（files_put()、fs_put()，最后一个共享者才真正关闭文件），然后把子进程过继给 init，把自己设为 ZOMBIE 并唤醒父进程，然后 sched()，再也不会回来。有两样东西 exit() 不能当场释放：内核栈（它正运行在这一页上，直到最后的 swtch 为止）和退出码 xstate（父进程还没来取）。所以要由父进程在自己的内核栈上调用 wait() 回收。

sys_wait() 只是一层薄包装：用 argaddr(0, &p) 从 trapframe 的 x0 取出用户传入的 int *status，再调用 wait(p)。p 是用户虚拟地址，内核不能直接解引用，只能通过用户页表 copyout。

```
int wait(uint64 addr)
{
  acquire(&wait_lock);
  for(;;){
    havekids = 0;
    for(np = proc_head; np; np = np->next){
      if(np->parent == p && !np->is_thread){
        acquire(&np->lock);
        havekids = 1;
        if(np->state == ZOMBIE){                // 找到一个已退出的子进程
          pid = np->pid;
          if(addr != 0) copyout(p->vm->pagetable, addr,
                                (char *)&np->xstate, sizeof(np->xstate));
          freeproc(np);                         // 释放 vmspace 和内核栈
          release(&np->lock); release(&wait_lock);
          return pid;
        }
        release(&np->lock);
      }
```

```
    }
    if(!havekids || p->killed){ release(&wait_lock); return -1; }
    sleep(p, &wait_lock);                          // 有孩子但都还活着
  }
}
```

wait() 只在一种情况下睡眠：有子进程，而且它们都还在运行。已经有 ZOMBIE 时它立即回收并返回 pid；没有子进程或自己被 kill 时立即返回 −1。睡眠的通道是父进程自己的 struct proc 地址，与 exit() 中的 wakeup(p->parent)、reparent() 中的 wakeup(initproc) 对应。父进程从扫描到睡下一直持有 wait_lock，子进程也必须先拿 wait_lock 才能唤醒，所以唤醒不会丢失。被唤醒后回到 for 循环开头重新扫描，因为可能同时有几个子进程退出，也可能是 reparent 引起的唤醒。

freeproc() 依次 vmspace_put()（引用计数归零才释放用户内存）、free_kstack()（trapframe 也在这一页里，一起释放），最后把状态改回 UNUSED。父进程如果始终不调用 wait()，子进程就一直以 ZOMBIE 状态占着内核栈和槽位，这就是“僵尸进程”（user/zombie.c 演示了这一点）。

sh 执行每条命令都是 fork → 子进程 exec → 父进程 wait，所以命令结束才出现下一个提示符。init 在循环里不停地 wait()：返回 login/sh 的 pid 时重新 fork 一个，返回别的 pid 说明回收了一个过继来的孤儿。正因为要收养孤儿，init 绝不能退出。

5.7.3 在系统调用里睡眠不是异常

sleep → sched → swtch 都是普通的 bl/ret 函数调用，全部在 EL1 内完成，不经过向量表，也不碰 ESR_EL1、ELR_EL1、SPSR_EL1。一次 wait() 从头到尾只有一次异常进入和一次异常返回：

```
EL0  svc #0  (x7 = SYS_wait)
       |  <- 唯一一次异常：来源 EL0，同步 -> +0x400 el0sync
EL1  usavereg -> usertrap -> syscall -> sys_wait -> wait
       -> sleep -> sched -> bl swtch
     ... CPU 运行别的进程；子进程 exit -> wakeup(父)，被调度回来 ...
     swtch ret -> sched -> sleep -> wait 返回 pid -> usertrap 返回
     urestorereg ; eret        <- 异常返回
EL0  svc 的下一条指令，x0 = 子进程 pid
```

这次 el0sync 的处理可能持续好几秒，中间夹着许多其他进程的运行，但对硬件来说它一直没有结束。所以“在系统调用里睡眠”属于 el0sync 而不是 el1sync：格子由异常来自哪一级决定，而睡眠本身根本不是异常（详见第 3.2.4 节）。

5.8 睡眠、唤醒与同步原语

5.8.1 sleep/wakeup

xv6 的阻塞机制只有两个函数。sleep(chan, lk) 原子地释放条件锁 lk 并在 chan 上睡眠；wakeup(chan) 把所有睡在 chan 上的进程置为 RUNNABLE。“通道”只是一个地址，通常就是被等待对象的地址。

关键是“原子”二字：sleep() 先拿到 p->lock 才释放 lk，而 wakeup() 必须拿到 p->lock 才能修改进程状态，所以唤醒不可能落在“检查完条件、还没睡着”的间隙里，不会丢失唤醒。被唤醒的进程总要在 while 循环中重新检查条件，因为 wakeup 会唤醒同一通道上的所有进程。

5.8.2 内核 mutex 与 rw semaphore

`sleep/wakeup` 唤醒的是同一通道上的所有进程，用它直接做锁会有惊群，也不保证先来先得。所以内核在它之上实现了 FIFO、可睡眠的 `struct mutex`。内部 spinlock 只保护 owner 和 waiter 链表；锁忙时 waiter 放在当前进程的内核栈中并 `sleep()`，解锁时所有权直接交给队首 waiter，避免惊群和后来者插队。旧 `sleeplock` 已成为该 mutex 的兼容包装。

`struct rw_semaphore` 允许连续的一批 reader 同时进入，但队首 writer 必须等 active reader 归零后独占；writer 排队后，后来的 reader 不能越过它。inode 因此使用 `i_rwsem`：只读路径调用 `ireadlock()`，修改和首次加载调用 `ilock()` 写锁。`struct file.pos_lock` 则保护经 `dup/fork` 共享的文件偏移。

initcall 早于第一个用户进程运行，`myproc()` 可能为空。mutex 和写 rwsem 为此分别保存 `boot_owner/boot_writer`，允许启动 CPU 无竞争地获取；若此时发生竞争则立即报错，绝不在没有 `struct proc` 的上下文调用 `sleep()`。该规则修复了 FAT32 bootfs 读取配置时的 `panic: mutex_lock: no process`。

5.8.3 用户态同步原语

在 `sleep/wakeup` 之上，`kernel/core/sync.c` 提供了 64 个内核同步对象槽，用户通过带代数（generation）的整数句柄访问：

- `mutex`：记录持有者 pid，只有持有者能解锁；
- `semaphore`：计数信号量，`sem_wait` 在计数为 0 时阻塞；
- `event`：手动复位事件，`notify` 唤醒全部等待者并保持触发状态；
- `katomic`：跨进程的原子整数，支持 load/store/fetch_add/compare_exchange。

对象在 `fork()` 之前创建，父子进程就持有相同的句柄，可以跨越独立的地址空间同步。句柄中的代数防止对象销毁、重新分配之后旧句柄误用新对象。`prodcons` 示例用两个信号量（空槽、满槽）加一个互斥锁实现了容量为 4 的有界缓冲区。

5.9 多核软实时调度

原版 xv6 的调度器按数组下标轮询，找到第一个 RUNNABLE 的进程就运行。为了让高优先级任务被唤醒后能尽快抢占，项目加入了与 Linux 语义一致的三种策略（表 5.7）。

表 5.7: 调度策略

策略	优先级	行为
`SCHED_OTHER`	nice −20 ~ 19	分时；时间片 = (20 − nice)/2 个节拍（10 ms ~ 200 ms）
`SCHED_FIFO`	1 ~ 99	一直运行到阻塞、主动让出或被更高优先级抢占，没有时间片
`SCHED_RR`	1 ~ 99	同 FIFO，但同优先级之间每 100 ms 轮转

选择. `pick_next()` 在允许当前 CPU 运行的 RUNNABLE 进程中选有效优先级最高的；同优先级选 `rq_seq` 最小的（最早入队）。实时任务的有效优先级是 100 + rt_priority，普通任务是 0，所以任何实时任务都排在普通任务前面。被更高优先级抢占、时间片还有剩余的实时任务保留原来的 `rq_seq`，回到队首。

抢占. 进程变成 RUNNABLE 时，`resched_hint()` 检查它允许运行的各个 CPU：如果有空闲 CPU，什么都不用做；否则找到当前优先级最低的 CPU，若新进程优先级更高，就设置该 CPU 的 `need_resched`，并通过 ARM-local mailbox 0 向它发送核间中断（IPI）。目标 CPU 在中断返回、系统调用返回或内核态中断返回三处检查 `need_resched`，调用 `yield()`。

节流. 为防止失控的 FIFO 任务卡死整个系统，每个 CPU 每秒最多给实时任务 950 ms；用完后实时任务的有效优先级降到 −1，让 shell 和内核工作线程在剩下的 50 ms 里运行——与 Linux 默认的 `sched_rt_runtime_us` 相同。

用户可以用与 Linux 同名的 `chrt`、`taskset`、`nice`、`renice` 命令设置这些属性，用 `rtlat` 测量唤醒延迟（第 8 章）。

到这里，进程可以被创建、调度、睡眠和回收，但第一个进程在 `forkret()` 里调用的 `prepare_namespace()` 还没有展开：它要读的根文件系统在 SD 卡上，要经过块缓存、日志、inode 和 VFS 几层。下一章从磁盘开始，自底向上讲文件系统。

第六章 文件系统

本章内容

- xv6 文件系统的分层
- 块设备：从 ramdisk 到 SD 卡
- xv6fs 的磁盘格式：superblock、inode、目录与 `mkfs`
- 块缓存、日志与 inode
- 统一 VFS：`super_block`、`inode_ops`、`file_operations` 与一张挂载表
- FAT32 与启动分区：bootfs、BPB、FAT 表、目录项与长文件名
- 根文件系统放在哪里：块设备表、内核命令行 `root=` 与 PARTUUID
- Linux 式命名空间引导：rootfs、devtmpfs、ext2 根、MS_MOVE 与 chroot
- 烧录 SD 卡

6.1 分层结构

xv6 的文件系统是一叠职责单一的层（图 6.1）。本项目保留了日志、块缓存这些中间层，在 inode 层之上加了一层 Linux 式的 VFS（每个文件系统一张 `inode_ops` 函数表），在下面把块设备从 VirtIO 磁盘换成了真正的 SD 卡。

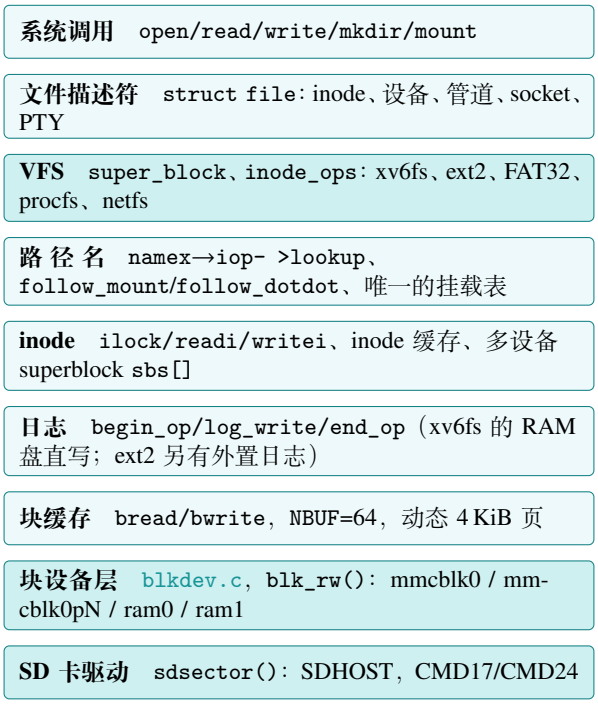


图 6.1: 文件系统分层（深色为本项目新增或替换的层）

本章大体自底向上：先是最底层的块设备（6.2 节），然后是 xv6fs 的磁盘格式和块缓存、日志、inode 三层，再往上是把多种文件系统接在一起的 VFS 和 FAT32 启动分区。有了这些零件，6.7、6.8 两节回到第 5 章留下的问题：第一个进程怎样在 `forkret()` 里找到根分区、挂上 ext2，并把它变成 `/`。最后一节讲怎样把这一切烧到 SD 卡上。

6.2 块设备：从 ramdisk 到 SD 卡

6.2.1 三个阶段

块设备驱动经历了三次替换：

1. **ramdisk**：刚移植到 `raspi3b` 时，用 QEMU 的 loader 把 `fs.img` 装到物理地址 `0x07000000`，`bread/bwrite` 直接 `memmove`。简单，但改动不会写回宿主机；
2. **Arasan SDHCI**（`kernel/drivers/mmc/host/sd.c`）：BCM2837 的 EMMC 控制器，GPIO48 ~ 53 设为 ALT3。QEMU 用 `-drive if=sd` 把 `fs.img` 接到 SD 总线，数据终于可以持久保存；
3. **SDHOST**（`kernel/drivers/mmc/host/sdhost.c`）：后来要驱动板载 Wi-Fi，而 Wi-Fi 芯片只能接在 Arasan 控制器上（GPIO34 ~ 39）。于是外置 SD 卡改由 Broadcom 自己的 SDHOST 控制器（`0x3f202000`）驱动，GPIO48 ~ 53 改为 ALT0。

两种 SD 驱动对上都提供同一个接口：

```
int sdsector(uint32 sector, void *buffer, int write);   // 读写一个 512 字节扇区
```

xv6 的块是 1 KiB，因此一个块对应两个扇区：块 N 是扇区 $2N$ 和 $2N+1$。

6.2.2 SD 卡初始化

无论哪个控制器，SD 卡协议层的初始化顺序都一样（SD 物理层规范 [13]）：

```
400 kHz 识别时钟
CMD0                复位，卡进入 idle
CMD8                检查电压范围，确认 SD 2.0
CMD55 + ACMD41      反复发送直到卡完成上电；OCR.CCS=1 表示 SDHC（按扇区寻址）
CMD2                读 CID
CMD3                取得相对地址 RCA
CMD7                选中卡，进入 transfer 状态
(SDSC) CMD16        设置块长 512
切换到 25 MHz 工作时钟
```

读一个扇区发 CMD17，写一个扇区发 CMD24，数据通过数据寄存器以轮询 PIO 方式搬运 128 个 32 位字。驱动用一把自旋锁串行化对控制器的访问。

6.2.3 命令超时与恢复

真机在 shell 里执行 `ls` 时偶尔 panic “fat32: read FS.IMG”。在驱动里加入事务阶段编号后得到：

```
sd: I/O failure lba=295956 write=0 phase=1 status=17f0207 irq=18000 control1=e0807
```

`phase=1` 表示失败发生在 CMD17 的命令阶段；中断状态 `0x18000` 是 ERROR 加 COMMAND TIMEOUT。更麻烦的是，超时后控制器的命令/数据状态机停在 inhibit 状态，之后的每条命令都会失败。仅仅清中断位不够，还要用 `CONTROL1` 的软件复位位分别复位命令线和数据线，等硬件自动清零这两位。修复后的 `sdsector()` 对同一扇区最多重试三次，每次失败后先 `recover_io()`。这只复位主机控制器内部的状态机，卡本身仍处于 transfer 状态，不需要重新初始化。

6.3 xv6 文件系统格式：superblock、inode 与目录

有了能读写扇区的块设备，下一步是看扇区里装的是什么。这一节看 xv6 自己的文件系统格式（xv6fs）：磁盘上每一块放的是什么，文件和目录怎样表示，一个路径怎样一步步变成 SD 卡上的扇区。xv6fs 现在不再是根文件系统（根是 p3 上的 ext2，6.8.9 节），但它仍然是理解 inode 层的基础：启动时的 rootfs、devtmpfs 两个 RAM 盘用的就是这种格式，SD 卡的 p2 上也还保留着一个 xv6fs 分区，ext2 的外置日志就放在它尾部。下面的数字取自 `mkfs` 为 p2 生成的 32 MiB 镜像 `fs.img`。

6.3.1 superblock 与磁盘布局

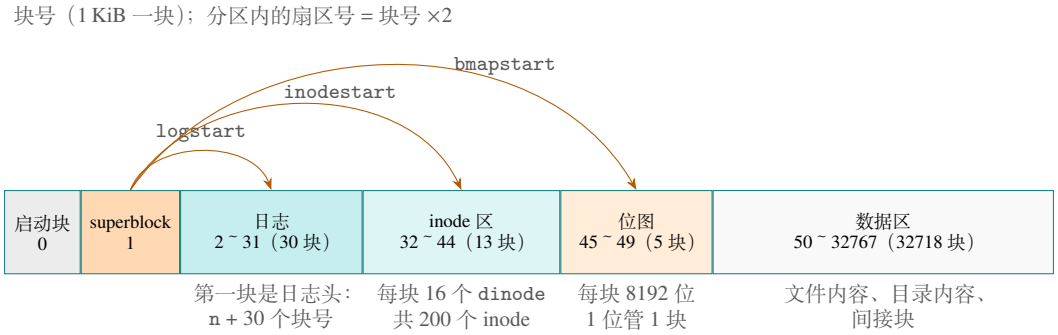


图 6.2: `mkfs` 生成的 32 MiB xv6fs 镜像布局。所有数字都由 `FSSIZE`、`NINODES`、`LOGSIZE` 算出，并写进第 1 块的 superblock

superblock 是第 1 块（第 0 块是从 Unix 继承下来的“启动块”，xv6 不用），内容就是 8 个 32 位整数：

表 6.1: `fs.img` 的 superblock（`struct superblock`，`include/kernel/fs.h`）

字段	值	含义与来源
`magic`	0x10203040	`FSMAGIC`；`fs_readsuper()` 和 `is_xv6fs()` 都检查它
`size`	32768	总块数，`FSSIZE`；32 MiB
`nblocks`	32718	数据块数 = `size` − 元数据块数 50
`ninodes`	200	`mkfs` 里的 `NINODES`
`nlog`	30	`LOGSIZE` = `MAXOPBLOCKS`×3
`logstart`	2	紧跟 superblock
`inodestart`	32	2 + 30；inode 区 200/16 + 1 = 13 块
`bmapstart`	45	32 + 13；位图 32768/8192 + 1 = 5 块

这些数都是 `mkfs` 在宿主机上算出来写进去的（运行 `make fs.img` 时会打印 `nmeta 50 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 5) blocks 32718 total 32768`）。内核启动时 `fsinit()` 读第 1 块、核对魔数、交给日志层，之后所有“某个东西在哪一块”的问题都由 superblock 加两个宏回答：

```
#define IPB           (BSIZE / sizeof(struct dinode))   // 每块 16 个 inode
#define IBLOCK(i, sb) ((i) / IPB + sb.inodestart)       // inode i 所在的块
#define BPB           (BSIZE * 8)                       // 每个位图块管 8192 块
#define BBLOCK(b, sb) ((b) / BPB + sb.bmapstart)        // 块 b 的位所在的位图块
```

日志区。 30 块中第一块是日志头 `struct logheader`：一个计数 `n` 加上 `n` 个“这些块属于哪个目标块号”。其余是被修改的块的副本。提交时先写副本、再写日志头（这一写就是提交点）、再把副本拷回原位、最后把 `n` 清零；`fsinit()` → `initlog()` 在启动时检查日志头，`n` 不为 0 就重做一遍。

位图. 每一位对应一块，1 表示已用。`mkfs` 把前 `freeblock` 块（元数据加上镜像里已经写入的全部文件内容）一次性标成已用；运行时 `balloc()` 从头扫描位图找第一个 0 位，置 1、`log_write`、再把新块清零。早期 `FSSIZE` 只有 1000 块时，光是打包进镜像的程序就用掉 986 块，首次启动创建 `/etc` 配置时 `balloc` 找不到 0 位，panic “out of blocks” ——这就是 `FSSIZE` 改成 32768 的原因。

与原版 xv6 不同的参数.

- `FSSIZE`= 32768：文件系统 32 MiB。原来只有 1000 块，加入网络、SSH 等程序后，打包就用掉了 986 块，首次启动创建 `/etc` 配置时 panic “balloc: out of blocks”；
- `DIRSIZ`= 30：文件名最长 30 字节，目录项从 16 字节变为 32 字节，为了让 TFTP 下载的 `kernel8-xv6_wifi.img` 这样的长文件名能通过普通 `ls` 显示；
- inode 有 12 个直接块和 1 个一级间接块，单个文件最大 12 + 256 = 268 块，即 268 KiB。

笔记 分区再大也不会改变可分配的块数——它由 superblock 里的 `size` 字段决定。修改 `FSSIZE` 或 `DIRSIZ` 之后必须重新生成并整体烧录 `fs.img`，只替换内核会让新内核去解释旧格式的磁盘。

6.3.2 inode：文件的元数据与块映射

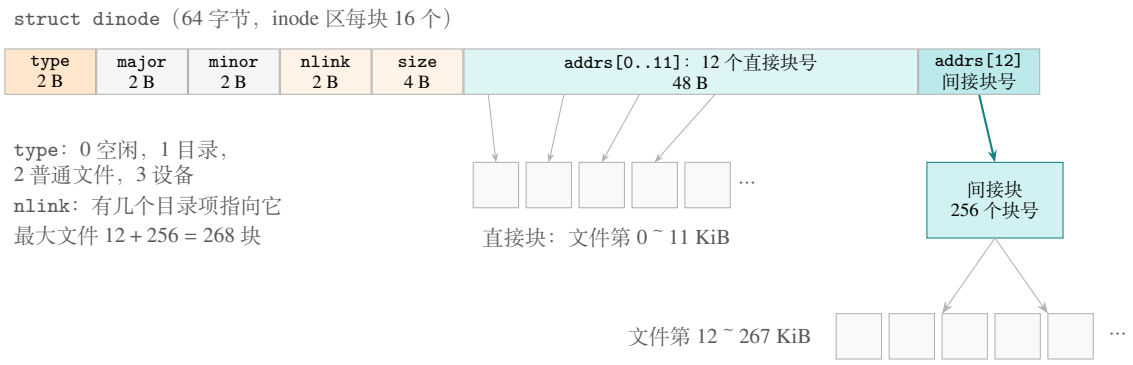


图 6.3: 磁盘上的 inode 和它的块映射。`bmap(ip, bn)` 把文件内第 `bn` 块翻译成磁盘块号：小于 12 直接查 `addrs[bn]`，否则读间接块再查第 `bn-12` 项；遇到 0 就当场 `balloc()` 一块

每个文件、目录和设备节点都由一个 64 字节的磁盘 inode 描述（图 6.3）。它不包含文件名——名字在目录里——只有类型、链接数、大小和数据块的位置。内存里的 `struct inode` 在此之外再加上 `dev`、`inum`、引用计数 `ref`、睡眠锁和 `valid` 标志（是否已从磁盘读入）。`iget()` 只在 inode 缓存里占一个槽、不读盘；`ilock()` 第一次加锁时才 `bread(IBLOCK(...))` 把磁盘 inode 拷进来；修改后 `iupdate()` 写回。

块映射由 `bmap()` 完成：

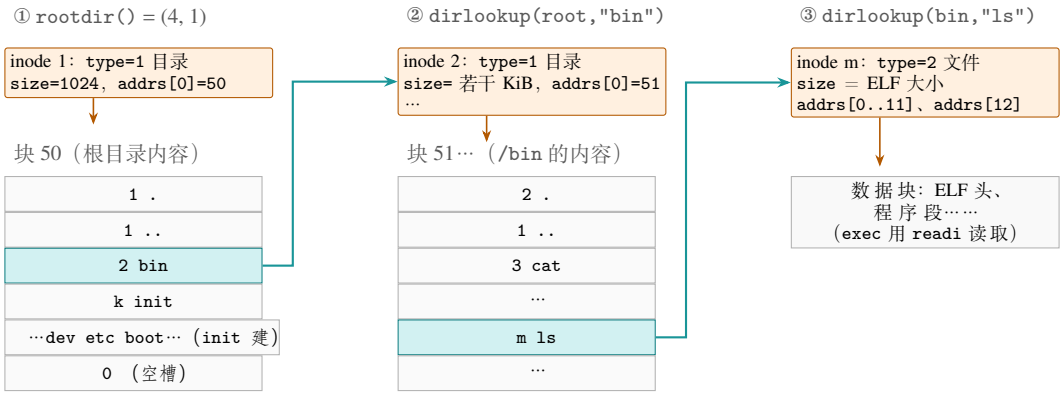
```
static uint bmap(struct inode *ip, uint bn) {
  uint addr, *a;
  struct buf *bp;
  if(bn < NDIRECT){                                  // 前 12 块：直接块
    if((addr = ip->addrs[bn]) == 0)
      ip->addrs[bn] = addr = balloc(ip->dev);
    return addr;
  }
  bn -= NDIRECT;
  if(bn < NINDIRECT){                                // 之后 256 块：经间接块
    if((addr = ip->addrs[NDIRECT]) == 0)
      ip->addrs[NDIRECT] = addr = balloc(ip->dev);
    bp = bread(ip->dev, addr);
```

```
    a = (uint*)bp->data;
    if((addr = a[bn]) == 0){
      a[bn] = addr = balloc(ip->dev);
      log_write(bp);
    }
    brelse(bp);
    return addr;
  }
  panic("bmap: out of range");
}
```

所以单个文件最大 12+256 = 268 块，即 268 KiB。这是 xv6 根文件系统最明显的限制：Wi-Fi 固件、640×480 的 BMP（900 KB）、原始帧（375 KB）都放不下，只能放在 FAT32 的 `/boot`（6.6 节）。

根文件系统里几个特殊的 inode 号：1 号永远是根目录（`ROOTINO`），`namex()` 遇到以 `/` 开头的路径就从它开始；2 号是 `mkfs` 建的 `/bin`；之后是按 `Makefile` 顺序装入的每个用户程序。`init` 是唯一一个在根目录下也有名字的程序（启动代码执行的是 `exec("init")`），所以 `/init` 和 `/bin/init` 是同一个 inode 的两个目录项，`mkfs` 把它的 `nlink` 设为 2。

6.3.3 目录与路径查找



每个目录项 32 字节：`ushort inum` + `char name[30]`，每块 32 项；`inum=0` 表示空槽。
`/init` 与 `/bin/init` 是同一个 inode k 的两个名字，所以它的 `nlink=2`。

图 6.4: `namei("/bin/ls")` 的查找过程。目录本身也是文件，内容是一串目录项；`namex()` 每次用 `skipelem()` 取出一个路径分量，在当前目录的内容里 `dirlookup()`，得到下一个 inode。块号 50、51 是 `mkfs` 的分配顺序决定的，k、m 取决于 `Makefile` 里程序的排列顺序

目录就是 `type=1` 的文件，内容是一串 32 字节的 `struct dirent`：2 字节 inode 号加 30 字节文件名（`DIRSIZ` 从原版的 14 扩到 30，见 6.3 节），一块恰好 32 项，`inum=0` 表示空槽。`mkfs` 把目录大小向上取整到整块。目录操作只有两个基本函数：`dirlookup(dp, name)` 顺序读目录内容、比较名字，命中就 `iget()` 对应的 inode；`dirlink(dp, name, inum)` 先确认不重名，再找第一个空槽（没有就追加在末尾）写入新目录项。

路径查找是 `namex()`，图 6.4 画的是 `namei("/bin/ls")`：

```
if(*path == '/') ip = rootdir();                    // 绝对路径从 fs_root，即 (4, 1) 开始
else             ip = idup(myproc()->cwd);          // 相对路径从当前目录开始
while((path = skipelem(path, name)) != 0){          // 依次取出 "bin"、"ls"
  ilock(ip);
  if(ip->type != T_DIR){ iunlockput(ip); return 0; }
  if(nameiparent && *path == '\0'){ iunlock(ip); return ip; }   // 停在父目录
```

```
  if((next = dirlookup(ip, name, 0)) == 0){ iunlockput(ip); return 0; }
  iunlockput(ip);
  ip = next;
}
```

注意加锁的顺序：每一步只锁当前目录，查到下一级以后先 `iunlockput()` 当前目录再进入下一级，任何时刻最多持有一把 inode 锁，所以不会因为两个进程以相反顺序查找路径而死锁。`nameiparent()` 在最后一个分量之前停下，返回父目录并把最后一个名字留给调用者——`create`、`unlink`、`link` 都需要它。

根目录的内容分两部分：`mkfs` 写入的 `.`、`..`、`bin`、`init`；以及启动时创建的 `dev`（内核在 `prepare_namespace()` 里建出这个空目录，再把 devtmpfs 挂上去，`console`、`ttyS0`、`input/event0`、`video0`、`mmcblk0p3` 等设备节点都在 devtmpfs 里）、由 `init` 首次启动时创建的 `etc`（`fstab` 等配置）、`proc`、`boot`、`mnt/ext2`、`root`、`usr/bin`。其中 `dev`、`proc`、`boot`、`mnt/ext2` 只是空目录，挂载以后会被挂载表“盖住”（6.5 节）。设备节点是 `type=3` 的 inode，不占数据块，只在 `major/minor` 里记下交给哪个驱动。

6.3.4 从路径到 SD 扇区

把这一节和前面几节串起来，在 p2（块设备 4）的 xv6fs 镜像上读 `/bin/ls` 的第 0 字节要经过这些层（块设备表、`blk_rw()` 和分区回调 `part_rw()` 在 6.8.6 节介绍）：

```
namei("/bin/ls")              -> inode m（经 inode 1、inode 2 的目录内容）
readi(ip, ..., off=0, n)      -> bn = off / 1024 = 0
bmap(ip, 0)                   -> 块号 B = ip->addrs[0]
bread(4, B)                   -> 块缓存未命中，调用 blk_rw(b, 0)
blk_rw: mmcblk0p2             -> 分区内扇区 2B，共 2 个扇区
part_rw(p2, 2B, 2)            -> 加上分区起点：卡上扇区 208896 + 2B
mmc_rw(mmcblk0, ...)          -> SDHOST 以 PIO 读 2 × 512 字节
```

一个 1 KiB 的 xv6 块对应两个 512 字节扇区。写入的路径与此相同，只是中间多了日志：`writei()` 修改的块经 `log_write()` 记入日志，`end_op()` 提交时才真正经 `blk_rw()` 写到 SD 卡。

6.3.5 mkfs 怎样生成 fs.img

`mkfs` 是一个在宿主机上运行的普通 C 程序，它和内核共用 `include/kernel/fs.h`，因此磁盘格式只有一份定义：

1. 按 `FSSIZE`、`NINODES`、`LOGSIZE` 算出各区大小，把 32768 块全部写 0，再在第 1 块写入 superblock；
2. `ialloc(T_DIR)` 得到 inode 1，写入 `.` 和 `..`（都指向自己）；再建 inode 2 作为 `/bin`，在根目录里加 `bin` 项；
3. 对命令行上的每个程序（`_ls`、`_cat`…去掉前缀下划线）分配一个 inode、在 `/bin` 里加目录项、用 `iappend()` 把文件内容逐块追加——它和内核的 `bmap()` 一样，前 12 块直接放、之后经间接块；`init` 额外在根目录里加一项；
4. 把两个目录的大小取整到整块；最后 `balloc(freeblock)` 把已经用掉的前 `freeblock` 块在位图里标成 1。

`mkfs` 只按顺序分配块和 inode，不需要日志，也不处理删除；所有整数都用 `xint()`/`xshort()` 写成小端序，与 AArch64 内核一致。

6.3.6 更新 p2 上的镜像

xv6fs 的格式一旦改动（例如 `DIRSIZ` 从 14 改成 30），就必须整体重写 p2。现在的办法只有一个：把 SD 卡插到开发机上，用 `make install-rpi3-rawfs` 或 `dd` 写入新的 `fs.img`（6.9 节）。

早先 p2 还是根文件系统时，内核提供过一个受保护的字符设备 /dev/sdroot，配合用户态 /bin/dd 在系统里把 /boot/fs.img 写回 p2：打开时冻结根文件系统，逐扇区写入并读回校验，结束后必须断电重启。它依赖“设备 1（root）”这层包装，在 /dev/root 改成直接指向分区（6.8.6 节）时与 /bin/dd 一起删掉了，include/uapi/device.h 里只留下一行注释。块设备文件 /dev/mmcblk0p2 在用户态是只读的，运行中的系统无法覆盖自己的根分区。

6.4 块缓存、日志与 inode

上一节是磁盘上的静态格式，这一节是内核里读写它的三层：块缓存把扇区读进内存，日志保证多块修改的原子性，inode 层在它们之上提供文件。这三层基本沿用 xv6 原样，这里只说明本项目中与它们相关的两点；inode 层后来被 VFS 接管成“所有文件系统共用的那一层”，见 6.5 节。

日志. 一次文件系统操作可能修改多个块（例如创建文件要改 inode、目录、位图）。begin_op()/end_op() 之间的所有修改先写进日志区，提交时一次性写回原位置；启动时 fsinit() 检查日志，把已提交但未写完的事务重做一遍。这样即使在写的过程中断电，文件系统也只会处于“操作前”或“操作后”两种状态之一。对 SD 卡来说这很重要：真机经常是直接拔电源关机的。

inode 睡眠锁. inode 操作可能触发 SD 卡读写，耗时很长，不能持有自旋锁，所以每个内存 inode 有一把睡眠锁。项目中出现过一次典型的锁配对错误：在为文件描述符层加入 FD_DEVICE、FD_VNODE、FD_SOCKET 等分支时，fileread() 的 FD_INODE 分支丢了 iunlock()。

```
} else if(f->type == FD_INODE){
  ilock(f->ip);
  if((r = readi(f->ip, 1, addr, f->off, n)) > 0)
    f->off += r;
  iunlock(f->ip);       // 这一行被误删
}
```

现象非常迷惑：启动卡在“exec-login: inode found”之后，调试输出显示 /bin/login 的 inode 被 PID 1 锁着，而 valid=0。原因是 init 读 /etc/fstab 时没有解锁，关闭文件后引用计数归零，这个 inode 缓存槽被 iget() 复用给了 /bin/login——槽换了主人，锁却还“记得”上一任的持有者。

笔记 iput() 不能代替 iunlock()：前者管“有多少引用”，后者管“是否正在独占访问内容”。一个有用的调试断言是：iget() 复用 ref==0 的槽时，若发现锁仍被持有，立即 panic 并打印旧的 dev/inum/owner。

6.5 VFS 中间层

前面几节讲的是一种文件系统怎样工作。系统里实际同时有好几种：RAM 盘上的 xv6fs、SD 卡上的 ext2 和 FAT32、现场生成的 procfs、经网络访问的 netfs。VFS 是让它们共用一套系统调用、一条路径解析和一张挂载表的那一层。

6.5.1 从路径前缀表到统一 VFS

最初的 ext2 驱动只能通过两个专用系统调用 ext2read、ext2readdir 访问，普通的 ls、cat 完全不知道它的存在。第一版 VFS 的办法是在 xv6 原有的路径解析外面再包一层：vfs.c 维护一张“路径前缀挂载表”，sys_open() 先用 findmount() 做最长前缀匹配，命中 /boot、/mnt/ext2 这样的挂载点就把相对路径交给后端的 vnode_ops（参数都是路径），打开结果是一种新的描述符 FD_VNODE；不命中才走原生的 namei()。

这个方案让 ls、cat、mv 很快就能访问其他文件系统，但系统里从此有两套东西并存：两张挂载表（vfs.c 的路径前缀表和 fs.c 的 inode 层挂载表）、两种打开的文件（FD_INODE 与 FD_VNODE）、两条路径解析（findmount() 与 namex()）。open、mkdir、chdir、exec 每个系统调用都要先问一句“这个路径归谁”。到了要把 ext2 当根文件系统时问题就暴露出来了：ext2 的目录不是 xv6 inode，进不了 inode 层挂载表，也就不能 MS_MOVE、不能 chroot（6.8.9 节）。

Linux 的做法是反过来：所有文件系统，包括“原生”的那个，都通过同一组函数表接入；路径解析只有一条，每走一级就调用当前目录所在文件系统的 lookup；挂载表也只有一张。本项目最终也改成了这种结构。对照如下：

表 6.2: xv6 统一 VFS 与 Linux VFS 的对应

xv6	作用	Linux
struct super_block（supers[]）	一个已挂上的文件系统实例：类型、函数表、根 inode 号、只读	struct super_block
struct inode_ops	路径、目录结构和 inode 元数据操作	inode_operations
struct file_operations	打开后的读写、同步、轮询和映射操作	file_operations
struct inode（itable）	内存中的文件对象，带 iop 和默认 i_fop	struct inode
struct fsmount（mtab）	“目录 X 被文件系统 Y 的根盖住”	struct mount
namex() + iop->lookup	逐级解析路径	link_path_walk() + ->lookup
pathfs_iops	按路径工作的小文件系统的通用实现	simple_*、libfs

xv6 没有 dentry 缓存：目录项不单独缓存，lookup 每次都到文件系统里查。当前实现已经把 file_operations 从 inode_ops 分开：目录名字与元数据操作走 inode->iop，一次打开后的数据操作走 file->f_op。暂未引入 Linux 的 address_space_operations 和 page cache；文件系统的 read_at/write_at 完成文件块映射后仍访问现有 bcache。

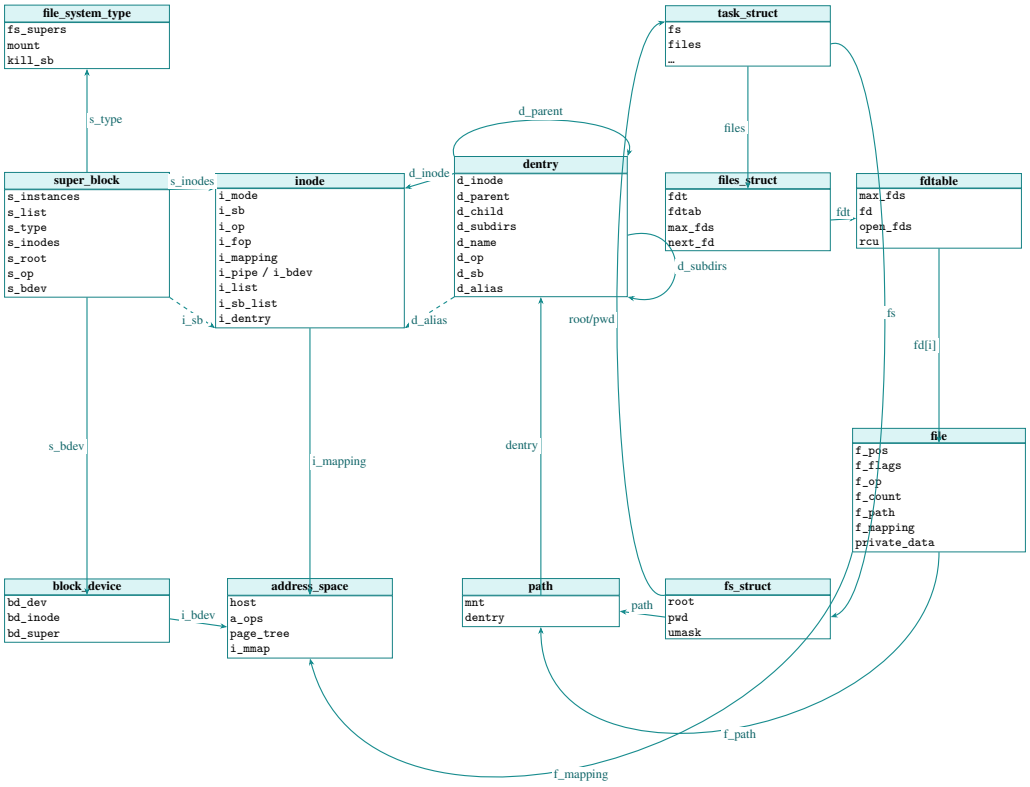


图 6.5: Linux 文件系统各核心数据结构之间的引用关系

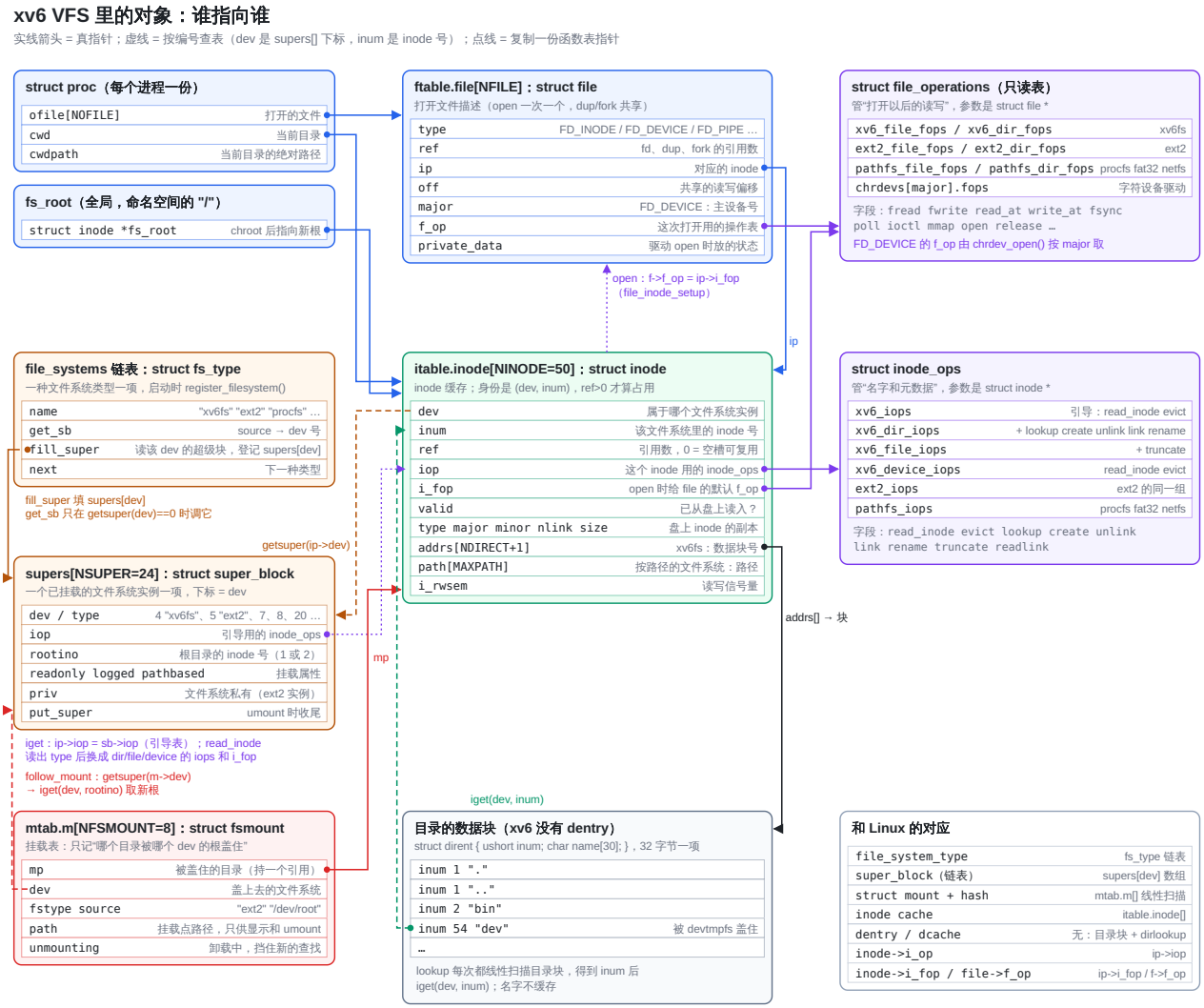


图 6.6: 当前 xv6 VFS 里的对象：谁指向谁。实线是真指针，虚线是按编号查表（dev 是 supers[] 下标，inum 是 inode 号），点线是复制一份函数表指针；右下角是与 Linux 的对应。图中 struct proc 的 ofile[]、cwd、cwdpath 在合入 clone 标志以后分别移进了 files_struct 和 fs_struct（5.5 节），指向关系不变

图 6.5 的左半是文件系统侧：file_system_type 描述一种文件系统，把它的每个实例（super_block）用 s_instances 串成链，s_type 指回类型；一个 super_block 通过 s_inodes 挂起该文件系统内所有内存 inode，块设备上的文件系统还有 s_bdev 指向 block_device。每个 inode 用 i_sb 指回所属超级块、i_mapping 指向 address_space（页缓存）。右半是目录与进程侧：dentry 是路径中的一个目录项，d_inode 指向 inode，d_parent/d_subdirs/d_child 把目录项连成树，d_alias 让同一 inode 的多个硬链接挂在一条链上；path 把挂载点 vfsmount 与 dentry 配对。task_struct 经 fs 指向 fs_struct（保存 root、pwd 两个 path），经 files 指向 files_struct，后者用 fdt 指向 fdtable，fd[i] 数组的每一项指向一个打开的 file；file 再用 f_path、f_mapping 分别指回 path、address_space（f_op 指向的 file_operations 操作表未单独画框）。本项目保留了其中的 superblock、inode cache、每进程 fd 表和打开文件对象，并同样分开 inode_ops 与 file_operations；尚未实现的是 dentry cache、address_space 和 page cache。图 6.6 则是本项目代码里实际的对象：进程经 ofile[]、cwd 指向 ftable 和 itable，fs_root 是全局的 /，supers[] 与 mtab.m[] 按设备号把它们串起来；没有 dentry，目录项就是目录数据块里的 struct dirent。

6.5.2 super_block、inode_ops 与 file_operations

include/kernel/vfs.h 定义 super_block 和 inode_ops，include/kernel/device.h 定义文件系统与设备共用的 file_operations。super_block 按设备号索引，一个设备号就是一个文件系统实例：

```
// kernel/vfs.h
struct super_block {
  int    used;
  int    dev;          // 设备号，即这个实例的编号
  char   type[12];     // "xv6fs" "ext2" "fat32" "procfs" "netfs"
  struct inode_ops *iop;
  uint   rootino;      // 根目录的 inode 号：xv6fs 1，ext2 2，按路径的为 hash("/")
  int    readonly;
  int    logged;       // 写操作要分批走 xv6 日志（只有 xv6fs）
  int    pathbased;    // 对象以路径命名（FAT32、procfs、netfs）
  void   *priv;        // 文件系统私有数据
};
```

块设备上的文件系统用块设备号（ext2 根是它所在的分区，p3 为 5；rootfs 7、devtmpfs 8），没有块设备的文件系统用 vfs.h 里的伪设备号（PROCFS_DEV 20、FATFS_DEV 21、NETFS_DEV 22）。登记由各文件系统自己完成：fs_readsuper() 登记 xv6fs（由 xv6fs_fill_super() 调用），ext2_register_super() 登记 ext2，vfsmount() 登记按路径的文件系统。

inode_ops 只描述路径、目录结构和 inode 元数据操作；file_operations 描述一次打开之后的数据操作。struct inode 同时保存当前 iop 和供 open 复制的默认 i_fop：

```
struct inode_ops {
  int  (*read_inode)(struct inode*);              // ilock() 第一次加锁时填充 inode
  void (*evict)(struct inode*);                   // iput() 放掉最后一个引用时
  struct inode *(*lookup)(struct inode *dp, char *name);   // dp 已加锁
  struct inode *(*create)(struct inode *dp, char *name,
                          short type, short major, short minor);
  int  (*unlink)(struct inode *dp, char *name);
  int  (*link)(struct inode *dp, char *name, struct inode *ip);
  int  (*rename)(struct inode *odp, char *oname, struct inode *ndp, char *nname);
  int  (*truncate)(struct inode*, uint64 size);
  int  (*readlink)(struct inode*, char*, int);
```

```
};

struct file_operations {          // include/kernel/device.h
  int  (*open)(struct file*);
  void (*release)(struct file*);
  int  (*fread)(struct file*, int user_dst, uint64 dst, int n);
  int  (*fwrite)(struct file*, int user_src, uint64 src, int n);
  int  (*read_at)(struct file*, int user_dst, uint64 dst, uint64 off, uint n);
  int  (*write_at)(struct file*, int user_src, uint64 src, uint64 off, uint n);
  int  (*fsync)(struct file*, int dataonly);
  int  (*poll)(struct file*, int events);
  int  (*ioctl)(struct file*, uint64 cmd, uint64 arg);
  uint64 (*mmap)(struct file*, uint64 addr, uint64 len,
                 int prot, int flags, uint64 off);
};

struct inode {                    // kernel/file.h
  uint dev, inum;  int ref;
  struct inode_ops *iop;          // 路径与 inode 元数据操作
  const struct file_operations *i_fop; // open 时复制到 file->f_op
  char path[MAXPATH];             // 只用于按路径的文件系统
  struct rw_semaphore i_rwsem;    // reader 并发，writer 独占
  int valid;
  short type, major, minor, nlink;  uint64 size;
  uint addrs[NDIRECT+1];          // xv6fs 的块地址
  uint32 fsdata[4];               // 其他文件系统的私有数据
};
```

size 扩成 64 位，因为 ext2 和 FAT32 上的文件可以超过 4 GiB；文件名分量的上限 NAMEMAX 是 128，xv6fs 自己仍只存 14 个字符。

itable.lock 仍是短临界区 spinlock，只保护 inode cache 槽查找、引用计数和复用；每个 inode 的内容改由 writer 公平的 i_rwsem 保护。ilock/iunlock 对应写锁，创建、写入、删除、截断和首次从磁盘填充 inode 时使用；ireadlock/ireadunlock 对应读锁，路径查找、普通读取、stat、pread 和 chdir 检查可以并发。struct file 另有 pos_lock mutex，串行化经 dup/fork 共享的 f->off。

FAT32 的全局锁也从 spinlock 改为 FIFO、可睡眠 mutex，因为持锁区会进行 SD I/O。initcall 读取 bootfs 时还没有当前进程，mutex 用 boot_owner = &cpus[cpuid()] 表示启动 CPU；此阶段只允许无竞争获取，若发生竞争则报错而不调用 sleep()。这修复了 rootdev: bootfs ... 之后的 panic: mutex_lock: no process。

xv6 的 inode 缓存原样保留，只在三个地方挂上钩子：

- iget(dev, inum) 在 itable 里找或分配一项，先设 ip->iop = getsuper(dev)->iop、ip->i_fop = NULL。此时 iop 是只含 read_inode/evict 的 bootstrap 表，因为磁盘类型尚未读出；
- ilock() 发现 valid==0 时调用 bootstrap iop->read_inode()。后端读出磁盘 inode 后，按 T_DIR/T_FILE/T_DEVICE 把 iop/i_fop 换成目录、普通文件或设备对应的最终操作表；
- iput() 放掉最后一个引用时调用 iop->evict()，原来“链接数为 0 就截断并释放”的代码成了 xv6_evict()。

ext2 根 inode 的两次读取. ext2_fill_super(5) 为了拒绝损坏的根文件系统，会直接执行 readinode(fs, EXT2_ROOT_INO, &root)，把 inode 2 读进栈上的 struct einode root 并检查它是否为目录。这一步不是

VFS inode 装入：没有 iget()、没有 ilock()，也不会设置某个 struct inode 的 iop/i_fop。验证通过后，ext2_register_super() 只把 bootstrap ext2_iops 登记成 supers[5].iop。

真正的 VFS 根 inode 在 prepare_namespace() 的 kchdir("/root") 中惰性装入：

```
kchdir("/root")
  -> klookup -> namei -> namex
       临时 rootfs 查到挂载点 /root
       -> follow_mount -> fs_dev_root(5) -> iget(5, rootino=2)
            valid=0, iop=sb->iop=&ext2_iops, i_fop=NULL
  -> ilock(ip)
       valid==0 -> ip->iop->read_inode(ip) -> e2_read_inode()
       -> readinode(fs, 2, &einode) -> e2fill()
            目录：iop=&ext2_dir_iops, i_fop=&ext2_dir_fops
            文件：iop=&ext2_file_iops, i_fop=&ext2_file_fops
       valid=1
  -> 确认 T_DIR, p->cwd=ip
```

因此可以概括为：ilock() 借助 inode 从 sb->iop 继承的 bootstrap read_inode，让具体文件系统读取磁盘类型并挂接最终 iop/i_fop。ext2_fill_super() 是第一次读取磁盘根 inode 做验证；kchdir() 中的 ilock() 通常才是第一次填充 itable[] 里的 VFS 根 inode。若它已被更早的代码装入、valid==1，ilock() 只取得锁，不重复读盘。

6.5.3 一次跨文件系统的路径解析

namex() 只改了一行：原来调用 dirlookup()，现在调用 ip->iop->lookup(ip, name)。挂载点的穿越仍由 6.8.8 节的 follow_mount()/follow_dotdot() 完成，判断“是不是文件系统的根”改用 isfsroot()：比较 ip->inum 与该设备 super_block 的 rootino。以 ext2 根上的 /dev/input/event0 和 /proc/1/status 为例（inode 号仅作示意）：

```
namei("/dev/input/event0")
  rootdir()              (dev 5, ino 2)      ext2 根, rootino = 2
  e2_lookup("dev")       (dev 5, ino 76)     follow_mount: 被设备 8 盖住
                      -> (dev 8, ino 1)      fs_dev_root(8): devtmpfs 的根
  xv6_lookup("input")    (dev 8, ino 5)      查 devtmpfs 的目录块
  xv6_lookup("event0")   (dev 8, ino 6)
namei("/proc/1/status")
  e2_lookup("proc")      (dev 5, ino 27)     follow_mount: 被设备 20 盖住
                      -> (dev 20, hash("/"))           procfs 的根
  pfs_lookup("1")        (dev 20, hash("/1"))         path = "/1"
  pfs_lookup("status")   (dev 20, hash("/1/status"))  path = "/1/status"
```

反方向的 .. 也一样：在 (dev 8, ino 1) 上遇到 ..，isfsroot() 为真，follow_dotdot() 先退回挂载点 (dev 5, ino 76)，再由 ext2 查它的 ..。

6.5.4 文件描述符与系统调用

打开任何文件系统上的普通文件，得到的都是 FD_INODE；FD_VNODE 和 vnode 一起删掉了。各系统调用只做与文件系统无关的事，具体工作交给函数表：

- sys_open() 完成只读与 O_TRUNC 检查后调用 file_inode_setup()，把 inode->i_fop 复制到 file->f_op，再执行可选的 f_op->open(file)；字符设备则由 chrdev_open() 根据 major 选择驱动 fops；

- `fileread/filewrite` 调用 `file->f_op->fread/fwrite`。通用 helper 用 `pos_lock` 保护共享偏移，再调用同一张表的 `read_at/write_at`；`pread/pwrite` 直接用显式偏移，不改变 `file->off`；
- xv6fs（`sb->logged`）的写 helper 仍按日志容量分批执行 `begin_op/end_op`，ext2 和按路径文件系统把整段交给自己的 `write_at`。各磁盘后端完成文件块映射后仍通过 `bread/bwrite` 使用 bcache；
- `exec()` 和模块加载没有用户 fd，使用 `file_inode_read_locked()` 进入 `inode->i_fop->read_at`；`O_TRUNC/ftruncate` 暂时仍走 `iop->truncate`，因为改变文件长度属于 inode 元数据修改；
- 目录读取：xv6fs 的目录本来就是 `struct dirent` 数组。其他文件系统通过 `vfs_dir_read()` 把自己的目录项“翻译”成 xv6 的 dirent 流，每 16 字节偏移一项，`ls` 不用改。名字超过 14 字符时截断显示，`open` 时仍可用全名；
- `fs_create()`、`fs_unlink()`、`fs_link()`、`fs_rename()` 先检查文件系统是否可写、是否提供该操作；`link`、`rename` 还要求两端在同一设备上，跨文件系统一律拒绝（与 Linux 的 `EXDEV` 相同）；`unlink`、`rename` 遇到挂载点返回失败（`EBUSY`）。

`fileinit()` 与打开文件对象。 `fileinit()` 只初始化系统级 `ftable.lock` 并调用 `epollinit()`，不读取文件系统，也不设置 inode 操作表。`ftable.file[NFILE]` 位于 BSS；`filealloc()` 寻找 `ref==0` 的槽并初始化一次打开实例，`fdalloc()` 把该指针放进当前进程的 `p->ofile[fd]`。因此 fd 是进程私有编号，并不是全局 file 数组的下标：两个进程分别 `open()` 同一路径，即使都得到 fd 3，也会占用两个不同的 `struct file`，各自保存 `off/flags/private_data`，而它们共同引用同一个 inode。`fork()` 继承或 `dup()` 复制时才由 `filedup()` 增加同一个 file 对象的引用，并共享文件偏移。

Linux 的语义相同，但不使用固定的全局数组：`task->files->fdtable.fd[fd]` 指向从系统级 slab cache 动态分配的 `struct file`。两次独立 `open()` 仍产生两个全局管理的 file 对象，`fork/dup` 才共享同一个对象。

顺带补上了 xv6fs 自己的 `rename`：`xv6_rename()` 在同一个日志事务里先加新名字、再清掉旧目录项，目前只支持普通文件，目标已存在时拒绝。

6.5.5 当前目录与 exec

第一版 VFS 里，最长前缀匹配只认绝对路径，而原生 xv6 的当前目录是一个 inode 指针 `p->cwd`，两者合在一起有两个明显的漏洞：`cd /mnt/ext2` 只会进入原生根文件系统里那个被挂载点遮住的空目录；`exec()` 只调用 `namei()` 和 `readi()`，挂载点上的程序根本无法运行。

当时的修复是让路径在进入系统调用时就变成绝对路径。`struct proc` 增加一个字段 `cwdpath`，记录当前目录的规范化绝对路径（`fork`、`clone` 复制它，`prepare_namespace()` 在 chroot 之后置为 `/`）。`sysfile.c` 的 `abspath()` 把相对路径接在 `cwdpath` 后面，再逐段处理 `.` 和 `..`：

```
cwdpath = /mnt/ext2/d      输入 ../a.txt     ->  /mnt/ext2/a.txt
cwdpath = /bin             输入 ../etc       ->  /etc
cwdpath = /                输入 mnt/ext2/d/e ->  /mnt/ext2/d/e
```

xv6 没有符号链接，所以按文字处理 `..` 与沿目录树向上走的结果完全相同。统一 VFS 之后这层处理仍然保留：`open`、`mkdir`、`unlink`、`rename`、`link`、`mknod`、`mount`、`chdir`、`exec` 一进来就调用 `abspath()`。不同的是，现在所有文件系统上的目录都有 inode，`chdir()` 只剩一条路：`namei()` 找到目录、检查类型、换掉 `p->cwd`、更新 `cwdpath`，不再需要“目录只以路径存在”的特殊情况。

`exec()` 读 ELF 的来源也只剩一种：

```
struct execsrc {                  // kernel/fs/exec.c
  struct inode *ip;               // 任何文件系统上已加锁的 inode
};

static int srcread(struct execsrc *src, uint64 dst, uint off, uint n) {
```

```
  return file_inode_read_locked(src->ip, 0, dst, off, n);
  // 内部构造临时 file，经 inode->i_fop->read_at 读取
}
```

begin_op() → namei() → ilock() 之后检查是普通文件，读 ELF 头、程序头和 loadseg() 都调用 srcread()。这样 /mnt/ext2 和 /boot 上的程序都能直接运行，ext2 当根文件系统时 /init 也是这样被加载的：

```
$ cd /mnt/ext2/d
$ cat /bin/echo > e
$ ./e hi from ext2
hi from ext2
$ cd /
$ mnt/ext2/d/e relative exec
relative exec
```

6.5.6 三类实现

xv6fs（xv6_iops）. xv6_iops 是读取磁盘类型前的 bootstrap 表。xv6_read_inode() 识别目录后挂接 xv6_dir_iops+xv6_dir_fops，识别普通文件后挂接 xv6_file_iops+xv6_file_fops；设备 inode 的 fops 留到 chrdev_open() 按 major 选择。xv6_lookup() 就是 dirlookup()，而文件的 read_at/write_at 最终调用 readi/writei，继续经过 bmap、bread 和 xv6 日志。根文件系统、rootfs、devtmpfs 都使用这一组表。

ext2（ext2_iops）. ext2 本来就有 inode 号，直接对上：bootstrap ext2_iops.read_inode=e2_read_inode；后者读出磁盘 inode，在 e2.open[] 占一个引用，并由 e2fill() 按类型切换为 ext2_dir_iops/ext2_dir_fops 或 ext2_file_iops/ext2_file_fops。e2_lookup() 在目录块里查名字并返回 iget(dev, ino)，文件 fops 的 read_at/write_at/fsync 进入 ext2 的块映射、bcache 和外置日志。e2_evict() 释放 e2.open[] 引用；只要还有进程持有已删除 inode，它仍是孤儿，最后一个 iput() 才真正回收。

按路径的文件系统（pathfs_iops）. FAT32 的磁盘上没有 inode 号，procfs 的对象是现场生成的，netfs 的协议以路径为键。它们共用 vfs.c 里的一套通用实现：bootstrap pathfs_iops 读取 stat 后切换到目录或普通文件的 iops/fops；内存 inode 的 path 字段记下对象在该文件系统内的路径，inode 号取路径的 31 位 FNV 哈希。iget_path() 对这类文件系统除了比较 inode 号还比较路径，哈希冲突也不会认错对象。pfs_lookup() 把目录路径和名字拼起来（处理 . 与 ..），问后端 stat 是否存在；读写、建立、删除、改名都转成后端按路径的调用。三个后端各提供一张小表：

```
static struct pathfs_ops fat32_pops = {      // kernel/fs/vfs.c
  .stat = fat_vstat, .read = fat_vread, .write = fat_vwrite,
  .readdir = fat_vreaddir, .create = fat_vcreate, .truncate = fat_vtruncate,
  .rename = fat_vrename, .openwrite = fat32openwrite, .fsync = fat_vfsync,
};
static struct pathfs_ops proc_pops  = { .stat = proc_vstat,  .read = proc_vread,  .readdir =
    proc_vreaddir };
static struct pathfs_ops netfs_pops = { .stat = netfs_vstat, .read = netfs_vread, .readdir =
    netfs_vreaddir };
```

FAT32 不能原地改写文件，只能追加：pfs_open() 要求以写方式打开的文件长度为 0（新建的，或带 O_TRUNC 已被截断的），然后用 fat32openwrite() 取得写句柄，存在 inode 的 fsdata 里供之后的追加写使用。procfs 和 netfs 只读。

- **procfs**（/proc）：没有磁盘数据，列目录时遍历进程表生成每个 PID 的目录，读 status 时持锁生成快照。另有 /proc/mounts、/proc/cmdline、/proc/meminfo、/proc/iomem、/proc/devices 等诊断文件；
- **ext2**（/mnt/ext2 或 /）：支持 1/2/4 KiB 块，遇到 ext4 extent 等不兼容特性就拒绝挂载，可以和 Linux 交换文件；可读写，带外置事务日志，见第 10 章；
- **FAT32**（/boot）：挂载启动分区，支持根目录的读、创建、截断、顺序写和重命名，支持 ASCII 长文件名。这让系统可以在运行时更新自己的内核：tftp 下载 kernel8-xv6_wifi.img 到 /boot，重启即生效。

6.5.7 由用户态决定挂载

挂载策略仿照嵌入式 Linux：内核只负责探测驱动和挂根，init 读取 /etc/fstab 决定其余的挂载。首次启动时 init 创建：

```
proc                /proc     procfs ro 0 0
sysfs               /sys      sysfs  rw 0 0
bootfs              /boot     fat32  rw 0 0
192.168.0.195:5640  /mnt/net  netfs  ro 0 0
```

ext2 已经是根，/mnt/ext2 那一行在当前 init.c 里被注释掉了，只在以 xv6fs 为根时才需要把 p3 另挂一次。

然后逐行调用 mount(source, target, fstype, flags)。内核里的 vfsmount() 分两步：先按类型登记 super_block（procfs 强制只读；ext2 的 ext2_get_sb() 先试 lookup_bdev(source)，source 不是设备节点时用已打开的 ext2 实例，或扫描 0x83 分区逐个 fill_super，见第 6.8.9 节），再 namei(target) 找到挂载点目录，调用与 prepare_namespace() 相同的 fs_mount() 挂进那一张挂载表。启动日志出现 “vfs: mounted fat32 at /boot read-write” 才说明 FAT32 真的挂上了；挂载点忙、或者同一个文件系统已经挂在别处时，打印 “vfs: cannot mount …(busy or already mounted)”，/boot 就仍是一个空目录。

6.6 FAT32 与启动分区

前面几节讲的是 xv6 自己的文件系统，以及把各种文件系统接到同一棵目录树上的 VFS。SD 卡上还有一个 xv6 无法回避的 FAT32 分区：树莓派的固件只认 FAT32，内核镜像、config.txt、Wi-Fi 固件都必须放在这里。本节先看这个分区里放了什么、谁来读，再看 FAT32 在磁盘上的组织，最后逐个讲 kernel/fs/fat/fat32.c 里的函数。

6.6.1 bootfs 里有什么，谁来读

图 6.7 按读者把启动分区的文件分成三组。

第一组由 GPU 读，在 ARM 执行第一条指令之前。 树莓派上电后先运行的是 VideoCore GPU：片内 ROM 读 bootcode.bin，它再装入 start.elf 和 fixup.dat。start.elf 解析 config.txt：

```
arm_64bit=1                 # 以 AArch64 启动 ARM 核
enable_uart=1               # 打开 Mini UART，固定 core 时钟
uart_2ndstage=1             # 固件自己的启动日志也从串口输出
disable_commandline_tags=1  # 不往内存里写 ATAG/设备树
core_freq=250               # VPU 时钟 250 MHz（Unicam 的下限，串口波特率也依赖它）
core_freq_min=250
kernel=kernel8-xv6_wifi.img # 内核镜像文件名
camera_auto_detect=0        # 不让固件占用摄像头
```

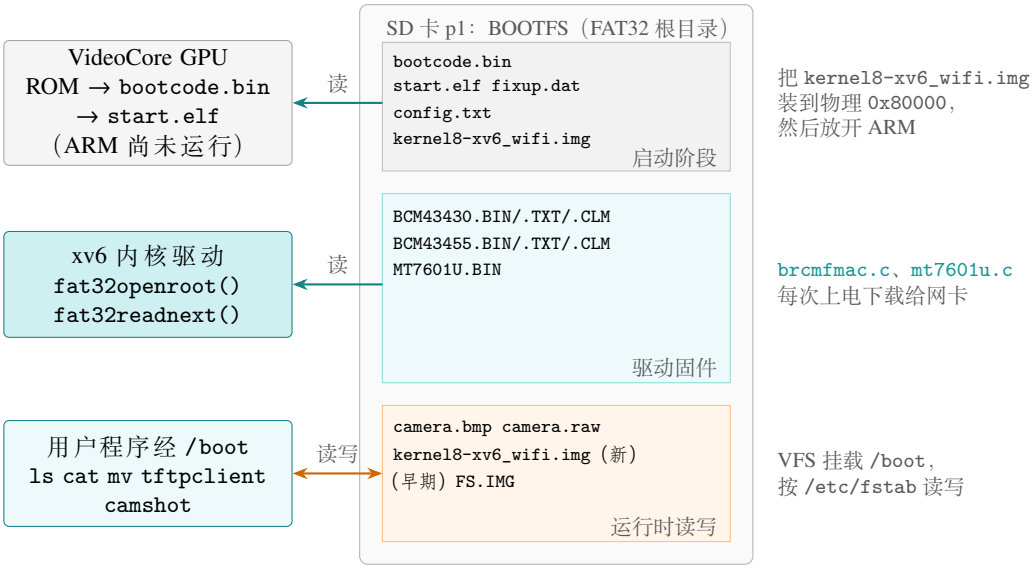


图 6.7: 启动分区里的文件和它们的三类读者。同一个 FAT32 分区，先被 GPU 固件读（ARM 还没开始执行），再被 xv6 内核驱动读，最后作为 /boot 被用户程序读写

然后把 kernel8-xv6_wifi.img 原样拷到物理地址 0x80000，才放开 ARM 核（第 3 章）。所以内核镜像必须是裸二进制而不是 ELF，固件也从不理会 xv6 的根文件系统。

第二组由 xv6 内核驱动读。 两块 Wi-Fi 网卡每次上电都要由主机下载固件：BCM43430/43455 需要 .BIN（R4 的执行代码）、.TXT（NVRAM）和 .CLM（法规数据库），MT7601U 需要 MT7601U.BIN。驱动在 main() 里初始化，这时还没有任何进程，更谈不上挂载点，所以它们直接调用 fat32openroot() 按 8.3 短名在根目录里找文件，再用 fat32readnext() 顺序读出。这也是挂起 bootfs 的 rootdev_init() 必须排在 brcmfmac_driver_init() 之前的原因。

第三组由用户程序经 /boot 读写。 进入用户态以后，init 按 /etc/fstab 把同一个分区挂到 /boot（6.5 节）。于是 ls /boot、cat /boot/config.txt 都走普通的 open/read；tftpclient 把新内核下载成 /boot/kernel8-xv6_wifi.img，重启即生效；camshot 把超过 xv6 单文件上限的照片写成 /boot/camera.bmp，关机后插到电脑上就能看到。

6.6.2 磁盘上的 FAT32

FAT32 的结构可以用一句话概括：*文件内容按簇存放，簇与簇之间的先后关系记在一张表（FAT）里，目录本身也是一个文件。*图 6.8 是它在 SD 卡上的位置和内部划分。

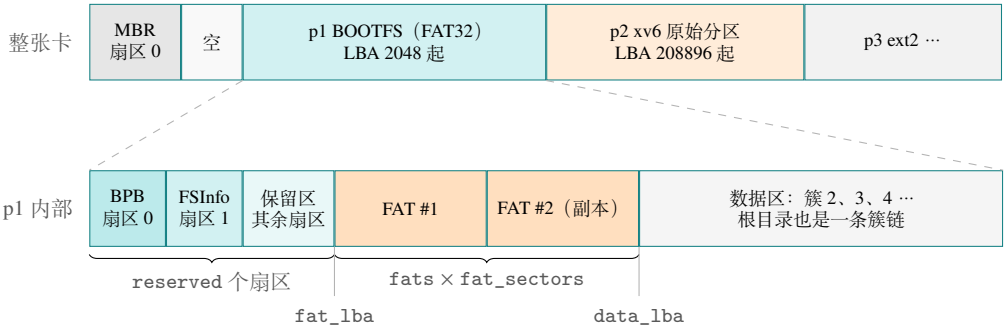


图 6.8: SD 卡与 FAT32 分区的磁盘布局。fat32mount() 从 BPB 读出各区的大小，算出 fat_lba 和 data_lba；簇号从 2 开始编号

MBR 和分区表。 整张卡的第 0 扇区是 MBR：偏移 446 起是 4 个 16 字节的分区表项，最后两个字节是签名 `0x55 0xAA`。每个表项里第 4 字节是分区类型（`0x0B`/`0x0C` 是 FAT32），第 8、12 字节是起始 LBA 和扇区数。读分区表的是块层：SD 驱动 `add_disk()` 之后，`blkdev.c` 的 `msdos_partition()` 解析这四个表项，把每个主分区登记为块设备 `mmcblk0pN`，记下起点、扇区数、类型和 PARTUUID，而不是 FAT32 驱动。`rootdev_init()` 再在这些块设备里挑选：把第一个 FAT32 分区的起点交给 `fat32mount()`，用 `is_xv6fs()` 找 block 1 带 xv6 superblock 魔数的分区做根文件系统。两件事互不依赖，所以根文件系统和启动分区可以各在各的分区里。FAT32 驱动的工作从 `fat32mount()` 拿到分区起点开始。

BPB：分区的第一个扇区。 FAT32 分区的第 0 扇区叫 BPB（BIOS Parameter Block），记录了整个分区的几何参数。`fat32mount()` 用到的字段如表 6.3。

表 6.3: `fat32mount()` 读取的 BPB 字段

偏移	字段	用途
11	每扇区字节数	必须是 512，否则拒绝
13	每簇扇区数 `spc`	必须是 2 的幂；一簇通常 4 KiB 或更大
14	保留扇区数 `reserved`	FAT 表从 `partition_lba + reserved` 开始
16	FAT 份数 `fats`	通常为 2，第二份是备份，写入时要同步
32	总扇区数	与 FAT 大小一起算出总簇数
36	每份 FAT 的扇区数	数据区从 `fat_lba + fats×fat_sectors` 开始
44	根目录首簇	通常为 2；根目录也是一条簇链
48	FSInfo 扇区号	内有“下一个空闲簇”的提示
510	`0x55 0xAA`	签名

有了这些字段，任何簇号都能换算成 SD 卡上的绝对扇区号：

```
fat_lba  = partition_lba + reserved;
data_lba = fat_lba + fats * fat_sectors;
static uint32 cluster_lba(uint32 cluster) {     // 簇号从 2 开始
  return diskmap.data_lba + (cluster - 2) * diskmap.sectors_per_cluster;
}
```

`fat32mount()` 对每个字段都做了合法性检查（2 的幂、不越界、不溢出）——SD 卡上的数据是外部输入，一个损坏的 BPB 不应该让内核算出一个指向别处的 LBA 然后去写它。

6.6.3 FAT 表与簇链

FAT 表是一个巨大的 32 位整数数组，下标就是簇号，第 c 项的值是“簇 c 之后的下一个簇”。只有低 28 位有效，高 4 位保留且写入时要保持原值；0 表示空闲，`0x0FFFFFF8` 及以上表示链尾（EOC）。一个 512 字节的扇区装 128 项，所以查第 c 项只需读一个扇区：

```
static uint32 fat_next(uint32 cluster) {
  uint32 sector = diskmap.fat_lba + cluster / 128;   // 128 = 512 / 4
  uint32 pos    = (cluster % 128) * 4;
  if(sdsector(sector, scratch, 0) < 0) panic("fat32: read FAT");
  return le32(scratch + pos) & 0x0fffffffU;
}
```

`fat_set_next()` 是对应的写操作，它要把同一个扇区在每一份 FAT 里都读改写一遍，保证两份副本一致。图 6.9 下半部分画的就是一条簇链：从目录项里拿到首簇，然后反复 `fat_next()` 直到 EOC。

6.6.4 目录项与长文件名

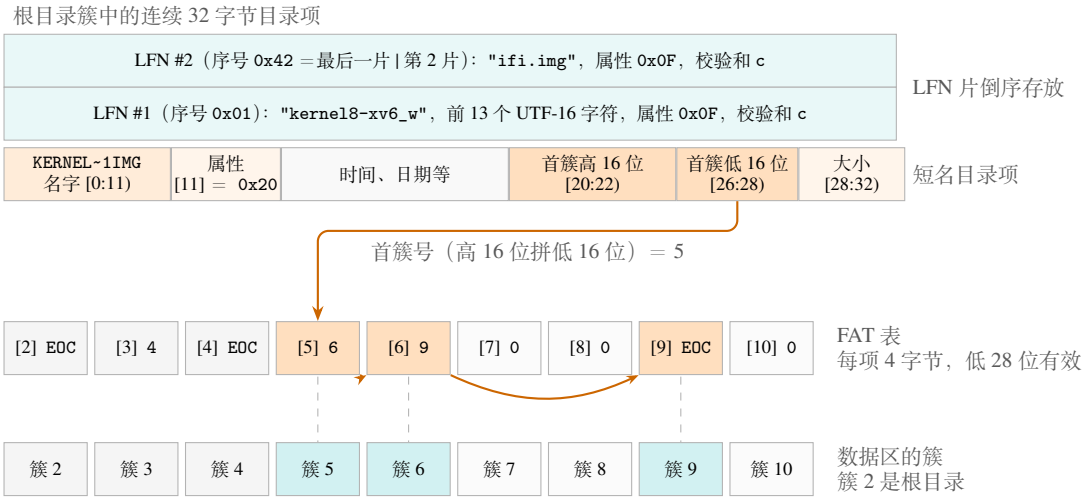


图 6.9: 长文件名、目录项与簇链。文件 `kernel8-xv6_wifi.img` 占两片 LFN 加一个短名目录项；短名项里的首簇号是 5，沿 FAT 表 5→6→9 直到 EOC 就是它的全部数据簇

目录的内容是一串 32 字节的目录项（图 6.9 上半部分）。短名目录项里有 8.3 格式的名字（11 字节，大写、空格填充）、属性字节、首簇号（拆成高低两个 16 位字段，这是 FAT16 向 FAT32 扩展时留下的痕迹）和文件大小。扫描目录时，首字节有两个特殊值：`0x00` 表示后面再没有目录项，`0xE5` 表示这一项已被删除。属性字节的 `0x10` 表示子目录，`0x08` 表示卷标。

长文件名（LFN）是后来加的：属性等于 `0x0F` 的目录项不是文件，而是一片名字，每片装 13 个 UTF-16 字符，倒序排在它所属的短名目录项前面。第一片（物理上最靠前的）序号带 `0x40` 标志，表示“最后一片”；每片都存着短名的校验和，以便发现“LFN 和短名不配套”（例如老系统删了短名但留下了 LFN）。`fat_find_path()` 扫描目录时把遇到的 LFN 片按序号拼进一个缓冲区，遇到短名项时检查三件事——片序号连续到 1、校验和与短名一致、拼出的名字与要找的名字相同（不区分大小写）——满足才算长名匹配；否则再按短名比较。

创建文件时反过来：`fat32createfile()` 先用 `fat_make_short_alias()` 造一个 `KERNEL~1.IMG` 这样的短名（取前 6 个字母数字、加 `~1` ~ `~9` 避免重名），算出需要几片 LFN，用 `fat_reserve_root_slots()` 在根目录里找到足够的连续空位（不够就给根目录分配新簇），按倒序写入 LFN 片和短名项，最后补一个全 0 的目录项作为结束标记。

本项目的 FAT32 驱动只处理根目录：查找、创建、枚举、重命名都在根目录簇链里进行，不支持子目录，LFN 也只支持 ASCII 字符。对启动分区来说这已经足够——上面列出的所有文件都在根目录里。

6.6.5 函数一览与调用关系

`fat32.c` 的函数可以按图 6.10 分成三层：

- **挂载**：`fat32mount(lba, sectors)` 解析 BPB、读 FSInfo 里的空闲簇提示；`fat32_is_boot_sector()` 判断一个扇区是不是 FAT32 引导扇区，`rootdev.c` 用它识别“没有分区表、整张卡就是一个 FAT32”的 superfloppy 格式；`fat32init()` 只初始化 `fat32_lock`。分区表扫描和 xv6 根分区识别都不在这个文件里。
- **对外接口**：只读的 `fat32openroot()`、`fat32openpath()`、`fat32statpath()`、`fat32readdirroot()`、`fat32pread()`、`fat32readerinit()`/`fat32readnext()`；写入的 `fat32createfile()`、`fat32openwrite()`、`fat32writefile()`、`fat32truncatefile()`、`fat32renamefile()`。VFS 的 `fat32_pops` 把它们包装成按路径的操作，再经 `pathfs_iops` 成为 `/boot` 上的 inode 操作（6.5 节）。
- **目录与 FAT**：上一小节的查找和 LFN 函数，以及 `fat_next()`、`fat_set_next()`、`fat_alloc_cluster()`、`fat_free_chain()`、`fat_mark_fsinfo_unknown()`。

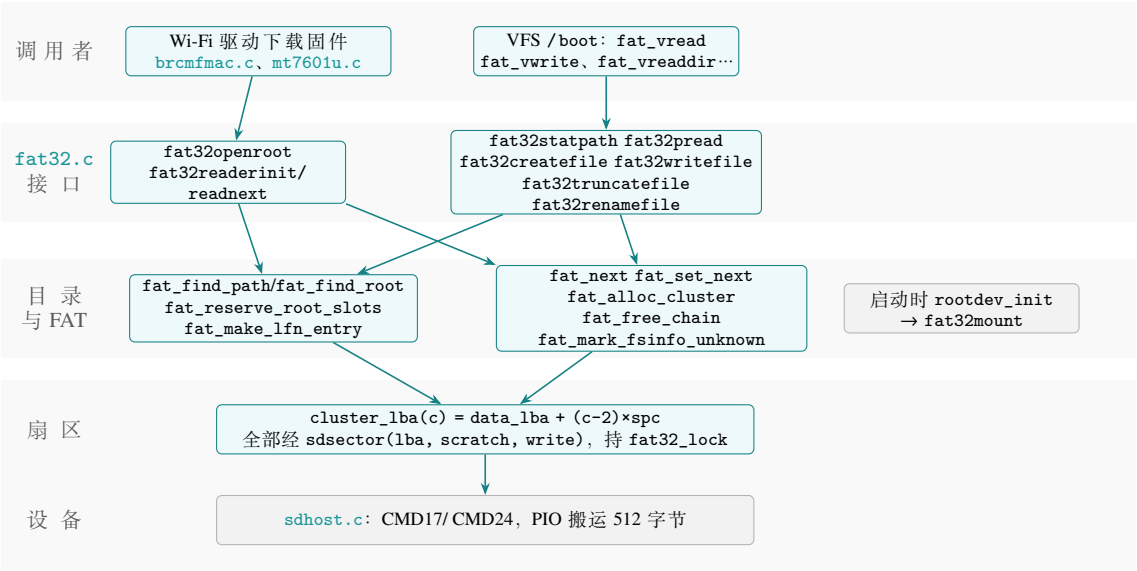


图 6.10: FAT32 驱动的调用关系。两类调用者共用同一组目录与 FAT 操作，最终都落到一个 512 字节的 `scratch` 扇区缓冲和 `sdsector()` 上

两个读接口的区别值得一提。`fat32pread(file, offset, …)` 是无状态的：每次调用都从首簇沿链走 ⌊offset/簇大小⌋ 步，适合 VFS 偶尔的随机读。如果用它按 512 字节分块读一个 600 KiB 的 Wi-Fi 固件，总共要走 $O(n^2)$ 次 `fat_next()`。`fat32reader` 则记住当前簇和簇内偏移，顺序读时每跨一个簇只查一次 FAT——固件下载就用它。

6.6.6 写入：分配、追加与截断

`fat32writefile()` 只支持追加：要求写入位置正好等于文件当前大小。写入过程是：

1. 重新按路径查找目录项，核对首簇和大小与调用者手里的 `fat32_file` 一致，防止文件已被别人改过；
2. 写入跨进新簇时调用 `fat_alloc_cluster()`：从 FSInfo 提示的位置开始找 FAT 值为 0 的项，把它标成 EOC（每份 FAT 都写），把这一簇清零；再用 `fat_set_next()` 把它挂到链尾；
3. 按扇区写数据：整扇区直接写，不满一个扇区的先读出来再改再写回；
4. 最后一次性更新目录项里的首簇和大小。

`fat32truncatefile()`（`O_TRUNC`）的顺序恰好相反：先把目录项的首簇和大小清零并写回，再释放旧簇链。这个顺序的意义在于断电时的后果：先改目录项，最坏情况是旧簇成了没人引用的“丢失簇”，浪费一点空间；如果先释放簇链，断电后目录项会指向已被标为空闲、随时可能分给别人的簇，那才是真正的损坏。分配和释放之后，`fat_mark_fsinfo_unknown()` 把 FSInfo 里的空闲簇计数标成“未知”（`0xFFFFFFFF`），只更新“下一个空闲簇”提示，避免留下一个过时的计数让别的系统误信。

两处性能问题都是真机上暴露出来的，修法相同——缓存当前 FAT 扇区：

- `fat_alloc_cluster()` 最初对每个候选簇调用一次 `fat_next()`，同一个扇区要重复读 128 次，根目录一长就像卡死了一样；现在扫描时只在跨扇区时才读；
- `fat_free_chain()` 最初对每个簇做一次读和两份 FAT 的读改写，截断一张 320×240 的 BMP 要五百多次 SD 事务（第 7 章 7.9.11 小节）；现在在一个扇区内批量清除链项，换扇区时才把整扇区写到每份 FAT。

并发与局限。 所有接口都持同一把自旋锁 `fat32_lock`，共用一个 512 字节的 `scratch` 扇区缓冲，没有块缓存，也没有日志。路径解析、LFN 字符、目录槽这些较大的临时数组也放在一个静态工作区 `fat_work` 里，而不是放在栈上——每个进程的内核栈只有 4 KiB，它们曾经把创建文件的调用链撑爆（第 7 章 7.9.11 小节）；既然有 `fat32_lock` 串行化，一个工作区就够用。这让实现很简单，代价也很清楚：持锁期间 `sdsector()` 以 PIO 方式

等待 SD 卡，一次大文件写会长时间占着这把锁；同一时间只能有一个写者；写入途中断电可能损坏分区。启动分区平时几乎只读，偶尔更新内核或写一张照片，这些代价是可以接受的，但修改它之前最好备份。

6.7 根文件系统放在哪里

有了块设备、文件系统和 VFS，接下来回答启动时的第一个问题：根文件系统在哪个分区上。启动时 SD 驱动的 `sdhost_probe()` 用 `add_disk()` 把整张卡登记为块设备 2（`mmcblk0`），块层随即读第 0 扇区的 MBR，把每个主分区登记为 `mmcblk0pN`（块设备 $2+N$）。`rootdev_init()` 自己不登记任何设备，只在这些块设备里选出根文件系统所在的那一个，也就是 Linux 所说的 ROOT_DEV（表 6.4）。`bio.c` 通过 `blk_rw()` 派发 I/O：分区的回调 `part_rw()` 加上分区起点，再交给整卡的 `mmc_rw()`。

内核命令行与 `root=`. 树莓派上 `cmdline.txt` 本来由 VideoCore 固件 `start.elf` 读取，再写进设备树交给 Linux 内核。xv6 的 `config.txt` 设了 `disable_commandline_tags=1`，固件就不再处理它；`cmdline.c` 在内核挂上 FAT32 启动分区后自己读 `/cmdline.txt`。文件不存在或没有启动分区（QEMU 裸镜像）时，用编译期内置的 `CONFIG_CMDLINE`（默认空，即自动探测）。仓库根目录有两份模板：`cmdline.txt`（`root=/dev/mmcblk0p2 rootfstype=xv6fs`）和 `cmdline_ext2.txt`；`make install-rpi3-ext2root` 把后者装成启动分区上的 `cmdline.txt`，内容是本书采用的 `root=/dev/mmcblk0p3 rootfstype=ext2`。`rootdev_init()` 用 `cmdline_get("root", ...)` 查询目标分区，支持三种格式：`/dev/mmcblk0p3`、`mmcblk0p3` 或 `PARTUUID=SSSSSSSS-03`（MBR 磁盘签名加分区号）；`rootfstype=` 指明根分区上的文件系统类型。

根块设备不是 FAT32 的一部分. `rootdev.c` 不解释任何文件系统格式。它只把 `root=` 解析成一个分区，再把 FAT32 启动分区交给 `fat32.c` 挂起来，供 `/boot` 和固件加载使用。根分区就是块设备本身，例如 `mmcblk0p3` 是块设备 5，没有另外包一层“根设备”。分区里是不是 ext2，要等阶段二挂载时由 `ext2_fill_super()` 读超级块才知道；读写经 `part_rw()` 和 `mmc_rw()`，不调用任何 FAT32 函数。

表 6.4: 根文件系统的存放方式

方式	识别条件与地址换算	适用场景
ext2 分区	`root=` 指定的分区（如 `mmcblk0p3`，块设备 5）；挂载时由 `ext2_fill_super()` 校验（6.8.9 节）；扇区 = 分区起点 + 块号 ×2	真机
裸镜像	没有 MBR；ROOT_DEV 是整卡 `mmcblk0`（块设备 2）；扇区 = 块号 ×2	QEMU

早期还支持把镜像存成 FAT32 根目录里的 `FS.IMG` 文件，经 `fat32mapsector()` 查块到扇区的映射；这种模式连同那两个映射函数已经删除。

ext2 分区 是真机上的做法：SD 卡分成 4 个区，p1 是 FAT32 启动分区，根文件系统是 p3 上的 ext2，与启动分区互不干扰。MBR 类型只是 0x83（macOS 也能建这种分区）。启动日志（节选）：

```
rootdev: bootfs mmcblk0p1 lba=2048 mounted as FAT32
Kernel command line: root=/dev/mmcblk0p3 rootfstype=ext2
rootdev: root=/dev/mmcblk0p3 rootfstype=ext2 -> mmcblk0p3
rootdev: ROOT_DEV = 5 (mmcblk0p3)
blkdev: cache self-test ok (mmcblk0 sector 0: 1 device read, MBR signature, second read hit)
```

p2 上的 xv6 分区. 根分区本身不需要识别，但 `rootdev_init()` 还要为 ext2 找到外置日志所在的分区：块层登记完分区以后，它为了找 ext2 外置日志所在的分区，对每个非 FAT 主分区（类型 0x7f 的优先）调用 `is_xv6fs()`（`kernel/drivers/block/rootdev.c`）：

```
static int is_xv6fs(struct blkdev *bd) {
  // xv6 的第 1 块是 superblock；块大小 1024，所以它在分区内第 2 个扇区
  if((bd->nsect != 0 && bd->nsect < FSSIZE * SECTORS_PER_BLOCK) ||
     bdev_read(bd, SECTORS_PER_BLOCK, sector) < 0 ||
     le32(sector) != FSMAGIC || le32(sector + 4) != FSSIZE)
    return 0;
  return 1;
}
```

判断只看两样东西：superblock 开头的魔数 0x10203040，以及紧跟着的 size 是否等于内核编译时的 FSSIZE。MBR 里的分区类型只决定自动探测的先后，不作为判断依据——macOS 的 diskutil 只能把这个分区建成 Linux 的 0x83 类型，和真正的 ext2 分区一样，靠魔数才能分开。size 的检查同样重要：如果有人改了 FSSIZE 重新编译内核却没重新烧 fs.img，这里会拒绝，而不是让新内核去解释旧格式的磁盘。识别成功后启动日志打印：

```
rootdev: xv6 filesystem on mmcblk0p2 start=208896 sectors=819200 size=400 MiB
```

此后块缓存对设备 4 的每一次读写都经过 blk_rw()：它按 b->dev 查到 mmcblk0p2，把块号换成分区内的扇区号（1 块 = 2 扇区），再调用分区的回调：

```
static int part_rw(struct blkdev *bd, uint32 sector, uchar *data,
                   uint32 count, int write) {   // kernel/drivers/block/blkdev.c
  struct blkdev *disk = bd->disk;               // mmcblk0
  return disk->rw(disk, bd->start - disk->start + sector, data, count, write);
}
```

扇区号 = 分区起点 208896 + 块号 ×2，然后由整卡的 mmc_rw() 交给 SDHOST。整条路径不经过 fat32.c 的任何函数。分区有 400 MiB，文件系统只用前 32 MiB；可分配多少块由 superblock 决定，与分区大小无关。

6.8 Linux 式命名空间引导

内核刚启动时，块设备驱动可能还没准备好，根文件系统在哪里也可能还不知道，可 / 这个目录必须先存在。Linux 的办法是先在内存里放一个临时的文件系统充当 /，等找到真正的根文件系统再把它换上来。PC 发行版和嵌入式系统“找”的方式完全不同。本节先把几个名字分清楚，再比较这两种做法，最后说明 xv6 为什么、以及怎样模仿嵌入式 Linux。

6.8.1 三个“根”：rootfs、根目录 / 与根文件系统

讨论根的切换时，有三样东西很容易混为一谈，本书刻意用不同的写法区分它们（表 6.5）。

表 6.5: 本书对三个“根”的写法

写法	是什么	生命周期
rootfs（英文）	Linux 里一个具体的临时文件系统实例，名字就叫 rootfs：内核在 init_mount_tree() 中建立的 ramfs/tmpfs，是第一个被挂上的文件系统。initramfs 就解压在它里面	开机即有，永远不会被卸载；切换后被压在真正的根文件系统下面，内容被清空或闲置
根目录 /	名字空间里的一个位置：路径解析的起点（Linux 中是进程的 fs->root）	一直存在，但“/ 是哪个文件系统的哪个目录”会变：先是 rootfs，切换后是根文件系统的根目录
根文件系统（中文）	设备上那个真正的文件系统：/dev/mmcblk0p2 上的 ext4、xv6 的 p2、本书的 p3 ext2，里面有 /sbin/init、/bin、/etc	由 root= 指定，挂上之后才成为 /

笔记 rootfs 这个英文名和“根文件系统”的字面意思几乎一样，正因如此才容易混淆。可以拿 Java 来类比：Java 里有一个类，它的名字就叫 `Class`（there is a class named `Class`）。同样，Linux 里有一个临时文件系统，它的名字就叫 rootfs，而它恰恰不是我们通常说的根文件系统，只是开机时临时占住 `/` 的那一个。本书中，英文 rootfs 只指这个临时文件系统，中文“根文件系统”只指设备上真正的那个，“根目录”或 `/` 只指路径的起点。

图 6.11 画出了三者在启动过程中的关系：`/` 先指向 rootfs；根文件系统先挂在 rootfs 的 `/root` 上；MS_MOVE 和 chroot 之后，`/` 指向根文件系统的根目录，rootfs 被压在下面。

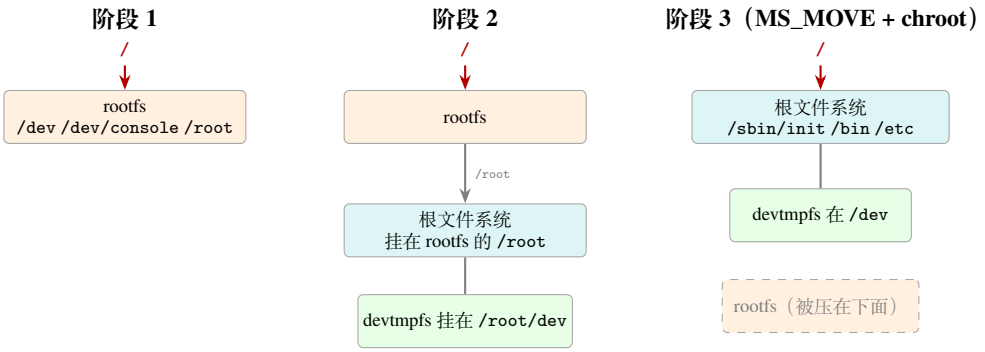


图 6.11: 根目录 `/` 在启动中指向哪里（嵌入式 Linux 的流程，xv6 照此实现）

6.8.2 PC 上的 Linux：initramfs 与用户态找根

PC 发行版（Ubuntu、Fedora 等）面对的硬件不可预测：根文件系统可能在 NVMe、SATA、USB、RAID、LVM 甚至加密盘或网络上。发行版不可能把所有驱动都编进内核，而盘符（`sda`、`sdb`）又会随插拔顺序变化，所以 bootloader 传的通常是 `root=UUID=…`，这里的 UUID 是 ext4、xfs 等写在文件系统超级块里的 UUID。内核里解析 `root=` 的函数（`name_to_dev_t()`，新内核中叫 `early_lookup_bdev()`）只认设备名、`PARTUUID=`、`PARTLABEL=` 和主次设备号，不会去读各种文件系统的超级块，因此找根的工作必须交给用户态：

1. GRUB 把内核 `vmlinuz` 和一个 cpio 压缩包 `initrd.img`（initramfs）一起装进内存，并传入 `root=UUID=…`；
2. 内核照例建立 rootfs 作为 `/`，再把 initramfs 解压进 rootfs。initramfs 里有一整套小型用户态：`/init` 脚本、busybox、`udevd`、`blkid`、内核模块；
3. `kernel_init_freeable()` 发现 rootfs 里有 `/init`，就不再调用 `prepare_namespace()`，直接在用户态运行这个 `/init`；
4. `/init` 挂上 `/proc`、`/sys`，把 devtmpfs 挂到 rootfs 的 `/dev`，启动 `udevd`，用 `modprobe` 加载 `nvme.ko`、`ahci.ko` 等驱动，设备节点随之出现；
5. 用 `blkid` 读各分区的超级块，找出 UUID 与 `root=` 相符的那个（必要时先 `cryptsetup` 解密、激活 LVM 或 RAID），把它挂到 `/root`（initramfs-tools）或 `/sysroot`（dracut）；
6. `switch_root`：把 `/dev`、`/proc`、`/sys`、`/run` 用 MS_MOVE 搬进新根，删掉 rootfs 里的全部文件以释放内存（rootfs 本身不能卸载），然后 `mount --move newroot /`、`chroot .`，最后 `exec /sbin/init`（通常是 systemd）。

6.8.3 嵌入式 Linux：内核按 `root=` 直接挂载

嵌入式板子的硬件是固定的：一张 SD 卡或 eMMC，驱动直接编进内核，分区布局在出厂时就定好。U-Boot 或固件在 `bootargs` 里传入 `root=/dev/mmcblk0p2 rootfstype=ext4 rootwait`，内核自己完成全部工作，不需要 initramfs：

1. `init_mount_tree()` 建立 rootfs 作为 `/`；没有 initramfs 时，`default_rootfs()` 在 rootfs 里建出 `/dev`、`/dev/console` 和 `/root`；
2. `devtmpfs_init()` 建立 devtmpfs，驱动注册时内核自动在里面建设备节点；

3. rootfs 里没有 `/init`，于是 `prepare_namespace()` 出场：`rootwait` 时等设备出现，再用 `name_to_dev_t()` 把 `root=` 解析成设备号 `ROOT_DEV`；
4. `mount_root()`：在 rootfs 里建一个设备节点 `/dev/root`，把它挂到 `/root`，然后 `chdir("/root")`；
5. `devtmpfs_mount()` 把 devtmpfs 挂到 `dev`，也就是 `/root/dev`（`CONFIG_DEVTMPFS_MOUNT` 打开时）；
6. `mount(".", "/", MS_MOVE)` 把根文件系统从 `/root` 挪到 `/`，盖住 rootfs；`chroot(".")` 让进程的根目录指向它；
7. 运行 `/sbin/init`（或 `init=` 指定的程序）。

两种做法的对比见表 6.6。

表 6.6: PC 发行版与嵌入式 Linux 挂载根文件系统的对比

	PC 发行版（有 initramfs）	嵌入式（无 initramfs）
谁来找根	用户态：initramfs 里的 `/init`、udev、blkid	内核：`prepare_namespace()`、`name_to_dev_t()`
`root=` 写法	`UUID=`（文件系统 UUID）、LVM、LUKS、NFS 等任意形式	`/dev/mmcblk0p2`、`PARTUUID=`、主次设备号
驱动	模块，放在 initramfs 里按需加载	编进内核
rootfs 里有什么	解压后的 initramfs：一个完整的小型用户态	只有 `/dev`、`/dev/console`、`/dev/root`、`/root`
`/dev`	devtmpfs + udev 动态建节点	devtmpfs，内核建节点
切换根目录	用户态 `switch_root`：搬 `/dev` 等、清空 rootfs、MS_MOVE、chroot	内核：MS_MOVE + chroot
追求	兼容性、通用性	快、精简、专盘专用

两种做法有三点相同：rootfs 总是第一个 `/`；根文件系统总是先挂在 rootfs 的某个目录下，再 MS_MOVE 到 `/`；rootfs 从来不会被卸载。

6.8.4 xv6 的选择：模仿嵌入式 Linux

xv6 跑在硬件固定的树莓派 3B+ 上：只有一张 SD 卡，驱动全部编进内核，没有内核模块，也没有 udev、blkid 这样的用户态工具，分区由 MBR 描述。这正是嵌入式 Linux 的处境，所以 xv6 走嵌入式路线：内核自己读 `root=`，自己找到分区并挂载。表 6.7 把上一小节的每一步对应到 xv6 的函数。

表 6.7: 嵌入式 Linux 的启动步骤与 xv6 的实现

嵌入式 Linux	xv6
`bootargs` 中的 `root=`	BOOTFS 上的 `cmdline.txt`，由 `cmdline.c` 自己读；固件经 `disable_commandline_tags=1` 不再处理它
`name_to_dev_t()`	`name_to_bdev()`：`/dev/mmcblk0pN`、`mmcblk0pN`、`PARTUUID=`
`ROOT_DEV`	`rootdev_dev()`：选中分区自己的块设备号（ext2 根 p3 是 5）
`register_filesystem()`	同名：xv6fs、ext2、procfs、sysfs、fat32、netfs 挂进 `file_systems` 链表
rootfs（ramfs）	RAM 盘 ram0 上的 xv6 文件系统
`default_rootfs()`	在 rootfs 中建 `/dev`、`/dev/console`、`/root`
devtmpfs	RAM 盘 ram1 上的 xv6 文件系统，节点来自 `register_chrdev()` 和块设备表
`create_dev("/dev/root")`	`kmknod("/dev/root", T_DEVICE, BLOCKDEV, rootdev_dev())`
`mount_root()`	`mount_root()`：`vfs_get_sb(rootfstype, "/dev/root")` 经 `get_sb/fill_super` 读超级块、恢复日志，再 `fs_mount()` 挂到 `/root`
`devtmpfs_mount()`	`devtmpfs_mount()`：挂到 `/root/dev`
`mount(".", "/", MS_MOVE)`	`fs_move_mount()`
`chroot(".")`	`fs_set_root()`
`run_init_process()`	initcode 执行 `exec("/init")`

xv6 与 Linux 仍有三处不同（表 6.8）。

表 6.8: xv6 与 Linux 根命名空间引导的差别

	Linux	xv6
rootfs、devtmpfs	ramfs / tmpfs，不经过块设备	两个 RAM 盘（ram0、ram1）上的 xv6 文件系统，开机时由 `ramfs_mkfs()` 直接在内存里格式化；因为 xv6 的 inode 层建在 buffer cache 上，这样不用另写一套文件系统
`root=` 无效	panic：VFS: Unable to mount root fs	打印原因，退回自动探测（0x7f 优先，再看 superblock 魔数），先保证能启动
chroot	每个进程有自己的 `fs->root`	只有一个挂载命名空间，`fs_set_root()` 对所有进程生效；用户态没有 chroot 系统调用

6.8.5 启动调用链总览

Linux 把找根和挂根都放在 `prepare_namespace()` 里，xv6 则分在两个时刻做：`main()` 里只能用自旋锁、不能睡眠，所以这里只读 MBR、登记块设备、决定根文件系统在哪个分区；真正读根文件系统的超级块、恢复日志、搭建目录树，要用块缓存的睡眠锁，必须等到第一个进程在 `forkret()` 里运行。第三个时刻是用户态的 `/init`，它按 `/etc/fstab` 挂其余的文件系统。

下面给出 ext2 作根文件系统时的完整调用链。左边是函数调用，缩进表示调用关系；右边是这一步做了什么；中间的 (a)–(d) 标出“`root=` 这串字符怎样一步步变成真正的扇区”，后文会单独解释。后面四个小节按阶段一、二、三依次展开。

启动分区上的 `cmdline.txt` 写成 `root=/dev/mmcblk0p3 rootfstype=ext2`。和 Linux 一样，ext2 在阶段一只登记“有这种文件系统”，打开分区、读超级块、恢复日志都推迟到阶段二 `mount_root()` 调用它的 `fill_super` 时才做：

```
main()                                  kernel/init/main.c：只在 CPU 0 上运行，此时
                                        还没有进程，只能用自旋锁，不能睡眠
  binit()                               初始化块缓存：64 个缓冲区，每个一页
  iinit()                               初始化内存 inode 表（itable，50 项）
    loginit()                           提前建立日志锁：阶段二要往 RAM 盘里写东西
    fs_mountinit()                      挂载表的锁，表为空；
                                        register_filesystem(&xv6fs_type)，
                                        供 RAM 盘上的 rootfs、devtmpfs 使用
  do_initcalls()                        按（级别，序号）调用各模块的 initcall
    [subsys] blkdev_init()              建立块设备表；登记主设备号 BLOCKDEV
    [device] sdhost_probe()             初始化 SD 卡
      add_disk(2, "mmcblk0", 5, mmc_rw)
                                        整张卡登记为块设备 2
        blk_add_partitions()            块层自己读第 0 扇区的 MBR
          add_partition()               主分区 N 登记为块设备 2+N（p3 是 5），
                                        回调 part_rw()：加上分区起点后交给 mmc_rw()
    [late]   rootdev_init()             ======== 阶段一 ========
      fat32mount(p1)                    挂上启动分区，读 cmdline.txt
      check_rootfstype()                rootfstype=ext2 -> ROOTFS_EXT2
      name_to_bdev("/dev/mmcblk0p3")
                                  (a)   root= 换成块设备 5：ROOT_DEV = 5
      自动探测 xv6 分区                  仍然找 p2：ext2 的外置日志在 p2 尾部，
                                        以后由 rootdev_xv6_part() 取得
    [late 5] ext2init()             (b) 只 register_filesystem(&ext2_type)，
                                        把 "ext2" 挂进 file_systems 链表，不读盘
    [late 6] vfsinit()                  登记 procfs、sysfs、fat32、netfs
```

```
  userinit()                          造出第一个进程，当前目录暂时为空
  scheduler()                         调度第一个进程，它从 forkret() 开始运行
forkret()                             kernel/core/proc.c：从这里起可以睡眠，
                                      可以使用块缓存
  prepare_namespace()                 ======== 阶段二 ========
    1. ramfs_create(0, 7, "ram0")     分配 32 KiB 内存当作块设备 7，在里面
                                      格式化出空的 xv6 文件系统：rootfs
       fs_set_root(rootfs 根目录)     "/" 从此指向 rootfs 的 (7,1)
       kmknod(...)                    在 rootfs 里建 /dev、/dev/console、/root
    2. devtmpfs_init()                同样建出块设备 8 (ram1)，按已注册的
                                      驱动和块设备表逐个建设备节点；此时
                                      devtmpfs 还没有挂到任何目录上
    3. kmknod("/dev/root", BLOCKDEV, 5)
                                (c)   rootfs 里的节点 (7,5)，次设备号 = ROOT_DEV
       mount_root("/dev/root", "/root")
         vfs_get_sb("ext2", "/dev/root", VFS_MOUNT_ROOTLOG)
           get_fs_type("ext2")        沿 file_systems 链表找到 ext2_type
           ext2_get_sb()
             lookup_bdev("/dev/root")
                                (d)   namei 到 (7,5)：T_DEVICE、主设备号
                                      BLOCKDEV、次设备号 5 -> dev 5
             getsuper(5) == 0         dev 5 还没有超级块
             ext2_fill_super(5)       blkdev_get(5)：p3 的起点、长度、读写回调
               setup_ext2()           读并校验 ext2 超级块、块组描述符、特性位
               ext2_journal_init()    在 p2 内第 65536 扇区起 xj_init()，
                                      有已提交事务就先重放
               readinode(2)           根 inode 必须是受支持的目录
               ext2_register_super(5) supers[5]：类型 ext2、ext2_iops、
                                      rootino 2、priv = &ext2_instances[5]
         klookup("/root")             (7,4)
         fs_mount((7,4), 5)           mtab.m[0]：(7,4) 被 dev 5 的根目录盖住
       kchdir("/root")                follow_mount：m[0] -> getsuper(5)
                                      -> iget(5, 2)：valid=0，继承 ext2_iops
                                      -> ilock -> e2_read_inode -> e2fill
                                      -> 挂接 ext2_dir_iops/ext2_dir_fops，
                                         当前目录 = (5,2)
       kmknod("dev")                  ext2 上没有 /dev 就建一个 (e2_create())
       devtmpfs_mount("dev")          m[1]：ext2 的 /dev (5,76) 被 dev 8 盖住，
                                      此时路径是 /root/dev
    4. fs_move_mount(".", "/")        m[0].mp 从 (7,4) 改成 (7,1)，路径改写为
                                      "/" 和 "/dev"
       fs_set_root(当前目录)          "/" = (5,2)，rootfs 被压在下面
  usertrapret()                       回到用户态
initcode：exec("/init")               ======== 阶段三 ========
  /init：mount_fstab()                /proc、/sys、/boot、/mnt/net；ext2 那一行
                                      因 dev 5 已经是 m[0] 而被拒绝 (busy)
```

图 6.12 把这条调用链画成了带文件行号和启动日志的调用树：A 段是启动早期只登记类型，B 段是 prepare_namespace() 里挂 ext2 根，C 段是 /init 的 mount_fstab()。橙色序号 ⓪–⑥ 标出 ext2 根特有

的几步：ext2init() 登记、vfs_get_sb("ext2")、lookup_bdev() 得到 dev 5、ext2_fill_super(5) 读盘、ext2_register_super() 登记 supers[5]、klookup("/root") 找挂载点、kchdir("/root") 经 follow_mount() 进入 ext2。

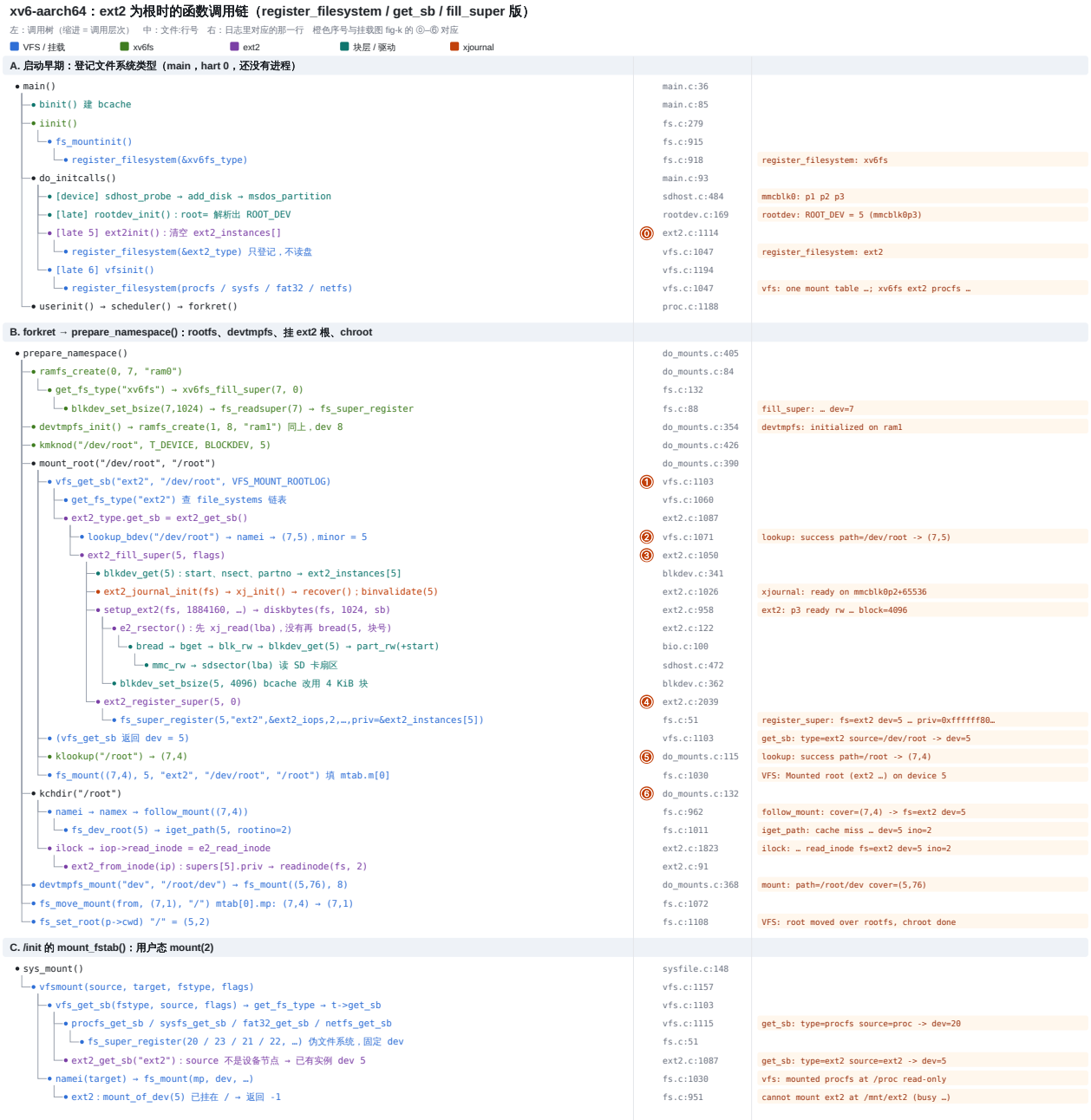


图 6.12: ext2 作根时的函数调用链（register_filesystem / get_sb / fill_super 版）：左为调用树，中为文件行号，右为对应的启动日志

ext2 根的 (a)–(d). (a) 把 root= 解析成块设备 5（mmcblk0p3）并记为 ROOT_DEV。(b) ext2init() 只在类型链表里登记 ext2_type，不碰 SD 卡。阶段二的 (c) 让 /dev/root 的次设备号等于 5，(d) lookup_bdev() 从节点读回 5，ext2_fill_super(5) 才第一次打开 p3：分区起点与长度来自块设备表 blkdev_get(5)，而不是 ext2 自己再去解析 MBR；p3 与块设备 5 的联系在 SD 驱动探测时就由 add_partition() 建好了，part_rw() 让“设备 5 的扇区 *s*”等于“卡上扇区 p3 起点 + *s*”。它直接读取 inode 2 只为验证根目录，然后登记 bootstrap ext2_iops；并没有调用 ilock()。挂载表建立以后，kchdir("/root") 经 follow_mount() 和 iget(5,2) 得到 valid=0 的

VFS inode，随后的第一次 `ilock()` 才通过 `ext2_iops.read_inode` 进入 `e2_read_inode/e2fill`，挂接根目录最终的 `iop/i_fop`。之后在新根的 `/dev` 上挂 devtmpfs、MS_MOVE、chroot，这几步与文件系统类型无关。

`/dev/root` 是什么。 第 3 步里的 `mount_root("/dev/root", "/root")` 很容易被读成“把 `/dev/root` 挂到 `/root`”，好像挂上去的是 `/dev/root` 这个东西本身。其实 `/dev/root` 只是 rootfs 里的一个设备节点：一个名字加一对设备号（主设备号 `BLOCKDEV`，次设备号是块设备表里的编号），它本身既不是文件系统，也不含数据。`mount_root()` 的两个参数，一个说“到哪里找设备”，一个说“挂在哪个目录”，真正的工作由它下面三步完成：`lookup_bdev("/dev/root")` 把名字换成设备号 5；`ext2_fill_super(5)` 从设备 5 读出 ext2 的超级块并重放日志；`fs_mount((7,4), 5)` 才把这个文件系统的根目录盖到 rootfs 的 `/root` 目录上。命令行 `mount /dev/root /root` 也是同样的意思：挂上去的是节点所指设备上的文件系统，而不是节点。

`/dev/root` 不是一个单独的设备，它就是 Linux 的 `ROOT_DEV`：“这次启动用作根文件系统的那个块设备”，这里是 5。早先的实现另外登记过一个“设备 1（root）”包住 p2，读写回调 `root_blk_rw()` 负责块号换算，`/dev/sdroot` 也经它更新根镜像；现在这一层已经删掉，ext2 的外置日志通过 `rootdev_xv6_part()` 直接取得 p2 的块设备。

可以把这三个阶段和图 6.11 对上：阶段一结束时，内核只知道根文件系统在哪个分区，`/` 还不存在；阶段二的第 1 步让 `/` 指向 rootfs（图中阶段 1），第 3 步把根文件系统挂到 `/root`（图中阶段 2），第 4 步 MS_MOVE 和 chroot 之后 `/` 指向根文件系统（图中阶段 3）。阶段三在这棵树上继续挂其他文件系统。

6.8.6 阶段一：登记块设备、选出 ROOT_DEV

阶段一要回答“根文件系统在哪个块设备上”，为此先要有一张块设备表。原来 `bio.c` 的 `bread/bwrite` 直接调用 `rootdev_rw()`，只能访问一个设备；现在 `blkdev.c` 仿照 Linux 的 gendisk 维护 `blk.dev[NBLKDEV]`，每项描述一个设备：

```
// include/kernel/blkdev.h (节选)
struct blkdev {
  int      dev;      // BLKDEV_MMC=2, BLKDEV_MMC_PART(n)=2+n, BLKDEV_RAM(n)=7+n
  char     name[16]; // "mmcblk0", "mmcblk0p2", "ram0" ...
  uint32   start;    // 在硬件上的起始扇区 (整盘为 0)
  uint32   nsect;    // 扇区数 (0 = 未知, 不做越界检查)
  uint32   bsize;    // 块缓存的块大小 (512 / 1024 / 4096 B)
  blk_rw_fn rw;      // I/O 回调: 整盘 mmc_rw, 分区 part_rw, RAM 盘 ram_rw
  void     *priv;    // 驱动私有数据 (RAM 盘的页)
  struct blkdev *disk;   // 分区所属的整盘; 整盘为 0
  int      partno;   // 分区号, 整盘为 0
  uchar    ptype;    // 分区的 MBR 类型 (0x0c FAT32、0x83 Linux、0x7f xv6)
  char     uuid[12]; // PARTUUID "ssssssss-pp"
};
```

这张表由三个 initcall 先后填好，`main()` 里只剩一句 `do_initcalls()`：

1. `blkdev_init()`（subsys 级）初始化设备表，并注册主设备号 `BLOCKDEV`，让块设备以后能以 `/dev/mmcblk0p3` 这样的文件出现；
2. SD 驱动的 `sdhost_probe()`（device 级）初始化卡，调用 `add_disk(2, "mmcblk0", 5, …, mmc_rw)`；块层的 `blk_add_partitions()` 读 MBR，`add_partition()` 把主分区 N 登记为块设备 $2+N$，回调是 `part_rw()`。和 Linux 一样，驱动只登记整盘，分区由块层自己扫描；
3. `rootdev_init()`（late 级）列出分区，挂上 FAT32 启动分区，由 `cmdline_init()` 读出 `cmdline.txt`（6.7 节）；`check_rootfstype()` 读 `rootfstype=ext2`，`name_to_bdev()` 把 `root=/dev/mmcblk0p3` 换成块设备

5；它不读 p3 的内容，分区上是不是 ext2 要到阶段二挂载时才检查。最后记下 ROOT_DEV，打印 `rootdev: ROOT_DEV = 5 (mmcblk0p3)`。它不再登记任何设备。

从此 `bio.c` 的读写入口是 `blk_rw(b, write)`：按 `b->dev` 查表，把块号换成扇区号，调用该设备的回调。为了容纳 ext2 的 4 KiB 块，`buf.data` 从静态数组改为 `kalloc()` 分配的页，`NBUF` 从 30 扩到 64；新增的 `buf.error` 让非根设备的 I/O 失败只置位而不 panic，根设备（`rootdev_dev()`）失败仍 panic。

阶段一结束时，ROOT_DEV 已经选定，但根分区的超级块还没有被读过，`/` 也还不存在。随后 `ext2init()` 只把 ext2 登记进文件系统类型链表（`register_filesystem()`），不读盘，`vfsinit()` 登记其余几种文件系统并初始化 netfs，`userinit()` 造出第一个进程，此时它的当前目录还是空的。

笔记 块设备文件（`/dev/mmcblk0p3` 等）经过 buffer cache，用户态只读；`lseek(fd, 0, SEEK_END)` 返回分区大小。它们的节点由阶段二的 devtmpfs 建出来。

6.8.7 阶段二：`forkret()` 中的 `prepare_namespace()`

第一个进程被调度后进入 `forkret()`，它在返回用户态之前调用一次 `prepare_namespace()`（`kernel/init/do_mounts.c`，文件名与 Linux 的 `init/do_mounts.c` 相同）。这个函数分四步，把图 6.11 的三个阶段依次走一遍（图 6.13）。

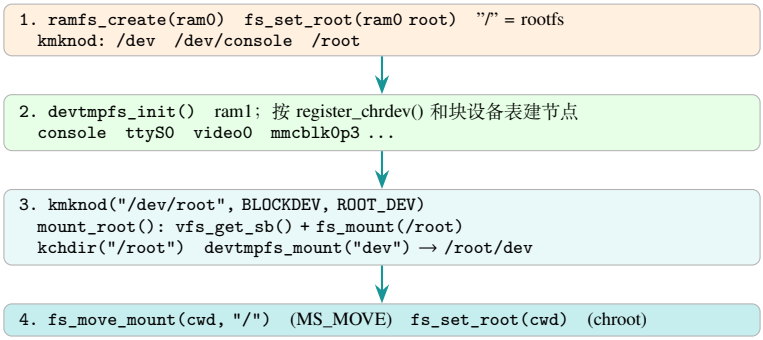


图 6.13: `prepare_namespace()` 的四步（颜色与图 6.11 对应）

第 1 步：rootfs 成为 /. `ramfs_create()` 先用 `ramdisk_create()` 分配一个 32 KiB 的 RAM 盘并登记为块设备 ram0（dev 7），再用 `ramfs_mkfs()` 直接在这块内存里写出一个空的 xv6 文件系统（超级块、inode 区、位图、只含 `.` 和 `..` 的根目录，没有日志区），最后 `fs_readsuper()` 读回超级块。`fs_set_root()` 让 `/` 指向它的根目录，进程的当前目录也设在这里。接着像 Linux 的 `default_rootfs()` 一样建出 `/dev`、`/dev/console` 和 `/root`。

第 2 步：devtmpfs. `devtmpfs_init()` 用同样的办法建出 ram1（dev 8），遍历已注册的字符设备（`chrdev_name()` 给出 `console`、`input/event0` 这样的路径）和块设备表，为每个设备建一个节点。之后才注册的驱动由 `devtmpfs_notify()` 交给工作队列补建。此时 devtmpfs 还没挂到任何地方。

第 3 步：挂上根文件系统. 先在 rootfs 里建设备节点 `/dev/root`（主设备号 `BLOCKDEV`，次设备号是 ROOT_DEV = 5；6.8.5 节调用链中的 (c)）。`mount_root()` 按 `rootdev_fstype()` 取类型名，调用 `vfs_get_sb("ext2", "/dev/root", VFS_MOUNT_ROOTLOG)`：`get_fs_type()` 在类型链表里找到 `ext2_type`，它的 `ext2_get_sb()` 用 `lookup_bdev()` 解析节点，从 inode 的次设备号得到 5 (d)。设备 5 还没有超级块，于是调用 `ext2_fill_super(5)`：读并校验 ext2 超级块和块组描述符，重放 p2 尾部的外置日志，检查根 inode 2，再为设备 5 登记 `super_block`（类型 ext2、函数表 `ext2_iops`、根 inode 2，6.5 节）。这一步对应 Linux 的 `fill_super`，读操作经 `blk_rw()` 和 `part_rw()` 落到 p3 上。最后 `fs_mount()` 把设备 5 上文件系统的根目录挂到 rootfs 的 `/root` 上，启动日志打印：

```
VFS: Mounted root (ext2 filesystem) on device 5 (/dev/root = mmcblk0p3).
```

`kchdir("/root")` 之后，相对路径 dev 指的就是 ext2 里的 /dev，`devtmpfs_mount("dev")` 把 devtmpfs 挂上去。ext2 上没有这个目录时，内核先经 `e2_create()` 建出这个挂载点。

第 4 步：MS_MOVE 与 chroot. `fs_move_mount()` 把“挂在 /root”的根文件系统改为“挂在 rootfs 的根目录上”，挂在它下面的 devtmpfs 随之从 /root/dev 变成 /dev；`fs_set_root()` 再把 / 改指根文件系统的根目录。rootfs 从此被压在下面，只剩 /dev/root 这个名字还留在它里面。

`prepare_namespace()` 返回后，`forkret()` 运行 `blkdev_selftest()`（读 mmcblk0 扇区 0 两次，确认第二次命中缓存），然后经 `usertrapret()` 回到用户态，执行 initcode。

6.8.8 支撑这四步的机制：超级块、日志、挂载表与 namex()

上面四步调用的 `fs_readsuper()`、`fs_mount()`、`fs_move_mount()`、`fs_set_root()` 都在 fs.c。原来的 xv6 只有一个根文件系统，所以既不需要挂载表，路径解析也直接从常量 `iget(ROOTDEV, ROOTINO)` 开始；现在同一时刻有三个文件系统（rootfs 和 devtmpfs 用 xv6fs 磁盘格式，根文件系统是 ext2），这几处都要改。

每个设备一份超级块（第 1、2、3 步）. 全局的 `struct superblock sb` 改为 `sbs[NBLKDEV]`，balloc、ialloc 等通过 `getsb(dev)` 取得对应设备的 xv6 磁盘超级块。`fs_readsuper(dev)` 读块 1、检查魔数并存入 `sbs[dev]`，同时在 VFS 的 `supers[]` 里登记一个内存中的 `super_block`（类型、函数表、根 inode 号，6.5 节）。rootfs 和 devtmpfs 走的就是这条路，不做日志恢复；ext2 根则由 `ext2_fill_super()` 登记 `supers[5]`，不用 `sbs[]`。`sbs[]` 是 xv6fs 自己的磁盘格式，后者是所有文件系统共用的“实例描述”，`supers[]` 是所有文件系统共用的“实例描述”，相当于 Linux 里 `struct ext2_sb_info` 与 `struct super_block` 的关系。

RAM 盘不走日志（第 1、2 步）. 第 1、2 步在 rootfs 和 devtmpfs 里建目录和节点时，还没有任何日志。日志锁由 `iinit()` 里的 `loginit()` 提前建立，`log_write()` 遇到不属于日志设备的块（`log.size==0 || b->dev!=log.dev`）就直接 `bwrite()` 写穿。RAM 盘的内容重启即消失，不需要崩溃一致性；根文件系统的一致性由第 3 步 `ext2_fill_super()` 打开的外置日志负责（6.8.9 节）。

挂载表与路径穿越（第 3、4 步）. fs.c 新增一张挂载表，每项记录“目录 mp 被设备 dev 的根目录覆盖”。这是系统里唯一的挂载表：rootfs 上的根文件系统、devtmpfs，以及阶段三挂上的 procfs、FAT32、ext2 都是其中的一项：

```
// kernel/fs/fs.c
struct fsmount {
  int          used;
  struct inode *mp;          // 被覆盖的目录（持引用）
  uint         dev;          // 挂载上来的文件系统设备号
  char         fstype[12], source[24];
  char         path[MAXPATH]; // 显示路径（用于 /proc/mounts）
};
```

`namex()` 在两处查这张表：

```
// 向下：lookup 到被覆盖的目录 -> 替换为被挂载文件系统的根目录
ip = follow_mount(next);

// 向上："..." 从被挂载文件系统的根目录出发 -> 先退回挂载点目录
if(namecmp(name, "..") == 0)
    ip = follow_dotdot(ip);
```

第 3 步的 fs_mount() 往表里加一项，于是解析 /root/... 时，follow_mount() 会在 root 这一级从 rootfs 跳进根文件系统（ROOT_DEV）。“是不是某个文件系统的根目录”由 isfsroot() 判断：比较的是该设备 super_block 里的 rootino（xv6fs 为 1，ext2 为 2），所以同样的跳转对 ext2 也成立。同一个文件系统不能挂两次，fs_mount() 发现设备已在表里就拒绝。第 4 步的 fs_move_mount() 只是把这一项的 mp 从 rootfs 的 /root 改成 rootfs 的根目录，并把所有子挂载的显示路径一并改写（/root/dev → /dev）。

/ 从哪里开始（第 1、4 步）。 namex() 解析绝对路径时调用 rootdir()，它返回 fs_root——第 1 步设为 rootfs 的根目录，第 4 步由 fs_set_root() 改为根文件系统的根目录。以切换后的两个路径为例：

```
namei("/bin/sh")    rootdir() = (dev 5, inode 2)  -> bin -> sh        ext2 根
namei("/dev/tty")   rootdir() = (dev 5, inode 2)  -> dev
                    follow_mount: dev 被覆盖 -> (dev 8, inode 1) -> tty  devtmpfs
```

follow_dotdot() 遇到 fs_root 时原地不动，所以 /.. 仍是 /，进程无法退回被压住的 rootfs。另外，fs_create() 是 kmknod() 和系统调用共用的入口，它检查文件系统是否可写，再调用目录所在文件系统的 iop->create；sys_unlink()、sys_rename() 用 fs_is_mountpoint() 拒绝删除或移走挂载点。

6.8.9 ext2 根的挂载细节

cmdline.txt 里的 root=/dev/mmcblk0p3 rootfstype=ext2 让 p3 上的 ext2（第 10 章）成为 /。有了统一的 VFS（6.5 节），ext2 只在两处有自己的代码：

1. 阶段一，rootdev_init() 按 root= 找到 mmcblk0p3，把 ROOT_DEV 记为 5、rootdev_fstype() 记为 ext2；它还会找 xv6 分区（rootdev_xv6_part()），因为 ext2 的外置日志在那里，找不到也不 panic。ext2init() 只登记文件系统类型，不读盘；
2. 阶段二，/dev/root 的次设备号是 5，mount_root() 调用 vfs_get_sb("ext2", "/dev/root")，由 ext2 自己的 get_sb 和 fill_super 打开分区、恢复外置日志、登记 supers[5]。

其余步骤与文件系统类型无关：ext2 先挂到 rootfs 的 /root，kchdir("/root")，在 ext2 自己的 /dev 上挂 devtmpfs（目录不存在时内核经 e2_create() 建出它），再 MS_MOVE 和 chroot。

图 6.14 从进程、内核态数据结构、硬件三层看 mount_root("/dev/root", "/root")：rootfs 里 /dev/root 节点 (7,5) 给出设备号，/root 节点 (7,4) 成为挂载点；mtab.m[0] 以 mp 指向 (7,4)、以 dev = 5 连到 supers[5]；supers[5] 的 rootino = 2 和 iop 把路径查找带进 ext2 的根目录 (5,2)；ext2 的私有状态和块设备表 blk.dev[] 的一项再落到 p3 的超级块、组描述符和 inode 表上，外置日志在 p2 尾部。

笔记 早先的实现不是这样。那时 ext2 只能挂在 vfs.c 的“路径前缀挂载表”里，它的目录不是 xv6 inode，既进不了 fs.c 的挂载表，也不能成为 MS_MOVE 和 chroot 的对象，只好用 mount_ext2_root() 直接挂到 /，devtmpfs 则挂在被遮住的 rootfs 的 /dev 上，再靠 fs_native_covers() 让 /dev 开头的路径绕过 ext2。两张挂载表、两套路径解析，正是把 VFS 改成 Linux 那样“每个文件系统一张函数表、一张挂载表”的直接原因；改完以后这些特殊处理全部删掉了。

ext2 挂载用到的四张表。 ext2 的挂载按 Linux 的 register_filesystem → get_sb → fill_super 进行，途中要查、要填四张表（表 6.9）。它们各管一件事：类型表回答“认不认识 ext2”；supers[] 回答“dev 5 上是什么文件系统、根 inode 号多少、用哪套 inode_ops”；挂载表只记录覆盖关系，“目录 mp 被 dev 号为 dev 的文件系统的根目录盖住”；inode 表是内存里的 inode 缓存。

挂载项里没有指向超级块的指针，只有一个整数 dev。follow_mount() 发现当前目录被某一项盖住时，用 getsuper(m->dev) 取 &supers[dev]，再由 fs_dev_root(dev) 以 (dev, sb->rootino) 调用 iget()；iget_path() 填槽时令 ip->iop = sb->iop，以后对这个 inode 的 lookup、read 都进入 ext2（图 6.15）。相比之下，mp 是真正的指针，指向 inode 表里被盖住的那一格，并持有它的一个引用。Linux 的 struct mount 直

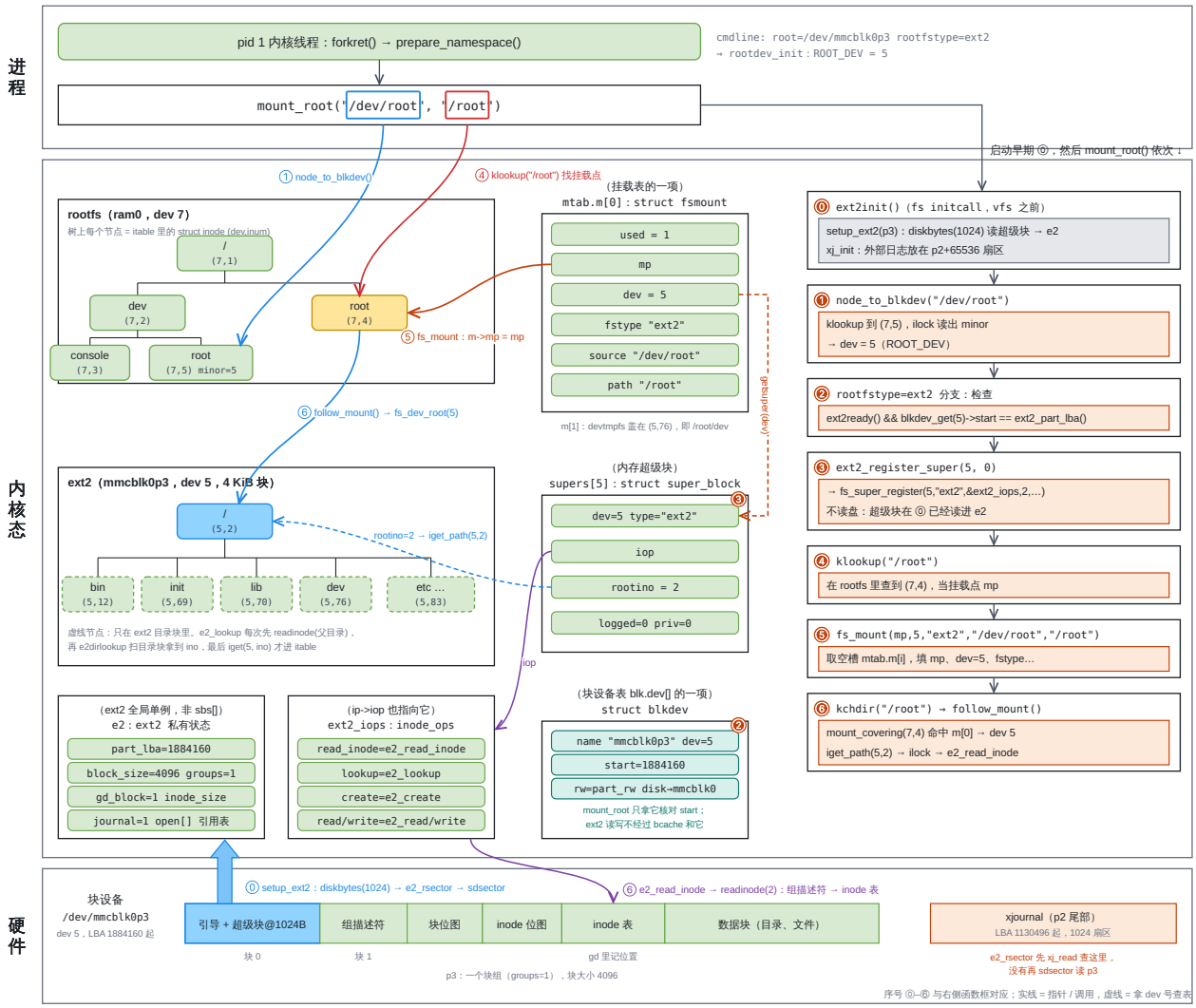


图 6.14: ext2 根挂载全景：rootfs 与 ext2 两棵目录树、挂载表、超级块、块设备表和 p3 的盘上布局。图中 ⓪ 画的是早期版本在 `ext2init()` 里就读超级块；当前代码把这一步移到了 `ext2_fill_super()`（图 6.12 的 ③）

表 6.9: ext2 挂载涉及的四张表（ext2 作根文件系统时）

表	按什么查	ext2 在其中的那一项
`file_systems` 链表（`vfs.c`）	类型名	`ext2_type`：`get_sb = ext2_get_sb`，`fill_super = ext2_fill_super`
`supers[24]` 数组（`fs.c`）	dev 号即下标	`supers[5]`：类型 ext2、`ext2_iops`、`rootino` 2、`priv` 指向 `ext2_instances[5]`
`mtab.m[8]` 数组（`fs.c`）	被盖住的 inode，或 dev	`m[0]`：`mp`、`dev` 5、`source` `/dev/root`、`path`
`itable.inode[50]` 数组（`fs.c`）	(dev, inum)	(5,2) 是 ext2 根目录，(5,76) 是它的 `/dev`

接存 `mnt.mnt_sb` 指针和 `mnt_root` dentry，挂载点用 (`mnt_parent`, `mnt_mountpoint`) 这一对来标识，查找走 `mount_hashtable`；xv6 用 dev 当下标、线性扫描 8 项，作用相同。

inode 表只按 (dev, inum) 认 inode，不记名字。名字到 inode 号的对应在目录块里，每走一级都由 `dirlookup()`（ext2 是 `e2_lookup()`）扫描目录块；xv6 没有 Linux 的 dentry cache。挂载表里的 `path` 是挂载点在整个命名空间里的绝对路径，只供 `/proc/mounts` 显示和 `umount` 按名字查找使用，路径解析从不看它。

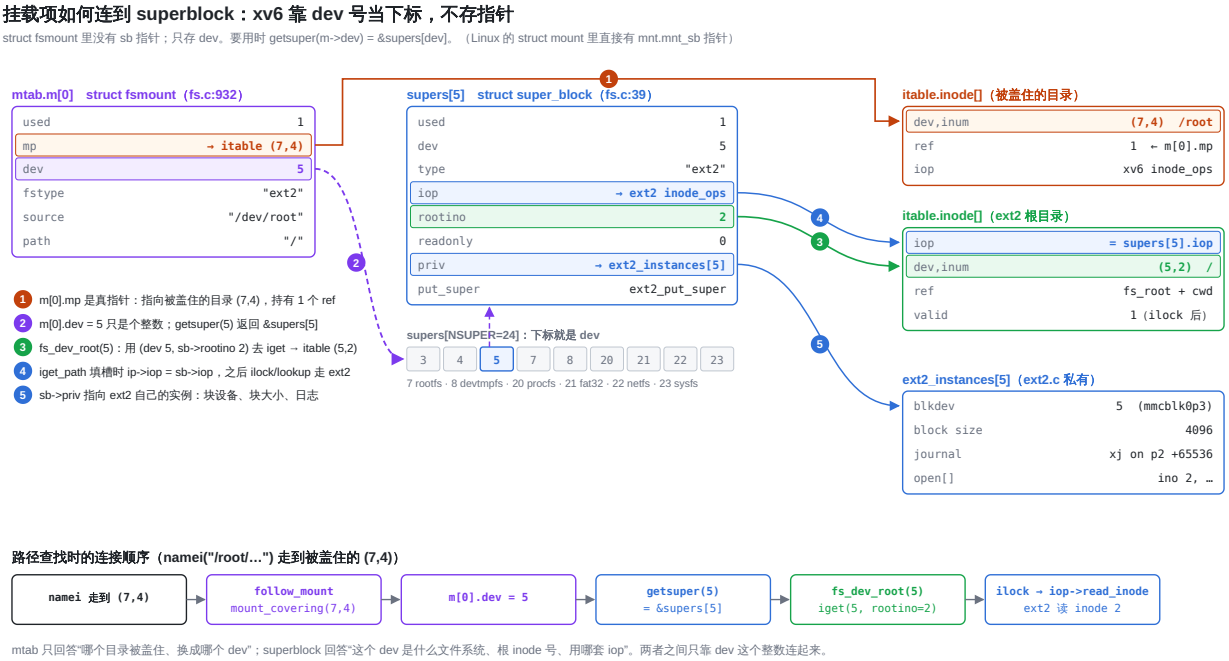


图 6.15: 挂载项怎样连到超级块：`mp` 是指针，指向被盖住的目录；`dev` 只是 `supers[]` 的下标，再经 `rootino`、`iop`、`priv` 进入 ext2

`mount_root()` 怎样找到 ext2. 把第 6.8.5 节调用链里的 (c)、(d) 展开来看：类型来自 `rootfstype=`，设备号来自 rootfs 里节点 `/dev/root` 的次设备号，驱动来自块设备表，`fill_super` 来自类型表。每一张表都是更早的阶段填好的。

每一步改了哪张表. 从 `fs_mount()` 起，挂载表与 inode 表的变化如图 6.16 和表 6.10：图中每一行是一次挂载操作，左边是 `m[0]`–`m[5]` 与 `fs_root`，右边是 `mp` 指向的 inode 及其引用数。(7,*x*) 是 rootfs 的 inode，(5,*x*) 是 ext2 的 inode，(8,1) 是 devtmpfs 的根目录。

表 6.10: ext2 作根时挂载表与 inode 表的变化

步骤	挂载表	inode 表
`fs_mount((7,4), 5)`	`m[0]` = {mp (7,4)，dev 5，path `/root`}	(7,4) 引用 1，由 `m[0].mp` 持有
`kchdir("/root")`	不变	`follow_mount()` 命中 `m[0]`，取得 (5,2)；当前目录 = (5,2)，(7,1) 引用 2→1
`devtmpfs_mount()`	`m[1]` = {mp (5,76)，dev 8，path `/root/dev`}	(5,76) 引用 1，由 `m[1].mp` 持有
`fs_move_mount()`	`m[0].mp` 改为 (7,1)；path 改为 `/`，`m[1]` 改为 `/dev`	(7,1) 引用 1→2，(7,4) 引用 1→0
`fs_set_root()`	不变	`fs_root` = (5,2)；(7,1) 只剩 `m[0]` 持有，路径上已不可达

fstab 里的 ext2 那一行. 阶段三 `/init` 照常执行 `ext2 / mnt/ ext2 ext2 rw 0 0`。`ext2_get_sb()` 的 `lookup_bdev("ext2")` 失败（source 不是设备节点），于是取已经打开的 ext2 实例，得到 dev 5；`getsuper(5)`



图 6.16: mtab 的变化以及 mtab 指向的 itable inode：每一行是一次挂载操作

已存在，不再 `fill_super`；最后 `fs_mount()` 发现 `mount_of_dev(5)` 已是 `m[0]`，拒绝挂第二次。

`exec` 和当前目录本来就按 inode 工作（6.5.5 节），`init`、`sh`、`login` 不用改。启动日志：

```
Kernel command line: root=/dev/mmcblk0p3 rootfstype=ext2
rootdev: root=/dev/mmcblk0p3 rootfstype=ext2 -> mmcblk0p3
ext2: p3 ready rw start=1884160 sectors=248463360 block=4096 groups=948
VFS: Mounted root (ext2 filesystem) on device 5 (/dev/root = mmcblk0p3).
devtmpfs: mounted
VFS: root moved over rootfs, chroot done; running /init
```

ext2 分区里要有 `/init` 和 `/bin` 下的程序。`make install-rpi3-ext2root RPI3_EXT2_DEV=/dev/rdisk4s3` 先用 `mke2fs` 重建 `fs_ext2.img`，再用 e2fsprogs 的 `debugfs` 写入 `/init`、`/bin/*`、`/lib/libc.so`、`/lib/ld-xv6.so` 和模块，最后用 `dd` 覆盖指定分区。因此该命令**会清除**分区原有文件；`/etc` 下的配置由 `init` 第一次启动时创建。

限制：ext2 驱动没有实现设备节点和硬链接（`ext2_iops` 里没有 `link`，`create` 拒绝 `T_DEVICE`），所以 `mknod` 只在 devtmpfs 的 `/dev` 里有效；`/etc/fstab` 里“ext2 挂到 `/mnt/ext2`”那一行会失败，因为同一个文件系统已经挂在 `/`，挂载表不允许挂两次，启动时会看到一行 `vfs: cannot mount ext2 at /mnt/ext2`，不影响使用。

6.8.10 阶段三：`/init` 与用户态看到的结果

initcode 执行 `exec("/init")`。这时 `namex()` 从 `fs_root` 出发，用根文件系统自己的 `lookup`（ext2 的 `e2_lookup()`）找到 `/init`。`/init` 先打开 `/dev/ttyS0`：devtmpfs 已经提供了节点，它不再 `mknod`，只有在打不开时（旧内核、没有 devtmpfs）才退回在根文件系统上建静态节点。然后 `mount_fstab()` 读 `/etc/fstab`，经 `sys_mount()` → `vfsmount()` 为 procfs、FAT32 和 ext2 登记 `super_block`，再用 `fs_mount()` 挂进同一张挂载表。

挂载表的内容可以从 procfs 看到：`/proc/mounts` 就是 `fs_mounts_format()` 的输出，`ro/rw` 取自各自的

`super_block`；`/proc/cmdline` 是 `cmdline_get_all()` 的内容：

```
$ cat /proc/cmdline
root=/dev/mmcblk0p3 rootfstype=ext2
$ cat /proc/mounts
/dev/root / ext2 rw 0 0
devtmpfs /dev devtmpfs rw 0 0
proc /proc procfs ro 0 0
bootfs /boot fat32 rw 0 0
```

rootfs（ram0）被根文件系统盖住以后不再列出，和新版 Linux 一致；`/dev/root` 这个名字只存在于 rootfs 里，正是 Linux 的习惯。`/etc/fstab` 里的 `/mnt/ext2` 因为 ext2 已经是 `/` 而没有挂上，所以不在表里。

6.8.11 用 fs-trace 看 rootfs 与 devtmpfs

前面几节讲的是代码怎么走，这一节换个角度：打开 `include/kernel/fs_trace.h` 里的跟踪开关，启动一次，把内核打印的 `fs-trace:` 日志整理出来，看看启动结束时两个 RAM 文件系统在“磁盘”上到底是什么样子。日志里有这几类记录：

- `fill_super`：读块 1 得到的磁盘 superblock 各字段；`register_super`：登记到 VFS 的 `super_block`；
- `iget_path/xv6_read_inode`：inode 缓存是否命中，以及 dinode 在哪个磁盘块的第几个位置；
- `bmap`：文件块号经 `addrs[]` 换成磁盘块号；
- `dirlookup`：逐项读出的目录项（偏移、名字、inode 号），以及命中或未命中。

下面看 rootfs（dev 7）和 devtmpfs（dev 8）。它们是内核在启动时用 `ramfs_mkfs()` 在内存盘 ram0、ram1 上现做出来的，内容随启动生成、重启即消失，各有自己的用途。两者借用了同一种磁盘格式（xv6fs）和同一张函数表 `xv6_iops`，所以日志里的 `type` 都是 `xv6fs`。ext2 根文件系统的盘上布局另见第 10 章。表 6.11 是两者的 superblock。

表 6.11: 日志中 rootfs 与 devtmpfs 的 superblock（`fill_super` 与 `register_super`）

字段	rootfs（dev 7）	devtmpfs（dev 8）	含义
`magic`	10203040	10203040	格式标识
`size`	32	64	总块数
`nblocks`	27	57	数据块数
`ninodes`	16	48	inode 数
`nlog`	0	0	日志块数
`logstart`	2	2	日志起始块
`inodestart`	2	2	inode 区起始块
`bmapstart`	4	6	位图起始块
来源	ram0	ram1	
`rootino`	1	1	VFS 根 inode 号
日志所在行	1	215	

每块 1024 字节，dinode 64 字节，所以一块放 16 个 dinode；inode 区占 $\lfloor \texttt{ninodes}/16 \rfloor + 1$ 块，位图占 $\lfloor \texttt{size}/8192 \rfloor + 1$ 块，其后才是数据区。按这个规则算出的区间见表 6.12。数据区的第一块分别是 5、7，正好是两个根目录 inode 的 `addrs[0]`——日志里 `root inode loaded` 之后打印的就是这个值，两边对得上，说明 `ramfs_mkfs()` 确实是按这个布局写的。

表 6.12: 按 superblock 算出的块区间

	boot	super	日志	inode 区	位图	数据区
rootfs（dev 7）	0	1	–	2–3	4	5–31
devtmpfs（dev 8）	0	1	–	2–5	6	7–63

由 inode 号也能直接算出 dinode 的位置：块号 = `inodestart` + ⌊ino/16⌋，块内第 `ino` mod 16 个，字节偏移再乘 64。例如 devtmpfs 的 `/dev/input` 是 ino 5，在块 2 + ⌊5/16⌋ = 2 的第 5 个，字节偏移 320。

下面两张图把日志里读到过的 inode 和块画在一起。左边是 inode 区，每行一个 dinode：槽号、inode 号、路径和类型；右边是数据区，目录块逐项列出目录项。黑色箭头是 inode 的 `addrs[]` 指向数据块，蓝色箭头是目录项里的 inode 号指回 inode 区；绿色底的目录项是这次启动时新建的。读法和 ext2 的 inode 块映射图一样，只是 xv6 的 inode 只有 12 个直接块和 1 个一级间接块。

rootfs（dev 7）. 这是 `prepare_namespace()` 第 1 步在 ram0 上造出的临时根，只有 32 块。图 6.17 里能看到整棵树：

```
/          ino 1   块 5    目录项：dev -> 2 (off 64)   root -> 4 (off 96)
  dev/     ino 2   块 6    目录项：console -> 3 (off 64)   root -> 5 (off 96)
    console  ino 3   T_DEVICE   第 1 步建出的 /dev/console
    root     ino 5   T_DEVICE   第 3 步建出的 /dev/root，次设备号 = ROOT_DEV = 5
  root/    ino 4   块 7    只有 . 和 ..，后来被 ext2 根盖住
```

五个 dinode 都在 inode 块 2 的第 1 ~ 5 个位置，三个目录各占一个数据块。所有目录项都是这次启动建的：日志里每次建立都先是一次 `dirlookup: miss`，然后 `ialloc` 分配新 inode，新目录还会接着写入自己的 `.` 和 `..`。`nlog=0`，没有日志区：`log_write()` 发现块不属于日志设备，直接写穿，这正是 6.8.8 节说的 RAM 盘不走日志。

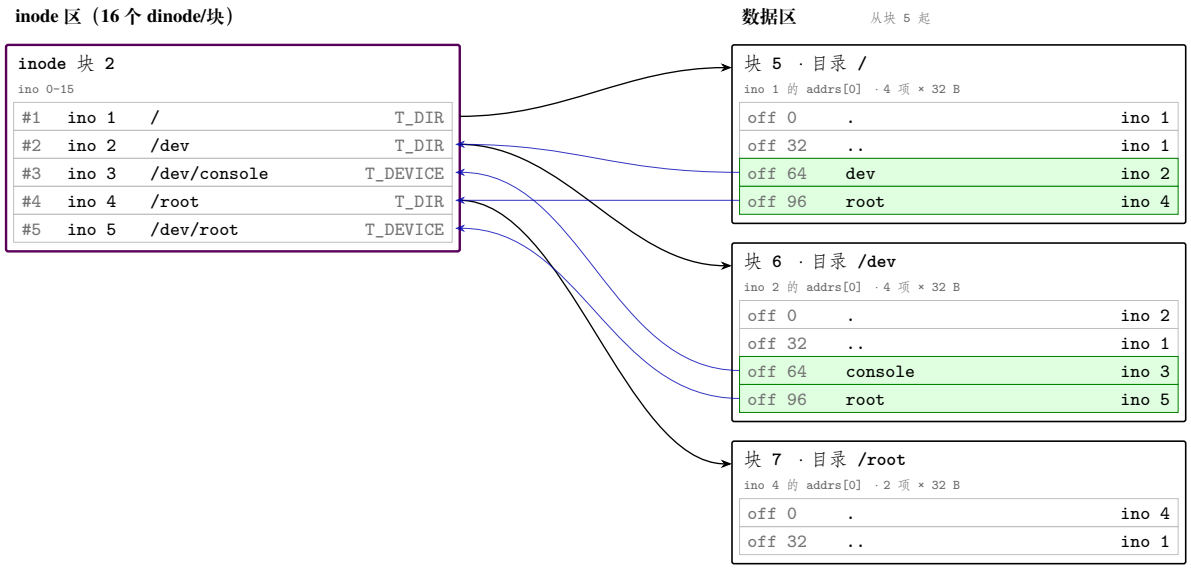


图 6.17: rootfs（dev 7）的磁盘映射：/、/dev、/root 三个目录块与 inode 块 2

读图 6.17：inode 号、槽号与目录项偏移. 图里有三种编号，初读容易混在一起。

inode 号是 inode 的编号。`fs.h` 定义 `ROOTINO` 为 1：编号 0 不用，因为目录项里 `inum==0` 表示空槽，所以根目录取第一个可用号 1。`ramfs_mkfs()` 把根写在 ino 1，之后 `ialloc()` 从 1 往上找第一个空闲 dinode，编号就按创建顺序发出：`/dev` 是 2，`/dev/console` 是 3，`/root` 是 4，第 3 步的 `/dev/root` 是 5。ext2 的约定不同，inode 1 留给坏块表，根是 2；所以 VFS 判断“是不是根目录”时比较的是各自 `super_block` 的 `rootino`，而不是写死的 1。

槽号（图左的 `#N`）是 dinode 在 inode 块里的位置，完全由 inode 号算出，dinode 自己并不存编号：

$$\text{块号} = \texttt{inodestart} + \lfloor \text{ino}/16 \rfloor, \qquad \text{槽号} = \text{ino} \bmod 16, \qquad \text{块内字节} = \text{槽号} \times 64.$$

rootfs 只有 16 个 inode，全在块 2，所以 `#N` 恰好等于 ino N；`#0` 对应 ino 0，不用。inode 区从哪一块开始看 `inodestart`：rootfs、devtmpfs 都是块 2（前面只有 boot 和 superblock，没有日志区）。

目录项偏移（图右的 `off N`）是目录项在目录数据块里的位置，每项 32 字节，按写入顺序排。每个目录创建时先写 `.`（指自己）和 `..`（指父目录），所以 `/dev` 的块 6 第一项 `.` 指向 ino 2，第二项 `..` 指向 ino 1；根目录没有父目录，它的 `..` 指回自己。目录项和槽号靠目录项里写的 inode 号连起来，两者顺序并不一致：块 6 的 off 32 对应 `#1`，off 96 对应 `#5`。

还要注意图里有两个都叫 `root` 的目录项：块 5（`/` 的内容）off 96 的 `root` 是目录 `/root`（ino 4，挂根文件系统用的挂载点），块 6（`/dev` 的内容）off 96 的 `root` 是设备节点 `/dev/root`（ino 5）。inode 只用 `addrs[]` 指向数据块，指向 inode 的只有目录项，所以从 ino 1 到 ino 5 要走两层，这正是日志第 2127 行查找 `/dev/root` 的过程：

$$\text{ino } 1 \to \texttt{addrs[0]} \to \text{块 } 5 \to \texttt{dev} \to \text{ino } 2 \to \texttt{addrs[0]} \to \text{块 } 6 \to \texttt{root} \to \text{ino } 5$$

dinode 的 64 字节。 “inode 1” 同时指三样东西：编号 1；磁盘块 2 第 1 个槽里的 64 字节 `struct dinode`；`ilock` 之后 `itable[]` 里的内存副本 `struct inode`（多了 `dev`、`ref`、锁和 `iop`）。`iget(7, 1)` 只在 `itable` 里记下 dev 7、ino 1，置 `valid=0`，并不读盘；`ilock` 发现 `valid==0` 才调用 `xv6_read_inode()`，按上面的公式读块 2、取出偏移 64 处的 dinode：

```
struct dinode {            // kernel/fs.h
  short type;              // 2 B
  short major;             // 2 B
  short minor;             // 2 B
  short nlink;             // 2 B
  uint  size;              // 4 B
  uint  addrs[NDIRECT+1];  // 13 x 4 = 52 B: addrs[0..11] 直接块, addrs[12] 一级间接块
};                         // 共 64 B, IPB = 1024 / 64 = 16
```

rootfs 的 ino 1 在启动结束时（4 个目录项，`size=128`）逐字段如表 6.13。`nlink` 是 3：`ramfs_mkfs()` 做好时为 1（日志第 23 行的 dump 是在建 `/dev` 之前打印的，那时 `size` 还是 64、`nlink` 还是 1），之后 `xv6_create()` 每建一个子目录就给父目录 `nlink++`，因为子目录的 `..` 也指向它；建了 `/dev`、`/root` 两个子目录，就是 1 + 2 = 3。根目录由两部分组成：ino 1 这 64 字节是关于它的信息，块 5 的 1024 字节是它的内容（目录项表），两者靠 `addrs[0]=5` 连起来；图 6.17 右边写 “块 5 · 目录 /”，意思是 “ino 1 的 `addrs[0]` 指向的内容块”。

表 6.13: rootfs 中 ino 1 的 dinode（块 2，偏移 64–127）

dinode 内偏移	块 2 内偏移	字段	值	小端字节
0	64	`type`	1（`T_DIR`）	`01 00`
2	66	`major`	0	`00 00`
4	68	`minor`	0	`00 00`
6	70	`nlink`	3	`03 00`
8	72	`size`	128	`80 00 00 00`
12	**76**	`addrs[0]`	**5**	`05 00 00 00`
16	80	`addrs[1]`	0	`00 00 00 00`
⋮	⋮	⋮	0	⋮
60	124	`addrs[12]`	0（无间接块）	`00 00 00 00`

把块 2 从偏移 64 开始 dump 出来就是：

```
块2+64:  01 00 00 00 00 00 03 00  80 00 00 00 05 00 00 00
块2+80:  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
块2+96:  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
块2+112: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
```

`addrs[0]=5` 就在块 2 内第 76–79 字节，即整个 ram0 的第 $2 \times 1024 + 76 = 2124$ 字节。`xv6_read_inode()` 用 `memmove` 把 `addrs[]` 拷进内存 inode 之后，它就有了两份：日志里 `root inode loaded` 下面打印的、`bmap()` 查

块号时用的，都是内存那一份。

块 2 的全部 16 个槽. inode 区不按目录分组，它是整个文件系统一张平铺的表，dinode 放在哪一块只看 inode 号。rootfs 的 `ninodes` 是 16，`ialloc()` 只会发出 ino 1–15，所以整个 rootfs 的对象，不论在第几层目录，都在块 2；按公式算出的块 3 对应 ino 16–31，这些号不存在，块 3 一直为空。`/dev/console` 在块 2，只因为它的号是 3。表 6.14 是 `prepare_namespace()` 结束时块 2 的全部内容：`type=0` 的槽是空闲的，`ialloc()` 找的就是它们；两个设备节点只有 `major/minor`，没有数据块；目录的 `size` 等于目录项数乘 32。层级关系只记在数据区的目录项里，dinode 本身没有“父目录”字段，表中的路径是从目录树反推出来标上的。

表 6.14: rootfs 块 2 的 16 个 dinode（`prepare_namespace()` 结束时）

槽	块内偏移	inode	路径	type	major	minor	nlink	size
#0	0–63	0	不用	0	0	0	0	0
#1	64–127	1	/	1 T_DIR	0	0	3	128
#2	128–191	2	/dev	1 T_DIR	0	0	1	128
#3	192–255	3	/dev/console	3 T_DEVICE	1 CONSOLE	0	1	0
#4	256–319	4	/root	1 T_DIR	0	0	1	64
#5	320–383	5	/dev/root	3 T_DEVICE	7 BLOCKDEV	5	1	0
#6–#15	384–1023	6–15	空闲	0	0	0	0	0

`addrs[0]` 只有三个目录非零：#1 为 5，#2 为 6，#4 为 7，其余全为 0。

devtmpfs（dev 8）. `devtmpfs_init()` 在 64 块的 ram1 上为 11 个设备建节点（图 6.18）。根目录块 7 里依次是 `console`、`tty`、`ttyS0`、`input`、`video0`、`mmcblk0`、`mmcblk0p1 ~ p3`、`ram0`、`ram1`，共 13 项 416 字节；`input` 是唯一的子目录（ino 5，块 8），里面只有 `event0`。设备节点都是 `T_DEVICE`，不占数据块，只在 dinode 里记 `major/minor`。

日志还暴露了一个原版 xv6 就有的现象：建每个节点时，根目录 8:1 都是 `iget` 未命中、重新从块 2 读 dinode，前后 11 次。原因是每建完一个节点，根目录的引用就降到 0，而 xv6 的 inode 表只认 `ref>0` 的项，引用为 0 的槽下次就被当作空槽重新初始化，不保留缓存的内容。

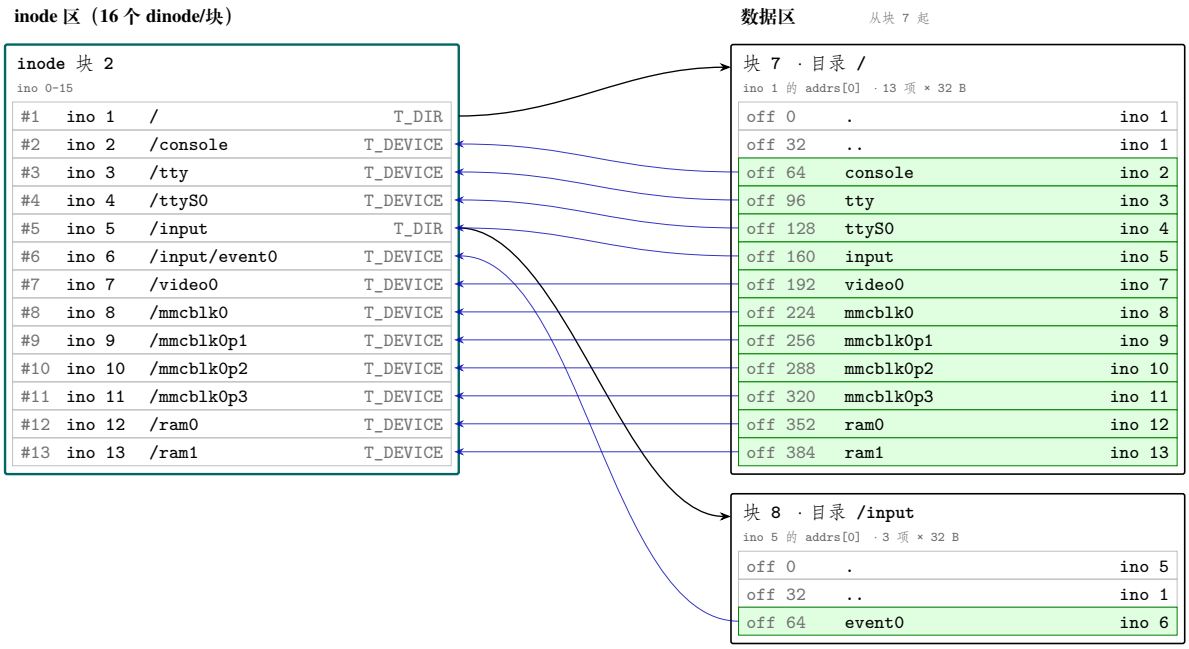


图 6.18: devtmpfs（dev 8）的磁盘映射：13 个 inode 都在 inode 块 2，根目录块 7 与 input 目录块 8

6.9 烧录 SD 卡

6.9.1 分区布局

前面几节讲的是内核怎样找到并挂上根文件系统；最后看这张 SD 卡在宿主机上怎样做出来。一张为 xv6 准备的卡分成四个区（表 6.15），本书的配置以 p3 上的 ext2 为根。

表 6.15: SD 卡分区（63.8 GB 卡的实例）

分区	名称	格式	起始 LBA	用途
p1	BOOTFS	FAT32	2048	固件、`config.txt`、`cmdline.txt`、内核、Wi-Fi 固件；挂到 `/boot`
p2	XV6FS	原始	208896	xv6fs 镜像（400 MiB，用到 32 MiB）；其后的空间放 ext2 的外置日志 xjournal（第 10 章）
p3	EXT2	ext2	1030144	根文件系统（`root=/dev/mmcblk0p3`）
p5	SPFS	原始	123654144	预留给其他教学文件系统

6.9.2 步骤

以 macOS 为例（`disk4` 必须先用 `diskutil list` 确认，写错磁盘会毁掉数据）：

```
# 1. 卸载整张卡并重新分区
diskutil unmountDisk force /dev/disk4
diskutil partitionDisk /dev/disk4 4 MBR \
  FAT32 BOOTFS 100MiB  UFSD_EXTFS XV6FS 400MiB \
  UFSD_EXTFS EXT2 R    UFSD_EXTFS SPFS 500MiB

# 2. 写入 p2 上的 xv6fs 镜像（只能写分区 2，绝不能写整卡 /dev/rdisk4）
make fs.img
make install-rpi3-rawfs RPI3_XV6_DEV=/dev/rdisk4s2

# 3. 在 BOOTFS 中放入树莓派官方固件（bootcode.bin、start.elf、fixup.dat 等）

# 4. 生成 fs_ext2.img 并 dd 到 p3，同时把内核、config.txt、
#    cmdline_ext2.txt（装成 cmdline.txt）和 Wi-Fi 固件装进 BOOTFS
make install-rpi3-ext2root RPI3_EXT2_DEV=/dev/rdisk4s3
```

`Makefile` 对设备变量做了检查：`RPI3_EXT2_DEV`、`RPI3_XV6_DEV` 必须是 `/dev/diskNsM` 或 `/dev/rdiskNsM` 形式的分区设备，指向整卡则拒绝执行；BOOTFS 没有挂载也会报错。这些防护是吃过亏之后加上的——`dd` 写错一个字母就会覆盖整张卡的分区表。

6.9.3 日常更新

改了内核之后不必每次拔卡：运行 `make`，内核会被拷到宿主机的 TFTP 目录；在树莓派上执行

```
tftp 192.168.0.195 kernel8-xv6_wifi.img
```

文件就会下载到 `/boot`（FAT32 启动分区），重启后生效。用户程序变化时，重新执行 `make install-rpi3-ext2root` 把新的 `fs_ext2.img` 写到 p3；只有 xv6fs 格式本身变化时，才需要重新烧录 p2。

到这里，从 SD 卡扇区到 `open("/bin/sh")` 的整条路径都已经打通。下一章转向挂在各条总线上的设备：块设备只是其中一种，控制台、USB、网络和摄像头各有自己的中间层，但都建立在同一个设备模型之上。

第七章 驱动架构

本章内容

- Linux 2.6 风格的总线/设备/驱动模型
- kobject 对象模型与 sysfs：/sys 视图、属性文件、uevent 与 /sys/class 符号链接
- 中断上半部与 workqueue 下半部
- 控制台、Mini UART 与 TTY
- USB 与 HID 键盘：描述符、枚举、split 事务、URB、真机排错
- I2C（BSC）
- 网络协议栈
- SDIO 无线网卡 BCM43430/43455
- USB 无线网卡 MT7601U
- CSI-2 摄像头 OV5647：D-PHY、CSI-2 包、Unicam FIFO、DMA 双缓冲与帧通知，HTTP MJPEG 推流及其流水线提速
- Windows 风格窗口系统：消息驱动、win32k、USER32/GDI 与窗口合成

第 6 章的块设备只是众多设备中的一种。本章先搭两副“骨架”：设备模型回答设备、驱动和总线怎样互相找到，上下半部回答中断里来不及做的工作交给谁。然后按依赖关系依次看几类驱动：最早需要的串口控制台与 TTY；协议层次最多的 USB 键盘；只用来配置传感器的 I2C；网络协议栈和挂在它下面的两块无线网卡（一块走 SDIO，一块走 USB）；最后是把 I2C、DMA、中断和用户态图像处理串在一起的 CSI-2 摄像头。

7.1 设备模型

7.1.1 为什么不再只用 devsw[]

原版 xv6 只有一张字符设备函数表：inode.major 直接索引 devsw[major].read/write。它够用来驱动一个控制台，但表达不了“硬件设备”“驱动程序”“总线”以及“二者何时绑定”。随着驱动从一个变成十几个，项目引入了 Linux 2.6 设备模型的最小子集（include/kernel/device.h）：

```
struct bus_type {
  char *name;
  int  (*match)(struct device *dev, struct device_driver *drv);
  int  (*probe)(struct device *dev);
  void (*remove)(struct device *dev);
};
struct device_driver {
  char *name;  struct bus_type *bus;
  int  (*probe)(struct device *dev);
  void (*remove)(struct device *dev);
};
struct device {
  char *name;  int id;  struct device *parent;  struct bus_type *bus;
  struct resource resource[DEVICE_RES_MAX];  int nresource;  // MMIO、IRQ
  struct device_driver *driver;  void *driver_data;
};
struct file_operations {
  int  (*read)(int user_dst, uint64 dst, int n);
  int  (*write)(int user_src, uint64 src, int n);
  int  (*open)(struct file *f);                 // 可选：每次 open 的私有状态
  void (*release)(struct file *f);
```

```
  int  (*fread)(struct file *f, int user_dst, uint64 dst, int n);
};
```

设备和驱动可以按任意顺序注册：先注册的一方等着，后注册的一方触发总线扫描，`match()` 成功就调用驱动的 `probe()`。驱动在 `probe()` 中初始化硬件，然后用 `register_chrdev(major, name, fops)` 注册字符设备，或者 `register_netdev()` 注册网卡。

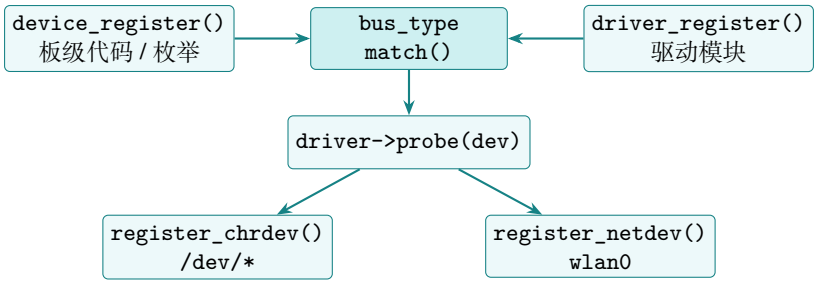


图 7.1: 设备与驱动的绑定

项目中有三条总线：

- **platform**：SoC 内固定的设备（Mini UART、SDHOST、Arasan、DWC2），无法枚举，由板级代码静态描述，按名字匹配；
- **usb**：DWC2 枚举出的 USB 设备，按 VID/PID 或接口类别匹配；
- **sdio**：Arasan 枚举出的 SDIO function，按 vendor/device/function 匹配。

字符设备的主设备号见表 7.1。设备节点由内核的 devtmpfs 按已注册的驱动自动创建（`do_mounts.c`），`init` 只在 devtmpfs 不可用时才用 `mknod` 补建；`cat /proc/devices` 可以看到全部总线、设备、资源和已绑定的驱动。

表 7.1: 字符设备主设备号

major	名称	节点
1	console	/dev/console
2	tty（当前进程的控制终端）	/dev/tty
3	ttyS0（Mini UART）	/dev/ttyS0
4	input（evdev）	/dev/input/event0
5	camera	/dev/video0

7.1.2 kobject：一切皆对象

上面对象只是“数据”，要在运行时看见并操作它们，就得把它们挂上一棵统一对象树，再由一棵伪文件系统把树渲染成目录与文件。Linux 用 `kobject`（`lib/kobject.c`）与它的渲染器 sysfs（`fs/sysfs`）完成此事，本工程照搬同一分层（`kernel/drivers/base/kobject.c`、`kernel/fs/vfs.c`）。

`/sys` 里没有“设备目录”“驱动文件”的特判：唯一的节点类型就是 `struct kobject`——一个带名字、父指针、引用计数与 ktype 的对象。`bus_type`、`device`、`device_driver` 各自**内嵌**一个 kobject 并给出一个 `kobj_type`，于是 `/sys/bus`、`/sys/devices`、驱动目录全都出自同一个通用遍历器：

```
struct attribute { char *name; int mode; };      // 一个 sysfs 文件
struct sysfs_ops {
  int (*show)(struct kobject*, struct attribute*, char *buf, int size);
  int (*store)(struct kobject*, struct attribute*, char *buf, int size);
};
struct kobj_type {
  void (*release)(struct kobject*);               // refcount 归零时回收
  struct sysfs_ops *sysfs_ops;                    // show/store 分派
```

```
  struct attribute **default_attrs;              // 该类对象的默认属性
};
struct kobject {
  char name[KOBJ_NAME_MAX];  struct kobject *parent;
  struct kobj_type *ktype;   struct attribute **attrs; // 实例属性叠加在默认之上
  struct kobject *link;      // 非空 => 本节点是指向 link 的符号链接
  int refcount;              // 内联的 Linux kref
  void *priv;                // 回指外层对象
};
```

kobject_add 把节点挂到父目录并持有父引用；kobject_put 递减引用、归零时调 ktype->release() 再释放对父的引用；kobject_create_and_add 从固定池切一个只读目录节点（/sys/devices、/sys/bus、/sys/class 及每个类目录都由它创建）。

7.1.3 sysfs：把对象树渲染成文件系统

sysfs 不存数据，它只是那棵 kobject 树的一个**渲染视图**。本工程里它和 procfs、netfs 一样是路径式伪文件系统：init 用 sysfs /sys sysfs rw 0 0 把它挂到 SYSFS_DEV，一组 sys_* 钩子把路径翻译成 (kobject, attribute)。

- sys_resolve(path)：按 / 逐段走 kobject_child_by_name；末段若匹配不上子目录，就回退到当前对象的 kobject_find_attr，据此判定“目录 / 属性文件 / 不存在”。遇到 link 节点先解引用（stat-follows-link）。
- sys_vreaddir：先列子 kobject，再列属性文件——正是 Linux sysfs 的 readdir 顺序。
- sys_vread / sys_vwrite：读就是把字节交给 ktype->sysfs_ops->show()；写先按属性的 mode 位拒绝只读文件（对应 sysfs 的 -EACCES），再交给 store()。shell 的 > 带 O_TRUNC，故属性文件的 truncate 是无副作用的 no-op。

文件不落地、只由 (对象, 属性) 现算，所以新增一个注册了 kobject 的驱动就会多出对应的 /sys 文件，渲染器代码一行不改——与 Linux 的 drivers/base/core.c 加 fs/sysfs 分工一致。左半的对象树与右半渲染出的 /sys 视图见图 7.2。

7.1.4 属性文件与 DEVICE_ATTR

每个具体属性是一个 device_attribute（attribute 加一对 show/store），用与 Linux 同名同形的宏声明：

```
static DEVICE_ATTR_RO(ioaddr);        // console.c: MMIO 物理地址，只读
static DEVICE_ATTR_RW(test_pattern);  // ov5647.c: 测试图样，可写
```

设备对象自带的默认属性对齐 Linux：name、subsystem、driver、modalias、uevent；驱动目录有 name；总线目录有 name 与 drivers_probe。

7.1.5 uevent 与 drivers_probe：手工触发热插拔

- uevent(0644)：读它打印该设备的热插拔环境变量(DRIVER=、MODALIAS=)；写一个动作(add/change/remove)相当于把一次 uevent 手工补发一遍——未绑定且能算出 MODALIAS 的设备会走 request_module()，与自动装载链复用同一入口。对应 Linux 的 kobject_uevent()，只是省掉了 netlink 到用户态 udev 的传递。
- drivers_probe（0200）：往 /sys/bus/<bus>/drivers_probe 写一个设备名，总线重新扫描并在设备未绑定时尝试 bind()，与 Linux 用它“重新触发一次匹配”的行为一致。

7.1.6 /sys/class 与符号链接

/sys/bus 反映总线拓扑，而 /sys/class/< 类名 > 反映**功能分组**：跨总线、干同一件事的设备链接到一起（tty、video4linux……）。设备若声明了 class，device_register() 就懒建类目录并放一个**符号链接**指回 /sys/devices 下的真身（Linux device_add_class_symlinks / sysfs_create_link）：

```
/sys/class/tty/bcm2837-mini-uart  ->  /sys/devices/bcm2837-mini-uart
/sys/class/video4linux/ov5647     ->  /sys/devices/ov5647
```

渲染器在解析路径时对链接解引用，所以顺着类链接一路 cat 到设备属性完全透明。为把链接显式呈现，本工程补了一个最小 readlink(2)（SYS_readlink，仅 sysfs 实现了 readlink inode 钩子，其它文件系统如同 Linux 对非链接返回 EINVAL），ls 据此打印 name -> target。

笔记 与 Linux 的差异：本工程没有通用 VFS 符号链接与 lstat，链接只在 sysfs 渲染器内部解引用；readlink 返回绝对路径 /sys/devices/...，而 Linux 返回相对路径 ../../devices/...。此外 uevent 未走 netlink 用户态 udev，/sys/.../power 电源管理尚未实现。

7.2 上半部与下半部

中断处理程序运行时，本 CPU 的中断是关着的，而且它打断的可能是任何代码，所以不能睡眠、不能长时间运行。真正耗时的工作——解析 USB 报告、执行几十次 SDIO 命令、处理一个 TCP 段——要推迟到可以调度、可以睡眠的上下文中去做。项目实现了与 Linux 同名的 workqueue（include/kernel/workqueue.h）：

```
struct work_struct {
  void (*func)(struct work_struct *work);
  struct work_struct *next;
  int pending, running, rerun;
  struct workqueue *wq;
};
int  queue_work(struct workqueue *wq, struct work_struct *work);
int  queue_delayed_work(struct workqueue *wq, struct delayed_work *dw, uint64 delay);
void cancel_work_sync(struct work_struct *work);
```

每个 workqueue 有自己的内核线程（kworker），在 ps 中可以看到。几条原则贯穿所有驱动：

- 上半部只做三件事：确认中断来源、屏蔽或清除该中断、queue_work()；
- 每个设备使用私有的 work 和 workqueue，慢设备（如 SDIO 的 CMD53）不会阻塞别的下半部；
- 同一个 work 在执行期间再次被排队，只记一个 rerun 标志，不会无限堆积；
- 移除设备时先 cancel_work_sync()，等正在运行的 work 返回后才释放设备内存。

定时器中断每 10 ms 推进一次 delayed-work 时钟，TCP 重传、Wi-Fi 连接超时等都用 delayed work 实现，而不是在定时器中断里直接运行协议代码。网络接收还在 workqueue 之上实现了 NAPI 式的预算轮询：中断只调度一次，下半部每轮最多处理 16 个报文，没处理完就重新排队，处理完才重新打开中断。

7.3 控制台、UART 与 TTY

有了设备模型和下半部，就可以看具体驱动了。先从最简单、也是启动后最早用到的一个开始：串口控制台。它只有一根收线和一根发线，却已经涉及中断生产者与进程消费者、行规程和会话，后面各节的套路在这里都能看到雏形。

7.3.1 Mini UART

Mini UART 驱动（kernel/drivers/tty/uart.c）在 probe() 中完成与早期汇编相同的初始化，再打开接收中断：

```
REGS_AUX->enables |= 1U;                // 使能 AUX 中的 Mini UART
REGS_AUX->mu_control = 0;               // 暂停收发
REGS_AUX->mu_ier = 0;
REGS_AUX->mu_lcr = 3;                   // 8 位数据
REGS_AUX->mu_iir = 0xc6;                // 清空 FIFO
REGS_AUX->mu_baud_rate = 270;           // 115200 @ core_freq 250 MHz
REGS_AUX->mu_ier = MU_IER_RX_ENABLE;    // 接收中断
REGS_AUX->mu_control = 3;               // 打开收发
```

发送使用一个 32 字节的软件环形缓冲区：uartputc() 把字符放进缓冲区，缓冲区满时睡眠；uartstart() 在发送 FIFO 有空位时搬字符过去，并打开“发送空”中断；uartintr() 读空接收 FIFO、把字符交给 consoleintr()，再继续发送。缓冲区空了就关掉发送中断，否则会一直触发。内核 printf 在 panic 等场合使用同步的 uartputc_sync()，不依赖中断。

7.3.2 控制台与行规程

consoleintr() 实现了最基本的行规程：回显、退格、整行缓冲、Ctrl-D 结束输入、Ctrl-P 打印进程表、Ctrl-C 向前台进程组发送 SIGINT。读进程在 consoleread() 里睡眠，直到收到一整行。

7.3.3 TTY 层

/dev/console、/dev/ttyS0、/dev/tty 共用 Mini UART 这一个硬件后端，但含义不同：

- /dev/console 是系统控制台，内核日志直接写 UART，不经过文件系统；
- /dev/ttyS0 是具体的串口终端，init 在上面启动 login；
- /dev/tty 是“当前进程的控制终端”，按调用者的 ctty 动态转发：串口会话转到 ttyS0，SSH 会话转到对应的 PTY 从端。

login 调用 tty_attach() 成为新会话和进程组的组长，并把 ttyS0 设为控制终端；之后 fork 出的 shell 和命令继承会话、进程组和控制终端。shell 运行外部命令时把它设为前台进程组，Ctrl-C 产生的 SIGINT 只发给这个组，所以能终止 ping、tftp 或整条管道，而不会杀掉 shell 和 login。

一个由此暴露的问题是：旧版 ps 通过系统调用让内核 printf 直接打印进程表，结果在 SSH 会话里执行 ps，输出却出现在串口上。内核 printf 写的是系统控制台，不看调用者的 fd 1。现在 sys_ps() 把快照格式化到用户缓冲区，由 ps 自己 write(1, ...)。

7.3.4 绑定：设备、驱动、设备文件与会话

kernel/drivers/tty/ 下四个文件分工明确：uart.c 只管 Mini UART 寄存器、32 字节发送环和中断，自身不暴露任何设备文件；console.c 是终端本体，持有唯一的输入环 cons 和行规程；tty.c 是会话层，管理前台进程组和 /dev/tty 的转发；pty.c 是 SSH 用的伪终端，自带一个迷你行规程。它们在四个地方绑定起来（图 7.3）：

1. **设备 ↔ 驱动**：consoleinit() 先 platform_device_register(&uartdev)，再 platform_driver_register(&uartdrv)，platform_match() 按名字 "bcm2837-mini-uart" 把两者配上，调用 uart_probe()，它执行 uartinit() 并注册 major 1（/dev/console）；
2. **设备文件 ↔ 函数**：ttyinit() 再注册 major 3（/dev/ttyS0，与 console 用同一组 consoleread/consolewrite）和 major 2（/dev/tty，ttycurrentread/write 按 p->ctty 转发）。devtmpfs 为每个设备

节点建 T_DEVICE inode；open() 时 chrdev_open() 用 inode 的 major 查 chrdevs[]，把 fops 填进 f->f_op；

3. **硬件 ↔ 中断**：这一侧是写死的——devintr() 里直接判断 irq == UART0_IRQ 再调 uartintr()，没有走 request_irq，uartdev 的 IRQ 资源只用于展示；
4. **进程 ↔ 终端**：init 打开 /dev/ttyS0 作为 fd 0/1/2；login 调 tty_attach() 设置 sid/pgid/ctty，子进程经 fork 继承；sh 每执行一条命令都用 tty_set_foreground() 切换前台进程组。

内核 printf 不经过这张表：consputc() 直接调 uartputc_sync()。

7.3.5 数据通路与调用链

输入有两个生产者，最终都进入 consoleintr()：UART 走 GIC UART0_IRQ → devintr → uartintr → 逐字节 consoleintr（中断上下文）；USB 键盘走 DWC2_IRQ → usbkbd_irq_complete → queue_work → usbkbd_rx_work → consoleintr（工作队列，进程上下文）。输出有两条路：write() 经 consolewrite 逐字节 uartputc 进 32 字节发送环，环满就睡；printf 和回显经 consputc → uartputc_sync，关中断忙等 FIFO，绕过发送环。图 7.4 是分层，图 7.5 把 read、串口中断输入、write、printf/回显、Ctrl-C 五条调用链逐步标出了文件、行号和拿的锁。

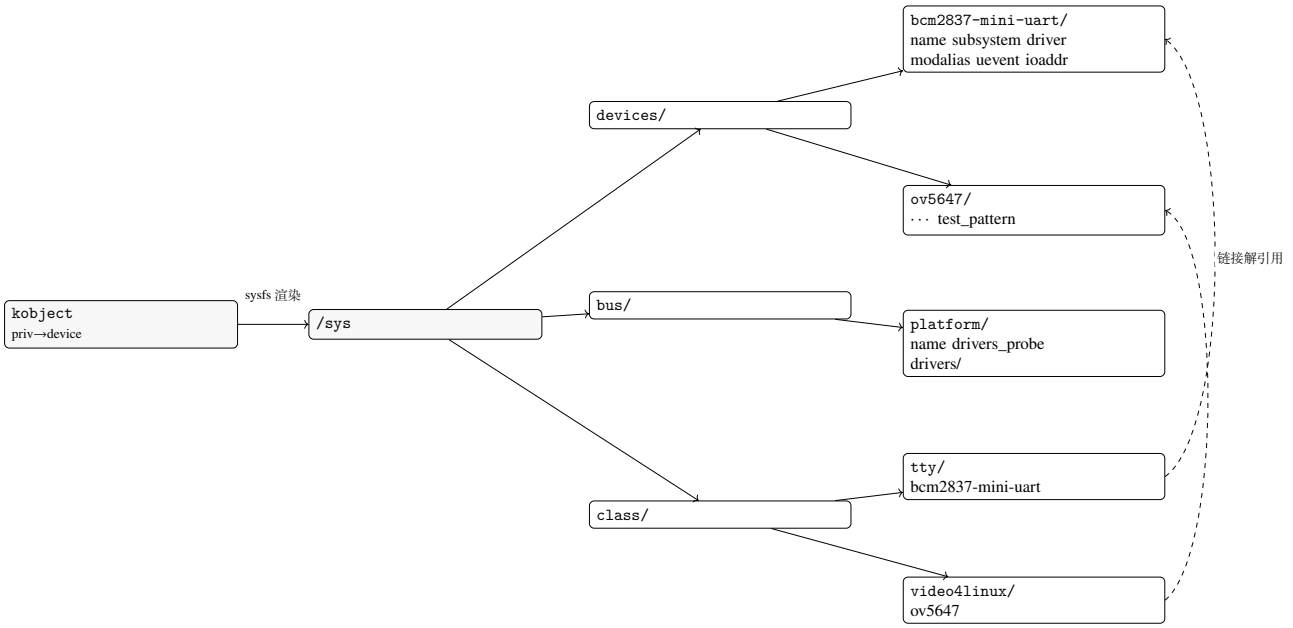


图 7.2: 左：`device.c` 内嵌的 kobject 对象树；右：sysfs 把同一棵树渲染成的 `/sys` 目录。属性文件是 (对象, 属性) 现算出来的视图，`/sys/class` 下则是链接回 `/sys/devices` 真身的符号链接。

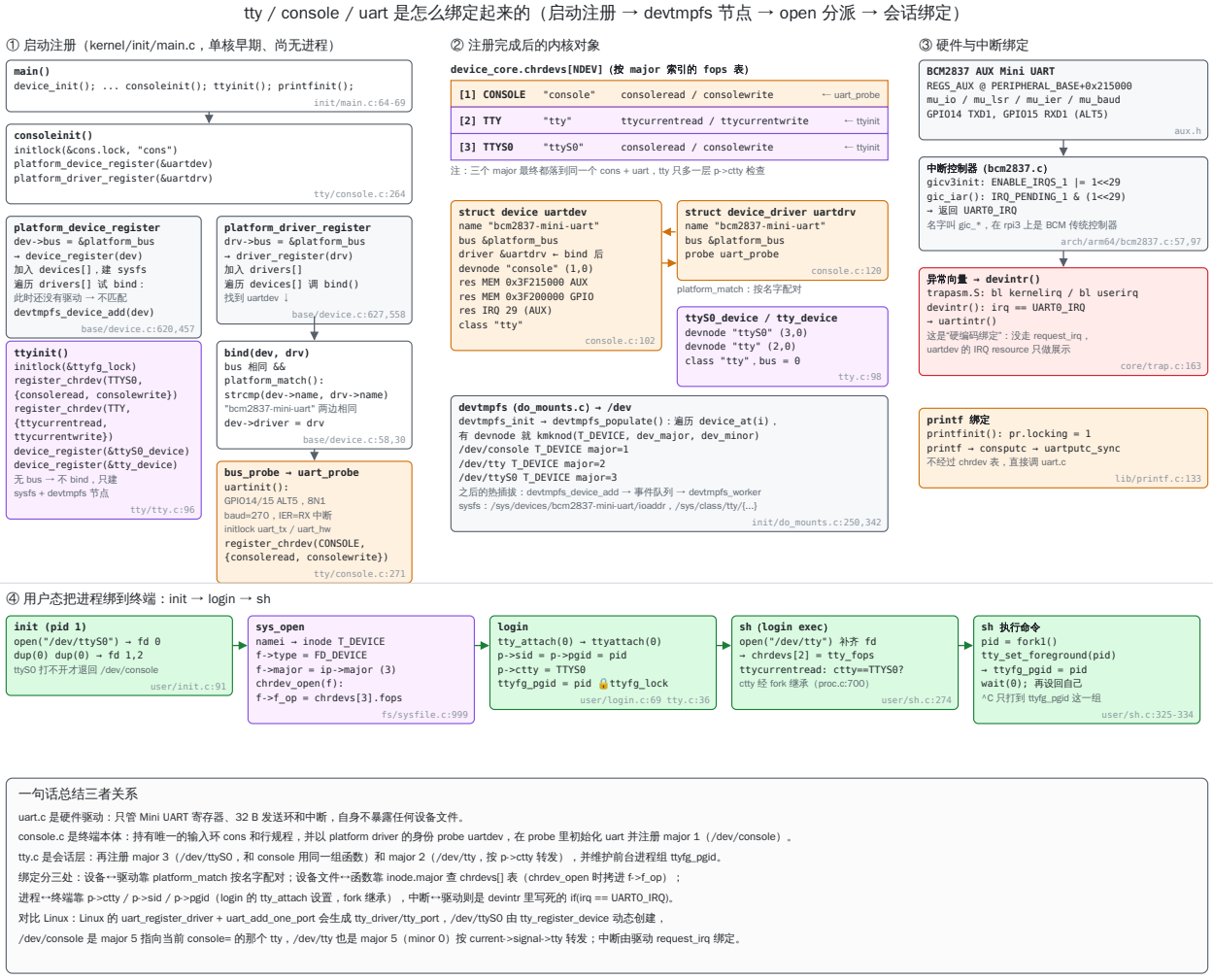


图 7.3: tty、console、uart 的绑定：启动注册、platform 匹配与 probe、`chrdevs[]`、devtmpfs 节点、open 分派、会话绑定

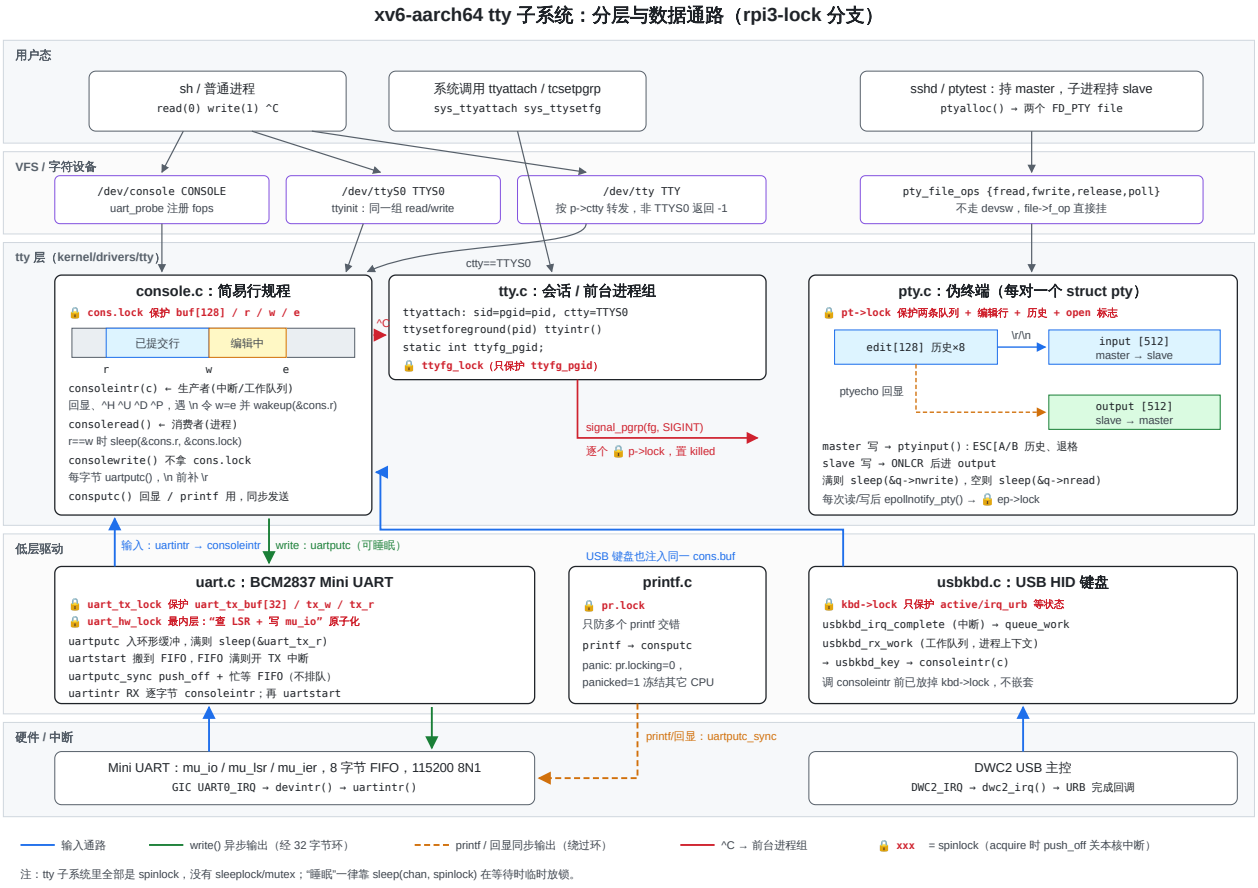


图 7.4: tty 的分层与三条数据通路

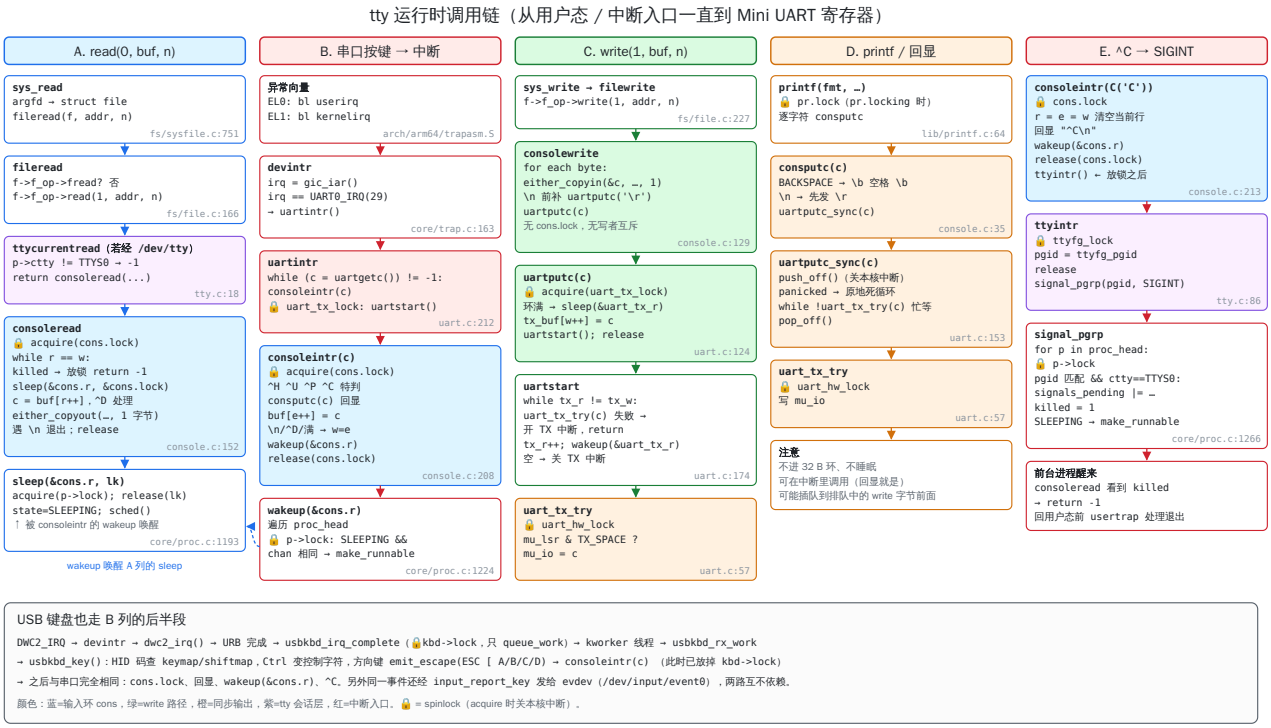
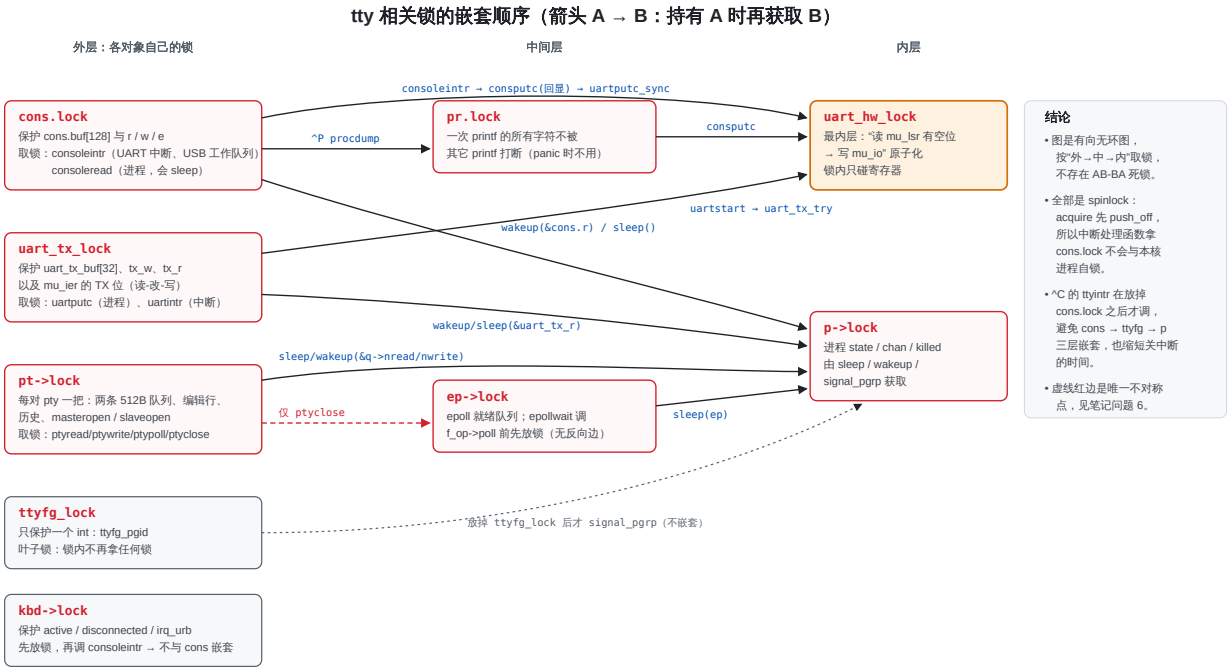


图 7.5: tty 的五条运行时调用链：read、串口中断输入、write、printf/回显、Ctrl-C

7.3.6 锁

tty 子系统里全部是自旋锁，没有 mutex；需要等待时一律 `sleep(chan, lk)`。`acquire()` 先 `push_off()` 关本核中断：`consoleintr` 和 `uartintr` 在中断里运行，若持锁时同一核再进中断去拿同一把锁就会自锁。



sleep / wakeup 通道（sleep(chan, lk) 在 p->lock 保护下放掉 lk，防止丢失唤醒）

chan	配对锁	谁睡（条件）	谁唤醒
`&cons.r`	`cons.lock`	consoleread：r == w（还没有完整一行）	consoleintr：收到 \n / ^D / 缓冲满，或 ^C 清空
`&uart_tx_r`	`uart_tx_lock`	uartputc：tx_w == tx_r + 32（环满）	uartstart：每搬走 1 字节（进程或 TX 中断里）
`&q->nread`	`pt->lock`	ptyread：队列空且对端仍打开	ptywrite / ptyinput 收到 \n / ptyqput 满时 / ptyclose
`&q->nwrite`	`pt->lock`	ptyqput：队列满（512）且读端仍打开	ptyread 读走数据后 / ptyclose
（任意 chan）	`p->lock`	上面任何一个睡眠者	signal_pgrp / kill：置 killed 后 make_runnable，读者醒来检查 killed 返回 -1

图 7.6: tty 的锁、嵌套顺序与 sleep/wakeup 通道

表 7.2: tty 相关的锁

锁	保护什么
`cons.lock`	128 字节输入环 `cons.buf` 与 `r/w/e` 下标；中断、工作队列、进程都会拿
`uart_tx_lock`	32 字节发送环、`tx_w/tx_r`、`mu_ier` 的发送中断位
`uart_hw_lock`	把“查 `mu_lsr` 有空位”和“写 `mu_io`”合成一步，防止两个核抢到最后一个 FIFO 槽而丢字节；最内层
`pr.lock`	一次 `printf` 的字符不与别的 `printf` 交错；panic 时不再加锁
`ttyfg_lock`	只保护前台进程组 `ttyfg_pgid`；叶子锁
`pt->lock`	一对 PTY 的两条 512 字节队列、编辑行、历史、打开状态
`kbd->lock`	USB 键盘的 `active/disconnected/irq_urb`；叶子锁

嵌套顺序（A → B 表示持 A 再拿 B）是一张有向无环图（图 7.6），所以不存在 AB-BA 死锁：

```
cons.lock     -> uart_hw_lock            回显 consputc
cons.lock     -> pr.lock -> uart_hw_lock Ctrl-P procdump
cons.lock     -> p->lock                 sleep / wakeup
uart_tx_lock -> uart_hw_lock             uartstart
uart_tx_lock -> p->lock                  sleep / wakeup
pt->lock      -> p->lock;  pt->lock -> ep->lock (仅 ptyclose)
ep->lock      -> p->lock                 epollwait 的 sleep
```

有几处是刻意这样写的：`consoleintr` 处理 `Ctrl-C` 时先放 `cons.lock` 再调 `ttyintr()`，避免形成 cons → ttyfg → 所有 `p->lock` 的长链；`ttyintr` 读出 pgid 后就放掉 `ttyfg_lock`，再 `signal_pgrp`；`usbkbd_rx_work` 先放 `kbd->lock` 再注入字符；`epollwait` 先放 `ep->lock` 再调 `f_op->poll`。

为什么 sleep 不丢唤醒。 `consoleread` 持 `cons.lock` 发现 `r == w`，调 `sleep(&cons.r, &cons.lock)`，它先拿 `p->lock` 再放 `cons.lock`；另一个核上的 `consoleintr` 要 `wakeup` 就必须拿到这个 `p->lock`，而它要等 `sched()` 切走后才由调度器释放。所以“检查条件”和“变成 SLEEPING”对唤醒方是原子的（图 7.7(a)）。`uartputc` 睡在 `&uart_tx_r`，PTY 睡在 `&q->nread/&q->nwrite`，结构相同。

持锁拷贝与 mmap。 `consoleread` 和 PTY 的读写在自旋锁下 `copyout/copyin`。有了 mmap 的按需调页之后，目标页可能还没缺页进来；`uva2ka_fault()` 发现调用者持有自旋锁就直接失败而不睡眠（第 4.10 节）。`sys_read/sys_write` 进入驱动之前已经用 `fault_in_user_range()` 把缓冲区缺页进来，所以正常的 `read(0, buf, n)` 不受影响。



图 7.7: (a) 读一行与 `Ctrl-C` 的双核时序；(b) PTY 回显死锁

7.3.7 已知问题

1. **PTY 回显会让 sshd 等它自己**（图 7.7(b)）。`ptyecho` 把回显写进与从端输出共用的 512 字节 `output` 队列，队列满就 `sleep`；而 master 一侧唯一的读者正是单线程的 sshd。`cat` 大文件把队列写满时用户再敲一个键，sshd 就在 `writen(master)` 里永远睡下去。Linux 的 n_tty 用独立的 `echo_buf`，写不下就留到下次。
2. **PTY 行规程丢掉所有控制字符**：`ptyinput` 只接受 `ch >= ' '`，所以经 SSH 时 `Ctrl-C`、`Ctrl-D`、Tab 都不起作用；`signal_pgrp` 也只给 `ctty == TTYS0` 的进程发信号。

3. **回显在关中断状态下忙等串口**：115200 波特率每字节约 87 μs，`Ctrl-U` 删一整行要关中断约 33 ms。Linux 在中断里只把字节放进 flip buffer，行规程和回显放到 `flush_to_ldisc` 工作队列里做。
4. **`consolewrite` 的写者之间没有互斥**：两个进程同时写时按字节交错；`printf` 和回显绕过发送环，可能比之前排队的用户字节先出来。Linux 对应的是 `atomic_write_lock`。
5. `ttyattach` 不持 `p->lock` 就写 `sid/pgid/ctty`；`ptyclose` 持 `pt->lock` 通知 epoll，是锁图里唯一一条 pt → ep 边。

7.3.8 与 Linux tty 的对照

表 7.3: xv6-aarch64 与 Linux 的 tty 锁

方面	xv6-aarch64	Linux
tty 对象	全局 `cons`，只有一个终端	每个终端一个 `tty_struct` + `tty_port`，有引用计数
硬件发送	`uart_tx_lock` + `uart_hw_lock`	`uart_port->lock`（`spin_lock_irqsave`）
中断收字节	直接在中断里跑行规程	flip buffer + `flush_to_ldisc` 工作队列
读/写	`cons.lock` + `sleep`；写无互斥	`atomic_read_lock`/`atomic_write_lock` mutex
行规程	写死在 `console.c`/`pty.c`	`ldisc_sem` 保护可切换的 ldisc
前台组/会话	`ttyfg_pgid` + `ttyfg_lock`	`tty->ctrl.lock` 保护 `pgrp/session`
pty	一个 `struct pty` + 一把锁	两个互为 link 的 `tty_struct`，各有 ldisc

总的来说，xv6 用很少几把自旋锁和 sleep/wakeup 就实现了“中断生产、进程消费”的终端，锁顺序是干净的；和 Linux 相比，缺的主要是把行规程从中断里拆出来，以及进程侧的读写 mutex，上面的问题 1、3、4 都源于这两点。

7.4 USB 与 HID 键盘

USB 键盘是本项目里协议层次最多的驱动：从主机控制器的寄存器，到 Hub、事务转换器、描述符、HID 报告，再到输入子系统和控制台。本节沿着“插上键盘—识别—配置—读按键”的顺序，把每一层讲清楚：先看硬件路径和三层软件的分工，再看一次事务在总线上如何由 Token、Data、Handshake 三相包组成，然后顺着描述符与 control 传输完成枚举，然后处理高速 Hub 后面低速设备的 split 时序，接着用 URB 和中断把一份按键报告送到输入子系统与 `/dev/input/eventN`，最后是真机上从 STALL 走到第一份报告的九个阶段。Hub 中断驱动的热插拔、复合 HID 接口和驱动模块的自动装载放在第 9 章。

7.4.1 三层软件与硬件路径

USB 驱动分三层：

- **HCD**（`kernel/drivers/usb/host/dwc2.c`）：复位 DWC2 控制器、强制主机模式、开 DMA；给根端口和 Hub 端口上电、复位；用 host channel 发起 control、bulk、interrupt 传输；
- **USB core**（`kernel/drivers/usb/core/usb.c`）：`struct usb_device`、按 ID 或接口类别匹配驱动、Linux 风格的 URB 接口；
- **类驱动**：HID 键盘（`usbkbd.c`）、CDC-ECM 网卡（`usbnet.c`，用于 QEMU）、MT7601U 无线网卡（`mt7601u.c`）。

一份按键从键盘走到用户进程，要穿过这样一条链：

```
USB 键盘 (低速 1.5 Mb/s)
  -> LAN951x 高速 Hub (TT 完成速率转换)
  -> DWC2 主机控制器 (ch6 interrupt-IN + DMA)
  -> USB bus: struct usb_device
  -> HID Boot Keyboard 类驱动 (usbkbd.c)
  -> input core: struct input_dev
```

```
+-> evdev -> /dev/input/eventN
+-> 控制台处理 -> consoleintr()
    -> xv6 console/TTY 行规程
    -> /dev/ttyS0 或前台 shell 的 read()
```

末尾分出的两支是关键。键盘输入并不直接写用户进程的缓冲区：`usbkbd.c` 把 HID usage 转换成字符后调用 `consoleintr()`，于是串口驱动已经具备的回显、退格、`Ctrl-C`、阻塞读和唤醒语义被整套复用；同一份报告另外生成原始按键事件，进入 `/dev/input/eventN`。一份 HID 报告、两个消费者，这条分发路径在 7.4.8 小节展开。

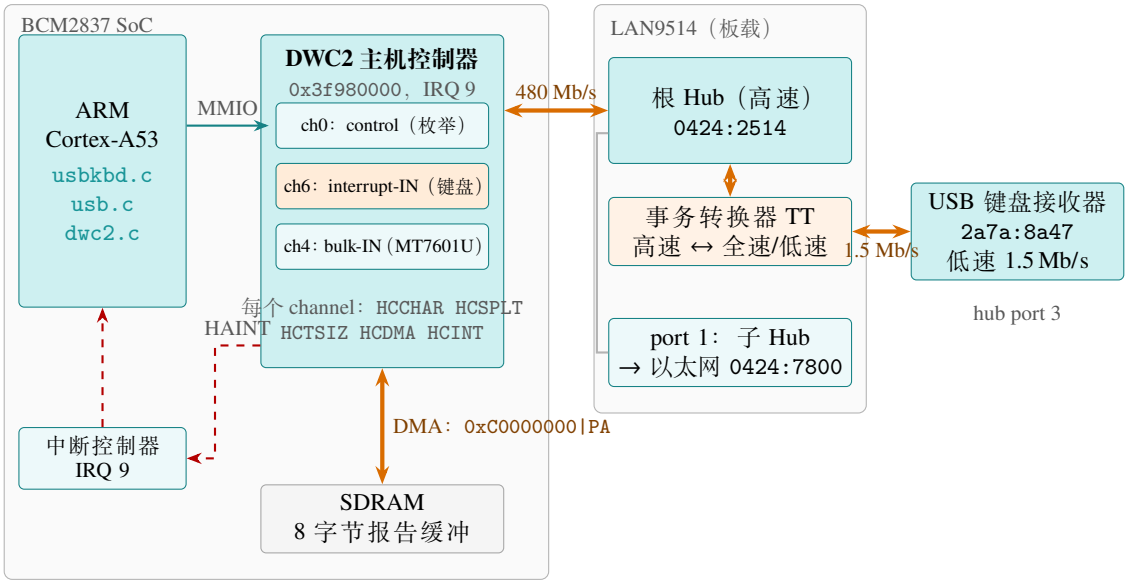


图 7.8: USB 键盘的硬件路径。DWC2 用 host channel 发起事务、用 DMA 读写内存；树莓派 3 的 USB 口都在板载 LAN9514 Hub 后面，低速键盘与 Hub 之间靠事务转换器（TT）完成速率转换

图 7.8从控制器内部展示 channel、DMA 和 IRQ，图 7.9则从外部连接展示两级 Hub、链路速率和一次 split 事务。有几个特点和前面的 Wi-Fi、摄像头都不一样：

主机发起一切. USB 是严格的主从结构，设备永远不能主动说话。即使是名为“interrupt”的传输类型，也只是“主机承诺每隔 `bInterval` 毫秒来问一次”：主机发 IN 令牌，设备有数据就回 DATA，没有就回 NAK。所谓“键盘中断”，其实是主机控制器替 CPU 轮询，问到数据后才用 IRQ 9 通知 CPU。

host channel 与 DMA. DWC2 有 8 个 host channel，每个 channel 是一组寄存器，描述“对哪个设备的哪个端点做一次什么传输”：`HCCHAR`（设备地址、端点号、方向、传输类型、最大包长、是否低速）、`HCSPLT`（split 事务参数）、`HCTSIZ`（字节数、包数、DATA0/1）、`HCDMA`（缓冲区的总线地址）、`HCINT`（完成、ACK、NAK、NYET、错误等状态）。本项目固定分工：ch0 做枚举时的 control 传输，ch6 给键盘的 interrupt-IN，ch4 给 MT7601U 的 bulk-IN。和 Arasan 不同，DWC2 自己用 DMA 读写内存，所以缓冲区在提交前要 clean、完成后要 invalidate 缓存，地址也要换成 `0xC0000000|PA` 的总线地址。

一切都在 Hub 后面. 树莓派 3 的 SoC 只有一个 USB 口，直接接板载 LAN9514，它内部是一个高速 Hub 加一块 USB 以太网卡（在 3B+ 上还有第二层 Hub）。外面插的键盘一般是低速（1.5 Mb/s）设备，而 DWC2 和 Hub 之间跑的是高速（480 Mb/s）。两种速率之间靠 Hub 里的事务转换器（TT）衔接，这就是后面 split transaction 的由来。

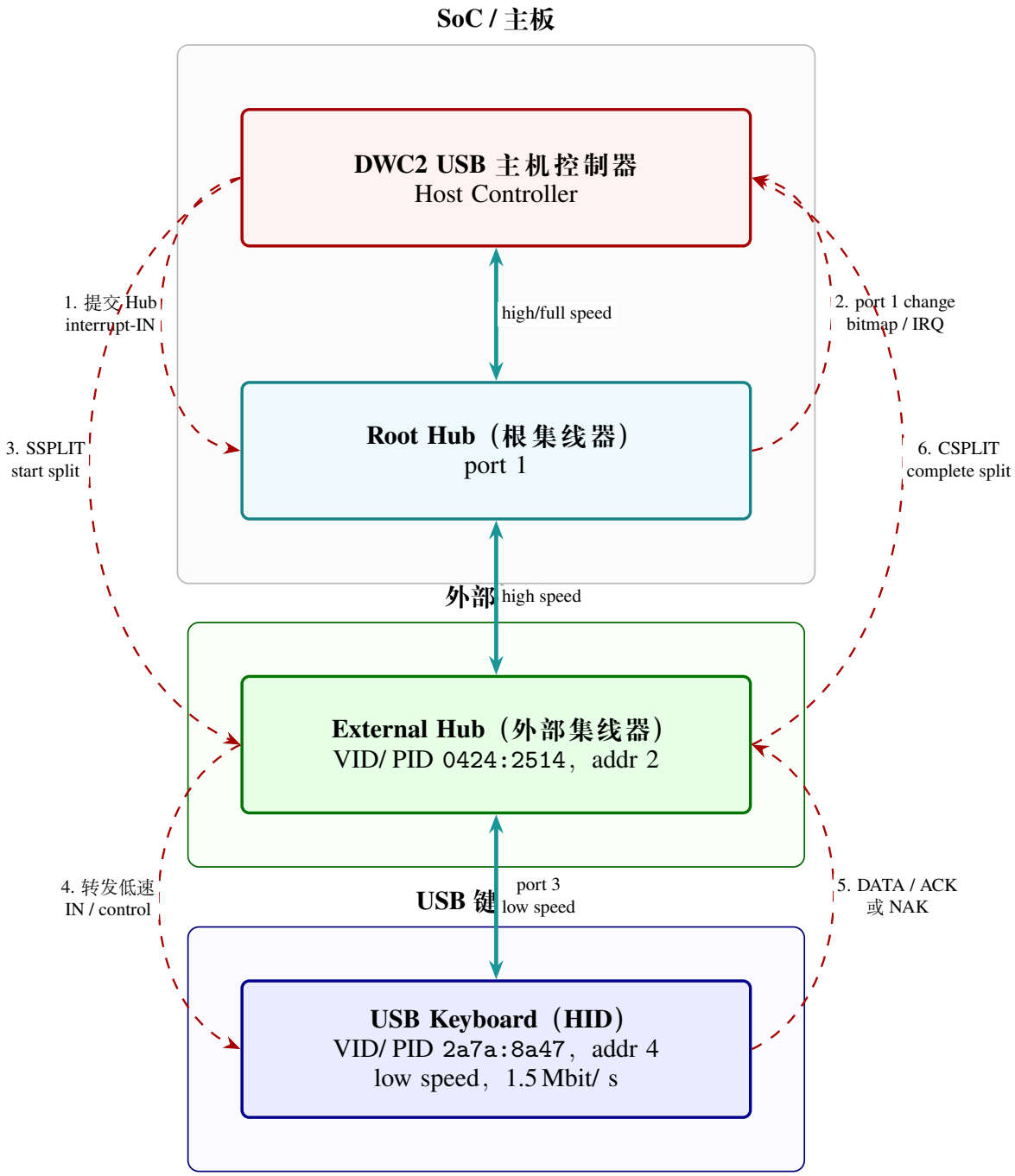


图 7.9: USB 键盘的物理连接与 split transaction 数据路径。实线表示总线连接和速率，红色虚线表示一次端口变化通知以及低速事务的 SSPLIT–设备响应–CSPLIT 往返。Hub interrupt-IN 不是设备主动把数据推给 CPU，而是 DWC2 周期提交事务；端口变化完成后才产生 host-channel IRQ

7.4.2 内核软件框架：五层栈与三条抽象边界

上一小节把驱动粗分成 HCD、USB core、类驱动三层；落到代码里，一份键盘输入实际要穿过**五层**软件栈，外加一层正交的通用设备模型（图 7.10）。这套分层照搬 Linux 的 `drivers/usb` 加 `drivers/base`：相邻两层之间由一张函数指针表（ops）隔开，上层只认这张表、不认下层的具体实现——于是换一块控制器（改 `dwc2.c`）不动任何类驱动，加一个类驱动（`usbmouse.c`、`mt7601u.c`）也不动 core 和 HCD。

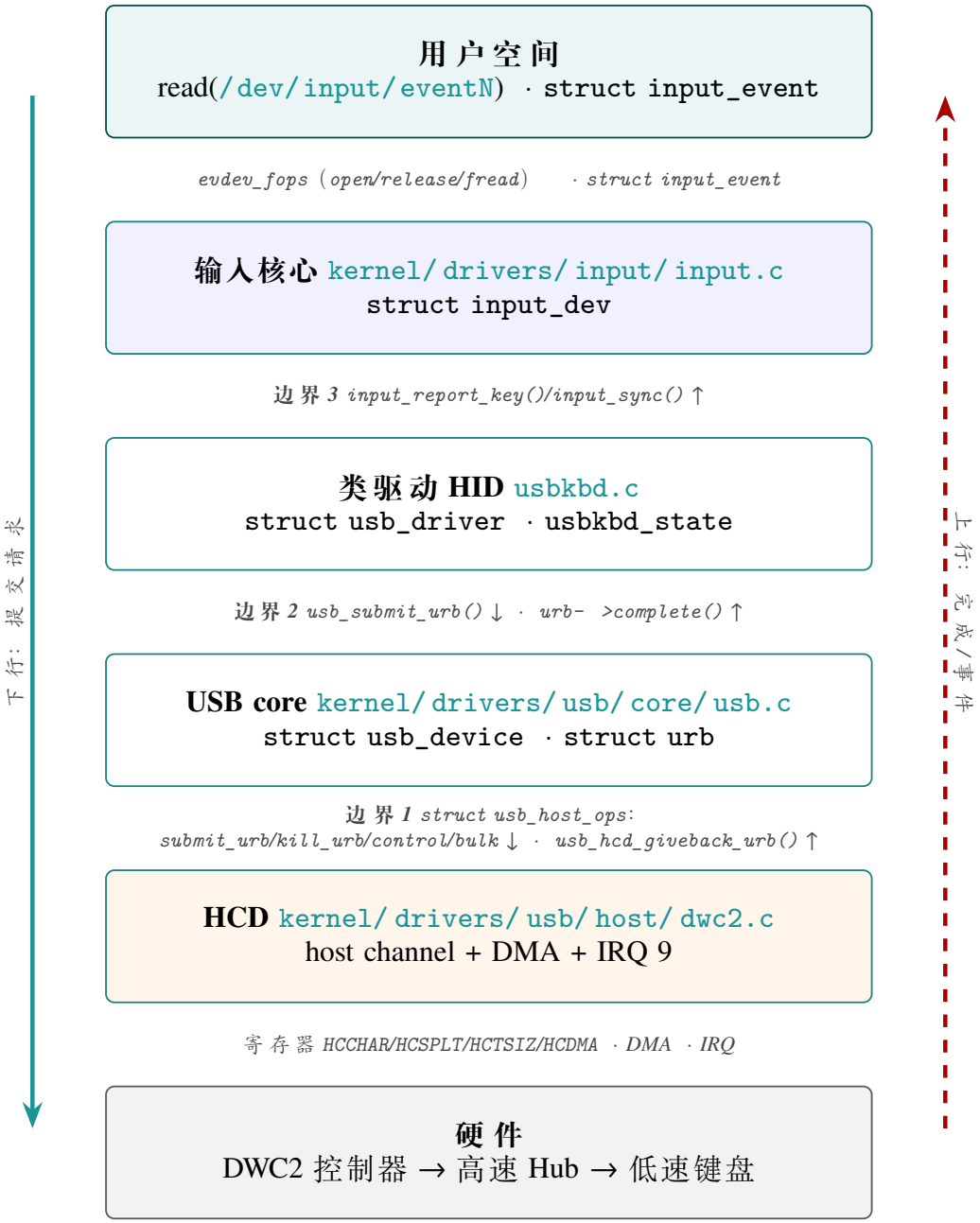


图 7.10: USB 键盘在内核里的五层软件栈。相邻层之间是一张 ops 回调表（图中三条加粗编号的边界），它把上层的调用与下层实现解耦。左实线是自上而下提交请求的路径，右虚线是自下而上送回完成与事件的路径；两条路径合起来构成一次 URB 的往返

边界 1：`usb_host_ops`——core 与 HCD 之间。 这是本项目里 Linux `usb_hcd` 的等价物（`usb.h`）。core 层只通过这张表驱使硬件：下行有 `submit_urb/kill_urb`（异步 URB）和 `control/bulk/bulk_rx_arm`（同步请求）；上行则只留一个入口 `usb_hcd_giveback_urb()`——HCD 完成一笔传输后必须回到这个函数，由 core 再把结果转

交给发起者的 urb->complete 回调。整张表在 dwc2.c 里实例化为 dwc2_usb_ops，枚举出的每个设备、乃至合成的根 Hub，都把 udev->ops 指向它。

边界 2：urb 与 urb->complete——core 与类驱动之间. 类驱动不直接碰寄存器，而是填一个 Linux 风格的 struct urb（端点号、方向、缓冲区、complete 回调），交给 usb_submit_urb()。下行链是：

```
usbkbd_submit / usb_submit_urb(urb)          # usbkbd.c -> usb.c
  -> urb->dev->ops->submit_urb(urb)          # 函数指针跳转：core -> HCD
     -> dwc2_submit_urb                      # dwc2.c
        -> 占用 interrupt_rx[] 槽位，编程 ch6 的 HCCHAR/HCSPLT/HCTSIZ/HCDMA
```

上行链把完成一路送回用户空间，中间两次跨越“硬中断 → 工作队列”的上下半部边界：

```
IRQ 9 -> dwc2_irq -> dwc2_interrupt_urb_work # 硬中断只 schedule_work
  -> usb_hcd_giveback_urb(urb, status, len)  # HCD -> core（唯一入口）
     -> urb->complete(urb)                   # 函数指针跳转：core -> 类驱动
        -> usbkbd_irq_complete -> queue_work(usbkbd_wq, rx_work)
           -> usbkbd_rx_work -> input_report_key()/input_sync()   # 边界3
              -> evdev_pass_event -> wake_up(client)
                 -> read(/dev/input/eventN) 返回一个 input_event   # 用户空间
```

边界 3：input_dev——类驱动与输入核心之间. 类驱动解析完 HID 报告后，只调用 input_report_key()/input_sync() 把语义化事件丢进输入核心，完全不知道 evdev、控制台或任何具体消费者。输入核心再把它分发给每个打开的 evdev_client（图里最上面那条边界），同时另一路经 consoleintr() 复用串口 TTY 的回显与行规程——一份报告、两个消费者。

正交的一层：通用设备模型. 上面五层是“数据通路”，还有一层横向的注册/绑定机制把它们接起来，即 device.c 的 struct bus_type。USB core 向设备模型注册一条名为 usb 的总线（usb_bus，带 match/probe/remove/modalias 四个钩子），类驱动 usb_register_driver()、枚举出的设备 usb_device_register() 分别把自己挂上去；一旦设备和驱动在总线上相遇，usb_match() 按 class/subclass/protocol 或 ID 表判定匹配，usb_probe() 调用类驱动的 probe()。热插拔与自动装载正来自这一层：匹配失败时 modalias 生成 usb:cXXsXXpXX 触发 request_module() 去加载对应的 .ko（详见 §9.1 与 §9.2 鼠标自动装载一节）。

与 Linux 的对应与简化. 表 7.4 给出每层对应的 Linux 路径。本项目有两处刻意的简化：其一，没有独立的 usb_hcd 对象，也没有单独的 hub-driver 层——dwc2.c 同时充当 platform 驱动（HCD）、根 Hub 的模拟者（dwc2_root_hub 复用自己的 dwc2_usb_ops）和总线枚举者；其二，usb_host_ops 把异步 URB 接口和遗留的同步 control/bulk 混在同一张表里，类驱动在枚举阶段可以直接调用同步接口，运行期才走 URB。

表 7.4: 本项目 USB 软件栈各层与 Linux 的对应

本项目	Linux 对应	职责
dwc2.c（usb_host_ops）	drivers/usb/host/dwc2 + usb_hcd	把 URB 翻译成 host channel 寄存器与 DMA，收完成中断
usb.c	drivers/usb/core	URB 生命周期、giveback、设备/驱动注册入口
device.c（bus_type）	drivers/base	总线/设备/驱动模型，match–probe、热插拔、自动装载
usbkbd.c、mt7601u.c	drivers/hid/usbhid、drivers/net/usb	类驱动：解析各自协议的报告/报文
input.c + evdev	drivers/input + drivers/input/evdev	事件分发到 /dev/input/eventN

7.4.3 事务格式：Token、Data、Handshake 三相

上一小节说“主机发起一切”，但一次到底“发起”了什么？USB 总线上被独立调度的最小单位既不是字节也不是包，而是**事务**（transaction）。一次事务由主机发起，按固定顺序串起最多三种**包**（packet）：主机先发一个 **Token** 包，声明“要对哪个设备的哪个端点做一次什么操作”；接着是一个可选的 **Data** 包，方向取决于 Token 的类型；最后是一个 **Handshake** 包，由接收方确认或拒绝。规范里的图 8-30（bulk 事务）和图 8-36（control SETUP 事务）画的正是这套三相结构在不同传输类型下走过的分支。

每个包的场（field）布局是固定的：以 `SYNC` 开场让接收方对齐位边界，紧跟一个 `PID` 场标明包的类型，中间是地址/负载与校验，末尾以 `EOP`（end of packet）收束，如图 7.11。

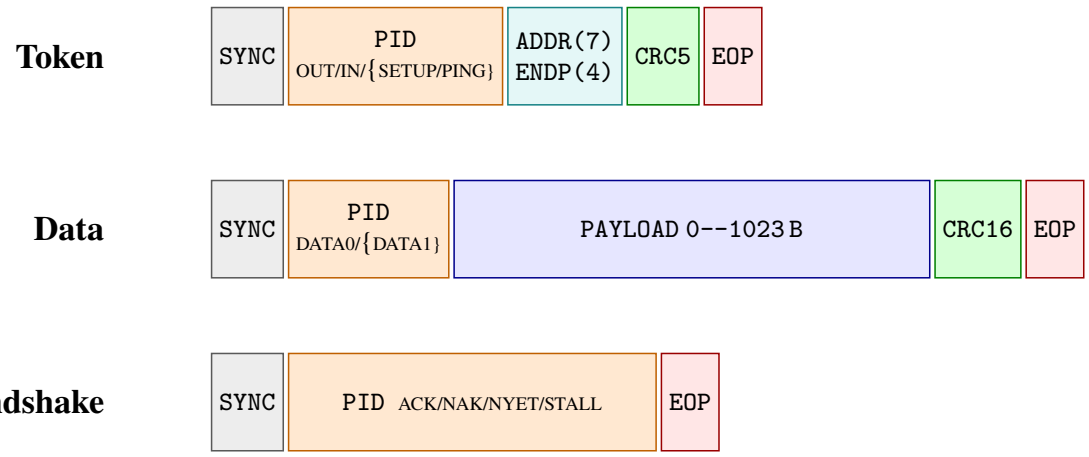


图 7.11: USB 三种包的场布局。Token 包用 CRC5 保护“地址 + 端点”，Data 包用 CRC16 保护负载，Handshake 包只有 SYNC/PID/EOP、既无负载也无 CRC 场。关键区别：**PID 场本身占 8 位（4 位本体再取反重发一次）**，双重编码已足以自校验，所以 PID 不需要额外 CRC，握手包也因此不必带 CRC

PID 决定了一个包在事务里扮演什么角色，表 7.5 按 Token/Data/Handshake 三类把本项目会遇到的 PID 列全。

表 7.5: PID 分类与各传输类型下的作用

类别	PID	含义与在本项目中的位置
Token	`OUT`	主机 → 设备：后面跟一个主机写往该端点的 Data 包（EP0 的 control OUT、ch4 之外的下发命令）
Token	`IN`	主机 → 设备：请求该端点回一个 Data 包。ch6 键盘每隔 `bInterval` 发一次 IN，ch4 MT7601U 的 bulk-IN 也是 IN
Token	`SETUP`	只在端点 0，启动一次 control 传输的 SETUP 阶段（枚举、`SET_CONFIGURATION` 等）
Token	`PING`	仅高速 bulk OUT：先探测端点是否还有缓冲，再决定发不发 Data，避免把数据丢进满的端点
Data	`DATA0/DATA1`	数据包，二者交替出现做 PID 序列翻转（toggle），用来发现丢包或重复包；EP0 的 SETUP 恒用 DATA0
Handshake	`ACK`	接收方 → 发送方：已正确收到
Handshake	`NAK`	设备 → 主机：“收到了但暂时不能传”——IN 端点没数据、或 OUT 端点缓冲满。是流控不是错误，主机下一帧重试
Handshake	`NYET`	高速设备 → 主机：“收到了但还没准备好”，与 NAK 区分；split 事务用它携带本帧是否完成的信息
Handshake	`STALL`	设备 → 主机：端点被软件停止（halt），不是重试能解决的，要驱动显式清除

三相结构在三种传输类型下走的路径略有不同，图 7.12 把控制 SETUP、IN、OUT 三条主线并排画出（框的颜色标出这个包由谁发出：灰框 = 主机、白框 = 设备）。

把这套三相格式对回本项目的三个 channel 就很清楚了：ch0 的枚举是反复做 control 事务——先 SETUP 三相把“读描述符/设配置”的请求送进去，再用若干 IN 或 OUT 事务搬运数据，最后用一个反方向的空 Data 事务做 STATUS；ch6 键盘每隔 `bInterval` 发一次 IN 事务，设备没按键就回 NAK、有按键才回 DATA1/DATA0；ch4 MT7601U 的 bulk-IN 同样发 IN，靠 DATA0/DATA1 的翻转做流控。下一小节的 split 事务，本质就是把这里的三相包在高速 Hub 的事务转换器（TT）处拆成两拍：主机先发一个 SSPLIT 触发 TT 去总线上跑完整事务，隔 1 ~

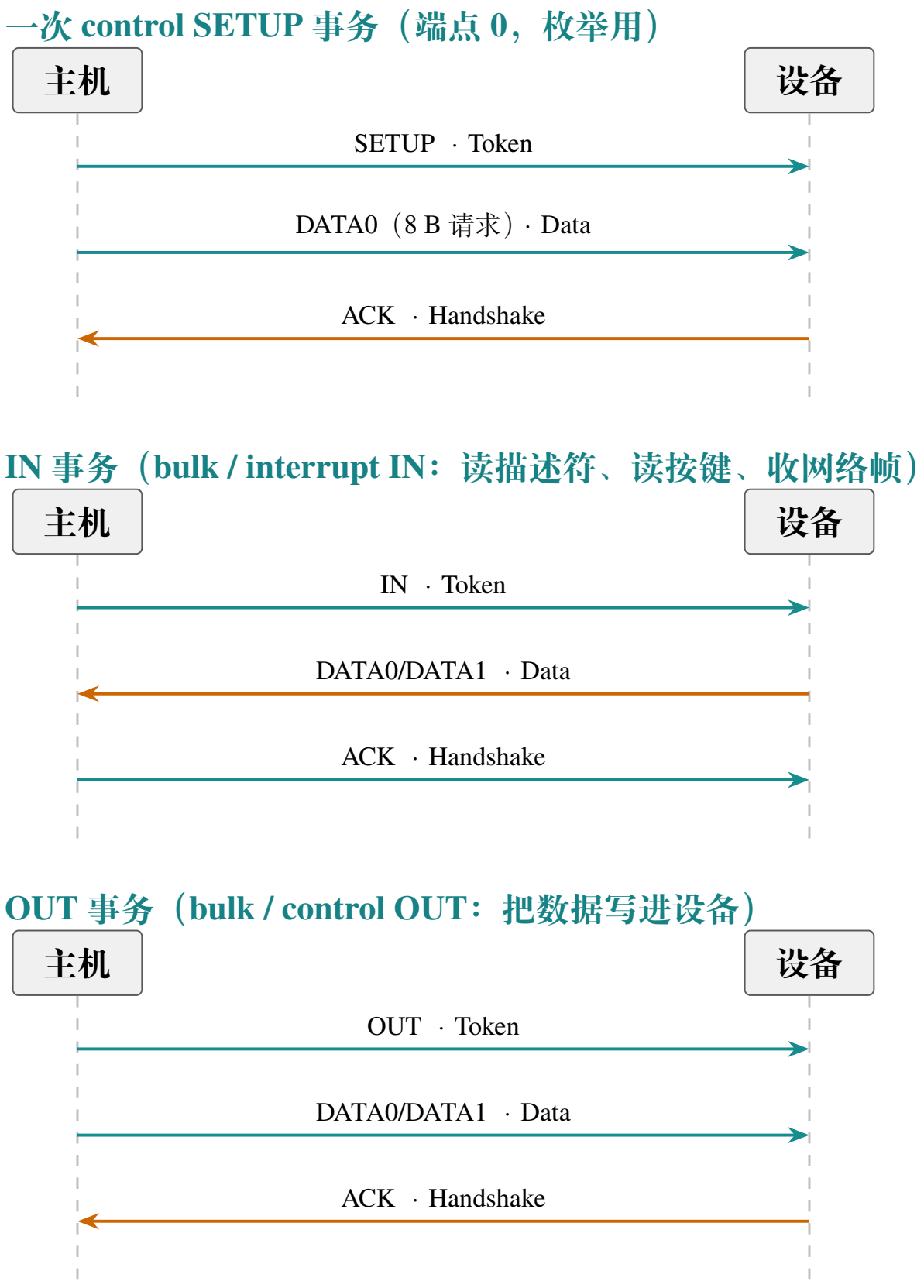


图 7.12: 三相事务的三条主线（对应规范图 8-30 的 bulk 与图 8-36 的 control SETUP，只画成功分支）。蓝色箭头 = 主机发出，橙色箭头 = 设备发出。注意 Data 包的方向：SETUP 与 OUT 由主机写向设备，IN 由设备回给主机；Handshake 永远来自本事务的接收方。失败分支不在此图：设备无数据回 `NAK`、端点被停回 `STALL`（都结束本帧、由主机择机重发），高速 bulk OUT 在 OUT 令牌前还插一次 `PING` 探测缓冲

2 个微帧再发 CSPLIT 把结果（DATA、ACK、NAK 或 NYET）取回来。

规范按“可靠性、时限、方向”把这同一套三相事务组合成**四种传输类型**（表 7.6）。前三者都靠 Handshake 与 DATA0/DATA1 翻转保证不丢不错，只有 isochronous 干脆去掉握手、发出去就不管，用可靠性换实时性；bulk 与 interrupt 同样可靠，区别只落在“有没有周期承诺”这一条上。本项目的落点很清楚：枚举走 control（ch0），键盘和 Hub 状态走 interrupt（ch6、ch7），MT7601U 网卡的报文走 bulk（ch4），isochronous 未使用。

表 7.6: USB 四种传输类型，以及它们在本项目里落在哪个 channel

类型	可靠性与时限	典型数据	本项目
Control	保证送达；不承诺延迟但优先调度，主机发起、双向	枚举、发命令、取描述符	ch0（EP0）
Interrupt	保证送达；有周期上限（`bInterval` 轮询），每包很小	键盘按键、鼠标、Hub 状态变化	ch6（键盘）、ch7（Hub）
Bulk	保证送达；无时限，只在剩余带宽里搬运，量大	U 盘、打印、网络报文	ch4（MT7601U bulk-IN/OUT）
Isochronous	不重传、无握手；预留带宽、严格实时	音频、视频流	（本项目未用）

7.4.4 描述符：设备、配置、接口与端点

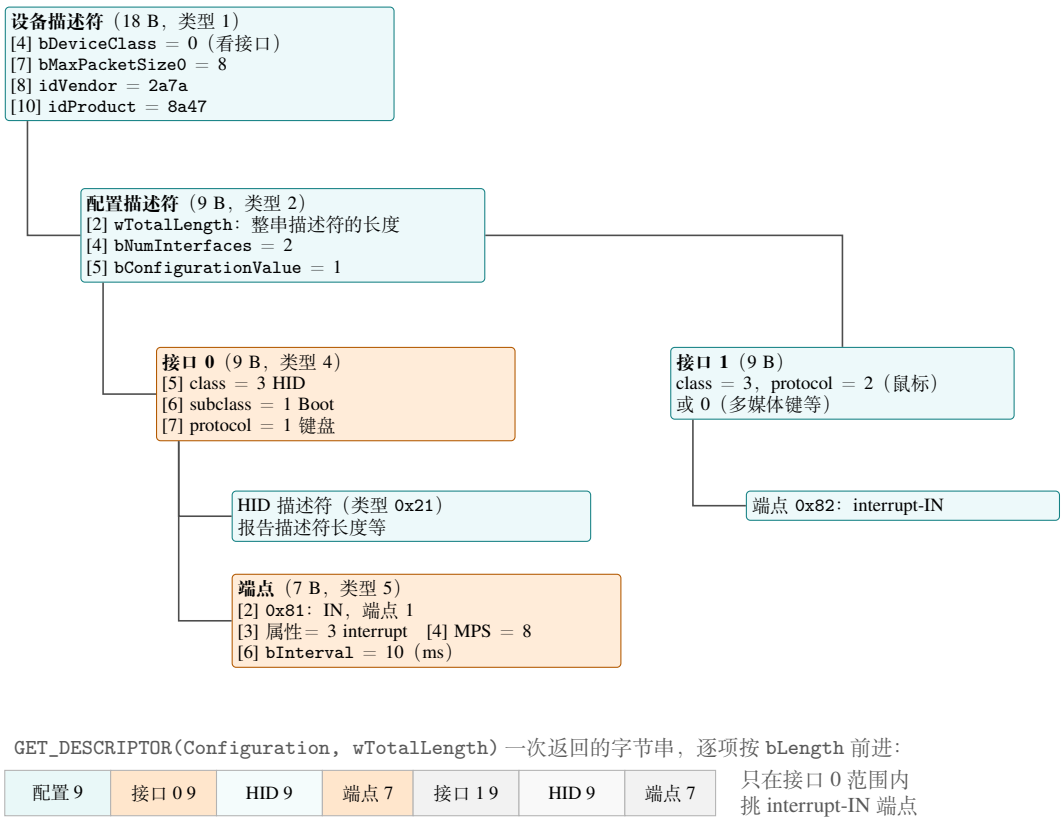


图 7.13: 一个 USB 键盘接收器的描述符层次（示意，方括号里是字节偏移；VID/PID 取自真机日志）。驱动按类别匹配，而不是按 VID/PID：先找 class/subclass/protocol = 3/1/1 的 Boot Keyboard 接口，再只在这个接口里找 interrupt-IN 端点

USB 设备用一组描述符介绍自己（图 7.13）。每个描述符的第 0 字节是长度 `bLength`，第 1 字节是类型。层次是：一个设备有若干配置（几乎总是 1 个）；一个配置包含若干接口，每个接口是一项独立的功能，有自己的 class/subclass/protocol；一个接口有若干端点，端点是真正收发数据的地方，有编号、方向、传输类型、最大包长和轮询间隔。端点 0 是所有设备都有的默认控制端点，枚举全靠它。

很多键盘接收器是复合设备：接口 0 是键盘，接口 1 是鼠标或多媒体键。所以驱动不能看设备描述符的 class（它通常是 0，意思是“看各个接口”），也不能凭 VID/PID 白名单，而要遍历配置描述符：

```
// GET_DESCRIPTOR(Configuration) 先取 9 字节拿到 wTotalLength，再一次取全部
for(int off = 0; off + 9 <= total && ctrl_buf[off] >= 2; off += ctrl_buf[off]){
  if(ctrl_buf[off + 1] == 4 &&                  // 接口描述符
     ctrl_buf[off + 5] == 3 &&                  // class: HID
     ctrl_buf[off + 6] == 1 &&                  // subclass: Boot Interface
     ctrl_buf[off + 7] == 1){                   // protocol: Keyboard
    sel->interface_number = ctrl_buf[off + 2];
    return 1;
  }
}
```

`off += ctrl_buf[off]` 按每个描述符自己的长度前进，所以中间夹着的 HID 描述符（类型 `0x21`）等不认识的类型会被自然跳过；`ctrl_buf[off] >= 2` 防止一个长度为 0 的坏描述符造成死循环。找到键盘接口后，后面的端点只在这个接口的范围里挑（`current_interface` 限定），否则复合设备里鼠标的端点会把键盘的覆盖掉。

7.4.5 枚举：control 传输与请求序列

端点 0 上的 control 传输分三个阶段：SETUP 阶段主机发出 8 字节的请求（`bmRequestType`、`bRequest`、`wValue`、`wIndex`、`wLength`），可选的 DATA 阶段按最大包长收发数据，STATUS 阶段反方向发一个空包确认。`control()` 就是在 ch0 上依次执行这三个阶段。这三个阶段各自怎么走、PID 怎么翻转、方向和数据量在什么时候谈定，正是“设备如何把配置数据交给主机、又如何在真正搬数据之前把方向与长度准备好”的关键，图 7.14 把它摊开。

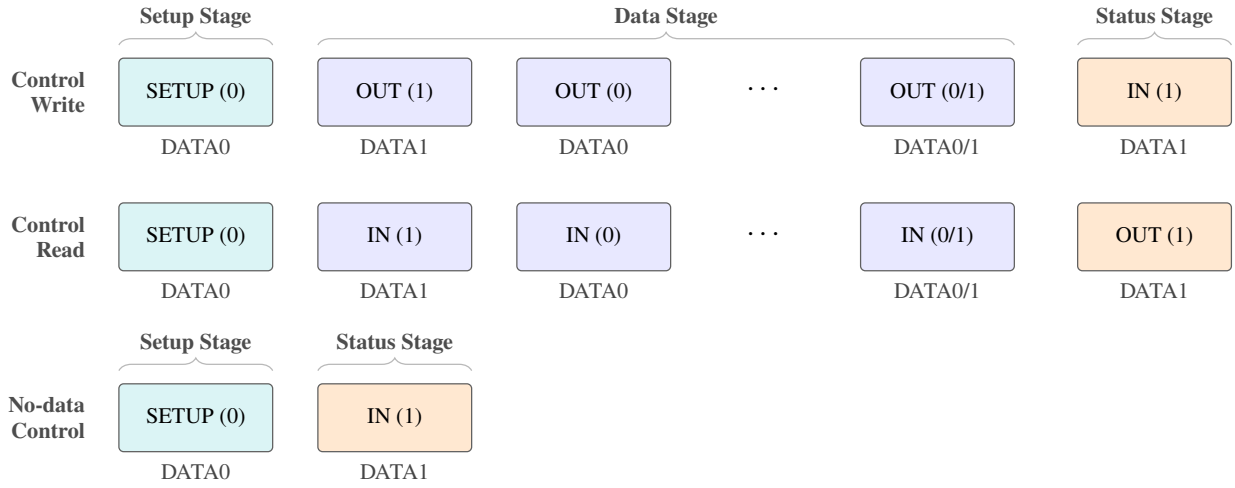


图 7.14: 端点 0 control 传输的三种序列（对应规范图 8-37）。框里是令牌（SETUP/OUT/IN）与其后的数据 PID 编号，框下是实际的数据包类型。Data 阶段可长可短（· · · 表示按 `wLength` 重复若干包），也可能根本没有（No-data Control）。注意三处不变量：SETUP 恒用 DATA0；Data 阶段第一包用 DATA1、之后逐包翻转；STATUS 恒为反方向的零长 DATA1

SETUP 阶段：一次事务把“要什么、给多少、往哪个方向”先谈定。 8 字节的 SETUP 包就是“为传送数据做准备”的地方：`bmRequestType` 的 bit 7（D 位）决定随后 Data 阶段的方向——0 为 OUT（主机写向设备，Control Write），1 为 IN（设备回给主机，Control Read）；`bRequest` 加 `wValue`/`wIndex` 说明“要哪一份数据”（例如 `GET_DESCRIPTOR` 的描述符类型与索引）；`wLength` 说明“最多给多少字节”。SETUP 永远用 DATA0，并且把 EP0

的数据翻转复位，使 Data 阶段从 DATA1 起步。若 `wLength`= 0（`SET_ADDRESS`、`SET_CONFIGURATION`、`SET_IDLE` 这类），就没有 Data 阶段，直接进入 STATUS，即图中的 No-data Control。

逐字节拆解：`GET_DESCRIPTOR(Device, 8)`. 这条请求最能说明“SETUP 谈请求、Data 拿数据”的分工。名字里的 `Device` 和 8 分别落在 SETUP 包的第 3 字节（`wValue` 高字节＝描述符类型 1）和第 6 ~ 7 字节（`wLength` = 8）上——也就是说，整条 *8 字节请求都在 SETUP 阶段发出*，而“取回的 8 个字节”发生在随后的 Data 阶段。它是一次完整的 control 传输，而不是“只属于某一个阶段”：SETUP 事务把这 8 字节以 DATA0 写进设备，Data 阶段设备用 IN 事务把设备描述符的前 8 字节回传，STATUS 阶段主机再发一个反方向的零长包收尾。表 7.7 逐位拆开这 8 字节。

表 7.7: `GET_DESCRIPTOR(Device, 8)` 在 SETUP 阶段发出的 8 字节请求（USB 为小端，多字节字段低字节在前）

字节	字段	取值	逐 bit / 含义
0	`bmRequestType`	`0x80`	`1000 0000`：bit 7 D = 1 → Data 阶段设备到主机（IN，Control Read）；bit 6–5 type = `00` → Standard 请求；bit 4–0 recipient = `00000` → 发给设备本身
1	`bRequest`	`0x06`	`GET_DESCRIPTOR`
2	`wValue` 低	`0x00`	`bDescriptorIndex` = 0：第 0 号（唯一一份）设备描述符
3	`wValue` 高	`0x01`	`bDescriptorType` = 1 → DEVICE 描述符
4–5	`wIndex`	`0x0000`	Standard 对设备的请求不用它，置 0
6–7	`wLength`	`0x0008`	Data 阶段最多回传 8 字节

为什么先只取 8 字节：此刻主机还没读到 `bMaxPacketSize0`（设备描述符第 7 字节），只能按规范允许的最小值 8 来假定 EP0 包长，于是把 `wLength` 也设成 8，恰好拿到含该字段的头 8 字节；得知真实包长后再发 `GET_DESCRIPTOR(Device, 18)` 取完整描述符。Data 阶段这 8 字节是单个 DATA1 包（长度= `wLength` = 8），一次 IN 事务即可搬完——它是低速设备时，这一个 IN 事务在总线上又拆成一对 SSPLIT/CSPLIT。

Data 阶段：方向沿用 SETUP，逐包翻转 DATA1/DATA0. 数据阶段是若干个普通的 IN 或 OUT 事务，方向由 SETUP 定死、整段不变。第一包用 DATA1，之后 DATA0/DATA1 交替；每包至多 mps 字节，包数由 `wLength` 与端点 0 的最大包长共同决定（`packets = (len+mps-1)/mps`）。低速键盘 EP0 的 mps 只有 8，所以 18 字节的设备描述符要分 3 包：DATA1(8) + DATA0(8) + DATA1(2)。当某包短于 mps、或累计已达 `wLength`，数据阶段即结束。

STATUS 阶段：反方向的零长 DATA1，兼作握手与翻转复位. 数据阶段之后，主机发一个方向相反的零长数据事务：Control Write 的 STATUS 是 IN、Control Read 的 STATUS 是 OUT（图里最右一列）。这个包恒为 DATA1、长度为 0，作用有三——确认整个控制传输成功、把方向反过来做最后一次握手、并复位 EP0 的数据触发器为下一次 SETUP 做准备。设备若在这一步回 STALL，表示它拒绝了刚才的请求。两条不变量值得单独记住：**status 阶段在任何 control 传输里都省不掉**（SETUP 之后无论有没有 Data，都以一个反方向零长 DATA1 收尾），而 **status 的方向永远与 Data 阶段相反**——Data 方向由 SETUP 的 D 位定，status 就取反方向；因此 `GET_DESCRIPTOR`（Control Read）的 status 是 OUT，`SET_CONFIGURATION`（No-data、D=0）的 status 是 IN。

对回本项目：`control()` 如何走这三相. `control()` 在 ch0 上按 SETUP→(Data)→STATUS 依次调用 `channel_xfer()`：SETUP 用 `PID_SETUP` 带 8 字节请求；数据阶段按方向以 `in=1/0` 反复搬运 mps 大小的块并翻转 PID；STATUS 用反方向、`len=0`、DATA1。挂在 Hub 后面的低速设备，这三相里的每一个事务又要各自拆成一对 SSPLIT/CSPLIT（下一小节）——所以“读 18 字节描述符”在总线上其实是 3 个 IN 事务、每个再走 split 两拍，这也解释了为什么必须逐包翻转、少取一包就会读到缓冲区里的残值。

表 7.8 是识别一个 Hub 后面的键盘所依次发出的 control 请求。

先取 8 字节再取 18 字节是 USB 枚举的惯例：在知道端点 0 的最大包长之前，只能假定它是最小的 8 字节，而设备描述符第 7 字节正好告诉我们真实值。端口状态的例子：真机日志里 Hub port 3 是 `0x301`（供电、低速、已连接），复位后变成 `0x303`（多了“使能”）；根 Hub 和端口上电之后并不是只读一次状态，而是在复位枚举前最

表 7.8: 从 Hub 端口到可用键盘的请求序列

对象	请求	bmRequestType	作用
Hub	SET_FEATURE(PORT_POWER)	0x23	给下游端口上电，之后最多等 500 ms 连接去抖
Hub	GET_STATUS	0xa3	端口状态：bit 0 连接、bit 1 使能、bit 8 供电、bit 9 低速、bit 10 高速
Hub	SET_FEATURE(PORT_RESET)	0x23	复位端口，设备进入默认状态、地址为 0
设备 0	GET_DESCRIPTOR(Device, 8)	0x80	只取 8 字节，得到端点 0 的最大包长
设备 0	GET_DESCRIPTOR(Device, 18)	0x80	完整设备描述符：class、VID、PID
设备 0	SET_ADDRESS(n)	0x00	分配唯一地址；把父 Hub、端口、速度记入 usb_route[n]
设备 *n*	GET_DESCRIPTOR(Config, 9)	0x80	得到 wTotalLength
设备 *n*	GET_DESCRIPTOR(Config, total)	0x80	全部接口和端点，挑出 Boot Keyboard
设备 *n*	SET_CONFIGURATION	0x00	激活配置，端点开始可用
接口	SET_PROTOCOL(boot)	0x21	让键盘用固定的 8 字节 Boot 报告格式
接口	SET_IDLE(0)	0x21	只在按键状态变化时才回报告

多等 500 ms，让 PORT_CONNECTION 稳定下来，冷启动较慢的 Hub、无线键盘接收器和低速键盘都靠这段去抖时间。如果等满之后仍然是 0x100，含义是控制器只看到 PORT_POWER，线上并没有设备的电气连接——此时 HID 驱动和 ch6 都还没有参与，不该往协议层去找原因。根 Hub 和嵌套的 Hub 用同一套请求递归扫描，深度限制为两层，防止一个异常的拓扑让启动无限递归。扫描到的端点属性按 USB 地址记录在 usb_ep0_mps[] 里，Hub、低速键盘和鼠标不再共用一个全局的端点 0 包长。

把这一整套判断画成控制流就是图 7.15。它有五个阶段，值得注意的不是步骤本身，而是三处分支判据：class 9 决定要不要扫描下游端口，端口状态决定这个端口是否继续，class/subclass/protocol 决定这个接口是不是 Boot Keyboard。递归那一跳（子 Hub 回到端口扫描）和低速那一支（LSPDDEV 加 split 事务）是真机上最容易出错的两处——前者决定嵌套 Hub 能不能被发现，后者决定 EP0 的每个包要不要单独走一遍 SSPLIT/CSPLIT。

SET_PROTOCOL(boot) 很关键。HID 设备本来用“报告描述符”描述自己的报告格式，解析它需要一个不小的解释器；而 Boot 协议是 BIOS 时代为了不解析报告描述符而规定的固定格式，所有键盘都必须支持。用它，驱动只需理解图 7.16 这 8 个字节。

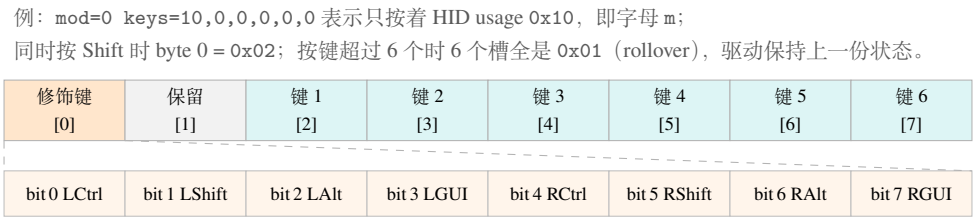


图 7.16: HID Boot 键盘报告（8 字节）。byte 0 是 8 个修饰键的位图，byte 2 ~ 7 是当前按着的最多 6 个键的 usage 编号

最后，usb_device_register() 把设备交给 USB 总线，总线按 class/subclass/protocol 匹配到 usbhid-keyboard 驱动，调用 usbkbd_probe()：发送上面两个 HID 请求，分配每设备的 usbkbd_state 和 URB，注册 input_dev，然后第一次提交 URB。

枚举与配置的软件函数调用链. 图 7.17回答的是“这些报文由哪一个 C 函数发起、设备何时进入通用设备模型，以及键盘的 interrupt-IN 请求最终在哪里启动”；它与 §9.1 热插拔一节的总线时序图（图 9.1）互为表里——那张图讲“USB 线上先后发送了什么”，这张图讲“这些报文由谁发起”。图中同一行的浅灰等宽字是这一步对应的串口日志，把函数调用与真机实际输出逐行对齐。启动代码先注册 USB 总线与类驱动，随后注册 DWC2 platform 设备和驱动；因此 dwc2_probe() 发现并注册 USB 设备时，device_register() 可以立即找到已经存在的 usbkbd_driver 并完成绑定。

调用链中有三个容易混淆的边界：

1. **USB 设备地址与 DWC2 channel 不是同一个编号。**SET_ADDRESS(4) 改变的是总线上的设备地址；ch0、ch6、

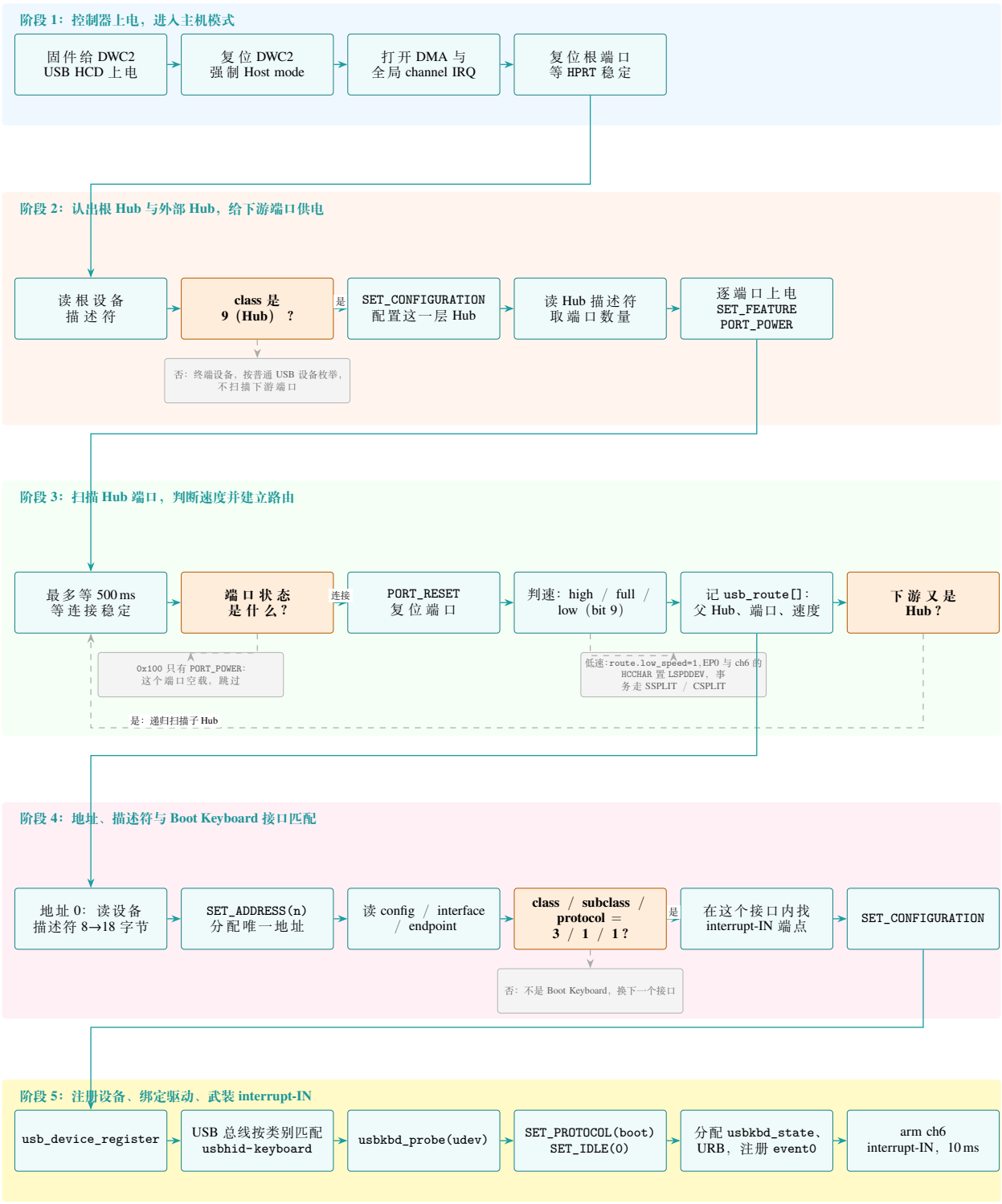


图 7.15: USB 键盘枚举与配置的决策流程。从上到下是五个阶段：控制器上电、识别 Hub 并供电、扫描端口与判速、描述符与接口匹配、注册绑定与武装 interrupt-IN。橙色框是判断，灰色框是提前退出的分支，虚线表示递归与低速分支。整条链上只有地址、配置和接口三类判断与“这是一只键盘”有关，其余部分对任何 USB 设备都一样

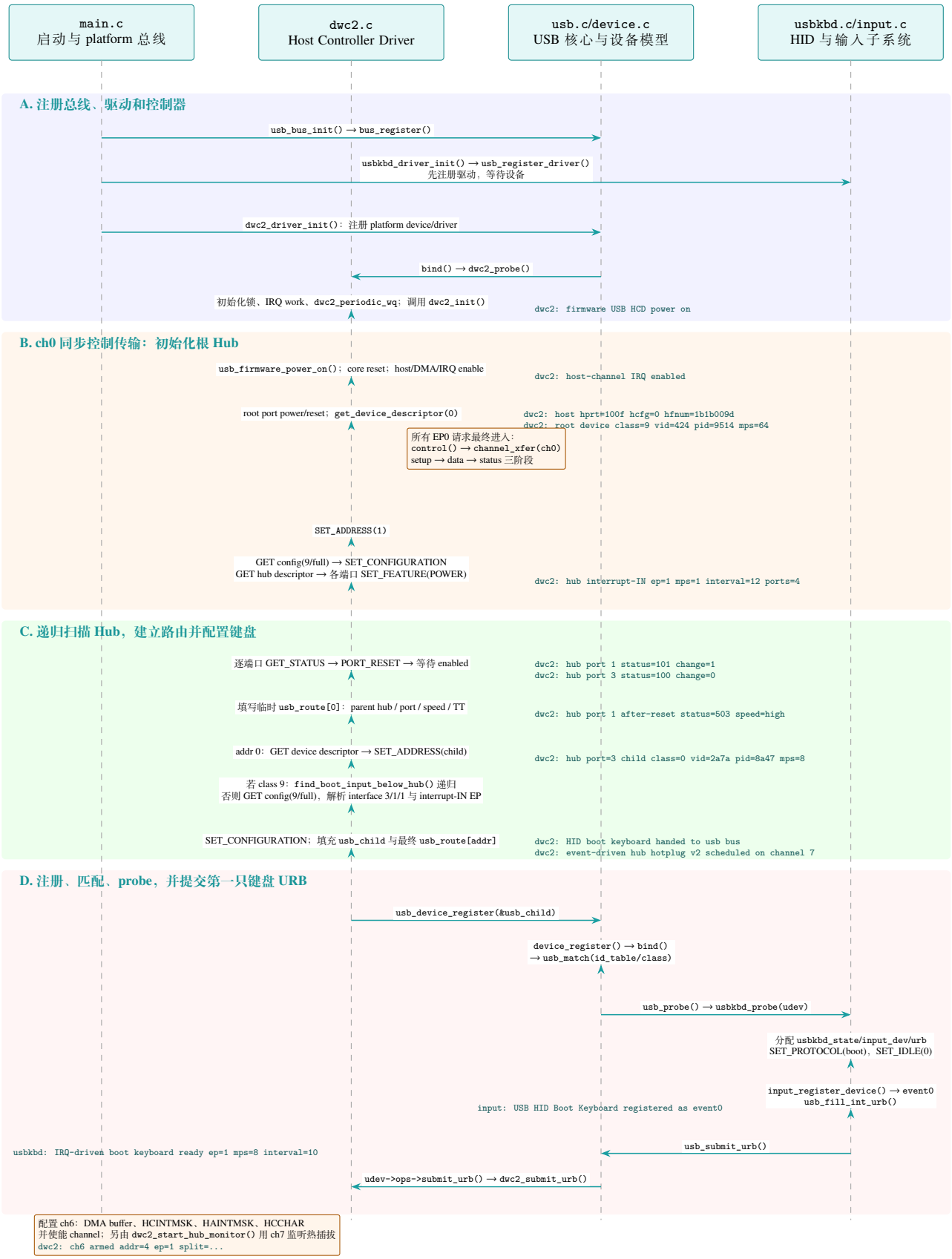


图 7.17: USB 键盘枚举与配置的软件函数调用链；同一行的浅灰等宽字是这一步实际打印的串口日志，可与真机输出逐行对照。图中的描述符和配置请求都由 `control()` 经 ch0 同步完成；设备完成 `SET_CONFIGURATION` 后才进入 USB 设备模型，类驱动的 `probe()` 注册 `event0`，最后由 `dwc2_submit_urb()` 把逻辑端点 EP1 映射到本机 host channel 6

ch7 是 DWC2 内部的事务执行上下文。`dwc2_submit_urb()` 把 URB 中的 device address、endpoint、方向、速度和 split route 写入 ch6 寄存器，才建立“ch6 正在服务 addr 4 的 EP1-IN”这一临时关系。

2. **枚举和运行采用不同执行模型。** `control()` 在启动阶段同步等待 ch0 完成，便于严格保持 USB 枚举顺序；键盘运行期则由 ch6 IRQ 完成 URB，再由 workqueue 解析 HID report，NAK 重试通过 `dwc2_periodic_wq` 的 delayed work 按到期时间重提。
3. **注册设备不是重新枚举。** `usb_device_register()` 只把已完成地址和配置阶段的 `usb_child` 交给设备模型；`usb_match()` 选择驱动，`usbkbd_probe()` 完成 HID 类请求、输入设备注册和第一只运行期 URB 的提交。

7.4.6 Split transaction：高速 Hub 后面的低速设备

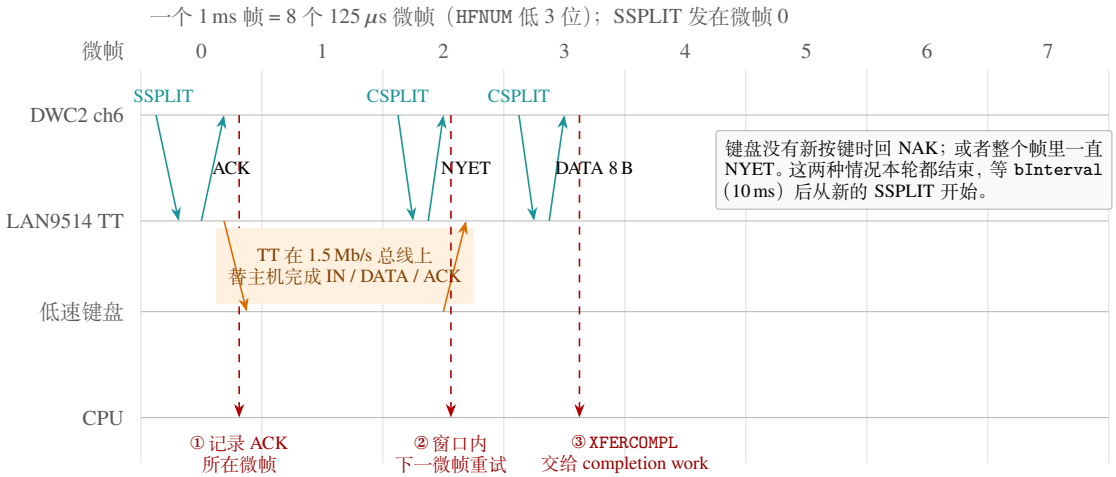


图 7.18: 低速键盘在高速 Hub 后面的一次 interrupt-IN 轮询。主机先向 TT 发 start-split，TT 在低速总线上替主机完成 IN 事务，主机再在随后的微帧里发 complete-split 取回结果。①、② 两种中间事件必须在中断里当场处理，晚一个帧就错过了窗口

DWC2 不能直接对低速设备说话，它要先把事务交给 Hub 里的 TT（图 7.18）：*start-split*（SSPLIT）告诉 TT“请替我对 port 3 上地址 4 的端点 1 做一次 IN”，TT 回 ACK 表示接受；TT 随后在低速总线上完成真正的事务；主机在随后的微帧里发 *complete-split*（CSPLIT）取结果，TT 要么回 NYET（还没完成），要么回数据，要么转达设备的 NAK。channel 寄存器里，这些信息体现为：

表 7.9: 真机日志中的寄存器样本解码

样本	含义
`HCCHAR=0x010e8808`	设备地址 4、端点 1、IN、interrupt 类型、低速（LSPDDEV）、最大包长 8
`HCSPLT=0x8001c083`	SPLTENA、COMPSPLT（当前是 CSPLIT）、XACTPOS=ALL、Hub 地址 1、端口 3
`HCINT=0x22`	`CHHLTD | ACK`：SSPLIT 已被 TT 接受
`HCINT=0x42`	`CHHLTD | NYET`：CSPLIT 暂时没有结果
`HCINT=0x03`	`CHHLTD | XFERCOMPL`：DMA 缓冲里已经是完整的报告

这里有三条规则，每一条都是在真机上踩过才写进代码的：低速设备的每一个事务都要在 `HCCHAR` 里置 `LSPDDEV`；CSPLIT 收到 NAK 说明这次事务已经结束（设备没数据），必须从新的 SSPLIT 重来，而不是继续发 CSPLIT；CSPLIT 收到 NYET 只能在同一个帧的窗口里重试，超出窗口就放弃本轮、等 `bInterval` 后重新开始。

HCSPLT：把一次事务拆成两拍的寄存器。 是否走 split 完全由 `HCSPLT` 决定，而 split 的两拍在寄存器层面只差一位，如图 7.19。`HUBADDR` 和 `PORT` 来自枚举时记下的路由表 `usb_route[addr]`[1]。剩下的问题是：这两拍由谁、

[1] `usb_route[]` 是 `dwc2.c` 里“设备地址 → (hub, port, low_speed)”的定长数组；枚举每发现一个 Hub 后面的设备，就把它的上游 Hub 地址和端口号记在这里，之后任何一次事务都据此决定要不要发 split、以及发给哪颗 TT。

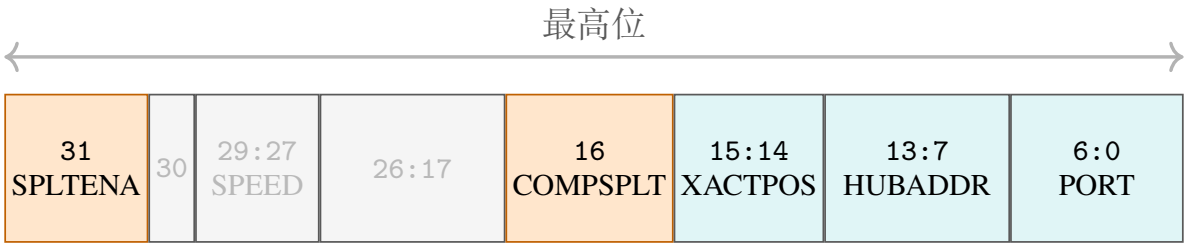


图 7.19: HCSPLT 的位域。只要 SPLTENA= 1，这次 channel 事务就不由 DWC2 自己驱动总线，而是交给 HUBADDR+PORT 指定的那颗 Hub 事务转换器（TT）代跑；两拍之间只有 COMPSPLT 一位不同（SSPLIT 为 0、CSPLIT 为 1），日志里 0x8000c083 与 0x8001c083 正是这一位之差。XACTPOS 本项目恒取 ALL（一次搬整包，不做首/尾微帧的分段）

在什么时刻发出。

SSPLIT→CSPLIT 的状态机. 两条传输路径共用同一套状态语义，核心是一个 complete_split 标志：

- complete_split= 0 时发的是 SSPLIT（HCSPLT 不带 COMPSPLT）。收到 ACK | CHHLTD 表示 TT 已接受、开始在低速总线上跑真正的事务，于是置 complete_split= 1、重发同一 channel——这一次带 COMPSPLT，即 CSPLIT。
- complete_split= 1 时收 NYET 表示 TT 还没跑完，保持 CSPLIT 原地重试，**绝不重发 SSPLIT 的负载**；收 XFERCOMPL 表示数据已进 DMA 缓冲，本轮结束；收 NAK 在 split 语义下等同于“事务结束”，要退回 complete_split= 0、等下一轮从新 SSPLIT 开始。

枚举用的 ch0 走 channel_xfer() 的同步轮询循环：SSPLIT 后 udelay(125) 再 goto restart_channel 发 CSPLIT，NYET 就再 udelay(125) 重试，NAK 则 udelay(100) 后重开一次 SSPLIT。此时 CPU 没有别的事，忙等无妨。键盘的 ch6 每 bInterval（10 ms）才轮询一次，不能占着 CPU 干等，于是把同一台状态机拆进中断：dwc2_interrupt_rx_start() 负责装 channel，dwc2_interrupt_rx_irq_fast() 在 IRQ 里就地完成 SSPLIT→CSPLIT 的翻转（下一小节展开）。

微帧窗口：CSPLIT 为什么必须在中断里发. split 的时序是硬性的：CSPLIT 不能紧跟 SSPLIT 立刻发，要等 SSPLIT 之后约两个微帧（1 微帧 = 125 μs）再发——太早 TT 还没准备好，太晚就跨帧错过窗口。dwc2_wait_frame_delta(base, 2) 就是读 HFNUM 忙等到“距 last_issue_frame 已过 2 个微帧”，并带一个 500 μs 的兜底超时。这里的关键是把 ssplit_ack_frame 记在 IRQ 时刻：早期版本在 kworker 里记，而 kworker 要等调度器给它 CPU，延迟是毫秒级、比微帧粗了一个数量级，于是 CSPLIT 总是发在 SSPLIT 之后好几个帧、永远错过。NYET 的重试同样被限制在“同一个帧”内：把 HFNUM 的低 3 位当帧内微帧号、其余当帧号，只要 now>>3 还等于 ssplit_ack_frame>>3 且重试次数 < 3，就在下一微帧重试；一旦跨到下一帧就放弃本轮，交给 bInterval 定时重开。这也是7.4.11 小节“上半部恰恰是唯一来得及的地方”那条原则的落点。

一次 split 只搬一个最大包，DATA 逐包翻转. 低速端点的最大包长只有 8 字节，而 18 字节的设备描述符要分 3 个数据包传。channel_xfer() 对 split 且 len>mps、又非 SETUP 的请求，按 mps 切片、每片递归做一次完整的 SSPLIT/CSPLIT 事务，并在片与片之间翻转 PID_DATA0/PID_DATA1。interrupt-IN 亦然：起始 pid 由 udev->interrupt_in_toggle 决定，一次成功轮询后翻转（^= 1）；设备端维护同样的 toggle，两边不一致就报 HCINT_DTERR（数据翻转错）。这正是7.4.11 里“阶段 4：读到了但其实没读全”的根因——旧代码把整个 18 字节当成一次 split 请求，只取回第一个包，后面 10 字节还是缓冲区里的 0。

full-speed 也要 split，但不置 LSPDDEV. 需要 split 与否取决于“设备是否挂在 Hub 后面”（`usb_route[addr].hub != 0`），而不是速度；`LSPDDEV`（`HCCHAR` bit17）则只在设备是 low-speed 时置。所以高速 Hub 后面的 full-speed 复合设备（如 `046d:c534` 无线接收器）同样要走 SSPLIT/CSPLIT，只是 `LSPDDEV= 0`——9.2 小节会看到这条规则在真机日志里的体现。

7.4.7 URB 与中断驱动的接收

USB core 提供与 Linux 同名的异步接口：

```
struct urb *usb_alloc_urb(void);
void usb_fill_int_urb(struct urb *urb, struct usb_device *dev, int ep, void *buf,
                      int len, void (*complete)(struct urb *), void *context, int interval);
int  usb_submit_urb(struct urb *urb);       // 提交给 HCD，立即返回
void usb_kill_urb(struct urb *urb);         // 同步取消
```

URB 的所有权在提交时交给 HCD，完成时由 `usb_hcd_giveback_urb()` 交还类驱动并调用 `complete` 回调，回调通过 `urb->context` 找回所属的键盘。`dwc2_submit_urb()` 把 ch6 配好就返回：

```
wr(HCINTMSK(6), HCINT_XFERCOMPL | HCINT_CHHLTD | HCINT_NAK |
                HCINT_ACK | HCINT_NYET | HCINT_ERRORS);
if(usb_route[addr].hub)                         // 在 Hub 后面：split 事务
  split = HCSPLT_SPLTENA | HCSPLT_XACTPOS_ALL |
          HCSPLT_HUBADDR(usb_route[addr].hub) | HCSPLT_PRTADDR(usb_route[addr].port) |
          (req->complete_split ? HCSPLT_COMPSPLT : 0);
wr(HCSPLT(6), split);
wr(HCDMA(6),  DWC2_DMA_BUS(urb->transfer_buffer));
wr(HCTSIZ(6), HCTSIZ_XFERSIZE(8) | HCTSIZ_PKTCNT(1) | HCTSIZ_PID(pid));
wr(HAINTMSK, rd(HAINTMSK) | (1U << 6));
wr(HCCHAR(6), HCCHAR_DEVADDR(addr) | HCCHAR_EPNUM(ep) | HCCHAR_EPTYPE(EPTYPE_INTERRUPT) |
              HCCHAR_MPS(8) | HCCHAR_EPDIR_IN |
              (usb_route[addr].low_speed ? HCCHAR_LSPDDEV : 0) | HCCHAR_CHENA);
req->last_issue_frame = rd(HFNUM) & 0x3fff;     // 记下发出时的微帧号
```

把一次按键的完整所有权转移摊开，是这样一条链：

```
usbkbd_probe()
  -> usb_alloc_urb() / usb_fill_int_urb(EP1 IN, 8 字节, interval 10 ms)
  -> usb_submit_urb(urb)              URB 标记为 submitted，所有权交给 HCD
  -> udev->ops->submit_urb()          DWC2 配置 ch6 DMA 后立即返回
  -> HCD 推进 SSPLIT/CSPLIT，有报告时 DMA 写入 buffer
  -> usb_hcd_giveback_urb(status, actual_length)   所有权交还类驱动
  -> urb->complete(urb)               usbkbd_irq_complete()
  -> queue_work(&usbkbd_wq, &kbd->rx_work)   用 urb->context 找回这台键盘
  -> usbkbd_rx_work() 解析报告，最后再次 usb_submit_urb()
```

每设备状态，不是全局单例. `usbkbd_state` 由 `usbkbd_probe()` 用 `kalloc()` 逐设备分配，保存在 `udev->dev.driver_data`，报告缓冲、上一份报告、capslock 状态、`rx_work`、自己的 `kbd->lock`，以及那只 interrupt-IN URB 全在这份私有状态里。完成回调只认 `urb->context`，不再依赖一个静态的“当前键盘”。输入设备名 `USB HID Boot Keyboard`、`phys` 为 `usb/dwc2/input0`、bustype、从描述符读到的 VID/PID 也在这里填好，再用 `input_set_capability()` 把 `hid_to_key[]` 里所有非零项登记成这台设备支持的按键能力位图。传

输、完成回调、work 和生命周期因此都是按设备走的：换一只键盘、再插一只鼠标，用的是同一段类驱动代码，而不是某个被硬编码的设备。

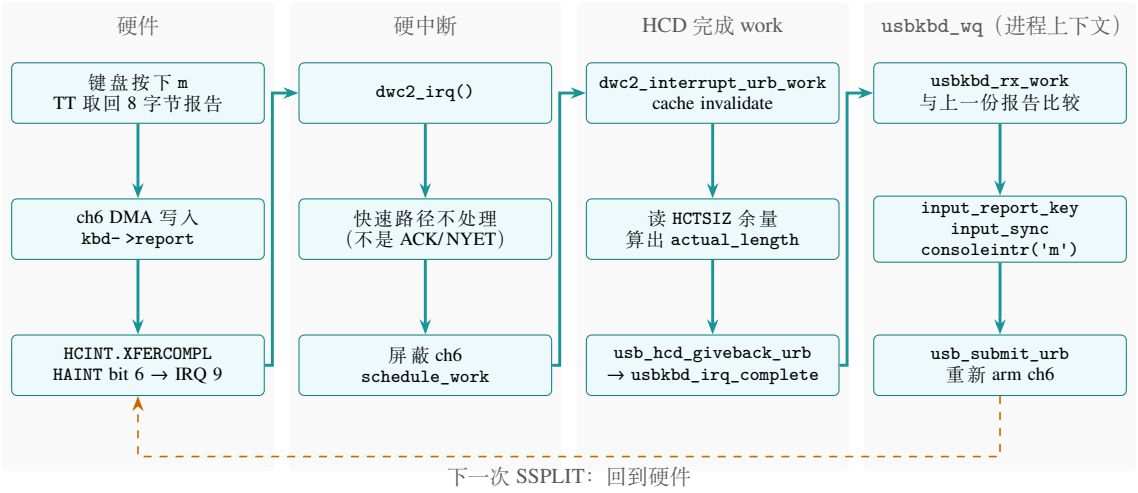


图 7.20: 一份键盘报告从硬件到 shell 的路径。每一列是一种执行上下文：硬中断只做与微帧时序有关的最少工作；HCD 的完成 work 负责缓存维护并把 URB 归还类驱动；HID 解析、输入事件和控制台都在键盘自己的 workqueue 里完成，最后重新提交 URB。键盘回 NAK（没有新按键）时，HCD completion 只登记 `bInterval` 期限，到期后由专用 DWC2 periodic workqueue 重发，URB 不归还类驱动

之后的事全由中断推动（图 7.20）。`dwc2_irq()` 先交给快速路径 `dwc2_interrupt_rx_irq_fast()`：SSPLIT 的 ACK 和窗口内的 NYET 在这里当场处理——记下当前微帧，忙等到“发出后第 2 个微帧”（最多 500 μs），立即重发 CSPLIT——因为它们有微帧级的截止时间，交给任何可调度的线程都来不及。只有最终结果（完成、无数据或错误）才屏蔽 ch6 并 `schedule_work`。HCD 的完成 work 做缓存维护、算实际长度并归还 URB；键盘的完成回调只 `queue_work(kbd->rx_work)`；`usbkbd_rx_work()` 在进程上下文里比较新旧报告、上报输入事件、调用 `consoleintr()`，最后重新 `usb_submit_urb()`。硬中断只管微帧时序，公共 `system_wq` 只判定控制器结果，专用 periodic workqueue 只负责到期重试，类驱动自己的 `usbkbd_wq` 只管 HID。

图 7.21 把同一条路径按时间顺序摊开，并分出两种结果：键盘回 NAK 时 URB 始终不归还类驱动，只有拿到完整报告才走 `usb_hcd_giveback_urb()`。

NAK 的无忙等周期重试. ch6 的最终 IRQ 仍通过 `schedule_work(&interrupt_rx.work)` 立即进入公共 `system_wq`。`dwc2_interrupt_urb_work()` 调用 `dwc2_interrupt_rx_complete()` 读取并清除 `HCINT`；若本轮是 NAK，或 CSPLIT 已越过当前 frame 的有效窗口，它不再执行毫秒级 `udelay(bInterval)`，而是调用：

```
mod_delayed_work(&dwc2_periodic_wq, &interrupt_rx.retry_work,
                 interval_jiffies);
```

这时 `retry_work` 先进入 workqueue 核心按 `expires` 排序的全局 `delayed_head`，并未进入可执行队列。CPU0 的 10 ms timer IRQ 调用 `workqueue_timer_tick()` 推进 `work_jiffies`；到期后才把其中的普通 `work_struct` 移入只有一个 kworker 的 `dwc2_periodic_wq`。该 worker 执行 `dwc2_interrupt_retry_work()`，从新的 SSPLIT 重新 arm ch6。等待期间没有 kworker 运行，原 URB 始终保持 submitted，也不会调用键盘类驱动。

拔出/取消时先在 `usb_lock` 下清除 `interrupt_rx.active` 并停止 ch6，再在锁外同步取消 delayed work：

```
cancel_delayed_work_sync(&interrupt_rx.retry_work);
```

到期回调也要取得 `usb_lock`，因此同步取消绝不能在持锁状态下等待；这个顺序同时保证取消返回后不会有旧 retry 重新 arm 已经释放的 USB 端点。

拔出设备时顺序相反：`usbkbd_remove()` 先把 `active=0`、`disconnected=1`，再 `usb_kill_urb()`——它停掉 ch6 的 DMA、屏蔽 `HCINTMSK/HAINTMSK` 对应位、同步取消 HCD 自己的完成 work，并等待正在运行的 completion 返

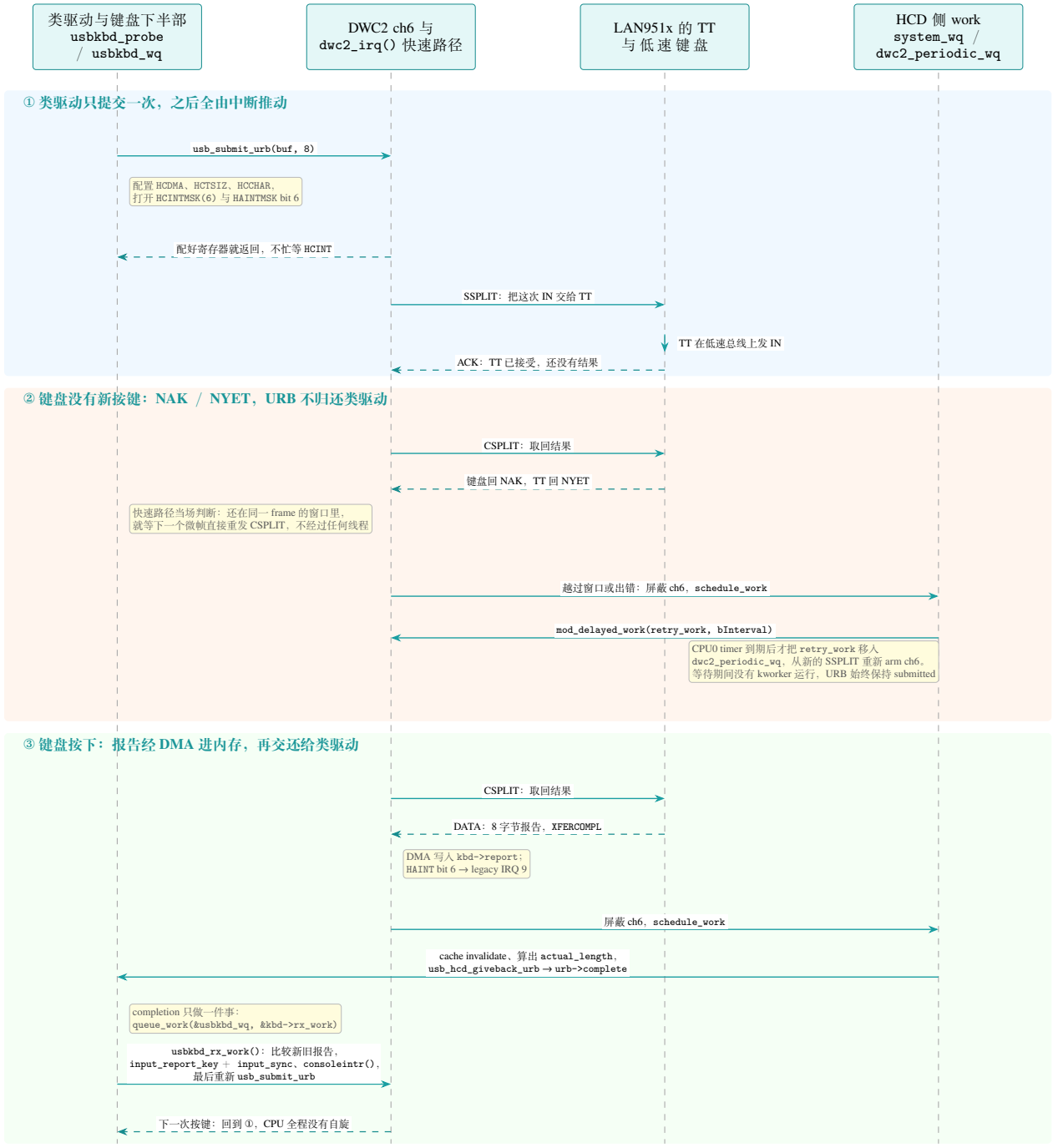


图 7.21: 运行期一次 interrupt-IN 轮询的完整时序。类驱动提交 URB 之后就不再参与：DWC2 与 Hub 的 TT 用 SSPLIT / CSPLIT 完成低速事务，有微帧截止时间的中间事件在硬中断里当场推进。上半段是键盘没有新按键的情况，控制器只登记 `bInterval` 期限，到期后由专用 periodic workqueue 重发，URB 不归还类驱动；下半段才是真正的一份报告，依次经过 DMA、IRQ、HCD completion、`usbkbd_wq`，最后落到控制台与 `event0`

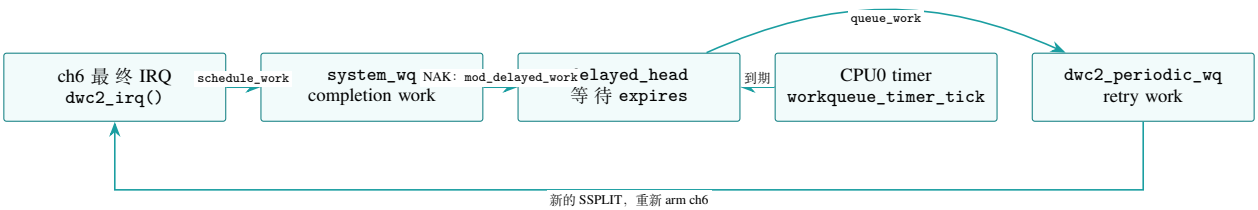


图 7.22: 空闲键盘 NAK 后的调用链。`system_wq` 与 `dwc2_periodic_wq` 使用同一种 FIFO/kworker 执行机制；定时语义来自二者之外的 `delayed_head`。因此 `mod_delayed_work()` 会立即返回，不会把等待转移成另一个 kworker 上的忙等

回；然后 `cancel_work_sync(&kbd->rx_work)`；最后才注销 input 设备、释放 URB 和这份私有状态。这两步的返回语义就是保证本身：`usb_kill_urb()` 返回后不会再有 DWC2 completion 摸到这只 URB，`cancel_work_sync()` 返回后再也没有键盘下半部访问设备私有内存，正对应 Linux 在 `disconnect()` 里对 `usb_kill_urb()` 的要求。

7.4.8 HID 报告与输入子系统

一份 8 字节报告到手，`usbkbd_rx_work()` 要同时喂两个消费者：evdev 要的是“哪个键按下、哪个松开”，控制台要的是“打出了什么字符”。两者的判断都建立在同一个动作上——把新报告和 `kbd->previous` 逐槽比较。唯一需要提前特判的是 rollover：按键多到键盘自己无法分辨时，六个键槽会被全部填成错误 usage（1 ~ 3），这份报告并不说明任何键的真实状态，驱动必须原样保留上一份。若把它当成“全部松开”，就会先上报一串假松开，下一份报告再把同样的键当作新按下，同一个字符因此重复两遍。

之后的事由输入子系统（`kernel/drivers/input/input.c`、`include/kernel/input.h`）完成。类驱动只用到七个接口：

```
struct input_dev *input_allocate_device(void);
void input_set_capability(struct input_dev *dev, int type, int code);
int  input_register_device(struct input_dev *dev);
void input_report_key(struct input_dev *dev, int code, int value);
void input_sync(struct input_dev *dev);
void input_unregister_device(struct input_dev *dev);
void input_free_device(struct input_dev *dev);
```

`probe()` 用前三项登记设备，运行期只用中间两项上报状态变化，remove 时用后两项收尾；事件去重、按键状态、环形队列、阻塞读、`SYN_REPORT` 和 `SYN_DROPPED` 全部由 core 负责，`usbkbd.c` 里只剩 USB 与 HID 的职责。core 再把事件分发给两个“处理者”：

- **evdev**：`/dev/input/event0`。每次 open 都分配一个独立的客户端环形缓冲区（`fops.open` 设置 `f->private_data`），读者在自己的缓冲区上睡眠；溢出时插入 `SYN_DROPPED`；
- **控制台**：把按键转成字符（处理 Shift、Caps Lock、Ctrl），送入 `consoleintr()`。所以 USB 键盘和串口一样可以登录、输入命令、按 `Ctrl-C`。

这正是 Linux 的分层：硬件驱动只关心“产生了什么事件”，不关心“谁在读”。图 7.23 把这条分发路径画全：同一个 `usbkbd_rx_work()` 先经 input core 把键码发布成事件，再单独把字符送进控制台。

内核侧的按键序列. `usbkbd_rx_work()` 处理一份报告时的动作顺序值得摊开看，因为它同时喂了上面两个处理者：

1. 本次 `report[2..7]` 里有、`previous[2..7]` 里没有的 usage 是新按下：先 `input_report_key(kbd->input, hid_to_key[code], 1)`，再查 `keymap/shiftmap` 生成字符；
2. 上次有、本次没有的 usage 是松开，`input_report_key(..., 0)`；usage 1 ~ 3 是错误码，两端都跳过；
3. `report[0]` 的修饰位图逐个和 `previous[0]` 比较，变化的位用 `hid_to_key[224+i]`（0xe0 ~ 0xe7）上报按下或松开；
4. 收尾调用 `input_sync()`，产生一个 `EV_SYN/SYN_REPORT`，一份报告就是一个完整帧；
5. 普通字符、Backspace、Enter 以及方向键的 `ESC [ A/B/C/D` 转义序列全部送进 `consoleintr()`，与 UART 收到的字节走同一条路径；
6. 最后 `usb_submit_urb()` 重新 arm ch6。

字符映射是两张稀疏的表，一张出字符、一张出 Linux 键码：

```
// kernel/drivers/input/usbkbd.c，节选。两张表都按 HID usage 编号索引
static const uchar keymap[58]   = { [4]='a', [16]='m', [30]='1', [40]='\n',
                                    [42]='\b', [43]='\t', [56]='/' };
```

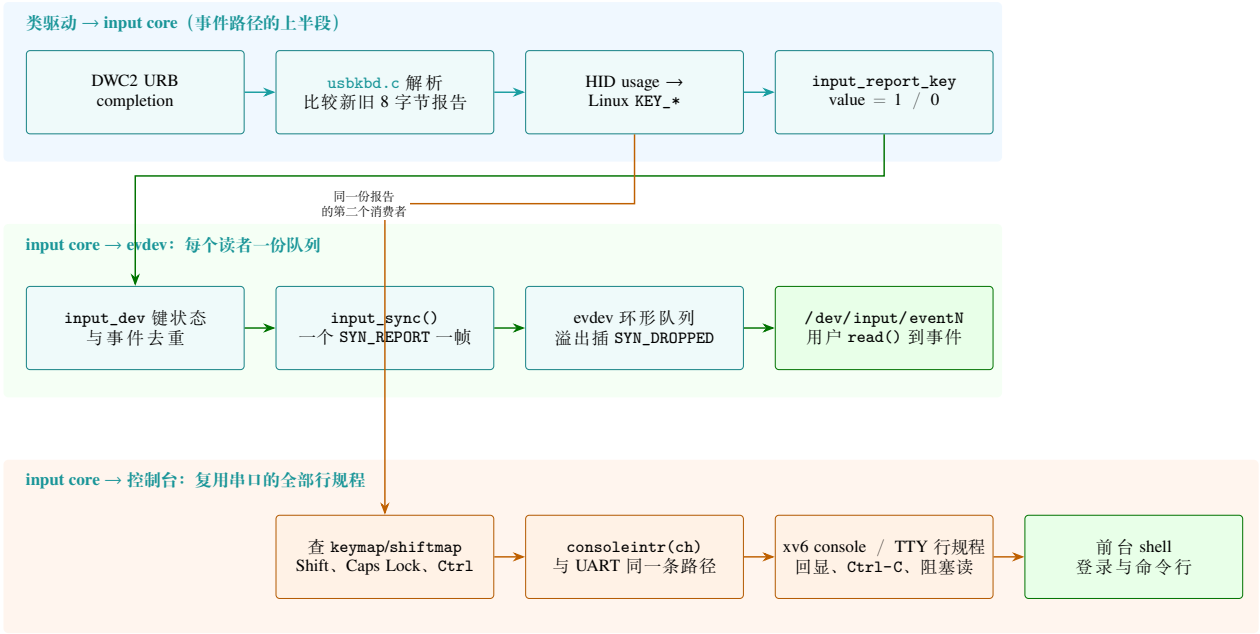


图 7.23: 一份 HID 报告进入输入子系统后的两条出路。上面两行是事件路径：类驱动解析报告、转成 Linux 键码，由 input core 去重、成帧，再复制到每个 open 自己的 evdev 队列；最后一行是字符路径：同一份报告查表转成字符后交给 consoleintr()，与 UART 收到的字节共用同一套行规程。两条路径互不依赖，因此读 /dev/input/eventN 的程序和登录会话可以同时存在，谁也不会把按键“读走”

```
static const uchar shiftmap[58] = { [4]='A', [16]='M', [30]='!', [45]='_',
                                    [47]='{' };
static const ushort hid_to_key[256] = { [4]=KEY_A, [40]=KEY_ENTER,
                                        [82]=KEY_UP, [225]=KEY_LEFTSHIFT };
```

keymap[]/shiftmap[] 只覆盖能直接打成 ASCII 的 usage 4 ~ 56（字母、数字、常用符号、Enter/Tab/Backspace/Esc/空格）：Shift 决定取哪一张，字母还要再异或一下软件维护的 Caps Lock 状态（usbkbd_key() 里 usage 57 只翻转这个标志，自己不产生字符）；Ctrl 加字母折成 c - 'a' + 1 这样的控制字符，所以 Ctrl-C 就是 0x03。hid_to_key[] 则是 HID usage 到 Linux KEY_* 编号的映射，覆盖普通键、四个方向键和 usage 0xe0 ~ 0xe7 的修饰键；probe() 遍历它一遍，把这台设备支持的键登记成 input 设备的能力位图，于是 evdev 的用户拿到的是与 Linux 一致的编号，而不是 HID 自己的一套。两张表都没登记的 usage——F1 ~ F12、多媒体键——既不产生字符也不上报事件，直接丢掉。

evdev 队列：每个读者一份。 /dev/input/eventN 的读语义按 Linux evdev 实现，一共四点：

- **minor 稳定绑定设备。** input_register_device() 从设备表里取一个空槽，槽号就是 eventN；open() 用 inode 的 minor 反查这张表，因此不会出现所有设备都“晋升”成 event0。devtmpfs 预先创建 /dev/input/event0..event3（主设备号 INPUT = 4，minor 0 ~ 3），空槽的节点还在但 open() 失败——这正是 Linux 的 ENODEV 语义，好处是不动态增删节点时也不会把旧节点错指给后来注册的设备。
- **每个 open 一个客户端。** evdev_open() 分配一个 evdev_client（EVDEV_BUFFER = 256 条事件的环形缓冲）挂到 dev->clients，两个读者各读各的，不会互相偷事件。
- **只暴露完整帧。** packet_head 标记最后一个完整帧的末尾，read() 最多读到那里；溢出时把尾巴丢掉并插入 SYN_DROPPED，读取者据此丢弃自己缓存的按键状态，等下一个 SYN_REPORT 再恢复。
- **断开是有顺序的。** input_unregister_device() 先把设备从表里摘掉、唤醒阻塞的读者，再让旧 fd 读完已经提交的完整帧，之后 read() 才返回错误。

每条事件都是一个 struct input_event：uint16 type、uint16 code、int value，read() 一次可以拿走多条。键盘发布 EV_KEY（value 1 按下、0 松开），Boot Mouse 还会发布 BTN_LEFT/BTN_RIGHT/BTN_MIDDLE 和

EV_REL 的 REL_X/REL_Y/REL_WHEEL，末尾统一跟一个 EV_SYN/SYN_REPORT。在真机上可以直接验证：

```
$ ls /dev/input
event0  event1  event2  event3
$ cat /proc/devices        # Character devices: 4 input/event0
$ evtest /dev/input/event0 # 每次按下/松开各一条 EV_KEY，帧尾一条 EV_SYN
```

引用计数与生命周期. input_dev 由 kalloc() 分配，初始引用属于类驱动；每个打开的 event fd 再加一个引用。usb_free_urb() 和 kfree(kbd) 发生在 remove 路径上，而 input_dev 的内存要等最后一个 fd 关闭才释放——这样“驱动已经卸载”和“用户还在读旧事件”两件事可以同时成立。注销路径因此不会睡眠等待用户空间关闭 /dev/input/event0。

7.4.9 中断传输：小、周期、低延迟

上一节把一份报告到手之后的软件分发讲完了，这一节回到总线上，看一次 interrupt 传输到底搬了些什么。§7.4 开头已经点破“interrupt”这个名字的误会：设备从不主动打断主机，是主机每 bInterval 毫秒替 CPU 问一次。interrupt 与 bulk 用的是同一套可靠机制（Handshake 加 DATA0/DATA1 翻转），唯一的区别就是这条周期承诺——主机必须按时来问，于是延迟有界；代价是每次问的量很小（不超过端点的 mps），正合 HID 按键这种“来得快、内容短”的负载。

把这三句话落到一次真实按键上。按下字母 l（没有配合任何修饰键）时，串口上会滚出三行：

```
dwc2: ch6 transfer complete addr=4 ep=1 bytes=8 data=0,0,f,0,0,0,0,0
usbkbd: channel 6 IRQ received
usbkbd: first HID report mod=0 keys=f,0,0,0,0,0
```

这三行正好是三层各说一句。第一行来自 HCD（dwc2.c）：ch6 这一轮 interrupt-IN 完成，HCINT 的传输完成位置位，DMA 已经把 bytes=8 个字节写进内存，于是把缓冲区原样打出来——addr=4 ep=1 说明这是对地址 4 的设备、端点 1（IN）的轮询。第二行是 USB core 经唯一入口 usb_hcd_giveback_urb() 回调进类驱动（边界 2，见 §7.4“内核软件框架”一节），usbkbd.c 报告“ch6 的事件到了”。第三行才是类驱动把这 8 字节拆开读：mod=0 是修饰位图，keys=f,... 是六个键槽的内容。

8 字节报文的逐字段格式. Boot Keyboard（HID boot protocol）的报告是定长 8 字节，一次 interrupt-IN 恰好搬一个完整报告，所以 bytes=8 就等于 mps，不需要像 bulk 那样靠“短包”标志结束：

表 7.10: 按下 l 时那 8 字节报告（0,0,f,0,0,0,0,0）的逐字段拆解

偏移	字段	本次值	含义
byte 0	Modifier 位图	0x00	八个修饰键各占一位（bit0–3 = 左 Ctrl/Shift/Alt/GUI，bit4–7 = 右侧四个），全 0 表示没按任何修饰键
byte 1	Reserved	0x00	Boot 协议固定保留字节，恒 0
byte 2	Key 0	0x0F	第一个键槽：HID usage 0x0f，即字母 l
byte 3–7	Key 1–5	0x00	其余五个键槽，0x00 = 该槽无按键（一份报告最多同时报 6 个键）

关键在于 byte 2 的 0x0f 是 *HID usage ID*，不是 ASCII。HID 键盘用途表从 0x04 起排字母，0x04 = a，所以 l 是第 12 个字母、usage = 0x04+11 = 0x0f；而 l 的 ASCII 是 0x6c，两者毫无关系。这正是 usbkbd.c 里 keymap[]/hid_to_key[] 要按 *usage* 编号索引（[4]='a'）而不能直接当字符输出的原因。六个键槽全 0x00 才表示“没键按下”，而 0x01 ~ 0x03 是保留的错误码（rollover 等），驱动会原样保留上一份报告——这条特判在 §7.4.8 展开。

总线上的一来一回. 这 8 字节是被一次普通的 IN 事务取回的，三相结构和 §7.4 里画的完全一样，只是没有 SETUP/STATUS 那套“先谈请求”的外壳：

```
主机每 bInterval(10 ms) 用 ch6 发起一次 IN:
  IN token    ADDR=4 ENDP=1              主机 -> 键盘   "EP1 有报告吗？"
键盘有新数据时:
  DATA0/DATA1  [8 字节负载] CRC16         键盘 -> 主机   <- 就是 data=0,0,f,0,0,0,0,0
  ACK                                    主机 -> 键盘   "收下"
键盘没有键按下时:
  NAK                                    键盘 -> 主机   "暂无数据", 本轮不算完成, ch6 稍后重新 arm
```

设备只在有新报告时才翻转 DATA0/DATA1，回 NAK 的空轮询保持翻转不变；主机侧对应地用 `udev->interrupt_in_toggle` 跟进，两边对不上就报 HCINT 的数据翻转错。因为键盘挂在高速 Hub 后面、自己是低速设备，上面这组 IN/DATA/ACK 实际是被拆成 SSPLIT/CSPLIT 两拍由事务转换器代跑的（§7.4 的 split transaction 一节）——但对类驱动而言，看到的仍是“ch6 完成、8 字节到手”这一个干净的事件。

数据到手之后的事——与 `kbd->previous` 逐槽比较、`input_report_key()` 上报、查 `keymap` 生成字符送 `consoleintr()`、再 `usb_submit_urb()` 重新 arm ch6——就是 §7.4.8 的内容了。下一小节把这份一模一样的 8 字节报告拿去和 bulk 对照，看“周期承诺”这一条差异如何决定键盘非用 interrupt 不可。

7.4.10 Bulk 传输：量大、可等待、不许出错

前面反复出现的键盘，其实不是 bulk 设备：它那份 8 字节 Boot 报告走的是 interrupt-IN（ch6）。把这份报告拿来和 bulk 对照，最能划清两者的边界。bulk 的定位是“数据多、什么时候到都行、但一个字节都不能错”——它和 interrupt 一样靠 Handshake 与 DATA0/DATA1 翻转保证可靠，却**放弃了周期承诺**：不预留带宽，只在每帧里别人用完剩下的时间见缝插针地搬，一次可以推很多包，发不动时靠设备回 NAK 做背压，主机稍后重发，直到回 ACK。

表 7.11: 同一份 8 字节键盘报告，走 interrupt 与“假设走 bulk”的差别

维度	走 Interrupt（HID 的实际做法）	走 Bulk（假设）
调度承诺	每 `bInterval`（此处 10 ms）必被轮询一次	只在剩余带宽里跑，无时间承诺
单次数据	≤ mps（8 字节），为小报告量身	高速下一个包 512 字节，搬 8 字节纯属浪费
延迟	有界，按键“立刻”出现	可能长时间不服务，按键忽快忽慢
端点	interrupt-IN EP1（描述符里属性 = 3）	需要 bulk 端点，键盘根本没有

一次 bulk 事务仍是三相，只是没有 SETUP/STATUS 那套“先谈请求”的结构，纯做单向搬运：OUT 方向主机发 `OUT` token 加 Data 包，设备回 ACK（收下）、NAK（缓冲区满，稍后再试）或 STALL（端点被 halt）；IN 方向主机发 `IN` token，设备回 Data、NAK（暂无数据）或 STALL。数据翻转保证主机不会把重传包误当成新数据，短于 mps 的包标志一次 bulk 传输结束。

本项目里真正的 bulk 用户是 MT7601U 无线网卡的 bulk-IN/bulk-OUT（ch4，详见 §7.8）：收空中进来的网络帧、发待发送的报文，数据量随流量起伏、又绝不容错，正是 bulk 的教科书场景——与“小、周期、低延迟”的键盘 interrupt 恰好互补。

7.4.11 真机排错：从 STALL 到第一份报告

这个驱动在 QEMU 上很快就能工作，到了真机（低速无线键盘接收器 `2a7a:8a47` 插在 LAN9514 的 port 3）却经历了九个阶段才读到第一个按键。表 7.12 按时间顺序列出每一步的现象、根因和修复；每一步只把串口日志实际证明的东西当作结论——ACK 不等于下游数据已经返回，“ready”也不等于收到了按键。

其中几个阶段值得多说几句。

表 7.12: USB 键盘真机排错的九个阶段

	现象	根因	修复
1	port 3 状态 `0x301`，复位后 `0x303`	低速设备在高速 Hub 后面	记录父 Hub、端口、速度，走 split 事务
2	`control setup failed req=6 addr=0, intr=0x0a`	SETUP 的 SSPLIT 没设 `LSPDDEV`，TT 回 STALL	按 `usb_route` 给所有低速事务置 `LSPDDEV`
3	GET_DESCRIPTOR 数据阶段返回 −2（超时）	CSPLIT 收到 NAK 后仍反复发 CSPLIT	NAK 时清 `complete_split`，下一帧重发 SSPLIT；NYET 才继续 CSPLIT
4	设备描述符 `class=0 vid=0 pid=0`	18 字节跨 3 个 8 字节包，被当成一个 split 请求，只收到前 8 字节	按最大包长分包，每包单独 SSPLIT/CSPLIT，完整收到一包后翻转 DATA0/1；检查长度、类型和 VID
5	“not a boot keyboard”	复合设备的后一个接口覆盖了键盘接口的属性	先定位 Boot Keyboard 接口，端点只在该接口内选
6	ready 之后又出现 `req=6 addr=4` STALL	绑定键盘后仍继续 CDC-ECM 网卡探测，向键盘要配置描述符	HID 交给 USB 总线后直接返回
7	按键无响应，反复 `hcint=0x42`	把每个 NYET 都当作“TT 还在忙”，永久重发 CSPLIT	记录 SSPLIT ACK 的帧号，同帧内最多重试 3 次；跨帧则结束本轮，等 `bInterval`
8	`ssack` 比 `issue` 晚几十个微帧	ACK 交给 kworker 处理，调度延迟毫秒级，CSPLIT 窗口早过了	SSPLIT ACK 和窗口内 NYET 改在 IRQ 快速路径里处理
9	快速路径启用后仍偶尔跨帧	快速路径之前有一行 `printf`；115200 波特率下一行要几毫秒	中间状态不打印，只在本轮结束后限量打印 `ch6 final`

阶段 4：“读到了”但其实没读全。 低速设备端点 0 的最大包长只有 8 字节，18 字节的设备描述符要分 3 个数据包传。经过 split 事务时，每个包都要单独走一遍 SSPLIT/CSPLIT，而且收完一个完整的包后要把 DATA0/DATA1 翻转。旧代码把整个 18 字节当成一次 split 请求，只取回了第一个包，后面 10 字节还是缓冲区里的 0——于是日志说 `vid=0 pid=0`。现在 `channel_xfer()` 对 split 请求按最大包长切片，`get_device_descriptor()` 还检查长度、类型和 VID 不为 0，宁可失败也不接受半截数据。

阶段 7 ~ 8：时间窗口。 周期性的 split 事务有严格的时序：CSPLIT 应在 SSPLIT 之后第 2 个微帧开始，并且只能在同一个帧内重试。早期代码对 NYET 无条件重试，永远等不到结果；改成按帧号判断窗口后，又发现记录下来的“ACK 帧号”比实际晚了几十个微帧——因为 ACK 是由 kworker 处理的，而 kworker 要等调度器给它 CPU，延迟是毫秒级的，比 125 μs 的微帧粗了一个数量级。解决办法是把这两种有截止时间的中间事件移到中断上下文里（`dwc2_interrupt_rx_irq_fast()`），只把“这一轮已经结束”的结果交给线程。这是“上半部只做最少的事”这条原则的一个反例：对有硬实时要求的状态推进，上半部恰恰是唯一来得及的地方。

阶段 9：观察者效应。 快速路径加上以后，问题偶尔还在。原因是 `dwc2_irq()` 开头还留着一行调试 `printf`：串口 115200 波特率下每个字符约 87 μs，一行几十个字符就是好几个毫秒，足够错过整个窗口。这是典型的“加日志就改变时序”的 Heisenbug。最终版本在有截止时间的分支里一行都不打印，只在本轮结束之后限量打印 `dwc2 ch6 final`；如果确实需要跟踪中间事件，应该写进内存里的 trace 环形缓冲，事后由进程上下文打印。

限量日志各自证明什么。 排错时最容易把“请求已经发出”当成“数据已经回来”，所以这四行日志的边界要背下来：

- `dwc2: ch6 armed addr=4 ep=1 split=...`：`HCSPLT`、`HCCHAR`、`HCTSIZ`、`HCINTMSK(6)`、`HAINTMSK` 和 `GINTMSK` 都已配置，channel 已使能。仅此而已，键盘的一个字节都还没到。
- `dwc2 ch6 final p=... int=... uf=... issue=... age=... ssack=...`：本轮已经走到“不再需要满足当前 CSPLIT 实时期限”的出口才会打印。
- `usbkbd: channel 6 IRQ received`：HCD 已归还 URB，键盘的 completion 把工作排进了 `usbkbd_wq`。
- `usbkbd: first HID report mod=0 keys=10,...`：workqueue 读到了 DMA 缓冲里的报告，`keys=10` 是 usage `0x10`，即字母 `m`。

`ch6 final` 那一行的字段就是为阶段 8 和 9 准备的：`p` 是本轮处在哪个 split 阶段（`p=1` 表示 complete-split），`int` 是 `HCINT`，`uf` 是 IRQ 采到的微帧号（`HFNUM` 低 3 位），`issue` 是本次 channel 使能时的计数器，`age` 是从发出到 IRQ 经过的微帧数，`ssack` 是最近一次 SSPLIT 被 ACK 时的计数器。对比 `issue`、`age` 和 `ssack` 就能分辨

“总线慢”还是“软件来晚了”。因为中间状态一律不打印，这行日志还做了限量（前 12 条），空闲键盘的 NAK 轮次也不消耗这个额度。HCINT 常用取值的解码见表 7.9：0x22 是 CHHLTD|ACK（SSPLIT 已被 TT 接受），0x42 是 CHHLTD|NYET（CSPLIT 还没有结果），0x03 才是 CHHLTD|XFERCOMPL（DMA 里是完整报告）。

诊断顺序. 上面四条日志本身就是一条顺序链：停在哪个标记之前，就说明卡在那一层——armed 之前是 channel 配置，armed 与 final 之间是总线和 split 时序，final 之后是 workqueue 与 completion。只有 first HID report 一直是好的而控制台没有字符时，才需要往下查键码映射、输入子系统和 /dev/input/eventN。

从两层轮询到中断驱动. 最早的实现虽然端点类型叫 interrupt-IN，CPU 侧的处理方式仍然是轮询，而且是两层：

```
第一层：ARM Generic Timer IRQ
  -> workqueue_timer_tick()
  -> kbd->poll_work 到期，进入 usbkbd_wq
  -> usbkbd_poll() 主动发起一次 interrupt-IN
第二层：udev->ops->interrupt()
  -> dwc2_usb_interrupt() -> channel_xfer(ch6)
  -> while: rd(HCINT(6))          CPU 忙等本轮完成 / NAK / 出错 / 超时
  -> 解析报告，queue_delayed_work(..., 1) 排下一轮
```

这里要把两件事分开：USB 协议上主机必须发起 IN 事务，这是总线规则；但这些事务由 DWC2 硬件按 bInterval 排布、完成后用 IRQ 通知 CPU，还是让 CPU 定时自己启动并忙等，则是实现选择。旧实现选了后者，于是每个 tick 都有一次调度开销，而每次事务又占着一个 CPU 自旋。改成 URB 之后，keyboard.poll_work 和 usbkbd_poll() 被删掉，取而代之的是每设备的 kbd->rx_work（只有 URB completion 会调度它）、host ops 里的 submit_urb()/kill_urb()，以及 HCINTMSK(6) 打开 XFERCOMPL/CHHLTD/NAK/ACK/NYET 与错误事件。空闲键盘返回 NAK 时 CPU 仍要处理一次很短的 IRQ 和 completion，但等待下一个 bInterval 的阶段只保存在 delayed-work 的到期链表里，不再由 CPU 或 kworker 自旋。

现在的执行链把“谁在等”换成了“谁被叫醒”，完整的一条链是这样：

```
usbkbd_probe()
  -> 首次 arm 长期 interrupt-IN DMA 请求，随即返回，不等待传输
DWC2 硬件按 bInterval 调度 IN 令牌
  -> 低速键盘经 LAN951x 的 TT 执行 SSPLIT/CSPLIT
  -> 有报告时 DMA 写入缓冲：HCINT(6).XFERCOMPL -> HAINT -> IRQ 9
dwc2_irq()                          硬中断 / HCD 快速路径
  -> SSPLIT ACK：就地记下 HFNUM，到 issue+2 微帧提交 CSPLIT
  -> 窗口内 NYET：下一个微帧直接重试 CSPLIT
  -> XFERCOMPL / 超出窗口 / 错误：屏蔽 ch6，schedule_work
DWC2 completion work                HCD 可调度上下文
  -> 读 HCINT/HCTSIZ 与 DMA 实际长度
  -> NAK 或本轮无数据：把 retry_work 登记到 delayed_head
  -> 最终完成：usb_hcd_giveback_urb()
CPU0 的 10 ms timer
  -> workqueue_timer_tick() 推进 work_jiffies
  -> 到期后移入 dwc2_periodic_wq，从新的 SSPLIT 重新 arm ch6
     （URB 始终保持 submitted，不归还类驱动）
usbkbd_irq_complete()               URB completion
  -> 用 urb->context 找回 usbkbd_state，queue_work(kbd->rx_work)
usbkbd_wq                           进程上下文下半部
  -> 解析报告、input_report_key()/consoleintr()
```

```
-> 再次 usb_submit_urb(), arm 新的一轮
```

图 7.21 是同一条链的时间顺序版本，两种结果（NAK 与完整报告）在那里分开画。

7.5 I2C

USB 是高速、复杂的外设总线；I2C 则是板上芯片之间最简单的低速控制总线。本项目只用它配置摄像头传感器，放在这里是为 7.9 节做准备。

BCM2837 的 I2C 控制器叫 BSC（Broadcom Serial Controller）。项目目前只用到 BSC0，用来配置摄像头传感器。在树莓派 3B+ 上，摄像头排线口的 I2C 走 GPIO44/45 的 ALT1 功能，而不是 40 针排针上的 GPIO0/1。

一次传输的过程是：写从机地址寄存器 A、数据长度 DLEN，清 FIFO，向控制寄存器 C 写入 I2CEN | ST（读还要加 READ），然后轮询状态寄存器 S：TXW 表示可以继续写 FIFO，RXD 表示 FIFO 有数据可读，DONE 表示传输完成，ERR 表示从机没有应答，CLKT 表示时钟拉伸超时。分频寄存器 DIV = 核心时钟 / 100 kHz。在真机上，BSC0 对摄像头传感器始终返回 NACK，驱动最后改用 GPIO44/45 上的软件 SCCB，排查过程见 7.9.6 小节。

```
int i2c_write(int bus, uint8 addr, const uint8 *buf, int len);
int i2c_read(int bus, uint8 addr, uint8 *buf, int len);
int i2c_write_read(int bus, uint8 addr, const uint8 *wbuf, int wlen,
                   uint8 *rbuf, int rlen);     // 先写寄存器地址，再读
```

摄像头寄存器的访问很短（写是 2 字节地址加 1 字节数据，读是 2 字节地址后读 1 字节），所以全部用轮询加超时，不用中断。为了在真机上区分“控制器没配好”和“线上没有设备”，驱动还带了一个用 GPIO 模拟 I2C 的诊断路径：直接把 GPIO44/45 当开漏输出，按位产生起始、地址、应答和停止条件。

这里的“GPIO 模拟”不是另一种协议，只是把本来由 BSC0 状态机完成的动作改成 CPU 逐位执行。GPIO44/45 设为 ALT1 时，芯片内部的复用器把引脚接到 BSC0 的 SDA0/SCL0；设为普通 INPUT/OUTPUT 时，BSC0 与引脚断开，i2c.c 用方向切换模拟开漏。当前 OV5647 路径在每个软件事务开始时先用 bsc_force_off(0) 停止 BSC0，再接管引脚，因而两个发送者不会同时驱动线路。

7.5.1 开漏电气规则与总线时序

I2C/SCCB 空闲态是 SDA=SCL=1。软件驱动从不主动输出 1：bb_low(pin) 先把输出锁存器写成 0，再把引脚改为 OUTPUT，从而主动拉低；bb_release(pin) 把引脚改为 INPUT（高阻态），由相机板上的上拉电阻把它拉高。这样主机释放 SDA 后，从机可以在第九个时钟把 SDA 拉低作 ACK，不会发生“主机推高、从机拉低”的电气冲突。SCL 也使用同样的开漏模拟，所以上升沿比推挽输出慢，却能等待线路真正回高并发现 clock stretching 或总线短路。

SDA 只能在 SCL 低电平时作为普通数据变化；两个例外定义事务边界：SCL 为高时 SDA 由高变低是 START，由低变高是 STOP。写一位时，主机先在 SCL 低时设置 SDA，再释放 SCL；从机在高电平期间采样，随后主机把 SCL 拉低。读一位时，主机先释放 SDA，SCL 升高后采样 GPIO 电平，再拉低 SCL。每八位后还有第九个应答时钟：发送方释放 SDA，接收方拉低表示 ACK、保持高表示 NACK。

函数	SDA/SCL 动作	协议含义
bb_start()	释放 SDA、SCL；SDA 拉低；SCL 拉低	START
bb_write_byte()	8 个数据时钟，释放 SDA 后产生第 9 拍	从机 ACK/NACK
bb_read_byte()	释放 SDA 后采 8 位，第 9 拍由主机回应	ACK 或末字节 NACK
bb_stop()	SDA 拉低；释放 SCL；释放 SDA	STOP

当前实现的末字节 NACK 没有单独的 NoAck() 函数，而是调用 bb_read_byte(..., 0)：第九拍保持 SDA 释放，随后 bb_stop() 在 SCL 为高时让 SDA 从低回高。若还要继续读下一个字节，则参数为 1，主机在第九拍主动拉低 SDA 发送 ACK。

7.5.2 OV5647 寄存器收发序列

OV5647 的 7 位地址为 0x36，因此线上的写地址字节是 0x6c，读地址字节是 0x6d；寄存器索引是 16 位。写 0x3034=0x1a 的完整序列是：

```
START -> 6c -> ACK -> 30 -> ACK -> 34 -> ACK -> 1a -> ACK -> STOP
```

读取芯片 ID 高字节 0x300a 时，硬件 BSC 路径能够表达 repeated START；当前真机模块则使用实测稳定的 SCCB 分离事务，在写完寄存器索引后先 STOP，再以读方向重新 START：

```
START -> 6c -> ACK -> 30 -> ACK -> 0a -> ACK -> STOP
START -> 6d -> ACK -> 56 <- NACK -> STOP
```

第一段的 ACK 均由 OV5647 产生；第二段的 0x56 由 OV5647 驱动 SDA，最后的 NACK 由主机产生，表示只读一个字节。地址、模式、曝光、增益等控制量走这条低速总线；真正的像素数据走两 lane CSI-2，由 Unicam DMA 直接写内存。因此软件 SCCB 降到约 25 kHz 只会延长上电配置，不会限制视频帧率。

驱动中的调用路径. 传感器驱动只用两个函数访问寄存器。v8 起 ov5647_use_bitbang 恒为 1，所以两者都直接走软件 SCCB：

```
static int ov5647_write(uint16 reg, uint8 val) {           // kernel/drivers/media/i2c/ov5647.c
  uint8 b[3] = { reg >> 8, reg & 0xff, val };               // 索引高、低字节，数据
  return i2c0_bitbang_write_read(OV5647_ADDR, b, 3, 0, 0);
}
static int ov5647_read(uint16 reg, uint8 *val) {
  uint8 b[2] = { reg >> 8, reg & 0xff };
  return i2c0_bitbang_write_read(OV5647_ADDR, b, 2, val, 1);
}
```

i2c0_bitbang_write_read() 把一次寄存器访问完整地包在 i2c_lock 里：

```
acquire(&i2c_lock);                     // 自旋锁：整个事务期间本 CPU 关中断
bsc_force_off(0);                       // 停掉 BSC0，避免两个发送者同时驱动线路
old_sda = gpio_get_fsel(44);            // 记下 ALT1，事务结束后恢复
old_scl = gpio_get_fsel(45);
if(bb_recover_bus() < 0) goto out_stop; // 若从机把 SDA 卡在低电平，补时钟并 STOP
if(wlen){                               // 写阶段：地址+写，再写 wlen 个字节
  if(bb_start() < 0 || bb_write_byte(addr << 1) < 0) goto out_stop;
  for(int i = 0; i < wlen; i++)
    if(bb_write_byte(wbuf[i]) < 0) goto out_stop;
  bb_stop();                            // SCCB 分离事务：先 STOP
}
if(rlen){                               // 读阶段：重新 START，地址+读
  if(bb_start() < 0 || bb_write_byte((addr << 1) | 1) < 0) goto out_stop;
  for(int i = 0; i < rlen; i++)         // 除最后一个字节外都回 ACK
    if(bb_read_byte(&rbuf[i], i + 1 < rlen) < 0) goto out_stop;
}
```

```
result = 0;
out_stop:
bb_stop();
gpio_fsel(44, old_sda); gpio_fsel(45, old_scl);
release(&i2c_lock);
```

任何一个字节收不到 ACK，`bb_write_byte()` 就返回 −1，事务立即跳到 `out_stop` 发 STOP 结束。上层的 `ov5647_write_array()` 对每个寄存器最多重试三次，每次间隔 2 ms。

一位、一个字节要多长时间。 所有时间都来自两个原语。`bb_delay()` 用 `CNTVCT_EL0` 忙等 CNTFRQ/50000 个计数，即 20 µs。`bb_low()` 先动作，再等一个 `bb_delay()`。`bb_release()` 把引脚改成输入后，最多等 100 µs 让线路真的回到高电平（上拉的上升时间，或从机拉住 SCL 做 clock stretching），然后再等 20 µs；超时返回 −1，表示总线被短路或卡住。由此可以算出各个阶段的时长（表 7.13）。

表 7.13: 软件 SCCB 各阶段的时长（线路正常回高时）

阶段	动作顺序（每步约 20 µs）	时长
START	释放 SDA，释放 SCL，拉低 SDA，拉低 SCL	80 µs
写 1 位	设置 SDA（拉低或释放），释放 SCL（从机在高电平期间采样），拉低 SCL	60 µs
第 9 拍 ACK	SDA 改输入，等待；释放 SCL，等待；读 SDA（低 = ACK）；拉低 SCL	80 µs
写 1 字节	8 位 + ACK	560 µs
读 1 位	释放 SCL 后读 SDA，拉低 SCL（SDA 一直由从机驱动）	40 µs
读 1 字节	8 位 + 主机的 ACK 或 NACK（60 µs）	380 µs
STOP	拉低 SDA，释放 SCL，释放 SDA	60 µs

所以“约 25 kHz”指的是半周期 20 µs 这个名义值；写数据时 SCL 高 20 µs、低 40 µs，实际是 16.7 kHz，读数据时才是 25 kHz。OV5647 允许的 SCCB 时钟上限是 400 kHz，这里慢了一个数量级以上，换来的是克隆模块和长排线上足够的建立、保持时间裕量。图 7.24 是一次事务开头的波形：START 之后发送地址字节 `0x6c`，第九拍由 OV5647 拉低 SDA 应答。

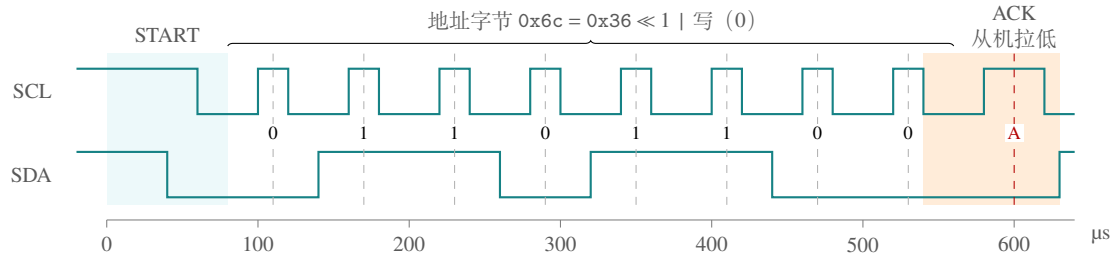


图 7.24: 软件 SCCB 发送 START 和地址字节 `0x6c`（0110 1100）的波形，横轴是由 `bb_delay()` 决定的时间。虚线是 SCL 高电平中点，从机在这里采样数据位；红色虚线是主机采样 ACK 的时刻；橙色区域里 SDA 由 OV5647 驱动——它在第 8 个时钟下降沿之后拉低 SDA，第 9 个时钟下降沿之后再释放

ACK 与 NACK 的细节。 第九拍不能用 `bb_release(44)`：这个函数会等 SDA 回高，而从机此时正要把它拉低应答，于是每次都会白等满 100 µs 的超时。`bb_write_byte()` 因此直接把 SDA 改成输入，等 20 µs，再升起 SCL，在高电平中间读 `gpio_level(44)`。读方向则反过来：`bb_read_byte()` 在 8 个时钟期间一直不碰 SDA，只在 SCL 为高时读电平；第九拍由主机决定，还要读下一个字节就拉低 SDA（ACK），最后一个字节则释放 SDA（NACK），告诉从机停止发送，接着的 STOP 结束事务。

完整事务与总耗时。 图 7.25 按时间比例画出了一次寄存器写和一次寄存器读，并用颜色标出每一段由谁驱动 SDA。

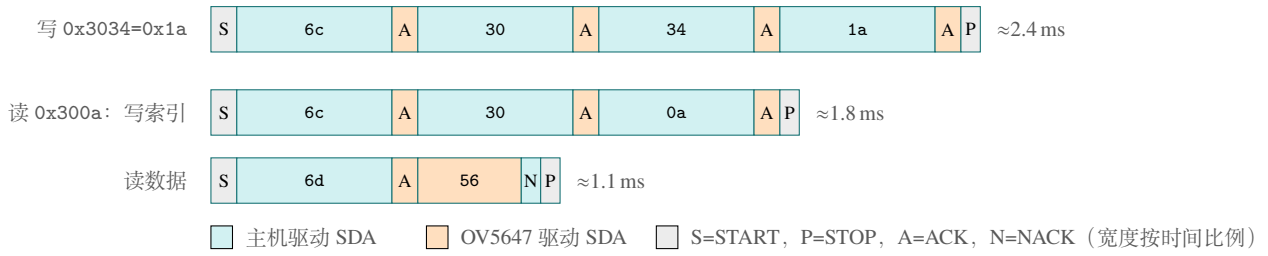


图 7.25: 寄存器写与分离式寄存器读的组成。读操作是两个独立事务：先写入 16 位寄存器索引并 STOP，再以读方向重新 START，读一个字节后主机 NACK

一次寄存器写约 2.4 ms，读一个寄存器约 2.9 ms（再加上事务开始时 `bb_recover_bus()` 检查总线空闲的约 0.1 ms）。读芯片 ID 要读 `0x300a` 和 `0x300b` 两个寄存器，约 6 ms。开始出流时 `ov5647_stream_on()` 要写 51 个公共寄存器、29 个模式寄存器、10 个控制寄存器，再加上几次读改写，总共近百次事务，约 0.25 s。这也说明了为什么连续取流之后每帧就快了：寄存器只在上电时配置一次，之后的帧都不再经过这条低速总线。

还有一个代价：`i2c_lock` 是自旋锁，持锁期间本 CPU 关中断，所以每个寄存器事务会让这个 CPU 关中断约 2.5 ms。配置期间这会抬高软实时任务的延迟，这也是以后应该把 SCCB 改成睡眠锁加 `bb_delay()` 让出 CPU 的原因。

写事务末尾的两个 STOP. 只写不读时（`rlen = 0`），写阶段末尾已经调用过一次 `bb_stop()`，跳到 `out_stop` 后又调用一次。第二次调用时 SCL 和 SDA 都是高电平，`bb_stop()` 先拉低 SDA——在 SCL 为高时这正是一个 START，随后释放 SDA 又构成 STOP。于是总线上多出一对空的 START/STOP。I2C/SCCB 从机会把它当作一个没有地址的空事务忽略，所以不影响功能，但用逻辑分析仪抓波形时会看到它。

7.6 网络协议栈

在讲两块无线网卡之前，先看看它们的上层。`kernel/net/net.c` 参考 MIT xv6 网络实验的分层，逐步扩展成了一个小而完整的协议栈：

- `struct net_device`：每块网卡一个，保存 MAC、IP、掩码、网关、DNS 和发送函数 `start_xmit`；驱动收到帧后调用 `net_rx_dev(dev, frame, len)`；
- Ethernet、ARP（表项以 `(ip, dev)` 为键，多网卡互不干扰）、IPv4、ICMP、UDP、DHCP；
- 路由表：最长前缀匹配，同长度按 metric 选择，DHCP 成功后自动加直连路由和默认路由；
- TCP：被动打开（`listen/accept`），每连接 8 KiB 异步发送队列，单段停等式重传；
- socket 文件描述符和 epoll：socket、PTY、管道都是 `struct file`，可以统一 `read/write/epoll`。

在这之上，用户态实现了 `ping`、`dhcp`、`tftp`、`netdns`，以及一个基于 TweetNaCl 的 SSH 服务器（第 8 章）。

7.7 SDIO 无线网卡：BCM43430/43455

7.7.1 硬件路径

树莓派 3B 板载 BCM43430，3B+ 板载 BCM43455。它们通过 SDIO 接在 Arasan SDHCI 控制器（`0x3f300000`，GPIO34 ~ 39 ALT3，IRQ 62）上。这里有两个容易混淆的概念：Arasan 是 SDIO *主机控制器*，负责产生 SDIO 命令和时钟、搬运数据；BCM43455 是 *FullMAC* Wi-Fi 芯片，片内有一颗 Cortex-R4 运行厂商固件，处理大部分 802.11 MAC 工作。

7.7.2 硬件结构：Arasan、SDIO 总线与 FullMAC 芯片

和摄像头一样，先把参与的部件和它们之间的通路理清楚（图 7.26）。

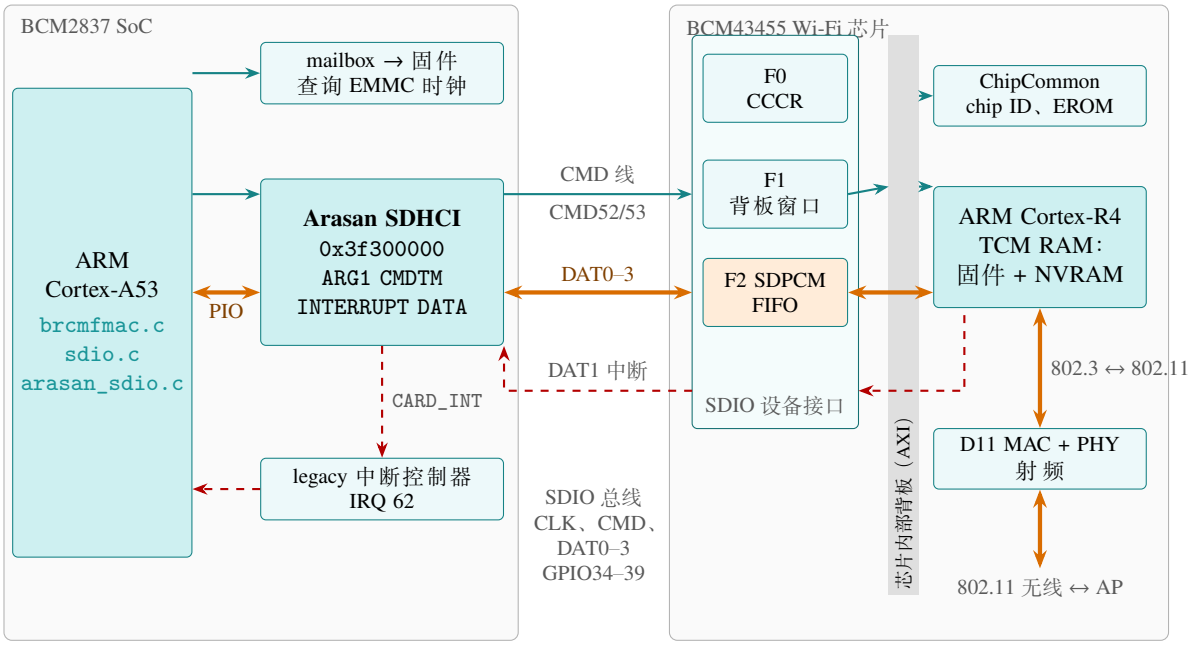


图 7.26: SDIO Wi-Fi 的硬件拓扑。细实线是控制面：CMD 线上的 CMD52/CMD53 经 F0 配置总线、经 F1 访问芯片内部背板（读 chip ID、下载固件、控制 CR4 复位）；粗线是数据面：CPU 以 PIO 方式经 Arasan 的 DATA 寄存器读写 F2 的 SDPCM FIFO，芯片固件再与 802.11 帧互转；虚线是事件：固件经 DAT1 发出卡中断，成为 BCM2837 的 IRQ 62

两颗处理器。 这条链路两端各有一颗 CPU：一端是运行 xv6 的 Cortex-A53，另一端是 BCM43455 片内的 Cortex-R4。R4 执行主机下载进去的厂商固件，负责扫描、认证、关联、管理帧、ACK 与重传、用主机装入的密钥做 CCMP 加解密，以及 802.11 帧与以太网帧之间的转换——这就是 *FullMAC* 的含义。主机看到的是一个“收发以太网帧、另外接受若干控制命令”的设备，802.11 的细节全藏在固件里。7.8 节的 MT7601U 恰好相反，是 SoftMAC：802.11 帧由主机驱动自己组装。

SDIO 总线。 两者之间是 SDIO 总线：一根时钟线 CLK、一根命令线 CMD、四根数据线 DAT0 ~ 3，在 3B+ 上走 GPIO34 ~ 39 的 ALT3。CMD52 只用命令线，命令里带地址和一个字节，响应里带回一个字节，用来读写寄存器；CMD53 在命令线上发出地址和长度，数据走四根 DAT 线，用来搬运固件和网络帧。DAT1 还有第二个用途：总线空闲时，卡可以把它拉低来请求中断，所以 Wi-Fi 芯片不需要单独的中断引脚。

一个 SDIO 设备，三个 function。 SDIO 卡在协议上分成若干 function，各有独立的 17 位寄存器地址空间：

- **F0** 是公共寄存器区 CCCR，设置总线宽度、使能其他 function、打开中断；
- **F1** 是进入芯片内部的“窗口”。BCM43455 内部的 ChipCommon、SDIO 核、R4 和 TCM RAM 挂在一条背板总线上，地址空间远大于 17 位，所以 F1 只开一个 32 KiB 的窗口：先把目标地址的高位写进窗口寄存器，再用低 15 位访问。读 chip ID、解析 EROM、让 R4 停机、下载固件和 NVRAM、写复位向量，全都经过这个窗口；
- **F2** 是固件启动之后才就绪的 FIFO，运行时所有控制命令、事件和网络帧都从这里进出。

Arasan 是主机控制器，而且工作在 PIO 模式。 Arasan SDHCI 负责产生 SDIO 时钟、把命令编码到 CMD 线上、按 4 位总线收发数据。驱动发一条命令的方式是写 `ARG1`（参数）和 `CMDTM`（命令号与传输方式），然后等 `INTERRUPT`

里的完成位。有数据阶段时，本项目没有使用控制器的 DMA，而是 CPU 等 `READ_RDY/WRITE_RDY`，再一个 32 位字一个 32 位字地读写 `DATA` 寄存器。这和摄像头形成鲜明对比：Unicam 的数据完全不经过 CPU，而这里每个字都要 CPU 亲手搬运。这样实现简单，也没有缓存一致性问题，代价是吞吐受 CPU 和单次传输长度限制——实机曾在 512、256、128 字节的 CMD53 上出现数据 CRC 错误，驱动逐级减半，最后稳定在 64 字节。这个上限是运行时探测出来的，完成多块传输和时序调优之后应当重新评估。

中断要过三道闸门. 固件有帧要交给主机时，DAT1 上的卡中断要穿过三层开关才能变成 CPU 的 IRQ：芯片内部 SDIO 核的 `HOSTINTMASK` 决定哪些事件送到 DAT1；F0 的 CCCR 里的 `INT_ENABLE` 打开 master 位和 F1 位（`sdio_claim_irq()`）；Arasan 的 `IRPT_EN` 打开 `CARD_INT`，BCM2837 的 legacy 控制器再打开 IRQ 62。任何一层没开，接收都只能靠轮询。

固件从哪里来. 芯片每次上电都是空的，固件、NVRAM、CLM 三个文件放在 SD 卡的 FAT32 启动分区里，由另一个控制器 SDHOST（`0x3f202000`）读出。两个 SD 控制器各管一条总线：SDHOST 接外置 SD 卡，Arasan 接板载 Wi-Fi，混用会造成 GPIO 和时钟的所有权冲突。

7.7.3 SDIO 枚举

Arasan 驱动 probe 后创建 `mmc_host`，SDIO core 依次发送 CMD0（复位）、CMD5（协商 I/O 电压，等待就绪）、CMD3（取 RCA）、CMD7（选中），再用 CMD52 读 CCCR、FBR 和 CIS，切换到 4 位总线，最后把三个 function 注册到 `sdio` 总线。`brcmfmac` 驱动按 vendor `0x02d0`、device `0xa9a6/0xa9bf`、function 1 匹配。三条 SDIO 命令的分工是：CMD5 识别和上电；CMD52 单字节读写寄存器；CMD53 批量传输。

7.7.4 固件装载

FullMAC 芯片每次上电都要由主机下载固件。驱动从 FAT32 启动分区读取三个文件（所以必须排在挂起 bootfs 的 `rootdev_init()` 之后）：

- `BCM43455.BIN`：芯片 CR4 的执行代码；
- `BCM43455.TXT`：板级 NVRAM 参数（天线、校准）；
- `BCM43455.CLM`：各国信道和功率的法规数据库。

Function 1 是“背板窗口”：设置窗口寄存器后，可以用 CMD52/CMD53 访问芯片内部的 ChipCommon、SDIO core、CR4 和片内 RAM。驱动读 chip ID、解析 EROM 找到各个核和 RAM，让 CR4 停机，用 CMD53 把固件和 NVRAM 写进 RAM，写入复位向量后放开 CR4，等待固件就绪的 mailbox 中断，最后才打开 Function 2。Function 2 是运行时的数据 FIFO。

7.7.5 SDPCM 与 BCDC

运行时的每一次 F2 传输都是分层封装的（图 7.27）。

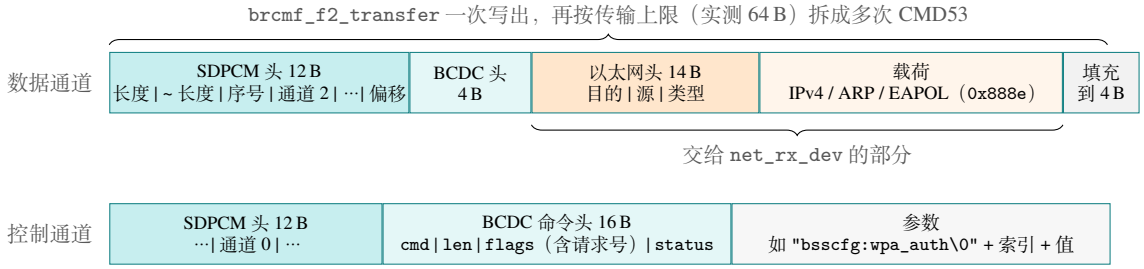


图 7.27: F2 上一帧的封装。SDPCM 是总线一层的帧格式，用通道号区分控制、事件和数据；控制通道里是 BCDC 命令（ioctl/iovar），数据通道里是 BCDC 数据头加一个完整的以太网帧

发送控制命令后，下一个到达的帧不一定就是响应——事件和数据帧可能插在前面。接收分发函数必须按通道分流，只有请求号相同的控制帧才是对应的响应。

7.7.6 WPA2 四次握手

这版固件不处理 WPA2 握手，所以主机自己完成四次握手（`kernel/drivers/net/wireless/wpa_crypto.c`）：用 PBKDF2 从口令和 SSID 算出 PMK；收到 M1 后生成 SNonce，派生 PTK，计算 MIC，发 M2；收到 M3 后验证 MIC，用 AES key unwrap 解出组密钥 GTK；把成对密钥和组密钥装进芯片，发 M4。握手完成后才放行普通数据帧，随后执行 DHCP。

7.7.7 数据流与并发

图 7.28 把一帧数据在两个方向上经过的函数排成了层。两个方向的触发方式不同：

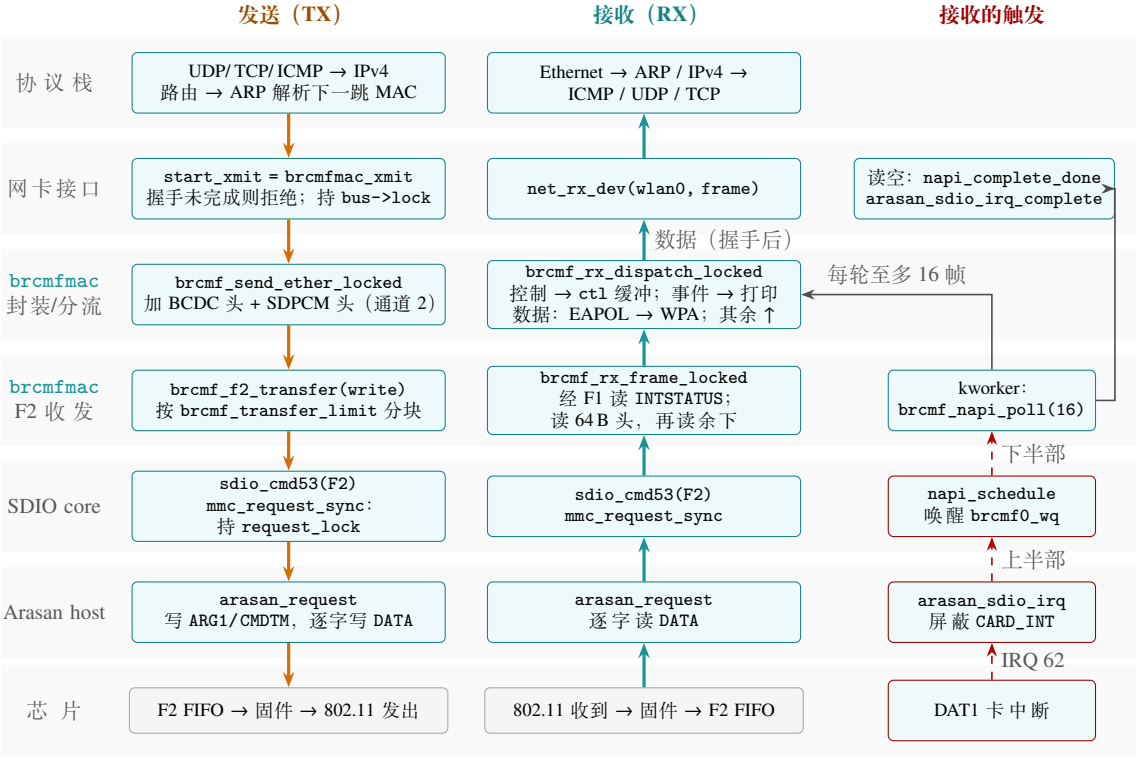


图 7.28: SDIO Wi-Fi 的数据流。发送在调用者的上下文里一路同步向下；接收由 DAT1 卡中断触发，上半部只屏蔽中断并调度 NAPI，真正的 CMD53 读取和分流在 `brcmf0_wq` 内核线程里完成，读空后才重新打开卡中断。两个方向都要经过 `mmc_request_sync()` 的 host 锁，且每个字都由 CPU 经 `DATA` 寄存器搬运

发送是同步的。 协议栈选好路由、解析出下一跳 MAC 以后，调用 `wlan0` 的发送函数 `brcmfmac_xmit()`。握手完成之前它直接拒绝普通数据。之后在调用者自己的上下文里（可能是系统调用，也可能是 TCP 重传 work）持 `bus->lock` 加上 BCDC 头和 SDPCM 头，经 `brcmf_f2_transfer()` 拆成若干次 CMD53 写进 F2，函数返回时数据已经交给了芯片。

接收是中断触发、线程执行的。 DAT1 卡中断到来时，上半部 `arasan_sdio_irq()` 只做三件事：在 Arasan 上屏蔽 `CARD_INT`、清掉状态位、调用 `napi_schedule()` 唤醒设备私有的 `brcmf0_wq` 内核线程。CMD53 要等总线、可能睡眠，绝不能在中断里做。下半部 `brcmf_napi_poll()` 在 kworker 里循环：经 F1 读 SDIO 核的 `INTSTATUS`，看到“有帧”就从 F2 读 64 字节的头、校验 SDPCM 长度和它的反码、再读剩余部分，然后按通道分流。每轮最多处理 16 帧；没读空就让 NAPI 重新排队，读空了才 `napi_complete_done()` 并重新打开 `CARD_INT`。卡中断是

电平触发的，在芯片里的帧被取走之前一直有效，所以必须先屏蔽、读空再打开，否则 CPU 会被同一个中断反复打断。

同步的控制命令。 控制面是请求/应答式的：`brcmf_dcmd()` 发出一个带递增请求号的 BCDC 命令，然后反复调用分流函数，直到读到请求号相同的控制帧。事件和数据帧可能插在应答前面，所以分流函数不能假定“下一帧就是回复”，必须按通道放到各自的去处：控制帧存进 `ctl` 缓冲，事件帧打印出来，数据帧中的 EAPOL 交给 WPA2 握手逻辑，其余在握手完成后交给 `net_rx_dev()`。

两把锁，两个粒度。 `bus->lock` 以设备为粒度，保证同一块网卡的 SDPCM 序号、收发缓冲和连接状态不会被发送路径、NAPI 和连接 watchdog 同时改动。`host->request_lock` 以控制器为粒度，保证 Arasan 那一套 `ARG1/CMDTM/INTERRUPT/DATA` 寄存器在一条 CMD52/CMD53 完成之前不被另一条覆盖。7.9.11 小节的 `CMD52 failed status=df0001` 就是缺了后一把锁造成的。两把都是睡眠锁或在进程上下文中持有，因为一次 SDIO 事务可能要等上毫秒级的超时。

从轮询到中断的演进。 这条接收路径经历过三个版本：最早 CPU0 在每次定时器中断里直接轮询 F2，完整的接收和协议处理都跑在中断上下文里；后来有了 DAT1 中断，但仍等下一次定时器才去读；现在中断直接调度 NAPI，定时器只负责推进 delayed-work 的时钟（连接 watchdog、TCP 重传）。早期每 100 ms 轮询一次时，SSH 输入一个字符要等 100 ~ 300 ms 才回显；改成中断驱动后，这个延迟基本消失。

7.8 USB 无线网卡：MT7601U

MT7601U 是一块常见的 USB 无线网卡（VID `148f`, PID `7601`）。与 BCM43455 最大的不同在于它是 *SoftMAC*：芯片只负责射频、基带和收发帧，扫描、认证、关联、加密握手等 802.11 逻辑都要由主机驱动完成（表 7.14）。

表 7.14: 两块无线网卡的对比

	BCM43430/43455	MT7601U
总线	SDIO（Arasan）	USB（DWC2 + LAN951x 集线器）
类型	FullMAC：固件完成扫描、关联	SoftMAC：主机构造 802.11 帧
固件	BIN + NVRAM + CLM，经背板写入 RAM	`MT7601U.BIN`，经 Bulk OUT 写入 MCU 的 ILM/DLM
控制	BCDC 命令	MCU 命令（EP8 发、EP5 收）和寄存器读写
数据	SDPCM 数据通道里的以太网帧	TXWI 描述符 + 802.11 帧，EP8 发、EP4 收
网卡名	`wlan0`	`wlan1`

下面按一次真机启动的日志，把 MT7601U 从“被枚举”到“`wlan1` 注册”的过程对到 `dwc2.c`、`usb.c`、`mt7601u.c` 上，依次看 USB 拓扑、探测时序、主机与芯片之间的报文格式，以及 802.11 关联和 WPA2 握手。

7.8.1 USB 拓扑与枚举

树莓派 3B 的 DWC2 只有一个根端口，板上固定接着 SMSC LAN9514 集线器，网卡插在集线器的外部端口上（图 7.29）。`dwc2_probe()` 调用 `dwc2_init()`，先枚举集线器，再逐个端口上电（`SET_PORT_FEATURE(POWER)`）、读状态（`GET_PORT_STATUS`）、复位，给端口下的设备分配地址。新设备复位后地址为 0：主机先在地址 0 上读 8 字节设备描述符拿到 EP0 最大包长，再 `SET_ADDRESS`，然后才读完整描述符。

```
dwc2: firmware USB HCD power on                 经 VideoCore 邮箱给 USB 上电
dwc2: host-channel IRQ enabled                  打开主机通道中断（之后 EP4 接收靠它）
dwc2: host hprt=100f hcfg=0 hfnum=1b1b009d      HPRT0：已连接、连接变化、已使能、使能变化、上电，高速
dwc2: root device class=9 vid=424 pid=9514 ...  根端口上是集线器 → SET_ADDRESS 1、SET_CONFIG
dwc2: hub port 1 status=101 change=1            0x0101：已连接 + 上电；change 1：连接变化
```

```
dwc2: hub port 1 after-reset status=503 ...      0x0503：再加上已使能、高速（bit10）
dwc2: hub port=1 child class=255 vid=424 pid=ec00    板载以太网，addr 2，跳过
dwc2: hub port 2 status=100 change=0             只有上电位：空端口（轮询 50×10 ms 后放弃）
dwc2: hub port 3 status=100 change=0             同上
dwc2: hub port 4 status=101 change=1             网卡插在端口 4
dwc2: hub port=4 child class=0 vid=148f pid=7601     is_mt7601() 命中，addr 3，不再看其余端口
dwc2: MT7601U configured; bulk-in=4,5/512 bulk-out=8/512
dwc2: MT7601U handed to usb bus                  usb_device_register(&usb_child)
```

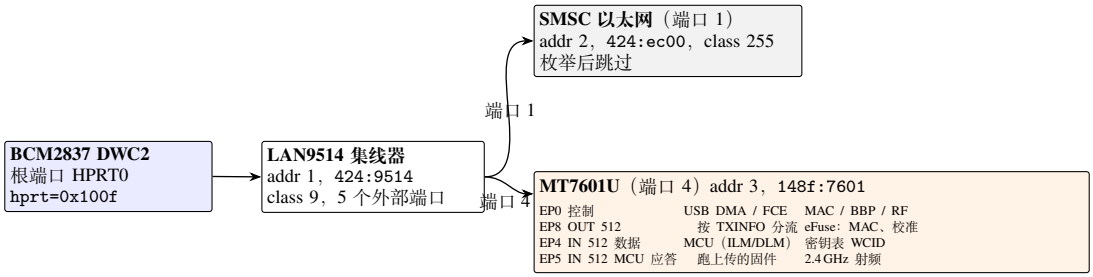


图 7.29: MT7601U 所在的 USB 拓扑与芯片内部结构

复位后的端口状态 0x0503 里 bit10 表示高速（480 Mbit/s），正因为如此，后面 bulk 端点的最大包长才是 512。

7.8.2 设备、配置、接口与端点描述符

USB 设备用一组描述符介绍自己。打开 USB_ENUM_TRACE 后，dwc2_trace_device_descriptor() 和 dwc2_trace_config_descriptor() 把它们逐项打印出来：

```
usb-enum: device addr=0 usb=201 class=0 subclass=0 protocol=0 mps0=64 vid=148f pid=7601 configs=1
usb-enum: config addr=3 value=1 total=74 interfaces=1 attributes=80 maxpower=160 mA
usb-enum:  interface=0 alt=0 eps=8 class=255 subclass=255 protocol=255
usb-enum:   endpoint=84 IN type=2 mps=512 interval=0
usb-enum:   endpoint=85 IN type=2 mps=512 interval=0
usb-enum:   endpoint=8 OUT type=2 mps=512 interval=0
usb-enum:   endpoint=4 OUT type=2 mps=512 interval=0     （5、6、7、9 同）
```

设备描述符 （18 字节，类型 1）. bcdUSB=0x0201；设备类为 0，表示类别由接口描述符给出；bMaxPacketSize0=64 是 EP0 的最大包长；idVendor/idProduct=148f/7601，is_mt7601() 和驱动的 id_table 都按这一对匹配；只有一种配置。

配置描述符 （9 字节，类型 2）. wTotalLength=74 是配置描述符连同其后所有接口、端点描述符的总长，bConfigurationValue=1 是 SET_CONFIGURATION 用的编号，bmAttributes=0x80 表示总线供电、不支持远程唤醒，bMaxPower=80（单位 2 mA，即 160 mA）。dwc2_init() 分两次读：先读 9 字节拿到总长，再一次读满 74 字节。$74 = 9 + 9 + 8 \times 7$，正好是一个配置、一个接口、八个端点。

接口描述符 （9 字节，类型 4）. 唯一的接口 0、备用设置 0、8 个端点（不含 EP0）。类别、子类、协议都是 0xFF，即厂商自定义：没有通用的类驱动可用，只能按 VID/PID 绑定专用驱动；HID 键盘的接口类是 3，就可以按类别匹配（7.1 节）。

端点描述符 （7 字节，类型 5）. `bEndpointAddress` 的 bit7 是方向（1 为 IN），低 4 位是端点号；`bmAttributes` 的低 2 位是传输类型（0 控制、1 等时、2 bulk、3 中断）；高速 bulk 端点的 `wMaxPacketSize` 必须是 512，`bInterval` 对 bulk 无意义。表 7.15 是八个端点的用途。

表 7.15: MT7601U 的端点与 DWC2 主机通道

地址	方向/类型	本驱动的用途	主机通道
0x00	控制	寄存器读写、固件 DMA 配置（厂商请求）	ch0，同步
0x84	IN bulk 512	EP4：802.11 帧（多帧聚合）	ch4 异步，4 个 16 KiB 缓冲；初始化期间 ch3 同步清空
0x85	IN bulk 512	EP5：MCU 命令应答	ch4 异步，命令发出前预挂
0x08	OUT bulk 512	EP8：固件、MCU 命令、全部 802.11 帧	ch5，同步
0x04–0x07, 0x09	OUT bulk 512	未用（Linux 用作 WMM 队列和管理队列）	—

`dwc2_init()` 的选取规则是：解析循环跳过非端点、非 bulk 的描述符，第一个 bulk IN 记为 `bulk_in_ep`，第二个记为 `bulk_in_ep2`，第一个 bulk OUT 记为 `bulk_out_ep`，三者缺一就放弃；然后 `SET_CONFIGURATION(1)`，打印 `bulk-in=4,5/512 bulk-out=8/512`。DWC2 没有通用的 URB 调度，端点固定绑在主机通道上：ch0 做控制传输，ch3 做同步 bulk IN，ch4 做异步 bulk IN（5 个槽：EP4 四个、EP5 一个，轮流调度），ch5 做 bulk OUT。发往芯片的所有东西都走 EP8，靠包头的 TXINFO 区分去向。

7.8.3 探测时序：从枚举到 `wlan1`

设备在 `dwc2_probe()` 里就登记到 usb 总线上了，这时 mt7601u 驱动还没有登记。`main()` 故意把 `mt7601u_driver_init()` 放在 `rootdev_init()` 挂好 FAT32 之后，因为 probe 要从 `/boot` 读固件。`driver_register()` 会回头遍历已登记的设备，按 `id_table` 匹配到 `148f:7601`，再调用 `mt7601u_probe()`。先登记设备、后登记驱动也能正确绑定，这正是驱动核心的作用。

图 7.30 用四条泳道画出整个过程：主机控制器驱动 `dwc2.c`、驱动核心的 usb 总线（`usb.c`/`device.c`）、网卡驱动 `mt7601u.c`、芯片本身。它把“控制器先枚举设备、根文件系统就绪后驱动才绑定、probe 再下载固件”这三个容易混淆的时间点分开。USB core 中的设备对象可以先于驱动存在；这是为什么必须等 `/boot` 可读后才登记 `mt7601u` 驱动，而不必重新枚举硬件。

对应的函数调用链如下：

```
main()
  usb_bus_init()                        登记 usb 总线
  dwc2_driver_init() -> dwc2_probe()    [1] 枚举：集线器 addr 1，以太网 addr 2，网卡 addr 3
    usb_device_register(&usb_child)     [2] 设备进总线；驱动表里还没有 mt7601u，暂不绑定
  ...
  rootdev_init()                        挂上 FAT32 启动分区
  mt7601u_driver_init()                 [3] driver_register -> usb_match 命中 -> usb_probe
    mt7601u_probe(udev)
      read32(0x0000), read32(0x1000)    [4] ASIC 0x76010001，高 16 位必须是 0x7601
      mt7601u_validate_firmware()         /boot/MT7601U.BIN：32 B 头 + ILM 45380（含 64 B IVB）+ DLM 0
      mt7601u_load_firmware()             DEV_MODE=1 复位 MCU，配 FCE/PDMA
        4 次 bulk OUT：ILM 14336×3 + 2308   每块写 FCE_DMA_ADDR/LEN，等 COM_REG1 bit31
        DEV_MODE=0x12 + 64 B IVB          启动 MCU，轮询 COM_REG0 == 1
      mt7601u_read_eeprom()             [5] eFuse 每次 16 B，共 256 B：MAC、频偏、RSSI、LNA
      mt7601u_basic_hw_init()             开 WLAN 时钟等 PLL，复位 MAC，开 USB DMA，写 MAC 地址
      mt7601u_mcu_channel_init()          MCU 命令 FUN_SET(Q_SELECT=1)，此后应答走 EP5
      mt7601u_full_hw_init()            [6] MAC 表 44 项，BBP 表 59 项
                                          WCID 地址表、共享密钥模式、WCID 属性表清零
```

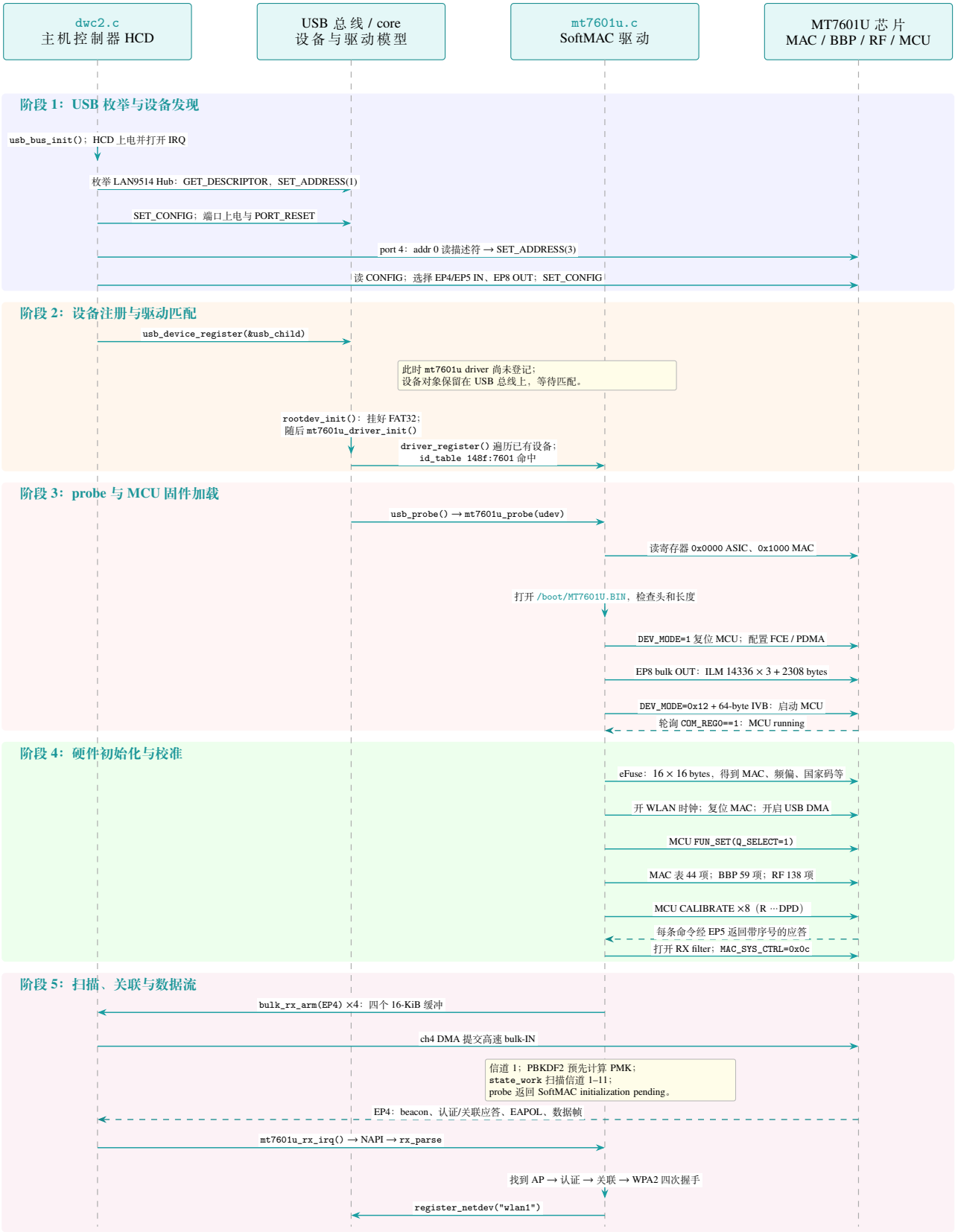


图 7.30: MT7601U 从 USB 枚举、延迟驱动匹配、MCU 固件下载、射频校准到 SoftMAC 关联和数据接收的完整时序。设备先注册在 USB core，待 FAT32 上的固件可读后才绑定驱动；运行时 EP4 承载接收，EP5 返回 MCU 应答，EP8 承载固件、MCU 命令和发送帧

```
                                    RF 表 138 项
                                    calibrate(R) → VCO → calibrate(TXDCOC) → rxdc_cal()
                                    → set_bw_filter(1) → calibrate(LOFT, TXIQ, RXIQ, DPD)
                                    → rxdc_cal()；开接收过滤，MAC_SYS_CTRL = 0x0c
    mt7601u_set_channel(1)        [7] RANDOM_WRITE 写 RF R17–R20，VCO，BW 滤波
    wpa_pbkdf2()                    PMK 预先算出
    bulk_rx_arm(EP4) ×4             挂 4 个 16 KiB 接收缓冲
    queue_delayed_work(state_work)  扫描交给后台状态机，probe 返回
EP4 完成中断 -> mt7601u_rx_irq()   [8] napi -> rx_parse -> rx_mgmt / rx_eapol
state_work                          扫描 1→11 -> 认证 -> 关联 -> WPA2 -> register_netdev(wlan1)
```

固件文件 45412 字节，正好是 32 字节头加 `ilm_len` 45380，`dlm_len` 为 0，所以日志里只有 ILM 的上传进度。PBKDF2 要做 4096 轮 HMAC-SHA1，放在 probe 里预先算，而不是等收到 M1 时在中断路径里算。

校准和换台都靠需要应答的 MCU 命令，它们走 `mt7601u_mcu_command()`：

```
mt7601u_mcu_command(cmd, payload, wait=1)
  seq = ++mcu_seq & 0xf
  bulk_rx_arm(EP5, mt_mcu_resp)          先挂好应答缓冲（Linux 的应答 URB 也一直挂着）
  dwc2_usb_bulk(EP8, TXINFO|载荷|0)       发命令
  最多等 1500 ms：
    bulk_rx_complete(EP5)                中断未开时直接读 HCINT(4) 判断完成
    未完成 → mt7601u_drain_data_ep()      EP4 接收环还没挂时，同步读空 EP4 并丢弃
    完成   → 核对序号与事件类型
```

清空 EP4 是这里的关键。芯片的接收 DMA 按顺序吐数据，EP4 上没人读的帧会把 EP5 的应答堵在后面；Linux 在校准之前就挂好了 EP4 的接收 URB，所以不会遇到。xv6 的 probe 在 `main()` 里同步执行，中断和调度器都还没启动，只能边等应答边清 EP4。日志里的

```
mt7601u: MCU cmd 12 seq 9 answered after 2 ms; drained 1684 bytes from EP4
```

就是初始换台时的现场：MAC 已经在收信标，读走积压的 1684 字节后，应答 2 ms 就到了。

7.8.4 主机与芯片之间的四种报文

MT7601U 带一个 MCU，主机要先上传固件，之后和芯片的交互分两条通道。

寄存器读写：EP0 厂商请求。 读一个 32 位寄存器是一次 IN 控制传输；写则拆成两次 16 位的 OUT 控制传输，第二次地址加 2：

```
读  bmRequestType=0xC0  bRequest=7 (MULTI_READ)  wValue=0       wIndex=reg    wLength=4  -> 小端 4 B
写  bmRequestType=0x40  bRequest=2 (WRITE)       wValue=v&0xffff wIndex=reg    wLength=0
    bmRequestType=0x40  bRequest=2 (WRITE)       wValue=v>>16    wIndex=reg+2  wLength=0
     (FCE 寄存器用 bRequest=0x42；bRequest=1 (DEV_MODE) 复位或启动 MCU)
```

BBP（数字基带）和 RF（模拟射频）寄存器不能直接访问，要把“寄存器号 + 值 + BUSY/KICK 位”写进 `BBP_CSR_CFG`（0x101c）或 `RF_CSR_CFG`（0x0500），再轮询忙位清零。写一个 RF 寄存器就要几次控制传输，RF 表 138 项要走几百次。运行中的 MCU 固件也会访问 `RF_CSR_CFG`，所以换信道时的频率表改由 MCU 命令 `RANDOM_WRITE`（12 号）代写，地址加 0x80000000 表示 RF，和 Linux 一样。

bulk 包：同一个 EP8，靠 TXINFO 分流。 每个 bulk OUT 包以 4 字节 TXINFO 开头、以 4 字节 0 结尾，中间按 4 字节对齐（图 7.31）：

- 固件块：TXINFO 的端口字段为 2（CPU_TX），后接最多 14336 字节固件。发送前写 `FCE_DMA_ADDR` 和 `FCE_DMA_LEN` 给出目标地址，发送后把 `CPU_DESC_IDX` 加一，等 `COM_REG1` 的 bit31；
- MCU 命令：TXINFO 还带命令标志（bit30）、命令号和 4 位序号。`FUN_SET`（1）不等应答；`RANDOM_WRITE`（12）和 `CALIBRATE`（31，参数 id 为 1 R、4 LOFT、5 TXIQ、6 BW、7 DPD、8 RXIQ、9 TXDCOC）要在 EP5 上等序号相同的应答；
- 802.11 帧：TXINFO 置 802.11 位（bit19），后接 20 字节 TXWI（速率、是否要 ACK、WCID、MPDU 长度）和完整的 802.11 帧。管理帧和 EAPOL 帧置 WIV 位，明文发送；握手完成后数据帧帧头置 Protected、清 WIV、用 WCID 1，由硬件按该 WCID 的 IV 表生成 CCMP 头并加密。

接收：EP4 上的聚合包。 一次 bulk IN 可能带回多帧，每段是 4 字节长度、28 字节 RXWI（含 CRC/ICV/MIC 错误位和 MPDU 长度）、MPDU 和 4 字节尾。`mt7601u_rx_parse()` 逐段拆开，管理帧交给 `rx_mgmt()`，数据帧交给 `rx_data()`。EP5 只用来收 MCU 应答，前 4 字节的 bit16–19 是序号，bit20–23 是事件类型。

硬件解密后的数据帧要多看一眼 RXWI。下面是握手后收到的一个组播帧：

```
rx data fc=4208 rxinfo=1520a2 len=76 hdr=32 llc=aa aa 3 0 0 0 8 0
```

`fc=0x4208` 是 From-DS、Protected 的数据帧；`rxinfo` 里 bit5 组播、bit7 本 BSS、bit16 已解密，PN_LEN（bit19–21）为 2——这时硬件保留了 8 字节 CCMP 头，所以 LLC 头从第 24+8 = 32 字节开始，`aa aa 03 00 00 00 08 00` 是 LLC/SNAP 加上 IPv4 的类型号。`rx_data()` 把它改写成以太网帧（目的地址取 addr1，源地址取 addr3），放进 8 帧的环形队列；NAPI 上下文在锁内把帧拷出、释放锁后再 `net_rx_dev()` 交给协议栈，做法同 `brcmf_napi_poll()`。

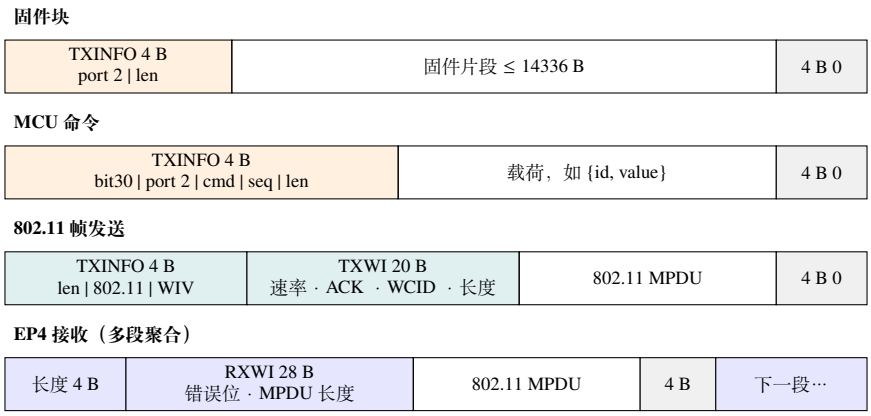


图 7.31: EP8 上的三种 bulk OUT 包与 EP4 上的接收包

EP4 由 DWC2 的主机通道中断驱动：probe 末尾预先挂 4 个 16 KiB 缓冲，每完成一个，中断里调用 `mt7601u_rx_irq()` 调度 NAPI，解析完立即重新挂上，这样总有缓冲在等数据。缓冲区由 DWC2 用 DMA 写入，总线地址要加 `0xC0000000` 别名（第 4 章）。

7.8.5 启动日志分析：USB 总线上发生了什么

7.8.3 节给出了函数调用链，上一节给出了报文格式，这一节换个角度：把 `mt7601u_probe()` 打印的每一行，翻译成它在 USB 上实际发出的传输。所有传输都发往设备地址 3，区别只在端点。

两种传输。 控制传输走 EP0，分三个阶段：SETUP（8 字节：`bmRequestType`、`bRequest`、`wValue`、`wIndex`、`wLength`）、可选的 DATA、方向相反的零长度 STATUS。MT7601U 的寄存器都用厂商请求访问（表 7.16），读一个寄存器是 1 次控制传输，写一个是 2 次。bulk 传输走 EP8 OUT 和 EP4/EP5 IN，每包最多 512 字节，DWC2 的 DMA 自动切包，PID 在 DATA0/DATA1 间交替以发现丢包；IN 方向主机不停发 IN 令牌，设备没有数据就回

NAK，收满或收到短包（不足 512 字节）时传输结束。EP8 上每个包都是“4 字节 TXINFO + 内容 + 4 字节 0”，TXINFO 告诉芯片这是固件、MCU 命令还是 802.11 帧。

表 7.16: MT7601U 的 EP0 厂商请求

操作	bmRequestType	bRequest	wValue	wIndex
读 32 位寄存器（DATA IN 4 字节）	0xC0	7 MULTI_READ	0	地址
写 32 位寄存器（分两次）	0x40	2 WRITE	低 / 高 16 位	地址 / 地址 +2
写 FCE（固件 DMA）寄存器	0x40	0x42	值	地址
复位 / 启动 MCU（可带数据）	0x40	1 DEV_MODE	模式	0

(1) 芯片身份。 `USB device 148f:7601 ASIC=76010001 MAC=76010500` 是两次 EP0 控制读：

```
SETUP C0 07 0000 0000 0400 -> IN 01 00 01 76 -> STATUS   0x0000 ASIC_VERSION = 0x76010001
SETUP C0 07 0000 0010 0400 -> IN 00 05 01 76 -> STATUS   0x1000 MAC_CSR0    = 0x76010500
```

VID/PID 是 dwc2 从设备描述符读来的；这里再读芯片自己的版本寄存器，确认高 16 位是 0x7601。

(2)(3) 不走 USB。 `driver WPA2-v4` 只是打印；`firmware ... validated` 从 SD 卡的 FAT32 读 `/boot/MT7601U.BIN` 并检查 32 字节头：ILM 45380 字节（前 64 字节是 IVB，即中断向量），DLM 为 0，32+45380 = 45412。

(4) 上传固件。 `ILM firmware upload ...` 把代码写进 MCU 的指令内存，分 4 块（$14336 \times 3 + 2308$）。每块是一组传输：

```
EP0  40 42 a840 0230 ... 0236   FCE_DMA_ADDR / FCE_DMA_LEN: 目标地址和长度
EP8  bulk OUT 2316 B:  04 09 00 10 | 2308 B 固件 | 00 00 00 00
EP0  读 0x09a8 = 3, 写 0x09a8 = 4   CPU_DESC_IDX 加一：新的一块到了
EP0  读 0x0734 = 0xc9040000        COM_REG1 bit31 = 1：这块收完了
```

TXINFO `04 09 00 10` 即小端 0x10000904：低 14 位 0x904 = 2308 是长度，bit27–29 = 2 表示发往 CPU_TX 端口（交给 FCE/DMA，而不是射频）。14336 字节的一块加上首尾 8 字节共 14344 字节，DWC2 把它切成 28 个 512 字节的包和一个 8 字节短包。

(5) 启动 MCU。 `MCU firmware running` 前是一次带数据阶段的控制写和一次轮询：

```
EP0  SETUP 40 01 0012 0000 4000 + DATA OUT 64 B (IVB)   DEV_MODE 0x12：带中断向量启动 MCU
EP0  读 0x0730 -> 1                                     COM_REG0 == 1：固件在跑
```

(6) eFuse。 `EEPROM ... permanent mac=...`：每 16 字节一块，读 EFUSE_CTRL（0x24），写入块地址和 KICK，轮询 KICK 清零，再读 4 个数据寄存器（0x28–0x34）。一块约 8–10 次控制传输，读满 256 字节约 150 次。偏移 4 是 MAC，`freq=96` 是晶振频偏，`lna` 是接收增益，`temp` 是参考温度。

(7) 基础初始化。 `MAC/BBP/RF bootstrap ready` 仍全走 EP0：开 WLAN 时钟并轮询 CMB_CTRL 等 PLL；MAC_SYS_CTRL 先写 3 复位再写 0；写 USB_DMA_CFG（0x238）打开 bulk 收发 DMA 和接收聚合——此后芯片才会经 EP4 送帧；MAC 地址写入 0x1008/0x100c；频偏写入 RF R12。

(8) 第一条 MCU 命令。 `MCU command channel ready ... async-prearmed` 是 FUN_SET(Q_SELECT=1)：

```
EP8  bulk OUT 16 B:  08 00 10 50 | 01 00 00 00 01 00 00 00 | 00 00 00 00
                     TXINFO 0x50100008: bit30 命令、端口 2、cmd 1、seq 0、长度 8
```

意思是“以后命令的应答从 EP5 送出”，本身不要应答。`async-prearmed` 指之后每条要应答的命令，发出前先在 EP5 上挂好接收缓冲（DWC2 ch4）。

(9) 寄存器表。 `MAC/BBP/RF register table initialized` 全是 EP0 控制写（表 7.17）。BBP 和 RF 寄存器不能直接寻址，要把“寄存器号 + 值 + 忙位”写进间接端口 0x101c 或 0x0500，再轮询忙位清零，芯片内部才把值送到基带或射频。这是探测过程中 USB 往返最多的一段。

表 7.17: 寄存器表阶段的控制传输次数

表	每项需要的控制传输	合计
MAC 44 项	写 2 次	≈ 90
BBP 59 项	读忙位、写 2 次、再读忙位	≈ 240
WCID 地址、密钥模式、WCID 属性	516 个寄存器，各写 2 次	≈ 1000
RF 138 项	读 KICK、写 2 次、再读 KICK	≈ 550

(10) 校准。 `RF calibration complete` 之前，主机发了 8 条 CALIBRATE（31 号）命令——R、TXDCOC、BW×2、LOFT、TXIQ、RXIQ、DPD——并逐条等应答；中间穿插经 EP0 读写 BBP 158/159 的 RX DC 校准。以 R 校准为例：

```
EP8  bulk OUT 16 B:  08 00 f1 51 | 01 00 00 00 00 00 00 00 | 00 00 00 00
                     0x51f10008: 命令、cmd 31、seq 1、长度 8; 载荷 {id = 1 (R), value = 0}
EP5  bulk IN  16 B:  08 00 01 4a | ...
                     0x4a010008: bit31-30 = 01 命令应答、seq 1、事件 0 = 完成
```

主机靠 seq 把应答和命令配对。最后写 RX 过滤 0x1400 和 `MAC_SYS_CTRL` = 0x0c 打开收发，从这一刻起空中的信标开始在接收 DMA 里排队，等着从 EP4 送出。

(11) 初始换台。 下面这行日志来自 `mt7601u_set_channel(1)` 的 RANDOM_WRITE（12 号）：

```
mt7601u: MCU cmd 12 seq 9 answered after 2 ms; drained 1684 bytes from EP4
```

它在总线上是：

```
EP8  bulk OUT 40 B:  20 00 c9 50 | 11 00 00 80 99 00 00 00 | 12 00 00 80 ... | 00 00 00 00
                     0x50c90020: 命令、cmd 12、seq 9、长度 32 = 4 对 {地址, 值}
                     0x80000011 = RF bank 0 的 R17, 值 0x99 ...
```

seq 是 9，因为前 8 条校准用掉了 1–8。MAC 已经在收，EP4 里排着 1684 字节信标；接收 DMA 按顺序吐数据，不读走它们，EP5 的应答就出不来。驱动等应答时用 ch3 同步读空 EP4（此时还没挂接收缓冲，读出的帧丢弃），应答 2 ms 就到了。

(12) 挂上接收。 `EP4 receive uses DWC2 IRQ with 4 buffers` 是 4 次 `bulk_rx_arm(EP4, 16 KiB)`：在 ch4 上填 `HCDMA`（总线地址）、`HCTSIZ`（16384 字节 = 32 包，DATA0/1）、`HCCHAR`（地址 3、EP4、IN、bulk、512），打开中断。之后 DWC2 自动反复发 IN 令牌：网卡没数据回 NAK，有数据就送一个以短包结尾的聚合包，触发 ch4 中断，经 `dwc2_irq()`、`mt7601u_rx_irq()`、NAPI 到 `mt7601u_rx_parse()` 拆帧，随即重挂。4 个缓冲轮转，EP4 总有人在收，(11) 那种拥堵不再出现。

(13) probe 返回。 `transport and MCU ready; SoftMAC initialization pending` 之后不再有同步 USB 通信；扫描、认证、关联和握手由 `state_work` 推进：发帧走 EP8（802.11 TXINFO + 20 字节 TXWI），收帧靠 EP4 中断，换信道再发 RANDOM_WRITE/CALIBRATE 并等 EP5。

表 7.18 汇总了探测期间各端点的用量。

表 7.18: 探测期间各端点的用量

端点	做了什么	次数
EP0 控制	读版本、FCE、启动 MCU、eFuse、寄存器表、BBP/RF 间接访问、RX DC 校准	两千次以上
EP8 bulk OUT	4 块固件、FUN_SET、8 条校准、换台 RANDOM_WRITE	十几次
EP5 bulk IN	每条要应答的命令一次 16 字节应答	9 次（seq 1–9）
EP4 bulk IN	打开 MAC 接收后信标排队；先同步读空，再交给 4 个异步缓冲	持续

7.8.6 空口：扫描、认证、关联、WPA2 四次握手

MT7601U 是 SoftMAC 网卡：芯片只负责收发帧和加解密，扫描、认证、关联和 WPA2 状态机都在 `mt7601u.c` 里，由两样东西推进：EP4 接收中断送来的帧，以及按截止时间重排的延迟工作 `state_work`。图 7.32 是这次启动的交换过程。

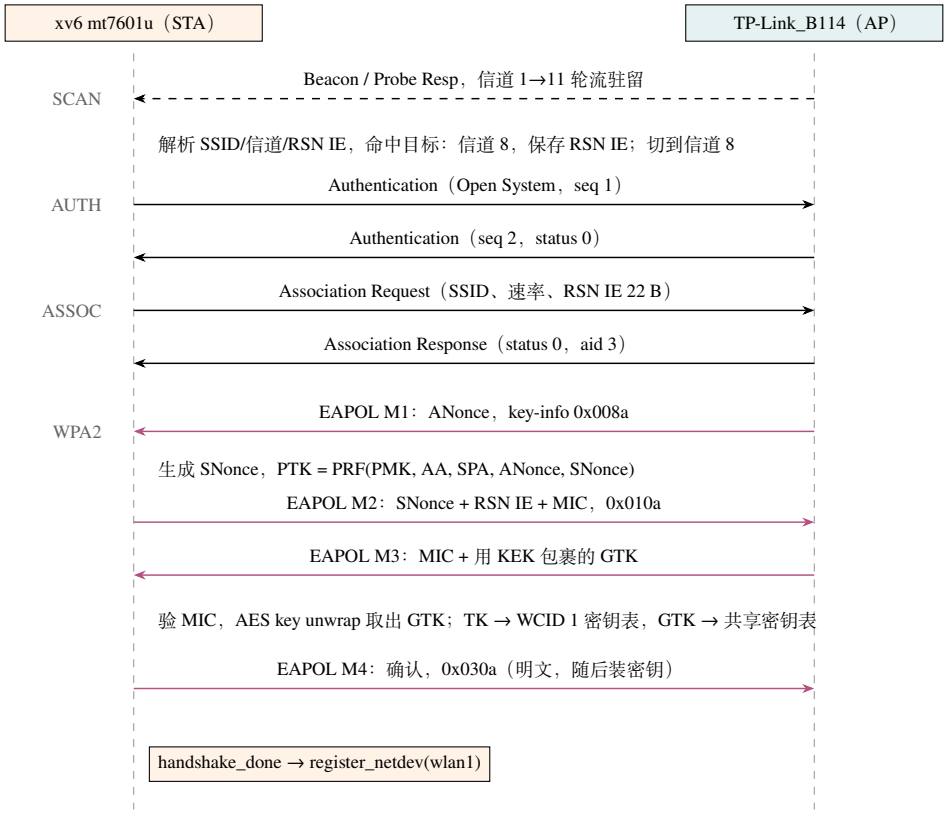


图 7.32: MT7601U 的扫描、认证、关联与 WPA2 四次握手

下面按报文逐一看这次交换。

一帧怎样从主机到空中。 主机发出的每个 802.11 帧都是 EP8 上的一个 bulk 包，层层包装如下：

```
EP8 bulk OUT:  TXINFO 4 B | TXWI 20 B | MPDU (802.11 头 24 B + 帧体) | 补齐到 4 B | 00 00 00 00
TXINFO  = 长度 | bit19 802.11 包 | bit24 WIV (置位 = 硬件不加 IV、不加密)
TXWI    = rate_ctl 0 (CCK 1 Mbit/s) · ack_ctl 1 (要 ACK) · WCID · len_ctl = MPDU 长度
```

FCS（4 字节 CRC）由芯片在发射时附加；对方回的 ACK 和必要的重传也由芯片硬件完成，主机看不到。管理帧用 WCID 0xff、置 WIV；关联后的数据帧（包括 EAPOL）用 WCID 1。接收方向相反：EP4 上每段是 4 字节长度 + 28 字节 RXWI + MPDU（FCS 已去掉）+ 4 字节尾。

本机地址 SA = `90:de:80:14:95:94`，AP 的 BSSID = `f0:09:0d:cb:b1:14`。表 7.19 列出握手前的全部报文（与日志的对应：Beacon 对应 `AP ssid=...`，两个 Authentication 对应 `authentication request sent` 和 `... accepted`，关联请求和应答对应 `rsn=22 bytes` 和 `associated aid=3`，M1、M2、M4 各有一行日志，M3 没有），帧控制字段按空中字节顺序写（第 1 字节含类型和子类型，第 2 字节含 ToDS/FromDS/Protected 等标志）。

表 7.19: MT7601U 与 AP 之间的报文

报文	方向	帧控制	帧体要点
Beacon	AP→	80 00	时间戳 8 B、间隔、能力；IE：SSID(0)、速率 (1)、DS 参数 (3) = 信道 8、RSN(48) 22 B
Authentication	→AP	b0 00	算法 00 00（Open System）、序号 01 00、状态 00 00；MPDU 30 B
Authentication	AP→	b0 00	算法 0、序号 2、状态 0 = 成功
Association Req	→AP	00 00	能力、监听间隔 0a 00；SSID IE `00 0c "TP-Link_B114"`；速率 IE `01 08 82 84 8b 96 0c 12 18 24`；AP 的 RSN IE 原样附上；MPDU 74 B
Association Resp	AP→	10 00	能力、状态 00 00、AID `03 c0`（高两位恒为 1，AID = 3）
EAPOL M1	AP→	08 02	数据帧、FromDS、明文；LLC `aa aa 03 00 00 00 88 8e`；EAPOL `02 03 00 5f`；key-info 0x008a；ANonce
EAPOL M2	→AP	08 01	ToDS、明文（WIV=1、WCID 1）；key-info 0x010a；SNonce；MIC；key data = RSN IE（帧体 95+22 B）
EAPOL M3	AP→	08 02	key-info 含 Install/ACK/MIC/Secure/加密数据（典型 0x13ca）；key data 用 KEK 包裹：RSN IE + GTK KDE
EAPOL M4	→AP	08 01	key-info 0x030a；回显 M3 的重放计数器；无 nonce、无 key data；明文

扫描. `rx_mgmt()` 只看子类型 8（beacon）和 5（probe response），从固定字段之后逐个解析 IE：0 号是 SSID，3 号（DS Parameter Set）给出 AP 实际所在的信道，48 号是 RSN。这次 AP 的 RSN IE 是 22 字节：

```
30 14 | 01 00 | 00 0f ac 04 | 01 00 00 0f ac 04 | 01 00 00 0f ac 02 | cc cc
  长度    版本      组密码 CCMP      1 个成对密码 CCMP     1 个 AKM: PSK       RSN 能力
```

日志 `keys installed ... group-cipher=4` 打印的就是这里组密码的类型字节 4（CCMP）。RSN IE 要原样放进关联请求和 M2，AP 会逐字节比对，防止降级攻击。

认证与关联. Open System 认证只是交换两个空的认证帧：本机发序号 1，AP 回序号 2、状态 0。关联请求带上能力字段、监听间隔 10、SSID、支持的速率（`0x82 0x84 0x8b 0x96` 是最高位置 1 的基本速率 1/2/5.5/11 Mbit/s，后四个是 6/9/12/18 Mbit/s 的 OFDM 速率）和 RSN IE。AP 回的 AID 是 3，日志 `associated aid=3`。三个地址字段都是 A1 = 接收者、A2 = 发送者、A3 = BSSID；STA 和 AP 之间的管理帧中 A1、A3 都是 AP。

EAPOL-Key 帧. 四次握手的四个报文都是数据帧，帧体是 LLC/SNAP（以太类型 0x888e）加 EAPOL。EAPOL 头 4 字节（版本、类型 3 = Key、长度），其后是 95 字节固定部分和可变的 key data：

```
偏移  0   描述符类型 2 (RSN)      1-2  key-info            3-4   密钥长度 (16)
      5   重放计数器 8 B           13   nonce 32 B
     45   EAPOL-Key IV 16 B        61   RSC 8 B             69    保留 8 B
     77   MIC 16 B                 93   key data 长度        95    key data
```

key-info 的位：bit0–2 描述符版本（2 = HMAC-SHA1 + AES 包裹），bit3 成对密钥，bit6 Install，bit7 ACK（需要对方应答），bit8 MIC 有效，bit9 Secure，bit12 key data 已加密。于是 M1 的 0x008a = 版本 2 + 成对 + ACK，M2 的 0x010a = 版本 2 + 成对 + MIC，M4 的 0x030a 再加上 Secure。`rx_eapol()` 用 “ACK 置位且 MIC 未置位” 认出 M1，用 0x0188（成对 + ACK + MIC）认出 M3。重放计数器由 AP 递增：M1 为 1，M2 原样回显（日志 `replay=0001`），M3 为 2，M4 回显 M3 的值。

密钥推导. 握手不在空中传送任何密钥，双方各自从 PMK 推导：

```
PMK = PBKDF2-HMAC-SHA1(口令, SSID, 4096 轮, 32 B)                  probe 时预先算好
PTK = PRF-512(PMK, "Pairwise key expansion",
```

```
              min(AA,SPA) || max(AA,SPA) || min(ANonce,SNonce) || max(ANonce,SNonce))
    = KCK[0..15] | KEK[16..31] | TK[32..47] | ...                    wpa_derive_ptk()
MIC = HMAC-SHA1(KCK, MIC 字段清零后的整个 EAPOL 帧) 的前 16 B          wpa_eapol_mic()
GTK = AES-Key-Unwrap(KEK, M3 的 key data) 中的 GTK KDE                wpa_aes_unwrap()
```

收到 M1 时本机已经有 ANonce 和 SNonce，可以算出 PTK，于是 M2 能带上 MIC；AP 收到 M2 后用同样的公式得到同一个 PTK，验 MIC 通过就证明双方口令一致。M3 的 key data 解包后是 AP 的 RSN IE 加 GTK KDE：`dd 16 00 0f ac 01 | 01 00 | GTK 16 B`，其中 01 的低 2 位是 GTK 序号 1。

装密钥. M4 用明文发出之后，`mt7601u_install_ccmp_keys()` 把密钥写进芯片的密钥存储器（全是 EP0 控制写）：

```
0x8000 + 32×1 = 0x8020   TK 16 B                    WCID 1 (AP) 的成对密钥
0xa000 +  8×1 = 0xa008   01 00 00 20 00 00 00 00     IV 初值：PN = 1, ExtIV 位, key id 0
0xa800 +  4×1 = 0xa804   (4 << 1) | 1 = 9           属性：CCMP、成对
0xac00 + 32×1 = 0xac20   GTK 16 B                    BSS 0 的共享密钥槽 1
0xb000                   4 << (4×1) = 0x40           槽 1 的密码类型：CCMP
```

日志里的 `attr=11009 skey-mode=11049` 来自先读后改的旧代码：读回的值混入了上一次写入的内容，现在两处都改为直接写入 0x9 和 0x40。

加密数据帧. 装好密钥后，`mt7601u_net_xmit()` 发出的数据帧帧控制为 `08 41`（ToDS + Protected），TXINFO 的 WIV 清零：芯片按 WCID 1 的 IV 表在 802.11 头后插入 8 字节 CCMP 头（PN 自动递增），用 TK 加密帧体并追加 8 字节 MIC。接收方向，AP 发来的单播帧（`08 42`）用 PTK 解密，组播和广播帧用槽 1 的 GTK 解密；RXWI 的 DECRYPT 位表示已解密，PN_LEN 非零时 CCMP 头仍留在帧里，所以 `rx_data()` 从第 32 字节开始找 LLC 头（7.8.4 节）。

几个时序细节：

- 扫描在信道 1 ~ 11 间轮换，每个信道驻留 50 个 tick；换信道要经 `RANDOM_WRITE` 改 RF bank 0 的 R17–R20，再做 VCO 校准和 BW 滤波校准。换台在 `dev->lock` 外做，失败后退避，连续 8 次失败就停在当前信道；
- 在相邻信道也能听到 AP 的信标，所以命中目标后先切到 DS 参数 IE 给出的信道再认证；
- 认证、关联各自最多重试 3 次，响应超时 100 tick；关联后启动 500 tick 的看门狗，收不到 M1 就重新关联，每收到一个结构正确的 EAPOL-Key 帧都会把看门狗往后推；
- 先发 M4、后装密钥：IEEE 802.11 规定认证者收到 M4 后才装密钥，加密的 M4 它解不开，只会一再重发 M3；
- 收到 AP 发来的解除认证或解除关联帧（例如空闲后的 `reason=6`），清掉握手状态，回到认证阶段自动重连。

握手完成后，`mt7601u_service()` 调用 `register_netdev()` 注册 `wlan1`。网络层的路由表决定一个包从 `wlan0` 还是 `wlan1` 发出，两块网卡可以同时工作。

7.8.7 调试实录

让这块网卡真正收发 IP 包，前后改了十处左右。表 7.20 按出现顺序列出现象、原因和修改，每一行都对应一条能在日志里看到的证据。

最后一次启动的结果是 `dhcp: dev=wlan1 address=192.168.0.219 gateway=192.168.0.1`，随后 ARP 解析出网关，`ping` 局域网主机有回应。

表 7.20: MT7601U 调试记录

现象	原因	修改
换台失败，0x0500 总读到同一怪值，无限刷屏	主机直接写 RF_CSR_CFG，与运行中的 MCU 抢间接端口；换台在自旋锁内，失败无限重试	频率表改用 RANDOM_WRITE；换台挪到锁外，退避，8 次后停止
LOFT 校准无应答	校准顺序少了 RXDC 和 BW 滤波	按 Linux mt7601u_init_cal() 补全
第三条 MCU 命令总无应答	EP4 的帧堵在 EP5 应答前面	等应答时交替清空 EP4
EP5 预挂后 1500 ms 超时	probe 时中断未开，完成标志永不置位；EP4 占满 4 个槽	直接轮询 HCINT(4)；槽位改为 5；ch4 轮转
刷屏数分钟	2 ms 轮询 + 每包 3 行跟踪，串口比轮询还慢	空轮询不打印；关 USB_XFER_TRACE
认证超时	在相邻信道发认证	先切到 AP 的信道
M4 重发	M4 加了密，且先装密钥后发	先明文发 M4，再装密钥
组播帧 ICV 错误，DHCP 无 Offer	WCID 属性、共享密钥模式、WCID 地址表未初始化	按 Linux 初始化三张表
解密帧被静默丢弃	PN_LEN 非零时 CCMP 头仍在帧里	rx_data() 跳过 8 字节
聚合包里多帧只交付一帧	单槽接收缓冲	8 帧环形队列，锁外交付
空闲后 reason=6	AP 已清掉客户端	处理 deauth，自动重连

7.8.8 现状与待改进

- **没有保活**：长时间空闲会被 AP 清掉，重连后要重新 dhcp；应定时向 AP 发 null-data 帧；
- **寄存器读偶尔拿到旧值**：装密钥时曾从 0xb000 读回上一次写入 WCID 属性的值，疑为控制传输 IN 的缓冲问题，相关寄存器现在都直接写入，根因待查；
- **热插拔刚刚补上**：最初集线器端口只在启动时扫描一次，运行中拔插不会被发现。第 9 章的 Hub 中断热插拔加上之后，拔出网卡会调用 mt7601u_remove()：先停 NAPI 和 state_work，再注销 wlan1；
- **Wi-Fi 参数写在驱动里**：SSID 和口令写死在 mt7601u.c，而 /etc/wifi.conf 经 wifi_connect() 只交给板载芯片的 brcmfmac_connect()。应改成从配置传入，也避免口令进入代码仓库；
- **只用一个 bulk OUT**：MT7601U 有多个 OUT 端点（Linux 驱动按 AC 队列分开），这里固件、MCU 命令和数据帧都走 EP8，负载高时 MCU 命令会排在数据帧后面；
- **探测期间忙等**：mt7601u_delay_ms() 轮询计数器，固件上传和校准都在 main() 里同步完成，会拉长启动时间。

7.9 CSI-2 摄像头：OV5647

最后一个驱动几乎用上了本章前面所有的东西：mailbox 打开电源和时钟，I2C 配置传感器，Unicam 接收器经 DMA 把帧写进内存，中断通知帧完成，字符设备与 mmap 把缓冲交给用户态，用户程序再把它编码、推流到网络上。

7.9.1 路线选择

树莓派官方的摄像头栈运行在 GPU 固件里（start_x.elf + MMAL），ARM 端只是发命令。本项目选择与 Linux 主线相同的“ARM 直接驱动”路线：内核自己通过 I2C 配置传感器，打开电源域和时钟，用 Unicam 接收 MIPI CSI-2 数据并 DMA 到内存。硬件是 Camera Module v1.3（OmniVision OV5647），接在 3B+ 的 CSI1 口。

7.9.2 硬件结构：I2C、CSI-2、Unicam 与 DMA

摄像头驱动之所以涉及这么多部件，是因为一帧图像从产生到进入内存要经过几条性质完全不同的通路（图 7.33）。弄清楚每条通路传什么、由谁发起，后面的代码就很好读了。

传感器有两个接口。 OV5647 对外有两个互不相干的接口。一个是低速的 *SCCB* 控制口（OmniVision 对 I2C 的兼容实现，两根线 SDA/SCL，约 100 kHz），CPU 通过它读芯片 ID、写一百来个配置寄存器：分辨率、binning、

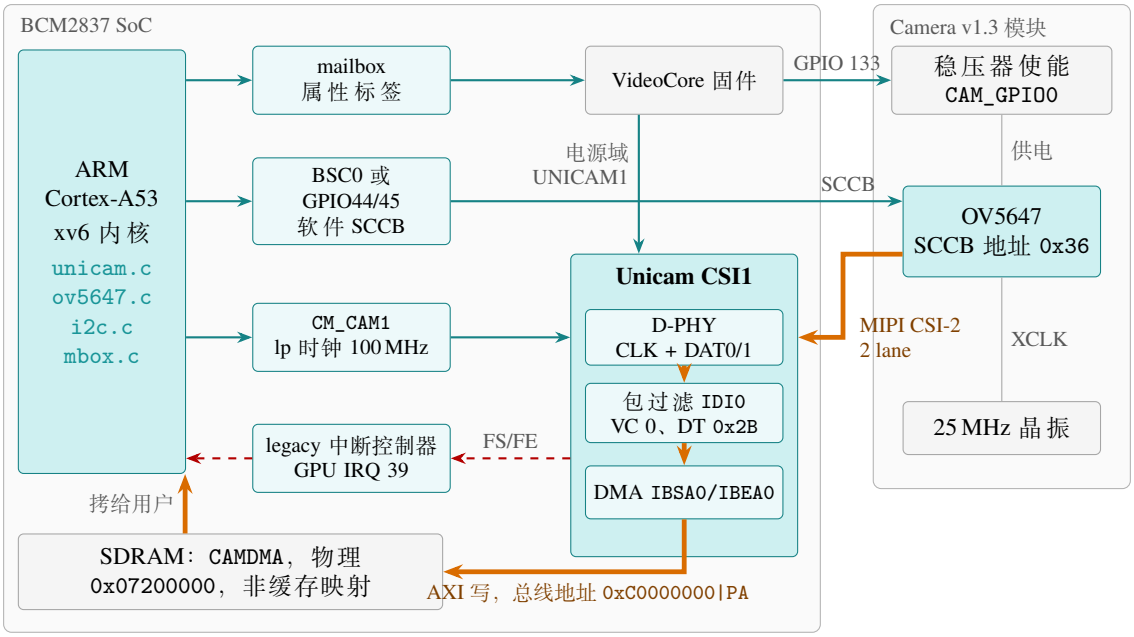


图 7.33: 摄像头的硬件拓扑。细实线是控制面：mailbox 上电和电源域、SCCB 写传感器寄存器、lp 时钟；粗线是数据面：CSI-2 → Unicam → DMA → 内存；虚线是事件：Unicam 的帧开始/结束中断（IRQ 39），另有 10 ms 的 watchdog 兜底（7.9.8 小节）

帧长、曝光、增益、MIPI 输出方式、测试图案，最后写“退出待机”让传感器开始出图。另一个是高速的 *MIPI CSI-2* 输出口：一对时钟线加两对数据线（lane），每对都是差分的 D-PHY 链路，空闲时处于低功耗的 LP-11 状态，传数据时切换到高速模式。图像只走 CSI-2，配置只走 SCCB——I2C 再慢也不影响帧率，它只决定“开机配置”要花多久，这也是驱动后来能退到软件 SCCB 的原因。

CSI-2 链路上传的是包。 传感器把每一帧编码成一串 CSI-2 包：帧开始（FS）和帧结束（FE）是不带数据的短包；中间每一行像素是一个长包，包头里有虚拟通道号（VC）和数据类型（DT，RAW10 是 `0x2B`），包尾有 CRC。接收方据此知道一帧何时开始、何时结束、每个包该放到哪里，CRC 和 ECC 错误也是在这一层被发现的（`STA` 里的 CRCE、PBE 等位）。

Unicam 是接收器，也是 DMA 主设备。 SoC 一侧的 Unicam CSI1 把上面的事情全做了，它内部可以分成三段：

- **D-PHY 前端**：接收差分信号，完成低功耗态和高速态之间的切换。判断“稳定进入高速态”要等若干个 lp 时钟周期（`CLT/DLT` 里的 settle 和终端使能计数），所以它需要时钟管理器 `CM_CAM1` 提供的 100 MHz lp 时钟——时钟频率不对，settle 时间就跟着错，表现为链路误码；
- **包处理**：解析包头、校验 CRC，FS/FE 短包变成 `ISTA` 里的帧开始/结束事件；长包经 `IDI0` 按 VC 和 DT 过滤，只有 VC 0、RAW10 的行数据才往下走。本项目不让 Unicam 解包（`IPIPE= 0`），行数据保持 CSI-2 原样的 10 位打包格式；
- **DMA 写入**：Unicam 自己就是 AXI 总线上的主设备，把每行按 `IBLS` 的行跨度写到 `IBSA0` ~ `IBEA0` 之间的内存，写到哪里由 `IBWP` 指示。这里的“DMA”不是 BCM2837 通用的 DMA 控制器，而是 Unicam 内置的写引擎；它看到的是 VideoCore 总线地址，所以内核写进去的是 `0xC0000000 | PA`（第 4 章“三种地址视图”）。

内存一侧的约束。 Unicam 只认一段起止地址，所以缓冲区必须物理连续，不能用 `kalloc()` 逐页拼；它放在 `PHYSTOP` 以上的 `CAMDMA` 区，分配器永远碰不到。这段内存映射成非缓存的普通内存：DMA 写进 DRAM 的数据，CPU 直接读就是最新的，不需要在每帧前后作废缓存行。

电源和时钟由谁管。 摄像头模块上的稳压器由 CAM_GPIO0 使能，这根线接在 VideoCore 管理的 GPIO 扩展器上，Unicam 所在的电源域也归固件管，ARM 只能通过 mailbox 发属性标签请固件代办。CM_CAM1 时钟则是普通的时钟管理器寄存器，内核直接写。传感器自己的像素时钟来自模块上的 25 MHz 晶振，不需要 SoC 提供。

CPU 只在两头出现。 把这些放在一起看，CPU 的角色就清楚了：开始时它是配置者——经 mailbox 上电、写时钟、经 SCCB 配传感器、经 MMIO 配 Unicam；数据开始流动以后，传感器 → Unicam → 内存这条路完全不经过 CPU；CPU 只需要对 FS/FE 事件作出反应，在恰当的帧开始前换掉 DMA 地址（图 7.37），最后把整帧交给用户。早期版本为了安全屏蔽了中断，由 CPU 每 1 ms 读一次 STA/ISTA；现在以 CSI1 中断（GPU IRQ 39）为主、10 ms watchdog 兜底，见 7.9.8 小节。

顺序约束。 几条通路之间有严格的先后关系，驱动的步骤顺序就是由它们决定的：先上电，SCCB 才可能应答；传感器先停在 LP-11，Unicam 才能在干净的空闲态上完成模拟复位和校准；接收器先使能、传感器后出流，否则第一个 FS 可能丢失；停止时反过来，先停传感器再停接收器，避免接收器在一帧中间被关掉。表 7.21 列出了这些部件在 Pi 3B+ 上的具体资源。

7.9.3 sensor_ops：仿照 V4L2 子设备

桥接驱动 unicam.c 不认识任何具体传感器，传感器驱动也不碰 Unicam。两者之间只有一层接口，对应 Linux 的 v4l2_subdev_ops：

```
struct sensor_ops {
  int  (*power_on)(struct camsensor *s);              // 供电、读芯片 ID、进入 LP-11
  void (*power_off)(struct camsensor *s);
  int  (*stream_on)(struct camsensor *s, const struct cam_mode *mode);
  int  (*stream_off)(struct camsensor *s);
  int  (*set_ctrl)(struct camsensor *s, int id, int value);   // 如测试图案
};
struct camsensor {
  char *name;  const struct sensor_ops *ops;
  const struct cam_mode *modes;  int nmodes;          // 宽、高、行字节数、CSI-2 数据类型
  struct camsensor_bus bus;                           // lane 数、是否非连续时钟、虚拟通道
  int warmup_frames;                                  // 开流后丢弃的帧数
};
```

启动时 ov5647_init() 调用 camsensor_register()，camera_init() 按注册顺序逐个调用 power_on，第一个读到正确芯片 ID 的传感器被绑定到 /dev/video0。要支持 Camera v2（IMX219），只需新写一个实现了 sensor_ops 的文件。

7.9.4 板级资源

表 7.21 列出了从 Raspberry Pi 设备树和 Linux 驱动中查到的资源。

供电和电源域都要通过 VideoCore 固件的 property mailbox：内核在一块 16 字节对齐的缓冲区里写好“标签”（如 0x00038041 设置扩展 GPIO、0x00038030 设置电源域），把地址写入 mailbox 寄存器，等固件把结果写回同一块缓冲区。项目把原来分散在 SD、USB 驱动里的私有 mailbox 代码收拢成了带锁的 kernel/drivers/firmware/mbox.c。电源域编号是一个容易出错的地方：设备树里的 UNICAM1 是 13，发给固件时要加 1。

表 7.21: 摄像头相关的板级资源（Pi 3B+）

项目	值
传感器 I2C	BSC0 走 GPIO44/45（ALT1），从机地址 0x36，芯片 ID 0x5647
传感器供电	固件扩展 GPIO 5，mailbox 编号 128 + 5 = 133
传感器时钟	摄像头板上自带 25 MHz 晶振
接收器	Unicam CSI1，寄存器 0x7e801000，lane 时钟门控 0x7e802004
中断	GPU IRQ 39
电源域	UNICAM1：设备树编号 13，发给固件时要加 1，即 14
lp 时钟	时钟管理器 CM_CAM1，100 MHz
VPU 时钟	Unicam 要求至少 250 MHz，config.txt 中 core_freq=250
CSI-2 链路	2 条数据 lane，非连续时钟，RAW10（数据类型 0x2B）

7.9.5 硬件连接与工作流程

把表 7.21 里的资源和表 7.23 里的排线引脚画到一起，就得到图 7.34。左边是 BCM2837 芯片里与摄像头有关的部件，中间是 15 芯排线，右边是 Camera Module v1.3 板上的器件。

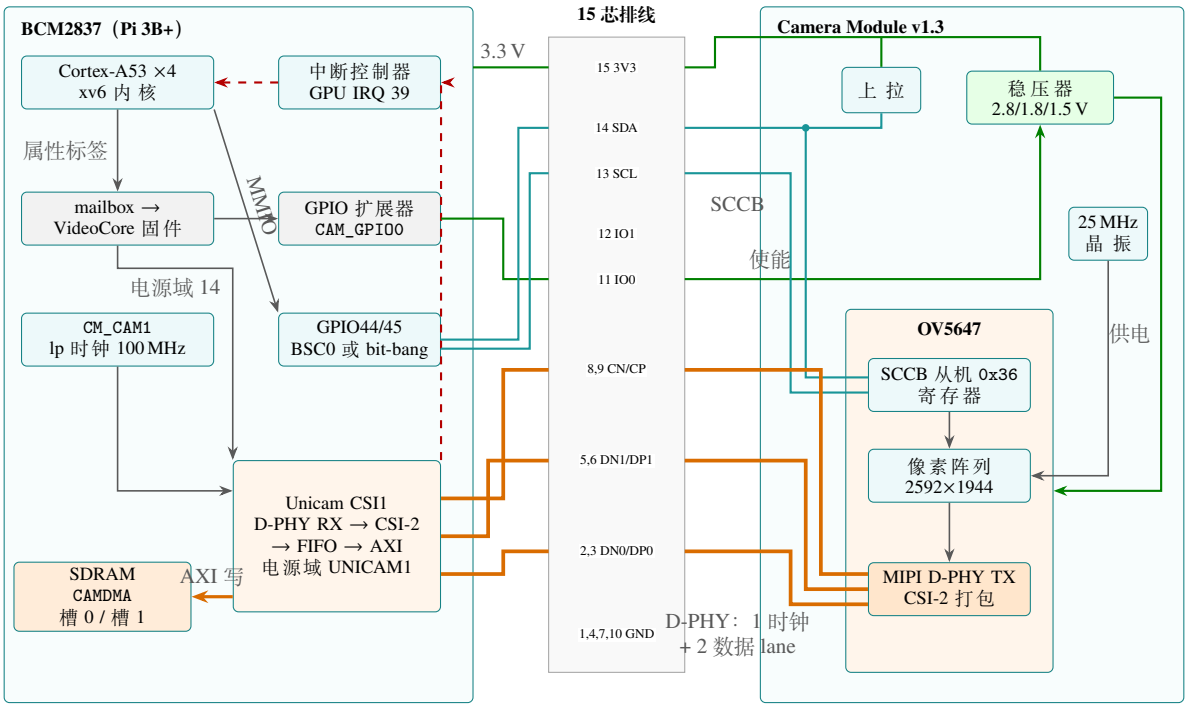


图 7.34: 摄像头的硬件连接。橙色粗线是 D-PHY 差分对（每条线代表一对 P/N）；蓝色是 SCCB 的 SCL/SDA，开漏、靠相机板上的上拉电阻回高；绿色是供电：排线第 15 脚的 3.3 V 和第 11 脚 CAM_IO0 的使能；虚线是帧事件中断；灰色细线是芯片内部的控制路径

图上有三点值得注意。第一，CPU 碰不到相机的电源：CAM_IO0 接在 VideoCore 管理的 GPIO 扩展器上，Unicam 的电源域也归固件管，内核只能经 mailbox 请固件代办。第二，传感器的像素时钟来自相机板上的 25 MHz 晶振，SoC 只给 Unicam 自己提供 lp 时钟（CM_CAM1），两边并不共用时钟；D-PHY 的时钟 lane 是传感器发出的源同步时钟，接收端用它采样数据 lane。第三，相机板和 SoC 之间只有两类信号线：慢速、双向的 SCCB 两根线，和快速、单向（相机到 Pi）的三对差分线。

结合连线看一次完整的工作流程. 按时间顺序，摄像头工作时这些线上依次发生的事情见表 7.22。每一步都标出了哪根线在动、由哪段代码驱动、串口上能看到什么。

图 7.35 把同样的过程画成各条线上的活动随时间的变化。可以看到，SCCB 只在开头的配置阶段（第 4 ~ 7 步）和最后的停流阶段忙碌；进入出流以后，所有的数据都走 D-PHY lane，SCCB 一直空闲，CPU 只在每帧结

表 7.22: 按连线看摄像头的工作流程

	线路	发生了什么	代码 / 日志
1	mailbox（芯片内）	固件打开 UNICAM1 电源域	`mbox_set_domain(14,1)`
2	第 11 脚 CAM_IO0	先拉低 20 ms 让模块放电，再置高；相机板稳压器上电，25 MHz 晶振起振，等 100 ms	`ov5647_power_on()`；`CAM_GPIO0 ... state=1`
3	芯片内	`CM_CAM1` 输出 100 MHz lp 时钟，打开三条 lane 的时钟门控	`cam1_clock_on()`
4	第 13/14 脚 SCL/SDA	读 `0x300a/0x300b`，应得 `56 47`	`software SCCB power-on chip-id=5647`
5	第 2/3、5/6、8/9 脚	写停流序列，三条 lane 停在 LP-11（两根线都为高）	`lanes in LP-11`
6	芯片内	Unicam 模拟复位、设 settle 时间、行跨度、VC/DT 过滤；DMA 指向 dummy；打开 IRQ 39	`unicam_start()`
7	第 13/14 脚	约 90 次寄存器写（约 0.25 s）：公共表、640×480 模式表、曝光/增益、测试图，最后退出软件待机	`programming common registers (51)` 等
8	第 8/9 脚	时钟 lane 进入 HS；非连续时钟，包与包之间回到 LP	`sensor streaming`
9	第 2/3、5/6 脚	每帧：FS 短包，480 个 800 字节的 RAW10 长包，FE 短包；前 30 帧写进 dummy 供曝光收敛	状态 WARMUP→STREAMING
10	芯片内 AXI	FS 锁存槽地址；行数据经 FIFO 写进 `CAMDMA` 槽	槽 NEXT→ACTIVE
11	IRQ 39	FE 产生中断；`unicam_irq()` 把槽放进完成队列，唤醒读者	`frame captured by CSI1 IRQ 39`
12	第 13/14、11 脚	关闭 `/dev/video0`：SCCB 写停流，lane 回 LP-11；停 Unicam，CAM_IO0 拉低，关时钟和电源域	`cam_stream_stop()`

束时被 IRQ 39 打断一次。这就是为什么把 SCCB 降到 25 kHz 只会让开机慢一点，不会影响帧率。

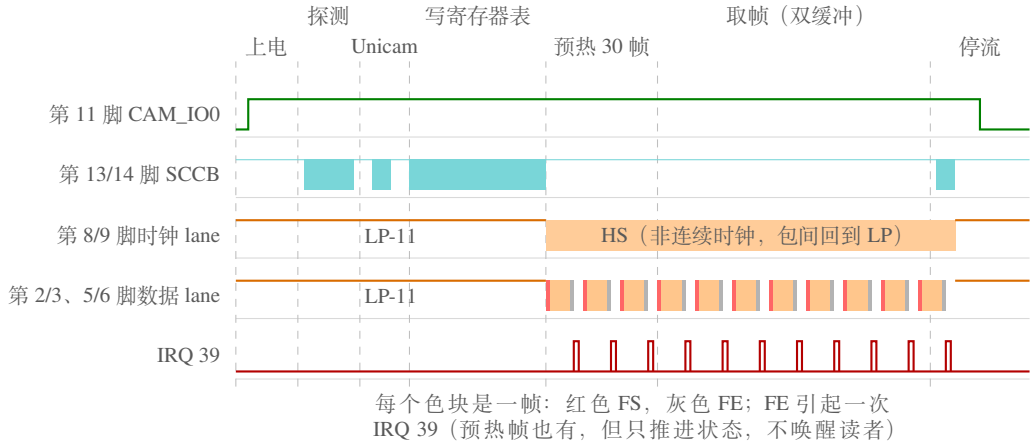


图 7.35: 各条线在一次工作周期中的活动（时间轴示意，不按比例）。CAM_IO0 在上电时先低后高、停流时拉低；SCCB 只在配置和停流时有事务；lane 在退出软件待机之后才从 LP-11 进入高速传输

一帧数据在连线上走过的路。 落到某一帧上，路径是：OV5647 的像素阵列在 25 MHz 晶振驱动下曝光、读出，经合并（binning）和跳采（skipping）缩到 640×480，每 4 个 10 位像素打包成 5 字节；D-PHY TX 把每一行包成一个长包，字节轮流分到第 2/3、5/6 脚两对数据线上，第 8/9 脚提供采样时钟；Unicam 的 D-PHY RX 在 `CLT/DLT` 规定的 settle 时间后锁定高速信号，解串、校验 ECC/CRC，按 `IDI0` 只放行 VC 0、DT `0x2B` 的行数据；这些行经 FIFO 交给 AXI 写主机，按 `IBLS=` 800 的行跨度写进 `CAMDMA` 里在上一个 FS 锁存的槽。FE 短包到达时，Unicam 置 `ISTA.FEI` 并拉起 IRQ 39，CPU 这才第一次介入这一帧：把槽移进完成队列、唤醒读者。整帧 384 000 字节从传感器到内存，CPU 没有搬过一个字节。

7.9.6 上电与探测

启动时 `main()` 先调用 `ov5647_init()` 登记传感器驱动（此时不碰硬件），再由 `camera_init()` 注册字符设备 `CAMERA`（主设备号 5，`/dev/video0`），然后对每个登记过的传感器先检查其默认模式 Unicam 能否接收，再调用 `power_on`。OV5647 的做法是：经 mailbox 把固件扩展 GPIO 5 先拉低 20 ms、再置 1，等 100 ms，把 GPIO44/45 交给传感器的控制总线，读寄存器 `0x300a/0x300b`，应得 `0x56 0x47`。下面几段讲这条控制总线是怎样从硬件 I2C 一步步退到 25 kHz 软件 SCCB 的。成功时打印：

```
camera: OV5647 on CSI1 (2 lanes) -> /dev/video0, 640x480 RAW10
```

mailbox. 供电走的是固件 property mailbox。电源控制现在与 Linux `gpio-raspberrypi-exp.c` 一致：先用 `GET_GPIO_CONFIG` 读出并保留 CAM_GPIO0 的极性，再设置方向和电平，最后用 `GET_GPIO_STATE` 回读，标准 Pi 3 应显示 `direction=output polarity=normal state=1`。消息本身按 Linux `rpi_firmware_property()` 的约定组织：标签的请求/应答长度字填 0；发送前清空 FIFO 里其他驱动超时后遗留的旧回复（日志里的 `stale replies K`）；只接受地址与本次请求相同的回复；判断成败时不看单个标签的 response 位，而看整条消息成功后返回结构的第一个字。这些规则是怎么来的，见 7.9.11 小节的问题 1。

控制总线：从 BSC0 到软件 SCCB. 读 16 位寄存器地址需要“写地址、不发 STOP、repeated START、读数据”的组合事务，`i2c_write_read()` 照 Linux `i2c-bcm2835.c` 实现了它（BSC 的写 FIFO 不能预填，要先启动空写事务、等 `TXW` 再填）。但在真机上，BSC0 对 `0x36` 始终 NACK，而 GPIO bit-bang 探测能得到 ACK，降速也无济于事。最终的驱动（`gpio-sccb-v8`）因此从第一次访问起就用 GPIO44/45 上约 25 kHz 的软件 SCCB：STOP/START 分离读法，拒绝任何不是 `0x5647` 的读数，失败就断电重来，不再扫描其他地址。BSC0 仍按 100 kHz 初始化并保留两段式中止，只是不再发事务。这条路是怎么一步步走过来的，见 7.9.11 小节的问题 2 ~ 6。

延迟探测. `/dev/video0` 在探测之前就已注册，所以 `open` 成功不代表绑定了传感器。启动时只探测一次，避免没有摄像头时拖慢开机；如果启动探测偶然失败，第一次 `read()` 或 `pattern=N` 写入会在拍照睡眠锁 `cam.busy` 下重新探测，最多三个完整的断电/上电周期，每次失败等 100 ms。

7.9.7 拍一帧

本小节和图 7.36、图 7.37 描述的是最初的单缓冲版本，7.9.10 和 7.9.11 小节的日志也来自它；现在的双缓冲、中断驱动版本见 7.9.8 小节。用户每 `read()` 一次 `/dev/video0` 就拍一帧，返回 40 字节的帧头（magic、宽、高、行字节数、格式、长度、序号、时间戳）加打包的 RAW10 数据。帧头里的 `format` 同时给出 Bayer 顺序（OV5647 是 BGGR，IMX219 是 RGGB），用户程序据此去马赛克。`cam_capture()` 的顺序与 Linux 相同——开始时先启动接收器再让传感器出流，结束时先停传感器再停接收器；整条调用链见图 7.36：

1. `mbox_set_domain(14, on)` 打开 UNICAM1 电源域；
2. `CM_CAM1` 时钟：PLLD_PER（500 MHz）÷5 = 100 MHz；若 BUSY 位不置起，就退回 19.2 MHz 晶振并打印提示；
3. `power_on`：上电、校验 ID、进入 LP-11；
4. 按传感器的总线参数配置 Unicam（照 Linux `unicam_start_rx()`）：lane 数、虚拟通道、数据类型、行字节数（16 的倍数），DMA 先指向一块大小为 0 的“假缓冲区”；
5. `stream_on`：依次写公共寄存器表、640×480 模式表、HTS/VTS/曝光/增益、缓存的测试图案，退出软件待机，开始发送；
6. 帧状态机：WARMUP 丢弃前 30 帧（约 0.5 s）让传感器自带的 AEC/AGC 收敛；ARMED 在下一个帧开始时把 DMA 地址换成真缓冲区；CAPTURE 一开始就写回假缓冲区；帧结束后进入 DONE；

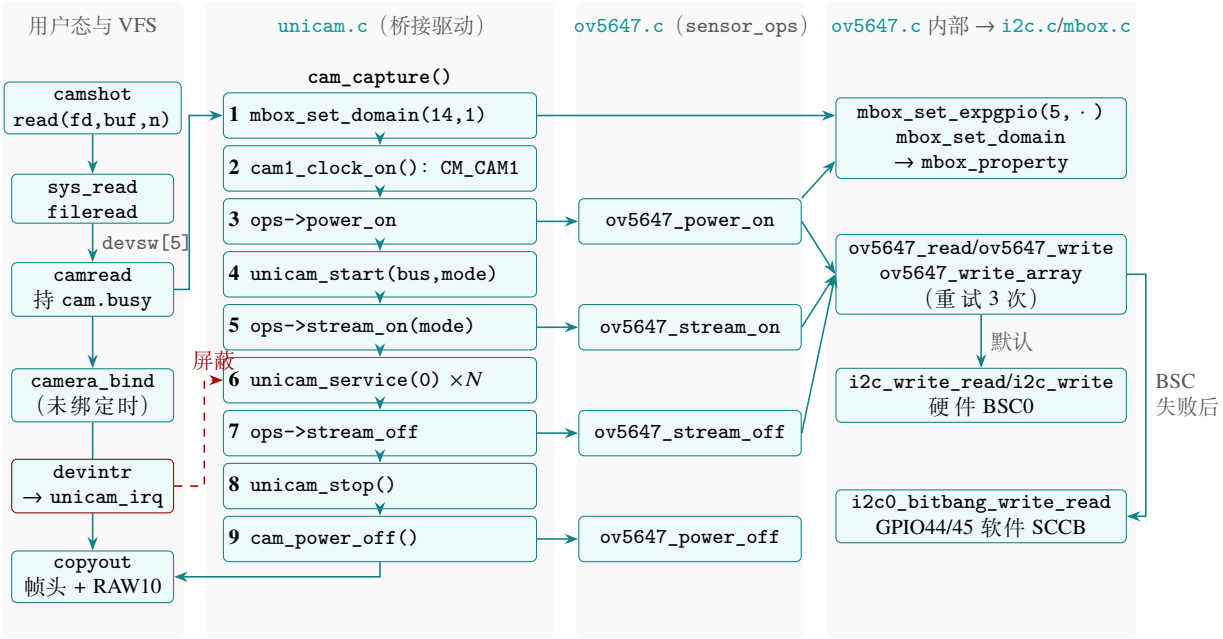
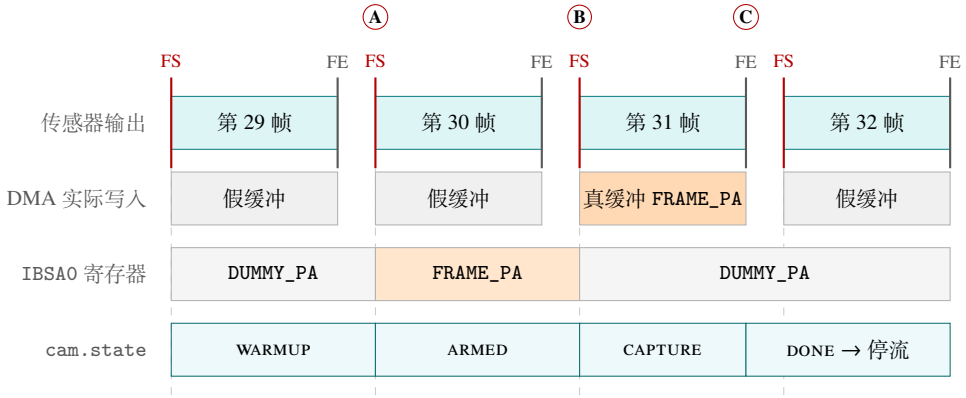


图 7.36: 一次 `read("/dev/video0")` 的调用关系。`unicam.c` 只通过 `sensor_ops` 调用传感器驱动；传感器驱动再经 mailbox 上电、经 BSC0 或软件 SCCB 写寄存器。第 6 步每 1 ms 轮询一次 Unicam 状态，直到 DONE 或 4 s 超时，是目前采集帧的唯一路径，中断入口已屏蔽；第 9 步的 `ov5647_power_off` 同样经 mailbox 断电

7. `stream_off` → 停 Unicam → `power_off` → 关时钟和电源域，把数据从 `CAMDMA`（非缓存映射，物理 0x07200000）拷给用户。

第 6 步的关键（图 7.37）是：写进 `IBSA0/IBEA0` 的新地址要到下一个帧开始才生效。所以真缓冲那一帧刚开始就把假缓冲写回去，这一帧会继续完整地写进真缓冲，之后的帧则落进假缓冲，缓冲区里恰好留下一整帧。假缓冲的大小编程为 0，是照搬 Linux 绕开循环缓冲模式越界写缺陷的做法。



A 第 30 个 FS（或 1.5 s 期限）：写入 `FRAME_PA`，进入 ARMED；本帧仍写假缓冲

B 下一个 FS：新地址生效，第 31 帧写进真缓冲；处理程序立刻写回 `DUMMY_PA`，进入 CAPTURE

C 第 31 帧的 FE：真缓冲里正好一整帧，进入 DONE；第 32 帧落进假缓冲

图 7.37: 帧状态机的时序。`IBSA0/IBEA0` 的新值要到下一个帧开始（FS）才被硬件锁存，所以“写真地址”和“写回假地址”都比它们生效的那一帧早一个 FS

为什么用轮询而不是中断。 设备树说 CSI1 是 GPU IRQ 39，但这个编号没有在真机上验证过；第一次打开它时 CPU0 陷入了清不掉的电平中断（7.9.11 小节问题 7）。所以目前屏蔽 IRQ 39，用 `CNTVCT_EL0` 计一个 4 秒期限、每 1 ms 轮询一次 Unicam 状态。现在真机上的日志是：

```
unicam: frame captured by polling; CSI1 IRQ 39 intentionally masked
```

它表示 sensor → CSI-2 → Unicam → DMA 的整条路径已经打通，只是 IRQ 路由还留待单独验证。

7.9.8 从 D-PHY 到内存：FIFO、双缓冲与帧通知

上一小节描述的是最初的单缓冲版本：每次 `read()` 都完整地上电、预热，挂一次真缓冲抓一帧，再轮询状态寄存器等它结束。现在的驱动把这条路径拆得更细：传感器一直出流，Unicam 在两个 DDR 槽之间轮换写入，帧结束由 CSI1 中断通知，读者在等待通道上睡眠。本小节从线缆开始，把一帧像素从传感器到用户进程所经过的每一层讲清楚。

排线与 D-PHY. 15 芯排线本身只是 15 根铜导体，不是 D-PHY。D-PHY 指的是 OV5647 内部的高速发送 PHY、线上的差分对和 Unicam 内部的接收 PHY 这一整套物理层。排线同时承载 D-PHY、低速 SCCB、控制线、电源和地（表 7.23，方向以 Pi 主板为准）。

表 7.23: 摄像头 15 芯排线的引脚

引脚	信号	含义
1, 4, 7, 10	GND	地；夹在差分对之间，提供回流路径并隔离
2 / 3	CAM_DN0 / CAM_DP0	D-PHY 数据 lane 0 的负/正线，输入到 Pi
5 / 6	CAM_DN1 / CAM_DP1	D-PHY 数据 lane 1 的负/正线
8 / 9	CAM_CN / CAM_CP	D-PHY 时钟 lane 的负/正线
11	CAM_IO0	3.3 V GPIO，模块电源使能（`CAM_GPIO0`）
12	CAM_IO1	3.3 V GPIO，用途取决于模块（时钟、LED 等）
13 / 14	SCL / SDA	3.3 V SCCB 控制总线，不属于 D-PHY
15	3V3	Pi 给相机板供电

每条 lane 都是一对差分线：P、N 两根线上是互补的电压，接收端只看两者之差，不把它们当成两根普通 GPIO 读 0 和 1。lane 有两种工作方式。*低功耗*（LP）方式下两根线各自是单端的 1.2 V 逻辑，两根都为高就是 LP-11，即停止态，传感器停流时 lane 就停在这里。*高速*（HS）方式下是约 200 mV 的小摆幅差分信号，速率可达每 lane 每秒数百兆比特。从 LP-11 经过一串规定的 LP 状态进入 HS 叫“开始传输”（SoT），HS 数据之后再回到 LP-11 叫“结束传输”（EoT）。时钟 lane 提供源同步时钟，两条数据 lane 并行，字节轮流分配到两条 lane 上。Unicam 的 `CLT`、`DLT` 寄存器设置的就是接收端判断“已稳定进入 HS”所需的 settle 时间和终端电阻打开时间，单位是 lp 时钟周期，所以 `CM_CAM1` 的时钟频率直接影响 D-PHY 能否正确收数。

CSI-2 包. D-PHY 之上的 CSI-2 协议层把一帧编码成一串包（图 7.38）。每一次 HS 突发（SoT 到 EoT 之间）携带一个包。短包只有 4 个字节：数据标识（2 位虚拟通道 VC、6 位数据类型 DT）、16 位数据字段和 8 位 ECC。帧开始（FS，DT `0x00`）和帧结束（FE，DT `0x01`）就是短包。长包先是 4 字节包头（数据标识、16 位字数、ECC），然后是数据，最后是 16 位 CRC。OV5647 的每一行像素都是一个 DT `0x2B`（RAW10）的长包，640 个像素按每 4 个像素 5 字节打包，正好 800 字节。



图 7.38: 一帧在 CSI-2 数据 lane 上的样子。每个包是一次独立的 HS 突发，包与包之间回到 LP 状态。FS、FE 短包标出帧的边界，每一行像素是一个 RAW10 长包

两层各管一件事：电气层的 LP→HS/SoT 只告诉接收 PHY“一段高速数据来了”；协议层的 FS/FE 短包才告诉 Unicam“帧从这里开始、到这里结束”。OV5647 和 Unicam 之间没有额外的握手线，SCCB 也不参与逐帧的握手，它只在出流之前把传感器配置好。所以开始的顺序是：CPU 先配置并使能 Unicam，再经 SCCB 让 OV5647 退出软件待机，此后的一切都由数据 lane 上的包驱动。

Unicam 内部：FIFO 与 AXI 写主机。 这里有两种容易混淆的“缓冲”（图 7.39）。第一种是 Unicam 内部的 FIFO，位于包解析器和 AXI 写主机之间，是 SoC 里的一个小硬件队列：它吸收 CSI-2 的突发数据与 DDR 写入之间的速率差。软件看不到它的地址，也不能给它分配大小；驱动能做的只是通过 PRI 设置 AXI 请求的普通/紧急优先级和阈值（PP=0xe，NP=8，PT=2，PE=1），通过 ICTL.IBOB 选择图像缓冲溢出时的行为，并从 STA.IFO、STA.OFO 读出输入/输出 FIFO 是否溢出过。第二种才是保存整帧的 DDR 图像缓冲。注意 IBLS= 800 是 DDR 里的行跨度，不是 FIFO 的大小。

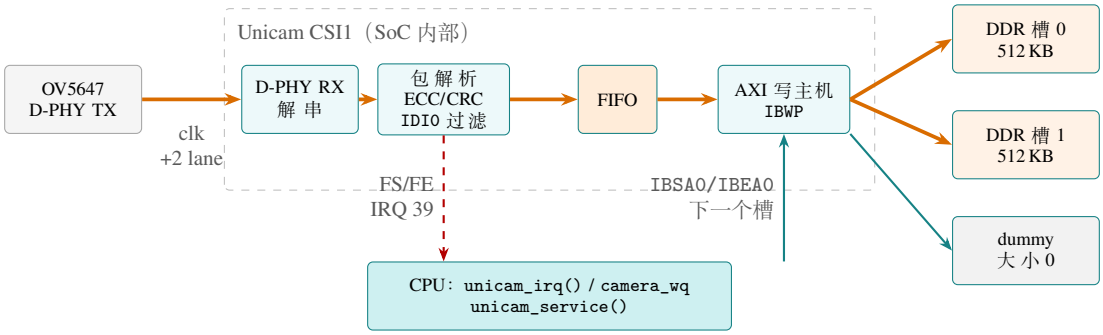


图 7.39: Unicam 的接收流水线。粗线是像素数据，由 Unicam 作为 AXI 主设备自己写进 DDR，不经过 CPU，也不使用 BCM2837 的通用 DMA 通道；虚线是帧事件；细线是 CPU 为下一帧排好的槽地址

DMA 缓冲区的布局。 图像缓冲放在 PHYSTOP 之上专门保留的 2 MB CAMDMA 区（物理 0x07200000）里，kalloc() 永远碰不到它。区内依次是两个 512 KB 的帧槽和一个 dummy 页：

```
CAMDMA_PA = 0x07200000
  slot 0   0x07200000 .. 0x0727ffff   512 KB（OV5647 640x480 用 384000 字节）
  slot 1   0x07280000 .. 0x072fffff   512 KB
  dummy    0x07300000                 编程为大小 0（IBEA0 == IBSA0）
写进 Unicam 的是总线地址：0xC0000000 | PA
```

Unicam 只认一段起止地址，所以每个槽必须物理连续。内核和用户两侧都把这段内存映射成 Normal-NC（非缓存普通内存）：Unicam 写进 DRAM 的像素，CPU 直接读到的就是最新数据，每帧前后不需要作废缓存行。Arm 不保证同一物理页用不同内存属性映射（例如内核可缓存、用户不可缓存）时的正确性，所以两侧的属性必须一致。

双缓冲：槽的状态机。 每个槽在五个状态之间循环（图 7.40）：FREE 表示空闲；NEXT 表示地址已写进 IBSA0/IBEA0，等下一个 FS 锁存；ACTIVE 表示 Unicam 正在往里写；DONE 表示一帧已完整写完，排在完成队列里；USER 表示被读者取走，正在被用户使用。

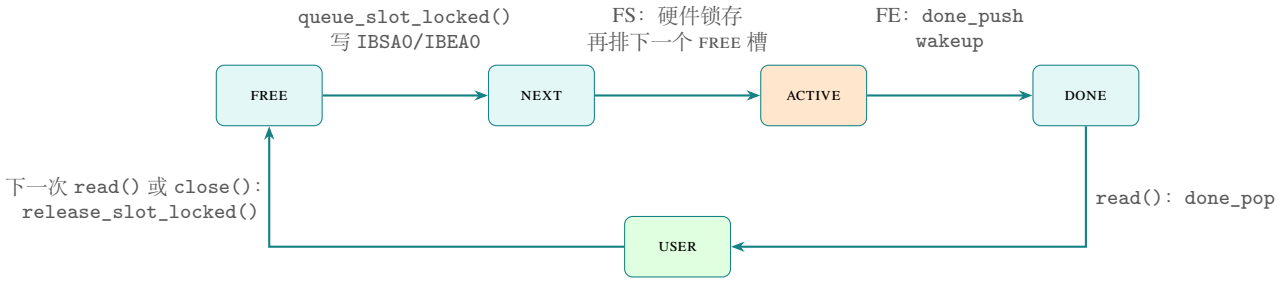


图 7.40: DMA 槽的状态机。若两个槽都处于 ACTIVE、DONE 或 USER，没有 FREE 槽可排，驱动就把 dummy 写进 IBSA0/IBEA0，下一帧被丢弃而不会覆盖用户仍在读的像素

整套机制建立在 Unicam 的一个硬件特性上：写进 IBSA0/IBEA0 的地址先进入影子寄存器，要到下一个 *FS*

才被锁存。所以软件可以在当前帧写入期间，提前为下一帧排好槽。中断处理里先处理 FE、再处理 FS，因为一次中断里两者可能同时出现：

```
// unicam_service()，持 cam.lock
if(fe && cam.active_slot >= 0){           // 帧结束：当前槽写完了
  int slot = cam.active_slot;
  cam.active_slot = -1;
  cam.buffers[slot].state = CAM_BUF_DONE;
  done_push_locked(slot);                 // 放进两项的完成队列
  wake = 1;
}
if((ista & ISTA_FSI) && cam.next_slot >= 0){  // 帧开始：硬件刚锁存了 next 槽
  cam.active_slot = cam.next_slot;
  cam.next_slot = -1;
  cam.buffers[cam.active_slot].state = CAM_BUF_ACTIVE;
  queue_slot_locked();                    // 为再下一帧排一个 FREE 槽，没有就排 dummy
}
if(wake)
  wakeup(&cam.done_count);
```

图 7.41 是连续出流时的情形：第 n 帧写进槽 0 时，槽 1 已经排好；读者处理槽 0 的同时，槽 1 接收第 n+1 帧。读者把槽 0 还回来以后，它又成为第 n+2 帧的目标。只有读者同时占着两个槽（例如处理得太慢），下一帧才会落进 dummy 被丢弃。

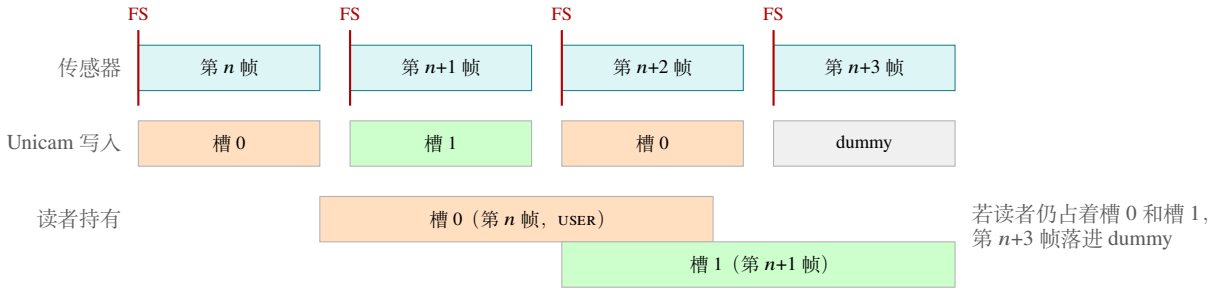


图 7.41: 双缓冲的时序（示意）。每个槽的地址都在前一个 FS 时排好；读者从 FE 之后开始处理，处理期间另一个槽继续接收

帧通知：中断为主，watchdog 兜底。 FE 到来时，Unicam 置 `ISTA.FEI`（以及 `STA.PIO`），并经 legacy GPU IRQ 39 向 CPU 发 CSI1 中断。`unicam_irq()` 调用 `unicam_service(1)`：读出并清除 `STA/ISTA`，推进上面的槽状态，在 `&cam.done_count` 上 `wakeup()`。读者那一侧不再忙等：

```
// cam_grab()
acquire(&cam.lock);
if(cam.done_count == 0 && cam.next_slot < 0 && cam.active_slot < 0)
  queue_slot_locked();                    // 第一次取帧：排第一个槽
while(cam.done_count == 0 && r_cntvct_el0() < deadline && !myproc()->killed)
  sleep(&cam.done_count, &cam.lock);      // 由 FE 中断或 watchdog 唤醒
slot = done_pop_locked();                 // DONE -> USER
release(&cam.lock);
```

这里有三处防御。第一，IRQ 39 来自设备树，没有在真机上验证过；早期打开它时 CPU0 陷进了清不掉的中断（7.9.11 小节问题 7）。所以 `unicam_irq()` 连续 100 次没有发现 Unicam 状态时就屏蔽这个中断，改为只靠

轮询。第二，camera_wq 每 10 ms 运行一次 unicam_poll_work()，调用同一个 unicam_service(0) 并唤醒读者；中断没有路由过来的板子靠它取帧，4 秒的截止时间也用不依赖定时器中断的 CNTVCT_EL0 计算。第三，“写槽地址”和“发布 NEXT 状态”在同一个 cam.lock 临界区里完成，set_image_buffer() 在 MMIO 写之后执行 dsb sy：另一个 CPU 上的中断处理看到 NEXT 状态时，硬件一定已经收到了对应的地址，不会出现“状态说已排好、寄存器里却还是旧地址”的竞态。

零拷贝 mmap. 普通的大缓冲 read() 仍返回“40 字节帧头 + RAW10”，要拷贝 384 KB。现在字符设备还提供 mmap：驱动的 cammmap() 用 remap_pfn_range() 把两个槽以只读、不可执行、Normal-NC 的属性映射到内核选定的 mmap 区地址（第 4.10 节；早期版本固定在 CAM_MMAP_BASE，现在由 mmap() 的返回值给出）。映射之后，传入恰好 40 字节缓冲的 read() 只复制帧头，帧头的 reserved 字段给出槽号，像素直接从映射里读：

```
uchar *dma = mmap(0, CAM_MMAP_BYTES, PROT_READ, MAP_SHARED, fd, 0);
struct cam_frame_hdr h;
if(read(fd, &h, sizeof(h)) == sizeof(h)){
  const uchar *raw10 = dma + h.reserved * CAM_SLOT_BYTES;
  ...                                  // 下一次 read() 或 close() 之前有效
}
```

被取走的槽处于 USER 状态，直到下一次 read() 或 close() 才归还，因此 Unicam 不会在用户读像素时覆盖它。这段 VMA 带 VM_PFNMAP 和 PTE_SPECIAL：fork() 的 dup_mmap() 照抄 PTE 而不复制物理页；munmap()、exec() 和进程退出只拆页表，绝不会把保留的 DMA 物理页交给 kfree()。映射带 VM_NOFILEREF，不持有 struct file，所以最后一个 fd 关闭时照旧停流；vm_ops 的 open/close 维护 camera_mapped 计数。

这一版已经通过完整构建；以 CSI1 中断为主的完成路径还需要烧录后，从真机日志 unicam: frame captured by CSI1 IRQ 39; watchdog remains armed 来确认。

7.9.9 用户程序 camshot

camshot 用纯整数运算完成图像处理（用户程序以 -mcpu=cortex-a53+nofp 编译）：解包 RAW10（每 4 个像素占 5 字节），减黑电平 16，BGGR 双线性去马赛克，灰度世界白平衡（增益限制在 0.5 ~ 4），把绿色通道的第 99 百分位拉到满幅，查表做 gamma 2.2 校正，写成 24 位 BMP。-t N 让传感器输出内部测试图案（1 彩条、2 色块、3 随机），这时改用固定的 1.0 白平衡和固定电平，不做统计——测试图案本身是校准过的，正好单独验证传感器、CSI-2、DMA、转换和写盘；-s 输出 320×240 半尺寸。

640×480 的 BMP 有 900 KB，超过原生 xv6 文件系统单文件 268 KB 的上限（MAXFILE= 12 + 256 块），所以默认写到以读写方式挂载的 FAT32 启动分区 /boot/camera.bmp，关机后把 SD 卡插到电脑上就能看到。为了让慢速写盘不被误判成“卡死”，camshot 在每个阶段打印一行（收到帧、解包、统计、打开输出、写头、转换），BMP 按 16 行一批写入（半尺寸图从 240 次 FAT32 写降到 15 次），启动时打印的版本串 converter fixed-pattern-v2 fat-progress batch16 用来确认 SD 卡上的程序确实已经更新。

7.9.10 一次成功拍照的日志解读

下面是 v8 驱动在真机上一次成功拍照的完整串口输出（camshot -t 1 -s -o /boot/cam02.bmp：传感器彩条、半尺寸、写到启动分区）。左边的 (1) ~ (6) 是为了讲解加的阶段编号，与下文 ① ~ ⑥ 一一对应：

```
(1) camera: driver gpio-sccb-v8 unicam-poll-v2
    ov5647: CAM_GPIO0 direction=output polarity=normal state=1
    i2c0: GPIO44/45 ALT1, 100000 Hz (core 250000000), BCM2837 two-stage abort
    ov5647: probing with 25 kHz GPIO SCCB; BSC0 bypassed
    ov5647: software SCCB power-on chip-id=5647
```

```
    ov5647: sensor outputs enabled
    ov5647: power-on complete, lanes in LP-11
    camera: OV5647 on CSI1 (2 lanes) -> /dev/video0, 640x480 RAW10
$ /bin/camshot -t 1 -s -o /boot/cam02.bmp
(2) camshot: converter fixed-pattern-v2 fat-progress batch16
    camshot: capturing...
    ov5647: CAM_GPIO0 direction=output polarity=normal state=1
    ... (与 (1) 相同的上电五行)
    ov5647: power-on complete, lanes in LP-11
(3) unicam: sensor powered, receiver setup begins
    unicam: receiver enabled, starting sensor stream
    ov5647: programming common registers (51)
    ov5647: programming mode registers (29)
    ov5647: programming controls (11)
    ov5647: streaming started
    unicam: sensor streaming, waiting for frame
(4) unicam: frame captured by polling; CSI1 IRQ 39 intentionally masked
(5) camshot: frame received 640x480, 384040 bytes
    camshot: unpacking RAW10...
    camshot: sensor test pattern; fixed WB/level
(6) camshot: preparing output /bovfs: boot create begin path=/boot/cam02.bmp sub=/cam02.bmp mode=601
    ot/cam02.bmp (230454 bytes)...
    vfs: boot create stat complete exists=0
    vfs: boot create directory-entry begin
    fat32: create lookup begin path=/cam02.bmp
    fat32: create lookup complete exists=0
    fat32: create reserve begin slots=2
    fat32: create reserve complete
    vfs: boot create directory-entry complete
    vfs: boot create truncate begin / complete
    vfs: boot create verify-stat begin / complete
    vfs: boot create openwrite begin / complete
    camshot: output opened; writing BMP header...
    camshot: converting and writing /boot/cam02.bmp (230454 bytes)...
    camshot: 320x240 frame 31, WB R x1.0 B x1.0, level x1.0 -> /boot/cam02.bmp
```

① **启动时的探测与绑定。** 这是 `main()` → `camera_init()` → `camera_bind()` → `ov5647_power_on()` 的输出。第一行是驱动版本串，用来确认 SD 卡上跑的确实是新内核。第二行来自 mailbox 的回读：CAM_GPIO0 是输出、极性正常、电平为 1，模块的稳压器已经打开。第三行由 `i2c0_init_camera_pins()` 打印：BSC0 仍按 100 kHz 初始化（分频 250 MHz/100 kHz = 2500），出错时采用“两段式中止”——先保持 `I2CEN` 清 FIFO 等 `TA` 归零，再关控制器（7.9.6 小节）。但紧接着的一行说明这次根本不用它：v8 直接用 25 kHz 软件 SCCB。读到 5647 之后，驱动写一小段寄存器打开传感器的输出引脚，再写停流序列让 MIPI 链路停在 LP-11 空闲态——Unicam 稍后要在这个干净的空闲态上做模拟复位和校准。最后一行表示绑定成功；绑定之后 `camera_bind()` 立即 `power_off()`，模块在两次拍照之间是断电的。

② **每次拍照都重新上电。** 用户 `read()` 进入 `camread()`，持 `cam.busy` 调 `cam_capture()`，后者再次 `ops->power_on()`，所以上电五行原样再出现一次。这是“每次拍照完整上电、预热、断电”这个设计的直接体现，

也是一次拍照要一秒左右的主要原因。

③ 先接收器、后出流。 `unicam: receiver enabled` 在 `starting sensor stream` 之前，顺序与 Linux 一致。随后 `ov5647_stream_on()` 写三张表：公共寄存器 51 项、640×480 模式 29 项、曝光增益等控制 11 项，再加上虚拟通道、测试图案、退出待机、MIPI 控制等几次读改写。它们全部经 25 kHz 软件 SCCB：一次 16 位地址的写是 4 个字节、约 36 个时钟，约 1.5 ms，九十多次写加起来约 0.15 秒——慢，但只在开流时发生一次。

④ 一帧到手。 `cam_capture()` 每 1 ms 调一次 `unicam_service(0)`，状态机经过 WARMUP（丢 30 帧）、ARMED、CAPTURE，在帧结束时进入 DONE（图 7.37）。“`intentionally masked`”说明 IRQ 39 是有意屏蔽的，不是丢了中断。

⑤ 数字对得上。 `read()` 返回 384040 字节：40 字节帧头加 $640 \times 480 \times 10/8 = 384000$ 字节 RAW10。测试图案模式下 `camshot` 用固定的 1.0 白平衡和电平，所以下一行直接是 `fixed WB/level`。

⑥ 创建输出文件。 这一段同时有用户态和内核两个来源的输出，所以第一行被“切开”了：`camshot` 的 `printf` 在用户态逐字符 `write(1, &c, 1)`，打到 `"/bo"` 时进程陷入 `open()`，VFS 的内核 `printf` 插了一整行进来，返回用户态后才接着打出 `"ot/cam02.bmp (230454 bytes)..."`。传给内核的路径并没有坏，`path=/boot/cam02.bmp` 写得清清楚楚。之后各行对应当时 `vfsopen()` 的创建流程（这份日志录于第一版 VFS；统一 VFS 之后同样的步骤由 `sys_open()` → `pfs_create()` → `O_TRUNC` 截断 → `pfs_open()` 完成，见 6.5 节）：

- `mode=601: 0x001|0x200|0x400`，即 `O_WRONLY|O_CREATE|O_TRUNC`；
- `stat complete exists=0: fat32statpath()` 正常返回，文件还不存在；
- `fat32: create lookup: fat32createfile()` 内部再查一次，确认不重名；
- `reserve begin slots=2: cam02.bmp` 本身就是合法的 8.3 名字，不需要 LFN，所以只要 2 个目录项——一个短名项，加一个全 0 的结束标记；根目录尾部空位够用，不必扩展目录簇；
- `truncate`：对刚建的空文件截断，首簇为 0，什么都不用释放；
- `verify-stat`、`openwrite`：重新确认目录项，并把首簇、大小装进 `fat32_file` 写句柄（当时在 vnode 里，现在在 inode 的 `fsdata` 里），供之后的追加写使用。

最后一行的 230454 是 54 字节 BMP 头加 $320 \times 240 \times 3 = 230400$ 字节像素；`frame 31` 是帧头里的序号，即帧结束计数——30 帧预热之后的第 31 帧。输出是 320×240，是因为 `-s` 把每个 2×2 Bayer 块合成一个像素，不是 Unicam 丢了一半分辨率。

这份日志证明了什么。 对照 7.9.2 小节的四条通路：mailbox 供电（CAM_GPIO0 回读）、SCCB 控制（读到 `5647`、三张表写完）、CSI-2 数据面与 DMA（`frame captured`、长度正确）、事件（轮询完成，IRQ 有意屏蔽）全部打通；用户态转换和 FAT32 写盘也都完成。它同时也是 7.9.11 小节问题 5、6（软件 SCCB 读 ID 不稳）和问题 11（新建文件停在 `directory-entry begin`）修好之后的验证记录。

7.9.11 真机调试记录

摄像头从“QEMU 上能探测到没有摄像头”到“真机写出第一张 BMP”，经历了下面十一个问题。前七个属于摄像头本身的控制面和数据面；后四个是摄像头作为第一个同时用到大块 DMA、长时间用户态计算、FAT32 大文件写和 Wi-Fi 并发的程序，逼出来的系统问题。每个问题都按同样的顺序记录：先是串口上看到的日志，再是分析，最后是解决方案。日志中凡是标注“格式”的，是按驱动源码里的 `printf` 格式写出的形式，其余是真机原样输出。

问题 1：mailbox 打开摄像头电源失败

日志输出.

```
ov5647: firmware GPIO 133 (camera power) failed, firmware rev N          (格式)
ov5647:   SET_GPIO_CONFIG: <结果> (status .., tag response .. = done D len L,
          first word W, stale replies K)
ov5647:   SET_GPIO_STATE:  <结果> (...)
ov5647: no reply at i2c0 0x36 (GPIO44/45), ...
```

问题分析. 传感器的稳压器由固件扩展 GPIO 133（CAM_GPIO0）使能，ARM 只能请 VideoCore 固件代办；固件说“不支持”，传感器就没电，I2C 上自然读不到 0x36。第一行的结果字段可以这样判读：`no reply` 是固件根本没回；`message rejected` 是整条消息被拒；`done 1 len 0` 是标签被应答但没有数据（QEMU 就是这样）；`len 8` 且 `first word 133` 是固件认识这个标签、但认为 GPIO 133 无效。对照 Linux 的 `rpi_firmware_property()` 和 `gpio-raspberrypi-exp.c`，旧的 mailbox 代码有三处不同：标签的请求长度字填了数据长度，Linux 填 0；发送前不清 FIFO，而 SD、USB 驱动各有一份私有 mailbox 代码，它们超时后留下的回复会被当成本次的回复读走；收到回复不核对地址。另外，旧代码在复制数据之前就因为单个标签的 response 位没置而判失败，Linux 并不这样判。最后一列 `stale replies` 就是为验证第二点加的：它统计发送前清掉了多少条别人的回复。

解决方案. 把各处私有代码收拢成带锁的 `kernel/drivers/firmware/mbox.c`，按 Linux 的约定重写：请求长度字填 0；发送前清空 FIFO；只接受地址与本次请求相同的回复；以“整条消息成功”加“返回结构第一个字为 0”判断 GPIO 操作成败。电源控制也按 Linux 的做法：先 `GET_GPIO_CONFIG` 读出并保留极性，再设置，最后 `GET_GPIO_STATE` 回读。修复后的日志是 `ov5647: CAM_GPIO0 direction=output polarity=normal state=1`。如果某个固件确实不处理这个标签，兜底方案是自制 `dt-blob.bin`，让固件启动时就把 p133 置为输出高。

问题 2：BSC0 对 0x36 始终 NACK

日志输出.

```
ov5647: no reply at i2c0 0x36 (GPIO44/45), BSC raw=90000131 flags=131 \
        now=... div=2500 fsel=55 levels=3
 (更早的内核把 raw 打成 -6ffffecf)
```

问题分析. 供电已经正常，BSC0 却在 repeated-START 和 STOP/START 两种读法下都报错。逐个字段看：`flags` 的 0x100 是 ERR（从机没应答），0x200 才是时钟拉伸超时；`div=2500` 是 250 MHz/100 kHz；`fsel=55` 表示 GPIO44/45 都是 ALT1；`levels=3` 表示 SDA、SCL 都是高电平，总线空闲、没有被谁拉住。控制器确实发出了事务，从机确实没应答——但仅凭这一行，还分不清是“传感器不在”还是“控制器这条路不通”。另外两个副产品：真 BCM2837 在 TA（transfer active）仍为 1 时撤掉 I2CEN，状态机可能冻结，QEMU 不模拟这一点；内核 `printf` 的 %x 把最高位为 1 的值当有符号数打印，0x90000131 成了 `-6ffffecf`。

解决方案. 出错时采用两段式中止：先保持 I2CEN、清 FIFO，等 NACK+STOP 让 TA 归零，再关控制器、清粘滞状态位；%x 改为无符号。为了回答“传感器在不在”，增加一个与 BSC 完全无关的 bit-bang 探测（问题 3）。

问题 3：bit-bang 探测时而 ACK、时而 NACK

日志输出.

```
camera: GPIO bitbang probe 0x36=ACK 0x10=NACK 0x1a=NACK        (某次启动，格式)
camera: GPIO bitbang probe 0x36=NACK 0x10=NACK 0x1a=NACK       (同一模块，另一次)
```

问题分析. bit-bang 探测把 GPIO44/45 临时切成开漏 GPIO，只发一个地址字节、读第九个时钟上的 ACK，不读写任何寄存器。它本该给出确定的答案，同一个模块的结果却不稳定，问题出在诊断代码自己：第九个时钟调用的是通用的 `bb_release()`，它释放 SDA 之后会等 SDA 回到高电平——这对数据位是对的，对 ACK 却正好相反，ACK 的定义就是从设备把 SDA 拉低。于是每次都白等 100 µs 超时，再去采样时从设备可能已经松开了 SDA。

解决方案. ACK 周期只把 SDA 切成输入，在 SCL 高电平的中间直接采样。修正以后结果稳定为 `0x36=ACK`。这个结论很有分量：bit-bang ACK 而 BSC NACK，说明传感器、地址、供电、上拉和排线都没问题，故障只在 BSC0 的复用或时序这一段。同时规定无论 bit-bang 结果如何，都继续执行问题 4 的恢复扫描。

问题 4：BSC 四档降速全部失败

日志输出.

```
ov5647: BSC retry 100000 Hz failed status=... now=... div=...    （格式）
ov5647: BSC retry 50000 Hz failed status=...
ov5647: BSC retry 25000 Hz failed status=...
ov5647: BSC retry 10000 Hz failed status=...
```

问题分析. 既然传感器在，就怀疑速率或寄存器写入顺序：依次用 100、50、25、10 kHz 重读芯片 ID，每档都按 Linux 的比例同时更新 `BSC_DIV` 和边沿采样延迟 `BSC_DEL`（不继承固件留下的值），并在写完地址、长度、FIFO 后加 `dsb sy`，保证配置在 START 之前到达控制器。四档全部是 `ERR`，速率和 MMIO 写序都被排除了；剩下的只能是 BSC0 到 GPIO44/45 这条路由本身，或者固件对 BSC0 的占用。继续在这里花时间，收益已经很低。

解决方案. 绕开 BSC0：用 GPIO 完成完整的软件 SCCB 事务（写寄存器地址、STOP、START、读一个字节），读到 `0x5647` 后打印 `using GPIO software SCCB; BSC0 bypassed`，此后传感器的所有读写都走这条路。寄存器表里 `0x0103=1` 会让传感器软件复位、期间不应答，写完这一项等 10 ms；普通寄存器写最多重试三次。软件 SCCB 慢，但只用来写一百来个寄存器，图像走 CSI-2，不受影响。

问题 5：软件 SCCB 偶发读出 0x3631，随后 BUS-STUCK

日志输出.

```
ov5647: software SCCB chip-id=3631 attempt=1                    （格式）
...
camera: full GPIO I2C scan bus stuck at 0x..
```

问题分析. 地址被 ACK 了，读回的却是 `0x3631`——它不是任何传感器的 ID，事务层面成功、数据位却采错了，说明时序处于边缘。旧代码随后的两个动作让情况更糟：每次软件事务前无条件送 9 个 SCL 恢复脉冲；读到错误 ID 后立即扫描整个 7 位地址空间。对一个状态已经不稳的模块，这些都是额外的扰动，扫描最终把总线卡住（BUS-STUCK）。

解决方案. （`gpio-sccb-v7`）软件时钟降到约 25 kHz（半周期约 20 µs）；任何不是 `0x5647` 的完整读数都被拒绝重读（日志 `rejected unstable chip-id=..`）；恢复脉冲只在发现 SDA 被拉低时才送，最多 9 个，然后统一发 STOP；只要 `0x36` 已经完成过寄存器事务，就不再做全地址扫描，而是打印 `address 0x36 responds but chip-id is unstable; power-cycle retry required` 后直接断电重来；上电前 CAM_GPIO0 先拉低 20 ms 让模块放电，置高后等 100 ms 再访问。

问题 6：0x36 ACK，但三次读 ID 都失败

日志输出.

```
ov5647: software SCCB ID read retry 1/3 (probe=ACK)          （格式）
ov5647: software SCCB ID read retry 2/3 (probe=ACK)
ov5647: software SCCB ID read retry 3/3 (probe=ACK)
ov5647: address 0x36 ACK but chip-id read failed; power-cycle retry required (fsel=.. levels=..)
```

问题分析. v7 在真机上仍会出现这种情况。看调用顺序就明白了：每次上电，驱动仍然先用 BSC0 读 ID，失败几次之后才切到 GPIO。问题 4 已经证明这些 BSC 事务注定失败，而它们留下的半截事务会让传感器的 SCCB 接口进入不确定状态，紧接着的软件读取就跟着失败。

解决方案. （gpio-sccb-v8）从第一次读 0x300a/0x300b 起就直接用 25 kHz 软件 SCCB，BSC0 只初始化、不再发事务，日志变为 `probing with 25 kHz GPIO SCCB; BSC0 bypassed` 和 `software SCCB power-on chip-id=5647`。如果 0x36 已 ACK 但读寄存器仍失败，本轮立即断电，交给 `camera_bind()` 做下一次完整的电源周期（用户首次 `read()` 时最多三次，每次间隔 100 ms），不再探测其他地址。7.9.10 小节的日志就是 v8 的样子。

问题 7：开始出流后整机无响应

日志输出.

```
unicam: receiver enabled, starting sensor stream
ov5647: streaming started
unicam: sensor streaming, waiting for frame
（此后再无任何输出，4 秒超时也没有触发）
```

问题分析. 第一次真机出流时打开了设备树给出的 CSI1 中断 GPU IRQ 39。“4 秒超时也没触发”是关键线索：超时是按 xv6 的 `ticks` 计的，而 `ticks` 由 CPU0 的通用定时器中断推进；如果它不走了，说明 CPU0 一直在处理别的中断。IRQ 39 是电平触发的，若这个编号实际不对应 Unicam，或者处理程序清不掉它的源，CPU0 一从中断返回就立刻再次进入，定时器中断永远得不到服务——超时机制本身依赖于它要监督的那条路径，所以失效了。

解决方案. bring-up 阶段屏蔽 IRQ 39，改用与中断无关的 `CNTVCT_EL0` 计 4 秒期限，每 1 ms 轮询一次 Unicam 的 `STA/ISTA`；无论成败都能返回，失败时打印 FS/FE 计数、`STA`、`IBWP`。成功时的日志是 `unicam: frame captured by polling; CSI1 IRQ 39 intentionally masked`—— “intentionally” 说明中断是有意屏蔽的，不是丢了。以后再验证中断时，也只在拍照期间打开，连续 100 次没有发现 Unicam 状态就自动关掉，并始终保留这个有界轮询作兜底。

问题 8：转换期间偶发 SDIO CMD52 failed

日志输出.

```
camshot: calculating white balance and level...
arasan-sdio: request CMD52 failed status=df0001 irq=100 c1=e0807
```

问题分析. 两行挨在一起，很容易得出“用户程序抢占了 Wi-Fi 工作线程”的结论，但这站不住：`camshot` 和 `brcmf0_wq` 都是 `SCHED_OTHER`、nice 0，用户态并不自动低于内核线程，SDIO 中断上半部也随时能打断用户程序。真正的线索在数值里：`status` 的 bit 0 是 `CMD_INHIBIT`，`irq=100` 只有卡中断、没有本次的 `CMD_DONE`——上一个事务还没结束，另一个就开始写寄存器了。原来 `sdio_cmd52()/sdio_cmd53()` 直接调用 Arasan 的

request()，而 Wi-Fi 的状态 work、NAPI 收发和 BCDC 控制请求可能在不同核上同时运行，共用同一组 ARG1/CMDTM/INTERRUPT/DATA 寄存器。

解决方案. 给 struct mmc_host 加睡眠锁 request_lock，所有 CMD52/CMD53 都经 mmc_request_sync() 串行化。锁的粒度是物理控制器而不是某个 function 或某块网卡，对应 Linux MMC core 的 host claim；用睡眠锁是因为一次请求可能要等命令、数据和超时（7.7.7 小节）。启动枚举时还没有进程，这时只有启动 CPU 发请求，直接执行、不取锁。此前为 camshot 加的 sched_yield() 撤回——它掩盖不了并发访问。

问题 9：执行不访存的指令却报数据中止

日志输出.

```
usertrap(): unexpected esr=0x92000004 ec=0x24
            elr=0x0000000000000b40 far=0xf3910003fda9a57b
```

问题分析. 更新 /bin/camshot 后，程序偶尔在第一行输出之前就退出。EC=0x24 是来自 EL0 的数据中止，可用当前 ELF 反汇编，ELR=0xb40 是 strncpy() 里一条纯寄存器的 sub，不可能访存；FAR 又是一个随机指针。CPU 实际执行的并不是磁盘上这个 ELF 的指令。exec() 通过内核高地址把新程序写进物理页，uvmsync_icache() 做了 dc cvau，最后却用 ic iallu——它只作废执行它的那个核的指令缓存。进程在一个核上装好，迁移到另一个核运行，那里的 I-cache 仍命中同一低地址上旧程序的指令：执行的是旧代码，用的却是新程序的数据和栈。

解决方案. 改为广播到 Inner Shareable 域的 ic ialluis，前面的逐行 dc cvau 和 dsb ish 保留（第 4 章故障二）。exec()、uvminit()、uvmcopy() 都经过这个函数。排查这类问题时，要用实际装进 SD 卡的那个 ELF 去反查 ELR，不能用另一次构建的符号文件。

问题 10：输出到 fixed WB/level 后长时间停顿

日志输出.

```
camshot: frame received 640x480, 384040 bytes
camshot: unpacking RAW10...
camshot: sensor test pattern; fixed WB/level
（长时间没有新输出）
```

问题分析. 看起来像转换死循环，但测试图案模式下转换并不做统计，下一步其实是以 O_TRUNC 打开已存在的 /boot/camera.bmp。截断要释放旧文件的簇链，旧的 fat_free_chain() 对每个簇都读一次 FAT、再对两份 FAT 各做一次读改写，覆盖一个 320×240 的 BMP 就要五百多次 SD 扇区事务，而且每次都是 PIO 轮询。

解决方案. fat_free_chain() 缓存当前 FAT 扇区，在其中批量清除属于这条链的项，换扇区时才把整扇区写到每份 FAT（6.6 节）。同时 camshot 在打开输出文件、写 BMP 头和开始转换之前各打印一行阶段日志，BMP 每 16 行写一次，并用版本串 converter fixed-pattern-v2 fat-progress batch16 确认 SD 卡上的程序已经更新——以后再停住，可以直接看出停在哪一步。

问题 11：新建文件停在 directory-entry begin

日志输出.

```
camshot: sensor test pattern; fixed WB/level
camshot: preparing output /boot/cavfs: boot create begin path=/boot/cam02.bmp sub=/cam02.bmp mode=601
m02.bmp (230454 bytes)...
vfs: boot create stat complete exists=0
```

```
vfs: boot create directory-entry begin
（此后再无输出）
```

问题分析。 先排除两个干扰：`/boot/cavfs: ... m02.bmp` 是用户态逐字符输出与内核 `printf` 交错的结果，路径本身没坏（7.9.10 小节）；`exists=0` 说明 `stat` 已经正常返回，阻塞点在 `fat32createfile()` 里。这和问题 10 不是同一个问题：`exists=1` 且停在 `truncate begin` 才是释放旧簇链慢。第一个猜测是“根目录要扩展，簇分配器扫描太慢”，于是先给 `fat_alloc_cluster()` 加了按扇区缓存和 FSInfo 提示，并加了 `fat32: extending root directory` 日志——真机上仍停在同一行，连新日志都没出现，说明根本没走到分配器。

真正的原因是内核栈。每个进程的内核栈只有 4 KiB，旧的 `fat32createfile()` 在栈上放了一个路径匹配结构、22 个目录槽和 256 字节的文件名，又嵌套调用 `fat_find_path()`，后者再放 256 字节文件名、520 字节 LFN 字符数组和 20 个目录槽。光这些数组就接近 3.5 KiB，加上 VFS、系统调用的栈帧和保存的寄存器，越过了 4 KiB。`stat` 只用到其中一层，所以没事。至于为什么是“停住”而不是一条报错：内核栈之间隔着不映射的保护页（第 4 章），越界先落进保护页、触发 EL1 数据中止；而 EL1 的异常入口要在同一个 `SP_EL1` 上压 272 字节保存现场（3.2 节），这一压又落在保护页里，异常入口自己再次中止，CPU 在异常向量里反复进出，串口上什么也不会出现。

解决方案。 把这些大数组移出栈，放进 FAT32 私有的静态工作区 `fat_work`。所有 FAT32 操作本来就持 `fat32_lock` 串行执行，一个工作区就够，不必加大每个进程的内核栈。反汇编确认栈帧降到 `fat32createfile` 464 字节、`fat_find_path` 176 字节。create 路径上加了更细的阶段日志（`create lookup`、`create reserve`、`extending root directory`），可以直接区分是查找、保留目录项还是分配簇出了问题；之前对分配器的优化也保留下来。修复后的完整日志见 7.9.10 小节。

小结。 回头看这十一个问题，有效的做法是一样的：让每一层都有自己的日志，日志里带上能判读的原始数值（`flags`、`fsel`、`levels`、`status`、`esr`、`exists`），然后只把日志实际证明了的东西当作结论。几次走弯路，都是因为把“两行日志挨在一起”当成了因果（问题 8），或者先修了猜测中的原因而没有确认程序是否真的走到那里（问题 11）。

7.9.12 连续取流与 HTTP 推流

7.9.10 小节的日志里，每次 `read()` 都完整地上电、预热、拍一帧、断电，一帧要一秒左右，这样只能拍照，做不了视频。现在的驱动把“上电并开始出流”和“抓一帧”拆开了，在此基础上，用户程序 `camserver` 把画面以 MJPEG 的形式通过 HTTP 推给浏览器。

内核：传感器一直出流。 `camread()` 第一次被调用时执行 `cam_stream_start()`：上电、启动 Unicam 和传感器的 stream、等预热结束，这时 Unicam 停在零长度的 dummy 缓冲上（状态 STREAMING）。第一次取帧时排上第一个槽，此后两个 DMA 槽轮流接收，读者从完成队列里取帧（7.9.8 小节）。传感器始终不停，所以从第二帧起不再有上电和预热的开销。只有 `/dev/video0` 的最后一个引用被关闭时，`camrelease()` 才调用 `cam_stream_stop()` 停流断电。

这带来一条使用约定：连续读帧的程序要一直开着同一个文件描述符。两个进程各自打开、读一帧、关闭，会让传感器反复重启。

用户态：一个进程扇出给所有观看者。 图 7.42 是 `camserver` 第一版的结构：抓帧、编码、发送在同一个循环里串行进行。后来它被拆成了三级流水线，见 7.9.13 小节。

之所以不用“每个连接 fork 一个子进程”的常见写法，有两个原因。第一，xv6 的 TCP 连接归调用 `accept` 的进程所有（内核检查 `c->owner == myproc()->pid`），不能交给子进程。第二，`/dev/video0` 只有一路传感器

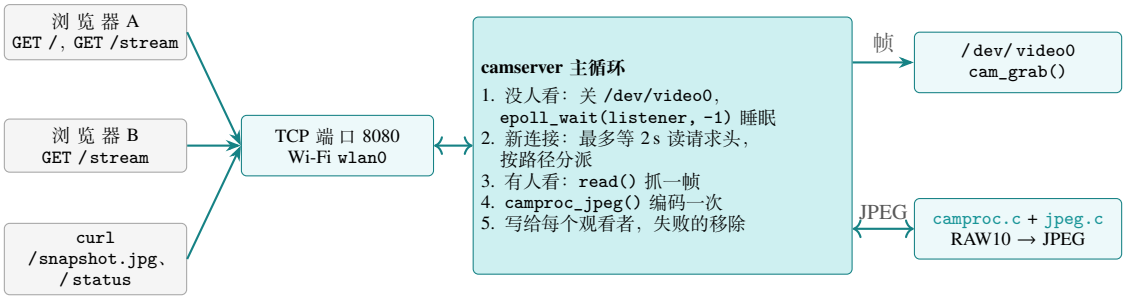


图 7.42: camserver 的结构。一次采集、一次编码，写给全部观看者；没人观看时传感器断电

流，多个进程分别打开会互相停流重启。所以 camserver 是单进程的：每抓一帧只编码一次，再依次写给所有观看者；N 个观看者只花一份采集和编码的代价。缺点是一个很慢的客户端会拖慢所有人：它的 TCP 发送队列满了，write() 就会阻塞。

新连接在帧与帧之间处理。主循环用一个只监视监听 socket 的 epoll：没人观看时 epoll_wait(-1) 睡眠；有人观看时超时为 0，只是看一眼有没有新连接。浏览器经常先建立“预连接”而不发请求，如果直接 read() 请求头，整个推流就会被卡住。因此读请求头之前，先用另一个 epoll 最多等 2 秒，等不到就关闭连接。

接口与 MJPEG 格式。 表 7.24 是服务器提供的路径。

表 7.24: camserver 的 HTTP 接口

路径	内容
/	HTML 页面，内嵌 <img src="/stream">
/stream	MJPEG 流，multipart/x-mixed-replace; boundary=frame
/snapshot.jpg	单帧 JPEG：正在推流时给最新一帧，否则临时上电拍一帧再断电
/status	纯文本：摄像头状态、尺寸、质量、观看人数、帧数、帧率、最近一帧大小
其他	404

MJPEG over HTTP 没有专门的协议，只是利用了 multipart/x-mixed-replace 这种内容类型：响应永远不结束，每一部分是一整幅 JPEG，浏览器收到新的一部分就替换掉上一幅。线上的字节流是这样的：

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace; boundary=frame
Cache-Control: no-cache, no-store, must-revalidate
Connection: close

--frame
Content-Type: image/jpeg
Content-Length: 18143

<18143 字节：FF D8 ... FF D9>
--frame
Content-Type: image/jpeg
Content-Length: 18144
...
```

每一部分都带 Content-Length，客户端不需要在 JPEG 数据里搜索边界串。

测试。 QEMU 的 raspi3b 没有摄像头，它的 USB 网卡在当前版本下也没有被 dwc2 驱动接受，无法端到端测试。因此先在宿主机上测试服务器逻辑：把 socket_listen、socket_accept、epoll_* 和 /dev/video0 换成

POSIX 的实现（假摄像头每 100 ms 产生一帧带移动红条的 640×480 RAW10），同一份 `camserver.c` 不加修改地编译运行：

```
/status            200, camera: idle
/                  200 text/html
/nope              404
静默预连接         2.0 s 后被丢弃，同时到来的 /status 正常返回
/snapshot.jpg      200 image/jpeg，解码为 640x480
5 个并发 /stream   前 4 个各收到 26~28 帧，每帧都能解码；第 5 个 503 too many viewers
观看者全部离开     camserver: no viewers, camera stopped
```

真机上这样运行：

```
$ wifi                  # 读 /etc/wifi.conf，关联并 DHCP，记下 wlan0 的 IP
$ camserver -s          # 先用 320x240；-q 调质量，-n 限制观看人数
camserver: listening on port 8080 (320x240, quality 75, max 4 viewers)
```

然后在同一局域网里用浏览器打开 `http://<IP>:8080/`。串口每 3 秒打印一行 `camserver: N.N fps, B bytes/frame, K viewer(s)`。帧率受三件事限制：等待下一帧的到来、用户态整数去马赛克和 JPEG 编码，以及 Wi-Fi 上的 TCP 吞吐；半尺寸 `-s` 能同时减轻后两项。

7.9.13 提高推流帧率

7.9.12 小节的第一版 `camserver` 在一个循环里依次做三件事：抓一帧、转成 JPEG、写给所有观看者。它正确，但慢：三段时间直接相加。本小节记录后来的提速过程，方法可以归结为四条：先测量，把串行改成流水线，去掉大块拷贝，再按瓶颈减少每帧的工作量。

先测量，再优化。 改动之前先给每一帧计时。`stats_frame()` 分别累加三段耗时：*capture* 是 `read()` 拿到一帧所花的时间，*jpeg* 是 `camproc_jpeg_raw()` 的时间，*send* 是把这一帧写进所有观看者 socket 的时间（被慢客户端阻塞也计在里面）。每 3 秒在串口打印一次平均值，`/status` 也给出同样的数字：

```
camserver: N.N fps, B bytes/frame, K viewer(s), capture/jpeg/send C/J/S ms, payload P kbit/s
```

真机上测得的三段约为 70.8、308.2、199.2 ms。串行时一帧要 70.8 + 308.2 + 199.2 ≈ 578 ms，帧率只有约 1.7 fps。三段里最慢的是 JPEG：640×480 的 RAW10 要在 ARM 上做整数去马赛克、白平衡、gamma 和 DCT 编码，用户程序又是以 `-mcpu=cortex-a53+nofp` 编译的，没有浮点和 NEON。

方法一：三级流水线。 三段工作分别由三个执行流承担：采集线程只负责从 `/dev/video0` 取帧，JPEG 线程只负责转换，主线程负责 HTTP 和发送。它们之间用两个深度为 2 的队列相连。这样相邻的三帧可以在不同的 CPU 核上同时处理：第 n 帧在发送时，第 n+1 帧在编码，第 n+2 帧在采集。稳定后的帧周期从三段之和变成三段中的最大值（图 7.43）：

$$T_{\text{串行}} = t_c + t_j + t_s \approx 578\,\text{ms} \quad\Longrightarrow\quad T_{\text{流水线}} = \max(t_c, t_j, t_s) \approx 308\,\text{ms},$$

理论上限从 1.7 fps 提高到约 3.2 fps；实际值还受调度、内存带宽和 Wi-Fi 波动影响。

为什么用线程而不用进程？xv6 的 TCP 连接记录了接受它的进程号（`c->owner`），已经连上的 socket 不能交给另一个进程。所以接受连接和发送都留在主线程，采集和 JPEG 用 `thread_create()` 创建的用户线程（第 8 章），它们与主线程共享地址空间，可以直接读写同一块缓冲。

方法二：RAW 帧零拷贝。 一帧 RAW10 有 384 KB。第一版用普通 `read()` 把它从内核拷进用户缓冲；现在采集线程只读 40 字节的帧头，JPEG 线程按帧头里的槽号，直接从 `/dev/video0` 的只读 `mmap` 里读像素（7.9.8 小节），

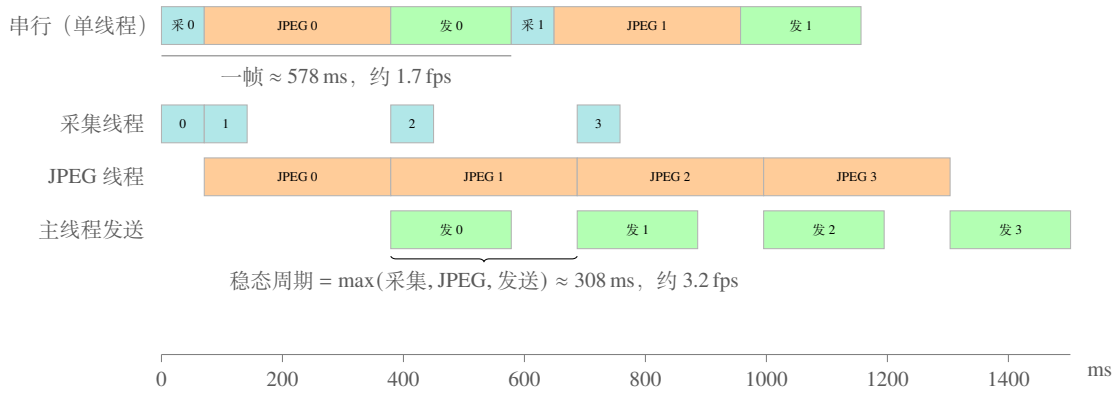


图 7.43: 串行与三级流水线的时间线（按实测的 70.8/308.2/199.2 ms 画，比例尺相同）。流水线中 JPEG 线程几乎一直在工作，帧周期由最慢的这一段决定

每帧省掉一次 384 KB 的拷贝。

零拷贝带来一个所有权问题：内核在同一个文件的下一次 `read()` 时归还上一个槽，如果 JPEG 线程还在读这个槽，像素就可能被 Unicam 的下一帧覆盖。解决办法是打开两个文件描述符，每个各持有一个内核槽，并用信号量 `camera_free[i]` 表示“第 i 个 fd 的槽已经用完”：

```
// 采集线程：轮流使用两个 fd
int ci = sequence++ % PIPE_DEPTH;
sem_wait(&camera_free[ci]);              // 等 JPEG 线程用完这个 fd 上一次的槽
int n = read(fd[ci], &job.hdr, sizeof(job.hdr));   // 只取帧头；内核归还旧槽、给出新槽
job.camera_index = ci;
raw_push(&job);

// JPEG 线程
struct raw_job raw = raw_pop();
const uchar *pixels = camera_dma + raw.hdr.reserved * CAM_SLOT_BYTES;
camproc_jpeg_raw(encoded_buf[bi], JPEG_CAP, &raw.hdr, pixels, half, wb, quality, &out.len);
sem_post(&camera_free[raw.camera_index]);  // 槽用完了，这个 fd 可以再 read
```

JPEG 结果只有几十 KB，主线程把最新一帧复制一份留给 `/snapshot.jpg`；大块的 RAW 数据始终留在 DMA 槽里。

方法三：有界队列与背压。 流水线的各段速度不同，必须有机制防止快的一段把慢的一段淹没。这里全部用计数信号量实现（表 7.25）。

表 7.25: 流水线中的同步对象

对象	初值	含义
`pipeline_start`	0	主线程有了第一个观看者时 `sem_post`，唤醒采集线程打开摄像头
`camera_free[0..1]`	1, 1	每个 camera fd 持有的 DMA 槽是否已被 JPEG 线程用完
`raw_free` / `raw_ready`	2 / 0	RAW 队列的空位 / 待编码的帧
`encoded_free` / `encoded_ready`	2 / 0	JPEG 缓冲的空位 / 待发送的帧
`convert_lock`	互斥锁	`camproc.c` 复用静态的 RAW 解包缓冲，单次快照不能与 JPEG 线程同时转换

两个队列都是单生产者、单消费者的环形缓冲。生产者写好元素后用 release 语义推进尾指针，消费者用 acquire 语义读，因此不需要额外的锁：

```
static void raw_push(const struct raw_job *job) {
  sem_wait(&raw_free);                                  // 背压：队列满就等
```

```
  uint tail = __atomic_load_n(&raw_tail, __ATOMIC_RELAXED);
  rawq[tail % PIPE_DEPTH] = *job;
  __atomic_store_n(&raw_tail, tail + 1, __ATOMIC_RELEASE);   // 先写元素，再发布
  sem_post(&raw_ready);
}
```

如果发送慢了（例如某个观看者网速很差，`write()` 阻塞），`encoded_free` 很快耗尽，JPEG 线程停在 `sem_wait()` 上；它不再归还 `camera_free`，采集线程也随之停下；内核那边两个槽都被持有，Unicam 自动把多余的帧写进 dummy 丢弃。慢的一端就这样一级一级地把压力传回传感器，而不会让内存无限增长。

观看者全部离开时，主线程把 `pipeline_run` 清零，采集线程退出内层循环，等两个 `camera_free` 都收回之后才关闭两个 fd（这时 JPEG 线程已不再读映射），最后一个引用关闭使内核停流断电。每次重新开始推流时 `pipeline_generation` 加 1，主线程丢弃上一代残留在队列里的帧。

方法四：减少每帧的工作量。 流水线只能把周期压到最慢的一段；要再快，就得让最慢的那段少干活。现有的两个开关作用在不同的阶段：

- `-s`（320×240）：每个 2×2 Bayer 块直接合成一个像素，省掉双线性去马赛克，输出像素减少 75%，同时减轻 JPEG 和发送两段；
- `-q 60`：降低 JPEG 质量，量化更粗，码流更短，主要减轻发送一段，对编码时间影响较小。

网络推流建议从 `camserver -s -q 60` 开始。

按测量结果定位瓶颈。 保持一个观看者，等至少 3 秒后读 `/status` 或看串口的那一行：`jpeg` 最大，瓶颈在 ARM 上的图像处理，先试 `-s`；`capture` 最大，检查 CSI/DMA 是否正常、帧是否靠 watchdog 而不是中断取得；`send` 最大，检查 Wi-Fi 信号、TCP 发送队列，或是否有一个慢客户端拖住了所有人。

还可以继续做的。 下面几项目前没有实现，按预期收益排列：

- **把 JPEG 拆到多个核上**：编码是按 16 行的 MCU 行进行的，可以把一帧分成上下两半交给两个 JPEG 线程各自编码，再把两段熵编码结果用 restart marker（`RST`n）拼接；Pi 3 有四个核，而现在只用上了三个执行流；
- **用查表和定点优化热点**：去马赛克和色彩转换对每个像素都做多次乘除，可以把 gamma、白平衡增益合并进一张查表，色彩空间转换改用移位和加法；
- **让发送不阻塞流水线**：给每个观看者独立的发送队列，慢客户端只丢自己的帧，不再拖慢别人；
- **可缓存的 DMA 映射**：现在为了正确性用 `Normal-NC`，CPU 读 384 KB 像素时每次都要访问 DRAM。改成可缓存映射可以加快 JPEG 一段，但必须在每个槽交还给用户前严格作废对应的缓存行（7.9.8 小节）。

7.9.14 当前状态

真机（Pi 3B+，Camera v1.3）上，v8 驱动从第一次访问起就用 25 kHz 软件 SCCB，稳定读到芯片 ID `0x5647` 并绑定到 `/dev/video0`；Unicam 以轮询方式拿到完整帧（单缓冲版本），`camshot -t 1 -s -o /boot/cam02.bmp` 能把彩条写成 BMP（7.9.10 小节）。驱动已经改为连续出流、DMA 双缓冲、以 CSI1 中断通知帧完成，并支持零拷贝 `mmap`（7.9.8 小节）；`camshot -v` 可以连续录成一串 JPEG，`camserver` 可以通过 HTTP 推送 MJPEG（7.9.12 小节）。还没有完成的工作有：

- 在真机上确认 CSI1 中断路径（日志 `frame captured by CSI1 IRQ 39`），watchdog 轮询保留为兜底；
- 查清 BSC0 硬件读不到芯片 ID 的原因（控制器路由还是固件占用），以便退回硬件 I2C；
- 更多模式：Linux 的 1296×972 等模式每帧超过 1.5 MB，需要扩大 `CAMDMA`，桥接驱动不用改；目前也还没有选择模式的接口；

- 只有两个槽，没有 `poll()`，也没有手动曝光；`camserver` 已改为三级流水线，JPEG 编码（约 308 ms/帧）是目前的瓶颈（7.9.13 小节）；
- `arasan_sdio.c`、`dwc2.c`、`sd.c` 里仍各有一份私有 mailbox 代码，应统一到 `mbox.c`；
- `config.txt` 不能打开固件的摄像头支持（`start_x=1`、`camera_auto_detect=1`），否则固件会占用同一个传感器和 I2C。

注 摄像头驱动是全书最能体现“分层”价值的例子：mailbox、I2C、时钟、电源域、CSI-2 接收器、DMA、帧状态机、字符设备、用户态图像处理，每一层都可以单独验证。真机调试也正是这样一层层推进的：先看 mailbox 回读，再用 bit-bang 判断 I2C 故障在控制器内外，再用传感器内部彩条（`camshot -t 1`）验证 CSI-2 和 DMA，最后才看真实画面。

7.10 Windows 风格窗口系统：win32k、消息与 GDI

前面的驱动各自提供一类设备接口，而图形桌面要解决的是另一种问题：键盘、鼠标和 framebuffer 都是全局资源，多个进程却必须各自拥有窗口，并且不能直接覆盖别人的画面。工程中的 `kernel/drivers/video/win32k.c` 与 `user/win32.c` 借用 Windows NT 的分层思想，构造了一套最小的消息驱动窗口系统。

它不是 Windows 二进制兼容层。这里的“Windows 风格”是指三条机制相似：应用用窗口过程处理消息；USER32/GDI32 风格的用户库负责控件与绘图 API；内核 win32k 负责跨进程共享且必须可信的窗口对象、输入路由和屏幕合成。完整的源码说明另见 `doc/readme/readme_win32k.md`。

7.10.1 边界：哪些对象在用户态，哪些必须进内核

图 7.44 给出了对象所有权。窗口应用链接 `user/win32.o`，调用 `CreateWindow`、`GetMessage`、`DispatchMessage` 和 `TextOut` 等 API；用户库再通过一个多路复用的 `win32k(op,a1,a2,a3)` 系统调用进入内核。

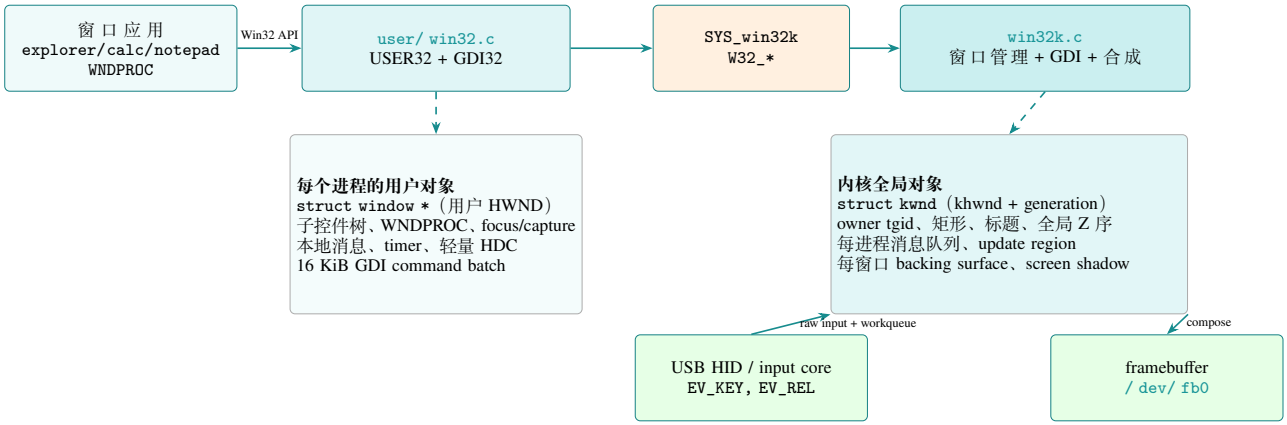


图 7.44: xv6 Windows 风格窗口系统的软件分层与对象所有权。只有顶层窗口进入内核；子控件和窗口过程留在所属进程

用户态对象. `user/win32.c` 的 `HWND` 是指向 `struct window` 的本进程指针。该对象保存窗口类、`WNDPROC`、文字、样式、父子链、控件私有数据、焦点等。`BUTTON`、`STATIC`、`EDIT`、`LISTBOX` 和弹出菜单都是用户态子窗口，不占用内核窗口槽。

内核对象. 每个顶层窗口对应一个 `struct kwnd`，记录 owner tgid、屏幕矩形、标题、全局 Z 序、无效区域以及一块内核 backing surface。内核 `HWND`（用户结构中的 `khwnd`）不是指针，而是

$$khwnd = (generation \ll 8) \mid (slot + 1).$$

低 8 位定位 `wins[MAXW]`，高位 generation 由 `index_of()` 校验；槽被复用后，旧句柄不会错误地命中新窗口。

只有顶层窗口进入内核，因为其位置、Z 序、激活状态和屏幕内容会影响别的进程；子控件只影响一个进程自己的客户区，由 USER32 在进程内管理即可。应用也不直接映射或写 /dev/fb0：它只能向自己的 backing surface 提交受检查的绘图命令，最终画面由内核合成。

真实 Windows 中，USER32.DLL 提供窗口消息 API，GDI32.DLL 提供绘图 API，win32k.sys 实现内核 USER/GDI 服务，现代桌面再由独立的 DWM 进程合成。本工程把 USER32 与 GDI32 合并在 user/win32.c，又把简化合成器放进 win32k.c；这是教学化简，不是对 Windows ABI 的复刻。

7.10.2 创建窗口与系统调用服务表

用户态 CreateWindow() 先从 windows[MAXWIN] 取得一个对象，再同步发送 WM_CREATE。应用通常在这里建立按钮、编辑框等子控件。只有 WM_CREATE 成功后，顶层窗口才构造 struct w32_create 并调用 W32_CREATE。这样第一条 WM_PAINT 到来时，整棵子控件树已经存在。

内核 sys_create() 分配 kwnd 和 $w \times h \times 4$ 字节 surface，建立调用进程的消息队列，把窗口插入 Z 序并返回 khwnd。所有窗口服务共用一个 syscall 入口，sys_win32k() 根据操作号分派：

表 7.26: SYS_win32k 的主要操作

类别	操作
生命周期	W32_CREATE、DESTROY、SHOW、MOVE
属性与 Z 序	SETTEXT、GETTEXT、GETINFO、ACTIVATE、ENUM
消息	GETMSG、POST、QUIT、SETSHELL
重绘	INVALIDATE、BEGINPAINT、ENDPAINT
GDI	GDI、BLIT、SCREEN

销毁、移动、改标题和绘制等操作都要满足 kwnd.owner == myproc()->tgid。进程退出时，win32k_exit(tgid) 清除其窗口、队列和 surface，防止悬挂的全局对象。

7.10.3 消息驱动：等待，而不是轮询

窗口程序的主循环通常只有三行：

```
MSG msg;
while(GetMessage(&msg, 0, 0, 0))
  DispatchMessage(&msg);
```

DispatchMessage() 根据 msg.hwnd 直接调用目标窗口的 WNDPROC。因此窗口过程是一个事件分派函数：WM_CREATE 初始化，WM_COMMAND 响应按钮，WM_CHAR 处理字符，WM_PAINT 绘图，WM_DESTROY 结束程序。

内核为每个 tgid 建立一个 struct kq，其中是 256 项环形队列。post_to() 在队列自旋锁 qlk 内写入消息，再把 q->kick 加一并 wakeup(q)；连续的 MOUSEMOVE 和 SHELLHOOK 会合并，避免慢应用被过时的移动消息淹没。sys_getmsg() 的顺序是：

1. 队列有消息，copyout() 一项到用户态；
2. 队列为空、但某窗口有 update region，合成一条低优先级 WM_PAINT；
3. 非阻塞或超时，返回 2；
4. 否则记下 q->kick，释放窗口锁 wm，在 qlk 下确认计数未变、队列仍空后 sleep(q, &qlk)，等待新消息、无效区域或 delayed work 定时器唤醒。计数保证“放锁之后、睡眠之前”到达的事件不会丢失唤醒。

所以“消息驱动”并不等于 CPU 不停检查消息。没有输入和重绘任务时，GUI 进程睡在内核队列上，不消耗时间片。

异步与同步。 PostMessage() 是异步的：顶层消息经 W32_POST 放入内核队列，子控件消息放入用户态本地队列。SendMessage() 是同步的，当前实现直接调用 w->proc(...)；它只适用于本进程对象，尚未实现 Windows

的跨进程同步消息。

7.10.4 输入如何找到 WNDPROC

图 7.45 的上半段是输入调用链。USB 键盘或鼠标把 `EV_KEY/EV_REL` 交给 input core；`win32k_input_event()` 当时可能处在持锁的输入回调中，所以只写 raw ring 并 `queue_work()`。真正的窗口逻辑由 `input_work_fn()` 在可调度上下文中执行。

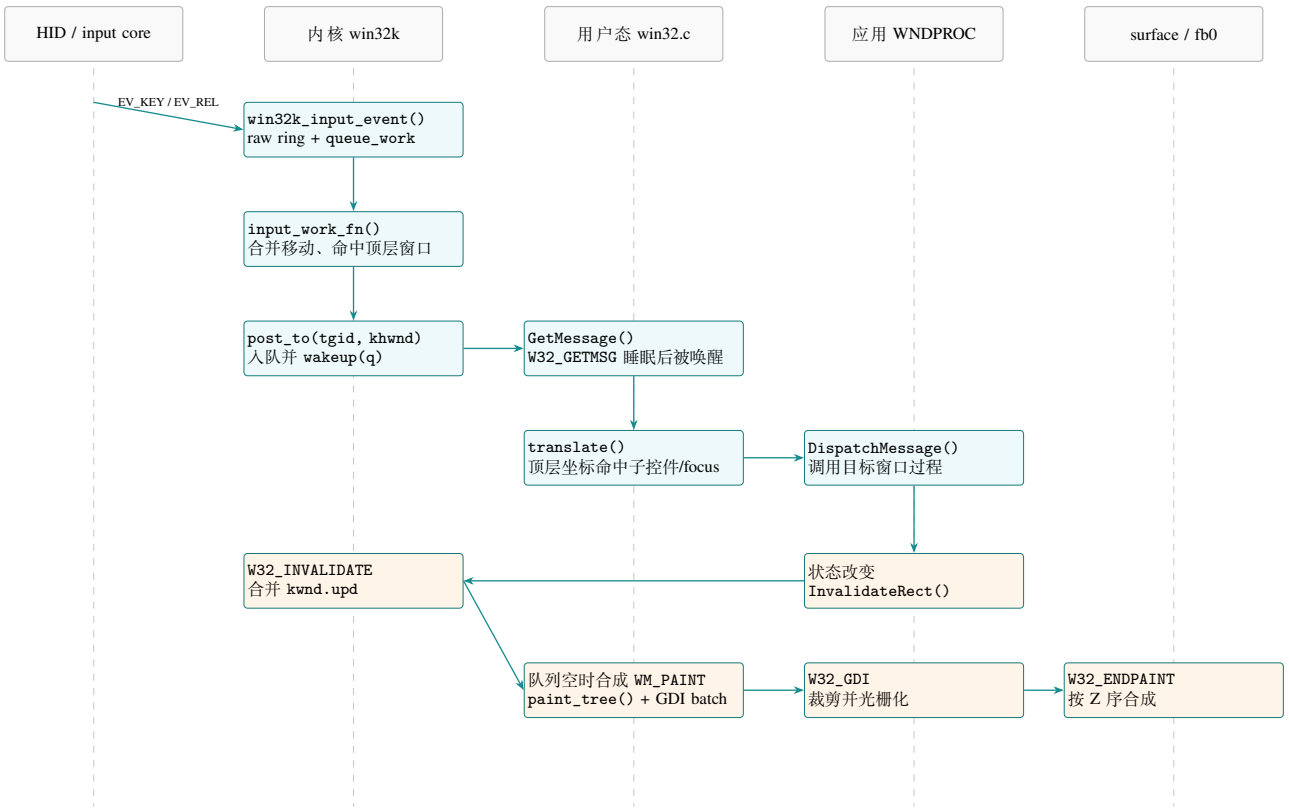


图 7.45: 上半：原始输入经内核顶层命中和用户态子控件路由成为 WNDPROC 调用；下半：状态变化经无效区域、`WM_PAINT` 和 GDI 批处理成为屏幕像素

内核从全局 Z 序顶端向下执行 `window_at()`，把鼠标送给命中的顶层窗口；键盘送给 active window。消息到达 `user/win32.c` 后，`translate()` 再用 `deepest_child()`、focus 和 capture 选择进程内的具体子控件。这是两级路由：

```
全局、跨进程：屏幕坐标 -> 顶层 khwnd -> owner tgid      （内核）
进程内：          顶层 HWND -> child/focus/capture -> WNDPROC（USER32）
```

标题栏拖动、最小化/关闭按钮、弹出菜单的外部点击取消和鼠标光标属于非客户区行为，直接由内核处理。应用因此不能通过伪造本地子控件改变别的进程的 Z 序或输入归属。

7.10.5 无效区域与 WM_PAINT

`InvalidateRect()` 不会立即绘图。用户库把子控件坐标换算成顶层窗口坐标，然后执行 `W32_INVALIDATE`；内核的 `runion()` 将矩形并入 `kwnd.upd`。只有普通消息处理完、队列为空时，`sys_getmsg()` 才合成 `WM_PAINT`，所以输入比重绘优先。

收到它后，`paint_toplevel()` 完成一次绘制事务：

1. `W32_BEGINPAINT` 返回顶层矩形、update rectangle 和 active 状态；内核把 `upd` 转移到 `painting` 并清空待更新区；

2. `paint_tree()` 先画窗口框架和客户区，再递归遍历可见子控件，并给各个 WNDPROC 发送 `WM_PAINT`；
3. GDI 原语进入批缓冲，批满或遍历结束时用 `W32_GDI` 提交；
4. `W32_ENDPAINT` 将刚刚绘制的区域按 Z 序合成到屏幕；窗口第一次绘制时合成整个 surface。

每个窗口保留完整 backing surface。因此窗口移动或从遮挡下重新露出时，内核通常只需重新合成已有像素，不必再次让应用 `WM_PAINT`。这与保留式桌面 surface 的思想相似。

7.10.6 GDI：HDC、批处理、光栅化与合成

用户态 `struct dc` 是轻量绘图状态：目标 HWND、客户区原点、clip rectangle、文字色、背景色和背景模式。它不是 framebuffer 指针。`FillRect`、`FrameRect`、`TextOut`、`DrawText` 和 `DrawEdge` 把坐标转换到顶层窗口空间，再向 16 KiB `batch[]` 追加 `w32_gdicmd`：

```
struct w32_gdicmd {
  unsigned short op, len;
  int x0, y0, x1, y1;
  uint32 color;
  int clip_left, clip_top, clip_right, clip_bottom;
  // GDI_TEXT 后面紧跟 4 字节对齐的文字
};
```

内核目前实现两条批命令：`GDI_FILL` 填矩形，`GDI_TEXT` 用固定 8×16 字体画字；`SetDIBitsToDevice()` 则先 flush 命令，再用 `W32_BLIT` 复制 COLORREF 像素块。

`sys_gdi()` 将批限制在 32768 字节，`copyin()` 后检查句柄 generation 和 owner，再把每条命令同时裁剪到 command clip 与 surface 边界。Win32 的 `COLORREF` 是 `0x00BBGGRR`，`gdi_pixel()` 根据 framebuffer 通道顺序转换后才写 surface。批处理既减少 syscall 次数，也阻止用户坐标或指针越过自己的窗口。

全局 `z[]` 按从底到顶记录窗口。`compose()` 采用 painter's algorithm：先填桌面，再逐层复制可见 surface 到缓存的 shadow screen，最后把损坏矩形一次写入非缓存 framebuffer 并重画光标。`window_to_screen()` 会尽量只复制没有被更高窗口遮住的片段；碎片过多时退回完整 `compose()`。shadow 还能在擦除内核鼠标光标时恢复其下方像素。

7.10.7 并发：锁的划分与前后缓冲

最初的实现用一把自旋锁保护全部窗口状态，合成也在锁内进行。xv6 的 `acquire()` 会关中断，而写非缓存 framebuffer 的大面积合成要数毫秒；在 Pi 3B 上，经板载 hub 的 TT 以 split 事务轮询的 500 Hz 鼠标因此不断错过 CSPLIT 时间窗口，拖动一阵后光标完全停住。现在的划分是：

表 7.27: win32k 的三把锁

锁	类型	保护对象与理由
`wm`	可睡眠 mutex	窗口表、Z 序、激活/捕获/拖动状态、合成与 shadow；合成在开中断下进行
`qlk`	自旋锁	各进程消息环与 `kick`；`GetMessage` 在其上 `sleep()`
`rawlk`	自旋锁	原始输入环；只在持设备锁的 input 回调里入队

加锁顺序固定为 `wm`→`qlk`。输入 worker 先处理完一整批原始事件，再执行一次鼠标移动；拖动时新旧矩形重叠就合并为一次合成，避免高回报率鼠标触发每秒数百次合成。

另一个只在多核真机上出现的问题是闪烁：GDI 命令原先直接写合成器读取的 surface，camview 每帧“先填背景、再贴图”，另一 CPU 上的合成若恰好落在两步之间，就把半成品显示出来。因此每个顶层窗口有前后两块 surface：

```
W32_GDI / W32_BLIT  ->  back          (绘制中，合成器看不到)
```

```
W32_ENDPAINT:  back[painting] -> surf   (front)  -> window_to_screen()
compose() / window_to_screen() 只读 surf
```

W32_GDI/W32_BLIT 光栅化时不持 wm：只有所有者会画、也只有所有者（或其退出）会释放 surface，而它画的是后缓冲。这些问题的现象、定位与验证记录在 doc/readme/readme_bug_fix.md。

7.10.8 实例与局限

以 user/calc.c 为例：WinInit() 注册内建控件；应用注册 CalcProc；WM_CREATE 创建显示框和 16 个按钮；主线程睡在 GetMessage()；一次鼠标点击经过 input core、内核顶层命中和用户态 child hit test 到达 ButtonProc；按钮同步给父窗口发送 WM_COMMAND；CalcProc 更新文字并使控件失效；最后低优先级 WM_PAINT 生成 GDI 批、更新 surface 并合成画面。这样就贯通了

输入 → 消息队列 → WNDPROC → 无效区域 → GDI 批 → surface → 屏幕.

当前实现仍有明确边界：顶层窗口、消息队列和用户窗口都是固定池；消息队列按 tgid 而不是每个 GUI 线程；本地或内核环形队列满时会丢消息；GDI 没有完整的 pen、brush、font、region 与设备对象表；每个窗口占前后两块 surface；surface 与合成都在 CPU/内核中，没有 GPU 加速和 alpha；合成器也没有像现代 Windows 那样独立成 DWM 进程。这些限制不妨碍它展示消息窗口系统最核心的控制流与权限边界。

本章的驱动最终都服务于用户程序：sh 从 TTY 读命令，sshd 经 socket 收发，camserver 用 mmap 取帧，窗口应用则通过 USER32/GDI 风格库使用 input 与 framebuffer。下一章回到用户态，看这些程序怎样经系统调用、线程和 libc 用上内核提供的接口。

第八章 用户进程、线程与 libc

本章内容

- 用户程序的形态与系统调用网关
- 从 init 到 shell
- 用户程序一览
- 用户线程：clone、texit、tjoin；Linux 风格 clone(flags) 与 vfork
- libc 与 POSIX 源码兼容层
- 在 xv6 里编译 C 程序
- 一个大型用户程序：sshd

前面几章一直站在内核这边：第 5 章造出第一个进程，第 6 章给它挂上根文件系统，第 7 章把终端、网络和摄像头做成了设备文件。本章换到用户态这一边来看：程序怎样进入内核、init 怎样把系统带到登录提示符、线程和 libc 怎样建立在 clone 和 mmap 之上，最后用 sshd 这个最大的用户程序把各层串起来。

8.1 用户程序的形态

8.1.1 为什么需要系统调用

进程隔离的第一个条件，是把所有用户程序放到特权最低的 EL0：在这一级，程序不能修改页表基址、不能屏蔽中断、不能访问没有映射给它的内存，也就无法改写内核为隔离所做的任何设置。否则，其他一切隔离手段都没有意义。

可是用户程序总要打印字符、读文件、发网络包，而这些都需要直接操作设备。我们不能让每个程序直接去写 UART 寄存器，而是由内核提供一组受控的入口——系统调用。它不能是普通的函数调用，因为每次调用都要把执行级别从 EL0 提升到 EL1，而提升级别只有一条路：产生异常。所以每个系统调用本质上都是一次同步异常：用户程序准备好参数，执行 svc 指令；硬件把它带到 EL1 的异常向量；内核检查参数、完成请求，然后执行普通的异常返回，从 svc 的下一条指令继续。受控入口是隔离的第二个条件；第三个条件——每个进程有自己独立的内存视图——由页表完成（第 4 章）。

8.1.2 链接与装载

xv6 原版把用户程序和用户库静态链接成一个 ELF64 文件，链接在虚拟地址 0，入口是 main。本项目在 make DYNAMIC_USER=0 时仍退回这一形态（默认形态改为动态链接，见第 8.5.2 节）：

```
_%: %.o $(ULIB)
    $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
```

-N 让代码段和数据段放在同一个可读写可执行的段里，exec() 只需处理一个 PT_LOAD，也没有 C 运行时启动代码：exec() 把 argc、argv 放进 x0、x1，把 PC 设为入口地址，main 返回后由用户库调用 exit()——实际上 xv6 的程序习惯在 main 末尾显式调用 exit(0)。这一形态简单，但代价是每个程序都各自携带一份 libc 的代码；把用户库改成共享的 libc.so、让程序默认以位置无关（PIE）ET_DYN 形式动态链接它，就是第 8.5.2 节的内容。

8.1.3 系统调用网关

user/usys.pl 只生成一个私有陷入网关 __syscall，而不是每个系统调用一条桩。它把系统调用号从第一个参数 x0 挪进 x7，其余参数整体下移一格，然后陷入 EL1：

```
.global __syscall
```

```
__syscall:
 mov x7, x0      // 第 0 个实参是系统调用号
 mov x0, x1      // 真正的参数 x1..x6 下移成 x0..x5
 mov x1, x2
 mov x2, x3
 mov x3, x4
 mov x4, x5
 mov x5, x6
 svc #0          // 陷入 EL1
 ret             // 返回值已在 x0
```

user/libc_syscall.c 里的公开 libc 函数（fork、read、write、exec……）都是对这个网关的薄包装：一个 sc() 帮助函数把 SYS_* 号作为第一个实参传给 __syscall，再用 SC0/SC1/…/SC6 按参数个数展开（I/P 只做到 long 的转换）：

```
extern long __syscall(long, long, long, long, long, long, long);

static long sc(long number, long a0, long a1, long a2, long a3, long a4, long a5){
  long result = __syscall(number, a0, a1, a2, a3, a4, a5);
  if(result < 0)                  // 内核目前只返回 -1，尚无 Linux 式 -errno
    errno = EIO;
  return result;
}

#define SC3(n,a,b,c) sc(SYS_##n, (a), (b), (c), 0, 0, 0)
int write(int fd, const void *buf, int count)
{ return SC3(write, I(fd), P(buf), I(count)); }
```

这样分层有两个好处。其一，svc 只出现在这一个网关里，应用和大部分 libc 都不再直接执行陷入指令，系统调用号这套内核 ABI 被收进 libc.so 内部。其二，在动态链接下（第 8.5.2 节），应用对 write 的调用经 PLT 落到共享 libc，再由 libc 内部经 __syscall 陷入，用户程序镜像里不再各自携带一份桩代码。

内核侧 syscall() 用 trapframe->x7 索引函数表，返回值写回 trapframe->x0。目前共有 70 多个系统调用：除了 xv6 原有的 21 个，还有同步对象、网络（UDP、ICMP、TCP socket、epoll）、挂载、TTY 会话、线程、调度属性、高精度时钟等。

笔记 系统调用号放在 x7，这是本项目沿用的约定；Linux AArch64 用的是 x8。所以 Linux 的二进制或汇编系统调用包装不能直接拿来用，第 8.5 节的 POSIX 层是源码兼容，不是二进制兼容。

8.1.4 内核如何分发系统调用

svc 进入 el0sync，usavereg 把全部用户寄存器存进 trapframe，usertrap() 检查 ESR_EL1 的异常类别是否为 0x15（来自 AArch64 的 SVC），然后打开中断、调用 syscall()（第 3.2 节）。分发函数在 kernel/core/syscall.c 中：

```
void syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->x7;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
```

```
    p->trapframe->x0 = syscalls[num]();
  } else {
    printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->x0 = -1;
  }
}
```

`syscalls[]` 是一张以系统调用号为下标的函数指针表。处理函数先检查调用号是否越界，越界就返回 −1，而不是像教学内核那样直接停机——一个用户程序传错调用号，不应该让整个系统死掉。

参数不需要搬运：用户态执行 `svc` 时它们就在 `x0` ~ `x5` 中，现在原样躺在 trapframe 里，各个 `sys_*` 函数用 `argint()`/`argaddr()`/`argstr()` 按编号取出：

```
static uint64 argraw(int n)
{
  struct proc *p = myproc();
  switch (n) {
  case 0: return p->trapframe->x0;
  case 1: return p->trapframe->x1;
  ...
  case 5: return p->trapframe->x5;
  }
  panic("argraw");
  return -1;
}
```

指针参数只取出数值，不在这里检查合法性；真正访问用户内存时由 `copyin()`/`copyout()` 查用户页表，非法地址会使系统调用返回 −1，而不会让内核出错。

返回值写回 `p->trapframe->x0`，而不是直接放进 `x0` 寄存器。原因是返回用户态时 `urestorereg` 会从 trapframe 恢复所有寄存器，直接写寄存器的值会被覆盖；只有改写保存的那一份，用户程序在 `svc` 之后才能在 `x0` 中看到它。

8.1.5 两个栈指针与返回级别

EL0 和 EL1 各有自己的栈指针。从用户态陷入时，硬件立刻把 `sp` 换成 `SP_EL1`（内核栈），用户栈指针原样留在 `SP_EL0` 中。即使内核从不使用 `SP_EL0`，也必须把它连同其他寄存器一起保存和恢复：陷入期间可能发生进程切换，下一个返回用户态的进程需要的是它自己的用户栈指针。所以 `usavereg` 把 `SP_EL0` 存进 trapframe 的 `sp` 字段，`urestorereg` 把它写回。

内核态的 `savereg` 则不需要恢复 `sp`：内核栈是同一个，即使中间发生了进程切换，`swtch()` 也已经把 `sp` 换成了正确的值。

`eret` 回到哪一级，由 `SPSR_EL1` 的模式字段决定，不需要另外指定，所以总是回到异常发生时所在的那一级。利用这一点，内核可以“伪造”一次从未发生过的异常来进入用户态：`userinit()` 在第一个进程的 trapframe 中写入 `elr = 0`、`spsr = 0`（模式 EL0t）、`sp = PGSIZE`，`usertrapret()` 一执行 `eret`，CPU 就第一次来到 EL0（第 5 章）。启动时从 EL2 降到 EL1 用的也是同一个技巧（第 3 章）。

8.2 从 init 到 shell

第一个用户进程 `initcode` 执行 `exec("/init")`（第 5 章）。`user/init.c` 做的事情相当于一个嵌入式 Linux 的启动脚本：

1. 打开 /dev/ttyS0 作为 fd 0，复制为 fd 1、2。/dev 是内核在 prepare_namespace() 里挂好的 devtmpfs，每个驱动注册时都已在其中建好节点（6.8.10 节）；只有在旧内核或根上没有 /dev 的情况下，init 才退回去用 mknod 在根盘上建静态节点；
2. 目录骨架（/proc、/sys、/boot、/etc、/bin 等）在构建根镜像时就由工具建好，init 不改动持久的目录布局，只在文件缺失时生成 /etc/hostname、/etc/passwd（默认账号 root）、/etc/rc、/etc/fstab 和 /etc/wifi.conf；
3. 按 /etc/fstab 逐行调用 mount()：默认是 /proc（procfs）、/sys（sysfs）、/boot（FAT32）和 /mnt/net（netfs）；
4. 如果存在 /etc/wifi.conf，先运行 /bin/wifi 连接无线网络；
5. 在一个循环里 fork 并 exec /bin/login；login 退出后回收它，再启动一个新的登录提示；同时回收所有过继来的孤儿进程。

login 校验密码后调用 tty_attach() 建立会话，切到 passwd 中指定的家目录，exec 指定的 shell。sh 在 xv6 原版基础上增加了：读取 /etc/rc、PATH 搜索（找不到时再回退到根目录，作为存储异常时的恢复路径）、export、exit/logout 内建命令，以及把外部命令设为前台进程组的作业控制。

8.3 用户程序一览

登录之后能用的命令都编译进了 UPROGS（表 8.1），构建时装进根镜像的 /bin。

表 8.1: 用户程序（UPROGS）

类别	程序
基本工具	sh、init、login、ls、cat、echo、grep、wc、mkdir、rm、ln、mv、touch、kill、ps、edit、file
文件系统	ext2ls、ext2cat、stressfs
网络	wifi、dhcp、ping、netdns、nettest、tftp、tcpd、epollserver、sshd
调度与同步	chrt、taskset、nice、renice、rtlat、spin、syncdemo、prodcons
线程与 POSIX	threadtest、posixtest
设备	camshot、ptytest、vmmap
开发	tcc
测试	usertests、forktest、grind、zombie

几个调度工具的用法与 Linux 相同，它们都是“先修改自己的调度属性，再 exec 目标程序”，利用了 fork/exec 对调度属性的继承：

```
chrt -f 80 rtlat          # 以 SCHED_FIFO 优先级 80 运行 rtlat
taskset 0x8 spin &        # 只允许在 CPU3 上运行
nice -n 10 grind          # 降低普通进程的时间片
chrt -f -p 60 6           # 修改 PID 6（某个 kworker）的策略
```

8.4 用户线程

8.4.1 设计

原来的模型是“一个 struct proc 对应一个独立的地址空间”。加入线程时，项目没有引入新的“线程控制块”，而是让每个线程仍然是一个完整的 struct proc——有自己的内核栈、trapframe、调度属性、TID 和 CPU 亲和性——只是同一线程组的成员共享同一个引用计数的 struct vmspace（图 8.1），以及描述符表 files_struct 和当前目录 fs_struct（第 5.5 节）。所以和 pthread 一样，一个线程打开的文件其他线程可以直接用，一个线程 close(fd) 后其他线程也看不到这个 fd 了。

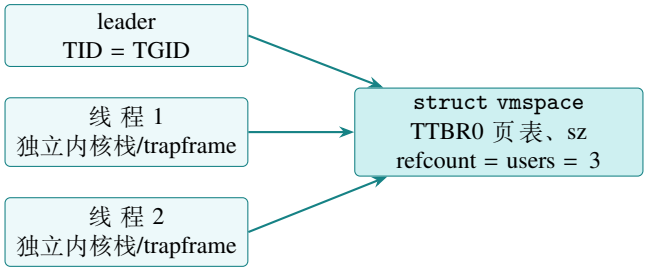


图 8.1: 线程组共享 vmspace

调度器切换到任何一个线程时，`switchuvm()` 都写入 `p->vm->pagetable`，所以同组线程看到相同的代码、全局变量、堆和彼此的栈；而它们的内核栈和 trapframe 各自独立，可以同时在多个 Cortex-A53 核上运行。

8.4.2 系统调用

表 8.2: 线程相关系统调用

调用	作用
`clone(entry, arg, stack_top)`	线程库专用（`SYS_clone`，libc 里叫 `thread_clone()`）：创建同一线程组的新调度实体，共享地址空间、fd 表和 cwd；`x0=arg`，`SP=stack_top`，`ELR=entry`
`clone(fn, stack, flags, arg, ...)`	glibc 形式（`SYS_clone_flags`），标志取 Linux 的值，见第 5.5 节
`vfork()`	`CLONE_VM|CLONE_VFORK|SIGCHLD`：父进程睡到子进程 exec 或退出
`texit(status)`	只退出调用线程；leader 调用时等同于整个进程退出
`tjoin(tid, &status)`	等待并回收同组中的指定线程（0 表示任意）
`gettid()` / `getpid()`	返回线程 ID / 线程组 ID

用户库在其上提供更安全的包装，它负责分配 16 字节对齐的 16 KiB 用户栈，并保证线程函数返回后自动调用 `texit()`：

```
static void thread_boot(void *opaque) {
  struct thread_start start = *(struct thread_start *)opaque;
  free(opaque);                          // 启动记录用完即释放
  start.fn(start.arg);
  texit(0);                              // 线程函数返回后自动退出
}

int thread_create(thread_t *thread, void (*fn)(void *), void *arg) {
  thread->stack = malloc(THREAD_STACK_SIZE + 16);
  struct thread_start *start = malloc(sizeof(*start));
  start->fn = fn;  start->arg = arg;
  uint64 top = ((uint64)thread->stack + THREAD_STACK_SIZE + 16) & ~15ULL;
  thread->tid = thread_clone(thread_boot, start, (void *)top);
  ...
}
```

用户栈要等 `thread_join()` 中内核回收了线程之后才释放，否则线程可能还在用这块栈。

8.4.3 clone 为什么“返回两次”

`fork()` 会返回两次：父进程中返回子进程的 pid，子进程中返回 0，因为子进程的 trapframe 是父进程的完整拷贝，只把 x0 改成了 0。Linux 的 `clone` 系统调用语义相同，所以 glibc 的 `clone()` 包装函数要在汇编里处理两次返回：调用前把函数指针和参数存进不会被系统调用破坏的寄存器；系统调用返回非 0 说明还在原线程，

直接返回调用者；返回 0 说明已经身处新线程，就取出函数指针调用它，函数结束后再调用 exit——新线程永远不会回到包装函数的调用者那里。

xv6-aarch64 的 clone(entry, arg, stack_top) 换了一种更简单的语义：内核直接把新线程的入口、参数和栈写进它的 trapframe，新线程第一次回到用户态时就从 entry 开始执行，用户态不需要任何汇编包装：

```
int threadclone(uint64 entry, uint64 arg, uint64 stack_top)
{
  if(entry == 0 || stack_top == 0)
    return -1;
  return do_clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND |
                  CLONE_THREAD, stack_top, 0, 0, entry, arg);
}

// do_clone() 里与线程有关的部分
  vm->refcount++; vm->users++;          // 共享同一个地址空间
  vmspace_put(np->vm);
  np->vm = vm;
  np->is_thread = 1;
  np->tgid = p->tgid;

  *(np->trapframe) = *(p->trapframe);
  np->trapframe->elr = entry;           // eret 之后从这里开始
  np->trapframe->x0  = arg;             // 第一个参数
  np->trapframe->x30 = 0;               // 不允许“返回”
  np->trapframe->sp  = stack_top;       // 新的用户栈，必须 16 字节对齐
```

x30 被清零，线程函数如果直接 ret 就会跳到地址 0 出错，所以用户库的 thread_boot() 包了一层，保证函数返回后调用 texit()。线程退出后变成 ZOMBIE，等 tjoin() 回收——与进程的 exit/wait 是同一种约定：先保留“尸体”，让等待者能取到退出状态，再由等待者释放资源。

glibc 形式的 clone(fn, stack, flags, arg, ...) 走另一条系统调用 SYS_clone_flags，内核语义与 Linux 相同（子进程从系统调用返回 0），所以它需要上面说的汇编包装：usys.pl 生成的 __clone 把 fn 和 arg 放在 x9/x10 里穿过 svc——子进程的 trapframe 是父进程的拷贝，这两个寄存器原样带过去——返回 0 时调用 fn(arg)，再以返回值 exit。vfork() 则是一段直接发 SYS_clone_flags 的汇编，x0 = 0x4111，stack = 0 表示子进程借用父进程的栈。

8.4.4 多核下的细节

- sbrk() 修改共享的页表时持有 vmspace.lock，改完执行 tlbi vmalle1is 广播失效，覆盖其他核上正在运行的同组线程；
- 用户态的 K&R malloc 加了一把基于 ldaxr/stxr 的锁；
- exec() 只在线程组只剩调用者时才允许执行；exit() 先终止并回收同组线程；kill(pid) 以 TGID 为目标，标记并唤醒整组；
- clone() 像 fork() 一样复制文件描述符引用——底层 struct file 和偏移共享，但描述符表独立。这与 Linux 的 CLONE_FILES 不同，是有意保留的简化。

threadtest 创建两个线程，用内核同步对象保护共享计数器，允许它们被调度到不同 CPU，预期输出 threadtest: PASS counter=2000。

8.4.5 实例：threadtest 的运行过程

图 8.2 的上半部分是 `exec("/bin/threadtest")` 时打印的两份快照，与第 5.4.3 节 ls 的情况几乎相同。struct proc、vmspace、内核栈用的正是 ls 退出后释放的那几页。有一点不同：新页表的物理页这次是递增分配的（0x6e99 → 0x6ea4），因为 ls 退出时按顺序还回了页，kalloc 后进先出，取出来的顺序就反过来了。

下半部分是按 `thread.c` 与 `threadclone()` 推出的线程结构，线程 tid 和线程栈地址为示意。`thread_create()` 先用 malloc 在堆里分配 16 KiB 的线程栈和一个 start 记录，再调用 `clone(thread_boot, start, 栈顶)`。内核为新线程分配独立的 struct proc、内核栈、trapframe 和 context，但让 `np->vm` 指向同一个 vmspace（`refcount` 从 1 增到 3），因此三个线程共用页表根 0x6e99000。新线程的 trapframe 从主线程复制后被改写：`elr = thread_boot`（libc 中 0xe35c），`sp` 设为线程栈顶，`x0` 设为 start 记录。线程第一次被调度时经 `forkret` → `usertrapret` → `eret` 进入 `thread_boot`，再进入 `worker`。全局变量 `counter` 和 `counter_lock` 位于程序的.bss（0x1a04、0x1a00），所有线程共享，所以要用 mutex 保护。线程栈只是堆中的普通内存，下方没有保护页，溢出时会悄悄覆盖相邻的堆数据。

8.5 libc 与 POSIX 源码兼容层

用户程序不直接写 `svc`，而是通过 libc。本节按演进顺序介绍它的几种形态：最早的静态库 `libc.a`；由内核装载的 `libc.so`，以及曾经在内核里做的代码页共享；当前源码树使用的用户态解释器 `ld-xv6.so`；还有实验分支上用 `mmap` 装库的 `ld.so`。最后是让 Linux 程序可以直接编译的 POSIX 层。

8.5.1 libc.a

在最简形态里，用户库被打包成一个静态库 `libc.a`（也就是 `make DYNAMIC_USER=0` 时保留的回退路径）：

- `ulib.c`：字符串、内存函数、gets、stat、atoi；
- `usys.S`：唯一的 `__syscall` 陷入网关；
- `printf.c`：printf/fprintf，支持 %d %x %p %s %c；
- `umalloc.c`：K&R 风格的 malloc/free，用 sbrk 向内核要内存；
- `thread.c`：上一节的线程包装；
- `posix.c`：POSIX 包装层；
- `tweetnacl.c`、`ssh_crypto.c`：密码学原语（sshd 使用）。

8.5.2 动态链接 libc.so

上面的静态 `libc.a` 让每个程序各自携带一份 libc 代码。本项目把默认形态改成共享库：`Makefile` 先把每个 libc 源文件编成位置无关对象（`-fPIC`），再用 `ld -shared` 链成 ET_DYN 的 `libc.so`，SONAME 取名 `libc.so`；用户程序则以 `-pie` 链接到它：

```
$U/%.pic.o: $U/%.c
    $(CC) $(CFLAGS) -fPIC -c -o $@ $<
$U/libc.so: $(LIBC_PIC_OBJS)
    $(LD) $(LDFLAGS) -shared --no-undefined -soname libc.so -o $@ $^
_%: %.o $(ULIB) $(USER_LINK_STAMP)
    $(LD) $(LDFLAGS) -pie --no-dynamic-linker -e main -o $@ ...
```

关键在最后一条：`-pie` 产出 ET_DYN 可执行文件，`--no-dynamic-linker` 则不写 PT_INTERP——因为本项目没有用户态 `ld.so`，动态装载的活由内核 `exec()` 自己完成。要退回上一节的静态形态，只需 `make DYNAMIC_USER=0`；构建用 `.dynamic-user`/`.static-user` 两个模式戳保证切换时重链所有用户程序，根文件系统构建再把共享库安装到 `/lib/libc.so`。

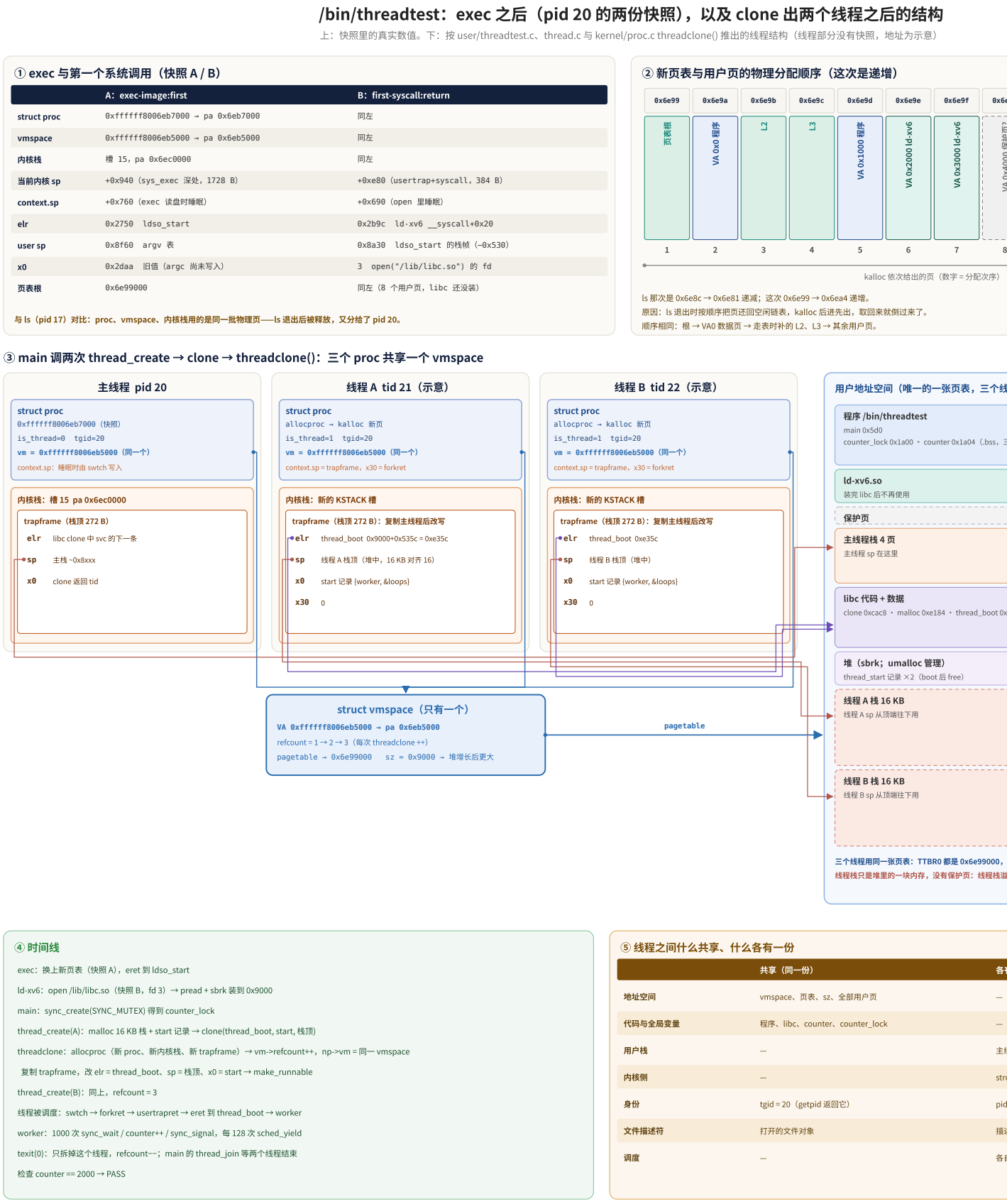


图 8.2: threadtest：exec 之后的两份快照（pid 20），以及 clone 出两个线程之后三个 proc 共享一个 vmspace 的结构

exec() 里的装载流程. 对每个 ET_DYN 程序，kernel/fs/exec.c：

1. 校验 ELF 头后按程序头把主镜像映射到虚拟地址 0，并记下 PT_DYNAMIC 位置。为此 loadseg() 放宽了老 xv6 的页对齐要求，只校验 GNU ld 保证的 p_vaddr ≡ p_offset (mod p_align)，以容纳非页对齐的段边界。
2. parsedynamic() 逐条读 DT_* 项，收集 DT_NEEDED、DT_SYMTAB、DT_STRTAB、DT_HASH、DT_RELA/RELASZ/RELAENT 与 DT_JMPREL/PLTRELSZ/PLTREL；符号数取自 ELF hash 表头，动态表必须以 DT_NULL 结束。
3. 策略校验：DT_NEEDED 必须恰好一个、且字符串正是 "libc.so"，随后打开硬编码路径 /lib/libc.so。任意依赖搜索留给未来的用户态 ld.so，不藏进内核。
4. libc 的载入基址取 libbase = PGROUNDUP(sz)，即紧跟主镜像之后的第一个空闲页，loadimage() 据此把 libc.so 也映射进本页表。
5. relocateimage() 先重定位 libc、再重定位主镜像：处理 .rela.dyn 与 .rela.plt，符号解析顺序为“主镜像优先、libc 兜底”，未定义的强符号会让 exec() 直接失败。
6. 校验入口合法、布置 argv 栈、把 PC 设为 mainimg.base + e_entry；由于重定位用 copyout 改写了用户代码与 GOT 页，返回用户态前必须 uvmsync_icache() 做 I-cache 维护。

支持的重定位类型. 装载器只认这份 AArch64 libc 实际会生成的四种 R_AARCH64_*，且全部在装载期一次性（eager）绑定，没有延迟 PLT：

```
switch(type){
case R_AARCH64_NONE:                        // 跳过
  continue;
case R_AARCH64_RELATIVE:                    // 基址 + addend
  value = img->base + rela.addend; break;
case R_AARCH64_ABS64:                       // 解析符号后 + addend
case R_AARCH64_GLOB_DAT:
case R_AARCH64_JUMP_SLOT:
  value = resolvesym(...) + rela.addend; break;
default:                                    // 其它类型一律失败
  printf("exec: unsupported AArch64 dynamic relocation %d\n", type);
  return -1;
}
```

真机上以默认配置启动、登录后，串口打印每个动态程序里 libc.so 的载入基址，这两行即证明主 ELF 与 libc.so 都被成功装载、符号被解析、重定位已完成：

```
exec: dynamic /bin/login, libc.so base=0x0000000000002000
xv6-rpi3 login: root
Password: xv6
Welcome root
exec: dynamic /bin/sh, libc.so base=0x0000000000003000
```

这里的 base 就是上文的 libbase：login 主镜像最大到 0x1a20，页对齐后落在第 2 页 0x2000；sh 更大，跨到第 3 页 0x3000。若某个程序没有打印 exec: dynamic 却能运行，说明它是静态 ET_EXEC 回退路径。

收敛与限制. 这是一个刻意为之的最小 **xv6 原生**装载 ABI，而不是完整的 ld.so：只支持 RELA（无 DT_REL）；只认上面四种动态重定位，带 GOT 数据重定位（ADR_GOT_PAGE/LD64_GOT_LO12_NC）的二进制会失败；只允许受信任的单个 /lib/libc.so，libc 自身不得再有依赖；没有 PT_INTERP、dlopen/dlsym、延迟 PLT 绑定、IFUNC、TLS、符号 interposition 与 ASLR；libc.so 每次 exec 都重新读盘并复制进本进程地址空间，进程间不共享只读

物理页。于是它消除了每个程序静态携带 libc 的磁盘冗余，却还没省下运行时的物理内存。可插拔 .ko 模块的装载器与它共用 include/kernel/elf.h 里的数据结构，却是两套独立逻辑：模块装载器处理 ET_REL、做的是指令位域修补，而这里处理 ET_DYN、做的是把基址与符号值写回 64 位槽。

8.5.3 一次库函数调用：PLT 与 GOT

动态链接最核心的问题是：程序编译时不知道 strlen 会被装到哪里，代码里又不能留绝对地址（否则代码页就要按进程改写，无法共享）。AArch64 的答案是“经过一张表间接跳转”。以 /bin/echo 为例，先看 ELF 里记了什么：

```
$ readelf -lW user/_echo
  PHDR      vaddr 0x40     0x188  R      程序头表自身
  INTERP    vaddr 0x1c8    0x00b  R      "/lib/ld.so"
  LOAD      vaddr 0x0      0x3ec  R E    ELF 头、.plt、.text、只读数据
  LOAD      vaddr 0x1ec0   0x158  RW     .dynamic、.got、.got.plt
$ readelf -dW user/_echo
  NEEDED  libc.so    HASH 0x1d8   SYMTAB 0x200   STRTAB 0x260
  PLTGOT  0x1fe8     JMPREL 0x280 PLTRELSZ 72    PLTREL RELA
$ readelf -rW user/_echo
  0x2000  R_AARCH64_JUMP_SLOT  write
  0x2008  R_AARCH64_JUMP_SLOT  exit
  0x2010  R_AARCH64_JUMP_SLOT  strlen
```

main 调用 strlen 时，bl 的目标是本程序里一段 16 字节的桩代码——PLT 项；桩代码从 GOT 读出真实地址再跳过去：

```
 368:  bl   310 <strlen@plt>        // main 里：调用 PLT 项
0000000000000310 <strlen@plt>:
 310:  adrp x16, 2000               // GOT 所在页，PC 相对
 314:  ldr  x17, [x16, #16]         // x17 = GOT[0x2010]
 318:  add  x16, x16, #0x10
 31c:  br   x17                     // 跳进 libc
```

GOT 在可写段里，构建出来时三个槽都指向 0x2d0，也就是 PLT0——glibc 做延迟绑定时的解析入口。本项目不做延迟绑定，PLT0 依赖的保留槽是 0，真跳过去会崩溃，所以必须在 main 之前把槽写好（急切绑定）。处理 R_AARCH64_JUMP_SLOT strlen @ 0x2010 时，在全局作用域里查到 libc 的 strlen（st_value = 0x4260），libc 若被映射在 0x10_0000_1000，就写入 0x10_0000_5260。此后每次库调用多一次 ldr 和一次 br，换来的是代码里没有任何绝对地址。

libc 自己也一样。它以 -shared、不带 -Bsymbolic 链接，内部对导出函数的调用同样走自己的 PLT（例如 putc 里的 bl write@plt），因此有 71 条 JUMP_SLOT，目标全在 RW 段的 .got.plt。这样程序可以覆盖 libc 的函数：符号查找先找程序，程序若自己定义了 malloc，libc 内部的 malloc 调用也会解析到它。表 8.3 是装载器支持的全部重定位类型。

表 8.3: 支持的动态重定位（S：符号地址，A：addend，B：模块基址）

类型	编号	计算	典型位置
R_AARCH64_NONE	0	跳过	
R_AARCH64_ABS64	257	S + A	数据里的绝对指针
R_AARCH64_GLOB_DAT	1025	S + A	.got：全局变量地址
R_AARCH64_JUMP_SLOT	1026	S + A	.got.plt：函数地址
R_AARCH64_RELATIVE	1027	B + A	模块内部的指针

8.5.4 共享 libc 代码页

上面的装载器让每个进程都有一份私有的 libc.so。可是 libc 是位置无关的、没有 DT_TEXTREL：所有重定位都落在它的 RW 段（GOT 与 .data），只读页在任何基址下都逐字节相同。用 readelf 看这份 libc：

```
LOAD  vaddr 0x0     memsz 0x9272   R-X   代码与只读数据：第 0..9 页
LOAD  vaddr 0xa278  memsz 0x390    RW-   GOT、.data、.bss：落在 0xa000 这一页
重定位：71 条 R_AARCH64_JUMP_SLOT，目标全部在 RW 段
```

所以只读页可以让所有进程共用一份物理内存，RW 页仍每个进程一份。第一版实现在 exec() 里专门缓存 /lib/libc.so 的代码页，第二次以后直接映射；现在这件事交给通用的页缓存（第 4.13 节）：shlib_map() 对每个代码页调用 pcache_getpage()，以用户只读、可执行、PXN 映射，并用 shlib_textsize() 保证只读段满足 p_vaddr == p_offset、且不与可写页共用一页。重定位若要写进共享的代码区，exec() 直接拒绝——那会让“共享”的页带上某个进程的基址。

（当前源码树改用用户态解释器 ld-xv6.so 之后，这段内核代码已经删除，libc 改由解释器加载，见第 8.5.5 节。）

真机上 login 和 sh 的 libc 基址不同（0x2000 与 0x3000，由各自主镜像的大小决定），这并不妨碍共享：代码页内容与装载地址无关，只是各进程把同一组物理页映射到不同的虚拟地址。

8.5.5 用户态解释器 /lib/ld-xv6.so

内核里的装载器只认一个 /lib/libc.so，还把 ELF 解析、符号查找、重定位都放在内核里。Linux 的做法是交给用户态：程序头里的 PT_INTERP 指名一个解释器，内核只映射主程序和解释器，其余由解释器在 EL0 完成。当前源码树就是这样做的，解释器叫 /lib/ld-xv6.so（user/ldso.c，约 350 行）。

构建。 程序以 -pie -dynamic-linker /lib/ld-xv6.so 链接，于是带上 PT_INTERP。解释器由 ldso.pic.o 和 usys.pic.o 链成，只借用 libc 的 __syscall 网关：

```
$U/ld-xv6.so: $U/ldso.pic.o $U/usys.pic.o
    $(LD) $(LDFLAGS) -shared -Bsymbolic -e ldso_start -o $@ $^
```

mkfs 新增 -I 选项，把它放进 /lib；ext2 根镜像也一并写入。readelf 显示它只有两个 PT_LOAD，入口 0x750 就是 C 函数 ldso_start，而且没有任何重定位：-fPIC 加 -Bsymbolic 让所有自身引用都在链接时绑定为 PC 相对访问。因此内核把它装到任何基址都能直接运行，它也不必像 glibc 的 ld.so 那样先重定位自己。

内核这一半。 exec() 删掉了约 460 行内核装载代码，只剩三件事：

1. readinterp() 找到唯一的 PT_INTERP，路径以 NUL 结尾、不超过 128 字节；ET_DYN 程序没有它就拒绝（exec: dynamic …has no valid PT_INTERP）；
2. 主程序装到 VA 0，解释器装在 interpbase = PGROUNDUP(sz)，两者都是 loadimage() 分配的私有页；入口改为 interpbase + e_entry；
3. 初始栈在 argv 之后放空的 envp 和 Linux 形式的辅助向量，argc 仍由 x0 传递：

```
sp -> argv[0..argc-1], NULL        <- x1; x0 = argc
      envp[0] = NULL
      AT_PHDR = e_phoff  AT_PHENT = 56  AT_PHNUM  AT_PAGESZ = 4096
      AT_BASE = interpbase（未使用）      AT_ENTRY = main   AT_NULL
```

解释器这一半。 ldso_start(argc, argv) 用 argv + argc + 2 找到 auxv，然后：

1. 由 AT_PHDR 找到主程序的 PT_DYNAMIC（主程序基址视为 0），解析 DT_*，符号个数取自 DT_HASH 的 nchain；

2. 要求 DT_NEEDED 恰好一个且就是 "libc.so"；
3. load_libc()：open("/lib/libc.so")，用 pread 读 ELF 头和程序头，按最大段尾 0xa650 调用 sbrk(0xb000) 向堆要 11 页，再把每个 PT_LOAD 用 pread 读到 base + p_vaddr；sbrk 的新页已经清零，.bss 不用另外处理；
4. 先重定位 libc（71 条 JUMP_SLOT），再重定位主程序；符号在“主程序 → libc”中线性查找，因此程序可以覆盖 libc 的函数；
5. 向 stderr 打印 ld.so: relocated libc.so in user space，以 (argc, argv) 调用 AT_ENTRY，再用返回值 exit。

按这次构建推算，/bin/sh 的地址空间是：程序 0x0–0x2e10，ld-xv6.so 0x3000–0x4d20，保护页和 4 页栈 0x5000–0x9fff，libc 0xa000–0x14fff，malloc 的堆从 0x15000 开始。所有页都是这个进程私有的，可读、可写、可执行。

表 8.4: 三种 libc 装载方式

	内核加载（第一步）	ld-xv6.so（当前）	ld.so（实验分支）
谁加载	内核 exec	用户态，pread	用户态，mmap
libc 位置	主程序之后	sbrk 堆起点	mmap 区
代码页	跨进程共享	每进程一份	页缓存共享
fork	整页复制	整页复制	写时复制
依赖	仅 libc	仅 libc	多个，/lib 搜索
内核需要	装载器	pread、sbrk	mmap、页缓存、COW

问题与建议. 对内核要求最少是 ld-xv6.so 最大的优点，表 8.4 也列出了它的代价。分析代码时还发现以下几点：

- **I-cache 一致性（真机隐患）**：libc 的代码是由内核 copyout 经数据缓存写进堆页的，之后没有做 I-cache 维护——exec 末尾的 uvmsync_icache() 在解释器运行之前就已结束，而 SCTLR_EL1 没有设置 UCI（bit 26），EL0 不能自己执行 dc cvau/ic ivau。Cortex-A53 的 I-cache 不侦听 D-cache，取指可能拿到旧内容，正是第 4.12 节故障二那一类问题；QEMU 不模拟缓存，所以看不出来。修法：像 Linux 一样打开 UCI，让解释器在读完代码段后逐行 dc cvau、dsb ish、ic ivau、dsb ish; isb；或者由内核提供一个 cacheflush(addr, len) 系统调用。
- **libc 与堆相邻**：用户 sbrk 负数缩到 libc 以下会把 libc 一起释放；每个进程多占 11 页，fork 时还要整页复制。
- **依赖“无重定位”**：以后若在 ldso.c 里写出带指针初值的全局变量，链接器会生成 R_AARCH64_RELATIVE，而解释器不处理它，结果是静默出错。建议在 Makefile 里检查 readelf -r 为空，或者用 AT_BASE 做自重定位。
- **模式戳未改名**：exec 现在拒绝没有 PT_INTERP 的 ET_DYN，但 .dynamic-user 戳没变；从旧链接方式切换过来时如果没有 make clean，开机会报 exec: dynamic /init has no valid PT_INTERP，随后 panic: init exiting。
- 每次 exec 都会打印两行（内核的 -> interpreter 和解释器的 relocated libc.so）；AT_PHDR 用 e_phoff 而不是 PT_PHDR；报错前缀仍是 ld.so:。

8.5.6 图解：/bin/echo 的文件布局与运行时绑定

本小节用两张图把前面的内容串起来。图中的偏移、地址和符号值都取自这次构建的 readelf -h -l -S -d -r --dyn-syms 和 objdump -DD user/_echo；libc 基址 0x9000 是按下文推算的值。

文件布局. 图 8.3 的中间一列是文件内容，按偏移从上到下排列（未按比例），每块都写出了解码后的内容。.comment 和 9 个 .debug_* 节（-g 生成，偏移 0x4b0–0xa3a）与加载无关，图中省略。

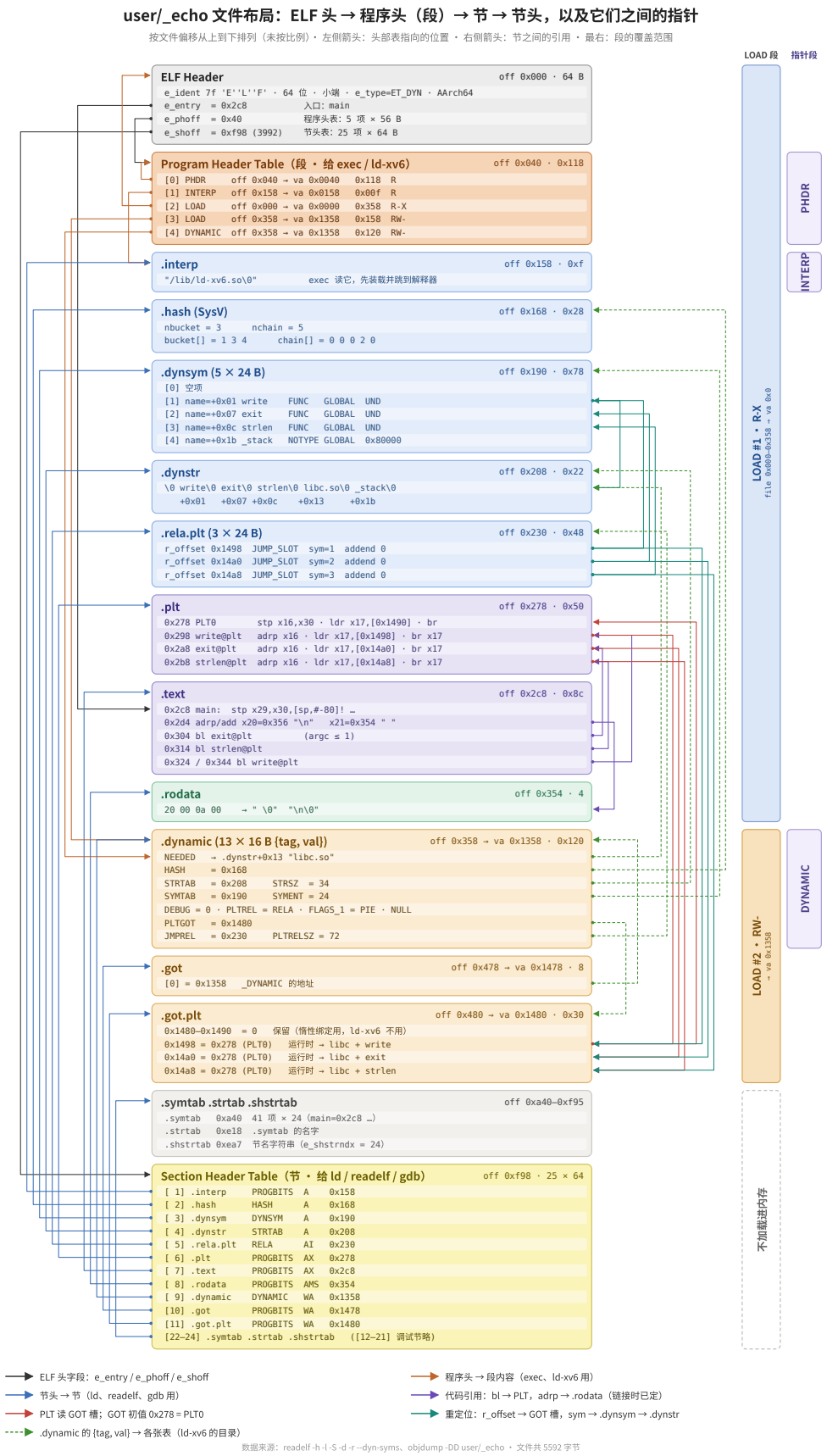


图 8.3: _echo 的文件布局：ELF 头、程序头（段）、节、节头，以及它们之间的指针

左侧箭头表示三张头部表指向文件的哪个位置：

- ELF 头的 `e_entry = 0x2c8` 指向 `main`，`e_phoff = 0x40` 指向程序头表，`e_shoff = 0xf98` 指向节头表；
- 程序头（橙色）给 exec 和 ld-xv6 看，按段描述；
- 节头（蓝色）给 `ld`、`readelf`、`gdb` 看，按节描述。

右侧箭头表示节与节之间的引用：

- 紫色：`bl` 跳到 PLT，`adrp/add` 取 `.rodata` 中的字符串，这些在链接时已经算好；
- 红色：PLT 存根读取 GOT 槽，GOT 槽的初值 `0x278` 又指回 PLT0；
- 青色：`.rela.plt` 的 `r_offset` 指向 GOT 槽，`sym` 指向 `.dynsym`，`.dynsym` 的 `st_name` 指向 `.dynstr`；
- 绿色虚线：`.dynamic` 的 `{tag, val}` 指向各张表。

最右两条色带分别标出两个 LOAD 段的覆盖范围，以及 PHDR、INTERP、DYNAMIC 三个“指针段”指向的内容。从图中可以读出以下几点：

1. **加载只看程序头。** exec 遍历程序头表，只处理 `PT_LOAD`：用 `vaddr + memsz` 调用 `uvmalloc`，用 `(vaddr, off, filesz)` 调用 `loadseg`。节头表、`.symtab`、`.strtab` 不在任何 LOAD 段里，`strip` 掉以后程序照样运行。
2. **节被打包进段。** `.interp` 到 `.rodata` 共 8 个节合成 LOAD #1（R-X，文件 `0x000–0x358` 映射到 VA 0）。`.dynamic`、`.got`、`.got.plt` 合成 LOAD #2（RW-，映射到 VA `0x1358`）。链接时没有用 `-z separate-code`，所以只读元数据、代码和常量同在一个 R-X 段。
3. **第二个 LOAD 段的 VA 不按页对齐。** ELF 只要求 `(vaddr - off) % align == 0`（`0x1358 - 0x358 = 0x1000`），所以 exec 去掉了旧版的 `vaddr % PGSIZE` 检查，并让 `loadseg` 能从页中间开始写入。`_echo` 的 `FileSiz == MemSiz`，没有 `.bss`。
4. **INTERP、DYNAMIC、PHDR 不单独加载。**它们都落在某个 LOAD 段内部，只起指路作用：INTERP 告诉内核先运行 `/lib/ld-xv6.so`；PHDR 和 `AT_PHDR` 相减得到主程序基址；DYNAMIC 是 ld-xv6 的目录。运行时没有节头可用，ld-xv6 只能通过 `DT_SYMTAB`、`DT_HASH`、`DT_JMPREL` 等找到各张表。
5. **PLT0 和 `GOT[0..2]` 用不上。**它们是为惰性绑定保留的。ld-xv6 在程序开始前一次性填好所有 GOT 槽，所以 PLT0 永远不会被执行；如果真的走到那里，它会跳到 `GOT[2]` 中的 0，现象就是 `elr=0` 的异常。

运行时绑定. 图 8.4 的左列是内核和磁盘上的三个文件，中列是进程的虚拟地址空间（地址向下增长），右列是步骤说明。每条箭头都标有步骤序号：1–3 由内核 exec 完成，4–6 由 EL0 的 ld-xv6 完成，7–8 是程序本身的运行。各步骤见表 8.5。

表 8.5: `exit()` 的绑定步骤

步骤	执行者	内容
1	内核	读 `/bin/echo` 的程序头，把 LOAD #1 装到 `0x0`，LOAD #2 装到 `0x1358`
2	内核	发现 `PT_INTERP`，把 `ld-xv6.so` 装到 `PGROUNDUP(0x14b0) = 0x2000`；分配保护页和 4 页栈（`0x4000–0x9000`），压入 argv、envp 和 auxv（`AT_PHDR=0x40`，`AT_BASE=0x2000`，`AT_ENTRY=0x2c8` 等）
3	内核	`elr = 0x2000 + 0x750 = 0x2750`（`ldso_start`），`eret` 回到 EL0
4	ld-xv6	用 `argv + argc + 2` 找到 auxv，用 `AT_PHDR` 扫描程序头，找到 `PT_DYNAMIC 0x1358`，读出 `NEEDED`、`SYMTAB`、`HASH`、`JMPREL`
5	ld-xv6	`open("/lib/libc.so")`，`sbrk` 得到基址 `0x9000`，`pread` 两个 PT_LOAD；先处理 libc 自己的 71 个 `JUMP_SLOT`
6	ld-xv6	处理 `_echo` 的 `.rela.plt`：`sym=2` 是 `"exit"`，在主程序中是 UND，于是查 libc 的 `.hash` 和 `.dynsym`，得到 `0x3cc0`，写入 `[0x14a0] = 0x9000 + 0x3cc0 = 0xccc0`；write、strlen 同理
7	ld-xv6	以 `(argc, argv)` 调用 `AT_ENTRY = 0x2c8`，即 `main`
8	程序	`bl exit@plt`（`0x2a8`）→ `ldr x17, [0x14a0]` 得到 `0xccc0` → `br x17` 进入 libc 的 `exit` → `svc #0`

关于这张图还有几点说明：

- **libc 基址 `0x9000` 的来历。**`ld-xv6.so` 的段止于 `0x3d20`，向上取页得 `0x4000`；保护页和 4 页栈占到 `0x9000`，所以 exec 结束时 `sz = 0x9000`，ld-xv6 的第一次 `sbrk` 就从这里开始。libc 代码段是 `0x9000–0x122ba`，数据段是 `0x132c0–0x13650`。程序更大时这些值整体上移，例如 `/bin/sh` 的 libc 在 `0xa000`。

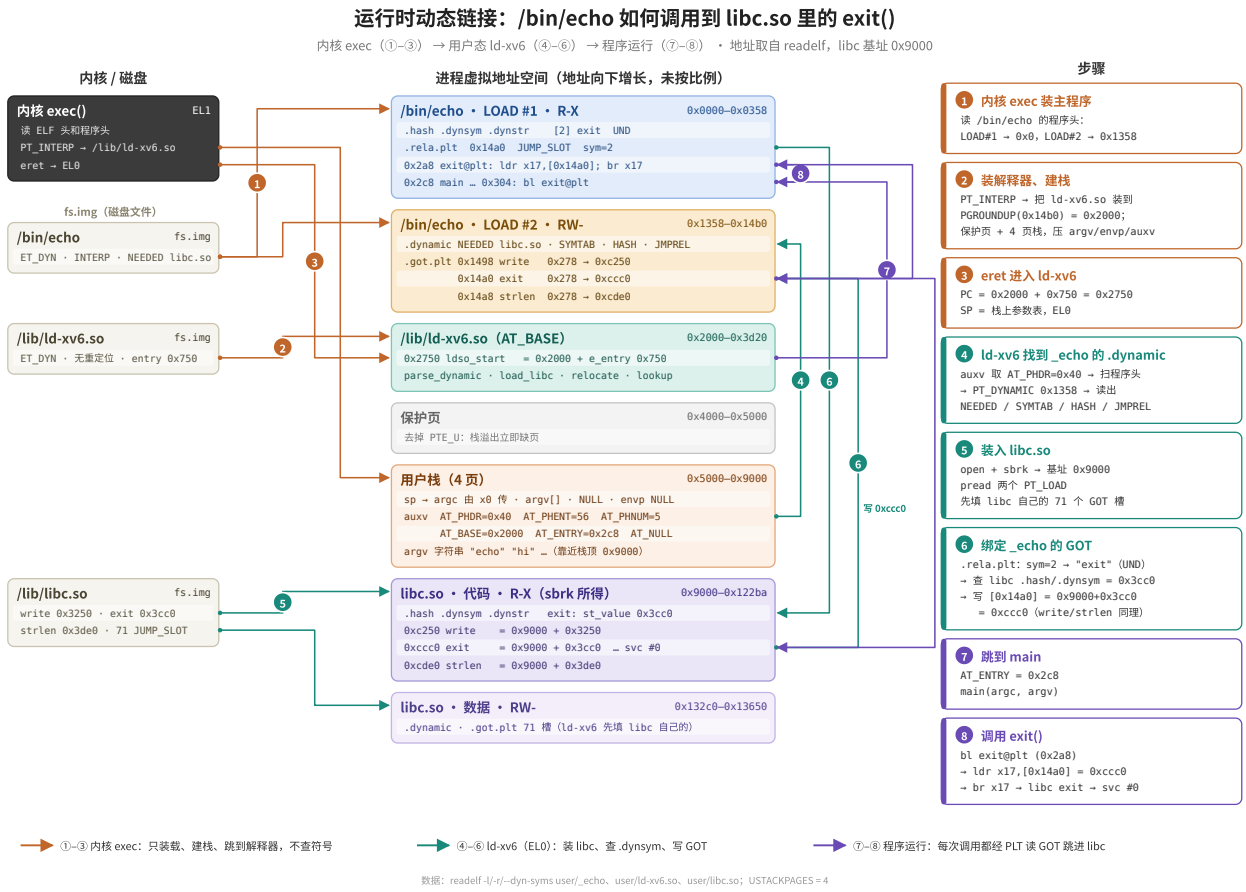


图 8.4: 运行时动态绑定：/bin/echo 如何调用到 libc.so 里的 exit()

- **内核从不查符号**。步骤 1–3 只负责按程序头摆放字节、建栈、跳到解释器；符号查找和 GOT 填写全部在 EL0 完成。
- **绑定只发生一次**。步骤 6 之后，每次库函数调用都只是“bl 到 PLT，读 GOT，br 跳转”三步，不再经过 ld-xv6。
- **真机注意 I-cache**。步骤 5 中 libc 的代码是以数据的形式写入堆页的，在 Cortex-A53 上执行之前需要先做 I-cache 维护，见第 8.5.5 节的“问题与建议”。

8.5.7 进一步：用 mmap 装载并共享（实验分支）

ld-xv6.so 让每个进程都有一份私有的 libc。实验分支 wip-cow-mmap-ldso 在第 4.13 节的引用计数、写时复制、页缓存和 mmap 之上，换了一个用 mmap 装库的解释器 /lib/ld.so：libc 的代码页全系统只有一份，数据页写时复制。本小节描述这个版本，它尚未合入当前源码树。

链接方式. 用户程序改为

```
USER_LD_MODE = -pie -dynamic-linker /lib/ld.so --hash-style=sysv
```

于是每个程序都带上 PT_INTERP /lib/ld.so 和 PT_PHDR；--hash-style=sysv 保证有 DT_HASH 供符号查找。模式戳改名为 .dynamic-ldso，旧的二进制会被重新链接。fs.img 和 ext2 根镜像同时安装 /lib/libc.so 与 /lib/ld.so（mkfs 支持多个 -L）。

exec() 的工作. 主程序照旧装到虚拟地址 0。若有 PT_INTERP，读出解释器路径，把它装在主镜像之后（只读页同样取自页缓存，所有进程共用一份 ld.so 代码），然后从解释器的入口开始运行。程序的信息通过辅助向量（auxv）传过去，标签取 Linux 的值：

```
x0 = argc   x1 = argv   x2 = auxv (以 {tag, value} 对，以 AT_NULL 结束)
AT_PHDR     主程序程序头的地址          AT_PHENT/AT_PHNUM  表项大小与个数
AT_PAGESZ   4096                        AT_BASE            ld.so 的装载地址
AT_ENTRY    主程序的入口
```

Linux 把 argc、argv、envp、auxv 全部放在栈上；这里沿用 xv6 “参数放在寄存器里”的传统，只多传一个 x2。没有 PT_INTERP 的程序仍走上一小节的内核装载器，作为兼容路径。

ld.so 的工作. user/ldso.c 是独立的（不依赖 libc——libc 正是它要装的东西），用原始 svc 做系统调用，全部符号 hidden，并以 -shared -Bsymbolic -z text 链接：

1. **自重定位**：根据 AT_BASE 处理自己 _DYNAMIC 里的 R_AARCH64_RELATIVE。在此之前只能用 PC 相对寻址访问代码和数据。实际构建出的 ld.so 没有任何重定位项，这一步通常什么也不做；
2. **找到程序**：由 AT_PHDR 与 PT_PHDR 算出主程序基址，读它的 PT_DYNAMIC；
3. **装载依赖**：对每个 DT_NEEDED（不含 / 的名字到 /lib 下找），先 mmap 文件第 0 页读出 ELF 头和程序头，再用一次 mmap(NULL, span, ...) 把整个镜像范围映射成文件内容（决定了基址），最后对其余段用 MAP_FIXED 按各自的地址和权限重新映射，并把 .bss 清零、不足的部分补匿名页。依赖按广度优先装载，可以有多个库；
4. **重定位**：先库后程序，处理 RELATIVE、ABS64、GLOB_DAT、JUMP_SLOT；符号用 SysV 哈希在全局作用域“程序 → 各库”中查找，所以程序可以覆盖（interpose）libc 的符号；
5. **运行**：以 (argc, argv) 调用 AT_ENTRY，用它的返回值调用 exit。

为什么库不能有文本重定位，程序却可以. 用户程序是以 -fno-pie 编译、-pie 链接的，只读段里的指针表（例如 usertests 的测试表）需要重定位，于是带 DT_TEXTREL。程序是 exec() 装进来的私有可写副本，改写无妨；库

的代码页却是页缓存里所有进程共用的只读页，所以 ld.so 只对库拒绝 DT_TEXTREL，libc 本身也以 -z text 链接来保证这一点。

结果. 开机后从 /init 起所有程序都经由 ld.so 启动。在 QEMU 中（用 gcc 13 构建，libc 代码段为 12 页，可写段从文件第 0xb000 页开始），vmmap 显示 init、sh、vmmap 三个进程里：

- libc 映射在 0x10_0000_1000 起的 mmap 区；12 个代码页在三个进程里是同一组物理页，每页 “shared by 4 mappings”（页缓存 + 3 个进程）；
- 文件第 0xb000 页既是代码段的尾巴（只读可执行），又是 .data 的开头（只读 + COW），因此被引用 7 次；
- 被重定位写过的 GOT 页已经各自复制成私有的 RW 页——只有真正被写的页才有副本；
- ld.so 的代码页也只有一份物理页。

与 Linux 相比还缺：环境变量与 AT_EXECFN、dlopen/dlsym、延迟绑定、IFUNC、TLS 与 ASLR；页缓存也只服务 mmap 和库代码，普通 read() 不经过它。

8.5.8 调试动态链接

出问题时先看是哪一条路径：内核打印 exec: dynamic /bin/x, libc.so base=… 的是内核内置加载器（没有 PT_INTERP 的程序）；不打印却正常运行的走 ld.so，在 vmmap 里能看到 libc 位于 0x10_0000_xxxx；DYNAMIC_USER=0 的静态程序两者都不经过。主机上常用的检查：

```
aarch64-elf-readelf -lW user/_sh                # 有没有 INTERP、PHDR；段布局
aarch64-elf-readelf -dW user/libc.so | grep TEXTREL   # 库必须没有
aarch64-elf-readelf -rW user/libc.so            # 重定位类型是否都受支持
aarch64-elf-objdump -s -j .got.plt user/_sh     # GOT 初值
```

表 8.6 列出常见的报错。

表 8.6: 动态链接的常见报错

信息	原因
exec: …cannot open interpreter /lib/ld.so	根文件系统里没装 ld.so（fs.img 或 ext2 镜像）
ld.so: cannot open /lib/libc.so	缺库，或 DT_NEEDED 名字不对
ld.so: undefined symbol foo	程序用了 libc 没有导出的函数
ld.so: unsupported relocation type	出现表 8.3 以外的类型，如 R_AARCH64_COPY
ld.so: text relocations are not supported	库里有没用 -fPIC 编译的对象
exec: unresolved dynamic symbol	同上，发生在内核内置加载器里
usertrap(): …ec=0x0 …elr=0x0，随后 init exiting	程序是 ET_DYN，内核却没有做动态装载：GOT 保持 0，第一次库调用跳到地址 0，执行 ELF 头 \x7fELF 得到未定义指令

最后一条是合并分支时真实遇到的：rpi3- kobject 的 Makefile 已经把程序链接成动态的，exec.c 却还是静态装载器，于是 init（pid 14）在第一次调用 libc 时就跳到了 0。完整的逐步说明见 readme/readme_dynamic_libc.md。

8.5.9 POSIX 层

目标是让只依赖常见 POSIX 接口的 C 源码能直接用 xv6 工具链重新编译。它由三部分组成：

1. include/ 提供标准名字的头文件：stdio.h、stdlib.h、string.h、unistd.h、fcntl.h、errno.h、time.h、pthread.h、semaphore.h、sys/types.h、sys/stat.h、sys/socket.h、netinet/in.h、arpa/inet.h、sys/epoll.h；
2. user/posix.c 把标准 API 映射到 xv6 已有的能力；

3. 内核补上无法只靠库函数实现的语义，如 `lseek()`，以及 `socket()` → `bind()` → `listen()` → `accept()` 的生命周期。

表 8.7: POSIX 接口到 xv6 的映射

POSIX 接口	实现
`open/read/write/lseek/fstat`	直接对应系统调用；`O_CREAT` 是 `O_CREATE` 的别名
`getpid/getppid/getcwd/gettid`	进程与当前工作目录查询
`dup/dup2`	复制、重定向文件描述符
`wait/waitpid`	回收僵尸子进程；`waitpid` 支持指定直接子进程、任意子进程 `-1` 与 `WNOHANG`
`readlink`	仅 `/sys` 实现（sysfs 符号链接），其它文件系统按 `EINVAL` 失败
`pthread_create/join/exit`	`clone/tjoin/texit`；`pthread_join` 同时回收用户栈
`pthread_mutex_*`、`sem_*`	内核同步对象；支持 `PTHREAD_MUTEX_INITIALIZER` 延迟初始化
`clock_gettime(CLOCK_MONOTONIC)`	`clock_us()`，读 `CNTVCT_EL0`
`sleep/usleep/nanosleep`	秒语义；原来按 tick 计的 `sleep` 改名为 `sleep_ticks`
`socket/bind/listen/accept`	统一的 socket 文件描述符，可用 `read/write/close/epoll`

`posixtest` 只包含标准头文件，验证文件读写与 `lseek`、`getcwd`、`dup2`、`waitpid` 与 `getppid`、POSIX 信号量、两个 pthread 并发更新共享变量、静态初始化的互斥锁以及 socket 生命周期。

这仍是第一阶段：系统调用失败只返回 −1，没有传回具体错误码，`errno` 也还不是线程局部的；`waitpid` 尚不支持进程组选择以及 `WUNTRACED/WCONTINUED`，`WIFEXITED` 暂时只对应 xv6 的普通退出状态；还缺条件变量、目录流和 TCP `connect()`。

8.6 在 xv6 里编译 C 程序

`tcc` 是一个面向教学的极小 C 编译器，运行在 xv6 用户态，直接输出 `exec()` 能加载的 ELF64 文件，不需要汇编器和链接器：

```
$ edit hello.c
int main() {
  int answer = 6 * 7;
  printf("hello from xv6 C\n");
  return answer - 42;
}
$ tcc hello.c -o hello
$ hello
hello from xv6 C
```

它支持整数局部变量、四则运算、`return`、只含字面量的 `printf`/`puts` 和注释。生成的代码用 `x29` 指向 256 字节的局部变量区，表达式结果放在 `x0`，字符串地址用 `adr` 指令取得，所以无需重定位；输出和退出直接用 `svc #0` 调用 `SYS_write` 和 `SYS_exit`。文件布局是一个 ELF 头、一个程序头、机器码和字符串池。

一个能在自己身上编译并运行程序的系统，才算是有了一点“自举”的味道。

8.7 一个大型用户程序：sshd

`sshd` 有一千多行，是整个系统里最复杂的用户程序，也是对内核各子系统的一次综合考验。它用到了：

- TCP socket 与 `accept`——网络协议栈；
- 密码学全部在用户态完成：基于 TweetNaCl[3] 的 X25519 密钥交换和 Ed25519 主机密钥签名，传输层用 AES-128-CTR 加密、HMAC-SHA256 校验；熵来自内核的硬件随机数发生器（`kernel/lib/rng.c`）；
- `/etc/passwd` 口令认证；

- PTY：为每个会话分配一对伪终端，shell 的 fd 0/1/2 指向 PTY 从端，`sshd` 读写主端并加密转发；
- 会话与控制终端：SSH 会话中的 `/dev/tty` 指向该会话的 PTY。

调试 `sshd` 时发现的问题推动了内核的多处改进：字符回显慢暴露了“网卡每 100 ms 轮询一次”和“TCP `write()` 同步等 ACK”两个瓶颈，促成了 SDIO 中断、workqueue 下半部、NAPI 和 TCP 异步发送队列；`ps` 输出跑到串口上，促成了 `sys_ps()` 改为返回缓冲区。

注 从 `_entry` 的第一条 `mrs` 指令，到一个可以通过 Wi-Fi 被 SSH 登录、能在多核上运行实时任务、能用 `tcc` 编译程序的系统，中间经过的每一层——串口、页表、进程、文件系统、驱动、协议栈、libc——都写在这本书里了。它们都还很简单，但每一层都在真机上跑过、出过错、被修好过。这也许就是“从裸机到操作系统”最有价值的部分。

到这里，从上电到 SSH 登录的主线已经走完。后面两章是建立在主线上的专题：第 9 章让 USB 设备可以在运行中插拔、驱动按需从 `.ko` 装载；第 10 章深入 ext2 驱动本身，看它怎样在没有掉电保护的 SD 卡上做到写入可恢复。

第九章 热插拔与驱动自动装载

第 7 章的 USB 键盘、MT7601U 都是“开机时插好、启动时枚举一次”的设备。本章讨论设备在运行中出现和消失：Hub 用自己的 interrupt-IN 端点报告端口变化，主机据此对新插入的设备重新枚举、拔出时拆除整棵子树；当总线上找不到匹配的驱动时，设备模型生成 MODALIAS 字符串，经 `request_module()` 从根文件系统的 `/lib/modules` 装载对应的 `.ko`。前者依赖第 7 章的 DWC2 主机通道、split 事务和 URB（9.1 节），后者依赖设备模型的 match/probe 与 `modalias` 钩子（7.1 节），最后用一份 Logitech 复合接收器的真机日志把两者串起来（9.2 节）。

9.1 Hub 中断驱动的 USB 热插拔

键盘按键报告和 USB 拓扑变化是两种不同的事件。按键数据由键盘自己的 interrupt-IN URB 接收；热插拔则由 LAN951x Hub 的 interrupt-IN endpoint 通知主机。当前 DWC2 为 Hub 的长期 URB 固定使用 channel 7，为唯一 HID 数据 URB 使用 channel 6。Hub 空闲时会对 interrupt-IN 返回 NAK，HCD 按 `bInterval` 重新调度请求；只有端口变化位图非零时才唤醒 Hub 事件 worker，不再每隔固定时间轮询所有端口的 EP0。

整个“插入键盘 → 枚举 → 可按键”的过程，可以串成一条从端口变化到第一个按键的总线时间线（图 9.1）：开机时四个端口都空载，控制器把 ch7 挂在 Hub 的 interrupt-IN 上等待热插拔；插入 port 3 之后由端口变化中断触发 reprobe，低速键盘的 EP0 枚举全部经 Hub 的 TT 拆成 SSPLIT/CSPLIT；驱动绑定完成后，周期 interrupt-IN 固定交给 ch6，直到某个按键的 HID 报告变成用户空间的事件。

```
Hub interrupt endpoint
  -> DWC2 channel 7 DMA receives change bitmap
  -> IRQ 9 records completion and queues hub_event_work
  -> dwc2_hub_wq examines changed ports only
  -> GET_STATUS / CLEAR_FEATURE for those ports
  -> compare connected bitmap
```

硬中断只处理 host-channel 完成并排入工作项，不在 IRQ 中进行 Hub control 传输。独立的 `dwc2_hub_wq` 根据 bitmap 中标记的端口发送 `GET_STATUS`，读取连接状态和变化位，再用 `CLEAR_FEATURE` 清除已处理的 port change feature。比较新旧 connected bitmap 后，只有拓扑确实变化才启动重枚举；因此空闲 NAK 不会造成无意义的全端口扫描。

设备拔出时，类驱动和 input/evdev 的资源按逆序回收：

```
usb_device_unregister(old_child)
  -> usbhid remove()
  -> mark device disconnected
  -> usb_kill_urb()
       -> stop/mask the HID host channel
       -> wait for completion callback
  -> cancel_work_sync()
  -> input_unregister_device()
       -> remove event-minor registration
       -> wake blocked event readers
  -> release driver-owned references
```

注销 event 设备会唤醒阻塞在 `read()` 的客户端，剩下的语义与 7.4.8 小节一致：旧 fd 先读完已提交的完整帧，内存等最后一个引用释放。热插拔在这里第一次真正考验了这套引用计数——拔掉的瞬间 `evtest` 往往还活

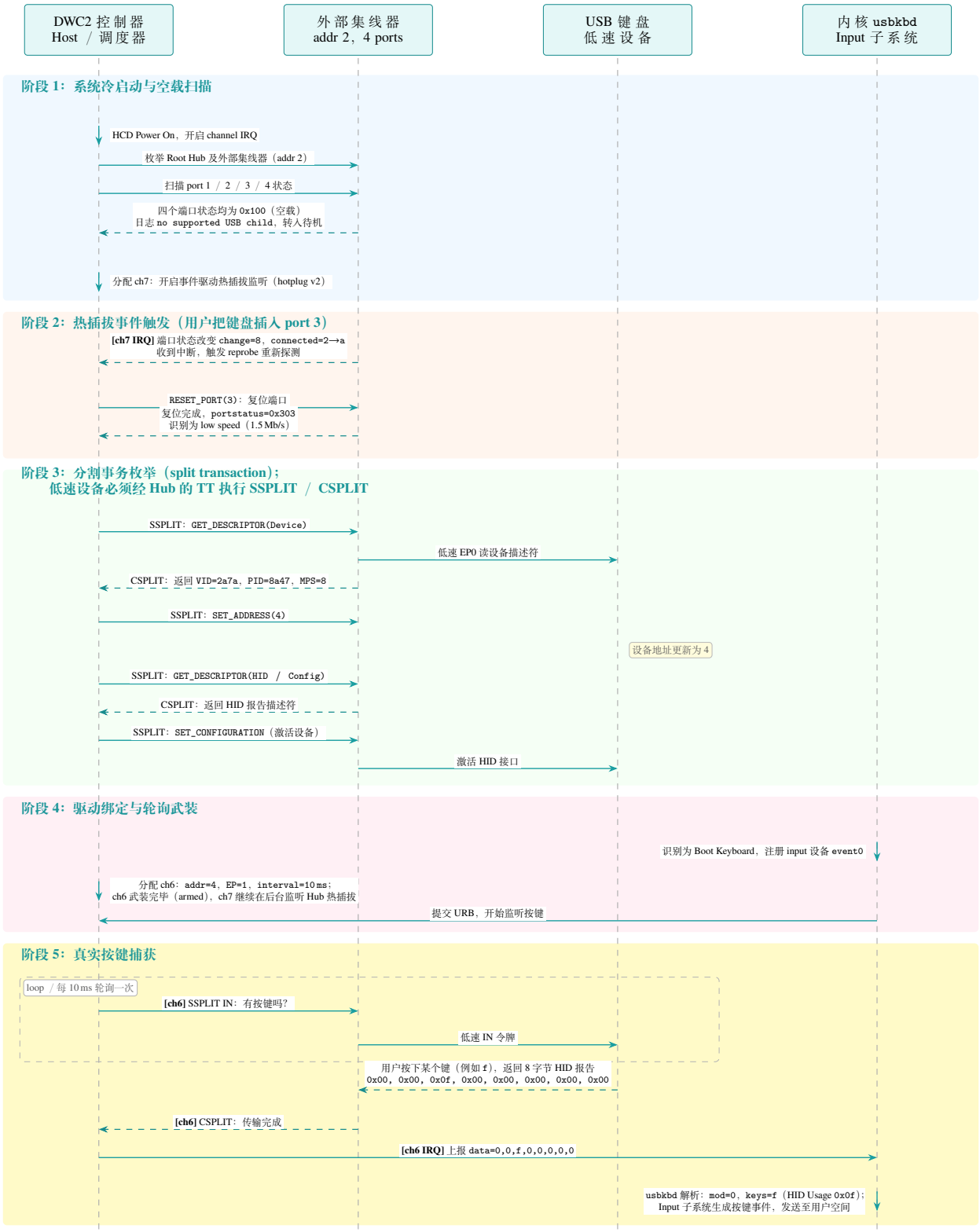


图 9.1: DWC2 经外部 Hub 热插拔枚举低速 HID 键盘、直到收到第一个按键的总线时序。ch7 只报告 Hub 的端口变化，worker 收到中断才 reprobe；EP0 枚举用 SSPLIT/CSPLIT 经 Hub 的 TT 转发；驱动绑定后 ch6 以 10 ms 周期轮询按键，HID 报告最终由输入子系统发布为事件

着，它必须看到断开错误而不是继续读到一个已经不属于自己的设备。

channel 7 怎么保证自己活着。 Hub 的 interrupt-IN 是有周期要求的：`dwc2_hub_interrupt_start()` 提交 ch7 时按当前 `HFNUM` 的奇偶选择 `HCCHAR.ODDFRM`，把事务安排在下一个 USB frame；同时在 `dwc2_hub_wq` 上用 `mod_delayed_work()` 登记一个 `bInterval` 期限的看门狗。正常的 NAK 完成 IRQ 会把这条期限往后推；如果控制器复位后的第一笔事务根本没有产生完成 IRQ，期限到期就会重新启动 ch7。它只恢复 Hub 的 interrupt URB，不做任何 `GET_STATUS`，因此不是退回旧的端口轮询。新一代监控还会先等一个 `bInterval` 再首次提交，确保触发重枚举的那只旧 URB 的 completion 已经完全退出。下面这三行里后两行各只打印一次，合起来正好证明 delayed work 和 DWC2 host channel 两段链路都在运行：

```
dwc2: event-driven hub hotplug v2 scheduled on channel 7
dwc2: hub ch7 scheduler alive active=0 urb-state=0
dwc2: hub ch7 first IRQ hcint=... hctsiz=...
```

一次真机热插 port 3 键盘时，连接位图从 `0x2` 变为 `0xa`：`0x2` 表示 port 1 已连接，`0xa` 则表示 port 1 和 port 3 已连接，位图的 bit 号就是 root hub 的端口号，所以新增设备来自 port 3。Hub interrupt URB 通知 worker 后，可看到类似日志：

```
dwc2: hub interrupt-IN ep=1 mps=1 interval=12 ports=4
dwc2: hub IRQ change=8 connected=2->a; reprobe
dwc2: hub port 3 status=301 change=1
dwc2: hub port 3 after-reset status=303 speed=low
dwc2: hub port=3 child class=0 vid=2a7a pid=8a47 mps=8
input: USB HID Boot Keyboard registered as event0
usbkbd: IRQ-driven boot keyboard ready ep=1 mps=8 interval=10
```

`status=0x301` 表示设备已连接、端口已供电且设备为低速；reset 后的 `0x303` 还包含端口使能状态。若插入的是 Boot Mouse，USB core 会绑定鼠标驱动并注册相应 input 设备。枚举和 probe 日志只证明新设备已绑定、接收 URB 已提交；要证明实际按键路径恢复，还应看到 channel 6 的 completion 和首份有效 HID report。

拔出后的短暂 `ch6 error` 是正常窗口。 键盘刚拔掉、Hub 的状态端点还没把变化报上来之前，常见这几行：

```
dwc2: keyboard ch6 error intr=82 hctsiz=... hcsplt=...
usbkbd: channel 6 transfer error; rearming
```

`0x82` 是 `CHHLTD | XACTERR`：ch6 仍在访问一个已经离线的低速端点，这在“Hub 状态端点的下一个周期到来之前”是必然窗口，不是键盘驱动的残留问题。判断依据是后续日志：只要接着出现 `hub IRQ change` 和 `... event0 disconnected`，remove 生命周期就已经收敛。反过来，如果只有 `ch6 error` 而没有 Hub 的变化事件，那要查的是 ch7 那条链路。

DMA 缓冲的所有权。 插回键盘时如果连续打印

```
dwc2: control data failed req=0 addr=1 result=-1
```

其中 `req=0, addr=1` 是事件 worker 向根 Hub 发端口 `GET_STATUS`，并不是周期拓扑轮询出错了。这里的根因很隐蔽：早期实现把 4 字节的端口状态数组放在 worker 的内核栈上，又直接把它交给 DWC2 做 DMA。缓存维护按 64 字节 cache line 向外取整，*invalidate* 时会把同一条 cache line 里的其它局部变量一起覆盖掉；而且这样一个短生命周期的栈对象本来就不该成为 DMA 目标。现在端口状态用独立的、静态存续且 64 字节对齐的 `dwc2_hub_port_status[64]`，控制传输仍只接收前 4 字节；Hub 状态位图、端口状态缓冲和枚举用的 `ctrl_buf`

三者彼此分离，DMA 与缓存的归属才不会互相污染。这也是本章反复出现的那条规则的又一次兑现：交给 DMA 的缓冲区必须由传输的 owner 独占，并且活得比传输久。

变化之后的恢复过程是这样的：HCD 先注销那个唯一的发布用 `usb_child`，停止并屏蔽相关 channel、清理待处理中断，然后重新初始化 DWC2 并扫描 Hub。新设备按地址 0 开始枚举，读取 Device Descriptor 前 8 字节确定 EP0 最大包长，设置 USB 地址、读取配置描述符并选择 Boot Keyboard 或 Boot Mouse 接口；随后 USB bus 匹配驱动，执行 probe，重新注册 input 设备并提交对应 HID URB。对低速键盘，地址路由还需记录父 Hub、端口和速度，以便 EP0 与 channel 6 的传输正确使用 split transaction。

与 Linux 还差什么。 把这套驱动和真正的 Linux USB 栈对比，剩下的差距是明确的几条：

- **重枚举的粒度**。Linux 只增删发生变化的 port 和 device；这里的一次拓扑变化会注销旧设备、停止并屏蔽相关 channel，然后重新初始化整个 DWC2。嵌套 Hub 的下游端口变化甚至不会出现在根 Hub 的连接位图里。
- **host channel 的分配**。ch7 固定给 Hub 的 interrupt URB；ch6 不再独占某一只 HID 数据 URB，而是由 `interrupt_rx[DWC2_INT_REQUESTS]` 数组在多个 interrupt-IN 请求之间轮转仲裁（同一时刻只有一个请求拥有 ch6）。这让键盘与鼠标可以共处一只 channel，但它是时分复用，不是 Linux 那种按设备/端点动态分配多 channel 的通用调度。
- **周期调度表**。空闲 NAK 已经用 `retry_work` 加 `dwc2_periodic_wq` 按 `bInterval` 重排，不再占着公共 worker 自旋；但 10 ms 的 delayed-work 粒度只适合这只低速键盘，通用的 HCD 需要以 USB frame/SOF 为不同端点各自维护周期表。
- **硬中断里的忙等**。为了满足 split 的 125 μs microframe 期限，快速路径最多在 IRQ 里等 500 μs 并重启 channel。Linux DWC2 用更完整的 SOF/periodic 状态机把这部分忙等压得更低。
- **多设备发布**。Boot HID 接口现在按接口逐个发布独立的 `usb_device`（`usb_hid_child[]`，最多 `DWC2_HID_CHILDREN=4` 个），各自保存地址、接口三元组与拔出生命周期，因此复合接收器的键盘接口与鼠标接口能被不同驱动分别认领。非 HID 的 CDC/bulk 设备（如 MT7601U、usbnet）仍共用单个 `usb_child`，与 HID 子设备尚未统一成一张按 Hub 端口排列的数组。

9.2 复合 HID 接口与鼠标驱动自动装载（真机日志 046d:c534）

7.4.11 节的那台接收器是 low-speed、单键盘接口。真机上更常见的是 Logitech 无线接收器 `046d:c534` 这类 **full-speed 复合 HID**：同一个 USB 地址上接口 0 是 Boot Keyboard（`3/1/1`）、接口 1 是 Boot Mouse（`3/1/2`）。这一节沿一份真实串口日志把“插入—split 枚举—双接口分别登记—键盘即时绑定—鼠标经 MODALIAS 从 ext2 根自动装载 `.ko`—拔出触发整树重枚举”走一遍，并说明 `dwc2.c` 的分接口发布与 `module.c` 的自动装载链在这里如何被验证。

拓扑与速度。 连接位图 `connected=2->a`（bit1|bit3）说明新增设备来自 port 3；复位后 `status=103` 的速度域是 **full-speed**。这是与低速键盘最容易混淆的一点：full-speed 设备在高速 Hub 之后同样要经 TT 做 split 事务，但 `HCCHAR` 的 `LSPDDEV` 位**不置**（只有 low-speed 才置）。表 9.1 用本日志的真实寄存器值把这条规则摊开。

一次 EP0 完成事务的 SSPLIT/CSPLIT 时序，与图 7.18 形状相同（只是本例是 full-speed），日志里的 `hfnum` 记录了两拍的帧号；interrupt-IN 阶段的 `dwc2 ch6 final p=0/1 ...` 打印则显示 ch6 在键盘与鼠标两只 URB 之间轮转：`p` 是 `req->complete_split`（0=SSPLIT、1=CSPLIT），`issue/ssack/age` 分别是发出帧、TT 接受 SSPLIT 帧与距发出的帧数，是 CSPLIT 有效窗口的判据。

分接口发布。 `dwc2.c` 的枚举在 `SET_CONFIGURATION` 后遍历整段配置描述符，对每个带 interrupt-IN 端点的 Boot HID 接口各建一个 `usb_device`（`usb_hid_child[]`），逐个 `usb_device_register()`。日志因此出现两行：

表 9.1: 鼠标接收器（046d:c534）日志中的报文解码

样本	含义
HCSPLT=0x8000c083	SPLTENA、XACTPOS=ALL、Hub 地址 1、端口 3；COMPSPLT= 0 即当前是 SSPLIT（错误行里的 0x8001c083 则 COMPSPLT= 1，为 CSPLIT）
HCCHAR=0x810c8808	地址 4、端点 1、IN、interrupt 类型、最大包长 8，LSPDDEV= 0：full-speed
HCINT=0x22	ACK | CHHLTD：TT 接受 SSPLIT
HCINT=0x23	XFERCOMPL | ACK | CHHLTD：CSPLIT 取回完整数据
HCINT=0x0a	STALL | CHHLTD：control status failed req=10 的 STALL（req=10=0x0a=SET_IDLE）
HCINT=0x82	XACTERR | CHHLTD：拔出前 HID ch6 error

```
dwc2: HID boot keyboard intf 0 handed to usb bus
dwc2: HID boot mouse   intf 1 handed to usb bus
```

接口 0 的 3/1/1 命中内置 usbkbd，同步 probe 并占用 ch6；接口 1 的 3/1/2 没有任何已注册驱动匹配（内置键盘只认 protocol=1），于是设备 core 走未匹配分支：用 bus 的 modalias() 生成别名并 request_module()。

表 9.2: Boot Mouse 的 MODALIAS 与 HID control、报告报文

项	内容
MODALIAS	usb:c03s01p02（class/subclass/protocol 的十六进制）；静态 alias 表映射到 usbmouse.ko
SET_PROTOCOL	SETUP：0x21 0x0b，wValue=0，wIndex=interface，wLength=0；失败才放弃 probe
SET_IDLE	SETUP：0x21 0x0a，同上；失败不致命
Boot Mouse 报告	4 字节：byte0 按钮位图（左/右/中），byte1 X、byte2 Y、byte3 滚轮（均为 signed char）；actual_length>=3 即接受
输入事件	按钮变化 input_report_key(BTN_*)；位移 input_report_rel(REL_X/Y/WHEEL)；末尾 input_sync()

自动装载链. 从设备注册到鼠标驱动绑定，跨越 USB core、设备 core 与模块加载器：

```
usb_device_register(接口1)  ->  device_register()  ->  无驱动匹配
  ->  usb_modalias() = "usb:c03s01p02"
  ->  request_module(alias)                    (见 device.c)
       ->  命中静态 alias 表 ->  queue_work(kmod_wq, slot)
            ->  kmod_run() ->  kmod_load("/lib/modules/usbmouse.ko")
                 ->  begin_op(); namei(); ilock()
                 ->  file_inode_read_locked(ip, ...)          % 经 inode->i_fop->read_at 分派
                 ->  load_module_image(): 模块区分配、符号解析、RELA 重定位
                 ->  run init: usbmouse_driver_init()
                      ->  init_workqueue(usbmouse_wq)        % 日志 "usbmouse_wq worker=0 pid=17"
                      ->  usb_register_driver() -> driver_register()
                           ->  重扫已登记设备 -> usb_match(接口1) 命中
                           ->  usbmouse_probe() -> usb_submit_urb()
                                ->  dwc2_submit_urb() -> dwc2_interrupt_schedule_locked()
                                     若 ch6 空闲则 rx_start，否则排入 interrupt_rx[] 等轮转
```

两个要点。其一，driver_register() 会重新遍历总线上**已存在**的设备，所以“设备先登记并异步排队装载、驱动后注册”的次序仍能绑定接口 1，无需手动 insmod。其二，kmod_load() 里读 .ko 必须走 file_inode_read_locked()，由 inode->i_fop->read_at 做 VFS 分派，不能用经典 xv6fs 的 readi()：readi() 的 bmap()->getsb() 假设设备带 xv6 超级块，在 ext2 根上会直接 panic: getsb: device has no xv6 filesystem。本日志的验收点正是这行：

```
request_module: loading /lib/modules/usbmouse.ko
```

```
module: loaded usbhid-mouse (1808 bytes)
```

出现这两行、且不再有 getsb panic，说明方案 A 的分接口发布、MODALIAS 自动装载与 ext2 上的模块读取在真机跑通。

本日志尚未证明的部分. 按“只承认证据”的原则，这次链路证明了**自动装载**，却没有证明**鼠标数据端到端接收**：全程未出现 `usbmouse: IRQ-driven boot mouse ready`，也没有 `usbmouse: first report`；拔出前是连续四次的 `dwc2: HID ch6 error intr=82 (XACTERR | CHHLTD)` 与 `usbkbd: channel 6 transfer error; rearming`。也就是说 full-speed 复合设备在共享 ch6、经 TT 的 split interrupt-IN 调度仍未收敛，键盘与鼠标争用同一 channel 的时序是主要嫌疑；拔出仍触发 `connected=a->2` 的**整棵拓扑重枚举**，而非只对 port 3 局部增删。下一步诊断应在 `usbmouse_probe()` 各返回前打印失败的 control 请求，并把 ch6 错误路径的 `HCSPLT/HCTSIZ/HCCHAR` 一并 dump，以区分是 SET_PROTOCOL 被 STALL、还是 interrupt-IN 的 CSPLIT 窗口错过。

后续定位：probe 为何失败. 之后在 QEMU 中用两个独立的 HID 设备（`-device usb-kbd -device usb-mouse`）复现，得到了确定的结论，问题不在 ch6 调度，而在三处与 USB 无关的代码：

1. **模块重定位.** `apply_rela()` 处理 `R_AARCH64_ADR_PREL_PG_HI21` 时页掩码写成 `~0xff`（应为 `~0xfff`），写 immhi 时掩码写成 `0x001ffffe`（应为 bit[23:5] 的 `0x00ffffe0`，旧值会清掉 Rd 低位，把 `adrp x2` 变成 `adrp x0`），`ADD_ABS_LO12_NC` 的 imm12 掩码也错了（应为 `0x003ffc00`）。结果是模块中字符串常量指针为 NULL、寄存器被改写，`input_register_device()` 因 `dev->name == 0` 返回失败。
2. **slab 对象未清零.** `input_allocate_device()` 复用的对象带着上一个设备的 `registered/disconnected` 标志；`evdev_open()` 的 `head/tail` 也是旧值。现在分配后先清零。
3. **挂根之前读 `.ko`.** kmod worker 是内核线程，调度器一启动就运行，早于 `forkret()` 中的 `prepare_namespace()`；开机即插鼠标会在挂根前 `namei()`，同样落入 getsb panic。`kmod_load()` 现在先调用 `wait_for_rootfs()` 睡眠到根就绪。

此外，`dwc2` 原先在 hub 上找到第一个 HID 设备就停止扫描，分开插的键盘与鼠标只有一个可用；现在由 `dwc2_publish_hid()` 为每个端口上的 HID 设备发布接口。修正后 QEMU 中出现：

```
input: USB HID Boot Mouse registered as event1
usbmouse: IRQ-driven boot mouse ready ep=1 mps=4 interval=10
```

Pi 3B 真机上的鼠标（500 Hz 全速设备）也完成了 probe 并收到首个报告：

```
usbmouse: IRQ-driven boot mouse ready ep=2 mps=20 interval=2
usbmouse: first report buttons=0 x=0 y=1 wheel=0
```

热插拔让设备和驱动可以在运行中来去，而 `.ko` 能被按需装载，前提是根文件系统上的 `/lib/modules` 可靠可读——上面日志里的 getsb panic 正是这一点出了问题。下一章就深入这块根文件系统背后的 ext2 驱动，看它怎样读写磁盘、怎样在断电后恢复一致。

第十章 可选工作

本章内容

- 为什么要在 xv6 里读写 ext2，分区与日志放在哪里
- ext2 的磁盘结构：superblock、块组、inode、目录
- 从文件偏移到块号：直接块与三级间接块
- 接入 VFS：按 inode 打开、孤儿 inode
- 持久化：`sdflush()` 写屏障
- 外置事务日志 xjournal：格式、提交协议、崩溃恢复
- 源码组织与编译，用 WAL 程序和随机断电做测试
- 在 MBR 卡上用 `fdisk` 分出 ext2 分区，用 `mke2fs` 格式化
- 安装到真机，以及一次“旧内核”问题的调试

第 2 ~ 8 章是系统的主线：没有它们，内核就启动不了、跑不了程序、用不了网络。本章收录的是主线之外的工作。它们不决定系统能否启动，但能把某个子系统做得更完整，也适合作为读者自己动手的练习。第一项是 ext2 移植：让 xv6 能可靠地读写一块 Linux 也认识的分区（第 6 章讲了它怎样成为根文件系统，本章讲它怎样读写磁盘），并在断电后保持一致；从驱动原理、编译测试，一直到给真卡分区、格式化和上机调试。代码在 `kernel/fs/ext2/ext2.c`、`kernel/fs/ext2/xjournal.c` 和 `user/waltest.c`，说明文档是 `doc/readme/readme_ext2.md`。

10.1 ext2 移植

10.1.1 动机与分区布局

xv6fs（第 6.3 节）只有 xv6 自己认识，开发机上的 macOS 或 Linux 无法直接打开它。要在两边交换文件，需要一种双方都懂的格式。FAT32 启动分区可以，但它只支持根目录、没有权限位，平时也不该频繁写。ext2 是 Linux 最经典的文件系统，结构简单、文档齐全，非常适合在教学内核里实现。

第 6.5 节介绍 VFS 时，ext2 后端还只能读：只支持直接块和一级间接块，默认按只读挂载。本节把它补成一个可写的后端，并解决可写之后立刻出现的问题：写到一半断电怎么办。

ext2 放在 SD 卡的第三个分区。它最初挂在 `/mnt/ext2` 当数据卷，根文件系统仍是 p2 上的 xv6fs；可读写、可恢复之后，它在 10-07 起成为根文件系统（`root=/dev/mmcblk0p3`，第 6.8.9 节），p2 则保留 xv6fs 镜像（图 10.1）。ext2 本身没有日志，这里给它配了一个外置的事务日志，放在 p2 尾部：p2 有 400 MiB，xv6fs 镜像只用前 32 MiB（`FSSIZE` = 32768 块，即 65536 扇区），后面的空间本来就闲着。本章后半的安装与调试记录（10.1.10 ~ 10.1.12 节）大多来自数据卷阶段，日志里因此还能看到 `/mnt/ext2`。

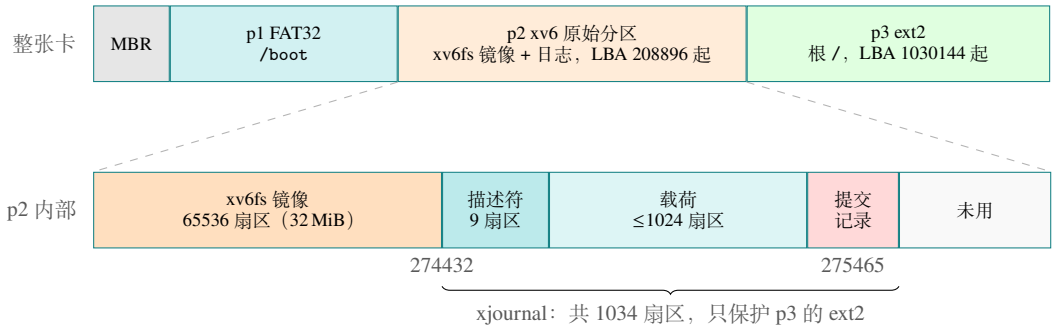


图 10.1: ext2 与外置日志在 SD 卡上的位置。下排数字是真机上的绝对 LBA：日志起点 = p2 起点 208896 + 65536

日志的位置在挂载时算出。`ext2init()` 只登记文件系统类型；`ext2_fill_super()`（第 6.8.9 节）打开分区、

读完超级块以后调用 `ext2_journal_init()`，向根块设备层（第 6.7 节）要 xv6 分区 p2 的块设备，再跳过 xv6 镜像：

```
struct blkdev *jbd = rootdev_xv6_part();                    // p2
struct blkdev *home = blkdev_get(fs->dev);                  // p3
uint32 jstart = FSSIZE * (BSIZE / SECTOR);                  // 65536
if(jbd && jbd->disk && jbd->nsect > jstart && home){
  if(xj_init(jbd, jstart, jbd->nsect - jstart, home) < 0)   // 先恢复日志
    return -1;                                              // 恢复失败：拒绝挂载
  fs->journal = 1;
} else
  printf("ext2: no raw xv6 root partition; external journal disabled\n");
```

`xj_init()` 在 ext2 被登记进 `supers[]`、对外可见之前检查并重放日志，所以之后任何人读到的位图和目录都已经是一致的。如果提交记录有效、但描述符、载荷、校验和或写回原位任何一步失败，`xj_init()` 返回 −1，整个挂载失败，绝不把没恢复好的 ext2 暴露出去。QEMU 裸镜像没有 p2，这时日志关闭，ext2 退回经块缓存的同步写。在开发机上重写 p2 时（第 6.3 节）只写 `fs.img` 的前 `FSSIZE` 块，不会碰到日志区。

10.1.2 ext2 的磁盘结构

ext2 把分区切成大小相同的块（1、2 或 4 KiB），再把连续的块分成若干块组。每个块组都有自己的位图和 inode 表，这样文件的数据块可以和它的 inode 放在同一个组里，减少磁头移动——这是为机械硬盘设计的，但结构本身对 SD 卡同样适用。图 10.2 是 4 KiB 块时第 0 组的布局。

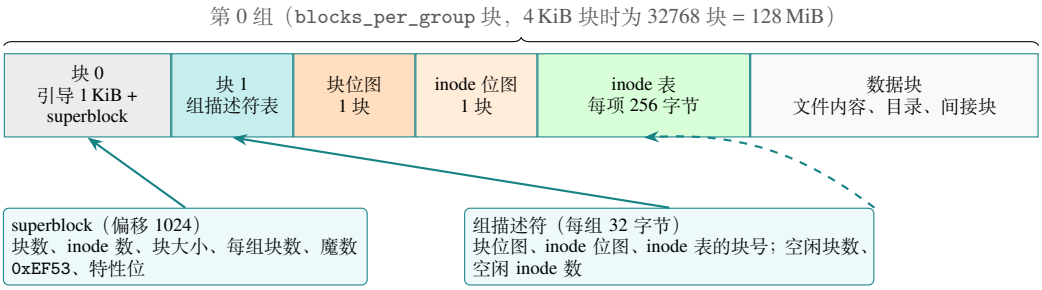


图 10.2: ext2 第 0 块组的布局。其他块组结构相同；开启 `sparse_super` 时只有第 0、1 和 3、5、7 的幂次组保存 superblock 与描述符表的备份

`setup_ext2()` 从 superblock 读取表 10.1 中的字段，并在用之前逐个检查。最重要的是三组特性位：`compat` 中的 `has_journal` 说明这是 ext3/ext4，`incompat` 中除 `filetype` 以外的任何一位（如 extent、64 位块号）都意味着本驱动看不懂磁盘布局。遇到这些就拒绝挂载，而不是把一个 ext4 分区当成 ext2 去写坏它。

表 10.1: `setup_ext2()` 读取的 superblock 字段

偏移	字段	用途
0 / 4	`inodes_count` / `blocks_count`	越界检查；块数 × 块大小不能超过分区
12 / 16	空闲块数 / 空闲 inode 数	分配、释放时加减
20	`first_data_block`	1 KiB 块时为 1，否则为 0
24	`log_block_size`	块大小 = 1024 ≪ 该值
32 / 40	每组块数 / 每组 inode 数	块号、inode 号到组号的换算
56	`magic`	必须是 `0xEF53`
58	`state`	第一次写时清除 `VALID_FS`
88	`inode_size`	动态版本里一般为 256
92 / 96 / 100	`compat` / `incompat` / `ro_compat`	拒绝 journal、extent、64bit 等

inode 与目录。 inode 号从 1 开始，2 号是根目录。找 n 号 inode 时，先算组号 $g=(n-1)/$每组 inode 数 和组内序号 $i=(n-1)$ mod 每组 inode 数，读第 g 个组描述符拿到 inode 表的块号，inode 就在表内偏移 $i\times$inode_size 处：

```
gd_off = (uint64)e2.gd_block * e2.block_size + group * e2.desc_size;
diskbytes(gd_off, inodebuf, e2.desc_size);
table = le32(inodebuf + 8);                                  // inode 表块号
off = (uint64)table * e2.block_size + (uint64)index * e2.inode_size;
diskbytes(off, inodebuf, e2.inode_size);
ip->mode  = le16(inodebuf);        ip->links = le16(inodebuf + 26);
ip->size  = le32(inodebuf + 4);
if((ip->mode & EXT2_S_IFMT) == EXT2_S_IFREG)                 // 普通文件的高 32 位
  ip->size |= (uint64)le32(inodebuf + 108) << 32;
for(int i = 0; i < 15; i++) ip->block[i] = le32(inodebuf + 40 + i*4);
```

`off` 必须用 64 位计算：一个几 GiB 的分区里，后面块组的 inode 表很可能位于 4 GiB 之后，32 位乘法会悄悄溢出，读到另一个位置的“inode”。文件大小同理，高 32 位放在 inode 偏移 108 处。

目录是普通的数据块，里面是一串变长目录项：inode 号（4 字节）、本项长度 `rec_len`（2 字节）、名字长度和文件类型（各 1 字节），然后是名字。删除一项时不移动后面的数据，而是把它的 `rec_len` 并入前一项；新增时找一个 `rec_len` 比实际需要大的项把它切开。`e2dirlookup()`、`e2diradd()`、`e2dirremove()`、`e2dirreplace()` 分别实现查找、插入、删除和改指向。

10.1.3 从文件偏移到块号

inode 的 `block[15]` 是一棵最多四层的树（图 10.3）：前 12 项直接指向数据块，第 13 项指向一个装满块号的间接块，第 14、15 项分别多套一层和两层。4 KiB 块时每个间接块能放 $n=1024$ 个块号，所以一个文件最多有

$$12+n+n^2+n^3 = 12+1024+1\,048\,576+1\,073\,741\,824 \text{ 块} \approx 4\text{ TiB}.$$

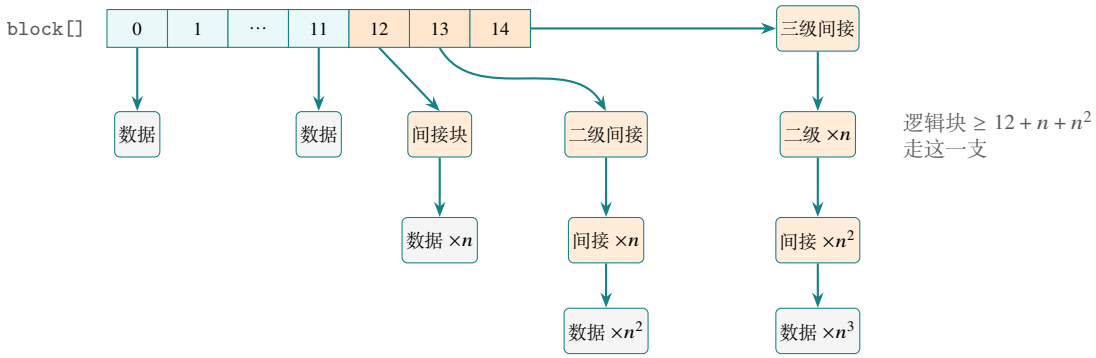


图 10.3: ext2 inode 的块指针树。块号为 0 表示空洞：读时返回全 0，写时才分配

`fileblock()` 按逻辑块号依次减去每一层能覆盖的块数，落到哪一层就沿哪一支往下读：

```
static uint32 fileblock(struct einode *ip, uint64 logical) {
  uint64 n = e2.block_size / 4;
  if(logical < 12) return ip->block[logical];
  logical -= 12;
  if(logical < n){                                           // 一级间接
    if(ip->block[12] == 0 || readblock(ip->block[12], ptrbuf[0]) < 0) return 0;
    return ptrbuf[0][logical];
  }
```

```
  logical -= n;
  if(logical < n*n){ ... 读 block[13]，再读 ptrbuf[0][logical / n] ... }
  logical -= n*n;                                          // 三级间接
  ... 读 block[14]，再依次取 logical / n²、(logical % n²) / n、logical % n ...
}
```

三层用三个独立的 4 KiB 缓冲 `ptrbuf[0..2]`，它们属于每个 ext2 实例（`struct ext2_fs`），由 `ext2_fill_super()` 用 `kalloc()` 分配，而不是局部数组——内核栈只有 4 KiB，放不下其中任何一个（第 7 章的摄像头调试记录里有过同样的教训）。同一实例的操作都持着该实例的锁，所以这些缓冲是安全的。

写的方向由 `set_fileblock()` 和 `get_or_alloc_fileblock()` 完成：路径上缺哪一层间接块就分配哪一层。截断时 `clear_fileblock()` 释放数据块，再检查它所在的间接块是否已经全为 0（`ptr_empty()`），是就把间接块也释放掉，逐层向上。`waltest large` 在偏移 5 GiB 处写一个字节：$5\,\text{GiB}/4\,\text{KiB} = 1\,310\,720 > 12 + n + n^2$，正好走到三级间接这一支，文件只占 4 个块。

10.1.4 分配与释放

分配一个数据块要改三处：块位图里的一位、组描述符里的空闲块数、superblock 里的空闲块数。`alloc_block()` 从首选的组开始找第一个空闲位：

```
for(uint32 bit = 0; bit < count; bit++){
  if((blockbuf[bit >> 3] & (1U << (bit & 7))) != 0) continue;
  if(freed_in_tx(first + bit)) continue;        // 本事务刚释放的块，暂不复用
  blockbuf[bit >> 3] |= 1U << (bit & 7);
  writeblock(bitmap, blockbuf);                 // 位图
  put16(gd + 12, le16(gd + 12) - 1);
  writegd(group, gd);                           // 组描述符
  adjust_super_count(12, -1);                   // superblock
  *out = first + bit;
  memset(blockbuf, 0, e2.block_size);
  writeblock_data(*out, blockbuf);              // 新块清零，直接写
  return 0;
}
```

两处细节和后面的日志有关。第一，`freed_in_tx()`：一个块如果在同一个事务里刚被释放，就不能立刻再分配出去并原地改写——事务一旦中止或者断电，释放会被撤销，这个块又回到原主人手里，内容却已经是别人的了。第二，新块直接清零而不经过日志：在事务提交之前，没有任何 inode 指向它，写什么都不会被看见。

inode 的分配（`alloc_inode()`）和释放（`free_inode()`）是同样的三步，只是改的是 inode 位图和空闲 inode 数；目录还要维护组描述符里的目录计数。

10.1.5 接入 VFS：按 inode 工作

第 6.5 节的 VFS 最初是“按路径”的：每次 `read`、`write` 都把路径传给后端重新查找。对 ext2 这会带来两个问题：每次读写都要从根目录走一遍；更糟的是，文件被重命名或删除以后，已经打开的描述符会突然指向别的文件，或者失效。Unix 的语义是：打开的是 inode，而不是名字。

过渡阶段的办法是给 `vnode_ops` 加一组句柄操作（`open/release/hread/hwrite/hstat/htruncate`），ext2 的句柄就是 inode 号。改成统一 VFS 以后，这组句柄操作连同 vnode 一起删掉了：ext2 以 bootstrap `ext2_iops` 接入 inode 缓存，第一次 `ilock()` 读出磁盘类型后，再挂接目录或普通文件各自的 `inode_ops/file_operations`：

```
static struct inode_ops ext2_iops = {        // bootstrap
```

```
  .read_inode = e2_read_inode,   // 读磁盘 inode，并在 fs->open[] 里占一个引用
  .evict      = e2_evict,        // 最后一个 iput()：ext2release()，孤儿在此回收
};
static struct inode_ops ext2_dir_iops = {
  .read_inode = e2_read_inode, .evict = e2_evict,
  .lookup     = e2_lookup,       // 查目录块，返回 iget(dev, ino)
  .create     = e2_create,  .unlink = e2_unlink,  .rename = e2_rename,
};
static struct inode_ops ext2_file_iops = {
  .read_inode = e2_read_inode, .evict = e2_evict, .truncate = e2_truncate,
};
static const struct file_operations ext2_file_fops = {
  .fread = generic_file_fread, .fwrite = generic_file_fwrite,
  .read_at = e2_file_read_at, .write_at = e2_file_write_at,
  .fsync = e2_file_fsync, .poll = generic_file_poll,
};
```

原来按路径的实现拆成了“核心函数 + 外壳”：`create_core()`、`mkdir_core()`、`unlink_core()`、`rename_core()`、`readdir_core()` 以父目录的 `einode` 和名字为参数，`e2_*` 操作直接调用它们；按路径的外壳只留给 `ext2read`、`ext2readdir` 两个老系统调用。每个实例的 `open[]` 表仍然存在，只是不再由 `open/close` 驱动，而由 inode 缓存驱动：`ilock()` 第一次填充 inode 时占一个引用，`iput()` 放掉最后一个引用时释放它。

有了引用计数，就能实现“删除仍被打开的文件”：`unlink` 或被 `rename` 覆盖时，目录项立即消失；如果 inode 的链接数降到 0 而它还被打开着，`drop_link()` 只把它标成孤儿，块不释放。旧描述符照常读写原来的数据，最后一次 `close` 让 inode 的引用降到 0，`e2_evict()` 调用 `ext2release()`，这时才在一个事务里截断并释放 inode。

`rename` 也补全了 POSIX 语义：目标已存在时原子地替换它（类型必须相同，目录必须为空）；把目录移进它自己的子树会被 `is_ancestor()` 拒绝——否则这棵子树会从根上脱落，形成一个环。

锁。 同一个 ext2 实例的操作由它自己的睡眠锁 `fs->lock`（`struct ext2_fs` 里，每个设备一把）串行化，不同分区互不阻塞。它与 inode 的睡眠锁有固定的顺序：先 inode 锁、后 `fs->lock`。`e2_create()` 在目录里加了新项以后，要先放掉 `fs->lock`，再加锁父目录 inode 刷新它的大小和链接数，否则会和持有目录 inode 锁再进入 ext2 的路径解析形成反序。最初用的是自旋锁，但 SD 卡是轮询 PIO，一次 `fsync` 最长可能等 5 秒，持自旋锁期间 CPU 一直关着中断，键盘和网络都会卡住。换成睡眠锁后，等待的进程会让出 CPU。

10.1.6 持久化：sdflush() 写屏障

`sdsector(lba, buf, 1)` 返回时，CMD24 的数据已经传给卡，但卡可能还在内部把它编程进 NAND。这时断电，这个扇区可能是新的、旧的，甚至是坏的。要建立“之前的写都已落盘”这样一个边界，需要单独的屏障：

```
int sdflush(void) {                                          // kernel/drivers/mmc/host/sd.c
  acquire(&sdlock);                                          // 与所有传输互斥
  asm volatile("dsb sy" ::: "memory");
  uint64 deadline = r_cntvct_el0() + 5ULL * r_cntfrq_el0();
  for(;;){
    uint32 status = 0;
    if(send_command(CMD_INDEX(13) | CMD_RSPNS_48 | CMD_CRCCHK_EN | CMD_IXCHK_EN,
                    sd_rca << 16, &status) == 0 &&
       (status & (1U << 8)) && ((status >> 9) & 0xf) == 4)  // READY_FOR_DATA, TRAN 态
```

```
      return 0;                                          // （省略解锁与超时）
    ...
  }
}
```

CMD13 读卡的状态寄存器。第 8 位 READY_FOR_DATA 置位、第 9 ~ 12 位的状态回到 TRAN（4），说明卡已经处理完之前的写，可以接受新命令。因为 sdflush() 持有块设备锁，屏障之后的写也不可能越过它。SD 6.x 规范还定义了一个可选的写缓存，需要专门的 Flush Cache 命令；本驱动没有启用它，所以 CMD13 就是当前支持的卡的持久化边界。

xv6 原生日志（第 6 章）也在四个位置用上了这个屏障：日志内容 → flush → 提交头 → flush → 写回原位 → flush → 清提交头 → flush。下面的 xjournal 用的是同一个思路。

10.1.7 外置事务日志 xjournal

要解决的问题。 一次 rename 要改新目录的数据块、旧目录的数据块，覆盖已有文件时还要改目标 inode、位图和计数，一共可能十几个扇区。这些写落在卡上的不同位置，断电可能发生在任意两次之间，留下“两个名字指向同一个 inode”或者“块已分配但没人用”的状态。ext2 本身对此无能为力，只能事后让 e2fsck 修。

xjournal 的思路与 xv6 原生日志相同：先把一次操作要写的所有元数据扇区完整地写到一块独立的区域，写一个“提交”标记，然后才写回原位。断电后只要看到提交标记就重放一遍，没看到就当这次操作没发生。区别在于它工作在 *512 字节扇区一层*，不依赖 xv6 的块缓存，也不知道 ext2 的格式，所以叫“外置”。

接口。 客户（这里是 ext2）在每个修改型操作前后调用 xj_begin() 和 xj_commit()；中间的写调用 xj_write()，它不写卡，而是把扇区存进事务缓冲；读调用 xj_read()，命中缓冲时返回未提交的新内容，这样同一个操作后面的步骤能看到前面的修改。缓冲是 128 个物理页（1024 个扇区），启动时分配。ext2 只在两个函数里接入它：

```
static int e2_rsector(uint32 lba, void *buf) {
  if(e2.journal && xj_read(lba, buf))                    // 先查事务缓冲
    return 0;
  return sdsector(lba, buf, 0);
}

static int e2_wsector(uint32 lba, const void *buf) {
  if(e2.journal && xj_active() && (!e2.direct || xj_contains(lba)))
    return xj_write(lba, buf);                           // 元数据：进事务
  return sdsector(lba, (void*)buf, 1);                   // 文件数据：直接写
}
```

ordered 模式。 文件内容不进日志，由 writeblock_data() 置 e2.direct 后直接写回原位。这借鉴的是 ext3 的 data=ordered：数据先落盘，引用它的元数据（inode 里的块号、文件大小）后提交。新分配的块在提交前没人能看到，所以永远不会出现“文件里有一段垃圾”；覆盖已有数据则不保证原子——这与 Linux 上的语义一致，需要原子更新的程序应该自己用 WAL（见下一小节的 waltest）。这样做让日志只承载元数据，一个事务 1024 个扇区的容量对绝大多数操作都绰绰有余。

磁盘格式与提交协议。 日志区分三部分（图 10.1）：9 个扇区的描述符（魔数 XJD1、序号、扇区数和每个扇区的原位 LBA，$16 + 4 \times 1024$ 字节恰好需要 9 个扇区）、最多 1024 个扇区的载荷，以及 1 个扇区的提交记录（魔数 XJC1、序号、扇区数、整个事务的校验和，再加一个对这 16 字节自身的校验和）。校验和用 32 位 FNV-1a，覆盖描述符和全部载荷。提交分四步，每步后面都有一次 sdflush()（图 10.4）：

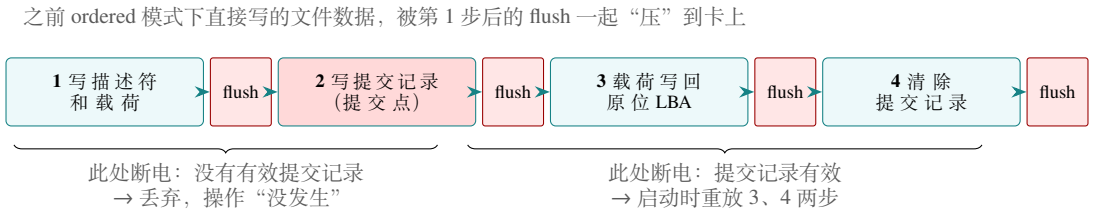


图 10.4: xjournal 的提交协议。四次 `sdflush()` 把四步严格排序；提交点之前和之后的断电有两种确定的结局

```
int xj_commit(void) {                                   // kernel/fs/ext2/xjournal.c, 节选
  ...
  uint32 sum = txsum(count);                            // 描述符 + 载荷的 FNV
  // 1. 描述符和载荷；这次 flush 也保证此前直接写的数据已经落盘
  for(uint32 i = 0; i < XJ_DESC; i++)  sdsector(xj.start + i, desc + i*512, 1);
  for(uint32 i = 0; i < count; i++)    sdsector(xj.start + XJ_DESC + i, slot(i), 1);
  if(sdflush() < 0) goto fail;
  // 2. 提交记录：这个扇区落盘的那一刻，事务就生效了
  make_commit(seq, count, sum);
  if(sdsector(xj.start + XJ_DESC + XJ_CAP, xj.commit, 1) < 0 || sdflush() < 0)
    goto fail;
  // 3./4. 写回原位并清除提交记录；失败也没关系，下次启动会重放
  if(write_home(count) < 0 || clear_commit() < 0)
    printf("xjournal: checkpoint of seq=%d incomplete; replay at boot\n", seq);
  return 0;
fail:
  printf("xjournal: commit of seq=%d failed before commit point\n", seq);
  return -1;
}
```

提交点是一个扇区的写入。SD 卡保证单个扇区的写要么完成、要么不生效吗？并不完全保证，所以提交记录自带一个校验和：写坏的提交记录校验不过，等同于没有。

崩溃恢复。 `recover()` 在 `xj_init()` 里执行，顺序是：读提交记录并校验它自身；读描述符，确认魔数、序号和扇区数与提交记录一致；读载荷，重算整个事务的校验和并与提交记录比较；全部通过才把载荷写回原位、清除提交记录。任何一步不符都丢弃——描述符或载荷写到一半时断电，校验和必然对不上。表 10.2 列出了各个断电时刻的结局。

表 10.2: 断电时刻与恢复结果

断电发生在	磁盘状态	启动时
第 1 步期间或之后	原位元数据未动；日志里有半个或一整个事务，但没有提交记录	无事可做，操作没发生
第 2 步期间	提交记录可能是旧的 0 扇区或写坏的扇区	校验失败，丢弃
第 3 步期间	原位一部分是新的、一部分是旧的	重放：`xjournal: replayed seq=N`
第 4 步期间	原位已全部更新，提交记录还在	再重放一次（幂等），结果相同
第 4 步之后	一切都是新的	无事可做

重放是幂等的：载荷是扇区的完整新内容，而不是“加 1”“置位”这样的增量，写一次和写两次结果一样。这让恢复本身被打断也没有关系，下次启动再来一遍即可。

ext2 怎样使用事务。 每个修改型操作的外形都一样，以 `rename` 为例：

```
int ext2rename(char *oldpath, char *newpath) {
  struct ext2_fs *fs = ext2_default();                    // 当前挂着的 ext2 实例
  acquiresleep(&fs->lock);
  tx_begin(fs);                                           // xj_begin(), 清空 freed[]
  int r = tx_end(fs, rename_locked(fs, oldpath, newpath) == 0);  // 成功则提交, 失败则中止
  releasesleep(&fs->lock);
  return r;
}
```

`tx_end(0)` 调用 `xj_abort()`，事务缓冲被直接丢掉，这次操作对磁盘的元数据没有留下任何痕迹。以前 `rename` 中途失败（比如目标目录满了又分配不到新块）会留下“新名字加上了、旧名字没删”的状态，有了事务就不会了。

少数操作会超过一个事务的容量：截断一个几 GiB 的文件要改成千上万个位图扇区。`tx_checkpoint()` 在剩余空间不足 96 个扇区时，先把 inode 以当前已截断到的大小写入事务并提交，再开一个新事务继续。每个分段的终点都是一个一致的状态：inode 的大小与它还引用的块相符，释放掉的块已经回到位图，只是文件还没截到最终长度。

最后，第一次写 ext2 时，`mark_write_session_dirty()` 绕过日志直接清除 superblock 的 `VALID_FS` 位并 flush。非根挂载可以 `umount`：`fs_umount()` 先标记 `unmounting` 挡住新的路径遍历，确认没有打开的文件、当前目录、活动 inode 和子挂载，再调用 `super_block.put_super = ext2_put_super`：用最后一个元数据事务把 `VALID_FS` 置回、flush、关闭外置日志、`binvalidate()`，最后释放超级块和实例。强制断电、根文件系统无法卸载或卸载失败时，接回开发机仍应执行 `e2fsck -f`；挂载时该位原本就是 dirty 的，正常卸载也会保留 dirty。

10.1.8 源码组织与编译

ext2 移植涉及的文件分布在内核、用户程序和构建脚本三处（表 10.3）。除了 `ext2.c` 和 `xjournal.c`，另外几处改动是为了让 ext2 真正可用：块设备层要提供写屏障，VFS 要能按 inode 工作，`exec` 和当前目录要能跨过挂载点（第 6.5.5 节）。

表 10.3: ext2 移植涉及的文件

文件	作用
`kernel/fs/ext2/ext2.c`	superblock 检查、inode 读写、直接到三级间接块、位图分配、目录项、事务封装、打开表与孤儿回收
`kernel/fs/ext2/xjournal.c`	外置事务日志：事务缓冲、四步提交、启动恢复
`kernel/drivers/block/rootdev.c`	`rootdev_xv6_part()` 告诉 ext2 p2 的块设备，日志就放在 xv6 镜像之后
`kernel/drivers/mmc/host/sd.c`	`sdflush()`：dsb 加 CMD13 轮询，作为持久化边界
`kernel/fs/ext2/ext2.c`（VFS 部分）	`ext2_type`（`ext2_get_sb`、`ext2_fill_super`）、bootstrap/目录/文件的 iops 与 fops、`ext2_register_super()`、`ext2_put_super()`：作为统一 VFS 的一种文件系统，可挂在 `/mnt/ext2` 或作根文件系统
`kernel/fs/exec.c`、`kernel/fs/sysfile.c`	`exec` 经 `inode->i_fop->read_at` 读 ELF；`open/read/write` 经 `file->f_op` 分派；`abspath()` 与 `cwdpath`
`user/waltest.c`	WAL 示例与语义、压力、断电测试
`Makefile`	`xjournal.o`、`rootdev.o`；`fs_ext2.img` 与 `install-rpi3-ext2` 目标

编译不需要额外步骤，`make` 会同时生成内核镜像 `kernel/kernel8-xv6_wifi.img` 和带 `waltest` 的根文件系统 `fs.img`。与 ext2 有关的构建变量如下：

```
RPI3_EXT2_DEV ?=              # ext2 分区, 例如 /dev/rdisk4s3; 默认为空, 防止误写整卡
EXT2_IMAGE    ?= fs_ext2.img  # 可选: 在开发机上预先格式化的 ext2 镜像
EXT2_SIZE     ?= 32M
EXT2_LABEL    ?= xv6-linux
```

```
MKE2FS        ?= ...          # 依次查找 PATH、/opt/homebrew 与 /usr/local 下的 e2fsprogs
```

e2fsprogs 在 macOS 上用 Homebrew 安装（`brew install e2fsprogs`）。它是一个“keg-only”包，`mke2fs` 不在默认的 `PATH` 里，所以 `Makefile` 会自己去 `/opt/homebrew/opt/e2fsprogs/sbin` 下找。

开发阶段先在 QEMU 里测试：用宿主机上的分区和格式化工具做一张与真机分区布局相同的 256 MiB 镜像（p1 FAT32，p2 写入 `fs.img`，p3 为 ext2），通过 `-drive if=sd` 挂给 `raspi3b` 机器。断电测试就是在这张镜像上做的（下一小节）。

10.1.9 测试：WAL 程序与随机断电

`user/waltest.c` 是一个极小的“数据库”：数据文件 `data.db` 里存一个值，修改时先写一条 redo 记录到 `redo.wal`。每条记录带魔数、事务号和校验和，提交顺序是：

```
写 redo.wal -> fdatasync -> 写 data.db -> fsync -> 清空 redo.wal -> fsync
```

它同时检验了两件事：`fsync` 是否真的建立了持久化边界，以及文件系统操作的语义是否正确。子命令如表 10.4。

表 10.4: `waltest` 的子命令

命令	检查什么
`init` / `commit 42` / `show`	正常提交与读取
`prepare 123`，断电，`recover`	只持久化 WAL 后断电，重启必须输出 `REDO tx=…value=123`
`fsops`	`mkdir`、`rename`、`unlink`、`ftruncate`、`pread`、`pwrite`
`semantics`	rename 覆盖已有文件；目录不能移入自己的子树；删除仍被打开的文件后，旧描述符仍可读写
`large`	5 GiB 处写一个字节，走三级间接块
`jstress`	无限循环：写文件、`rename` 覆盖、`unlink`，供随机断电使用

断电测试在 QEMU 里做：准备一个与真机分区布局相同的 256 MiB SD 镜像，运行 `waltest jstress`，在随机时刻直接杀掉 QEMU 进程，再从同一个镜像启动。每次重启后，把 p3 从镜像里取出来，在宿主机上运行 `e2fsck -fn`。

```
dd if=sd.img of=p3.img bs=512 skip=215040 count=309248
e2fsck -fn p3.img          # 退出码 0 表示没有任何错误
```

结果：第一轮随机断电 14 次，其中 3 次正好落在“已提交、未写回”的窗口——这时如果不重放，`e2fsck` 会报错（退出码 4）；三次启动都打印了 `xjournal: replayed seq=…`，重放后全部无错误。其余 11 次断电前后都没有错误。拆分出 `rootdev.c` 以后又断电 4 次，1 次重放（`seq=85`，27 个扇区），`e2fsck` 全部通过。

```
xjournal: replayed seq=85 (27 sectors)
xjournal: ready lba=198656 capacity=1024 sectors
ext2: p2 has no ext superblock
ext2: p3 ready rw start=215040 sectors=309248 block=4096 groups=2
```

（QEMU 镜像的 p2 起点是 133120，所以日志起点是 133120 + 65536 = 198656；真机上的数字见第 10.1.12 节。）

10.1.10 给 SD 卡分出 ext2 分区

新卡可以按第 6 章“烧录 SD 卡”一节，用 `diskutil partitionDisk` 一次建好四个分区。本小节记录的是另一种更常见的情况：卡上已经装好 xv6，p2 一直占到卡尾，需要从中腾出一个 p3。

为什么不用 diskutil。 `diskutil addPartition` 只能在 GPT 分区表上新增分区，而树莓派 3 的 Boot ROM 只认 MBR；`diskutil resizeVolume` 也缩不了 p2，因为 macOS 不认识 xv6 的格式。MBR 只是第 0 扇区里的 4 个 16 字节表项，用 macOS 自带的 `fdisk -e` 直接改表项即可：既不移动数据，也不重写分区里的任何扇区。

先看清现状。 这张卡共 250 347 520 个扇区（约 119 GiB）：

```
$ sudo fdisk /dev/disk4
Disk: /dev/disk4        geometry: 15583/255/63 [250347520 sectors]
 #: id  cyl  hd sec -  cyl  hd sec [     start -       size]
 1: 0C  128   0   1 - 1023   3  32 [     16384 -    1048576] Win95 FAT32L
 2: 83 1023   3  32 - 1023 127  63 [   1064960 -  249282560] Linux files*
 3: 00    0   0   0 -    0   0   0 [         0 -          0] unused
 4: 00    0   0   0 -    0   0   0 [         0 -          0] unused
```

p2 虽然占了 119 GiB，xv6 真正用到的只有开头 65 536 个扇区的镜像和紧随其后的 1034 个扇区日志。缩小表项之前，必须确认 p2 真的是 xv6 而不是 Raspberry Pi OS 的 ext4，并做好备份：

```
sudo dd if=/dev/rdisk4s2 bs=512 skip=2 count=1 2>/dev/null | xxd | head -1
# xv6: 以 4030 2010 0080 0000 开头 (FSMAGIC 0x10203040, FSSIZE 32768)
sudo dd if=/dev/rdisk4s2 of=~/xv6fs-p2-backup.img bs=512 count=66570
```

新布局。 p2 的起点不变，只把大小改成 400 MiB；p3 从下一个 2048 对齐的扇区开始，一直到卡尾（表 10.5）。

表 10.5: 加入 p3 之后的分区表

分区	类型	起始 LBA	扇区数	容量	用途
p1	0x0C	16384	1048576	512 MiB	BOOTFS，不变
p2	0x7F	1064960	819200	400 MiB	xv6 根，只缩小表项
p3	0x83	1884160	248463360	118.5 GiB	ext2

1884160+248463360 = 250347520，恰好用到卡尾。p2 的类型改成 0x7F 只是为了标明用途：`rootdev_init()` 优先选 0x7f，`ext2init()` 只尝试 0x83，就不会再去 p2 里找 ext2 superblock。

用 `fdisk -e` 修改。

```
$ diskutil unmountDisk /dev/disk4
$ sudo fdisk -e /dev/disk4
fdisk: could not open MBR file /usr/standalone/i386/boot0: No such file or directory
fdisk: 1> edit 2
         Partition id: 7F
         Do you wish to edit in CHS mode? n
         Partition offset: 1064960
         Partition size: 819200
fdisk:*1> edit 3
         Partition id: 83
         Do you wish to edit in CHS mode? n
         Partition offset: 1884160
         Partition size: 248463360
fdisk:*1> print
fdisk:*1> write
Device could not be accessed exclusively.
A reboot will be needed for changes to take effect. OK? [n] y
```

```
Writing MBR at offset 0.
fdisk: 1> quit
```

两条提示都无害。第一条说找不到 Intel Mac 的引导代码模板 boot0，只有 update、reinit 才会用到它，而这两个命令一个会覆盖引导代码，一个会清空分区表，这里都不能用；edit 和 write 会保留原有的引导代码和磁盘签名。第二条是因为 macOS 仍持有这张卡，回答 y 后把卡弹出、重新插入，系统就会重读分区表，不必重启电脑。

10.1.11 格式化 ext2

p3 必须格式化成本驱动认识的 ext2。直接对整个分区运行 mke2fs：

```
diskutil unmount /dev/disk4s3
sudo /opt/homebrew/opt/e2fsprogs/sbin/mke2fs -F -t ext2 -b 4096 \
  -O filetype,sparse_super,large_file,^has_journal,^extent,^64bit,^metadata_csum \
  -L xv6-linux /dev/rdisk4s3
sudo /opt/homebrew/opt/e2fsprogs/sbin/e2fsck -fn /dev/rdisk4s3
sudo /opt/homebrew/opt/e2fsprogs/sbin/dumpe2fs -h /dev/rdisk4s3   # 查看特性位
```

-O 里的每一项都对应 setup_ext2() 的一条检查（表 10.6）。新版 mke2fs 的默认配置偏向 ext4，所以哪怕指定了 -t ext2，也最好把不支持的特性明确关掉。

表 10.6: mke2fs 特性选项与驱动的对应关系

选项	特性位	驱动的态度
-b 4096	块大小	支持 1/2/4 KiB；4 KiB 时间接块能放 1024 个块号
filetype	incompat 0x0002	支持：目录项里带文件类型
sparse_super	ro_compat 0x0001	支持：只在部分块组保存 superblock 备份
large_file	ro_compat 0x0002	支持：文件大小的高 32 位
^has_journal	compat 0x0004	必须关：有 ext3 日志就拒绝挂载
^extent	incompat 0x0040	必须关：不认识 ext4 extent 树
^64bit	incompat 0x0080	必须关：块号和组描述符都按 32 位解释
^metadata_csum	ro_compat 0x0400	必须关：不维护元数据校验和

不要把 32 MiB 空镜像当作根文件系统。 Makefile 里的 install-rpi3-ext2 目标把开发机上的 fs_ext2.img 用 dd 写进分区。当该镜像不存在时，规则只执行 truncate -s 32M 和 mke2fs，里面只有 .、.. 和 lost+found，适合作为 /mnt/ext2 数据卷，却不能作为根文件系统。它只有 32 MiB：写进 118 GiB 的 p3 之后，superblock 记的仍是 8192 个块，分区的其余空间全部浪费，启动日志里会出现 groups=1。要使用整个分区，应当像上面那样直接对分区运行 mke2fs；已经写了镜像的话，可以用 resize2fs 扩展：

```
sudo /opt/homebrew/opt/e2fsprogs/sbin/e2fsck -f /dev/rdisk4s3
sudo /opt/homebrew/opt/e2fsprogs/sbin/resize2fs /dev/rdisk4s3
```

扩展后应有 248463360/8 = 31 057 920 个块，按每组 32 768 块计共 948 个块组。另外，fs_ext2.img 这个目标只依赖文件名，镜像已经存在时改了 EXT2_SIZE 也不会重新生成，要先删掉旧文件。

可启动的 ext2 根必须使用 install-rpi3-ext2root（第 6.9 节）。它依赖 phony 的 ext2root-image，每次都会重建镜像并用 debugfs 安装 /init、/bin/*、动态加载器、libc 和 /lib/modules 下的 .ko。这个镜像同样只有 32 MiB，要用满 p3，写入后照上面的办法 resize2fs 一次即可。两个安装目标共用同一个文件名，所以 install-rpi3-ext2 有时看似也能启动：那只是碰巧复用了前一次已经填充的镜像；make clean 后再次执行它就会烧入空卷。空卷仍能通过挂载和块缓存自检，但 initcode 的 exec("/init") 随后失败，最终显示 panic: init exiting。

10.1.12 安装与真机调试

下面是 ext2 还作为 /mnt/ext2 数据卷时的一次真机调试，根文件系统在 p2 上。现在以 ext2 为根时，安装只需一条 make install-rpi3-ext2root（第 6.9 节），但“先核对新内核的日志”这条教训同样适用。当时内核和 p2 根文件系统一起安装：

```
make
make install-rpi3 RPI3_XV6_DEV=/dev/rdisk4s2   # BOOTFS 挂载在 /Volumes/bootfs
```

install-rpi3 把内核、config.txt 和 Wi-Fi 固件复制到 BOOTFS，并把 fs.img 写入 RPI3_XV6_DEV 指定的 p2；它不会碰 p3 上的 ext2，也不会碰 p2 尾部的日志区。xv6 启动后，/etc/fstab 中 ext2 一行应为 rw（init 不会覆盖已有的 fstab，老卡要用 edit /etc/fstab 手工改一次）。

日志输出. 第一次插卡启动时看到的是：

```
ext2: mounted p3 read-only start=1884160 sectors=248463360 block=4096 groups=1
$ ls /mnt
.                               1 60 96
..                              1 1 1024
ext2                            1 2 4096
$ cd /mnt
exec cd failed
$ ls /mnt/ext2
exec ls failed
```

问题分析. 这几行日志里有三条线索：

- ext2: mounted p%d read-only 这句只在早期的只读 ext2 驱动里出现过，当前源码打印的是 ext2: p%d ready rw ...，前面还应有一行 xjournal: ready。在仓库里搜索这个格式串，只能在旧提交里找到——卡上 BOOTFS 中的内核根本不是新编译的那个；
- groups=1 说明 p3 上是那个 32 MiB 的 fs_ext2.img，而不是整分区的 ext2；
- ls /mnt 显示 ext2 的 inode 号为 2、大小为 4096——这是 ext2 根目录，说明挂载本身是成功的。cd 和 exec ls 的失败也与 ext2 无关：ls 刚刚还成功列出了 /mnt，失败发生在执行命令这一步，还没有访问 ext2。

解决方案. 用 make install-rpi3 RPI3_XV6_DEV=/dev/rdisk4s2 重新安装内核和 p2 根文件系统后，cd /mnt、ls /mnt/ext2 恢复正常，再按上一小节把 p3 改成整分区 ext2。新内核的启动日志应为：

```
rootdev: p2 boot=no type=0x7f (xv6 raw filesystem)
rootdev: raw xv6 root p2 lba=1064960 sectors=819200 size=400 MiB
rootdev: bootfs p1 lba=16384 mounted as FAT32
xjournal: ready lba=1130496 capacity=1024 sectors
ext2: p3 ready rw start=1884160 sectors=248463360 block=4096 groups=948
vfs: mounted ext2 at /mnt/ext2 read-write
```

日志起点 1130496 = 1064960 + 65536，即 p2 起点加上 xv6 镜像的长度。最后在真机上跑一遍：

```
cd /mnt/ext2
waltest init
waltest fsops
waltest semantics
```

这次调试的教训是：换了内核却没看启动日志的第一行，很容易把旧内核的问题算到新代码头上。每次安装后先核对一两条只有新版本才会打印的日志（这里是 `xjournal: ready` 和 `ready rw`），再开始测试。

10.1.13 当前边界

这仍是一个小型的 ext2 后端，与 Linux 的实现相比还缺不少：

- 不支持硬链接、符号链接、扩展属性、配额、HTree 索引目录和 GPT 分区表；
- 不支持 ext3 日志、ext4 extent、64 位块号和元数据校验和；
- 外置日志只有本内核认识。如果日志里还有已提交未写回的事务，要先在 xv6 里启动一次让它重放，再把卡插到 Linux 上；
- 孤儿 inode 只记在内存里，没有 ext3 那样的磁盘孤儿链表。“删除仍被打开的文件”之后断电，这个 inode 要靠 `e2fsck` 回收；
- 还没有 `umount`，接回开发机时应运行 `e2fsck -f`。

这些都是不错的后续练习。其中最有价值的大概是磁盘孤儿链表和 `umount`：前者只需要在 superblock 的 `s_last_orphan` 和 inode 的 `dtime` 字段里串一条链，在同一个事务里维护；后者让 `VALID_FS` 能在干净卸载时置回去，开发机就不必每次都做完整检查。

参考文献

[1] Arm Limited. *Arm Architecture Reference Manual for A-profile architecture*. ARM DDI 0487. Arm Limited. 2024.

[2] Arm Limited. *Arm Cortex-A53 MPCore Processor Technical Reference Manual*. ARM DDI 0500. Arm Limited. 2018.

[3] Daniel J. Bernstein et al. *TweetNaCl: a crypto library in 100 tweets*. 2014. URL: https://tweetnacl.cr.yp.to/.

[4] Broadcom Corporation. *BCM2835 ARM Peripherals*. Broadcom. 2012.

[5] Broadcom Corporation. *BCM2836 ARM-local peripherals (Quad-A7 control)*. Broadcom. 2014.

[6] Russ Cox, Frans Kaashoek, and Robert Morris. *xv6: a simple, Unix-like teaching operating system*. MIT PDOS, 2024. URL: https://pdos.csail.mit.edu/6.1810/xv6/book-riscv-rev4.pdf.

[7] Sergey Matyukevich. *raspberry-pi-os: Learning operating system development using Linux kernel and Raspberry Pi*. 2018. URL: https://github.com/s-matyukevich/raspberry-pi-os.

[8] Microsoft Corporation. *Microsoft Extensible Firmware Initiative FAT32 File System Specification*. Microsoft. 2000.

[9] MIPI Alliance. *MIPI Alliance Specification for Camera Serial Interface 2 (CSI-2)*. MIPI Alliance. 2017.

[10] Raspberry Pi Ltd. *Raspberry Pi firmware: boot files, config.txt and the mailbox property interface*. 2026. URL: https://github.com/raspberrypi/firmware/wiki.

[11] Raspberry Pi Ltd and Linux kernel developers. *Raspberry Pi Linux kernel source (drivers: brcmfmac, mt7601u, dwc2, bcm2835-unicam, ov5647, i2c-bcm2835)*. 2026. URL: https://github.com/raspberrypi/linux.

[12] rockytriton. *LLD: Low Level Devel – Raspberry Pi bare metal series*. 2021. URL: https://github.com/rockytriton/LLD.

[13] SD Association. *SD Specifications Part 1: Physical Layer Simplified Specification*. SD Association. 2020.

[14] SD Association. *SD Specifications Part E1: SDIO Simplified Specification*. SD Association. 2020.

[15] Rene Stange. *Circle: a C++ bare metal programming environment for the Raspberry Pi*. 2026. URL: https://github.com/rsta2/circle.

[16] USB Implementers Forum. *Universal Serial Bus Specification, Revision 2.0*. USB-IF. 2000.